70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A1

Sun'iy intellektga asoslangan kompyuter tizimlari arxitekturasini tahlil qilish va loyihalash

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A1
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A1
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu virtual laboratoriyada siz SHV (shaxsiy himoya vositalari) aniqlash tizimi uchun hisoblash platformasini raqamlar bilan tanlaysiz: talablardan FLOP/TFLOP/s yuklamasini hisoblaysiz, platformalarni talablar matritsasi bo'yicha baholaysiz, kechikish byudjetini simulyatsiya qilasiz, Roofline tahlilini quriysiz, kichik CNN'ning FLOP'ini torch bilan sanaysiz va Hardware–Software Co-design izlanish fazosini grid search bilan ko'rib chiqasiz. Real labdan farqi: tarmoq yo'qligi sababli ultralytics YOLO vaznlari yuklanmaydi — rasmiy hujjatlardagi GFLOP/parametr qiymatlari jadval sifatida kiritilgan; GPU yo'q, shuning uchun o'lchovlar 2 yadroli CPU'da kichik modelda bajariladi.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A1`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A1 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Talablardan yuklama hisobi: GFLOP, bayt, TFLOP/s
2. 2-qadam. Platformalar jadvali va talablar matritsasi bo'yicha tanlash skripti
3. 3-qadam. Kechikish byudjeti va Monte-Karlo simulyatsiyasi
4. 4-qadam. Arifmetik intensivlik va Roofline modeli
5. 5-qadam. Kichik CNN uchun FLOP va parametrlarni torch bilan hisoblash
6. 6-qadam. Hardware–Software Co-design: izlanish fazosini grid search bilan ko'rib chiqish
7. 7-qadam. Edge va markazlashgan arxitekturaning 3 yillik TCO taqqoslovi
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Talablardan yuklama hisobi: GFLOP, bayt, TFLOP/s

MAQSAD

Ssenariy talablari (16 kamera × 10 kadr/s) YOLOv8 variantlari uchun bir inferens GFLOP, vazn hajmi (FP16/INT8) va jami TFLOP/s talabiga aylantiriladi; samaradorlik koeffitsiyenti orqali kerakli cho'qqi TOPS topiladi. Kutilgan natija: yolov8s @ 640 uchun 28,6 GFLOP, 4,58 TFLOP/s haqiqiy, ~10 TOPS cho'qqi. Kodda KAMERALAR, FPS, SAMARADORLIK va MODELLAR jadvalini o'zgartirib, talab qanday o'sishini kuzating.

> **Kutilgan natija:** 12 qatorli jadval (4 model × 3 imgsz), etalon yolov8s@640: 4,58 TFLOP/s, cho'qqi 10,2 TOPS; tarmoq 64 Mbit/s, xom kadrlar ~1 GB/s.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Ssenariy talablaridan hisoblash yuklamasini (GFLOP, TFLOP/s) hisoblash
# Real labda ultralytics YOLO(...).info() modelni yuklab GFLOP'ni beradi. Sandbox'da tarmoq
yo'q,
# shuning uchun Ultralytics hujjatlaridagi rasmiy (640x640) qiymatlarni jadval sifatida
kiritamiz
# va boshqa kirish o'lchamlari uchun FLOP ~ (H*W) proporsionalligidan foydalanamiz.
import pandas as pd

# --- Ssenariy talablari (o'zgartirib ko'ring) ---
KAMERALAR = 16            # kameralar soni
FPS = 10                  # har kamera uchun kadr/s
SAMARADORLIK = 0.45       # platformaning cho'qqi unumdorligidan real erishiladigan ulush (30-
60 %)
INFERENS_S = KAMERALAR * FPS

# Ultralytics hujjatlari, 640x640 kirish: (parametrlar soni, GFLOP bir inferens uchun, COCO
mAP50-95)
MODELLAR = {
    "yolov8n": (3.15e6, 8.7, 37.3),
    "yolov8s": (11.17e6, 28.6, 44.9),
    "yolov8m": (25.90e6, 78.9, 50.2),
    "yolov8l": (43.70e6, 165.2, 52.9),
}


def model_profili(nomi, imgsz=640):
    """Bitta model varianti uchun statik yuklama ko'rsatkichlari."""
    params, gflop640, mAP = MODELLAR[nomi]
    gflop = gflop640 * (imgsz / 640) ** 2          # konvolyutsiya FLOP'i kirish maydoniga
proporsional
    talab_tflops = gflop * INFERENS_S / 1e3         # 160 inferens/s uchun jami talab
    return {
        "model": nomi,
        "imgsz": imgsz,
        "params_M": round(params / 1e6, 2),
        "vazn_FP16_MB": round(params * 2 / 1e6, 1),   # FP16: 2 bayt/parametr
        "vazn_INT8_MB": round(params * 1 / 1e6, 1),   # INT8: 1 bayt/parametr
        "GFLOP_inferens": round(gflop, 1),
        "TFLOP_s_talab": round(talab_tflops, 2),
        # samaradorlik koeffitsiyentini hisobga olgan holda kerakli CHO'QQI unumdorlik
        "kerakli_choqqi_TOPS": round(talab_tflops / SAMARADORLIK, 1),
        "mAP": mAP,
```

```
    }


qatorlar = [model_profili(n, s) for n in MODELLAR for s in (416, 512, 640)]
df = pd.DataFrame(qatorlar)

print("=== Yuklama hisobi ===")
print(f"Ssenariy: {KAMERALAR} kamera x {FPS} kadr/s = {INFERENS_S} inferens/s, samaradorlik
{SAMARADORLIK:.0%}")
print(df.to_markdown(index=False))

# Talablar matritsasidagi chegara: YOLOv8s @ 640
etalon = df[(df.model == "yolov8s") & (df.imgsz == 640)].iloc[0]
print("\n--- Etalon konfiguratsiya: yolov8s @ 640 ---")
print(f"Bir inferens: {etalon.GFLOP_inferens} GFLOP, vazn FP16 {etalon.vazn_FP16_MB} MB")
print(f"Jami talab: {etalon.TFLOP_s_talab} TFLOP/s haqiqiy -> cho'qqi kamida
{etalon.kerakli_choqqi_TOPS} TOPS")

# Kirish oqimi (dekoderdan oldin va keyin) — tarmoq va xotira talabi
bitrate_mbit = 4.0                                  # bitta 1080p10 H.264 oqim, Mbit/s
xom_kadr_MB = 1920 * 1080 * 3 / 1e6                 # dekodlangan RGB kadr
print(f"\nTarmoq: {KAMERALAR} x {bitrate_mbit} = {KAMERALAR * bitrate_mbit:.0f} Mbit/s (1
Gbit/s portning {KAMERALAR * bitrate_mbit / 10:.1f} %)")
print(f"Dekodlangan xom kadrlar: {INFERENS_S} x {xom_kadr_MB:.1f} MB = {INFERENS_S *
xom_kadr_MB / 1e3:.2f} GB/s xotira yozuvi")
df.to_csv("yuklama.csv", index=False)
print("yuklama.csv saqlandi")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Yuklama hisobi ===`, `yolov8s`

TAJRIBA QILIB KO‘RING

- KAMERALAR=64 va FPS=15 qilib, qaysi modellar 50 TOPS (Orin NX) chegarasiga sig'ishini aniqlang.
- SAMARADORLIK ni 0.3 va 0.6 qilib, kerakli cho'qqi TOPS qanday o'zgarishini jadvalga yozing.
- Yangi model qo'shing (masalan, yolov8x: 68,2 M parametr, 257,8 GFLOP) va uning vazn hajmini INT4 (0,5 bayt) uchun hisoblang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Platformalar jadvali va talablar matritsasi bo'yicha tanlash skripti

MAQSAD

Olti platforma (Orin NX, AGX Orin, A100, Coral Edge TPU, x86 CPU, Hailo-8) uchun haqiqiy unumdorlik, zaxira koeffitsiyenti, dekodlash yetarliligi va bir inferensga energiya hisoblanadi; so'ng 7 mezonli og'irlikli talablar matritsasi bo'yicha ball beriladi. Kutilgan natija: Orin NX ~96 %, Coral va CPU zaxira < 1 bo'lgani uchun yaroqsiz. Og'irliklarni va baholash funksiyalarini o'zgartirib, tanlov qanchalik barqarorligini tekshiring.

> **Kutilgan natija:** Ikki jadval: texnik ko'rsatkichlar (zaxira_koef: Orin NX ~4,9, A100 ~68, Coral 0,44) va og'irlikli ballar; TANLOV: Jetson Orin NX.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Platformalar jadvali va talablar matritsasi bo'yicha tanlash skripti
# Har platforma uchun haqiqiy unumdorlik (cho'qqi x samaradorlik), zaxira koeffitsiyenti,
# dekodlash yetarliligi va og'irlikli ball hisoblanadi. Qiymatlar rasmiy tavsiflardan
(taxminiy).
import pandas as pd

# Talab (1-qadam natijasi): yolov8s @ 640, 160 inferens/s
TALAB_TFLOPS = 28.6 * 160 / 1e3       # 4.58 TFLOP/s haqiqiy
KAMERA_OQIM = 16                      # 1080p10 oqimlar soni
GFLOP = 28.6

platformalar = pd.DataFrame([
    # nom, cho'qqi INT8 dense TOPS, samaradorlik, quvvat Vt, narx $, NVDEC 1080p30 oqim, INT8,
FP16, sanoat harorati, ekotizim (0-5)
    ("Jetson Orin NX 16GB",  50,  0.45, 25,  650,   7, True,  True,  True,  5),
    ("Jetson AGX Orin 64GB", 138, 0.45, 60,  2000, 14, True,  True,  True,  5),
    ("A100 PCIe 80GB",       624, 0.50, 300, 10000, 10, True,  True,  False, 5),
    ("Coral Edge TPU M.2",   4,   0.50, 2,   80,    0, True,  False, False, 2),
    ("x86 CPU 8 yadro",      2.5, 0.60, 65,  400,   0, True,  True,  True,  4),
    ("Hailo-8 M.2",          26,  0.50, 3,   200,   0, True,  False, True,  3),
], columns=["nom", "tops_peak", "eff", "vt", "narx", "nvdec_1080p30", "int8", "fp16",
"sanoat_t", "ekotizim"])

platformalar["tops_real"] = platformalar.tops_peak * platformalar.eff
platformalar["zaxira_koef"] = (platformalar.tops_real / TALAB_TFLOPS).round(2)
# 1080p10 oqim = 1/3 ta 1080p30 oqim
platformalar["dekod_yetarli"] = platformalar.nvdec_1080p30 * 3 >= KAMERA_OQIM
# bitta inferens uchun energiya (J) = quvvat / (sekundiga inferenslar soni)
platformalar["inferens_J"] = (platformalar.vt / (platformalar.tops_real * 1e3 /
GFLOP)).round(4)

print("=== Platformalarni baholash ===")
print(platformalar[["nom", "tops_peak", "tops_real", "zaxira_koef", "dekod_yetarli", "vt",
"narx", "inferens_J"]].to_markdown(index=False))

# --- Talablar matritsasi: (mezon, og'irlik, baholash funksiyasi -> 0..1) ---
# zaxira: 1 dan kichik = 0; 1-3 oralig'i chiziqli; 3 dan yuqori = 1
def ball_zaxira(z):
    return 0.0 if z < 1 else min(1.0, (z - 1) / 2)
```

```
MEZONLAR = [
    ("O'tkazuvchanlik (zaxira>=3)", 5, lambda r: ball_zaxira(r.zaxira_koef)),
    ("Kechikish (<=200 ms)",        5, lambda r: 1.0 if r.zaxira_koef >= 1 else 0.0),
    ("Ish harorati (+40 C)",        4, lambda r: 1.0 if r.sanoat_t else 0.0),
    ("Quvvat (<=60 Vt)",            3, lambda r: 1.0 if r.vt <= 60 else 0.0),
    ("Video dekodlash (16 oqim)",   4, lambda r: 1.0 if r.dekod_yetarli else 0.3),
    ("Narx (<=2000 $)",             3, lambda r: max(0.0, 1 - r.narx / 2000) if r.narx <= 2000
else 0.0),
    ("Dasturiy ekotizim",           3, lambda r: r.ekotizim / 5),
]
JAMI_OGIRLIK = sum(w for _, w, _ in MEZONLAR)

natija = []
for _, r in platformalar.iterrows():
    ballar = {m: round(f(r) * w, 2) for m, w, f in MEZONLAR}
    jami = round(sum(ballar.values()) / JAMI_OGIRLIK * 100, 1)
    # funksional talab (zaxira >= 1) bajarilmasa — platforma umuman yaroqsiz
    yaroqli = r.zaxira_koef >= 1
    natija.append({"nom": r.nom, **ballar, "jami_%": jami, "yaroqli": yaroqli})

jadval = pd.DataFrame(natija).sort_values(["yaroqli", "jami_%"], ascending=False)
print("\n--- Og'irlikli ballar (talablar matritsasi, jami 100 %) ---")
print(jadval.to_markdown(index=False))
tanlov = jadval[jadval.yaroqli].iloc[0]
print(f"\nTANLOV: {tanlov.nom} ({tanlov['jami_%']} %)")
print("Yaroqsiz (zaxira < 1):", ", ".join(jadval[~jadval.yaroqli].nom))
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Platformalarni baholash ===`, `TANLOV:`

TAJRIBA QILIB KO‘RING

- Narx og'irligini 3 dan 5 ga, haroratnikini 4 dan 1 ga o'zgartiring — tanlov o'zgaradimi?
- Talabni yolov8m @ 640 (12,6 TFLOP/s) ga oshiring va qaysi platformalar yaroqli qolishini ko'ring.
- Hailo-8 uchun apparat dekoder yo'qligini hisobga olib, CPU dekodlash quvvatini (+20 Vt) qo'shing va inferens_J ni qayta hisoblang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Kechikish byudjeti va Monte-Karlo simulyatsiyasi

MAQSAD

200 ms umumiy byudjet bloklar (tarmoq+dekodlash, oldindan ishlov, inferens, keyingi ishlov) orasida taqsimlanadi; har blok kechikishi lognormal taqsimot bilan 20 000 marta simulyatsiya qilinib p50/p95/p99 topiladi. Paket (batch) hajmi inferensni tejasa-da, kutish vaqtini oshirishi ko'rsatiladi. Kutilgan natija: batch=1 da p95 ~105 ms, batch=32 da p95 200 ms dan oshadi. Grafikda byudjet vs simulyatsiya ustunlari va batch egri chizig'i chiqadi.

> **Kutilgan natija:** Bloklar jadvali (p50/p95/p99), jami p95 ~105 ms, batch jadvali: 16 da ~150 ms, 32 da ~240 ms; 2 panelli grafik.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Uchdan-uchga kechikish byudjeti va uning Monte-Karlo simulyatsiyasi
# 200 ms umumiy byudjet bloklar orasida taqsimlanadi; har blok kechikishi tasodifiy (lognormal)
# deb olinadi va p50/p95/p99 hisoblanadi. Paket (batch) hajmi kutish vaqtini qanday oshirishini
ko'ramiz.
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
BYUDJET_MS = 200
INFERENS_S = 160                        # 16 kamera x 10 kadr/s
N = 20000                               # simulyatsiya qilinadigan kadrlar soni

# Blok: (nomi, byudjet ms, odatiy (median) ms, tarqalish sigma (lognormal))
BLOKLAR = [
    ("Tarmoq + dekodlash", 40, 25, 0.35),
    ("Oldindan ishlov",    10,  6, 0.25),
    ("Inferens",           60, 35, 0.20),
    ("Keyingi ishlov+mantiq", 20, 12, 0.40),
]
ZAXIRA = BYUDJET_MS - sum(b[1] for b in BLOKLAR)


def simulyatsiya(batch):
    """Har blok uchun N ta kechikish namunasini qaytaradi (ms)."""
    ustunlar = {}
    for nom, _, med, sigma in BLOKLAR:
        ustunlar[nom] = rng.lognormal(np.log(med), sigma, N)
    # Paketlash: inferens batch ta kadr to'plangancha kutadi; o'rtacha kutish (batch-1)/2 /
INFERENS_S
    if batch > 1:
        ustunlar["Paket kutish"] = rng.uniform(0, (batch - 1) / INFERENS_S * 1000, N)
        # batch ko'paysa, bir kadrga to'g'ri keladigan inferens vaqti kamayadi (GPU
samaradorligi oshadi)
        ustunlar["Inferens"] *= (1 + 0.25 * (batch - 1)) / batch
    return ustunlar


print("=== Kechikish byudjeti ===")
print(f"{'Blok':26s} {'byudjet':>8s} {'p50':>7s} {'p95':>7s} {'p99':>7s}")
jami = np.zeros(N)
p95_bloklar = []
for nom, byud, med, sigma in BLOKLAR:
```

```
    x = rng.lognormal(np.log(med), sigma, N)
    jami += x
    p95_bloklar.append(np.percentile(x, 95))
    print(f"{nom:26s} {byud:8d} {np.percentile(x, 50):7.1f} {np.percentile(x, 95):7.1f}
{np.percentile(x, 99):7.1f}")
print(f"{'Zaxira':26s} {ZAXIRA:8d}")
print(f"{'JAMI (batch=1)':26s} {BYUDJET_MS:8d} {np.percentile(jami, 50):7.1f}
{np.percentile(jami, 95):7.1f} {np.percentile(jami, 99):7.1f}")
print(f"Byudjetdan oshgan kadrlar ulushi: {np.mean(jami > BYUDJET_MS) * 100:.2f} %")

print("\n--- Paket hajmining ta'siri (uchdan-uchga) ---")
print(f"{'batch':>5s} {'p50':>7s} {'p95':>7s} {'p99':>7s} {'>200ms %':>9s}  inferens/kadr(ms)")
batchlar = (1, 4, 8, 16, 32)
p95_batch = []
for b in batchlar:
    u = simulyatsiya(b)
    e2e = sum(u.values())
    p95_batch.append(np.percentile(e2e, 95))
    print(f"{b:5d} {np.percentile(e2e, 50):7.1f} {np.percentile(e2e, 95):7.1f}
{np.percentile(e2e, 99):7.1f} {np.mean(e2e > BYUDJET_MS) * 100:9.2f}
{np.median(u['Inferens']):.1f}")

# --- Grafik: byudjet vs simulyatsiya p95 (stacked bar) va batch ta'siri ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
nomlar = [b[0] for b in BLOKLAR]
byudjetlar = [b[1] for b in BLOKLAR]
pastki = 0
for nom, byud, p95 in zip(nomlar, byudjetlar, p95_bloklar):
    ax1.bar("Byudjet", byud, bottom=pastki, label=nom)
    pastki += byud
ax1.bar("Byudjet", ZAXIRA, bottom=pastki, color="lightgray", label="Zaxira")
pastki = 0
for p95 in p95_bloklar:
    ax1.bar("Simulyatsiya p95", p95, bottom=pastki)
    pastki += p95
ax1.axhline(BYUDJET_MS, color="red", ls="--", label="200 ms chegara")
ax1.set_ylabel("ms")
ax1.set_title("Kechikish byudjeti taqsimoti")
ax1.legend(fontsize=7)
ax2.plot(batchlar, p95_batch, "o-")
ax2.axhline(BYUDJET_MS, color="red", ls="--")
ax2.set_xlabel("Paket hajmi (batch)")
ax2.set_ylabel("Uchdan-uchga p95, ms")
ax2.set_title("Paketlash kechikishni qanday oshiradi")
ax2.grid(alpha=0.3)
fig.tight_layout()
print("\nXulosa: batch 4-8 inferensni tejaydi, lekin 16+ da p95 200 ms chegaraga
yaqinlashadi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Kechikish byudjeti ===`, `Paket hajmining ta'siri`

TAJRIBA QILIB KO‘RING

- Tarmoq blokining sigma qiymatini 0.35 dan 0.8 ga oshiring (Wi-Fi kamera) — p99 qanday o'zgaradi?
- Byudjetni 50 ms ga tushiring va qaysi bloklar qayta loyihalanishi kerakligini aniqlang.
- Inferens medianini Orin NX o'rniga AGX Orin (3 baravar tez) uchun o'zgartirib, ruxsat etilgan maksimal batch'ni toping.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Arifmetik intensivlik va Roofline modeli

MAQSAD

Har platforma uchun tizza nuqtasi (P_peak / B_mem) hisoblanadi; YOLOv8s (FP16/INT8, batch 1/8), GEMM va GELU yuklamalari uchun AI = FLOP/bayt topilib, erishiladigan unumdorlik P = min(P_peak, AI·B_mem) va rejim (hisoblash yoki xotira bilan cheklangan) aniqlanadi. Kutilgan natija: YOLOv8s FP16 batch 1 Orin NX'da xotira bilan (X), A100'da hisoblash bilan (H) cheklangan; log-log Roofline grafigi chiziladi.

> **Kutilgan natija:** Tizza nuqtalari (A100 INT8 ~322, Orin NX ~490 FLOP/bayt), yuklama×platforma jadvali H/X belgilari bilan, Roofline grafigi.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Arifmetik intensivlik va Roofline modeli
# P = min(P_peak, AI * B_mem). Har platforma uchun tizza nuqtasi (ridge point) hisoblanadi,
# YOLOv8s yuklamasi turli batch'larda va boshqa yuklamalar (GEMM, elementlararo amal)
joylashtiriladi.
import numpy as np
import matplotlib.pyplot as plt

# Platforma: nom, cho'qqi unumdorlik (TFLOP/s yoki TOPS, dense), xotira o'tkazuvchanligi (GB/s)
PLATFORMALAR = [
    ("A100 PCIe (INT8 TC)",   624, 1935),
    ("A100 PCIe (FP16 TC)",   312, 1935),
    ("Orin NX (INT8 dense)",  50,  102),
    ("Coral Edge TPU",        4,   32),     # host DRAM orqali (taxminiy)
    ("x86 CPU 8 yadro (FP32)", 1.0, 70),
]

# Yuklama: FLOP va xotira trafigi (bayt) — soddalashtirilgan model
GFLOP = 28.6e9                # yolov8s @ 640, bir inferens
PARAMS = 11.17e6
AKTIV_BAYT_FP16 = 60e6        # faollashuvlar o'qish+yozish (taxminiy, FP16)


def yuklama_ai(batch, bayt_param):
    """Arifmetik intensivlik: vaznlar paket uchun bir marta o'qiladi, faollashuvlar har kadr
uchun."""
    flop = GFLOP * batch
    bayt = PARAMS * bayt_param + AKTIV_BAYT_FP16 * batch * (bayt_param / 2)
    return flop / bayt


print("=== Roofline tahlili ===")
print(f"{'Platforma':26s} {'P_peak':>8s} {'B_mem GB/s':>10s} {'tizza AI':>9s}")
for nom, peak, bw in PLATFORMALAR:
    print(f"{nom:26s} {peak:8.1f} {bw:10.0f} {peak * 1e12 / (bw * 1e9):9.0f}")

YUKLAMALAR = [
    ("YOLOv8s FP16, batch 1",  yuklama_ai(1, 2)),
    ("YOLOv8s FP16, batch 8",  yuklama_ai(8, 2)),
    ("YOLOv8s INT8, batch 1",  yuklama_ai(1, 1)),
    ("YOLOv8s INT8, batch 8",  yuklama_ai(8, 1)),
    ("GEMM 1024x1024 FP32",    2 * 1024 ** 3 / (3 * 1024 ** 2 * 4)),   # n/6 FLOP/bayt
    ("GELU elementlararo FP32", 8 / 8),                                    # ~8 FLOP / 8 bayt
```

```
]

print(f"\n{'Yuklama':26s} {'AI':>7s}  " + "  ".join(f"{p[0][:14]:>14s}" for p in PLATFORMALAR))
for nom, ai in YUKLAMALAR:
    hujayra = []
    for pnom, peak, bw in PLATFORMALAR:
        erishiladigan = min(peak, ai * bw / 1e3)        # TFLOP/s
        rejim = "H" if ai * bw / 1e3 >= peak else "X"   # H = hisoblash bilan, X = xotira bilan
cheklangan
        hujayra.append(f"{erishiladigan:9.1f} {rejim}")
    print(f"{nom:26s} {ai:7.1f}  " + "  ".join(f"{h:>14s}" for h in hujayra))
print("H = hisoblash bilan cheklangan, X = xotira bilan cheklangan (TFLOP/s erishiladigan)")

# Talab bilan solishtirish: 160 inferens/s uchun 4.58 TFLOP/s kerak
TALAB = 4.58
print(f"\nTalab: {TALAB} TFLOP/s. Orin NX INT8 batch 1 da erishiladigan: {min(50, yuklama_ai(1,
1) * 102 / 1e3):.1f} TFLOP/s")

# --- Roofline grafigi ---
ai_x = np.logspace(-1, 4, 200)
fig, ax = plt.subplots(figsize=(8, 5))
for nom, peak, bw in PLATFORMALAR:
    ax.plot(ai_x, np.minimum(peak, ai_x * bw / 1e3), label=nom)
for nom, ai in YUKLAMALAR:
    ax.axvline(ai, color="gray", ls=":", lw=0.8)
    ax.text(ai, 0.15, nom, rotation=90, fontsize=7, va="bottom", ha="right")
ax.set_xscale("log")
ax.set_yscale("log")
ax.set_xlabel("Arifmetik intensivlik, FLOP/bayt")
ax.set_ylabel("Erishiladigan unumdorlik, TFLOP/s")
ax.set_title("Roofline: platformalar va yuklamalar")
ax.grid(True, which="both", alpha=0.3)
ax.legend(fontsize=8)
fig.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Roofline tahlili ===`, `tizza AI`

TAJRIBA QILIB KO‘RING

- AKTIV_BAYT_FP16 ni 2 baravar oshiring (katta kirish) va Orin NX'da qaysi batch'dan boshlab rejim H ga o'tishini toping.
- Yangi platforma qo'shing (masalan, H100 SXM: 1979 TOPS INT8, 3350 GB/s) va tizza nuqtasini solishtiring.
- GEMM uchun n=256 va n=4096 nuqtalarini qo'shib, kichik matritsalar nima uchun xotira bilan cheklanishini tushuntiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Kichik CNN uchun FLOP va parametrlarni torch bilan hisoblash

MAQSAD

MiniYolo nomli kichik detektor tarmog'i uchun forward hook'lar bilan har qatlam FLOP'i formuladan (2·Kh·Kw·Cin·Cout·Hout·Wout) hisoblanadi va torch FlopCounterMode bilan 0 % farq tasdiqlanadi. So'ng kirish o'lchami (64/128/256) va batch (1/8) uchun haqiqiy CPU vaqti o'lchanib erishilgan GFLOP/s topiladi. Kutilgan natija: 128×128 da ~41,6 MFLOP, 61 K parametr; FLOP imgsz² ga proporsional; batch paketlash bir inferens vaqtini kamaytiradi.

> **Kutilgan natija:** Qatlamlar jadvali (5 ta Conv2d), jami 41,58 MFLOP hook va FlopCounterMode bir xil; o'lchov jadvali 12–40 GFLOP/s oralig'ida.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Kichik CNN uchun FLOP va parametrlarni hisoblash (torch) va CPU'da o'lchash
# Formula: Conv2d FLOP = 2*Kh*Kw*Cin/groups*Cout*Hout*Wout, Linear FLOP = 2*Nin*Nout.
# Qo'lda yozilgan hook'lar bilan hisoblab, torch'ning FlopCounterMode natijasi bilan
solishtiramiz,
# so'ng haqiqiy CPU vaqtini o'lchab erishilgan GFLOP/s ni topamiz.
import time
import torch
import torch.nn as nn
from torch.utils.flop_counter import FlopCounterMode

torch.manual_seed(0)
torch.set_num_threads(2)
torch.backends.nnpack.set_flags(False)   # sandbox CPU'da NNPACK yo'q — ogohlantirishlarni
o'chiramiz


class MiniYolo(nn.Module):
    """Juda kichik detektor-uslubidagi tarmoq: 4 ta conv + bosh (head)."""
    def __init__(self, cin=3, sinflar=4):
        super().__init__()
        self.backbone = nn.Sequential(
            nn.Conv2d(cin, 16, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.ReLU(),
            nn.Conv2d(64, 64, 3, stride=1, padding=1), nn.ReLU(),
        )
        self.head = nn.Conv2d(64, 5 + sinflar, 1)      # har hujayra: 4 koordinata + obyektlik
+ sinflar

    def forward(self, x):
        return self.head(self.backbone(x))


# --- Hook'lar bilan FLOP hisoblash ---
flop_jadval = []

def conv_hook(mod, inp, out):
    kh, kw = mod.kernel_size
    cin = mod.in_channels // mod.groups
    hout, wout = out.shape[-2:]
```

```
    flop = 2 * kh * kw * cin * mod.out_channels * hout * wout * out.shape[0]
    flop_jadval.append((mod.__class__.__name__, tuple(out.shape[1:]), flop))

def linear_hook(mod, inp, out):
    flop = 2 * mod.in_features * mod.out_features * out.shape[0]
    flop_jadval.append(("Linear", tuple(out.shape[1:]), flop))

model = MiniYolo().eval()
for m in model.modules():
    if isinstance(m, nn.Conv2d):
        m.register_forward_hook(conv_hook)
    elif isinstance(m, nn.Linear):
        m.register_forward_hook(linear_hook)

IMGSZ = 128
x = torch.randn(1, 3, IMGSZ, IMGSZ)
with torch.no_grad():
    model(x)

print("=== CNN FLOP hisobi ===")
print(f"{'Qatlam':8s} {'chiqish shakli':>18s} {'MFLOP':>9s}")
for nom, shakl, flop in flop_jadval:
    print(f"{nom:8s} {str(shakl):>18s} {flop / 1e6:9.2f}")
jami_hook = sum(f for _, _, f in flop_jadval)
params = sum(p.numel() for p in model.parameters())
print(f"Jami (hook): {jami_hook / 1e6:.2f} MFLOP, parametrlar: {params / 1e3:.1f} K, vazn FP32
{params * 4 / 1e6:.2f} MB")

# --- torch FlopCounterMode bilan tekshirish ---
flop_jadval.clear()
with torch.no_grad(), FlopCounterMode(display=False) as fc:
    model(x)
jami_torch = fc.get_total_flops()
print(f"Jami (torch FlopCounterMode): {jami_torch / 1e6:.2f} MFLOP  -> farq {abs(jami_torch -
jami_hook) / jami_hook * 100:.2f} %")

# --- FLOP kirish o'lchamiga qanday bog'liq ---
print("\n--- Kirish o'lchami va paket ta'siri (o'lchangan CPU vaqti, 2 ip) ---")
print(f"{'imgsz':>6s} {'batch':>5s} {'MFLOP':>9s} {'ms/inferens':>12s} {'GFLOP/s':>8s}")
for imgsz in (64, 128, 256):
    for batch in (1, 8):
        xx = torch.randn(batch, 3, imgsz, imgsz)
        flop_jadval.clear()
        with torch.no_grad():
            model(xx)
        flop = sum(f for _, _, f in flop_jadval)
        with torch.no_grad():
            for _ in range(3):                # isinish
                model(xx)
            vaqtlar = []
            for _ in range(10):
                t0 = time.perf_counter()
                model(xx)
                vaqtlar.append(time.perf_counter() - t0)
        t = sorted(vaqtlar)[len(vaqtlar) // 2]
        print(f"{imgsz:6d} {batch:5d} {flop / 1e6:9.1f} {t / batch * 1e3:12.3f} {flop / t /
1e9:8.2f}")

print("\nXulosa: FLOP kirish maydoniga (imgsz^2) proporsional; paketlash bir inferens vaqtini
kamaytiradi,")
print("chunki vaznlar keshda qoladi va BLAS yadrolari kattaroq matritsalarda samaraliroq
ishlaydi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== CNN FLOP hisobi ===`, `FlopCounterMode`

TAJRIBA QILIB KO‘RING

- Tarmoqqa nn.Linear bosh (head) qo'shib, linear_hook ishlashini va FLOP ulushini tekshiring.
- Birinchi conv'ning stride'ini 2 dan 1 ga o'zgartiring — FLOP necha baravar oshadi, vaqt-chi?
- torch.set_num_threads(1) bilan qayta o'lchab, 2 ipning tezlanishini batch 1 va 8 uchun solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Hardware–Software Co-design: izlanish fazosini grid search bilan ko'rib chiqish

MAQSAD

Dasturiy qarorlar (3 model × 3 imgsz × 2 format × 3 batch) va apparat qarorlari (6 platforma) birgalikda — 270 ta konfiguratsiya — zaxira, kechikish, quvvat, narx, harorat va format cheklovlari bo'yicha baholanadi. Yaroqli konfiguratsiyalar orasidan mAP bo'yicha eng yaxshilari, har platforma uchun eng yaxshi variant va narx–sifat Pareto chegarasi topiladi; grafikda yaroqli (rangli) va yaroqsiz (kulrang) nuqtalar ko'rsatiladi. Cheklov qiymatlarini o'zgartirib, izlanish fazosi qanday torayishini kuzating.

> **Kutilgan natija:** 270 konfiguratsiya, ~118 yaroqli; top-10 jadval (yolov8m AGX Orin / Orin NX INT8), Pareto ro'yxati (Hailo-8 → Orin NX → AGX Orin), scatter grafik.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Hardware-Software Co-design: izlanish fazosini to'liq ko'rib chiqish (grid search)
# Dasturiy qarorlar (model, kirish o'lchami, format, batch) x apparat qarorlari (platforma)
birgalikda
# baholanadi. Cheklovlar: o'tkazuvchanlik zaxirasi, inferens kechikishi, quvvat, narx, harorat,
format.
# Maqsad: cheklovlarni qanoatlantiruvchi eng sifatli (mAP) va arzon konfiguratsiyalar (Pareto
chegarasi).
import itertools
import pandas as pd
import matplotlib.pyplot as plt

KAMERALAR, FPS = 16, 10
INFERENS_S = KAMERALAR * FPS
INFERENS_BYUDJET_MS = 60       # kechikish byudjetidan inferensga ajratilgan qism
MIN_ZAXIRA = 1.5               # o'tkazuvchanlik zaxirasi (pik yuklama uchun)
MAKS_VT, MAKS_NARX = 60, 2000

# model: (GFLOP @640, COCO mAP @640)
MODELLAR = {"yolov8n": (8.7, 37.3), "yolov8s": (28.6, 44.9), "yolov8m": (78.9, 50.2)}
IMGSZ = (416, 512, 640)
FORMATLAR = {"FP16": 2, "INT8": 1}            # bayt/parametr (ma'lumot uchun)
BATCHLAR = (1, 4, 8)
# platforma: INT8 dense TOPS, samaradorlik, Vt, narx, NVDEC 1080p30 oqim, FP16 bor, INT8 bor,
sanoat harorati
PLATFORMALAR = {
    "Orin NX 16GB":  (50,  0.45, 25,  650,   7, True,  True,  True),
    "AGX Orin 64GB": (138, 0.45, 60,  2000, 14, True,  True,  True),
    "A100 PCIe":     (624, 0.50, 300, 10000, 10, True,  True,  False),
    "Coral TPU":     (4,   0.50, 2,   80,    0, False, True,  False),
    "Hailo-8":       (26,  0.50, 3,   200,   0, False, True,  True),
    "x86 CPU 8c":    (2.5, 0.60, 65,  400,   0, True,  True,  True),
}


def baholash(model, imgsz, fmt, batch, plat):
    gflop640, map640 = MODELLAR[model]
    tops, eff, vt, narx, nvdec, fp16, int8, sanoat = PLATFORMALAR[plat]
    gflop = gflop640 * (imgsz / 640) ** 2
    # sifat (mAP) proxysi: kichik kirishda pasayadi, INT8 kvantlash ~1 punkt yo'qotadi
    mAP = map640 - 12 * (1 - imgsz / 640) - (1.0 if fmt == "INT8" else 0)
```

```
    # format qo'llab-quvvatlanishi
    if (fmt == "FP16" and not fp16) or (fmt == "INT8" and not int8):
        return None
    tops_real = tops * eff * (0.5 if fmt == "FP16" else 1.0)   # FP16 odatda INT8 ning yarmi
    talab = gflop * INFERENS_S / 1e3
    zaxira = tops_real / talab
    # inferens kechikishi: paket hisoblash + paket to'lguncha kutish
    inferens_ms = batch * gflop / (tops_real * 1e3) * 1e3 * (1.3 if batch == 1 else 1.0)
    kutish_ms = (batch - 1) / INFERENS_S * 1e3
    kechikish = inferens_ms + kutish_ms
    dekod = nvdec * 3 >= KAMERALAR
    cpu_dekod_vt = 0 if dekod else 20            # apparat dekoder bo'lmasa CPU dekodlash
qo'shimcha quvvati
    jami_vt = vt + cpu_dekod_vt
    cheklovlar = {
        "zaxira": zaxira >= MIN_ZAXIRA,
        "kechikish": kechikish <= INFERENS_BYUDJET_MS,
        "quvvat": jami_vt <= MAKS_VT,
        "narx": narx <= MAKS_NARX,
        "harorat": sanoat,
    }
    return dict(model=model, imgsz=imgsz, fmt=fmt, batch=batch, platforma=plat, mAP=round(mAP,
1),
                zaxira=round(zaxira, 2), kechikish_ms=round(kechikish, 1), Vt=jami_vt,
narx=narx,
                yaroqli=all(cheklovlar.values()),
                buzilgan=",".join(k for k, v in cheklovlar.items() if not v) or "-")


qatorlar = []
for model, imgsz, fmt, batch, plat in itertools.product(MODELLAR, IMGSZ, FORMATLAR, BATCHLAR,
PLATFORMALAR):
    r = baholash(model, imgsz, fmt, batch, plat)
    if r:
        qatorlar.append(r)
df = pd.DataFrame(qatorlar)

print("=== Co-design izlanish fazosi ===")
print(f"Jami konfiguratsiyalar: {len(df)}, yaroqli: {df.yaroqli.sum()}")
print("\nCheklov buzilishlari (necha konfiguratsiyada):")
for k in ("zaxira", "kechikish", "quvvat", "narx", "harorat"):
    print(f"  {k:10s}: {df.buzilgan.str.contains(k).sum()}")

yaroqli = df[df.yaroqli].sort_values(["mAP", "narx"], ascending=[False, True])
print("\n--- Eng yaxshi 10 ta yaroqli konfiguratsiya (mAP bo'yicha) ---")
print(yaroqli.head(10).drop(columns=["yaroqli", "buzilgan"]).to_markdown(index=False))

# Pareto chegarasi: narx kamroq va mAP yuqoriroq
pareto = []
for _, r in yaroqli.sort_values("narx").iterrows():
    if not pareto or r.mAP > pareto[-1].mAP:
        pareto.append(r)
print("\n--- Pareto chegarasi (narx vs mAP) ---")
for r in pareto:
    print(f"  {r.narx:6d} $  mAP {r.mAP:5.1f}  {r.platforma:14s} {r.model} @{r.imgsz} {r.fmt}
batch {r.batch}  zaxira {r.zaxira}")

print("\n--- Har platforma uchun eng yaxshi yaroqli variant ---")
for plat, g in yaroqli.groupby("platforma"):
    r = g.iloc[0]
    print(f"  {plat:14s}: {r.model} @{r.imgsz} {r.fmt} batch {r.batch}  mAP {r.mAP}  zaxira
{r.zaxira}  {r.kechikish_ms} ms")
yoq = set(PLATFORMALAR) - set(yaroqli.platforma)
```

```
print("  Yaroqli varianti yo'q:", ", ".join(sorted(yoq)))

fig, ax = plt.subplots(figsize=(8, 5))
for plat, g in df.groupby("platforma"):
    ok = g[g.yaroqli]
    yo = g[~g.yaroqli]
    ax.scatter(yo.narx, yo.mAP, marker="x", alpha=0.3, color="gray")
    ax.scatter(ok.narx, ok.mAP, label=plat, s=40)
ax.plot([r.narx for r in pareto], [r.mAP for r in pareto], "k--", lw=1, label="Pareto
chegarasi")
ax.set_xscale("log")
ax.set_xlabel("Narx, $ (log)")
ax.set_ylabel("Sifat (mAP proxysi)")
ax.set_title("Co-design: yaroqli (rangli) va yaroqsiz (kulrang x) konfiguratsiyalar")
ax.legend(fontsize=8)
ax.grid(alpha=0.3)
fig.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Co-design izlanish fazosi ===`, `Pareto chegarasi`

TAJRIBA QILIB KO‘RING

- INFERENS_BYUDJET_MS ni 60 dan 20 ga tushiring — batch 8 variantlari yaroqli qoladimi?
- MAKS_NARX=5000 va harorat cheklovini olib tashlab (sanoat korpusi), A100 yaroqli bo'ladimi, Pareto o'zgaradimi?
- mAP proxy formulasini o'zgartiring (masalan, INT8 uchun −3 punkt) va eng yaxshi konfiguratsiya qanday siljishini yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Edge va markazlashgan arxitekturaning 3 yillik TCO taqqoslovi

MAQSAD

Uch variant (har sexda Orin NX; markaziy A100 server; markaziy L4 server) uchun apparat, elektr energiyasi (PUE bilan), tarmoq va texnik xizmat xarajatlari 3 yilga hisoblanadi; sexlar soni 1–24 oralig'ida tenglik nuqtasi qidiriladi va grafik chiziladi. Kutilgan natija: 16 kamera uchun Edge ~1,5 ming $, A100 ~26 ming $; 24 sexgacha markaz Edge'dan arzonlashmaydi. Elektr narxi, tarmoq ijara va server sig'imini o'zgartirib qaytadan hisoblang.

> **Kutilgan natija:** 1/2/4/8 sex uchun TCO jadvallari ($/kamera/oy bilan), tenglik nuqtasi xabari, 3 chiziqli TCO grafigi.

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Chekka (Edge) va markazlashgan arxitekturalarning 3 yillik TCO taqqoslovi
# TCO = apparat + elektr energiyasi + sovutish + tarmoq + texnik xizmat. Ikki ssenariy:
# (A) har sexda Jetson Orin NX (16 kamera uchun 1 modul), (B) markaziy serverda bitta A100
(yoki L4).
import numpy as np
import matplotlib.pyplot as plt

YIL = 3
SOAT_YIL = 24 * 365
NARX_KVT_SOAT = 0.08          # $ / kVt*soat (o'zgartirib ko'ring)
SEXLAR = (1, 2, 4, 8)          # sexlar soni, har birida 16 kamera

# Variant: nom, apparat $ (bitta birlik), quvvat Vt (bitta birlik), sovutish koef, tarmoq
$/yil, xizmat $/yil
VARIANTLAR = {
    "Edge: Orin NX har sexda": dict(apparat=650 + 350, vt=25 + 15, pue=1.1, tarmoq=0,
xizmat=150, birlik_har_sex=1),
    "Markaz: A100 server":     dict(apparat=10000 + 6000, vt=300 + 250, pue=1.6, tarmoq=1200,
xizmat=1500, birlik_har_sex=0),
    "Markaz: L4 server":       dict(apparat=2500 + 4000, vt=72 + 200, pue=1.6, tarmoq=1200,
xizmat=1000, birlik_har_sex=0),
}
# Markaz serveri sig'imi: A100 ~ 68x zaxira (1 sex), L4 (121 TOPS INT8 dense) ~ 13x zaxira
SIGIM_SEX = {"Edge: Orin NX har sexda": 1, "Markaz: A100 server": 12, "Markaz: L4 server": 3}


def tco(nom, sexlar):
    v = VARIANTLAR[nom]
    birlik = sexlar if v["birlik_har_sex"] else int(np.ceil(sexlar / SIGIM_SEX[nom]))
    apparat = birlik * v["apparat"]
    energiya = birlik * v["vt"] * v["pue"] / 1000 * SOAT_YIL * NARX_KVT_SOAT * YIL
    tarmoq = v["tarmoq"] * YIL * (sexlar if not v["birlik_har_sex"] else 0)
    xizmat = birlik * v["xizmat"] * YIL
    return dict(birlik=birlik, apparat=apparat, energiya=round(energiya), tarmoq=tarmoq,
xizmat=xizmat,
                jami=round(apparat + energiya + tarmoq + xizmat))


print("=== TCO 3 yil ===")
print(f"Elektr narxi {NARX_KVT_SOAT} $/kVt*soat, {YIL} yil, har sexda 16 kamera")
```

```
natija = {n: [] for n in VARIANTLAR}
for s in SEXLAR:
    print(f"\n--- {s} sex ({s * 16} kamera) ---")
    print(f"{'Variant':26s} {'birlik':>6s} {'apparat':>8s} {'energiya':>8s} {'tarmoq':>7s}
{'xizmat':>7s} {'JAMI $':>8s} {'$/kamera/oy':>11s}")
    for nom in VARIANTLAR:
        r = tco(nom, s)
        natija[nom].append(r["jami"])
        print(f"{nom:26s} {r['birlik']:6d} {r['apparat']:8d} {r['energiya']:8d}
{r['tarmoq']:7d} {r['xizmat']:7d} {r['jami']:8d} {r['jami'] / (s * 16) / (YIL * 12):11.2f}")

# Tenglik nuqtasi: markaz A100 qachon Edge'dan arzon bo'ladi?
sex_x = np.arange(1, 25)
edge = np.array([tco("Edge: Orin NX har sexda", s)["jami"] for s in sex_x])
a100 = np.array([tco("Markaz: A100 server", s)["jami"] for s in sex_x])
l4 = np.array([tco("Markaz: L4 server", s)["jami"] for s in sex_x])
teng = sex_x[np.argmax(a100 < edge)] if (a100 < edge).any() else None
teng_matn = str(teng) if teng else "24 sexgacha yo'q"
print(f"\nMarkaziy A100 Edge'dan arzon bo'ladigan sexlar soni: {teng_matn}")
print("Eslatma: TCO'da kechikish va yagona nosozlik nuqtasi (markaz) hisobga olinmagan — ular
talablar matritsasida.")

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(sex_x, edge / 1e3, "o-", label="Edge: Orin NX har sexda")
ax.plot(sex_x, a100 / 1e3, "s-", label="Markaz: A100 server")
ax.plot(sex_x, l4 / 1e3, "^-", label="Markaz: L4 server")
ax.set_xlabel("Sexlar soni (har birida 16 kamera)")
ax.set_ylabel("3 yillik TCO, ming $")
ax.set_title("Edge va markazlashgan arxitektura TCO")
ax.grid(alpha=0.3)
ax.legend()
fig.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== TCO 3 yil ===`, `Markaziy A100`

TAJRIBA QILIB KO‘RING

- NARX_KVT_SOAT ni 0.25 $ (Yevropa) qilib energiya ulushini solishtiring.
- Markaz uchun tarmoq xarajatini 0 qilib (mavjud optika), tenglik nuqtasi paydo bo'ladimi?
- Edge variantiga 3 yilda bir marta modul almashtirish (buzilish 10 %) xarajatini qo'shing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Yuklama hisobini kengaytirish** 1-qadam skriptiga yolov8x (68,2 M parametr, 257,8 GFLOP @640) va yolo11s (9,4 M, 21,5 GFLOP) qatorlarini qo'shing, KAMERALAR=32 uchun jadvalni qayta hosil qiling va qaysi modellar 50 TOPS dense cho'qqiga (samaradorlik 45 %) sig'ishini ro'yxat qilib hisobot bering.	oson	5
2	**Og'irliklar sezgirligi** 2-qadamda talablar matritsasining og'irliklarini uch xil ssenariyda (narx ustuvor, kechikish ustuvor, energiya ustuvor) o'zgartiring; har ssenariy uchun birinchi uchta platformani jadvalga yozing va tanlov qaysi og'irliklarda o'zgarishini aniqlang.	oson	5
3	**Kechikish byudjetini qayta taqsimlash** 3-qadamda umumiy byudjetni 100 ms ga tushiring, bloklar byudjetini o'zingiz qayta taqsimlang va simulyatsiyada p95 ≤ 100 ms bo'lishi uchun qaysi blok medianini necha foizga kamaytirish kerakligini raqam bilan ko'rsating (grafik bilan).	oson	5
4	**Roofline'ga yangi platformalar** 4-qadam Roofline tahliliga kamida ikkita yangi platforma (masalan, H100 SXM, Jetson Orin Nano, Apple M-seriya NPU) rasmiy tavsiflari bilan qo'shing, tizza nuqtalarini hisoblang va YOLOv8s INT8 batch 1 har platformada qaysi rejimda ishlashini jadval + grafik bilan asoslang; manbalarni ko'rsating.	o'rta	10
5	**Modelni o'lchash: nazariya va amaliyot** 5-qadamdagi MiniYolo'ni kengaytiring (kamida 8 conv qatlam, 256×256 kirish), hook va FlopCounterMode natijalarini solishtiring, so'ng torch.set_num_threads(1) va (2) bilan batch 1/4/16 uchun ms va GFLOP/s jadvalini tuzing; erishilgan GFLOP/s nazariy CPU cho'qqisining necha foizi ekanini baholang.	o'rta	10
6	**Co-design cheklovlar tahlili** 6-qadam grid search'ida har bir cheklovni navbat bilan olib tashlab (zaxira, kechikish, quvvat, narx, harorat) yaroqli konfiguratsiyalar soni qanday o'zgarishini jadvalga yozing; qaysi cheklov eng «qimmat» ekanini va uni yumshatish uchun qanday muhandislik yechimi borligini tushuntiring.	o'rta	10
7	**TCO sezgirlik tahlili** 7-qadamda elektr narxi (0,05–0,30 $/kVt·soat), tarmoq ijarasi (0–3000 $/yil) va server narxi bo'yicha sezgirlik tahlili o'tkazing: har parametr uchun tenglik nuqtasi (sexlar soni) qanday siljishini grafikda ko'rsating va 5 yillik muddat uchun xulosani qayta tekshiring.	o'rta	10
8	**Ikkinchi ssenariy: 64 kamera, markazlashgan ishlov** 1–4 va 6-qadamlar skriptlarini 64 kamera × 15 kadr/s, kechikish ≤ 500 ms, markaziy server xonasi ssenariysi uchun qayta sozlang: yuklama, platforma tanlovi (A100/L4/AGX Orin klaster), kechikish byudjeti (tarmoq 100 ms), Roofline va Co-design natijalarini bitta hisobotda keltiring va Edge ssenariysi bilan solishtiring.	qiyin	15
9	**Energiya byudjeti va termal cheklov modeli** 2- va 6-qadamlarga quvvat rejimi modelini qo'shing: Orin NX 10/15/25 Vt rejimlarida	qiyin	15

	cho'qqi unumdorlik mos ravishda 40/70/100 % deb olinsin, +40 °C muhitda passiv sovutishda 15 Vt rejimiga tushadi. Har rejim uchun zaxira koeffitsiyenti, yaroqli konfiguratsiyalar soni va bir inferens energiyasini (J) hisoblab, faol sovutish qarorini raqamlar bilan asoslang.		
10	**Strukturaviy sxema va interfeyslar generatori** Tanlangan konfiguratsiya (6-qadam natijasi) asosida strukturaviy sxemani avtomatik hosil qiluvchi skript yozing: bloklar ro'yxati, har interfeys uchun protokol, o'tkazuvchanlik (Mbit/s yoki MB/s — 1-qadam formulalaridan) va kechikish byudjeti (3-qadam natijasidan) jadval ko'rinishida chiqsin; 3 ta xavf va ularning kamaytirish choralarini ham skript matn sifatida chop etsin.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Shifoxona KT kesimlarida o'pka tugunlarini segmentatsiyalash serveri

o'rta

Soha: Tibbiyot

Viloyat ko'p tarmoqli shifoxonasida 4 ta KT apparat ishlaydi, har biri soatiga 12 ta tekshiruv o'tkazadi va har tekshiruvda o'rtacha 320 ta 512x512 kesim hosil bo'ladi. Radiolog hisobot yozguncha segmentatsiya natijasi butun tekshiruv uchun 90 s ichida tayyor bo'lishi kerak. Bemor ma'lumotlari shifoxona tarmog'idan chiqmasligi shart, shuning uchun bulut variantlari ko'rilmaydi. Tizim har KT xonasida (edge) yoki server xonasida (markaz) joylashishi mumkin; apparat byudjeti 12 000 $, quvvat chegarasi 400 Vt. Ertalabki pik soatlarda yuklama o'rtachadan 3 baravar yuqori.

Yuklama	4 KT x 12 tekshiruv/soat x 320 kesim (pik x3)
Modellar (GFLOP/kesim, Dice)	U-Net-lite 256^2: 9, 0,82; U-Net 512^2: 62, 0,86; nnU-Net 2.5D 512^2: 130, 0,89
SLO	butun tekshiruv <= 90 s, inferens ulushi <= 60 s
Platformalar	RTX A4000, L4, T4, Xeon 16 yadro (AMX), AGX Orin 64GB
Cheklovlar	byudjet 12 000 $, quvvat 400 Vt, ma'lumot ichkarida
Formatlar	FP16, INT8
TCO	5 yil, elektr 0,06 $/kVt*soat, KT soni 1-12

QADAMLAR

1. (s1) KAMERALAR/FPS o'rniga kesim/s hisoblang (o'rtacha va pik x3); 3 model x 2 kirish o'lchami (256, 512) uchun GFLOP, vazn MB (FP16/INT8), TFLOP/s talabi va kerakli cho'qqi TOPS (samaradorlik 0,4) jadvali; DICOM kirish oqimini (512^2 x 2 bayt/kesim) MB/s da hisoblang.
2. (s2) 5 platforma jadvali (cho'qqi FP16 TFLOP/s, INT8 TOPS, Vt, narx, FP16/INT8 qo'llab-quvvatlash); zaxira koeffitsiyenti pik yuklamaga nisbatan, bir kesim energiyasi (J); 6 mezonli og'irlikli matritsa (tekshiruv vaqti, joylashuv/maxfiylik, quvvat, narx, format, ekotizim) va TANLOV.
3. (s3) 90 s byudjetni bloklarga bo'ling: PACS'dan yuklash, resampling, inferens (320 kesim, batch 8/16/32), 3D post-processing, hisobot yozish; lognormal Monte-Karlo (10 000 tekshiruv) bilan p50/p95/p99 va byudjetdan oshgan tekshiruvlar ulushini toping; batch ta'siri grafigi.
4. (s4) Roofline: har platforma tizza nuqtasi; U-Net 512^2 FP16/INT8 batch 1/8/32 uchun AI (vaznlar + faollashuvlar ~180 MB/kesim) va rejim (H/X); 2.5D model uchun ham joylashtiring.
5. (s5) torch'da MiniUNet (3 ta conv enkoder stride 2 + 2 ta ConvTranspose2d dekoder + 1x1 chiqish) yozing; hook formulalari va FlopCounterMode (ConvTranspose uchun ham) 128^2/256^2 kirishda; batch 1/8 CPU vaqti va GFLOP/s.
6. (s6) Co-design grid: 3 model x 2 imgsz x 2 format x 3 batch x 5 platforma; cheklovlar zaxira >= 1,5 (pik), inferens <= 60 s/tekshiruv, quvvat, narx; Pareto narx-Dice; har platforma uchun eng yaxshi variant.

7. (s7) TCO 5 yil: edge (har KT xonasida AGX Orin) vs markaz (L4 server) vs markaz (A4000 ish stansiyasi), KT soni 1-12; tenglik nuqtasi va $/tekshiruv ko'rsatkichi.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z); Strukturaviy sxema matni: PACS -> ishlov -> model -> RIS interfeyslari (DICOM, HL7) o'tkazuvchanligi bilan

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar) — 25, Hisob-kitob va simulyatsiya to'g'riligi — 25, Grafiklar (Roofline, byudjet, co-design, TCO) — 15, Tahlil, tanlov asosi va xavflar — 25, Hisobot sifati — 10

2-variant. Paxta dalasida begona o'tlarni dron bortida aniqlash oson

Soha: Agro

Fermer xo'jaligi 300 gektar paxta maydonini kuzatish uchun dron sotib olmoqda. Dron 30 m balandlikda uchadi, bort kamerasi 1080p 15 kadr/s beradi va begona o'tlar (4 sinf) shu yerning o'zida aniqlanib, GPS bilan belgilanishi kerak, chunki dalada aloqa yo'q. Bort hisoblagichining quvvati 10 Vt dan oshmasligi, massasi 150 g dan kam va narxi 500 $ dan arzon bo'lishi kerak; muhit harorati yozda +45 C gacha. Bitta parvoz 25 daqiqa, kechikish 150 ms dan oshsa belgilangan koordinata 1 m dan ko'proq siljiydi.

Yuklama	1 kamera x 15 kadr/s (sinov: 30 kadr/s)
Modellar	yolov8n (8,7 GFLOP), yolo11n (6,5), yolov8s (28,6) @ 640; imgsz 320/480/640
SLO	kadr <= 150 ms uchdan-uchga
Platformalar	Orin Nano 8GB, Hailo-8L (RPi 5 bilan), Coral Edge TPU, RK3588, Orin NX
Cheklovlar	<= 10 Vt, <= 150 g, <= 500 $, +45 C
Energiya	batareya 60 Vt*soat, parvoz 25 daqiqa
TCO	3 yil, 1 dron vs 3 dron, ta'mir 15 %/yil

QADAMLAR

1. (s1) KAMERALAR=1, FPS=15 va 30 uchun 3 model x 3 imgsz jadvali: GFLOP, vazn INT8 MB, TFLOP/s va kerakli cho'qqi TOPS (samaradorlik 0,35); bir parvozdagi jami kadrlar va GB hisobini qo'shing.
2. (s2) 5 platforma (TOPS dense, samaradorlik, Vt, narx, massa g, dekoder, INT8/FP16, harorat): zaxira, inferens_J va parvoz davomida hisoblagich iste'moli (Vt*soat) ustunlari; 6 mezonli matritsa (zaxira, quvvat, massa, narx, harorat, ekotizim) va TANLOV; massa va quvvat og'irliklarini almashtirib tanlov barqarorligini tekshiring.
3. (s3) 150 ms byudjet: kamera olish + ISP, oldindan ishlov, inferens, NMS + GPS bog'lash; lognormal simulyatsiya 20 000 kadr, p95/p99; batch 1/2/4 kutish vaqti 30 kadr/s da.
4. (s4) Roofline 4 platforma uchun (Orin Nano 68 GB/s, Hailo-8L va Coral host DRAM taxminiy 20-30 GB/s, RK3588 25 GB/s): yolov8n INT8 batch 1/4 AI va rejim; nima uchun kichik model batch 1 da xotira bilan cheklanishini yozing.
5. (s5) MiniYolo'ni 4 sinf uchun 320x320 va 480x480 kirishda hook va FlopCounterMode bilan hisoblang; batch 1/4 CPU vaqti; FLOP imgsz^2 proporsionalligini jadvalda tasdiqlang.
6. (s6) Co-design grid: 3 model x 3 imgsz x INT8/FP16 x batch 1/2/4 x 5 platforma; cheklovlar zaxira >= 1,3, inferens <= 60 ms, <= 10 Vt, <= 500 $, <= 150 g; Pareto narx-mAP va eng yengil yaroqli variant.
7. (s7) TCO 3 yil: 1 dron bortida hisoblash vs tasvirlarni uyga olib kelib ish stansiyasida (RTX 3060, 1500 $) tungi ishlov; 1/2/3 dron uchun jadval; bort variantining asosiy afzalligi (kechikish) TCO'da yo'qligini izohlang.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar, hisob-kitob va jadvallar — 25, Tahlil va asoslangan tanlov — 25, Hisobot sifati (grafik, sxema, til) — 20

3-variant. Shahar chorrahalarida transport oqimini hisoblash va qoidabuzarlikni aniqlash o'rta

Soha: Transport

Shahar yo'l harakati boshqarmasi 12 ta chorrahaning har birida 4 tadan kamera (jami 48 ta, 1080p 25 kadr/s) o'rnatgan. Har kadrda transport turlari (6 sinf) aniqlanib, qizil chiroqda o'tish va to'xtash chizig'i buzilishi 100 ms ichida qayd etilishi kerak (svetofor kontrolleriga moslashuvchan signal uchun). Har chorrahada shkafda 35 Vt quvvat va -30...+50 C harorat; chorrahadan boshqarmaga 20 Mbit/s optika bor. Byudjet chorraha boshiga 3000 $. Arxitektura tanlovi: har chorrahada edge modul yoki boshqarmada markaziy GPU server.

Yuklama	48 kamera x 25 kadr/s (har chorrahada 4 x 25 = 100 inferens/s)
Modellar	yolov8n/s/m @ 640 (8,7/28,6/78,9 GFLOP), imgsz 416/640/800
SLO	p99 <= 100 ms (tarmoq 20 Mbit/s markaz uchun +30 ms)
Platformalar	Orin NX, AGX Orin, Hailo-8 (x2), L4 server (markaz), T4 server (markaz)
Cheklovlar	shkaf <= 35 Vt, -30...+50 C, <= 3000 $/chorraha
Tarmoq	chorraha -> markaz 20 Mbit/s, kamera oqimi 4 Mbit/s
TCO	3 yil, 12 chorraha, elektr 0,07 $/kVt*soat, optika ijarasi 600 $/yil

QADAMLAR

1. (s1) Bitta chorraha (100 inferens/s) va butun shahar (1200 inferens/s) uchun 3 model x 3 imgsz jadvali: GFLOP, TFLOP/s, kerakli cho'qqi TOPS (samaradorlik 0,45); 4 kamera x 4 Mbit/s oqim 20 Mbit/s kanalga sig'adimi - markaz varianti uchun tekshiring.
2. (s2) 5 platforma jadvali: edge uchun chorraha yuklamasi, markaz uchun 1200 inferens/s talab bilan zaxira; dekodlash yetarliligi (4 va 48 oqim 1080p25); 7 mezonli matritsa (zaxira, kechikish, harorat, quvvat, dekod, narx, ekotizim); edge va markaz uchun alohida TANLOV.
3. (s3) 100 ms byudjet ikki arxitekturada: edge (olish+dekod, ishlov, inferens, kuzatuv+qoida mantiqi) va markaz (+tarmoq 30 ms median, sigma 0,5); 20 000 kadr Monte-Karlo, p99 va oshgan kadrlar ulushi; batch 1/4/8 ta'siri; ikki panelli grafik.
4. (s4) Roofline: Orin NX, AGX Orin, L4, T4 (xotira 102/205/300/320 GB/s); yolov8s INT8 batch 1/4/8 AI va rejim; markaz serverida batch 8 bilan nima o'zgaradi.
5. (s5) MiniYolo 6 sinf, kirish 128/256/416; hook vs FlopCounterMode; batch 1/8 CPU vaqti va GFLOP/s; birinchi conv stride 1 bo'lsa FLOP necha baravar oshishini o'lchang.
6. (s6) Co-design grid 3 model x 3 imgsz x 2 format x 3 batch x 5 platforma; edge cheklovlari (zaxira >= 1,5, inferens <= 40 ms, <= 35 Vt, <= 3000 $, harorat) va markaz cheklovlari (zaxira >= 1,5 jami yuklama uchun, inferens <= 30 ms) alohida; Pareto narx-mAP ikkala holatda.
7. (s7) TCO 3 yil: edge Orin NX har chorrahada vs markaz L4 server(lar) vs markaz T4 server(lar); chorraha soni 1-30 bo'yicha tenglik nuqtasi; tarmoq ijarasi 0 va 600 $/yil holatlarini solishtiring.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z); Strukturaviy sxema: kamera -> edge/markaz -> svetofor kontrolleri interfeyslari (RTSP, MQTT/NTCIP) va o'tkazuvchanlik

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar) — 25, Hisob-kitob va simulyatsiya to'g'riligi — 25, Grafiklar (Roofline, byudjet, co-design, TCO) — 15, Tahlil, tanlov asosi va xavflar — 25, Hisobot sifati — 10

4-variant. 5G baza stansiyalari KPI oqimida anomaliyani real vaqtda aniqlash

oson

Soha: Telekom

Mobil operator 2 400 ta hujayra (sektor) uchun har sekundda 48 ta KPI (trafik, PRB bandligi, RSRP, uzilishlar va h.k.) oladi va har hujayra uchun 1 Hz chastotada anomaliya ehtimolini hisoblamoqchi. Model - kichik MLP yoki 1D-CNN (oxirgi 60 s oynasi); tasvir yo'q, shuning uchun yuklama kamera emas, hujayra x chastota bilan hisoblanadi. SLO: anomaliya 2 s ichida aniqlanishi kerak; tizim operatorning mavjud x86 serverlarida (GPU'siz) yoki kichik GPU'da ishlashi mumkin. Keyingi yilda hujayralar soni 2 baravar oshadi. Ma'lumotlar sklearn make_classification bilan sintetik hosil qilinadi.

Yuklama	2 400 hujayra x 1 Hz (kelajak: 4 800 x 1 Hz, sinov 10 Hz)
Modellar	MLP 48-128-64-1 (~0,04 MFLOP), 1D-CNN 60x48 (3 conv, ~3 MFLOP), kichik Transformer 60x48 (~25 MFLOP)
SLO	p99 <= 2 s uchdan-uchga, inferens <= 50 ms partiya uchun
Platformalar	x86 8 yadro (AVX2), Xeon 16 yadro (AMX), T4, Orin Nano
Byudjet	4 000 $, mavjud server xonasi
Partiya (batch)	1, 64, 2400 (butun davr)

QADAMLAR

1. (s1) KAMERALAR*FPS o'rniga HUJAYRALAR x CHASTOTA: 3 model uchun MFLOP/inferens, vazn MB (FP32/FP16/INT8), GFLOP/s talab (2 400 va 4 800 hujayra, 1 va 10 Hz), kerakli cho'qqi (samaradorlik 0,2, chunki kichik modellar past samarali); KPI oqimi (48 x 4 bayt x hujayra x Hz) kbayt/s.
2. (s2) 4 platforma (FP32 TFLOP/s, INT8 TOPS, Vt, narx, ekotizim): zaxira koeffitsiyenti 1D-CNN uchun, bir inferens energiyasi (mJ); 5 mezonli matritsa (zaxira, narx, quvvat, mavjud infratuzilma bilan moslik, ekotizim) va TANLOV; GPU bu yuklama uchun nima uchun ortiqcha ekanini raqam bilan ko'rsating.
3. (s3) 2 000 ms byudjet: Kafka/kolleksiya (median 300 ms, sigma 0,6), oyna yig'ish, inferens partiyada, ogohlantirish yozish; 20 000 davr Monte-Karlo; partiya 1/64/2400 uchun kutish vaqti (1 Hz davr ichida) va p99.
4. (s4) Roofline: MLP va 1D-CNN uchun AI (vazn + oyna baytlari) batch 1/64/2400 - hammasi xotira bilan cheklangan ekanini x86 (70 GB/s) va T4 (320 GB/s) uchun ko'rsating; qaysi batch'dan boshlab H rejimga o'tadi.
5. (s5) torch'da MLP va 1D-CNN (Conv1d) yozing, Linear/Conv1d hook formulalari va FlopCounterMode; batch 1/64/2400 CPU vaqti - bir inferens vaqti batch bilan necha baravar kamayadi.
6. (s6) Co-design grid: 3 model x 3 batch x FP32/INT8 x 4 platforma; cheklovlar zaxira >= 2 (4 800 hujayra), inferens <= 50 ms, narx <= 4 000 $; Pareto narx-sifat (sifat proxysi: MLP 0,90, 1D-CNN 0,94, Transformer 0,96 AUC).

7. (s7) TCO 3 yil: mavjud x86 server (faqat elektr+xizmat) vs yangi T4 server vs bulut GPU (0,35 $/soat); hujayralar 2 400 -> 9 600 o'sishida qaysi variant qachon arzon.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar, hisob-kitob va jadvallar — 25, Tahlil va asoslangan tanlov — 25, Hisobot sifati (grafik, sxema, til) — 20

5-variant. Elektr tarmog'i PMU o'lchovlarida nosozlikni aniqlash va tasniflash

o'rta

Soha: Energetika

Hududiy elektr tarmog'ida 400 ta PMU (fazor o'lchagich) 50 Hz chastotada 12 kanalli (3 faza kuchlanish/tok, chastota, burchak) ma'lumot yuboradi. 1D-CNN model 200 ms oynani (10 namuna) tahlil qilib, 8 turdagi nosozlikni (qisqa tutashuv, uzilish, tebranish...) aniqlaydi. Himoya avtomatikasi uchun uchdan-uchga kechikish 100 ms dan oshmasligi kerak, ya'ni nosozlik oyna tugaganidan 100 ms ichida tasniflanadi. Tizim podstansiyalarda (har birida 20 PMU, sanoat shkafi, -40...+55 C, 20 Vt) yoki dispetcherlik markazida joylashishi mumkin; markazga 2 Mbit/s radioreleyli kanal, kechikishi 15-40 ms.

Yuklama	400 PMU x 50 Hz (20 PMU/podstansiya, 20 podstansiya); oyna 10 namuna x 12 kanal
Modellar	1D-CNN-S (0,8 MFLOP), 1D-CNN-M (6 MFLOP), TCN (40 MFLOP), aniqlik 0,91/0,95/0,97
SLO	p99 <= 100 ms (himoya), markaz uchun kanal 15-40 ms
Platformalar	Orin Nano, Orin NX, Xilinx Kria K26 (1,4 TOPS INT8, 10 Vt), x86 sanoat PC (4 yadro), T4 server (markaz)
Cheklovlar	podstansiya <= 20 Vt, -40...+55 C, <= 1 500 $/podstansiya
Kanal	2 Mbit/s, PMU oqimi 12 x 4 bayt x 50 Hz
TCO	5 yil, elektr 0,05 $/kVt*soat, 20 podstansiya

QADAMLAR

1. (s1) Oyna/s hisobi (har PMU 50 Hz da har namuna uchun yangi oyna: 400 x 50 = 20 000 inferens/s; podstansiya 1 000 inferens/s); 3 model uchun GFLOP/s, kerakli cho'qqi TOPS (samaradorlik 0,25); PMU oqimi kbit/s va 20 PMU uchun 2 Mbit/s kanal yetarliligi.
2. (s2) 5 platforma: zaxira podstansiya (1 000 inf/s) va markaz (20 000 inf/s) uchun; inferens energiyasi (mJ); 7 mezonli matritsa (zaxira, kechikish, harorat, quvvat, narx, sertifikatlash/IEC 61850 ekotizimi, format); edge va markaz uchun TANLOV.
3. (s3) 100 ms byudjet: PMU -> konsentrator (10 ms), oyna yig'ish, inferens, rele buyrug'i (GOOSE 4 ms); markaz uchun +radiorele (median 25 ms, sigma 0,5); 20 000 oyna Monte-Karlo p99; partiya 1/20/400 ta'siri va qaysi arxitektura SLO'ni bajaradi.
4. (s4) Roofline: 1D-CNN-M va TCN batch 1/20/400 uchun AI; Orin Nano (68 GB/s), Kria (19 GB/s), x86 (40 GB/s), T4 (320 GB/s) tizza nuqtalari; rejim H/X.
5. (s5) torch'da 1D-CNN-M (Conv1d x3 + Linear) yozing; hook (2*K*Cin*Cout*Lout) va FlopCounterMode; batch 1/20/400 CPU vaqti; bir oyna uchun mks va GFLOP/s.
6. (s6) Co-design: 3 model x 3 batch x FP32/FP16/INT8 x 5 platforma; cheklovlar zaxira >= 2, inferens+kutish <= 40 ms, quvvat, narx, harorat; Pareto narx-aniqlik; Kria INT8 nima uchun chegarada ekanini tushuntiring.

7. (s7) TCO 5 yil: edge (20 podstansiyada Orin Nano/Kria) vs markaz T4 server + 20 kanal; podstansiya soni 1-40 tenglik nuqtasi; markaz arxitekturasidagi yagona nosozlik nuqtasi va kanal uzilishi xavfini xulosada baholang.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z); Strukturaviy sxema: PMU -> konsentrator -> model -> rele (IEC 61850 GOOSE/MMS) interfeyslari

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar) — 25, Hisob-kitob va simulyatsiya to'g'riligi — 25, Grafiklar (Roofline, byudjet, co-design, TCO) — 15, Tahlil, tanlov asosi va xavflar — 25, Hisobot sifati — 10

6-variant. To'lov tizimida kartali tranzaksiyalar firibgarligini real vaqtda skoring qilish o'rta

Soha: Moliya

Milliy to'lov tizimi o'rtacha 3 000 tranzaksiya/s (bayram pikida 12 000/s) qayta ishlaydi. Har tranzaksiya 64 ta belgidan iborat vektor bo'lib, firibgarlik skoringi avtorizatsiya zanjirida sinxron bajariladi: p99 kechikish 20 ms dan oshsa bank tomoni taym-aut beradi. Uchta model nomzod: gradient-busting o'rnini bosuvchi MLP, kengroq MLP va 32 so'nggi tranzaksiya ketma-ketligi uchun GRU. Ma'lumotlar markazida ikki zona (faol-faol) bo'lishi shart, ya'ni har zona to'liq yuklamani ko'tarishi kerak. Byudjet 40 000 $ (2 zona), har server 300 Vt chegara; GPU'siz x86 variant ham jiddiy ko'rib chiqiladi. Ma'lumotlar sklearn make_classification (imbalance 0,2 %) bilan sintetik.

Yuklama	3 000 tr/s o'rtacha, 12 000 tr/s pik; 2 zona, har biri 100 %
Modellar	MLP 64-256-64-1 (0,07 MFLOP, AUC 0,93), MLP 64-1024-512-1 (1,2 MFLOP, 0,95), GRU 32x64 h=128 (5 MFLOP, 0,97)
SLO	p99 <= 20 ms, inferens <= 5 ms
Platformalar	x86 32 yadro (AVX-512), Xeon AMX 32 yadro, T4, L4, A10
Cheklovlar	<= 40 000 $ (2 zona), <= 300 Vt/server, faol-faol
Partiya	1, 16, 128 (dinamik batching 2 ms oyna)
TCO	3 yil, elektr 0,10 $/kVt*soat, bulut muqobili 0,7 $/soat (L4)

QADAMLAR

1. (s1) 3 model uchun MFLOP, vazn MB, GFLOP/s talab (o'rtacha va pik), kerakli cho'qqi (samaradorlik 0,25 kichik partiya uchun, 0,5 katta partiya uchun); belgi oqimi (64 x 4 bayt x tr/s) MB/s va 32 tranzaksiyali ketma-ketlikni xotirada saqlash hajmi (10 M karta uchun).
2. (s2) 5 platforma (FP32 TFLOP/s, INT8 TOPS, Vt, narx): zaxira pik yuklamaga va bir zona uchun, tranzaksiya energiyasi (mJ); 6 mezonli matritsa (zaxira, kechikish, narx, quvvat, ekotizim/Triton, ishonchlilik); TANLOV va 'GPU shartmi' savoliga raqamli javob.
3. (s3) 20 ms byudjet: tarmoq kirish (2 ms), belgilarni Redis'dan olish (median 3 ms, sigma 0,6), dinamik batching kutish (0-2 ms), inferens, qaror va javob; 50 000 tranzaksiya Monte-Karlo p99; batch 1/16/128 ta'siri; 12 000 tr/s pikda kutish qanday kamayishini ko'rsating.
4. (s4) Roofline: MLP va GRU batch 1/16/128 AI; x86 (120 GB/s), T4 (320), L4 (300), A10 (600) tizza nuqtalari; GRU ketma-ket qadamlari nima uchun past AI berishini yozing.
5. (s5) torch'da 3 modelni (nn.Linear, nn.GRU) yozing; Linear hook va GRU uchun formula (3 x 2 x (in+h) x h x T) ni FlopCounterMode bilan solishtiring; batch 1/16/128 CPU vaqti va p50 mks/tranzaksiya.
6. (s6) Co-design grid: 3 model x 3 batch x FP32/FP16/INT8 x 5 platforma; cheklovlar zaxira >= 1,5 (pik, 1 zona), inferens+kutish <= 5 ms, narx 2 zona uchun, quvvat; Pareto narx-AUC; faol-faol uchun eng arzon ikki server.
7. (s7) TCO 3 yil: 2 x x86 server vs 2 x L4 server vs bulut L4 (2 zona, 24/7) vs bulut avtomasshtab (o'rtacha 30 % yuk); tr/s 3 000 -> 30 000 o'sishida tenglik nuqtasi.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar) — 25, Hisob-kitob va simulyatsiya to'g'riligi — 25, Grafiklar (Roofline, byudjet, co-design, TCO) — 15, Tahlil, tanlov asosi va xavflar — 25, Hisobot sifati — 10

7-variant. Onlayn imtihonda avtomatik proktoring: yuz va qarash yo'nalishini kuzatish o'rta

Soha: Ta'lim

Universitet masofaviy imtihonlarida bir vaqtda 1 500 ta talaba veb-kamera orqali kuzatiladi. Har talaba oqimidan 2 kadr/s olinib, yuz aniqlash, bir nechta odam borligi, qarash yo'nalishi va telefon aniqlash bajariladi; shubhali hodisa 3 s ichida nazoratchiga ko'rsatilishi kerak. Imtihon mavsumi yiliga 6 hafta, qolgan vaqtda tizim bo'sh turadi - shuning uchun bulut ijarasi va o'z serveri jiddiy solishtiriladi. Talaba brauzerida (WebAssembly, ~0,5 TFLOP/s) qisman ishlov berish ham variant. Byudjet 25 000 $, universitet server xonasida 2 kVt bo'sh quvvat bor.

Yuklama	1 500 talaba x 2 kadr/s = 3 000 inferens/s (pik 2 500 talaba)
Modellar	SCRFD-0.5G (0,5 GFLOP) + MobileFaceNet (0,45) + yolov8n telefon @ 320 (2,2); og'ir: SCRFD-2.5G + ArcFace-R50 (12,6) + yolov8s @ 416
SLO	hodisa <= 3 s, inferens zanjiri <= 500 ms
Platformalar	T4 server, L4 server, A10 server, Xeon 32 yadro, talaba brauzeri (WASM 0,5 TFLOP/s, 0 $)
Cheklovlar	<= 25 000 $, <= 2 kVt, yiliga 6 hafta faol
Tarmoq	talaba oqimi 0,5 Mbit/s (2 kadr/s JPEG) yoki faqat hodisalar (brauzer varianti)
TCO	3 yil, bulut T4 0,35 $/soat, L4 0,7 $/soat, faqat 6 hafta/yil

QADAMLAR

1. (s1) 2 ta model zanjiri (yengil/og'ir) uchun bir kadr GFLOP yig'indisi, 3 000 va 5 000 inferens/s uchun TFLOP/s va kerakli cho'qqi TOPS (samaradorlik 0,4); kiruvchi JPEG oqimi 1 500 x 0,5 Mbit/s va 1 Gbit/s port yetarliligi; brauzer varianti uchun talaba kompyuteriga tushadigan yuklama (GFLOP/s).
2. (s2) 5 platforma jadvali (brauzer satri: 1 500 x 0,5 TFLOP/s taqsimlangan, lekin ishonchsiz); zaxira, kadr energiyasi; 7 mezonli matritsa (zaxira, kechikish, narx, quvvat, maxfiylik, ishonchlilik, ekotizim); TANLOV ikki ssenariyda: 'hammasi serverda' va 'brauzer + server tasdiqlash'.
3. (s3) 3 000 ms byudjet: yuklash (median 400 ms, sigma 0,7 - uy interneti), navbat, zanjir inferensi (3 model ketma-ket), qoida mantiqi, nazoratchi UI; 20 000 kadr Monte-Karlo; batch 1/8/32 (3 000 inferens/s da kutish kichik) ta'siri.
4. (s4) Roofline: T4/L4/A10/Xeon tizza nuqtalari; SCRFD, MobileFaceNet, ArcFace-R50 FP16 batch 1/32 AI va rejim; kichik modellar uchun batch'ning rolini raqam bilan ko'rsating.
5. (s5) torch'da MiniFace (4 conv + global pool + Linear 128 embedding) 112x112 va MiniYolo 320x320; hook va FlopCounterMode; batch 1/32 CPU vaqti.
6. (s6) Co-design: 2 zanjir x imgsz (yolov8 uchun 320/416) x FP16/INT8 x batch 1/8/32 x 4 server platformasi; cheklovlar zaxira >= 1,7 (pik), zanjir inferensi <= 500 ms, narx, quvvat; Pareto narx-sifat (sifat proxysi: yengil 0,88, og'ir 0,95).

7. (s7) TCO 3 yil: o'z L4 server(lar)i (yil bo'yi elektr, 6 hafta foydali) vs bulut L4 faqat 6 hafta vs gibrid (brauzer + 1 server); talaba soni 500-5 000 uchun tenglik nuqtasi va $/imtihon.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar) — 25, Hisob-kitob va simulyatsiya to'g'riligi — 25, Grafiklar (Roofline, byudjet, co-design, TCO) — 15, Tahlil, tanlov asosi va xavflar — 25, Hisobot sifati — 10

8-variant. Shisha idish zavodida yuqori tezlikli nuqson nazorati (4 liniya) qiyin

Soha: Sanoat

Zavodda 4 ta ishlab chiqarish liniyasi har biri sekundiga 20 ta idish chiqaradi; har idish 3 ta kameradan (2048x1536, monoxrom) suratga olinadi, ya'ni jami 240 tasvir/s. Nuqsonli idish 30 ms ichida rad etuvchi pnevmatik klapanga signal berilmasa konveyerdan o'tib ketadi. Model - segmentatsiya + tasniflash (nuqson 7 sinf), tasvirni 1024x768 yoki 512x384 ga tushirish mumkin, lekin mayda yoriqlar 512 da yo'qoladi (sifat proxysi beriladi). Sex harorati +50 C, chang, har liniya shkafida 150 Vt; byudjet 4 liniya uchun 30 000 $; to'xtash 1 soati 8 000 $ zarar, shuning uchun zaxira va ishonchlilik muhim.

Yuklama	4 liniya x 20 idish/s x 3 kamera = 240 tasvir/s (2048x1536 mono)
Modellar (GFLOP @1024x768, sifat)	SegFormer-B0: 25, 0,90; U-Net-R18: 70, 0,94; U-Net-R34: 110, 0,96; 512x384 da FLOP/4, sifat -0,05
SLO	p99 <= 30 ms uchdan-uchga, inferens <= 15 ms
Platformalar	Orin NX, AGX Orin, RTX A4000 (sanoat PC), L4, Hailo-8 x4, Xeon 16 AMX
Cheklovlar	+50 C, <= 150 Vt/liniya, <= 30 000 $, zaxira >= 2 (ishonchlilik)
Kirish oqimi	2048x1536x1 bayt x 60 tasvir/s har liniya (GigE Vision)
TCO	5 yil, elektr 0,09 $/kVt*soat, to'xtash zarari 8 000 $/soat, buzilish ehtimoli yiliga 5 %

QADAMLAR

1. (s1) Liniya (60 tasvir/s) va zavod (240/s) uchun 3 model x 2 o'lcham jadvali: GFLOP, vazn FP16/INT8, TFLOP/s, kerakli cho'qqi (samaradorlik 0,45); GigE Vision oqimi (MB/s va Gbit/s) har liniya uchun va 10 GbE zarurligini aniqlang; xom kadr xotira yozuvi GB/s.
2. (s2) 6 platforma: zaxira liniya va zavod yuklamasiga, inferens_J, GigE dekod talab qilmasligini hisobga oling; 8 mezonli matritsa (zaxira >= 2, kechikish, harorat, quvvat, narx, ishonchlilik/MTBF, format, ekotizim); liniya-boshi (edge) va zavod-markaz TANLOVI.
3. (s3) 30 ms byudjet: kamera ekspozitsiya+uzatish (5 ms), oldindan ishlov, inferens, qaror+klapan (2 ms); lognormal Monte-Karlo 50 000 tasvir; batch 1/3/6 (3 kamera bir idish) ta'siri; p99 <= 30 ms ta'minlaydigan maksimal batch'ni toping; ikkita grafik.
4. (s4) Roofline: 6 platforma tizza nuqtasi; U-Net-R18 FP16/INT8 batch 1/3/6 AI (faollashuvlar 1024x768 uchun ~400 MB); kichik o'lcham 512x384 da AI qanday o'zgaradi; GEMM va elementlararo amal nuqtalari.
5. (s5) torch'da MiniSeg (enkoder 4 conv stride 2, dekoder 2 ConvTranspose2d, 7 sinf) 256x192 va 512x384 kirish (1 kanal); hook va FlopCounterMode; batch 1/3 CPU vaqti; ConvTranspose FLOP formulasini alohida tekshiring.
6. (s6) Co-design grid: 3 model x 2 o'lcham x 2 format x 3 batch x 6 platforma; cheklovlar zaxira >= 2, inferens+kutish <= 15 ms, quvvat, narx (4 liniya), harorat; Pareto narx-sifat; har cheklovni navbat bilan olib tashlab yaroqli soni jadvali (eng 'qimmat' cheklov).

7. (s7) TCO 5 yil: edge (har liniyada Orin NX/AGX) vs markaz L4 server (zaxira server bilan N+1) vs A4000 sanoat PC har liniyada; to'xtash zararini (buzilish 5 %/yil x tiklash 4 soat) qo'shing; liniya soni 1-12 tenglik nuqtasi; sezgirlik: elektr 0,05-0,2 $.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z); Strukturaviy sxema: GigE Vision kamera -> frame grabber -> model -> PLC (PROFINET/EtherCAT) interfeyslari va kechikish byudjeti jadvali; 3 ta xavf va kamaytirish chorasi (skript matn sifatida chop etsin)

Baholash (100 ball): Kod: parametrlangan, qayta ishlatiladigan funksiyalar — 25, Hisob, simulyatsiya va sezgirlik tahlili to'g'riligi — 20, Grafiklar va Pareto/tenglik nuqtasi tahlili — 20, Muhandislik xulosasi, strukturaviy sxema, xavflar — 25, Hisobot sifati — 10

9-variant. Aeroport bagaj rentgen tasvirlarida taqiqlangan buyumlarni aniqlash

o'rta

Soha: Xavfsizlik

Xalqaro aeroportda 6 ta rentgen skaner (ikki energiyali, 1024x768 tasvir) ishlaydi, har skanerdan tasma tezligi 0,25 m/s bo'lgani uchun o'rtacha 1,5 bagaj/s chiqadi, pik 2,5 bagaj/s. Har bagaj uchun 2 proyeksiya (yuqori va yon) tahlil qilinadi, 12 sinf taqiqlangan buyum aniqlanadi. Operator ekranida natija 1 s ichida ko'rinishi kerak; skanerlar terminalning turli qavatlarida; xavfsizlik talablari tufayli bulut mumkin emas, lekin terminal server xonasi bor. Byudjet 20 000 $, skaner shkafida 80 Vt; sifat kritik - mAP 0,80 dan past model qabul qilinmaydi.

Yuklama	6 skaner x 1,5 bagaj/s x 2 proyeksiya = 18 inferens/s (pik 30/s)
Modellar	yolov8m (78,9 GFLOP, mAP 0,84), yolov8l (165,2, 0,87), RT-DETR-R50 (136, 0,88) @ 640; imgsz 640/1024
SLO	p99 <= 1 000 ms, inferens <= 300 ms
Platformalar	Orin NX, AGX Orin, RTX A2000 (70 Vt, 153 INT8 TOPS, 650 $), T4 server, L4 server
Cheklovlar	<= 20 000 $, <= 80 Vt skanerda, mAP >= 0,80, bulut yo'q
Formatlar	FP16, INT8 (INT8 mAP -1,5 punkt; RT-DETR INT8 -3)
TCO	5 yil, 6 -> 12 skaner kengayish, elektr 0,08 $/kVt*soat

QADAMLAR

1. (s1) 18 va 30 inferens/s uchun 3 model x 2 imgsz jadvali: GFLOP, vazn, TFLOP/s, kerakli cho'qqi (samaradorlik 0,4); 1024x768x2 bayt x 2 proyeksiya tasvir oqimi MB/s har skaner va server uchun.
2. (s2) 5 platforma: zaxira pikka, inferens_J; 7 mezonli matritsa (zaxira, kechikish, sifat >= 0,80 ni qo'llab-quvvatlovchi model sig'imi, quvvat, narx, xavfsizlik/joylashuv, ekotizim); skaner-boshi va markaz TANLOVI.
3. (s3) 1 000 ms byudjet: skaner -> tasvir shakllantirish (median 150 ms), uzatish, oldindan ishlov, inferens (2 proyeksiya), operator UI render (80 ms); 20 000 bagaj Monte-Karlo; batch 1/2/4 (2 proyeksiya birga) ta'siri va p99.
4. (s4) Roofline: 5 platforma tizza nuqtasi; yolov8l va RT-DETR FP16/INT8 batch 1/2/4 AI va rejim; RT-DETR'ning attention qismi uchun alohida nuqta (AI past) qo'shing.
5. (s5) MiniYolo 12 sinf, 1 kanal (rentgen) kirish 256x192 va 512x384; hook va FlopCounterMode; batch 1/2 CPU vaqti; imgsz^2 proporsionallik.
6. (s6) Co-design: 3 model x 2 imgsz x 2 format x 3 batch x 5 platforma; cheklovlar zaxira >= 1,5, inferens <= 300 ms, quvvat, narx (6 skaner), mAP >= 0,80 (INT8 jarima bilan); Pareto narx-mAP; INT8 jarimasini -3 punktga oshirganda top-5 qanday o'zgaradi.
7. (s7) TCO 5 yil: har skanerda Orin NX/A2000 vs markaz T4/L4 server; 6 va 12 skaner uchun jadval, 1-16 skaner tenglik nuqtasi; markaz varianti uchun zaxira (N+1) serverni qo'shib qayta hisoblang.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar) — 25, Hisob-kitob va simulyatsiya to'g'riligi — 25, Grafiklar (Roofline, byudjet, co-design, TCO) — 15, Tahlil, tanlov asosi va xavflar — 25, Hisobot sifati — 10

10-variant. Kassasiz do'kon: xaridorlarni kuzatish va javondan olingan tovarni aniqlash qiyin

Soha: Chakana savdo

Chakana tarmoq 400 m2 maydonli kassasiz do'kon ochmoqda: shiftda 48 ta kamera (1080p 15 kadr/s), javonlarda 600 ta og'irlik sensori (10 Hz). Har kadrda odamlar aniqlanadi, poza (17 nuqta) baholanadi va qayta identifikatsiya (ReID) embedding olinadi; sensor hodisasi bilan birlashtirilib 'kim nima oldi' hal qilinadi. Xaridor chiqishida chek 2 s ichida shakllanishi kerak, lekin kuzatuv zanjiri har kadrda 300 ms dan oshmasligi lozim, aks holda traektoriyalar uziladi. Do'kon orqa xonasida 1,5 kVt va 2 ta server o'rni bor; byudjet 45 000 $; tarmoq 10 GbE. Savdo kuni 16 soat, tungi vaqtda tizim o'chadi.

Yuklama	48 kamera x 15 kadr/s = 720 kadr/s; har kadrda o'rtacha 3 odam -> poza+ReID 2 160 inferens/s; 600 sensor x 10 Hz
Modellar	yolov8s odam (28,6 GFLOP @640); RTMPose-m (1,9 GFLOP/odam); OSNet ReID (1,0 GFLOP/odam); og'ir: yolov8m + RTMPose-l (4,5)
SLO	zanjir p99 <= 300 ms/kadr, chek <= 2 s
Platformalar	2 x L4 server, 2 x T4 server, 1 x A100 server, 4 x AGX Orin, 2 x RTX 4090 ish stansiyasi
Cheklovlar	<= 45 000 $, <= 1 500 Vt, 2 server o'rni, 10 GbE
Dekodlash	48 x 1080p15 oqim (NVDEC 1080p30 soni: L4 - 8 muvozanat, T4 - 7, A100 - 10, AGX - 14, 4090 - 2 x NVDEC ~ 14)
TCO	3 yil, 16 soat/kun, elektr 0,11 $/kVt*soat, 1 -> 10 do'kon

QADAMLAR

1. (s1) Uch bosqichli zanjir (detektor + poza + ReID) uchun bir kadr jami GFLOP (3 odam bilan) va 720 kadr/s uchun TFLOP/s; yengil va og'ir zanjir; kerakli cho'qqi (samaradorlik 0,4); 48 oqim x 4 Mbit/s tarmoq, xom kadr GB/s, 600 sensor oqimi kbit/s.
2. (s2) 5 konfiguratsiya (server to'plamlari) jadvali: jami TOPS, dekoder sig'imi 48 oqim uchun, Vt, narx; zaxira, kadr energiyasi; 8 mezonli matritsa (zaxira, kechikish, dekod, quvvat, narx, o'rin soni, ishonchlilik N+1, ekotizim/DeepStream); TANLOV.
3. (s3) 300 ms byudjet: dekod, detektor, poza (odam soni bilan o'sadi: 1-8 odam), ReID, kuzatuv + sensor birlashtirish (median 20 ms, sigma 0,5); odam soni Poisson(3) tasodifiy; 20 000 kadr Monte-Karlo p99; batch 1/8/16 ta'siri; 'juma kechqurun' ssenariysida (6 odam/kadr) byudjet buziladimi.
4. (s4) Roofline: L4, T4, A100, AGX Orin, 4090 tizza nuqtalari; yolov8s, RTMPose-m, OSNet FP16/INT8 batch 1/8/32 AI va rejim; kichik ReID modelini katta batch'da yig'ish nima uchun zarurligini ko'rsating.
5. (s5) torch'da MiniYolo (1 sinf), MiniPose (4 conv + 17 kanal heatmap 64x48) va MiniReID (4 conv + GAP + Linear 128) uchun hook va FlopCounterMode; batch 1/8/32 CPU vaqti; zanjir jami ms/kadr 3 odam uchun.

6. (s6) Co-design grid: 2 zanjir x imgsz (detektor 512/640) x 2 format x batch 1/8/16 x 5 konfiguratsiya; cheklovlar zaxira >= 1,5, zanjir <= 150 ms inferens, dekod, quvvat, narx; Pareto narx-sifat (proxy: yengil 0,90, og'ir 0,95; INT8 -0,01); har platforma uchun eng yaxshi.
7. (s7) TCO 3 yil: do'kon ichida 2 x L4 vs 4 x AGX Orin vs 2 x 4090 vs markaziy A100 (bir necha do'kon uchun, do'kon -> markaz 1 Gbit/s ijarasi 2 400 $/yil); do'konlar soni 1-10; 16 soat/kun ish rejimini energiyada hisobga oling; tenglik nuqtasi.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z); Strukturaviy sxema: kamera/sensor -> dekod -> zanjir -> hodisalar ombori -> chek xizmati interfeyslari (RTSP, MQTT, gRPC) va o'tkazuvchanlik; Sezgirlik: odam/kadr 2-6 oralig'ida zaxira va p99 grafigi

Baholash (100 ball): Kod: parametrlangan, qayta ishlatiladigan funksiyalar — 25, Hisob, simulyatsiya va sezgirlik tahlili to'g'riligi — 20, Grafiklar va Pareto/tenglik nuqtasi tahlili — 20, Muhandislik xulosasi, strukturaviy sxema, xavflar — 25, Hisobot sifati — 10

11-variant. Omborda paketlarni saralash: shtrix-kod va manzil yorlig'ini o'qish

oson

Soha: Logistika

Pochta-logistika kompaniyasining saralash markazida 12 ta saralash liniyasi bor; har liniyadan sekundiga 4 ta paket o'tadi (1080p kamera, paket o'tganda 1 kadr). Har paket uchun yorliq aniqlanadi (detektor), so'ng shtrix-kod va manzil matni o'qiladi (OCR). Saralash qurilmasiga yo'nalish 400 ms ichida yuborilmasa paket 'noma'lum' tarmog'iga ketadi va qo'lda qayta ishlanadi (har biri 0,5 $ xarajat). Ombor isitilmaydi (-10...+40 C), har liniyada 40 Vt quvvat, markazda kichik server xonasi; byudjet 15 000 $.

Yuklama	12 liniya x 4 paket/s = 48 inferens zanjiri/s (pik 80/s)
Modellar	yolov8n yorliq @ 640 (8,7 GFLOP) + CRNN OCR 32x320 (1,2 GFLOP/qator, 3 qator); og'ir: yolov8s + SVTR-small OCR (3,5/qator)
SLO	p99 <= 400 ms, inferens zanjiri <= 150 ms
Platformalar	Orin Nano, Orin NX, Hailo-8 + x86, RK3588, T4 server (markaz)
Cheklovlar	<= 40 Vt liniyada, -10...+40 C, <= 15 000 $
Xato narxi	noto'g'ri yo'naltirilgan paket 0,5 $; o'qish aniqligi yengil 0,96, og'ir 0,985
TCO	3 yil, 12 -> 24 liniya, elektr 0,07 $/kVt*soat

QADAMLAR

1. (s1) 48 va 80 zanjir/s uchun 2 zanjir (detektor + 3 OCR qatori) GFLOP, TFLOP/s va kerakli cho'qqi (samaradorlik 0,4); OCR uchun 3 qator x 1,2 GFLOP hisobini alohida ko'rsating; bir paket tasviri (JPEG 300 KB) oqimi MB/s.
2. (s2) 5 platforma jadvali: zaxira liniya (4/s) va markaz (48/s) uchun, inferens_J; 6 mezonli matritsa (zaxira, kechikish, harorat, quvvat, narx, ekotizim) va TANLOV; xato narxi (0,5 $ x xato ulushi x paketlar/yil) ni og'ir model foydasiga qo'shimcha ustun qilib hisoblang.
3. (s3) 400 ms byudjet: suratga olish trigger (10 ms), uzatish, detektor, OCR (3 qator ketma-ket yoki batch 3), saralash buyrug'i (PLC 20 ms); 20 000 paket Monte-Karlo p99; OCR qatorlarini batch qilish kechikishni qanday kamaytiradi.
4. (s4) Roofline: Orin Nano/NX, RK3588, T4 tizza nuqtalari; yolov8n INT8 batch 1/4 va CRNN FP16 batch 3/12 AI va rejim.
5. (s5) MiniYolo (1 sinf yorliq) 256x256 va MiniCRNN (3 conv + GRU 64 + Linear 37 belgi) 32x160 kirish; hook va FlopCounterMode (GRU formulasini qo'lda qo'shing); batch 1/8 CPU vaqti.
6. (s6) Co-design: 2 zanjir x imgsz 480/640 x FP16/INT8 x batch 1/4/8 x 5 platforma; cheklovlar zaxira >= 1,5, inferens <= 150 ms, quvvat, narx (12 liniya), harorat; Pareto narx-aniqlik.
7. (s7) TCO 3 yil: har liniyada Orin Nano vs markaz T4 server; 12 va 24 liniya; xato xarajatini (paketlar/yil x xato ulushi x 0,5 $) TCO'ga qo'shib qaysi model-platforma eng arzon ekanini toping.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar, hisob-kitob va jadvallar — 25, Tahlil va asoslangan tanlov — 25, Hisobot sifati (grafik, sxema, til) — 20

12-variant. Shahar avtoturargohlari: kirish-chiqishda davlat raqamini tanish (ANPR) oson

Soha: Shahar

Shahar hokimligi 40 ta ko'p qavatli va ochiq avtoturargohni yagona to'lov tizimiga ulamoqchi. Har avtoturargohda 2 ta shlagbaum kamerasi (1080p) bor; avtomobil kelganda 2 kadr/s tahlil qilinadi va raqam 1 s ichida tanilib shlagbaum ochilishi kerak, aks holda navbat hosil bo'ladi. Avtoturargoh soatiga pikda 300 avtomobil qabul qiladi. Shlagbaum shkafida 15 Vt, harorat -25...+45 C; markaziy ma'lumot markazi mavjud, avtoturargohlar 4G/optika bilan ulangan (kechikish 30-150 ms). Byudjet avtoturargoh boshiga 1 200 $; 2 yilda avtoturargohlar soni 100 taga yetadi.

Yuklama	40 x 2 kamera x 2 kadr/s = 160 inferens/s (faqat harakat bo'lganda ~30 %)
Modellar	yolov8n raqam @ 416 (3,7 GFLOP) + CRNN 32x128 (0,5 GFLOP); og'ir: yolov8s @ 640 + SVTR (2,0)
SLO	p95 <= 1 000 ms (shlagbaum), inferens <= 200 ms
Platformalar	RK3588 (6 TOPS), Orin Nano, Hailo-8L + RPi 5, x86 mini PC (4 yadro), T4 server (markaz)
Cheklovlar	<= 15 Vt, -25...+45 C, <= 1 200 $/avtoturargoh
Tarmoq	4G (median 80 ms, sigma 0,7) yoki optika (median 20 ms)
TCO	3 yil, 40 -> 100 avtoturargoh, 4G ijarasi 120 $/yil

QADAMLAR

1. (s1) Bitta avtoturargoh (4 inferens/s) va markaz (160/s, pik 100 %) uchun 2 zanjir GFLOP, TFLOP/s, kerakli cho'qqi (samaradorlik 0,35); kadr JPEG (200 KB) oqimi 4G kanal (10 Mbit/s) ga sig'ishini tekshiring.
2. (s2) 5 platforma: zaxira avtoturargoh va markaz yuklamasiga, inferens_J; 6 mezonli matritsa (zaxira, kechikish, harorat, quvvat, narx, ekotizim); edge va markaz uchun TANLOV; 4G uzilganda edge ishlashda davom etishini mezon sifatida qo'shing.
3. (s3) 1 000 ms byudjet: harakat trigger, kadr olish, detektor, OCR, to'lov bazasi so'rovi (median 60 ms), shlagbaum rele (50 ms); markaz uchun +4G; 20 000 avtomobil Monte-Karlo p95/p99; 4G sigma 0,7 bo'lganda markaz varianti SLO'ni bajaradimi.
4. (s4) Roofline: RK3588 (25 GB/s), Orin Nano (68), x86 mini PC (40), T4 (320); yolov8n INT8 batch 1 va CRNN batch 1/8 AI va rejim.
5. (s5) MiniYolo (1 sinf) 256x256 va MiniCRNN 32x128 (3 conv + GRU 48 + Linear 40); hook va FlopCounterMode; batch 1/8 CPU vaqti; bir avtomobil uchun zanjir ms.
6. (s6) Co-design: 2 zanjir x imgsz 416/640 x FP16/INT8 x batch 1/2/4 x 5 platforma; cheklovlar zaxira >= 1,5, inferens <= 200 ms, quvvat, narx, harorat; Pareto narx-aniqlik (yengil 0,95, og'ir 0,985).
7. (s7) TCO 3 yil: har avtoturargohda RK3588/Orin Nano vs markaz T4 + 4G ijarasi; 40 va 100 avtoturargoh; tenglik nuqtasi 1-120 oralig'ida; xulosada 4G ishonchliligi xavfini yozing.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar, hisob-kitob va jadvallar — 25, Tahlil va asoslangan tanlov — 25, Hisobot sifati (grafik, sxema, til) — 20

13-variant. O'rmon xo'jaligida yong'in tutunini quyosh batareyali minora kameralaridan aniqlash oson

Soha: Ekologiya

Tog'li o'rmon xo'jaligi 10 ta kuzatuv minorasiga 4K panoramali kamera o'rnatmoqda; kamera 360 gradusni 6 ta 1280x720 bo'lakka bo'lib har 5 s da bir marta tahlil qiladi (0,2 kadr/s x 6 bo'lak). Minorada elektr yo'q: quyosh paneli 100 Vt + akkumulyator 1,2 kVt*soat, hisoblagichga o'rtacha 8 Vt ajratiladi (qish kunlari 4 Vt). Aloqa - LoRa/4G kam tezlikli (0,3 Mbit/s), shuning uchun video markazga yuborilmaydi, faqat ogohlantirish va kichik rasm. Tutun 60 s ichida aniqlanishi kerak; yolg'on ogohlantirish narxi yuqori (vertolyot chiqadi). Byudjet minora boshiga 900 $.

Yuklama	10 minora x 6 bo'lak x 0,2 kadr/s = 12 inferens/s (minora: 1,2/s)
Modellar	yolov8n tutun @ 640 (8,7 GFLOP, F1 0,86), yolov8s @ 640 (28,6, 0,91), yolov8s @ 1280 (114, 0,94)
SLO	aniqlash <= 60 s, bo'lak zanjiri <= 5 s
Platformalar	Orin Nano (7/15 Vt rejim), RK3588, Hailo-8L + RPi 5, Coral Edge TPU + RPi 4, x86 markaz serveri (faqat solishtirish)
Cheklovlar	<= 8 Vt o'rtacha (qish 4 Vt), <= 900 $, -30...+40 C
Energiya	quyosh 100 Vt panel, 1,2 kVt*soat akkumulyator, qishda 3 kun quyoshsiz
TCO	5 yil, akkumulyator almashtirish 3 yilda, 10 -> 30 minora

QADAMLAR

1. (s1) Minora (1,2 inferens/s) va butun tarmoq uchun 3 konfiguratsiya GFLOP, TFLOP/s, kerakli cho'qqi (samaradorlik 0,35); kunlik energiya (Vt*soat) = quvvat x 24 soat va 3 quyoshsiz kunda akkumulyator yetarliligini hisoblang.
2. (s2) 4 edge platforma + markaz satri: zaxira, inferens_J, kunlik Vt*soat; 6 mezonli matritsa (zaxira, energiya byudjeti, harorat, narx, ekotizim, oflayn ishlash); TANLOV; quyosh paneli 60 Vt bo'lsa tanlov o'zgaradimi.
3. (s3) 5 000 ms byudjet bo'lak uchun: kamera burilish+ekspozitsiya (median 800 ms), olish, inferens, vaqt bo'yicha tasdiqlash (3 ketma-ket kadr), LoRa yuborish (median 1 500 ms, sigma 0,6); 10 000 sikl Monte-Karlo; 60 s aniqlash talabiga 3-kadr tasdiq sig'adimi.
4. (s4) Roofline: Orin Nano 7 Vt rejimda (cho'qqi 40 %), RK3588, Coral, Hailo-8L; yolov8n/s INT8 batch 1/6 (6 bo'lak birga) AI va rejim.
5. (s5) MiniYolo (1 sinf tutun) 320x180 va 640x360 kirish; hook va FlopCounterMode; batch 1/6 CPU vaqti; 1 Vt*soat energiyada nechta inferens bajarilishini CPU o'lchovidan (TDP taxminiy 15 Vt) baholang.
6. (s6) Co-design: 3 konfiguratsiya x FP16/INT8 x batch 1/6 x 4 platforma x 2 quvvat rejimi (yoz/qish); cheklovlar zaxira >= 1,5, inferens <= 2 s, o'rtacha quvvat, narx; Pareto narx-F1 va energiya-F1.
7. (s7) TCO 5 yil: minora edge (platforma + panel + akkumulyator) vs 'rasmlarni 4G orqali markazga yuborish' (4G ijarasi 240 $/yil + markaz server); 10 va 30 minora; tenglik nuqtasi

yo'qligini yoki borligini ko'rsating.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar, hisob-kitob va jadvallar — 25, Tahlil va asoslangan tanlov — 25, Hisobot sifati (grafik, sxema, til) — 20

14-variant. Futbol stadionida o'yinchilar va to'pni ko'p kamerali real vaqt kuzatuvi

qiyin

Soha: Sport

Professional klub stadionida 8 ta statsionar 4K kamera (3840x2160, 50 kadr/s) o'yin maydonini qoplaydi. Har kadrda 22 o'yinchi + hakamlar + to'p aniqlanadi, poza baholanadi va murabbiy planshetiga 3D traektoriyalar 150 ms ichida yetkaziladi (taktik tahlil va ofsayd yordami). To'p kichik (4K da ~12 piksel), shuning uchun kirish o'lchamini 1280 dan pastga tushirish sifatni keskin pasaytiradi. Stadionda texnik xona (3 kVt, 42U shkaf) bor; o'yin haftasiga 1-2 marta 2 soat, qolgan vaqt tizim bo'sh. Byudjet 80 000 $; bulut variantida 8 x 4K oqimni yuborish uchun 400 Mbit/s kanal kerak.

Yuklama	8 kamera x 50 kadr/s = 400 kadr/s 4K; har kadrda ~25 obyekt -> poza 10 000 inferens/s
Modellar	yolov8m @ 1280 (316 GFLOP, mAP 0,78 to'p), yolov8l @ 1280 (661, 0,84), yolov8x @ 1920 (2 320, 0,88); RTMPose-m 1,9 GFLOP/odam
SLO	p99 <= 150 ms, inferens <= 60 ms
Platformalar	2 x A100 server, 4 x L4 server, 2 x RTX 4090 ish stansiyasi, 1 x H100 PCIe (1 513 INT8 TOPS dense, 350 Vt, 25 000 $), 8 x AGX Orin (kamera boshida)
Cheklovlar	<= 80 000 $, <= 3 000 Vt, 4K dekod 8 x 2160p50
Dekodlash	4K50 oqim = 4 x 1080p50 = 6,7 x 1080p30; NVDEC soni: A100 - 5 (1080p30 ~ 10), L4 - 4 (~ 8), 4090 - 2 (~ 14), H100 - 7, AGX - 14
TCO	3 yil, 100 soat/yil foydalanish, bulut A100 3 $/soat + 400 Mbit/s kanal 1 500 $/oy

QADAMLAR

1. (s1) 400 kadr/s uchun 3 detektor konfiguratsiyasi + poza (25 obyekt) GFLOP va TFLOP/s; kerakli cho'qqi (samaradorlik 0,5 katta batch'da); 4K kadr xom hajmi (24,9 MB) x 400 = GB/s xotira yozuvi; 8 x 50 Mbit/s HEVC tarmoq.
2. (s2) 5 konfiguratsiya jadvali: jami TOPS, dekod sig'imi (4K50 oqimlar), Vt, narx; zaxira, kadr energiyasi; 8 mezonli matritsa (zaxira, kechikish, dekod, quvvat, narx, sifat to'p, ishonchlilik, ekotizim); TANLOV va kamera-boshi (AGX) varianti nima uchun yiqilishini raqam bilan ko'rsating.
3. (s3) 150 ms byudjet: 4K dekod (median 12 ms), tile'larga bo'lish, detektor, poza (25 obyekt batch), ko'p kamerali triangulyatsiya (median 15 ms, sigma 0,4), planshetga Wi-Fi (median 20 ms, sigma 0,6); 20 000 kadr Monte-Karlo; batch 1/4/8 (8 kamera sinxron kadr) ta'siri; p99 <= 150 ms maksimal batch.
4. (s4) Roofline: A100, L4, 4090, H100 (3 350 GB/s), AGX Orin tizza nuqtalari; yolov8l @1280 FP16/INT8 batch 1/8 va RTMPose batch 25/200 AI va rejim; 4090 (1 008 GB/s) va L4 (300 GB/s) farqi.

5. (s5) MiniYolo 2 sinf (odam, to'p) 256x144 / 512x288 / 1024x576 kirish; hook va FlopCounterMode; batch 1/8 CPU vaqti; to'p uchun kichik obyekt oqibatida imgsz pasaytirib bo'lmasligini FLOP o'sishi bilan birga tahlil qiling.
6. (s6) Co-design grid: 3 detektor x 2 format x batch 1/4/8 x 5 konfiguratsiya (+ poza yuklamasi qo'shiladi); cheklovlar zaxira >= 1,3, inferens+kutish <= 60 ms, dekod, quvvat, narx; Pareto narx-mAP(to'p); har cheklovni olib tashlab yaroqli soni jadvali.
7. (s7) TCO 3 yil: o'z serveri (100 soat/yil, lekin 24/7 'idle' 15 % quvvat) vs bulut A100 soatbay + 400 Mbit/s kanal vs 2 x 4090 ish stansiyasi; foydalanish soatlari 50-1 000/yil bo'yicha tenglik nuqtasi grafigi.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z); Strukturaviy sxema: 4K kamera (SDI/IP) -> dekod -> detektor/poza -> triangulyatsiya -> planshet (WebSocket) interfeyslari va o'tkazuvchanlik; 3 ta xavf (dekod chegarasi, to'p sifati, Wi-Fi) va kamaytirish choralari

Baholash (100 ball): Kod: parametrlangan, qayta ishlatiladigan funksiyalar — 25, Hisob, simulyatsiya va sezgirlik tahlili to'g'riligi — 20, Grafiklar va Pareto/tenglik nuqtasi tahlili — 20, Muhandislik xulosasi, strukturaviy sxema, xavflar — 25, Hisobot sifati — 10

15-variant. Telekanallar uchun jonli efirda avtomatik subtitr (nutqni tanish) serveri o'rta

Soha: Media

Milliy teleradiokompaniya 24 ta telekanal uchun jonli efirda o'zbek va rus tillarida avtomatik subtitr yaratmoqchi. Audio 16 kHz mono, 10 s bo'laklarga bo'linib Whisper oilasidagi modelga beriladi; subtitr efirdan 3 s dan ko'p kechikmasligi kerak (qonun talabi). Yuklama tasvir emas, audio soniya bilan hisoblanadi: 24 kanal x 1 real vaqt = 24 audio-soniya/s, ya'ni har 10 s da 24 ta bo'lak. Yangiliklar vaqtida sifat (WER) ustuvor, tungi takrorlarda tejamkor model yetarli. Server xonasi mavjud, 1,5 kVt bo'sh; byudjet 30 000 $; bulut (ma'lumot chiqib ketishi) ruxsat etilmaydi.

Yuklama	24 kanal x real vaqt; bo'lak 10 s -> 2,4 bo'lak/s (30 s bo'lak: 0,8/s)
Modellar (GFLOP / 30 s audio, WER)	whisper-tiny: 60, 0,28; whisper-base: 120, 0,22; whisper-small: 400, 0,15; bo'lak 10 s uchun FLOP/3
SLO	subtitr kechikishi <= 3 s (bo'lak 10 s bo'lsa: inferens <= 2 s)
Platformalar	Xeon 32 yadro (AMX), T4 server, L4 server, A10 server, AGX Orin x3
Cheklovlar	<= 30 000 $, <= 1 500 Vt, bulut yo'q
Formatlar	FP16, INT8 (WER +0,01), batch 1/8/24 (24 kanal birga)
TCO	5 yil, 24 -> 48 kanal, elektr 0,08 $/kVt*soat

QADAMLAR

1. (s1) KAMERALAR*FPS o'rniga kanal x bo'lak chastotasi: 3 model x 2 bo'lak uzunligi (10/30 s) uchun GFLOP/bo'lak, TFLOP/s talab, kerakli cho'qqi (samaradorlik 0,3 - avtoregressiv dekoder past samarali); vazn FP16/INT8 MB; audio oqimi (16 kHz x 2 bayt x 24) kB/s.
2. (s2) 5 platforma: zaxira 24 va 48 kanal uchun, bo'lak energiyasi (J); 6 mezonli matritsa (zaxira, kechikish, narx, quvvat, WER sig'imi, ekotizim/CTranslate2); TANLOV; AGX Orin x3 varianti nima uchun quvvatda yutadi, lekin ekotizimda yutqazadi.
3. (s3) 3 000 ms byudjet: bo'lak to'planishi (10 s bo'lak uchun kechikish bo'lak uzunligiga teng - buni alohida ko'rsating: shuning uchun 'sirpanuvchi oyna' 2 s qadam), VAD, enkoder, dekoder (token soni 20-80, uniform), subtitr formatlash (SRT/WebVTT), efirga qo'shish (200 ms); 10 000 bo'lak Monte-Karlo; batch 1/8/24 kutish.
4. (s4) Roofline: Xeon (300 GB/s), T4, L4, A10, AGX Orin; enkoder (AI yuqori) va dekoder qadami (AI past, vazn o'qish ustun) nuqtalarini batch 1/8/24 uchun alohida joylashtiring; qaysi qism xotira bilan cheklangan.
5. (s5) torch'da MiniWhisper (Conv1d x2 + 2 qatlam TransformerEncoderLayer d=128 + Linear) 80x300 kirish; hook (Conv1d, Linear) va FlopCounterMode (attention FLOP'ini qo'lda qo'shing); batch 1/8/24 CPU vaqti.
6. (s6) Co-design: 3 model x 2 bo'lak x 2 format x 3 batch x 5 platforma; cheklovlar zaxira >= 1,5 (48 kanal kelajak), inferens <= 2 s, narx, quvvat; Pareto narx-WER; kun/tun rejimi uchun ikki xil tanlov.

7. (s7) TCO 5 yil: L4 server vs Xeon AMX server vs 3 x AGX Orin; 24 va 48 kanal; elektr 0,08 va 0,2 $ sezgirlik; $/kanal/oy.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar) — 25, Hisob-kitob va simulyatsiya to'g'riligi — 25, Grafiklar (Roofline, byudjet, co-design, TCO) — 15, Tahlil, tanlov asosi va xavflar — 25, Hisobot sifati — 10

16-variant. Muzey va ko'rgazmalarda vizual qidiruv kiosklari: eksponatni rasmdan tanish oson

Soha: Turizm

Yirik tarix muzeyi 15 ta sensorli kiosk va mobil ilova orqali 'rasmga ol - eksponat haqida o'qi' xizmatini ishga tushirmoqda: foydalanuvchi suratidan embedding (ResNet/EfficientNet) olinadi va 12 000 eksponatli bazadan eng yaqin 5 tasi topiladi. Kiosklar pikda har 2 s da 1 so'rov beradi (jami 7,5 so'rov/s), mobil ilova mavsumda yana 10 so'rov/s qo'shadi. Javob 1,5 s ichida ko'rinishi kerak, aks holda tashrif buyuruvchi ketadi. Muzeyda kichik server xonasi (500 Vt) bor; kioskning o'zida ham hisoblash mumkin (har kioskda 20 Vt). Byudjet 8 000 $. Baza sintetik: 12 000 x 512 o'lchamli normallashtirilgan vektorlar.

Yuklama	15 kiosk x 0,5 so'rov/s + mobil 10 so'rov/s = 17,5 so'rov/s (pik x2)
Modellar	MobileNetV3-L @224 (0,44 GFLOP, top-1 proxy 0,85), EfficientNet-B0 (0,78, 0,89), ResNet50 (8,2, 0,92); qidiruv 12 000 x 512 skalyar ko'paytma (12,3 MFLOP)
SLO	p95 <= 1 500 ms, inferens+qidiruv <= 300 ms
Platformalar	RK3588 (kioskda), Orin Nano (kioskda), x86 mini PC 6 yadro (server), T4 server, Apple M2 Mac mini (15,8 TOPS NPU)
Cheklovlar	<= 8 000 $, kiosk <= 20 Vt, server <= 500 Vt
Tarmoq	muzey Wi-Fi (median 40 ms, sigma 0,8), surat 300 KB
TCO	3 yil, 15 -> 40 kiosk, elektr 0,09 $/kVt*soat

QADAMLAR

1. (s1) 17,5 va 35 so'rov/s uchun 3 model + qidiruv GFLOP/so'rov, TFLOP/s, kerakli cho'qqi (samaradorlik 0,3); baza hajmi (12 000 x 512 x FP32/FP16/INT8) MB va surat oqimi MB/s.
2. (s2) 5 platforma: zaxira 'kioskda' (0,5 so'rov/s) va 'serverda' (35/s) uchun, so'rov energiyasi (J); 6 mezonli matritsa (zaxira, kechikish, narx, quvvat, ekotizim, boshqaruv qulayligi - bitta server vs 15 qurilma); TANLOV.
3. (s3) 1 500 ms byudjet: surat olish+siqish (median 200 ms), Wi-Fi uzatish, oldindan ishlov, embedding, qidiruv, kontent sahifasini yuklash (median 150 ms, sigma 0,5); 20 000 so'rov Monte-Karlo p95; batch 1/4/8 - past so'rov tezligida batch kutish nima uchun zararli.
4. (s4) Roofline: 5 platforma tizza nuqtasi; MobileNetV3 va ResNet50 FP16 batch 1/8 AI; qidiruv (matritsa-vektor, AI ~0,5 FLOP/bayt) xotira bilan cheklanganini ko'rsating va baza INT8 bo'lsa qanday tezlashadi.
5. (s5) torch'da MiniEmbed (5 conv + GAP + Linear 512) 224x224 va 160x160; hook va FlopCounterMode; batch 1/8 CPU vaqti; qidiruvni torch.mm bilan 12 000 x 512 da o'lchab ms qo'shing.
6. (s6) Co-design: 3 model x imgsz 160/224 x FP16/INT8 x batch 1/4 x 5 platforma; cheklovlar zaxira >= 1,5 (pik), inferens+qidiruv <= 300 ms, narx (15 kiosk yoki 1 server), quvvat; Pareto narx-top-1.

7. (s7) TCO 3 yil: har kioskda RK3588/Orin Nano vs markaz server (mini PC yoki T4) + Wi-Fi; 15 va 40 kiosk; tenglik nuqtasi; mobil ilova uchun baribir server kerakligini xulosada hisobga oling.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar, hisob-kitob va jadvallar — 25, Tahlil va asoslangan tanlov — 25, Hisobot sifati (grafik, sxema, til) — 20

17-variant. Qurilish kompaniyasida dron tasvirlaridan beton yoriqlarini tungi ommaviy tahlil qilish o'rta

Soha: Qurilish

Yirik qurilish kompaniyasi 30 ta obyektda har kuni dron bilan beton konstruksiyalarni suratga oladi: kuniga jami 24 000 ta 4000x3000 tasvir. Tasvirlar kechqurun ofisga keladi va ertalab 7:00 gacha (8 soat oynada) yoriq segmentatsiyasi va o'lchami hisoboti tayyor bo'lishi kerak; demak, bu real vaqt emas, o'tkazuvchanlik (throughput) va narx masalasi. Tasvir 1024x768 bo'laklarga (tile) bo'linadi: bir tasvirdan 16 ta bo'lak. Mavsumda (bahor-kuz) yuklama 2 baravar oshadi, qishda 3 baravar kamayadi. Ofisda 1 kVt, byudjet 20 000 $; bulut GPU (L4 0,7 $/soat, A100 3 $/soat) ruxsat etilgan, chunki tasvirlar maxfiy emas.

Yuklama	24 000 tasvir/kun x 16 bo'lak = 384 000 bo'lak / 8 soat = 13,3 bo'lak/s (mavsum: 26,7/s)
Modellar (GFLOP @1024x768, IoU)	SegFormer-B0: 25, 0,72; U-Net-R18: 70, 0,78; DeepLabV3-R50: 180, 0,82
SLO	butun partiya <= 8 soat (p99 emas, o'tkazuvchanlik)
Platformalar	RTX 4090 ish stansiyasi, L4 server, T4 server, Xeon 16 AMX, bulut L4/A100 soatbay
Cheklovlar	<= 20 000 $ (yoki bulut), <= 1 000 Vt, 8 soat oyna
Formatlar	FP16, INT8 (IoU -0,01), batch 8/32/64
TCO	3 yil, mavsumiylik (6 oy x2, 3 oy x1, 3 oy x1/3), elektr 0,1 $/kVt*soat

QADAMLAR

1. (s1) KAMERALAR*FPS o'rniga bo'lak/s (o'rtacha, mavsum, qish) uchun 3 model GFLOP, TFLOP/s, kerakli cho'qqi (samaradorlik 0,55 - katta batch); kunlik tasvir hajmi (JPEG 6 MB x 24 000) GB va diskdan o'qish MB/s.
2. (s2) 5 platforma (bulut satri: soatbay narx, 0 $ apparat): zaxira 8 soat oynaga nisbatan (mavsumda ham), bo'lak energiyasi; 6 mezonli matritsa (zaxira, narx, quvvat, mavsumiy moslashuvchanlik, ekotizim, ma'lumot ko'chirish vaqti); TANLOV.
3. (s3) 8 soatlik 'byudjet' bo'laklar uchun o'tkazuvchanlik simulyatsiyasi: tasvirni o'qish+tile (median 40 ms), inferens batch 8/32/64, hisobot (vektorlash, o'lcham), yozish; 384 000 bo'lakni navbat modeli bilan simulyatsiya qilib jami vaqt p50/p95 (10 marta takror) va 8 soatga sig'ish ehtimoli; batch kattalashganda jami vaqt qanday kamayadi.
4. (s4) Roofline: 4090, L4, T4, Xeon, A100 tizza nuqtalari; U-Net-R18 va DeepLabV3 FP16/INT8 batch 8/64 AI; katta batch'da hamma H rejimda bo'lishini ko'rsating.
5. (s5) torch'da MiniSeg (4 conv enkoder + 2 ConvTranspose2d + 1 sinf) 256x192 va 512x384; hook va FlopCounterMode; batch 1/8/32 CPU vaqti va GFLOP/s; batch bilan bir bo'lak vaqti qanchaga kamayadi.
6. (s6) Co-design: 3 model x 2 format x 3 batch x 5 platforma x 3 mavsum yuklamasi; cheklovlar zaxira >= 1,2, 8 soat oyna, narx, quvvat; Pareto narx-IoU; mavsumda bulutga 'to'lib ketish'

(burst) gibrid variantini qo'shing.

7. (s7) TCO 3 yil: o'z 4090/L4 (yil bo'yi) vs bulut L4 soatbay (faqat ishlagan soatlar, mavsumiylik bilan) vs gibrid (T4 o'zimizniki + mavsumda bulut); kunlik tasvir 10 000-60 000 oralig'ida tenglik nuqtasi.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar) — 25, Hisob-kitob va simulyatsiya to'g'riligi — 25, Grafiklar (Roofline, byudjet, co-design, TCO) — 15, Tahlil, tanlov asosi va xavflar — 25, Hisobot sifati — 10

18-variant. Yer osti ko'mir konida metan sensorlari va konveyer kameralarini birgalikda tahlil qilish qiyin

Soha: Ko'mir / kon sanoati

Yer osti ko'mir konida 3 ta lava (qazish uchastkasi) bor; har lavada 400 ta metan/CO/harorat sensori (10 Hz, 6 kanal) va 6 ta konveyer tasmasi kamerasi (720p, 10 kadr/s) o'rnatilgan. Sensor oynasi (5 s) 1D-CNN bilan portlash xavfi darajasiga tasniflanadi, kameralar tasma yirtilishi va begona jism (metall) ni aniqlaydi. Metan ogohlantirishi 500 ms ichida, konveyer to'xtatish buyrug'i 200 ms ichida berilishi kerak. Yer ostidagi qurilmalar portlashdan himoyalangan (Ex) korpusda, passiv sovutish, 15 Vt chegara, -10...+40 C; yer ustiga 100 Mbit/s optika (kechikish 5 ms), lekin optika uzilganda tizim yer ostida mustaqil ishlashi shart. Byudjet 3 lava uchun 25 000 $ (Ex korpus har qurilmaga +800 $). To'xtash 1 soati 15 000 $ zarar.

Yuklama	sensor: 3 x 400 x 10 Hz oyna = 12 000 inferens/s (oyna 50x6); kamera: 3 x 6 x 10 = 180 kadr/s
Modellar	1D-CNN 50x6 (2 MFLOP, F1 0,93), TCN (15 MFLOP, 0,96); yolov8n @ 640 (8,7 GFLOP, mAP 0,80), yolov8s (28,6, 0,86)
SLO	metan <= 500 ms p99, konveyer to'xtatish <= 200 ms p99
Platformalar	Orin Nano (Ex), Orin NX (Ex), Hailo-8 + ARM (Ex), RK3588 (Ex), yer usti: T4 server, L4 server
Cheklovlar	<= 15 Vt passiv (cho'qqi 70 %), Ex korpus +800 $, <= 25 000 $, oflayn ishlash shart
Quvvat rejimi	Orin NX 10/15/25 Vt -> 40/70/100 % cho'qqi; Orin Nano 7/15 Vt -> 50/100 %
TCO	5 yil, elektr 0,06 $/kVt*soat, Ex sertifikat 5 yilda yangilanadi (+500 $), to'xtash 15 000 $/soat

QADAMLAR

1. (s1) Ikki yuklama turi uchun alohida hisob: sensor oynalari (lava 4 000 inferens/s) va kameralar (lava 60 kadr/s): GFLOP, TFLOP/s, kerakli cho'qqi (sensor samaradorlik 0,25, kamera 0,45); jami lava talabi; sensor oqimi (6 x 4 bayt x 10 Hz x 400) kbit/s va 6 x 2 Mbit/s kamera oqimi 100 Mbit/s optika uchun.
2. (s2) 6 platforma: Ex quvvat rejimida (15 Vt, passiv -> cho'qqi 70 %) haqiqiy TOPS, zaxira lava talabiga, inferens_J; 8 mezonli matritsa (zaxira, kechikish, Ex/harorat, quvvat, narx + Ex korpus, oflayn ishlash, dekod 6 oqim, ekotizim); yer osti TANLOVI va yer usti server ('ikkinchi fikr' va arxiv) roli.
3. (s3) Ikki byudjet: 500 ms (sensor: konsentrator RS-485 median 60 ms, oyna, inferens, ogohlantirish sirena) va 200 ms (kamera: dekod, inferens, kuzatuv, PLC to'xtatish 20 ms); 20 000 hodisa Monte-Karlo har biri uchun p99; sensor partiyasi 1/400/4000 (bitta lava bir yo'la) ta'siri; optika uzilgan rejimda (yer osti mustaqil) byudjet bajariladimi.
4. (s4) Roofline: Orin Nano/NX (15 Vt rejim cho'qqisi), RK3588, Hailo-8, T4, L4; 1D-CNN batch 1/400/4000 va yolov8n INT8 batch 1/6 AI va rejim; sensor modeli nima uchun faqat katta

partiyada H rejimga chiqadi.

5. (s5) torch'da 1D-CNN (Conv1d x3 + Linear, 50x6) va MiniYolo 2 sinf 320x180; hook va FlopCounterMode; batch 1/400 (sensor) va 1/6 (kamera) CPU vaqti; ikkala yuklamani bitta jarayonda ketma-ket bajarib lava uchun ms/sikl.
6. (s6) Co-design grid: (2 sensor modeli x 3 partiya) x (2 detektor x imgsz 416/640 x 2 format x batch 1/6) x 4 Ex platforma x 2 quvvat rejimi; cheklovlar zaxira >= 2 (xavfsizlik), ikkala kechikish, 15 Vt, narx (3 lava + Ex), oflayn; Pareto narx-(F1+mAP)/2; har cheklovni olib tashlab yaroqli soni.
7. (s7) TCO 5 yil: har lavada Ex edge (Orin NX / RK3588) + yer usti arxiv server vs 'hammasi yer ustida' (T4/L4 + optika, oflayn talab buziladi - jarima sifatida yiliga 2 soat to'xtash 15 000 $/soat); lava soni 1-8 tenglik nuqtasi; Ex sertifikat va to'xtash zararining ulushi.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z); Strukturaviy sxema: sensor konsentrator (RS-485/Modbus) + kamera (RTSP) -> Ex edge -> sirena/PLC va yer usti server (optika) interfeyslari, oflayn rejim tavsifi; Quvvat rejimi tahlili: 10/15/25 Vt rejimlarida zaxira va yaroqli konfiguratsiyalar soni jadvali

Baholash (100 ball): Kod: parametrlangan, qayta ishlatiladigan funksiyalar — 25, Hisob, simulyatsiya va sezgirlik tahlili to'g'riligi — 20, Grafiklar va Pareto/tenglik nuqtasi tahlili — 20, Muhandislik xulosasi, strukturaviy sxema, xavflar — 25, Hisobot sifati — 10

19-variant. Bankomat tarmog'ida yuz orqali tasdiqlash va shubhali xatti-harakatni aniqlash oson

Soha: Bank

Tijorat banki 500 ta bankomatga kamera orqali 'yuz bilan tasdiqlash' va 'niqob/shlem/ikkinchi odam' kabi shubhali holatlarni aniqlashni joriy qilmoqda. Bankomat o'rtacha har 90 s da bir mijozga xizmat ko'rsatadi; tasdiqlash uchun 5 kadr, shubhali holat uchun sessiya davomida 1 kadr/s tahlil qilinadi. Tasdiqlash 800 ms ichida bo'lishi kerak, aks holda mijoz PIN rejimiga o'tadi. Bankomat ichida 10 Vt bo'sh quvvat va sanoat PC (2 yadro) bor; markaziy ma'lumot markazi 2 ta zonada; bankomat -> markaz kanali 2 Mbit/s (median 40 ms). Biometrik ma'lumot bankdan chiqmasligi shart; byudjet bankomat boshiga 300 $ yoki markazda 30 000 $.

Yuklama	500 bankomat x (5 kadr/90 s + 1 kadr/s x 30 s sessiya/90 s) ~ 0,39 inferens/s har biri = 195 inferens/s (pik 2x)
Modellar	SCRFD-0.5G (0,5 GFLOP) + MobileFaceNet (0,45, TAR 0,97) + yolov8n shubha @ 320 (2,2); og'ir: SCRFD-2.5G + ArcFace-R50 (12,6, TAR 0,995)
SLO	tasdiqlash p95 <= 800 ms, inferens <= 150 ms
Platformalar	mavjud bankomat PC (2 yadro, ~0,05 TFLOP/s, 0 $), Hailo-8L M.2 (bankomatga), Orin Nano, T4 server (markaz), L4 server (markaz)
Cheklovlar	<= 10 Vt bankomatda, <= 300 $/bankomat yoki <= 30 000 $ markaz, 2 zona
Tarmoq	2 Mbit/s, median 40 ms, sigma 0,5; kadr JPEG 150 KB
TCO	5 yil, 500 -> 1 200 bankomat, elektr 0,1 $/kVt*soat

QADAMLAR

1. (s1) Bankomat (0,39 inferens/s) va markaz (195/s, pik 390/s) uchun 2 zanjir GFLOP, TFLOP/s, kerakli cho'qqi (samaradorlik 0,3); 5 kadr x 150 KB ni 2 Mbit/s kanaldan yuborish vaqti (ms) - markaz varianti uchun byudjetga ta'sirini hisoblang.
2. (s2) 5 platforma: zaxira bankomat va markaz uchun (mavjud PC yengil zanjirni ko'taradimi?), inferens_J; 6 mezonli matritsa (zaxira, kechikish, narx, quvvat, biometrik maxfiylik, ekotizim); TANLOV ikki arxitektura uchun.
3. (s3) 800 ms byudjet: kamera (5 kadr yig'ish 200 ms), yuz aniqlash + embedding, markaz uchun +uzatish (5 x 150 KB / 2 Mbit/s) va kanal kechikishi, bazadan solishtirish (median 30 ms), javob; 20 000 sessiya Monte-Karlo p95; qaysi arxitektura SLO'ni bajaradi; batch 1/8/32 markazda.
4. (s4) Roofline: bankomat PC (20 GB/s), Hailo-8L, Orin Nano, T4, L4; MobileFaceNet va ArcFace-R50 FP16/INT8 batch 1/32 AI va rejim.
5. (s5) torch'da MiniFace (4 conv + GAP + Linear 128) 112x112 va MiniYolo 320x320 (3 sinf shubha); hook va FlopCounterMode; torch.set_num_threads(2) bilan batch 1/5 CPU vaqti - mavjud bankomat PC'ga o'xshash muhitda zanjir ms.

6. (s6) Co-design: 2 zanjir x FP16/INT8 x batch 1/5/32 x 5 platforma; cheklovlar zaxira >= 1,5 (pik), inferens <= 150 ms, narx (500 bankomat yoki markaz 2 zona), quvvat; Pareto narx-TAR.
7. (s7) TCO 5 yil: har bankomatda Hailo-8L (300 $) vs mavjud PC (0 $, yengil zanjir) vs markaz T4/L4 2 zona + kanal; 500 va 1 200 bankomat; tenglik nuqtasi; xulosada biometrik maxfiylik va kanal uzilishini hisobga oling.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar, hisob-kitob va jadvallar — 25, Tahlil va asoslangan tanlov — 25, Hisobot sifati (grafik, sxema, til) — 20

20-variant. Davlat xizmatlari markazida hujjatlarni raqamlashtirish: skan -> tasnif -> OCR -> maydon ajratish qiyin

Soha: Davlat xizmatlari

Respublika davlat xizmatlari agentligi 200 ta markazda arizalar, pasport nusxalari va ma'lumotnomalarni skaner qiladi: ish vaqtida jami 250 sahifa/min (pik 600/min), A4 300 dpi (2480x3508). Har sahifa uchun 4 bosqichli zanjir: hujjat turi tasnifi (12 sinf), matn bloklarini aniqlash, OCR (o'rtacha 40 qator), kalit maydonlarni ajratish (kichik Transformer). Operator skan qilgach 2 s ichida to'ldirilgan shaklni ko'rishi kerak (p95); aks holda navbat o'sadi. Shaxsiy ma'lumotlar - faqat davlat ma'lumot markazi (2 zona, 10 kVt) yoki markazning o'zida ishlov. Byudjet 120 000 $; har markazda 300 Vt va 1 server o'rni bor; markaz -> DPM kanali 20 Mbit/s (median 25 ms). Model zanjiri bosqichlari turli xil AI'ga ega, shuning uchun bir platforma hammasiga mos kelmasligi mumkin.

Yuklama	250 sahifa/min = 4,2 sahifa/s (pik 10/s); har sahifa: 1 tasnif + 1 detektor + 40 OCR qator + 1 maydon modeli
Modellar (GFLOP)	tasnif EfficientNet-B0 @224: 0,78; detektor DBNet-R18 @1024: 48; OCR SVTR-tiny 32x320: 0,7/qator (og'ir SVTR-small 3,5); maydon LayoutLM-lite 512 token: 22
SLO	p95 <= 2 000 ms sahifa, inferens zanjiri <= 800 ms
Platformalar	markazda: Orin NX, RTX A2000 PC, Xeon 8 AMX; DPM: 2 x L4 server, 2 x A10 server, 4 x T4 server
Cheklovlar	<= 120 000 $, markazda <= 300 Vt, DPM 2 zona, ma'lumot davlat tarmog'ida
Tarmoq	20 Mbit/s, skan JPEG 1,5 MB (markazdan DPM ga)
TCO	5 yil, 200 -> 400 markaz, elektr 0,08 $/kVt*soat, kanal 300 $/yil/markaz

QADAMLAR

1. (s1) Bir sahifa zanjiri jami GFLOP (40 OCR qator bilan) ikki konfiguratsiyada (tiny/small OCR); 4,2 va 10 sahifa/s uchun TFLOP/s, kerakli cho'qqi (bosqichlarga alohida samaradorlik: CNN 0,45, OCR kichik batch 0,2, Transformer 0,3); markaz (1,25 sahifa/min) uchun ham; 1,5 MB skan x sahifa/s ni 20 Mbit/s kanalga nisbatan tekshiring.
2. (s2) 6 platforma/konfiguratsiya: zaxira markaz va DPM yuklamasiga, sahifa energiyasi; 8 mezonli matritsa (zaxira, kechikish, narx, quvvat, maxfiylik/joylashuv, 2 zona ishonchlilik, ekotizim, boshqaruv 200 qurilma vs 1 klaster); 'markazda' va 'DPM' TANLOVI hamda gibrid (markazda tasnif+detektor, DPM'da OCR+maydon) varianti.
3. (s3) 2 000 ms byudjet: skaner (median 600 ms, sigma 0,2), uzatish (DPM uchun 1,5 MB / 20 Mbit/s + 25 ms), tasnif, detektor, OCR (40 qator: batch 1 ketma-ket vs batch 40), maydon modeli, shakl render (100 ms); 20 000 sahifa Monte-Karlo p95/p99 uch arxitekturada; OCR batch'lash kechikishni necha baravar kamaytiradi.
4. (s4) Roofline: Orin NX, A2000, Xeon, L4, A10, T4; 4 bosqich uchun alohida AI (DBNet yuqori, OCR qator past, LayoutLM o'rtacha) batch 1/40 va rejim; qaysi bosqich qaysi platformada

xotira bilan cheklangan - gibrid taqsimot asosi.

5. (s5) torch'da 3 kichik model: MiniCls (4 conv + GAP + Linear 12) 224x224, MiniOCR (3 conv + GRU 64 + Linear 100) 32x160, MiniLayout (2 qatlam TransformerEncoderLayer d=128, 128 token); hook va FlopCounterMode (attention va GRU qo'lda); batch 1/8/40 CPU vaqti; zanjir ms/sahifa.
6. (s6) Co-design grid: 2 OCR x OCR batch 1/8/40 x FP16/INT8 x 3 joylashuv (markaz/DPM/gibrid) x 6 platforma; cheklovlar zaxira >= 1,5 (pik), zanjir inferens <= 800 ms, narx (200 markaz yoki DPM), quvvat, maxfiylik; Pareto narx-aniqlik (tiny 0,96, small 0,985); har cheklovni olib tashlab yaroqli soni.
7. (s7) TCO 5 yil: har markazda Orin NX/A2000 vs DPM 2 zona L4/A10 + 200 kanal vs gibrid; 200 va 400 markaz; markaz soni 50-500 tenglik nuqtasi; sezgirlik: kanal 100-600 $/yil, elektr 0,05-0,2 $.

Topshiriladi: Kod (.py yoki .ipynb), har qadam alohida funksiya/bo'lim; Natijalar jadvallari (CSV yoki markdown); Grafiklar (PNG): kechikish byudjeti, Roofline, co-design, TCO; Hisobot (PDF): talablar matritsasi, tanlov asosi, strukturaviy sxema matni, xulosa (>= 150 so'z); Strukturaviy sxema: skaner -> markaz ishlov -> DPM zanjiri -> shakl xizmati interfeyslari (TWAIN, HTTPS/gRPC, navbat) va o'tkazuvchanlik jadvali; 3 ta xavf (kanal, OCR sifati, 2 zona sinxron) va kamaytirish choralari

Baholash (100 ball): Kod: parametrlangan, qayta ishlatiladigan funksiyalar — 25, Hisob, simulyatsiya va sezgirlik tahlili to'g'riligi — 20, Grafiklar va Pareto/tenglik nuqtasi tahlili — 20, Muhandislik xulosasi, strukturaviy sxema, xavflar — 25, Hisobot sifati — 10

70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A2

GPU va sun'iy intellekt tezlatkichlaridan foydalangan holda parallel hisoblashni tashkil etish

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A2
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A2
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu virtual laboratoriyada siz matritsa ko'paytirish va konvolyutsiyani — neyron tarmoqning asosiy yadrolarini — bir necha darajada parallellashtirasiz va bir xil metodika (isinish, takror, median) bilan o'lchaysiz: sof Python → NumPy vektorlash → Numba @njit va prange (OpenMP muqobili) → kesh bloklash → PyTorch iplari → multiprocessing → Amdahl qonuni. Muhim: sandbox'da CUDA va GPU mavjud emas, shuning uchun real labdagi CUDA yadrolari va Tensor Core o'lchovlari o'rniga 2 yadroli CPU'da parallelizm o'rganiladi; har qadam oxirida «GPU bo'lsa shu kod qanday o'zgaradi» izohi berilgan. Massivlar kichik (n ≤ 1024), chunki Numba birinchi kompilyatsiyasi va 2 yadro vaqt chegarasini talab qiladi.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A2`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A2 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Sof Python vs NumPy vektorlash
2. 2-qadam. Numba @njit va prange — OpenMP'ning Python muqobili
3. 3-qadam. Kesh bloklash (tiled) vs naive matritsa ko'paytirish
4. 4-qadam. Konvolyutsiyani to'rt usulda parallellashtirish
5. 5-qadam. PyTorch: GPU o'rniga ko'p yadro — 1 vs 2 ip, formatlar, yadro birlashtirish
6. 6-qadam. Ma'lumot parallelligi: multiprocessing bilan 2 jarayon
7. 7-qadam. Amdahl va Gustafson qonunlari: nazariya va 2 yadroda o'lchov
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Sof Python vs NumPy vektorlash

MAQSAD

Uch sikldagi sof Python matritsa ko'paytirish (n=64, 128) va NumPy BLAS chaqiruvi bir xil metodika bilan o'lchanadi, GFLOP/s va tezlanish hisoblanadi; so'ng 1M elementli GELU uchun list comprehension va vektorlangan NumPy solishtiriladi. Kutilgan natija: matmul'da 250–500×, GELU'da ~10× tezlanish; NumPy'ning oraliq massivlar tufayli xotira trafigi ko'rsatiladi. n va N ni o'zgartirib tezlanish qanday o'sishini kuzating.

> **Kutilgan natija:** Sof Python ~0,04 GFLOP/s, NumPy FP64 10–20 GFLOP/s, FP32 n=1024 da ~45 GFLOP/s; GELU 236 ms vs 21 ms.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Sof Python vs NumPy vektorlash: matritsa ko'paytirish va elementlararo amal
# Bu yerda GPU yo'q (CUDA mavjud emas) — avval bitta yadroda interpretator va vektorlangan
# kutubxona o'rtasidagi farqni o'lchaymiz. O'lchov metodikasi: isinish + bir necha takror,
median.
import time
import random
import numpy as np

random.seed(0)
np.random.seed(0)


def median_vaqt(f, takror=3):
    """Funksiyani takror marta bajarib median vaqtni (s) qaytaradi (1 isinish)."""
    f()
    v = []
    for _ in range(takror):
        t0 = time.perf_counter()
        f()
        v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


# --- (a) Matritsa ko'paytirish ---
def matmul_python(A, B, n):
    C = [[0.0] * n for _ in range(n)]
    for i in range(n):
        Ai = A[i]
        Ci = C[i]
        for k in range(n):                # ikj tartib: B qatori bo'ylab ketma-ket murojaat
            a = Ai[k]
            Bk = B[k]
            for j in range(n):
                Ci[j] += a * Bk[j]
    return C


print("=== Sof Python vs NumPy ===")
print(f"{'variant':24s} {'n':>5s} {'vaqt, ms':>10s} {'GFLOP/s':>9s}")
natijalar = {}
for n in (64, 128):
    A = [[random.random() for _ in range(n)] for _ in range(n)]
```

```
    B = [[random.random() for _ in range(n)] for _ in range(n)]
    flop = 2 * n ** 3
    t_py = median_vaqt(lambda: matmul_python(A, B, n), takror=1 if n >= 128 else 3)
    An, Bn = np.array(A, dtype=np.float64), np.array(B, dtype=np.float64)
    t_np = median_vaqt(lambda: An @ Bn, takror=5)
    # to'g'rilikni tekshirish
    xato = np.abs(np.array(matmul_python(A, B, n)) - An @ Bn).max()
    print(f"{'sof Python (3 sikl)':24s} {n:5d} {t_py * 1e3:10.2f} {flop / t_py / 1e9:9.4f}")
    print(f"{'NumPy @ (BLAS, FP64)':24s} {n:5d} {t_np * 1e3:10.3f} {flop / t_np / 1e9:9.2f}")
    natijalar[n] = t_py / t_np
    print(f"  tezlanish {t_py / t_np:8.0f}x, maks xato {xato:.1e}")

for n in (512, 1024):
    An = np.random.rand(n, n).astype(np.float32)
    Bn = np.random.rand(n, n).astype(np.float32)
    t_np = median_vaqt(lambda: An @ Bn, takror=5)
    print(f"{'NumPy @ (BLAS, FP32)':24s} {n:5d} {t_np * 1e3:10.2f} {2 * n ** 3 / t_np /
1e9:9.2f}")

# --- (b) Elementlararo amal: GELU faollashtirish 1M element ---
N = 1_000_000
x_list = [random.uniform(-3, 3) for _ in range(N)]
x_np = np.array(x_list, dtype=np.float32)
import math

def gelu_python(xs):
    return [0.5 * x * (1 + math.tanh(0.79788456 * (x + 0.044715 * x ** 3))) for x in xs]

def gelu_numpy(x):
    return 0.5 * x * (1 + np.tanh(0.79788456 * (x + 0.044715 * x ** 3)))

t_py = median_vaqt(lambda: gelu_python(x_list), takror=1)
t_np = median_vaqt(lambda: gelu_numpy(x_np), takror=5)
print(f"\n--- GELU, {N / 1e6:.0f}M element ---")
print(f"sof Python (list comprehension): {t_py * 1e3:8.1f} ms  ({N / t_py / 1e6:6.1f} M
element/s)")
print(f"NumPy (vektorlangan):            {t_np * 1e3:8.1f} ms  ({N / t_np / 1e6:6.1f} M
element/s)  tezlanish {t_py / t_np:.0f}x")
# NumPy'da har amal alohida o'tish: ~8 ta oraliq massiv (har biri 4 MB) yoziladi va o'qiladi
trafik = 8 * 2 * N * 4
print(f"NumPy samarali xotira trafigi: ~{trafik / 1e6:.0f} MB -> {trafik / t_np / 1e9:.1f} GB/s
(yadro birlashtirilmagan)")
print("\nXulosa: interpretator 1 element uchun ~100 ns sarflaydi; BLAS/vektor buyruqlar 100-
1000x tez.")
print("GPU bo'lsa: shu NumPy kodi CuPy'da o'zgarishsiz ishlaydi, lekin 1M element uchun yadro
ishga tushirish xarajati (~10 mks) sezilarli.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Sof Python vs NumPy ===`, `GELU`

TAJRIBA QILIB KO‘RING

- matmul_python'ni ijk tartibiga o'zgartiring — sof Python'da ham kesh ta'siri sezilarmi?
- GELU'ni np.float64 bilan qayta o'lchang: xotira trafigi 2 baravar oshganda vaqt qanday o'zgaradi?
- NumPy matmul uchun n=2048 qo'shing va GFLOP/s barqaror qoladimi, tekshiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Numba @njit va prange — OpenMP'ning Python muqobili

MAQSAD

@njit matmul (ikj) kompilyatsiya qilinadi, parallel=True + prange varianti 1 va 2 ipda (numba.set_num_threads) o'lchanadi, NumPy BLAS bilan solishtiriladi; alohida reduktsiya (sum x²) misoli 2 ipda xotira o'tkazuvchanligi cheklovini ko'rsatadi. Kutilgan natija: JIT kompilyatsiya ~0,3–1 s, prange 2 ipda ~1,6–1,9× tezlanish, BLAS yana 1,5× tezroq. Bu real labdagi `#pragma omp parallel for` va `reduction(+:s)` ning to'g'ridan-to'g'ri muqobili.

> **Kutilgan natija:** Kompilyatsiya vaqtlari, threading layer nomi, 4 qatorli jadval (seq ~16 GFLOP/s, prange 2 ip ~27, BLAS ~39), reduktsiya 11 → 22 GB/s.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Numba @njit va prange — OpenMP'ning Python muqobili
# @njit kodni mashina kodiga kompilyatsiya qiladi (JIT), parallel=True + prange esa siklni
# iplar orasida taqsimlaydi (OpenMP `#pragma omp parallel for` kabi). Sandbox'da 2 yadro bor.
import time
import numpy as np
import numba
from numba import njit, prange

numba.set_num_threads(2)


@njit(cache=False)
def matmul_seq(A, B, C):
    n, m, p = A.shape[0], A.shape[1], B.shape[1]
    for i in range(n):
        for k in range(m):                # ikj: B qatori bo'ylab ketma-ket murojaat (kesh
uchun qulay)
            a = A[i, k]
            for j in range(p):
                C[i, j] += a * B[k, j]


@njit(parallel=True, cache=False)
def matmul_par(A, B, C):
    n, m, p = A.shape[0], A.shape[1], B.shape[1]
    for i in prange(n):                   # tashqi sikl iplar orasida taqsimlanadi
(schedule(static) kabi)
        for k in range(m):
            a = A[i, k]
            for j in range(p):
                C[i, j] += a * B[k, j]


@njit(parallel=True, cache=False)
def yigindi_par(x):
    s = 0.0
    for i in prange(x.shape[0]):          # reduction(+:s) ga ekvivalent — numba avtomatik
aniqlaydi
        s += x[i] * x[i]
    return s
```

```
def olchov(f, *args, takror=5):
    v = []
    for _ in range(takror):
        args[-1][:] = 0.0
        t0 = time.perf_counter()
        f(*args)
        v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


n = 256
A = np.random.rand(n, n).astype(np.float32)
B = np.random.rand(n, n).astype(np.float32)
C = np.zeros((n, n), dtype=np.float32)
flop = 2 * n ** 3

print("=== Numba njit va prange ===")
print(f"Numba {numba.__version__}, iplar: {numba.get_num_threads()}, threading layer
tanlanadi...")

# Birinchi chaqiruv = kompilyatsiya vaqti (OpenMP'da bu kompilyator vaqti, bu yerda runtime'da)
t0 = time.perf_counter(); matmul_seq(A, B, C); t_komp_seq = time.perf_counter() - t0
C[:] = 0
t0 = time.perf_counter(); matmul_par(A, B, C); t_komp_par = time.perf_counter() - t0
print(f"JIT kompilyatsiya: seq {t_komp_seq:.2f} s, parallel {t_komp_par:.2f} s (bir marta,
keyin keshdan)")
print(f"Threading layer: {numba.threading_layer()}")

# --- Python etalon (kichik n) bilan to'g'rilik ---
etalon = A @ B
C[:] = 0; matmul_par(A, B, C)
print(f"Maks xato (par vs NumPy): {np.abs(C - etalon).max():.2e}")

print(f"\n{'variant':22s} {'iplar':>5s} {'vaqt, ms':>9s} {'GFLOP/s':>8s} {'tezlanish':>9s}")
t_seq = olchov(matmul_seq, A, B, C)
print(f"{'njit seq (ikj)':22s} {1:5d} {t_seq * 1e3:9.2f} {flop / t_seq / 1e9:8.2f} {1.0:9.2f}")
for iplar in (1, 2):
    numba.set_num_threads(iplar)
    t = olchov(matmul_par, A, B, C)
    print(f"{'njit parallel prange':22s} {iplar:5d} {t * 1e3:9.2f} {flop / t / 1e9:8.2f} {t_seq
/ t:9.2f}")
numba.set_num_threads(2)
t_np = olchov(lambda a, b, c: np.matmul(a, b, out=c), A, B, C)
print(f"{'NumPy BLAS (2 ip)':22s} {2:5d} {t_np * 1e3:9.2f} {flop / t_np / 1e9:8.2f} {t_seq /
t_np:9.2f}")

# --- Reduktsiya (yig'indi) 1 vs 2 ip ---
x = np.random.rand(4_000_000).astype(np.float64)
yigindi_par(x)                                  # kompilyatsiya
print("\n--- Reduktsiya: sum(x*x), 4M element ---")
for iplar in (1, 2):
    numba.set_num_threads(iplar)
    v = []
    for _ in range(7):
        t0 = time.perf_counter(); s = yigindi_par(x); v.append(time.perf_counter() - t0)
    t = sorted(v)[3]
    print(f"iplar={iplar}: {t * 1e3:7.2f} ms, {2 * x.nbytes / t / 1e9:5.1f} GB/s samarali
(o'qish), natija {s:.1f}")
print("Reduktsiya xotira bilan cheklangan: 2 ip hisobni emas, DRAM o'tkazuvchanligini
bo'lishadi.")
print("\nGPU bo'lsa: @njit o'rniga @cuda.jit, prange o'rniga cuda.grid(2) indekslari, massivlar
cuda.to_device bilan ko'chiriladi,")
print("vaqt o'lchashdan oldin cuda.synchronize() shart.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Numba njit va prange ===`, `Reduktsiya`

TAJRIBA QILIB KO‘RING

- prange'ni ichki (j) siklga ko'chiring — tezlanish nima uchun yo'qoladi (ip yaratish xarajati)?
- n=512 va n=1024 uchun o'lchab, 2 ip samaradorligi (tezlanish/2) o'lcham bilan qanday o'zgarishini jadvalga yozing.
- fastmath=True bayrog'ini qo'shing va GFLOP/s hamda maks xato qanday o'zgarishini solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Kesh bloklash (tiled) vs naive matritsa ko'paytirish

MAQSAD

Uch algoritm bir ipda solishtiriladi: naive ijk (B ustun bo'ylab — har qadam yangi kesh satri), ikj (qator bo'ylab, vektorlashadi) va bloklangan (tiled) variant; so'ng blok o'lchami 8…256 ta'siri o'lchanadi. Kutilgan natija: ijk → ikj ~10× tezlanish; kichik bloklar sikl xarajati tufayli sekin, 128–256 da ikj darajasiga yetadi. Kodda Numba'ning muhim nozik jihati ko'rsatilgan: range(jj, jj+bs) indekslari manfiy-tekshiruv tufayli vektorlashmaydi, slice bilan hal qilinadi.

> **Kutilgan natija:** n=256/512/1024 jadvali: naive ~1,7 GFLOP/s, ikj ~15–17, tiled ~12; blok o'lchami jadvali 5 → 15 GFLOP/s.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Kesh bloklash (cache tiling): naive (ijk) vs ikj vs bloklangan (tiled) matritsa
ko'paytirish
# GPU'dagi umumiy xotirali (shared memory) yadroning CPU muqobili — blok L1/L2 keshga sig'adi
va har
# yuklangan element ko'p marta qayta ishlatiladi. Numba bilan 1 ipda o'lchaymiz (algoritm
farqini ko'rish uchun).
import time
import numpy as np
from numba import njit

BS = 64   # blok o'lchami (o'zgartirib ko'ring: 16, 32, 128)


@njit
def matmul_ijk(A, B, C):
    n = A.shape[0]
    for i in range(n):
        for j in range(n):
            s = 0.0
            for k in range(n):            # B[k, j] — ustun bo'ylab, har qadam yangi kesh
satri
                s += A[i, k] * B[k, j]
            C[i, j] = s


@njit
def matmul_ikj(A, B, C):
    n = A.shape[0]
    for i in range(n):
        for k in range(n):
            a = A[i, k]
            for j in range(n):            # B[k, j] — qator bo'ylab ketma-ket, vektorlashadi
                C[i, j] += a * B[k, j]


@njit
def matmul_tiled(A, B, C, bs):
    """Bloklangan ikj. Muhim nozik jihat: numba `range(jj, jj+bs)` indekslarini manfiy bo'lishi
mumkin deb hisoblab
    har murojaatga tekshiruv qo'shadi va ichki sikl vektorlashmaydi (2 GFLOP/s). Qator
bo'lagini (slice) olib,
    0 dan boshlanadigan indeks bilan yursak — vektorlashadi. n bs ga karrali deb olinadi."""
```

```
    n = A.shape[0]
    for ii in range(0, n, bs):
        for kk in range(0, n, bs):
            for jj in range(0, n, bs):
                for i in range(ii, ii + bs):
                    Ci = C[i, jj:jj + bs]
                    for k in range(kk, kk + bs):
                        a = A[i, k]
                        Bk = B[k, jj:jj + bs]
                        for j in range(bs):
                            Ci[j] += a * Bk[j]


def olchov(f, A, B, C, *extra, takror=3):
    v = []
    for _ in range(takror):
        C[:] = 0
        t0 = time.perf_counter()
        f(A, B, C, *extra)
        v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


print("=== Kesh bloklash ===")
# kompilyatsiya (kichik massivda)
a0 = np.ones((8, 8), np.float32); c0 = np.zeros((8, 8), np.float32)
matmul_ijk(a0, a0, c0); matmul_ikj(a0, a0, c0); matmul_tiled(a0, a0, c0, 4)

print(f"{'variant':16s} {'n':>5s} {'xotira, MB':>10s} {'vaqt, ms':>9s} {'GFLOP/s':>8s}
{'xato':>8s}")
for n in (256, 512, 1024):
    A = np.random.rand(n, n).astype(np.float32)
    B = np.random.rand(n, n).astype(np.float32)
    C = np.zeros((n, n), np.float32)
    etalon = A @ B
    flop = 2 * n ** 3
    mb = 3 * n * n * 4 / 1e6
    for nom, f, extra in (("naive ijk", matmul_ijk, ()), ("ikj", matmul_ikj, ()), (f"tiled bs=
{BS}", matmul_tiled, (BS,))):
        if nom == "naive ijk" and n > 512:
            continue                        # sodda variant katta n da juda sekin (bir necha
soniya)
        t = olchov(f, A, B, C, *extra, takror=1 if n >= 512 else 3)
        print(f"{nom:16s} {n:5d} {mb:10.1f} {t * 1e3:9.1f} {flop / t / 1e9:8.2f} {np.abs(C -
etalon).max():8.1e}")

# Blok o'lchami ta'siri (n=1024)
n = 1024
A = np.random.rand(n, n).astype(np.float32); B = np.random.rand(n, n).astype(np.float32); C =
np.zeros((n, n), np.float32)
print(f"\n--- Blok o'lchami ta'siri, n={n} ---")
print(f"{'bs':>4s} {'blok hajmi (3 blok), KB':>24s} {'vaqt, ms':>9s} {'GFLOP/s':>8s}")
for bs in (8, 16, 32, 64, 128, 256):
    t = olchov(matmul_tiled, A, B, C, bs, takror=1)
    print(f"{bs:4d} {3 * bs * bs * 4 / 1024:24.1f} {t * 1e3:9.1f} {2 * n ** 3 / t / 1e9:8.2f}")
print("\nArifmetik intensivlik: ijk ~ 0.25 FLOP/bayt (har 2 FLOP uchun 8 bayt), tiled ~ bs/6
FLOP/bayt.")
print("Eslatma: bu serverda L3 kesh katta, shuning uchun n=1024 da ikj ham keshdan ishlaydi va
tiled undan oshmasligi mumkin;")
print("bloklash foydasi matritsa keshdan katta bo'lganda (n>=4096, 64+ MB) va GPU umumiy
xotirasida yaqqol ko'rinadi.")
print("GPU bo'lsa: blok = cuda.shared.array((TPB,TPB)), yuklash -> cuda.syncthreads() -> hisob;
TPB=16 da har element 16 marta qayta ishlatiladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Kesh bloklash ===`, `Blok o'lchami ta'siri`

TAJRIBA QILIB KO‘RING

- matmul_tiled'da slice o'rniga to'g'ridan-to'g'ri C[i, j] indekslashga qaytib, GFLOP/s 2 ga tushishini o'zingiz tekshiring.
- n=2048 (48 MB, keshdan katta) uchun faqat ikj va tiled bs=128 ni o'lchang — endi bloklash foyda beradimi?
- Blokni ikki darajali qiling (tashqi 256, ichki 32) va natijani yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Konvolyutsiyani to'rt usulda parallellashtirish

MAQSAD

3×3 konvolyutsiya (16→32 kanal, 96×96) Numba to'g'ridan-to'g'ri sikl (1 ip), prange (1 va 2 ip), NumPy im2col+GEMM va PyTorch conv2d (oneDNN, 1 va 2 ip, batch 8) bilan bajariladi; barcha natijalar etalon bilan solishtiriladi. Kutilgan natija: prange 2 ipda ~2×, im2col ~16×, torch 2 ip ~20× tezlanish; xato < 1e-5. Bu cuDNN'ning implicit GEMM g'oyasini CPU'da ko'rsatadi.

> **Kutilgan natija:** 81,4 MFLOP; jadval: Numba 1 ip ~2 GFLOP/s, prange 2 ip ~4, im2col ~33, torch 2 ip ~46 GFLOP/s; maks xato ~5e-6.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Konvolyutsiya (3x3) ni to'rt usulda: Numba to'g'ridan-to'g'ri (1 ip), Numba prange
(2 ip),
# NumPy im2col + GEMM, PyTorch conv2d (oneDNN, 2 ip). Hammasi bir xil natija berishi
tekshiriladi.
import time
import numpy as np
import numba
from numba import njit, prange
import torch
import torch.nn.functional as F

torch.set_num_threads(2)
torch.backends.nnpack.set_flags(False)
numba.set_num_threads(2)
np.random.seed(0)

CIN, COUT, H, W, K = 16, 32, 96, 96, 3
HO, WO = H - K + 1, W - K + 1
FLOP = 2 * K * K * CIN * COUT * HO * WO


@njit
def conv_seq(x, w, y):
    cin, h, wd = x.shape
    cout, ho, wo = y.shape
    for co in range(cout):
        for i in range(ho):
            for j in range(wo):
                s = 0.0
                for ci in range(cin):
                    for di in range(3):
                        for dj in range(3):
                            s += x[ci, i + di, j + dj] * w[co, ci, di, dj]
                y[co, i, j] = s


@njit(parallel=True)
def conv_par(x, w, y):
    cin, h, wd = x.shape
    cout, ho, wo = y.shape
    for co in prange(cout):                # chiqish kanallari iplar orasida taqsimlanadi
        for i in range(ho):
            for j in range(wo):
```

```
                s = 0.0
                for ci in range(cin):
                    for di in range(3):
                        for dj in range(3):
                            s += x[ci, i + di, j + dj] * w[co, ci, di, dj]
                y[co, i, j] = s


def conv_im2col(x, w):
    """Konvolyutsiyani matritsa ko'paytirishga keltirish (cuDNN'ning implicit GEMM g'oyasi)."""
    cols = np.empty((CIN * K * K, HO * WO), dtype=np.float32)
    r = 0
    for ci in range(CIN):
        for di in range(K):
            for dj in range(K):
                cols[r] = x[ci, di:di + HO, dj:dj + WO].ravel()
                r += 1
    return (w.reshape(COUT, -1) @ cols).reshape(COUT, HO, WO)


def olchov(f, takror=5):
    f()
    v = []
    for _ in range(takror):
        t0 = time.perf_counter(); f(); v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


x = np.random.rand(CIN, H, W).astype(np.float32)
w = (np.random.rand(COUT, CIN, K, K).astype(np.float32) - 0.5)
y1 = np.zeros((COUT, HO, WO), np.float32)
y2 = np.zeros((COUT, HO, WO), np.float32)
xt, wt = torch.from_numpy(x)[None], torch.from_numpy(w)

print("=== Konvolyutsiya ===")
print(f"Kirish {CIN}x{H}x{W}, yadro {COUT}x{CIN}x{K}x{K}, chiqish {COUT}x{HO}x{WO}: {FLOP /
1e6:.1f} MFLOP")
t0 = time.perf_counter(); conv_seq(x, w, y1); conv_par(x, w, y2); print(f"Numba kompilyatsiya:
{time.perf_counter() - t0:.2f} s")

etalon = F.conv2d(xt, wt)[0].numpy()
natija = []
natija.append(("Numba to'g'ridan-to'g'ri, 1 ip", olchov(lambda: conv_seq(x, w, y1)), y1))
numba.set_num_threads(1)
natija.append(("Numba prange, 1 ip", olchov(lambda: conv_par(x, w, y2)), y2))
numba.set_num_threads(2)
natija.append(("Numba prange, 2 ip", olchov(lambda: conv_par(x, w, y2)), y2))
natija.append(("NumPy im2col + GEMM", olchov(lambda: conv_im2col(x, w)), conv_im2col(x, w)))
with torch.no_grad():
    torch.set_num_threads(1)
    natija.append(("torch conv2d, 1 ip", olchov(lambda: F.conv2d(xt, wt)), etalon))
    torch.set_num_threads(2)
    natija.append(("torch conv2d, 2 ip", olchov(lambda: F.conv2d(xt, wt)), etalon))
    xb = xt.repeat(8, 1, 1, 1)
    t_b = olchov(lambda: F.conv2d(xb, wt))
    natija.append(("torch conv2d, 2 ip, batch 8", t_b / 8, etalon))

print(f"\n{'usul':30s} {'ms':>8s} {'GFLOP/s':>8s} {'tezlanish':>9s} {'maks xato':>10s}")
t_bazaviy = natija[0][1]
for nom, t, yy in natija:
    print(f"{nom:30s} {t * 1e3:8.2f} {FLOP / t / 1e9:8.2f} {t_bazaviy / t:9.1f} {np.abs(yy -
etalon).max():10.1e}")
print("\nXulosa: to'g'ridan-to'g'ri sikl -> prange (2x) -> GEMM'ga keltirish (BLAS) -> oneDNN
```

```
(vektorlash + bloklash).")
print("GPU bo'lsa: F.conv2d cuDNN'ga o'tadi (implicit GEMM/Winograd), FP16'da Tensor Core;
o'lchash uchun torch.cuda.synchronize() kerak.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Konvolyutsiya ===`, `torch conv2d`

TAJRIBA QILIB KO‘RING

- prange'ni chiqish kanallari (co) o'rniga qatorlar (i) bo'yicha qo'ying — yuk muvozanati o'zgaradimi?
- Yadroni 5×5 qilib FLOP va vaqtni qayta o'lchang; im2col matritsasi hajmi necha baravar oshadi?
- CIN=64, COUT=64, 224×224 (mustaqil ish topshirig'i) uchun faqat torch variantini o'lchang va GFLOP/s ni solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. PyTorch: GPU o'rniga ko'p yadro — 1 vs 2 ip, formatlar, yadro birlashtirish

MAQSAD

torch.cuda.is_available() False ekani ko'rsatiladi va GPU'dagi o'zgarishlar izohlanadi. CPU'da: (a) matmul n=128/512/1024 uchun torch.set_num_threads(1) vs (2) tezlanishi; (b) FP32 vs BF16 vs INT8 (torch._int_mm) — CPU'da maxsus buyruqlar (AVX512-BF16/VNNI) bo'lmasa past formatlar sekinroq, bu apparat–format mosligi (Co-design) darsidir; (c) GELU eager (7 alohida amal) vs birlashtirilgan F.gelu — xotira trafigi 7× kam. Kutilgan natija: 2 ip katta n da ~2×, BF16 CPU'da 20× sekin, F.gelu ~3× tez.

> **Kutilgan natija:** Iplar jadvali (1024 da 1,98× tezlanish), formatlar jadvali (FP32 ~50 GFLOP/s, BF16 ~2, INT8 ~4), GELU: eager ~130–220 ms, F.gelu ~45–80 ms.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. PyTorch: GPU o'rniga ko'p yadro — torch.set_num_threads(1) vs 2, formatlar va yadro
birlashtirish
# Sandbox'da CUDA mavjud emas (torch.cuda.is_available() == False). Real labdagi Tensor Core
o'lchovlari
# o'rniga CPU'da: (a) iplar soni ta'siri, (b) FP32 vs BF16 vs INT8 (CPU'da!), (c) GELU eager vs
birlashtirilgan.
import time
import torch
import torch.nn.functional as F

torch.manual_seed(0)
torch.backends.nnpack.set_flags(False)
print("=== torch iplar: 1 vs 2 ===")
print(f"torch {torch.__version__}, CUDA mavjud: {torch.cuda.is_available()}, CPU iplar
(standart): {torch.get_num_threads()}")
print("GPU bo'lsa: dev='cuda', tensorlar .to(dev), vaqt oldidan torch.cuda.synchronize();
FP16/BF16 Tensor Core'ga o'tadi.\n")


def olchov(f, takror=7):
    for _ in range(2):
        f()
    v = []
    for _ in range(takror):
        t0 = time.perf_counter(); f(); v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


# --- (a) Matritsa ko'paytirish: iplar soni va o'lcham ---
print(f"{'n':>5s} {'iplar':>5s} {'ms':>9s} {'GFLOP/s':>8s} {'tezlanish':>9s}")
for n in (128, 512, 1024):
    A = torch.randn(n, n); B = torch.randn(n, n)
    t1 = None
    for iplar in (1, 2):
        torch.set_num_threads(iplar)
        t = olchov(lambda: A @ B)
        if t1 is None:
            t1 = t
```

```
        print(f"{n:5d} {iplar:5d} {t * 1e3:9.3f} {2 * n ** 3 / t / 1e9:8.1f} {t1 / t:9.2f}")
print("Kichik n da 2 ip foyda bermaydi (ip sinxronizatsiyasi xarajati), katta n da ~2x ga
yaqinlashadi.")

# --- (b) Formatlar: FP32 vs BF16 vs INT8 — CPU'da tezlanish bormi? ---
torch.set_num_threads(2)
n = 512
A = torch.randn(n, n); B = torch.randn(n, n)
etalon = A @ B
print(f"\n--- Formatlar, n={n}, 2 ip (CPU) ---")
print(f"{'format':8s} {'ms':>9s} {'GFLOP/s':>8s} {'nisbiy xato':>12s}")
t32 = olchov(lambda: A @ B)
print(f"{'FP32':8s} {t32 * 1e3:9.3f} {2 * n ** 3 / t32 / 1e9:8.1f} {0.0:12.1e}")
Ab, Bb = A.bfloat16(), B.bfloat16()
tb = olchov(lambda: Ab @ Bb)
xb = ((Ab @ Bb).float() - etalon).norm() / etalon.norm()
print(f"{'BF16':8s} {tb * 1e3:9.3f} {2 * n ** 3 / tb / 1e9:8.1f} {xb.item():12.1e}")
# INT8: dinamik kvantlash (A3 mavzusi): masshtab * int8 matritsa
sa, sb = A.abs().max() / 127, B.abs().max() / 127
A8 = (A / sa).round().clamp(-128, 127).to(torch.int8)
B8 = (B / sb).round().clamp(-128, 127).to(torch.int8)
try:
    t8 = olchov(lambda: torch._int_mm(A8, B8))
    x8 = ((torch._int_mm(A8, B8).float() * sa * sb) - etalon).norm() / etalon.norm()
    print(f"{'INT8':8s} {t8 * 1e3:9.3f} {2 * n ** 3 / t8 / 1e9:8.1f} {x8.item():12.1e}
(torch._int_mm)")
except Exception as e:
    print(f"{'INT8':8s} CPU'da _int_mm qo'llab-quvvatlanmaydi: {type(e).__name__}")
print("CPU'da BF16/INT8 tezlanish bermasligi mumkin (AVX512-BF16/VNNI yo'q) — GPU Tensor
Core'da 2-8x. Bu apparat-format moslik (Co-design) misoli.")

# --- (c) Elementlararo amal: GELU eager (6-7 alohida yadro) vs birlashtirilgan F.gelu ---
N = 8 * 1024 * 1024                                  # 8M element = 32 MB FP32
x = torch.randn(N)
def gelu_eager(x):
    return 0.5 * x * (1 + torch.tanh(0.79788456 * (x + 0.044715 * x ** 3)))
print(f"\n--- GELU, {N // 2 ** 20}M element ---")
print(f"{'variant':22s} {'iplar':>5s} {'ms':>8s} {'samarali GB/s':>14s}")
for iplar in (1, 2):
    torch.set_num_threads(iplar)
    for nom, f, otish in (("eager (7 yadro)", lambda: gelu_eager(x), 7 * 2), ("F.gelu (1
yadro)", lambda: F.gelu(x, approximate="tanh"), 2)):
        t = olchov(f, takror=5)
        print(f"{nom:22s} {iplar:5d} {t * 1e3:8.2f} {otish * N * 4 / t / 1e9:14.1f}")
print("Eager rejimda har amal 32 MB o'qib-yozadi; birlashtirilgan yadro bitta o'tishda — xotira
trafigi 7x kam.")
print("GPU bo'lsa: torch.compile Triton bilan shunday birlashtirishni avtomatik qiladi
(sandbox'da C kompilyatori yo'q).")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== torch iplar: 1 vs 2 ===`, `GELU`

TAJRIBA QILIB KO‘RING

- n=4097 kabi 8 ga karrali bo'lmagan o'lchamni qo'shing — CPU'da ham unumdorlik tushadimi (GPU'dagi Tensor Core sharti bilan solishtiring)?
- GELU uchun N ni 1M ga tushiring: eager va fused farqi saqlanadimi (kesh ichida ishlaganda)?
- torch.float16 matmul'ni CPU'da sinab ko'ring va BF16 bilan vaqt/xato bo'yicha solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Ma'lumot parallelligi: multiprocessing bilan 2 jarayon

MAQSAD

600 ta sintetik kadr uchun Python+NumPy oldindan ishlov (Sobel, filtr, gistogramma) ketma-ket va multiprocessing.Pool (fork, 1 va 2 jarayon, chunksize 1/25/150) bilan bajariladi; tezlanish, samaradorlik va Pool yaratish xarajati o'lchanadi, natija ketma-ket bilan taqqoslanib tekshiriladi. Kutilgan natija: 2 jarayonda ~1,9–2,4× (chunksize kattaroq bo'lsa yaxshiroq), Pool ochish ~10–50 ms. Bu DDP'dagi «har qurilma o'z ulushini ishlaydi» g'oyasining CPU muqobili.

> **Kutilgan natija:** Ketma-ket ~0,6 s; jadval 6 qator (jarayonlar × chunksize), 2 jarayonda ~2× tezlanish; Pool xarajati xabari.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Ma'lumot parallelligi: multiprocessing.Pool bilan kadrlarni 2 jarayonga bo'lish
# Python GIL tufayli iplar CPU-og'ir Python kodini tezlashtirmaydi; jarayonlar (fork) esa
# har biri o'z interpretatoriga ega. Bu ko'p-GPU DDP'dagi "har qurilma o'z ulushini ishlaydi"
g'oyasining CPU muqobili.
import os
import time
import multiprocessing as mp
import numpy as np

KADRLAR = 600                  # 'video kadrlar' soni
H = W = 96


def kadr_ishlov(kadr):
    """Bitta kadr uchun oldindan ishlov: normalizatsiya, Sobel gradient, 3x3 o'rtacha filtr,
gistogramma.
    Kichik NumPy massivlar ustida ko'p amal — BLAS iplari ishlamaydi, bu Python+NumPy
yukidir."""
    x = kadr.astype(np.float32) / 255.0
    gx = np.zeros_like(x); gy = np.zeros_like(x)
    for _ in range(6):                                  # ishni og'irlashtirish (real quvurda
bir nechta bosqich)
        gx[:, 1:-1] = x[:, 2:] - x[:, :-2]
        gy[1:-1, :] = x[2:, :] - x[:-2, :]
        mag = np.sqrt(gx * gx + gy * gy)
        x = (x[:-2, :-2] + x[1:-1, :-2] + x[2:, :-2] + x[:-2, 1:-1] + x[1:-1, 1:-1] + x[2:,
1:-1]
             + x[:-2, 2:] + x[1:-1, 2:] + x[2:, 2:]) / 9.0
        x = np.pad(x, 1)
    hist, _ = np.histogram(mag, bins=16, range=(0, 1))
    return float(mag.mean()), hist.argmax()


def ketma_ket(kadrlar):
    return [kadr_ishlov(k) for k in kadrlar]


if __name__ == "__main__":
    rng = np.random.default_rng(0)
    kadrlar = [rng.integers(0, 256, (H, W), dtype=np.uint8) for _ in range(KADRLAR)]

    print("=== Ma'lumot parallelligi (multiprocessing) ===")
```

```
    print(f"CPU yadrolar (ko'rinadigan): {os.cpu_count()}, ruxsat etilgan:
{len(os.sched_getaffinity(0))}, kadrlar: {KADRLAR}")

    t0 = time.perf_counter(); etalon = ketma_ket(kadrlar); t_seq = time.perf_counter() - t0
    print(f"\nKetma-ket (1 jarayon): {t_seq * 1e3:7.0f} ms  ({KADRLAR / t_seq:.0f} kadr/s)")

    ctx = mp.get_context("fork")
    print(f"\n{'jarayonlar':>10s} {'chunksize':>9s} {'ms':>7s} {'kadr/s':>7s} {'tezlanish':>9s}
{'samaradorlik':>12s}")
    for n_proc in (1, 2):
        for chunksize in (1, 25, 150):
            with ctx.Pool(n_proc) as pool:
                pool.map(kadr_ishlov, kadrlar[:20])          # isinish
                t0 = time.perf_counter()
                natija = pool.map(kadr_ishlov, kadrlar, chunksize=chunksize)
                t = time.perf_counter() - t0
            assert natija == etalon, "natija mos emas!"
            print(f"{n_proc:10d} {chunksize:9d} {t * 1e3:7.0f} {KADRLAR / t:7.0f} {t_seq /
t:9.2f} {t_seq / t / n_proc * 100:11.0f} %")

    # Jarayon yaratish xarajati (DDP'da bu 'process group init' ga o'xshaydi)
    t0 = time.perf_counter()
    with ctx.Pool(2) as pool:
        pool.map(kadr_ishlov, kadrlar[:2])
    print(f"\nPool(2) yaratish + yopish xarajati: {(time.perf_counter() - t0) * 1e3:.0f} ms —
qisqa vazifalar uchun parallelizm o'zini oqlamaydi.")
    print("chunksize=1 da har kadr alohida pickle/IPC orqali yuboriladi; katta bo'lak IPC
xarajatini kamaytiradi.")
    print("\nGPU bo'lsa: ma'lumot parallelligi = DistributedDataParallel (har GPU o'z paketini
ishlaydi, gradientlar all-reduce);")
    print("bu yerda jarayonlar natijani faqat oxirida jamlaydi (reduce), aloqa xarajati ancha
kichik.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Ma'lumot parallelligi (multiprocessing) ===`, `Pool(2) yaratish`

TAJRIBA QILIB KO‘RING

- KADRLAR=60 qiling — qisqa vazifada tezlanish saqlanadimi? Pool xarajati ulushini hisoblang.
- pool.map o'rniga pool.imap_unordered ishlatib, birinchi natija kelgunicha vaqtni (kechikish) o'lchang.
- kadr_ishlov ichidagi takror sonini 6 dan 1 ga tushiring: IPC xarajati hisobga nisbatan qanday o'sadi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Amdahl va Gustafson qonunlari: nazariya va 2 yadroda o'lchov

MAQSAD

S(n)=1/((1−p)+p/n) jadvali p=0,5…0,99 uchun chiqariladi; so'ng «dastur» = Python ketma-ket qism + torch matmul 1 va 2 ipda o'lchanib, S(2) dan parallel ulush p baholanadi va shu p bilan 8 hamda 1024 yadrodagi (GPU) tezlanish bashorat qilinadi. Grafikda Amdahl egri chiziqlari, o'lchangan nuqta va Gustafson chiziqlari ko'rsatiladi. Kutilgan natija: faqat matmul p≈1, ketma-ket qism qo'shilganda p≈0,97 → 1024 yadroda tezlanish atigi ~30×.

> **Kutilgan natija:** Amdahl jadvali (p=0,95: n=inf da 20), o'lchov jadvali S(2) ~1,9–2,0, p baholari, 2 panelli grafik.

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Amdahl va Gustafson qonunlari: nazariy egri chiziqlar + 2 yadroda o'lchangan
parallel ulushni baholash
# S(n) = 1 / ((1-p) + p/n). O'lchangan S(2) dan p = (1 - 1/S) / (1 - 1/n) topiladi va
# shu p bilan 8, 64, 1000 yadrodagi (GPU!) tezlanish chegarasi bashorat qilinadi.
import time
import numpy as np
import torch
import matplotlib.pyplot as plt

torch.manual_seed(0)


def amdahl(p, n):
    return 1.0 / ((1 - p) + p / n)


def gustafson(p, n):
    return (1 - p) + p * n


print("=== Amdahl qonuni ===")
print(f"{'p':>5s} " + " ".join(f"{'n=' + str(n):>8s}" for n in (2, 4, 8, 64, 1024, 'inf')))
for p in (0.5, 0.8, 0.9, 0.95, 0.99):
    satr = [amdahl(p, n) for n in (2, 4, 8, 64, 1024)] + [1 / (1 - p)]
    print(f"{p:5.2f} " + " ".join(f"{s:8.2f}" for s in satr))

# --- O'lchov: "dastur" = ketma-ket qism (Python'da oldindan ishlov) + parallel qism (torch
matmul) ---
def olchov(f, takror=5):
    f(); f()
    v = []
    for _ in range(takror):
        t0 = time.perf_counter(); f(); v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]

n = 768
A = torch.randn(n, n); B = torch.randn(n, n)

def ketma_ket_qism():
    # parallellashtirilmagan ish: Python siklida kichik amallar (masalan, NMS, log yozish)
```

```
    s = 0.0
    for i in range(60000):
        s += (i % 7) * 0.5
    return s

def dastur(ketma_ket=True):
    if ketma_ket:
        ketma_ket_qism()
    for _ in range(4):
        C = A @ B
    return C

print("\n--- O'lchov: 1 ip vs 2 ip (torch.set_num_threads) ---")
print(f"{'variant':34s} {'1 ip, ms':>9s} {'2 ip, ms':>9s} {'S(2)':>6s} {'p (baho)':>9s}
{'S(8)':>6s} {'S(1024)':>8s}")
natijalar = []
for nom, kk in (("faqat matmul (p ~ 1)", False), ("ketma-ket qism + matmul", True)):
    torch.set_num_threads(1); t1 = olchov(lambda: dastur(kk))
    torch.set_num_threads(2); t2 = olchov(lambda: dastur(kk))
    S2 = t1 / t2
    p = min(1.0, max(0.0, (1 - 1 / S2) / (1 - 1 / 2)))
    natijalar.append((nom, p, S2))
    print(f"{nom:34s} {t1 * 1e3:9.1f} {t2 * 1e3:9.1f} {S2:6.2f} {p:9.3f} {amdahl(p, 8):6.2f}
{amdahl(p, 1024):8.2f}")
torch.set_num_threads(1); t_seq_qism = olchov(ketma_ket_qism)
print(f"Ketma-ket qismning o'zi: {t_seq_qism * 1e3:.1f} ms (o'lchangan) — Amdahl bahosi bilan
solishtiring.")

# --- Grafik ---
nn_ = np.array([1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024])
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
for p in (0.5, 0.9, 0.95, 0.99):
    ax1.plot(nn_, amdahl(p, nn_), label=f"Amdahl p={p}")
for nom, p, S2 in natijalar:
    ax1.plot(nn_, amdahl(p, nn_), "--", label=f"o'lchangan p={p:.2f} ({nom[:14]})")
    ax1.plot([2], [S2], "ko")
ax1.set_xscale("log", base=2); ax1.set_yscale("log", base=2)
ax1.set_xlabel("Protsessorlar soni n"); ax1.set_ylabel("Tezlanish S(n)")
ax1.set_title("Amdahl qonuni: tezlanish chegarasi"); ax1.grid(alpha=0.3, which="both");
ax1.legend(fontsize=7)
for p in (0.5, 0.9, 0.99):
    ax2.plot(nn_, gustafson(p, nn_), label=f"Gustafson p={p}")
    ax2.plot(nn_, amdahl(p, nn_), ":", color="gray")
ax2.set_xscale("log", base=2); ax2.set_yscale("log", base=2)
ax2.set_xlabel("Protsessorlar soni n"); ax2.set_ylabel("Masshtablangan tezlanish")
ax2.set_title("Gustafson (masala hajmi o'sadi) vs Amdahl (nuqtali)"); ax2.grid(alpha=0.3,
which="both"); ax2.legend(fontsize=7)
fig.tight_layout()
print("\nXulosa: GPU (minglab yadro) faqat p -> 1 bo'lganda samarali; shuning uchun paket
(batch) kattalashtiriladi (Gustafson)")
print("va ketma-ket qismlar (NMS, ma'lumot yuklash, CPU-GPU ko'chirish) alohida
optimallashtiriladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Amdahl qonuni ===`, `p (baho)`

TAJRIBA QILIB KO‘RING

- ketma_ket_qism sikl sonini 60 000 dan 600 000 ga oshiring — p va S(1024) qanday tushadi?
- Matritsa o'lchamini 768 dan 1536 ga oshiring (Gustafson: masala hajmi o'sadi) — p qanday o'zgaradi?
- S(2) o'lchovini 5 marta takrorlab p bahosining tarqalishini (min/max) yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Sof Python vs NumPy: o'lcham egri chizig'i** 1-qadam skriptida n = 16, 32, 64, 128, 192 uchun sof Python va NumPy matmul vaqtlarini o'lchab, log-log grafikda chizing; sof Python uchun vaqt ~n^3 ekanini (log-log og'ish koeffitsiyenti) hisoblab ko'rsating.	oson	5
2	**prange samaradorligi** 2-qadamda n = 128, 256, 512, 1024 uchun prange 1 va 2 ipda tezlanish va parallel samaradorlikni (tezlanish/2) jadvalga yozing; kichik n da samaradorlik past bo'lishining sababini ip yaratish xarajati orqali tushuntiring.	oson	5
3	**Blok o'lchami tadqiqoti** 3-qadamda bs = 8, 16, 32, 64, 128, 256, 512 uchun n=1024 va n=2048 da GFLOP/s o'lchang, grafik chizing va har n uchun eng yaxshi blokni hamda uning uch bloki egallagan KB hajmini L1/L2 kesh o'lchamlari bilan solishtiring.	oson	5
4	**Konvolyutsiya: parallellashtirish o'qi** 4-qadam Numba konvolyutsiyasini uch xil prange o'qi (chiqish kanali, qator, kanal+qator collapse) bilan amalga oshiring; 1 va 2 ipda vaqt, GFLOP/s va tezlanishni jadvalga kiriting, yuk muvozanati va kesh lokalligi nuqtai nazaridan eng yaxshi variantni asoslang.	o'rta	10
5	**im2col xotira va vaqt savdosi** 4-qadam im2col variantida CIN=16/32/64 va 96×96/160×160 kirishlar uchun cols matritsasining hajmini (MB) va vaqtni o'lchang; to'g'ridan-to'g'ri Numba va torch conv2d bilan solishtirib, qaysi hajmdan boshlab im2col xotira cheklovi tufayli foydasiz bo'lishini hisoblab ko'rsating.	o'rta	10
6	**Format va iplar matritsasi** 5-qadamda FP32, BF16, FP16 va INT8 formatlarida n = 256, 512, 1024 va 1, 2 ip uchun GFLOP/s va nisbiy xato jadvalini tuzing; CPU'da past formatlar nima uchun tezlashmasligini va GPU Tensor Core'da (A2 nazariy qismidagi jadval) qanday nisbat kutilishini solishtirib yozing.	o'rta	10
7	**Yadro birlashtirish qo'lda** 5-qadam GELU'sini Numba @njit(parallel=True) bilan bitta o'tishda (fused) yozing; eager torch, F.gelu va Numba fused variantlarini 1 va 2 ipda 8M element uchun o'lchab samarali GB/s ni hisoblang va xotira trafigi nazariy hisobi bilan solishtiring.	o'rta	10
8	**Multiprocessing vs threading vs Numba** 6-qadam kadr_ishlov vazifasini uch usulda parallellashtiring: multiprocessing.Pool(2), concurrent.futures.ThreadPoolExecutor(2) va Numba prange; har biri uchun tezlanish, natija to'g'riligi va xotira sarfini (psutil) o'lchang, GIL ta'sirini threading natijasida ko'rsating va qaysi usul qachon tanlanishini jadval bilan asoslang.	qiyin	15
9	**Amdahl: o'lchangan p va bashorat tekshiruvi** 7-qadam metodikasini 2–4-qadamlardagi barcha parallel variantlarga (prange matmul, prange konvolyutsiya, torch matmul, multiprocessing) qo'llab har biri uchun	qiyin	15

	p ni baholang; ketma-ket qism ulushini qo'lda o'lchab (profil) Amdahl bahosi bilan solishtiring, farq sabablarini (xotira o'tkazuvchanligi, sinxronizatsiya) tushuntiring va jamlovchi grafik chizing.		
10	**CPU Roofline va GPU bashorati** Barcha qadamlardagi o'lchovlarni (sof Python, NumPy, prange, tiled, im2col, torch conv2d, GELU eager/fused) arifmetik intensivlik–unumdorlik tekisligiga joylashtiring: sandbox CPU uchun cho'qqi GFLOP/s ni torch FP32 matmul'ning eng yaxshi natijasidan, xotira o'tkazuvchanligini reduktsiya (2-qadam) natijasidan oling; har nuqta qaysi chegara bilan cheklanganini tushuntiring va A2 matnidagi RTX 3060 ko'rsatkichlari bilan GPU'da kutilgan tezlanishni bashorat qiling.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. EKG monitoring: ko'p kanalli signalni FIR filtrlash va R-tishlarni sanash

oson

Soha: Tibbiyot

Kardiologiya bo'limining markaziy monitoring stansiyasiga 2 000 ta bemor kanalidan 500 Hz chastotali EKG keladi (har kanalda 20 s = 10 000 namuna). Har kanalga 64 koeffitsiyentli FIR polosali filtr qo'llanib, so'ng bo'sag'a bilan R-tishlar sanaladi va yurak urishi hisoblanadi. Tizim CPU serverida ishlaydi va 2 000 kanal 1 s ichida qayta ishlanishi kerak. Siz sof Python'dan boshlab vektorlash, Numba prange, torch conv1d va multiprocessing darajalarida tezlanishni o'lchaysiz. Signal sintetik: sinusoidalar yig'indisi + Gauss shovqini + davriy R-tish impulslari (numpy generator, seed 0).

Yadro	1D FIR filtr (64 tap) + bo'sag'a bilan R-tish sanash (reduktsiya)
O'lchamlar	kanallar 200 / 2 000, namuna 10 000 (float32), taplar 16 / 64
FLOP	2 x 64 x 10 000 x 2 000 = 2,56 GFLOP
Iplar/jarayonlar	Numba 1/2 ip, torch 1/2 ip, Pool 1/2 jarayon
Bo'lak (chunk)	kanallar bo'yicha 1 / 50 / 500
Vaqt chegarasi	2 000 kanal <= 1 s
Ma'lumot	sintetik EKG, seed 0

QADAMLAR

1. (s1) Sof Python FIR (ikki sikl: namuna x tap) 1 kanal x 10 000 namuna uchun vaqt va MFLOP/s; NumPy np.convolve(mode='valid') va 'sirpanuvchi oyna' (np.lib.stride_tricks.sliding_window_view @ h) bilan 200 va 2 000 kanal; tezlanish va maks xato jadvali.
2. (s2) @njit FIR (kanal, namuna, tap sikllari) va @njit(parallel=True) prange kanallar bo'yicha; JIT vaqti; 1 va 2 ipda ms, GFLOP/s, tezlanish; R-tish sanash reduktsiyasini (bo'sag'a oshib o'tish sonini) prange bilan yozib 1 vs 2 ip GB/s samarali o'qishni hisoblang.
3. (s3) Lokallik: tap siklini ichki/tashqi qilib (namuna-tap vs tap-namuna) va kanallarni 50 talik bo'laklarda ishlab (bo'lak L2 keshga sig'adi) GFLOP/s farqini o'lchang; taplar 16 vs 64 da arifmetik intensivlik qanday o'zgaradi.
4. (s4) FIR'ni 4 usulda: Numba 1 ip, Numba prange 2 ip, NumPy sliding_window + GEMM (im2col analogi: 10 000 x 64 matritsa x 64 vektor), torch F.conv1d (2 000 kanal batch, 1 va 2 ip); etalon bilan maks xato < 1e-4; jadval.
5. (s5) torch conv1d 1 vs 2 ip (200/2 000 kanal), float32 vs float64 vs bfloat16 vaqt va nisbiy xato; R-tish sanash eager (3 amal) vs bitta torch.count_nonzero(x[1:] > t) & (x[:-1] <= t) birlashtirilgan ifoda - samarali GB/s.
6. (s6) multiprocessing.Pool(1/2) bemorlar bo'yicha (kanal = vazifa), chunksize 1/50/500; tezlanish, samaradorlik, Pool yaratish xarajati; natija ketma-ket bilan bir xilligini assert qiling.
7. (s7) Amdahl: 'dastur' = Python'da hisobot shakllantirish (ketma-ket, 2 000 kanal uchun sikl) + prange FIR; 1 vs 2 ipda S(2), p bahosi, S(8)/S(1024) bashorati; Gustafson: kanallar 2 000 -> 20

000 (generatsiya bo'laklab) p qanday o'zgaradi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

2-variant. Sun'iy yo'ldosh dala tasvirlarida NDVI va tekstura indekslarini hisoblash o'rta

Soha: Agro

Agro-holding 2048x2048 piksellik 4 kanalli (qizil, yashil, ko'k, NIR) dala tasvirlarini har hafta qayta ishlaydi: NDVI elementlararo hisoblanadi, so'ng 5x5 o'rtacha va dispersiya (tekstura) filtrlari qo'llanib, parsellarga bo'lingan hisobot tuziladi. Bir mavsumda 3 000 tasvir bo'lgani uchun bitta tasvir 0,5 s dan tez ishlanishi kerak. Siz NDVI va 5x5 filtrni sof Python'dan torch conv2d gacha bo'lgan darajalarda parallellashtirib, qaysi daraja xotira, qaysi biri hisob bilan cheklanganini aniqlaysiz. Tasvirlar sintetik: silliq fon + parsel bloklari + shovqin (numpy, seed 1).

Yadro	NDVI = (NIR - R)/(NIR + R) (elementlararo) + 5x5 o'rtacha va dispersiya filtri
O'lchamlar	512 / 1 024 / 2 048 piksel, 4 kanal float32 (2048: 64 MB)
FLOP	5x5 filtr: 2 x 25 x 4,2 M = 210 MFLOP (2048); NDVI ~5 FLOP/piksel
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	satr bo'laklari 32 / 128 / 512 satr
Vaqt chegarasi	1 tasvir <= 0,5 s
Ma'lumot	sintetik 4 kanal tasvir, seed 1

QADAMLAR

1. (s1) NDVI: sof Python ikki sikl (512x512) vs NumPy (512/1024/2048) - ms, M piksel/s, tezlanish; NumPy'da oraliq massivlar (NIR-R, NIR+R, bo'lish) tufayli xotira trafigi MB va samarali GB/s.
2. (s2) @njit 5x5 o'rtacha filtr (to'g'ridan-to'g'ri 4 sikl) va prange satrlar bo'yicha 1/2 ip; JIT vaqti, GFLOP/s, tezlanish; NDVI o'rtachasi va dispersiyasini prange reduktsiya bilan hisoblab 1 vs 2 ip GB/s.
3. (s3) Lokallik: filtrni satr bo'laklarida (32/128/512 satr) ishlab har bo'lak keshda qolishini o'lchang; ustun-birinchi (j tashqi) sikl tartibini qo'shib necha baravar sekinlashishini ko'rsating; 'blok o'lchami ta'siri' jadvali va grafigi.
4. (s4) 5x5 filtrni 4 usulda: Numba 1 ip, prange 2 ip, NumPy 'ajraladigan' filtr (5x1 so'ng 1x5: 2x5 ko'paytma o'rniga 25) yoki im2col + GEMM, torch F.conv2d (1 va 2 ip, 4 kanal batch); dispersiyani E[x^2]-E[x]^2 bilan ikkita conv'dan oling; maks xato < 1e-4.
5. (s5) torch conv2d 1 vs 2 ip (512/1024/2048), FP32 vs BF16 vaqt va xato; NDVI eager (3 amal) vs torch.where bilan bitta ifoda / Numba fused - samarali GB/s 1 va 2 ip.
6. (s6) Pool(1/2) tasvirlar bo'yicha (20 ta 1024x1024 tasvir, har vazifa NDVI + filtr + parsel o'rtachasi), chunksize 1/5/10; tezlanish, Pool xarajati, natija tekshiruvi.
7. (s7) Amdahl: ketma-ket qism = parsel hisobotini Python lug'atida yig'ish, parallel qism = prange filtr; S(2), p, S(8)/S(64)/S(1024); Gustafson: 1024 -> 2048 -> 4096 (4096 uchun faqat 1 kanal, 64 MB) o'lchamlarda p o'sishi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

3-variant. Shahar yo'l tarmog'ida barcha juftliklar orasidagi eng qisqa yo'llar (Floyd-Warshall) qiyin

Soha: Transport

Jamoat transporti boshqarmasi 512 ta bekat/chorrahadan iborat tarmoq uchun barcha juftliklar orasidagi eng qisqa vaqt matritsasini har 10 daqiqada (trafik yangilanganda) qayta hisoblaydi. Floyd-Warshall algoritmi O(n^3) - 512 uchun 134 M yangilanish; sof Python'da bu daqiqalar, Numba'da soniyadan kam. Algoritmdagi k sikli ketma-ket (bog'liqlik bor), i-j sikllari parallellashadi; bloklangan (tiled) Floyd-Warshall esa GPU'dagi umumiy xotira yadrosining aynan o'zi. Siz sof Python'dan prange va bloklashgacha o'tib, k siklining ketma-ketligi Amdahl ulushiga qanday ta'sir qilishini o'lchaysiz. Graf sintetik: tasodifiy geometrik graf, og'irlik = masofa / tezlik (seed 2).

Yadro	Floyd-Warshall: D[i,j] = min(D[i,j], D[i,k] + D[k,j]); keyin 'min-plus' matritsa ko'paytirish varianti
O'lchamlar	n = 64 (Python), 256 / 512 / 1 024 (Numba; 1024 float32 = 4 MB)
Amallar	n^3 yangilanish: 512 -> 134 M, 1024 -> 1,07 G
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2 (manbalar bo'yicha Dijkstra muqobili)
Blok	tiled FW blok 16 / 32 / 64 / 128
Vaqt chegarasi	n=512 <= 2 s
Ma'lumot	tasodifiy geometrik graf, seed 2

QADAMLAR

1. (s1) Sof Python FW n=64 (3 sikl) vs NumPy vektorlangan ichki sikl (har k uchun np.minimum(D, D[:,k,None] + D[None,k,:])) n=64/256/512; ms, M yangilanish/s, tezlanish; NumPy variantida har k da yoziladigan oraliq massiv (n^2 x 4 bayt) trafigi GB/s.
2. (s2) @njit FW (k-i-j) va prange i bo'yicha (k tashqi, ketma-ket) 1/2 ip n=256/512/1024; JIT vaqti, M yangilanish/s, tezlanish; 'o'rtacha yo'l vaqti' va 'erishib bo'lmaydigan juftliklar soni' reduktsiyasini prange bilan hisoblang.
3. (s3) Bloklangan FW (3 fazali: diagonal blok, qator/ustun bloklari, qolganlari) blok 16/32/64/128 bilan 1 ipda; naive k-i-j bilan solishtiring; n=1024 da blok ta'siri jadvali; i-k-j va k-i-j sikl tartibi farqini (D[k,j] qator bo'ylab) o'lchang.
4. (s4) Bir k-qadamni 4 usulda: Numba 1 ip, prange 2 ip, NumPy broadcasting, torch torch.minimum(D, D[:,k:k+1] + D[k:k+1,:]) 1/2 ip; qo'shimcha: min-plus 'matritsa ko'paytirish' (D o D) log2(n) marta - torch'da (D[:,:,None] + D[None,:,:]).min(1) n=256 uchun xotira n^3 x 4 bayt = 64 MB; hammasi bir xil D beradimi (maks xato 0).
5. (s5) torch k-sikli 1 vs 2 ip n=512/1024, float32 vs float16 (masofalar 0-3 600 s oralig'ida xato), eager (qo'shish + minimum alohida) vs torch.minimum(D, torch.add(a, b, out=buf)) buferli variant - samarali GB/s.
6. (s6) Pool(1/2): manbalar bo'yicha bo'lish - har jarayon o'z i-satrlari to'plami uchun Dijkstra (heapq, Python) ni bajaradi (n=512, 64 manba); tezlanish, chunksize 1/8/32, natija FW bilan bir xilligi; nima uchun FW'ni jarayonlarga bo'lish qiyin (har k da sinxronizatsiya) - matn.

7. (s7) Amdahl: FW prange 1 vs 2 ip S(2) dan p; k-sikl boshidagi sinxronizatsiya (n ta bar'yer) xarajatini n=256 vs 1024 da solishtiring; S(8)/S(1024) bashorati; Gustafson: n 2 baravar oshganda ish 8 baravar, bar'yer 2 baravar - p qanday o'sadi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; Bloklangan FW uchun 3 fazaning sxemasi (matn/rasm) va blok o'lchami - kesh hajmi jadvali

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

4-variant. 5G massiv antenna: raqamli nur shakllantirish (beamforming) kompleks matritsa ko'paytirish o'rta

Soha: Telekom

Baza stansiyasining 128 antennali massividan har 1 ms da 20 000 ta kompleks namuna-vektor keladi; har vektorga 128x128 kompleks og'irlik matritsasi (128 nur) qo'llaniladi, so'ng har nur quvvati (|y|^2 yig'indisi) hisoblanadi. Real tizimda bu FPGA/GPU'da bajariladi, lekin siz CPU'da kompleks arifmetikaning parallellashtirish darajalarini o'rganasiz: kompleks ko'paytma 4 haqiqiy ko'paytma + 2 qo'shish, shuning uchun FLOP hisobi 8 x M x N x K. Ma'lumot sintetik: tasodifiy kanal matritsasi + 3 ta 'foydalanuvchi' yo'nalishi (steering vektorlar), seed 3.

Yadro	kompleks GEMM Y = X W (X: 20 000 x 128, W: 128 x 128, complex64) + nur quvvati reduktsiyasi
O'lchamlar	namuna 2 000 / 20 000, antenna 64 / 128, nur 128
FLOP	8 x 20 000 x 128 x 128 = 2,6 GFLOP
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	namuna bloklari 64 / 256 / 1 024
Vaqt chegarasi	20 000 namuna <= 100 ms (10 x real vaqt)
Ma'lumot	sintetik kanal + 3 steering vektor, seed 3

QADAMLAR

1. (s1) Sof Python kompleks matmul (Python complex, 3 sikl) 200 x 64 x 64 uchun vs NumPy complex64 @ (2 000/20 000 namuna, 64/128 antenna); ms, GFLOP/s (8MNK), tezlanish, maks xato; complex64 vs complex128 BLAS tezligi.
2. (s2) @njit kompleks matmul (haqiqiy/mavhum qismlarni alohida float32 massivlarda saqlab 4 ko'paytma) va prange namunalar bo'yicha 1/2 ip; JIT vaqti, GFLOP/s, tezlanish; nur quvvati sum|y|^2 prange reduktsiyasi 1 vs 2 ip GB/s.
3. (s3) Namunalarni 64/256/1024 lik bloklarda ishlab (W 128x128x8 bayt = 128 KB keshda qoladi) GFLOP/s; W'ni transpozitsiya qilib (nur bo'ylab ketma-ket) qaysi tartib vektorlashishini o'lchang; blok ta'siri jadvali.
4. (s4) Beamforming'ni 4 usulda: Numba 1 ip, prange 2 ip, NumPy complex64 GEMM, torch complex64 matmul 1/2 ip (yoki haqiqiy 2x2 blok ko'rinishida [Xr -Xi; Xi Xr] float32 GEMM - 4 real GEMM); etalon bilan maks xato; jadval.
5. (s5) torch matmul 1 vs 2 ip (2 000/20 000), complex64 vs complex128 vs 'real 2x2 blok float32' vs bfloat16 blok; nur quvvati eager (abs, kvadrat, yig'indi) vs torch.sum(y.real*y.real + y.imag*y.imag) vs Numba fused - GB/s.
6. (s6) Pool(1/2) 1 ms 'slotlar' bo'yicha (40 slot x 20 000 namuna generatsiya qilinadi), chunksize 1/5/20; tezlanish, IPC xarajati (20 000 x 128 x 8 bayt = 20 MB har slot!) va nima uchun natijani emas, faqat quvvat vektorini qaytarish kerak.
7. (s7) Amdahl: ketma-ket qism = steering vektorlarini Python'da hisoblash + nur tanlash (argmax, log), parallel = prange GEMM; S(2), p, S(8)/S(1024); Gustafson: namuna 20 000 -> 200 000

(bo'laklab) p o'zgarishi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

5-variant. Transformator radiatorida issiqlik tarqalishini 2D stensil (Jacobi) bilan modellashtirish o'rta

Soha: Energetika

Elektr tarmog'i kompaniyasi kuch transformatori radiatorining termal holatini real vaqtda baholash uchun 2D issiqlik o'tkazuvchanlik tenglamasini 1024x1024 to'rda Jacobi iteratsiyasi bilan yechadi (har iteratsiya 5 nuqtali stensil). Har 1 s da yangi chegara sharoiti (yuk toki) keladi va 200 iteratsiya 0,5 s ichida bajarilishi kerak. Stensil - xotira bilan cheklangan klassik yadro (AI ~0,5 FLOP/bayt), shuning uchun 2 ip tezlanishi hisob emas, xotira o'tkazuvchanligi bilan belgilanadi; vaqt bo'yicha bloklash (temporal tiling) va float32 bu chegarani yengillashtiradi. Boshlang'ich maydon va issiqlik manbalari sintetik (seed 4).

Yadro	5 nuqtali Jacobi stensil: u_new = 0,25 (u[i-1,j] + u[i+1,j] + u[i,j-1] + u[i,j+1]) + f; qoldiq normasi reduktsiyasi
O'lchamlar	to'r 256 / 512 / 1 024 / 2 048 (2048 float64 = 32 MB x 2)
FLOP	6 FLOP/nuqta; 1024^2 x 200 iter = 1,26 GFLOP
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2 (domen dekompozitsiyasi)
Blok	satr bo'laklari 16 / 64 / 256; vaqt bloki 1 / 2 / 4 iteratsiya
Vaqt chegarasi	200 iter @1024 <= 0,5 s
Ma'lumot	sintetik boshlang'ich maydon + 3 manba, seed 4

QADAMLAR

1. (s1) Sof Python Jacobi 128x128 x 10 iter vs NumPy slicing (u[1:-1,2:] + ...) 256/512/1024 x 200 iter; ms, M nuqta/s, GFLOP/s, tezlanish; NumPy variantida har iteratsiyada 4 ta oraliq massiv yozilishi - samarali GB/s.
2. (s2) @njit Jacobi (ikki sikl, ikki bufer almashinadi) va prange satrlar bo'yicha 1/2 ip; JIT vaqti; GFLOP/s va GB/s (o'qish 5 + yozish 1 qiymat/nuqta); qoldiq normasini (sum (u_new-u)^2) prange reduktsiya bilan har 10 iteratsiyada hisoblang - 1 vs 2 ip.
3. (s3) Lokallik: ustun-birinchi tartib (j tashqi) vs satr-birinchi; satr bo'laklari 16/64/256 (bo'lak x 3 satr keshda); vaqt bo'yicha bloklash: 2 va 4 iteratsiyani bitta o'tishda (ikkita qo'shimcha halo satr bilan) bajarib GB/s ga nisbatan GFLOP/s o'sishini ko'rsating; float32 vs float64.
4. (s4) Stensilni 4 usulda: Numba 1 ip, prange 2 ip, NumPy slicing, torch F.conv2d 3x3 yadro [[0,.25,0],[.25,0,.25],[0,.25,0]] 1/2 ip (padding bilan chegara sharti); 200 iteratsiyadan keyin maydonlar maks farqi < 1e-6 (float64) yoki < 1e-3 (float32).
5. (s5) torch conv2d stensil 1 vs 2 ip (512/1024/2048), float64 vs float32 vs bfloat16 (200 iter yig'ilgan xato!); qoldiq normasi eager (ayirma, kvadrat, yig'indi) vs torch.sum((a-b)**2) bitta ifoda vs Numba fused - GB/s.
6. (s6) Pool(1/2) domen dekompozitsiyasi: to'rni 2 ta gorizontal yarmiga bo'lib, har jarayon o'z yarmini 200 iteratsiya ishlaydi va har iteratsiyada halo satrni almashadi (mp.Pipe) - tezlanish va aloqa xarajati; muqobil: mustaqil 8 ta ssenariy (turli yuk toki) ni Pool'da bo'lish - qaysi biri samarali va nima uchun.

7. (s7) Amdahl: ketma-ket qism = chegara sharti va manbalarni Python'da yangilash (har iteratsiya), parallel = prange stensil; S(2), p, S(8)/S(1024); Gustafson: to'r 512 -> 2048 da p o'sishi; grafik; xulosada xotira chegarasi tufayli S(2) < 2 ekanini GB/s bilan asoslang.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

6-variant. Monte-Karlo bilan opsion narxlash va portfel xavfini (VaR) baholash

oson

Soha: Moliya

Investitsiya kompaniyasining xavf bo'limi har kuni 200 000 ta narx traektoriyasi (geometrik Broun harakati, 252 kunlik qadam) bilan Yevropa va Osiyo opsionlarini narxlaydi hamda portfel VaR'ini hisoblaydi. Hisob tungi oynada 5 daqiqa ichida tugashi kerak, lekin treyderlar 'nima bo'lsa' tahlilini kunduzi 2 s ichida kutishadi. Monte-Karlo - 'uyatli parallel' (embarrassingly parallel) masala: traektoriyalar mustaqil, faqat oxirida reduktsiya; siz bu p -> 1 holatda Amdahl va Gustafson qonunlari qanday ko'rinishini o'lchaysiz. Tasodifiy sonlar: numpy default_rng (seed 5), Numba'da har ip o'z seed'i bilan.

Yadro	GBM traektoriya: S_t+1 = S_t exp((r - s^2/2) dt + s sqrt(dt) Z) + to'lov funksiyasi + o'rtacha/kvantil reduktsiyasi
O'lchamlar	traektoriya 20 000 / 200 000, qadam 252, aktivlar 1 / 5 (korrelyatsiyalangan, Cholesky)
Amallar	200 000 x 252 = 50,4 M normal son + ~10 FLOP har biri
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Bo'lak	traektoriya bo'laklari 1 000 / 10 000 / 50 000
Vaqt chegarasi	200 000 traektoriya <= 2 s
Ma'lumot	S0=100, r=0,05, sigma=0,2, T=1; 5 aktiv korrelyatsiya matritsasi, seed 5

QADAMLAR

1. (s1) Sof Python (random.gauss, ikki sikl) 2 000 traektoriya vs NumPy vektorlangan (barcha traektoriyalar bir yo'la, 252 qadam sikli) 20 000/200 000; ms, M qadam/s, tezlanish; Yevropa call narxi Black-Scholes formulasi bilan solishtirilsin (xato < 1 %); NumPy'da 200 000 x 252 float64 = 403 MB oraliq massiv - xotira muammosini bo'laklab yechish.
2. (s2) @njit traektoriya sikli (traektoriya bo'yicha tashqi, qadam ichki, np.random.normal njit ichida) va prange traektoriyalar bo'yicha 1/2 ip; JIT vaqti; M qadam/s, tezlanish; to'lov o'rtachasi va dispersiyasi prange reduktsiya; 2 ipda natija 1 ipdagidan nima uchun farq qiladi (tasodifiy oqim) va buni qanday nazorat qilish.
3. (s3) Lokallik: 'qadam tashqi, traektoriya ichki' (NumPy uslubi, 200 000 x 8 bayt vektor har qadamda) vs 'traektoriya tashqi, qadam ichki' (Numba, 1 skalyar) tartiblarini o'lchang; bo'lak 1 000/10 000/50 000 traektoriya (bo'lak x 8 bayt L2 ga sig'adimi) ta'siri jadvali; Osiyo opsioni uchun yo'l o'rtachasini saqlash xotirani qanday oshiradi.
4. (s4) 5 aktivli korrelyatsiyalangan traektoriyalar: Z (5 x N) ni Cholesky L bilan ko'paytirish (kichik GEMM 5x5 x 5xN) 4 usulda - Numba 1 ip, prange 2 ip, NumPy L @ Z, torch 1/2 ip; har qadamda GEMM vs barcha qadamlarni (5 x 252N) bitta GEMM'da; etalon bilan maks xato.
5. (s5) torch: traektoriya generatsiyasi torch.randn + exp cumsum 1 vs 2 ip, float32 vs float64 (narx xatosi), eager (5 alohida amal) vs oldindan hisoblangan drift bilan bitta exp(cumsum(...)) - samarali GB/s; 200 000 x 252 float32 = 200 MB.

6. (s6) Pool(1/2) seedlar bo'yicha (har vazifa = 10 000 traektoriya o'z seed'i bilan, natija = yig'indi va kvadratlar yig'indisi), chunksize 1/2/5; tezlanish, Pool xarajati; natijani qaytarishda 10 000 x 8 bayt emas, 2 ta son yuborish IPC'ni qanday kamaytiradi.
7. (s7) Amdahl: faqat prange simulyatsiya (p ~ 1) va 'simulyatsiya + Python'da hisobot/kvantil (np.percentile emas, sorted) ' uchun S(2), p, S(8)/S(1024); Gustafson: 20 000 -> 200 000 -> 2 000 000 (bo'laklab) traektoriyada p -> 1; grafik; GPU'da (RTX 3060, 3 584 yadro) kutilgan tezlanishni bashorat qiling.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

7-variant. Milliy test natijalari: savollar statistikasi va korrelyatsiya matritsasi (X^T X) oson

Soha: Ta'lim

Test markazi 50 000 abituriyentning 200 ta savolga javob matritsasini (1 - to'g'ri, 0 - noto'g'ri) tahlil qiladi: har abituriyent bali (satr yig'indisi), har savol qiyinligi (ustun o'rtachasi), savollar orasidagi korrelyatsiya (200x200, X^T X orqali) va har savolning umumiy ball bilan nuqta-biserial korrelyatsiyasi. Natija imtihondan keyin 10 daqiqa ichida, 'nima bo'lsa' tahlili uchun 1 s ichida kerak. Siz reduktsiya va Gram matritsasi (X^T X) uchun sof Python'dan BLAS va prange gacha darajalarni o'lchaysiz; bu yerda X^T X - tor (200 ustun) matritsa ko'paytirish, AI balandligi N bilan o'sadi. Javoblar sintetik: 2 parametrli IRT modeli (qobiliyat, qiyinlik) bo'yicha Bernoulli (seed 6).

Yadro	satr/ustun yig'indilari (reduktsiya) + Gram matritsasi X^T X (50 000 x 200) + korrelyatsiya normalizatsiyasi
O'lchamlar	abituriyent 5 000 / 50 000, savol 50 / 200; X float32 (40 MB) va int8 (10 MB)
FLOP	2 x 50 000 x 200 x 200 = 4 GFLOP
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	abituriyent bloklari 500 / 5 000 / 50 000 (qisman Gram yig'indisi)
Vaqt chegarasi	to'liq tahlil <= 1 s
Ma'lumot	IRT 2PL sintetik javoblar, seed 6

QADAMLAR

1. (s1) Sof Python: ballar (satr yig'indisi) va Gram matritsasi 500 x 50 uchun uch sikl vs NumPy X.sum(1), X.T @ X 5 000/50 000 x 50/200 (float32 va float64); ms, GFLOP/s, tezlanish, maks xato; int8 matritsani float32 ga o'tkazish vaqtini alohida o'lchang.
2. (s2) @njit Gram (i-k-j tartib, simmetriyadan foydalanib yarmini hisoblash) va prange abituriyentlar bloklari bo'yicha (har ip o'z qisman G'ini yig'adi, so'ng qo'shiladi) 1/2 ip; JIT vaqti, GFLOP/s, tezlanish; ustun o'rtachasi (qiyinlik) prange reduktsiya 1 vs 2 ip GB/s.
3. (s3) Bloklash: X'ni 500/5 000/50 000 satrli bloklarda o'qib qisman Gram'larni yig'ish (blok x 200 x 4 bayt L2 ga sig'adimi); X satr-ustun (C) vs ustun-satr (Fortran) joylashuvida X.T @ X tezligi; blok ta'siri jadvali.
4. (s4) Nuqta-biserial korrelyatsiyani 4 usulda: Numba 1 ip (savollar bo'yicha sikl), prange 2 ip, NumPy ((X - p) * (score - m)[:,None]).mean(0) / std, torch 1/2 ip (bitta matvec X.T @ score_centered); maks xato < 1e-5; jadval.
5. (s5) torch X.T @ X 1 vs 2 ip (5 000/50 000), float32 vs bfloat16 vs int8 (torch._int_mm, 0/1 uchun aniq!) - vaqt va xato; korrelyatsiya normalizatsiyasi eager (4 amal) vs bitta ifoda - GB/s.
6. (s6) Pool(1/2) hududlar bo'yicha (14 ta hudud, har biri 3 500-4 000 abituriyent, har vazifa qisman yig'indilar + qisman Gram qaytaradi), chunksize 1/2/7; tezlanish, natija to'liq hisob bilan bir xilligi, IPC hajmi (200x200 float64 = 320 KB har vazifa).

7. (s7) Amdahl: ketma-ket qism = hisobot (200 savol uchun Python'da matn satrlari, saralash), parallel = prange Gram; S(2), p, S(8)/S(1024); Gustafson: abituriyent 5 000 -> 50 000 -> 500 000 (bloklab generatsiya) da p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

8-variant. Turbina vibratsiya monitoringi: ko'p kanalli FFT, Welch spektri va chastota cho'qqilari o'rta

Soha: Sanoat

Issiqlik elektr stansiyasining 2 048 ta vibratsiya sensori 25,6 kHz chastotada yoziladi; har 320 ms da (8 192 namuna) har kanal uchun Hann oynali FFT, Welch quvvat spektri (8 segment x 1 024) va 3 ta asosiy cho'qqi topiladi; natija 320 ms oynaning o'zida tayyor bo'lishi kerak (real vaqt). FFT O(N log N) - matmul'dan past AI, shuning uchun iplar tezlanishi va formatlar (float32/complex64) ta'siri boshqacha; oyna ko'paytirish va |X|^2 esa elementlararo amallar bo'lib birlashtirishga (fusion) nomzod. Signal sintetik: 1x, 2x aylanish garmonikalari + podshipnik chastotasi + shovqin (seed 7).

Yadro	Hann oyna x signal -> FFT -> |X|^2 -> Welch o'rtacha -> argmax cho'qqilar
O'lchamlar	kanal 256 / 2 048, namuna 8 192 (float32: 64 MB), segment 1 024
FLOP	~5 N log2 N = 0,53 MFLOP/kanal FFT; 2 048 kanal -> 1,1 GFLOP + Welch 8 segment
Iplar/jarayonlar	numpy.fft (1 ip), torch.fft 1/2 ip, Numba 1/2 ip (oyna, |X|^2, Welch), Pool 1/2
Bo'lak	kanal bloklari 16 / 128 / 1 024
Vaqt chegarasi	2 048 kanal <= 320 ms
Ma'lumot	sintetik vibratsiya, seed 7

QADAMLAR

1. (s1) Sof Python DFT (ikki sikl, N=512, 1 kanal) vs np.fft.rfft (N=512 va 8 192, 256/2 048 kanal, axis=1); ms, MFLOP/s (5 N log N formulasi), tezlanish, maks xato; float32 vs float64 kirishda rfft vaqti.
2. (s2) @njit Hann oyna + |X|^2 + Welch o'rtachasi (FFT'ni np.fft bilan qoldirib, elementlararo qismni Numba'da) va prange kanallar bo'yicha 1/2 ip; JIT vaqti; GB/s samarali; umumiy quvvat (sum |X|^2) prange reduktsiya 1 vs 2 ip; FFT ulushi vs elementlararo ulush profil jadvali.
3. (s3) Lokallik: kanallarni 16/128/1 024 lik bloklarda (blok x 8 192 x 4 bayt) ishlab rfft + Welch GFLOP/s; signal massivi (kanal, namuna) vs (namuna, kanal) joylashuvi (axis=0 FFT) farqi; blok ta'siri jadvali.
4. (s4) Welch spektrini 4 usulda: Numba 1 ip (FFT'siz: 1 024 nuqtali DFT'ni 1 024x1 024 kosinus/sinus matritsa bilan GEMM - 'DFT as GEMM'), prange 2 ip, NumPy rfft + reshape, torch.fft.rfft 1/2 ip (2 048 x 8 segment batch); maks xato < 1e-3 (float32); DFT-GEMM FLOP'i FFT'dan necha baravar ko'p, lekin GFLOP/s yuqori - tushuntiring.
5. (s5) torch.fft.rfft 1 vs 2 ip (256/2 048 kanal), float32 vs float64 (complex64/128); oyna x signal + |X|^2 eager (3 amal) vs torch.abs(rfft(x*w))**2 ... vs Numba fused oyna - samarali GB/s; cho'qqi topish torch.topk 3 vs Numba.
6. (s6) Pool(1/2) sensor guruhlari bo'yicha (16 guruh x 128 kanal, har vazifa 3 cho'qqi chastotasini qaytaradi), chunksize 1/2/8; tezlanish, IPC (128 x 8 192 x 4 = 4 MB har vazifa) ulushi; natija tekshiruvi.

7. (s7) Amdahl: ketma-ket qism = ogohlantirish qoidalari (Python'da 2 048 kanal sikli, cho'qqi chegara bilan solishtirish), parallel = torch rfft 2 ip; S(2), p, S(8)/S(1024); Gustafson: namuna 8 192 -> 65 536 (kanal 256) da p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

9-variant. Video-kuzatuv: kadrlar farqi orqali harakatni aniqlash va zona statistikasi oson

Soha: Xavfsizlik

Ob'ekt qo'riqlash xizmatining 16 ta kamerasi (240x320, 10 kadr/s) uchun oddiy, lekin keng tarqalgan 'harakat detektori' ishlaydi: ketma-ket kadrlar farqi, 5x5 quti filtri (box filter), bo'sag'a va 4x4 zonalar bo'yicha harakat piksellari ulushi. 16 kamera x 10 kadr/s = 160 kadr/s real vaqtda bajarilishi kerak, ya'ni bir kadrga 6 ms. Hamma amallar elementlararo yoki kichik filtr - ya'ni xotira bilan cheklangan; siz vektorlash, birlashtirish (fusion), prange va jarayonlar bu turdagi yadroga qancha foyda berishini o'lchaysiz. Kadrlar sintetik: statik fon + harakatlanuvchi to'rtburchak + shovqin (seed 8); 1 kamera x 300 kadr xotirada (23 MB), qolganlari jarayon ichida hosil qilinadi.

Yadro	|F_t - F_t-1| -> 5x5 quti filtr -> bo'sag'a -> 4x4 zona yig'indisi (reduktsiya)
O'lchamlar	kadr 240x320 uint8 (float32 ishlovda), kadrlar 300 / 1 kamera; 16 kamera Pool'da
Amallar	~60 FLOP/piksel (filtr 25 + farq + bo'sag'a) x 76 800 = 4,6 MFLOP/kadr
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2 (kameralar bo'yicha)
Blok	kadr paketlari 1 / 10 / 50 kadr
Vaqt chegarasi	<= 6 ms/kadr (160 kadr/s)
Ma'lumot	sintetik kadrlar, seed 8

QADAMLAR

1. (s1) Sof Python (ikki sikl, farq + bo'sag'a + zona yig'indisi, 5x5 filtrsiz) 10 kadr vs NumPy (farq, scipy.ndimage.uniform_filter 5x5 yoki kumulyativ yig'indi bilan quti filtr, reshape bilan zona yig'indisi) 300 kadr; ms/kadr, M piksel/s, tezlanish; NumPy'dagi oraliq massivlar soni va samarali GB/s.
2. (s2) @njit to'liq zanjir (bitta o'tishda farq + 5x5 yig'indi + bo'sag'a + zona hisoblagich) va prange satrlar bo'yicha 1/2 ip; JIT vaqti; ms/kadr, GB/s; zona hisoblagichlar reduktsiyasi (16 ta hisoblagich prange'da - poyga holati!) ni to'g'ri yozing (har ip o'z massivini yig'ib keyin qo'shadi).
3. (s3) Lokallik: 5x5 quti filtrni to'g'ridan-to'g'ri (25 o'qish) vs ajraladigan (5 gorizontal + 5 vertikal) vs integral tasvir (4 o'qish) bilan o'lchang; kadrlarni 1/10/50 talik paketlarda (paket x 300 KB keshga sig'adimi) ishlab GB/s; jadval.
4. (s4) Quti filtrni 4 usulda: Numba 1 ip, prange 2 ip, NumPy kumulyativ yig'indi (integral), torch F.avg_pool2d(stride 1, padding 2) yoki conv2d 5x5 (1/2 ip, 10 kadr batch); maks xato < 1e-4; jadval.
5. (s5) torch zanjir 1 vs 2 ip (10/50 kadr batch), uint8 -> float32 vs float16 (CPU) vaqt; eager (farq, abs, filtr, bo'sag'a, yig'indi alohida) vs iloji boricha birlashtirilgan ifoda vs Numba fused - samarali GB/s 1 va 2 ip.
6. (s6) Pool(1/2) kameralar bo'yicha (16 kamera x 300 kadr, har jarayon o'z kadrlarini seed bilan hosil qiladi va 4x4 zona statistikasini qaytaradi), chunksize 1/2/8; tezlanish, samaradorlik, Pool

xarajati; nima uchun kadrlarni emas, seed'ni yuborish kerak (IPC).

7. (s7) Amdahl: ketma-ket qism = ogohlantirish mantiqi va log (Python, har kadr), parallel = prange zanjir; S(2), p, S(8)/S(1024); Gustafson: kadr 240x320 -> 480x640 -> 960x1280 da p; grafik; xulosada 6 ms/kadr talabini qaysi usul qaysi ip sonida bajarishini jadvalda ko'rsating.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

10-variant. Tavsiya tizimi: 20 000 mahsulot embedding'lari orasida bloklangan kosinus o'xshashlik va top-k qiyin

Soha: Chakana savdo

Chakana savdo tarmog'ining onlayn do'koni 20 000 mahsulot uchun 128 o'lchamli embedding'lar asosida 'o'xshash mahsulotlar' (har mahsulot uchun top-10) ro'yxatini har kecha yangilaydi. To'liq o'xshashlik matritsasi 20 000 x 20 000 x 4 bayt = 1,6 GB - 1,5 GB xotiraga sig'maydi, shuning uchun hisob bloklanadi: 2 000 qatorli bloklar (160 MB) hisoblanib, darhol top-10 ajratiladi va blok tashlanadi. Bu GEMM (102 GFLOP) + argpartition (xotira bilan cheklangan) kombinatsiyasi; siz blok o'lchami, iplar va jarayonlarning jami vaqtga ta'sirini hamda Amdahl'da argpartition ulushini o'lchaysiz. Embedding'lar sintetik: 50 ta klaster markazi + shovqin, normallashtirilgan (seed 9).

Yadro	S = E E^T (bloklab) -> har qator top-k (argpartition) -> indekslar; normalizatsiya (reduktsiya)
O'lchamlar	mahsulot 2 000 / 20 000, o'lcham d = 64 / 128, k = 10
FLOP	2 x 20 000^2 x 128 = 102 GFLOP; S bloki 2 000 x 20 000 = 160 MB
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	qator bloki 250 / 1 000 / 2 000 / 5 000
Xotira chegarasi	<= 600 MB (1,5 GB muhit)
Ma'lumot	50 klaster + shovqin, seed 9

QADAMLAR

1. (s1) Sof Python kosinus o'xshashlik (3 sikl) 200 x 200 x 64 vs NumPy E @ E.T 2 000 x 2 000 (d 64/128); ms, GFLOP/s, tezlanish; top-10: Python sorted vs np.argpartition - alohida vaqt; 20 000 uchun to'liq matritsa xotirasini hisoblab, nima uchun bloklash shart ekanini ko'rsating.
2. (s2) @njit bloksiz skalyar ko'paytma + top-k (har qator uchun 'insertion' bilan 10 ta eng kattani saqlash, S saqlanmaydi!) va prange qatorlar bo'yicha 1/2 ip 2 000/20 000 (20 000 uchun vaqt chegarasi - 5 000 qatorda o'lchab ekstrapolyatsiya); JIT vaqti, GFLOP/s; normalizatsiya (satr normasi) prange reduktsiya 1 vs 2 ip.
3. (s3) Bloklash: blok 250/1 000/2 000/5 000 qator uchun GEMM (BLAS) + argpartition vaqtini alohida o'lchang, blok xotirasini (blok x 20 000 x 4 bayt) va psutil bilan haqiqiy RSS cho'qqisini yozing; eng yaxshi blok va uning sababi (GEMM samaradorligi vs kesh/xotira); jadval + grafik.
4. (s4) Bitta blokni (2 000 x 20 000) 4 usulda: Numba 1 ip (ikj), prange 2 ip, NumPy BLAS, torch matmul 1/2 ip; top-k uchun torch.topk vs np.argpartition; etalon bilan indekslar mosligi (to'plam tengligi, bir xil qiymatlar uchun tartib farqini hisobga oling).
5. (s5) torch matmul blok 1 vs 2 ip, float32 vs bfloat16 vs int8 (torch._int_mm, embedding'larni 127 ga masshtablab) - vaqt, top-10 ro'yxatlar mosligi (%); normalizatsiya eager (kvadrat, yig'indi, sqrt, bo'lish) vs F.normalize - GB/s.
6. (s6) Pool(1/2) qator bloklari bo'yicha (10 blok x 2 000, E fork orqali meros bo'ladi - nusxalanmaydi, natija 2 000 x 10 indeks), chunksize 1/2/5; tezlanish; 2 jarayon x BLAS 2 ip = 4

ip 2 yadroda - ortiqcha obuna (oversubscription) ta'sirini OMP_NUM_THREADS=1 bilan solishtiring.

7. (s7) Amdahl: GEMM (parallel, BLAS 2 ip) + argpartition (NumPy, 1 ip - ketma-ket) ulushlari: profil bilan p ni to'g'ridan-to'g'ri hisoblang va S(2) o'lchovidan olingan p bilan solishtiring; S(8)/S(1024) - GPU'da argpartition ham parallel bo'lsa (torch.topk) p qanday o'zgaradi; Gustafson: 20 000 -> 200 000 mahsulot (ekstrapolyatsiya) ; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; Xotira profili: blok o'lchami vs RSS cho'qqisi (psutil) grafigi

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

11-variant. Yetkazib berish nuqtalari orasida masofa matritsasi (haversine) va eng yaqin ombor oson

Soha: Logistika

Kuryerlik kompaniyasi shahar bo'ylab 4 000 ta yetkazib berish nuqtasi va 40 ta ombor koordinatalariga ega. Har kuni ertalab barcha nuqtalar orasidagi masofa matritsasi (haversine formulasi, 16 M juft) va har nuqta uchun eng yaqin ombor hamda 5 ta eng yaqin qo'shni hisoblanadi (marshrutlash uchun). Haversine - trigonometrik funksiyalarga boy elementlararo yadro (AI yuqori, ~25 FLOP/juft), shuning uchun vektorlash va prange yaxshi tezlanish beradi, lekin NumPy broadcasting 16 M x 8 bayt oraliq massivlar yaratadi. Koordinatalar sintetik: 6 ta tuman markazi atrofida normal taqsimot (seed 10).

Yadro	haversine(lat1, lon1, lat2, lon2) barcha juftliklar + argmin (eng yaqin ombor) + top-5 qo'shni
O'lchamlar	nuqta 500 / 4 000, ombor 40; matritsa 4 000^2 float32 = 64 MB
FLOP	~25 FLOP/juft x 16 M = 0,4 GFLOP (trig bilan)
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	qator bloklari 100 / 500 / 4 000
Vaqt chegarasi	4 000 nuqta <= 1 s
Ma'lumot	6 tuman markazi atrofida sintetik koordinatalar, seed 10

QADAMLAR

1. (s1) Sof Python haversine (math.sin/cos, ikki sikl) 500 x 500 vs NumPy broadcasting (lat[:,None], lon[None,:]) 500/4 000 float32 va float64; ms, M juft/s, tezlanish, maks xato (metrda); NumPy'da yaratiladigan oraliq massivlar soni va MB.
2. (s2) @njit haversine (ikki sikl, oldindan hisoblangan sin/cos lat) va prange qatorlar bo'yicha 1/2 ip; JIT vaqti; M juft/s, tezlanish; eng yaqin ombor (argmin 40 ta) va o'rtacha masofa prange reduktsiya 1 vs 2 ip.
3. (s3) Lokallik: qator bloklari 100/500/4 000 (blok x 4 000 x 4 bayt) bilan masofa + top-5 ni blokda hisoblab matritsani saqlamaslik; sin/cos ni oldindan hisoblash (4 000 x 4 qiymat) FLOP'ni necha baravar kamaytiradi; simmetriyadan foydalanib yarmini hisoblash; jadval.
4. (s4) Masofani 4 usulda: Numba 1 ip, prange 2 ip, NumPy broadcasting, torch (torch.sin/cos broadcasting yoki torch.cdist 3D dekart koordinatalarida - yer sharida xorda masofasi, 2 asin(c/2R) bilan haversine'ga aylantiring) 1/2 ip; maks xato < 1 m; jadval.
5. (s5) torch 1 vs 2 ip (500/4 000), float32 vs float64 vs float16 (xato metrda!), eager (8 alohida trig/arifmetik amal) vs iloji boricha kam oraliq massiv (in-place out=) vs Numba fused - GB/s.
6. (s6) Pool(1/2) tumanlar bo'yicha (6 tuman, har vazifa o'z nuqtalari uchun eng yaqin ombor va top-5 ni qaytaradi; nuqtalar fork orqali meros), chunksize 1/2/3; tezlanish, yuk muvozanati (tumanlar har xil kattalikda!) - imap_unordered bilan solishtiring.
7. (s7) Amdahl: ketma-ket qism = marshrut hisobotini Python'da shakllantirish (4 000 satr), parallel = prange masofa; S(2), p, S(8)/S(1024); Gustafson: 500 -> 4 000 -> 12 000 nuqta

(bloklab) p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

12-variant. Havo sifati xaritasi: stansiya o'lchovlarini IDW bilan shahar to'riga interpolyatsiya qilish oson

Soha: Shahar

Shahar ekologiya portali har 10 daqiqada 200 ta stansiyaning PM2.5 o'lchovlaridan 1 000 x 1 000 nuqtali xarita (teskari masofa og'irligi, IDW, p=2) tuzadi: har to'r nuqtasi uchun 200 stansiyagacha masofa, og'irlik 1/d^2 va og'irlikli o'rtacha. Jami 200 M masofa hisobi; xarita 5 s ichida tayyor bo'lishi kerak. Yadro - to'r nuqtalari bo'yicha 'uyatli parallel', ichki sikl esa reduktsiya (og'irliklar yig'indisi); NumPy broadcasting to'liq 1 000 000 x 200 x 4 bayt = 800 MB matritsa yaratadi - bo'laklash shart. Stansiyalar va qiymatlar sintetik: 3 ta ifloslanish manbai atrofida (seed 11).

Yadro	IDW: z(x) = sum(w_i z_i)/sum(w_i), w_i = 1/d_i^2 (200 stansiya) har to'r nuqtasi uchun
O'lchamlar	to'r 200x200 / 1 000x1 000, stansiya 50 / 200
FLOP	~8 FLOP x 1 M nuqta x 200 stansiya = 1,6 GFLOP
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Bo'lak	to'r satr bo'laklari 10 / 50 / 200 satr (bo'lak x 1 000 x 200 x 4 bayt)
Vaqt chegarasi	1 000x1 000 xarita <= 5 s
Ma'lumot	200 stansiya, 3 manba, seed 11

QADAMLAR

1. (s1) Sof Python IDW (3 sikl) 200x200 to'r x 50 stansiya vs NumPy broadcasting (to'r bo'laklab 50 satr) 200x200 va 1 000x1 000 x 200; ms, M nuqta-stansiya/s, tezlanish, maks xato; bo'laksiz broadcasting xotirasini hisoblab ko'rsating.
2. (s2) @njit IDW (to'r i, j tashqi, stansiya ichki - 2 ta akkumulyator) va prange satrlar bo'yicha 1/2 ip; JIT vaqti; GFLOP/s, tezlanish; xarita maksimumi va o'rtachasi prange reduktsiya 1 vs 2 ip.
3. (s3) Lokallik: stansiya siklini tashqi (200 ta to'liq xarita o'tishi) vs ichki (1 o'tish) qilib GB/s farqini o'lchang; satr bo'laklari 10/50/200 (NumPy variantida) ta'siri; stansiya koordinatalarini float32 massivda ketma-ket saqlash (SoA) vs (200, 2) (AoS); jadval.
4. (s4) IDW'ni 4 usulda: Numba 1 ip, prange 2 ip, NumPy bo'lakli broadcasting, torch (torch.cdist to'r-stansiya + og'irlik matvec: W @ z / W.sum(1)) 1/2 ip; maks xato < 1e-4; torch variantida cdist + 2 matvec 'GEMM'ga keltirish' g'oyasi.
5. (s5) torch 1 vs 2 ip (200x200 / 1 000x1 000), float32 vs float64 vs bfloat16 (1/d^2 da xato), eager (cdist, kvadrat, teskari, matvec, bo'lish) vs kam oraliq massiv - GB/s.
6. (s6) Pool(1/2) to'r satr bo'laklari bo'yicha (20 bo'lak x 50 satr; stansiyalar fork orqali), chunksize 1/2/10; tezlanish, Pool xarajati, natija massivlarini birlashtirish (np.vstack) vaqti; natija tekshiruvi.
7. (s7) Amdahl: ketma-ket qism = xaritani PNG'ga render qilish (matplotlib imsave) + Python hisobot, parallel = prange IDW; S(2), p, S(8)/S(1024); Gustafson: to'r 200x200 -> 1 000x1 000 -> 2 000x2 000 da p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

13-variant. Sun'iy yo'ldosh tasvirlarida o'zgarishni aniqlash: oynali statistika integral tasvir bilan o'rta

Soha: Ekologiya

Tabiatni muhofaza qilish agentligi ikki sanadagi 2048x2048 sun'iy yo'ldosh tasvirini (1 kanal, aks ettirish) solishtirib o'rmon kesilishi va suv havzasi o'zgarishini topadi: har piksel atrofida 11x11 oynada o'rtacha va dispersiya hisoblanadi (ikkala tasvir uchun), so'ng normallashtirilgan farq va bo'sag'a qo'llanadi. Oynali statistika sodda yo'l bilan 121 o'qish/piksel (1 GFLOP), integral tasvir (kumulyativ yig'indi) bilan esa 4 o'qish/piksel - algoritmik tezlanish parallel tezlanishdan katta bo'lishi mumkin; kumulyativ yig'indi esa 'prefix sum' - parallellashtirish uchun qiyin klassik yadro. Tasvirlar sintetik: Perlin uslubidagi silliq maydon + o'zgargan bloklar + shovqin (seed 12).

Yadro	11x11 oynali o'rtacha/dispersiya (to'g'ridan-to'g'ri va integral tasvir) + normallashtirilgan farq + bo'sag'a
O'lchamlar	tasvir 512 / 1 024 / 2 048 (float32: 16 MB x 2), oyna 5 / 11 / 21
FLOP	to'g'ridan-to'g'ri: 2 x 121 x 4,2 M x 2 tasvir = 2 GFLOP (2048); integral: ~20 FLOP/piksel
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	satr bo'laklari 32 / 128 / 512 (halo bilan)
Vaqt chegarasi	2048 juft tasvir <= 1 s
Ma'lumot	sintetik silliq maydon + o'zgargan bloklar, seed 12

QADAMLAR

1. (s1) Sof Python oynali o'rtacha (4 sikl) 256x256, oyna 5 vs NumPy: scipy.ndimage.uniform_filter va integral tasvir (np.cumsum ikki o'q bo'ylab + 4 burchak) 512/1024/2048, oyna 5/11/21; ms, M piksel/s, tezlanish; integral usulda vaqt oyna o'lchamiga bog'liq emasligini jadvalda ko'rsating; maks xato (float64 integral vs float32 - katastrofik qisqarish!).
2. (s2) @njit to'g'ridan-to'g'ri oynali o'rtacha/dispersiya va prange satrlar bo'yicha 1/2 ip (oyna 11, 1024/2048); JIT vaqti, GFLOP/s, tezlanish; o'zgargan piksellar soni (bo'sag'adan oshganlar) prange reduktsiya 1 vs 2 ip.
3. (s3) Lokallik va algoritm: (a) to'g'ridan-to'g'ri oyna, (b) ajraladigan (11 gorizontal + 11 vertikal), (c) sirpanuvchi yig'indi (har qadam 1 qo'shish 1 ayirish), (d) integral tasvir - 1 ipda GFLOP/s va ms; satr bo'laklari 32/128/512 (halo 5 satr) ta'siri; qaysi usul xotira, qaysi hisob bilan cheklangan.
4. (s4) Oynali o'rtachani 4 usulda: Numba 1 ip (sirpanuvchi), prange 2 ip, NumPy integral tasvir, torch F.avg_pool2d(11, stride 1, padding 5) yoki conv2d 1/2 ip; dispersiya E[x^2]-E[x]^2; maks xato < 1e-3; jadval.
5. (s5) torch avg_pool2d 1 vs 2 ip (1024/2048), float32 vs float64 vs bfloat16; integral tasvir torch.cumsum ikki o'q (prefix sum CPU'da 1 ip!) 1 vs 2 ip; normallashtirilgan farq eager (6 amal) vs bitta ifoda vs Numba fused - GB/s.

6. (s6) Pool(1/2) tasvir plitkalari bo'yicha (16 plitka 512x512 + 5 piksel halo, har vazifa o'zgargan piksel maskasini qaytaradi), chunksize 1/2/8; tezlanish, IPC (512x512 bool = 256 KB) ulushi; plitka chegaralarida natija ketma-ket bilan bir xilligini tekshiring.
7. (s7) Amdahl: ketma-ket qism = cumsum (torch/numpy 1 ip) va bog'langan komponentlar (scipy.ndimage.label, 1 ip), parallel = prange statistika; profil bilan p va S(2) dan p; S(8)/S(1024); Gustafson: 512 -> 2048 -> 4096 (1 tasvir 64 MB, ehtiyot) p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

14-variant. Futbol o'yinchilari traektoriyasini Kalman filtri bilan silliqlash (ko'p obyektli kichik matritsalar) o'rta

Soha: Sport

Sport analitika kompaniyasi o'yin videosidan olingan 22 o'yinchi + to'p koordinatalarini (25 kadr/s, 90 daqiqa = 135 000 kadr) Kalman filtri bilan silliqlaydi va tezlik/tezlanishni baholaydi: holat 4 o'lchamli (x, y, vx, vy), har qadamda bir nechta 4x4 va 4x2 matritsa amallari. Bu 'ko'p mayda matritsa' (batched small matmul) yadrosi: BLAS 4x4 uchun samarasiz, Python sikli esa 3 M qadam uchun juda sekin; vaqt o'qi bo'ylab bog'liqlik bor (parallellashmaydi), obyektlar bo'ylab mustaqil (23 ta - ip sonidan ko'p, lekin uncha ko'p emas). Trayektoriyalar sintetik: silliq harakat + o'lchov shovqini (seed 13).

Yadro	Kalman bashorat + yangilash: F x, F P F^T + Q, K = P H^T (H P H^T + R)^-1 (4x4, 4x2, 2x2 teskari)
O'lchamlar	obyekt 23 / 230 (10 o'yin birga), qadam 13 500 / 135 000
FLOP	~600 FLOP/qadam x 23 x 135 000 = 1,9 GFLOP
Iplar/jarayonlar	Numba 1/2, torch bmm 1/2, Pool 1/2
Paket	obyektlar paketi 1 / 23 / 230 (torch bmm uchun)
Vaqt chegarasi	1 o'yin (23 x 135 000) <= 2 s
Ma'lumot	sintetik traektoriyalar + shovqin, seed 13

QADAMLAR

1. (s1) Sof Python Kalman (ro'yxatlar bilan 4x4 amallar qo'lda yozilgan) 1 obyekt x 13 500 qadam vs NumPy (har qadamda np kichik massivlar - 'NumPy mayda massiv' xarajati!) vs NumPy vektorlangan obyektlar bo'yicha (23 x 4 x 4 einsum/matmul); ms, M qadam/s, tezlanish; maks xato.
2. (s2) @njit Kalman (4x4 amallar skalyar yoki kichik massiv) 1 obyekt va prange obyektlar bo'yicha 1/2 ip (23 va 230 obyekt); JIT vaqti; M qadam/s, tezlanish; o'rtacha tezlik va maksimal tezlanish prange reduktsiya 1 vs 2 ip; 23 obyekt 2 ipga qanday taqsimlanadi (12+11) - yuk muvozanati.
3. (s3) Lokallik: holat va kovariatsiyani (obyekt, 4) / (obyekt, 4, 4) (AoS) vs (4, obyekt) / (4, 4, obyekt) (SoA - ichki sikl obyektlar bo'ylab vektorlashadi) joylashuvida o'lchang; qadamlarni 100/1 000/13 500 lik bo'laklarda ishlab (o'lchov massivi keshda) GFLOP/s; jadval.
4. (s4) Bir qadamni 230 obyekt uchun 4 usulda: Numba 1 ip, prange 2 ip, NumPy np.matmul batched (230x4x4 @ 230x4x4) + np.linalg.inv 2x2 batched, torch.bmm 1/2 ip (+ torch.linalg.inv); 135 000 qadam uchun jami; maks xato < 1e-6; qaysi usulda 'bir qadam xarajati' (Python chaqiruv) ustun.
5. (s5) torch bmm zanjiri 1 vs 2 ip (23/230/2 300 obyekt), float32 vs float64 (kovariatsiya barqarorligi!), eager (8 ta bmm/add alohida) vs oldindan hisoblangan doimiy F P F^T + Q bloklari bilan kamroq chaqiruv - M qadam/s; 'batch kattalashgani sari vaqt/qadam/obyekt' grafigi.

6. (s6) Pool(1/2) o'yinlar bo'yicha (10 o'yin x 23 obyekt, har vazifa o'z traektoriyasini seed bilan hosil qiladi va tezlik statistikasini qaytaradi), chunksize 1/2/5; tezlanish, Pool xarajati; obyektlar bo'yicha bo'lish (23 vazifa) bilan solishtiring.
7. (s7) Amdahl: vaqt o'qi ketma-ket, obyektlar parallel - S(2) dan p; obyekt soni 23 -> 230 -> 2 300 da S(2) va p qanday o'sadi (Gustafson: ko'proq o'yin birga); S(8)/S(1024) - GPU'da 23 obyekt bilan nima bo'ladi (band bo'lmagan yadrolar); grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

15-variant. Video quvuri: 1080p kadrlarni 720p ga bilinear kichraytirish va RGB -> YUV konvertatsiya oson

Soha: Media

Onlayn video platformasi yuklangan rolik kadrlarini (1920x1080 RGB) oldindan ko'rish uchun 1280x720 ga kichraytiradi va kodlashdan oldin YUV (BT.709) fazosiga o'tkazadi. Bir rolik 500 kadr; 30 kadr/s real vaqtdan tez (ya'ni <= 33 ms/kadr) ishlanishi kerak. Bilinear interpolyatsiya - har chiqish pikseli uchun 4 ta kirish pikseli (gather), RGB -> YUV esa har pikselga 3x3 matritsa (elementlararo GEMM) - ikkalasi ham xotira bilan cheklangan; siz birlashtirish (ikkisini bitta o'tishda) va iplar/jarayonlar ta'sirini o'lchaysiz. Kadrlar sintetik gradient + harakatlanuvchi shakllar (seed 14); xotirada 20 kadr (124 MB), qolgani jarayon ichida hosil qilinadi.

Yadro	bilinear 1920x1080 -> 1280x720 (3 kanal) + RGB -> YUV 3x3 matritsa + Y gistogrammasi (reduktsiya)
O'lchamlar	kadr 1080p (6,2 MB uint8), kadrlar 20 xotirada / 500 jami
Amallar	bilinear ~12 FLOP + YUV 18 FLOP = 30 FLOP x 0,92 M piksel = 28 MFLOP/kadr
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Paket	kadr paketi 1 / 5 / 20
Vaqt chegarasi	<= 33 ms/kadr
Ma'lumot	sintetik kadrlar, seed 14

QADAMLAR

1. (s1) Sof Python bilinear (ikki sikl, 1 kanal, 192x108 -> 128x72) vs NumPy (indeks massivlari + 4 ta gather + og'irliklar, fancy indexing) to'liq kadr; RGB -> YUV NumPy (reshape(-1,3) @ M.T) ; ms/kadr, M piksel/s, tezlanish; NumPy'dagi oraliq massivlar (float32 3 kanal x 4 gather) MB.
2. (s2) @njit bilinear + YUV bitta o'tishda (chiqish pikseli bo'yicha) va prange chiqish satrlari bo'yicha 1/2 ip; JIT vaqti; ms/kadr, GB/s (o'qish 4x3 bayt + yozish 3 bayt / piksel); Y gistogrammasi (256 bin) prange'da to'g'ri reduktsiya (har ip o'z gistogrammasi).
3. (s3) Lokallik: chiqish satri bo'yicha o'tish (kirishning 2 satri keshda) vs ustun bo'yicha; kadrlarni 1/5/20 lik paketlarda ishlab GB/s; uint8 -> float32 konvertatsiyani alohida o'tish vs ichkarida qilish; jadval.
4. (s4) Kichraytirish + YUV'ni 4 usulda: Numba 1 ip, prange 2 ip, NumPy gather + GEMM (3x3), torch F.interpolate(mode='bilinear', align_corners=False) + einsum 1/2 ip (paket 5 kadr); maks xato <= 1 (uint8) - align_corners va yarim-piksel kelishuvini moslashtiring; jadval.
5. (s5) torch interpolate + YUV 1 vs 2 ip (paket 1/5/20), float32 vs float16 vs uint8->float32 vaqti; YUV eager (3 ta kanal ifodasi alohida) vs einsum/matmul bitta chaqiruv vs Numba fused - GB/s; antialias=True farqi.
6. (s6) Pool(1/2) roliklar bo'yicha (10 rolik x 50 kadr, har jarayon o'z kadrlarini seed bilan hosil qiladi va Y gistogrammasini qaytaradi), chunksize 1/2/5; tezlanish, samaradorlik; kadrlarni pickle bilan yuborish (6 MB) vs seed yuborish IPC farqi - o'lchab ko'rsating.

7. (s7) Amdahl: ketma-ket qism = kadrni PNG'ga yozish (1 ip, matplotlib/PIL) yoki metadata Python, parallel = prange kichraytirish; S(2), p, S(8)/S(1024); Gustafson: 720p -> 1080p -> 4K (1 kadr 25 MB) chiqishda p; grafik; 33 ms/kadr talabini qaysi usul bajaradi.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

16-variant. Sayohat tavsiya tizimi: ALS matritsa faktorizatsiyasini foydalanuvchilar bo'yicha parallellashtirish qiyin

Soha: Turizm

Sayohat platformasi 50 000 foydalanuvchi x 5 000 mehmonxona/tur reytinglarini (2 M baho, siyrak) rank-32 ALS (Alternating Least Squares) bilan faktorlaydi. Har iteratsiyada har foydalanuvchi uchun o'zi baholagan k_u ta obyekt vektorlaridan Gram (32x32) yig'ilib, 32x32 chiziqli tizim yechiladi; so'ng obyektlar uchun xuddi shunday. Foydalanuvchilar mustaqil (prange), lekin har biri uchun ish hajmi har xil (k_u 5 dan 500 gacha) - yuk muvozanati masalasi; Gram yig'ish - mayda matmul, solve - LAPACK. 10 iteratsiya 20 s ichida tugashi kerak. Reytinglar sintetik: yashirin faktorlar + shovqin, k_u Zipf taqsimotida (seed 15).

Yadro	ALS: har foydalanuvchi uchun A = V_u^T V_u + lambda I (32x32), b = V_u^T r_u, solve; so'ng obyektlar uchun
O'lchamlar	foydalanuvchi 5 000 / 50 000, obyekt 5 000, rank 16 / 32, baholar 200 k / 2 M (CSR)
FLOP	Gram: 2 x 2 M x 32^2 = 4,1 GFLOP; solve 50 000 x (2/3) 32^3 = 1,1 GFLOP har iteratsiya
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Rejalashtirish	prange statik vs foydalanuvchilarni k_u bo'yicha saralab taqsimlash
Vaqt chegarasi	10 iteratsiya <= 20 s
Ma'lumot	yashirin faktorlar + Zipf k_u, seed 15

QADAMLAR

1. (s1) Sof Python ALS foydalanuvchi qadami (ro'yxatlar, Gram 3 sikl, Gauss yo'q qilish) 100 foydalanuvchi vs NumPy (V[idx].T @ V[idx], np.linalg.solve) 5 000/50 000; ms, GFLOP/s, tezlanish; RMSE har iteratsiyada kamayishini jadvalda ko'rsating (to'g'rilik).
2. (s2) @njit ALS foydalanuvchi qadami (CSR indptr/indices/data, Gram ikki sikl, np.linalg.solve njit ichida) va prange foydalanuvchilar bo'yicha 1/2 ip; JIT vaqti (LAPACK chaqiruvi bilan); GFLOP/s, tezlanish; RMSE prange reduktsiya 1 vs 2 ip; 2 ipda tezlanish 2 dan past bo'lsa k_u notekisligi sababini profil bilan ko'rsating.
3. (s3) Lokallik va muvozanat: foydalanuvchilarni k_u bo'yicha kamayish tartibida saralab prange'ga berish vs tasodifiy tartib; V matritsasini (obyekt, rank) C-tartibda vs F-tartibda; Gram'ni V[idx] nusxalab GEMM (BLAS) vs nusxasiz to'plash (gather) - rank 16/32 uchun jadval.
4. (s4) Bir foydalanuvchi guruhi (1 000 ta) uchun Gram + solve 4 usulda: Numba 1 ip, prange 2 ip, NumPy (padding bilan batched: k_max gacha nol to'ldirilgan (1000, k_max, 32) tensor va np.matmul + np.linalg.solve batched), torch bmm + torch.linalg.solve 1/2 ip; maks xato < 1e-5; padding qancha ortiqcha FLOP keltiradi - hisoblang.
5. (s5) torch batched solve 1 vs 2 ip (1 000/10 000 foydalanuvchi), float32 vs float64 (A shartliligi, lambda ta'siri); Gram eager (transpose, bmm, add lambda) vs torch.baddbmm bitta chaqiruv - GFLOP/s; bfloat16 Gram xatosi.

6. (s6) Pool(1/2) foydalanuvchi bloklari bo'yicha (10 blok x 5 000; CSR va V fork orqali meros, natija U bloki 5 000 x 32 float32 = 640 KB), chunksize 1/2/5; tezlanish; Numba prange ichidagi LAPACK iplari + 2 jarayon = ortiqcha obuna - OMP_NUM_THREADS/numba.set_num_threads(1) bilan solishtiring.
7. (s7) Amdahl: foydalanuvchi qadami (parallel) + obyekt qadami (parallel) + RMSE va Python boshqaruv (ketma-ket): profil ulushlari va S(2) dan p; k_u notekisligi tufayli 'yuk muvozanati yo'qotishi' ni Amdahl'ning ketma-ket qismi kabi ko'rsating; S(8)/S(1024); Gustafson: 5 000 -> 50 000 foydalanuvchi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; Yuk muvozanati grafigi: har ipga tushgan k_u yig'indisi (statik vs saralangan taqsimot)

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

17-variant. Qurilish konstruksiyasining chekli elementlar modeli: siyrak SpMV va konjugat gradientlar qiyin

Soha: Qurilish

Loyiha instituti ko'p qavatli bino karkasining issiqlik/deformatsiya maydonini 100x100x100 = 1 M tugunli 3D to'rda modellashtiradi; natijaviy tizim siyrak (har qatorda 7 nol bo'lmagan element, CSR, 7 M nnz) va konjugat gradientlar (CG) usuli bilan yechiladi: har iteratsiya 1 SpMV + 2 skalyar ko'paytma + 3 AXPY. SpMV - xotira bilan cheklangan yadro (AI ~0,15 FLOP/bayt): 7 M nnz x (4 bayt qiymat + 4 bayt indeks) + vektorlar = ~70 MB har iteratsiya, hisob esa atigi 14 MFLOP. Shuning uchun 2 ip tezlanishi DRAM o'tkazuvchanligi bilan cheklanadi, format (float32/float64, int32 indeks) va qatorlarni qayta tartiblash (lokallik) esa real ta'sir beradi. 100 iteratsiya 3 s ichida bajarilishi kerak. Matritsa sintetik: 3D 7 nuqtali Laplas + diagonal ustunlik (seed 16).

Yadro	CSR SpMV y = A x + CG iteratsiyasi (dot, axpy); 1 M x 1 M, 7 nnz/qator
O'lchamlar	to'r 32^3 / 64^3 / 100^3 (nnz 0,23 M / 1,8 M / 7 M)
FLOP va bayt	SpMV 14 MFLOP, ~70 MB trafik (float32 + int32) -> AI ~0,2
Iplar/jarayonlar	Numba 1/2, torch.sparse 1/2, scipy.sparse (1 ip), Pool 1/2
Blok	qator bloklari 1 000 / 10 000 / 100 000; ELL/CSR format
Vaqt chegarasi	100 CG iteratsiyasi (100^3) <= 3 s
Ma'lumot	7 nuqtali Laplas + diagonal, seed 16

QADAMLAR

1. (s1) Sof Python CSR SpMV (indptr sikli) 32^3 vs scipy.sparse.csr_matrix @ x va NumPy 'stensil shaklida' (7 ta siljitilgan massiv qo'shish, matritsasiz) 32^3/64^3/100^3; ms, GFLOP/s va GB/s, tezlanish; CSR vs stensil xotira trafigi (indeks saqlanmaydi!) farqi.
2. (s2) @njit CSR SpMV va prange qatorlar bo'yicha 1/2 ip; JIT vaqti; GB/s samarali (nnz x 8 + 2 x n x 4 bayt / vaqt) - DRAM chegarasi bilan solishtiring (A2 2-qadam reduktsiya natijasi); CG'dagi dot va norma prange reduktsiya 1 vs 2 ip; 100 iteratsiya CG qoldiq egri chizig'i (to'g'rilik: ||r|| < 1e-6).
3. (s3) Lokallik: tugunlarni tabiiy (i,j,k) vs tasodifiy almashtirilgan tartibda raqamlash - SpMV vaqti (x ga murojaat lokalligi); float64 + int64 vs float32 + int32 formatlar; ELLPACK (n x 7 zich) vs CSR; qator bloklari 1 000/10 000/100 000 (prange chunk) ta'siri; jadval + GB/s.
4. (s4) SpMV'ni 4 usulda: Numba 1 ip, prange 2 ip, scipy.sparse (1 ip), torch.sparse_csr_tensor @ x 1/2 ip (yoki torch stensil: F.conv3d 3x3x3 yadro 7 nuqta - 'matritsasiz' variant); maks xato < 1e-5; qaysi usul GB/s bo'yicha DRAM chegarasiga eng yaqin.
5. (s5) torch CG (sparse mm + dot + axpy) 1 vs 2 ip (64^3/100^3), float32 vs float64 (CG yaqinlashuvi!); AXPY'lar eager (3 alohida) vs torch.add(alpha=) in-place vs Numba fused 'SpMV + dot' (bir o'tishda) - GB/s; iteratsiya vaqtida SpMV vs vektor amallari ulushi.
6. (s6) Pool(1/2) qator bloklari bo'yicha SpMV (A va x fork orqali meros, natija y bloki qaytadi) - tezlanish va nima uchun 100 iteratsiya CG'ni Pool bilan qilish befoyda (har iteratsiyada x ni

jarayonlarga yuborish 4 MB); muqobil: 8 ta mustaqil yuk holati (o'ng tomon) ni Pool'da bo'lish - tezlanish.

7. (s7) Amdahl: CG'da parallel (SpMV, prange) va ketma-ket (skalyar alpha/beta hisobi, Python sikl boshqaruvi, 100 x 2 bar'yer) ulushlari; S(2) dan p va profil p; S(8)/S(1024) - xotira chegarasi tufayli Amdahl bashorati nima uchun optimistik (GB/s chegara modelini qo'shing); Gustafson: 32^3 -> 100^3 -> 160^3 (float32, 29 M nnz, ehtiyot) ; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; Roofline-uslubidagi nuqta: SpMV AI va erishilgan GFLOP/s ni CPU xotira chegarasiga nisbatan (grafik)

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

18-variant. Kon seysmik monitoringi: 3D to'lqin tenglamasini 7 nuqtali stensil bilan modellashtirish qiyin

Soha: Ko'mir / kon sanoati

Ko'mir koni geofizika xizmati tog' jinslarida seysmik to'lqin tarqalishini 192x192x192 3D to'rda (7 nuqtali Laplas stensili, ikkinchi tartibli vaqt sxemasi) 50 vaqt qadamida modellashtiradi: har qadam 7 M katak x 10 FLOP, ikki bufer almashinadi. Portlash ishlaridan keyin natija 10 s ichida kerak. 3D stensil - GPU'dagi klassik 'shared memory tiling' yadrosi; CPU'da u xotira bilan cheklangan (AI ~0,6), lekin 3D bloklash (kesh plitkalari) va vaqt bo'yicha bloklash AI'ni oshiradi. Z o'qi bo'ylab prange yuk muvozanati va halo almashinuvini ko'rsatadi. Tezlik modeli sintetik: 3 qatlam + qazilgan bo'shliq, manba Ricker impulsi (seed 17).

Yadro	3D 7 nuqtali stensil: u_new = 2u - u_old + c^2 dt^2 (lap u) + manba; energiya reduktsiyasi
O'lchamlar	to'r 64^3 / 128^3 / 192^3 (float32: 28 MB x 3 bufer), qadam 50
FLOP	10 FLOP x 7,1 M x 50 = 3,5 GFLOP (192^3)
Iplar/jarayonlar	Numba 1/2, torch conv3d 1/2, Pool 1/2 (Z bo'yicha domen)
Blok	3D plitka 8 / 16 / 32 (y-z tekislikda), vaqt bloki 1 / 2
Vaqt chegarasi	192^3 x 50 qadam <= 10 s
Ma'lumot	3 qatlam tezlik modeli + bo'shliq, Ricker manba, seed 17

QADAMLAR

1. (s1) Sof Python 3D stensil 32^3 x 5 qadam (3 sikl) vs NumPy slicing (6 ta siljitilgan massiv) 64^3/128^3/192^3 x 50 qadam; ms, M katak/s, GFLOP/s, tezlanish; NumPy'da har qadam yaratiladigan oraliq massivlar (6 x 28 MB) va samarali GB/s; energiya saqlanishi tekshiruvi (to'g'rilik).
2. (s2) @njit stensil (i, j, k sikllari, k ichki - ketma-ket xotira) va prange eng tashqi (i) o'q bo'yicha 1/2 ip; JIT vaqti; GFLOP/s va GB/s (7 o'qish + 1 yozish + u_old); to'lqin energiyasi sum(u^2) prange reduktsiya 1 vs 2 ip; prange'ni ichki o'qqa ko'chirib nima bo'lishini o'lchang.
3. (s3) 3D bloklash: y-z plitkalari 8/16/32 (plitka x 3 tekislik x 4 bayt keshda), sikl tartibi (i-j-k vs k-j-i), vaqt bo'yicha bloklash 2 qadam (halo 2 katak); har varianti uchun GFLOP/s va erishilgan AI; 'plitka hajmi - L2 kesh' jadvali; 192^3 da bloklash foyda beradimi.
4. (s4) Stensilni 4 usulda: Numba 1 ip, prange 2 ip, NumPy slicing, torch F.conv3d (3x3x3 yadro, 7 nol bo'lmagan koeffitsiyent) 1/2 ip; 50 qadamdan keyin maydon maks farqi < 1e-3 (float32); torch conv3d 27 ko'paytma bajaradi (FLOP 4x ko'p) - lekin GFLOP/s qanday.
5. (s5) torch conv3d 1 vs 2 ip (64^3/128^3/192^3), float32 vs float64 vs bfloat16 (50 qadamda dispersiya xatosi); vaqt sxemasi eager (2u, -u_old, c^2 lap, + manba alohida) vs torch.addcmul / bitta ifoda vs Numba fused - GB/s.
6. (s6) Pool(1/2) Z (i) o'qi bo'yicha domen dekompozitsiyasi: 2 yarim to'r, har qadamda halo tekisligi (192x192x4 bayt = 147 KB) mp.Pipe orqali almashinadi; tezlanish va aloqa ulushi; muqobil: 8 ta mustaqil manba joylashuvini Pool'da bo'lish (p ~ 1) - solishtiring.

7. (s7) Amdahl: ketma-ket qism = manba qo'shish + chegara (PML soddalashtirilgan) Python'da har qadam + halo almashinuvi, parallel = prange stensil; S(2) dan p va profil p; S(8)/S(1024) va xotira chegarasi tuzatishi; Gustafson: 64^3 -> 192^3 -> 256^3 (3 x 64 MB) ; grafik; GPU (RTX 3060, 360 GB/s) uchun kutilgan M katak/s bashorati.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; 3D plitka sxemasi (matn/rasm) va 'plitka hajmi - kesh' jadvali

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

19-variant. Kredit skoring: logistik regressiyani gradient tushishi bilan o'qitish (matvec + fused sigmoid) o'rta

Soha: Bank

Bankning kredit bo'limi 500 000 mijoz x 64 belgi (daromad, qarz yuki, tarix...) ma'lumotida logistik regressiya modelini to'liq gradient tushishi bilan qayta o'qitadi (haftasiga, 200 epoxa). Har epoxa: z = X w (matvec), p = sigmoid(z), gradient X^T (p - y), log-loss (reduktsiya) - ya'ni 2 ta tor GEMV (AI past: 64 FLOP/qator per 256 bayt) + elementlararo amallar. Bu yadro xotira bilan cheklangan (X 128 MB har epoxa ikki marta o'qiladi), shuning uchun X'ni bloklab bir o'tishda ikkala GEMV'ni va sigmoid'ni birlashtirish (fused) asosiy tezlanish manbai. 200 epoxa 60 s ichida tugashi kerak. Ma'lumot sklearn make_classification (500 000 x 64, class_sep 1,0, seed 18); to'g'rilik - sklearn LogisticRegression koeffitsiyentlari bilan solishtirish.

Yadro	GEMV X w, sigmoid, GEMV X^T r, log-loss reduktsiyasi (epoxa)
O'lchamlar	mijoz 50 000 / 500 000, belgi 32 / 64 (float32: 128 MB), epoxa 20 / 200
FLOP	4 x 500 000 x 64 = 128 MFLOP/epoxa, trafik 256 MB/epoxa -> AI 0,5
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2 (bo'laklar bo'yicha gradient)
Blok	qator bloklari 1 000 / 10 000 / 100 000
Vaqt chegarasi	200 epoxa (500 000 x 64) <= 60 s
Ma'lumot	sklearn make_classification, seed 18

QADAMLAR

1. (s1) Sof Python epoxa (ikki sikl, math.exp) 2 000 x 64 vs NumPy (X @ w, 1/(1+np.exp(-z)), X.T @ r) 50 000/500 000 x 32/64; ms/epoxa, GFLOP/s va GB/s, tezlanish; 20 epoxadan keyin loss va sklearn bilan koeffitsiyent farqi; NumPy'da sigmoid 4 ta oraliq massiv (500 000 x 4 bayt) trafigi.
2. (s2) @njit epoxa (qator bo'yicha: z_i, p_i, r_i va darhol gradientga qo'shish - X bir marta o'qiladi!) va prange qatorlar bo'yicha 1/2 ip (gradient 64 ta akkumulyator - har ip o'z massivida, so'ng yig'iladi); JIT vaqti; ms/epoxa, GB/s; log-loss prange reduktsiya 1 vs 2 ip.
3. (s3) Lokallik: X C-tartib (qator = mijoz) vs F-tartib (ustun = belgi) da GEMV va GEMV^T tezligi; qator bloklari 1 000/10 000/100 000 (blok x 64 x 4 bayt) bilan 'blok ichida ikkala GEMV' - X bir o'tishda; jadval + GB/s; float32 vs float64 trafik farqi.
4. (s4) Epoxani 4 usulda: Numba 1 ip (fused), prange 2 ip, NumPy BLAS (2 GEMV + sigmoid), torch (X @ w, torch.sigmoid, X.T @ r) 1/2 ip; 20 epoxadan keyin w maks farqi < 1e-4; jadval; qaysi variantda X necha marta o'qiladi.
5. (s5) torch epoxa 1 vs 2 ip (50 000/500 000), float32 vs bfloat16 X (xotira 2x kam - tezlashadimi? xato?) vs float64; sigmoid + loss eager (6 amal) vs F.binary_cross_entropy_with_logits (fused) vs Numba fused - GB/s; torch.compile yo'q - izoh.
6. (s6) Pool(1/2) ma'lumot parallelligi: X bo'laklari bo'yicha qisman gradient (10 bo'lak x 50 000, X fork orqali meros, natija 64 float + loss), chunksize 1/2/5, 20 epoxa (har epoxada Pool qayta ishlatiladi - yaratmang!); tezlanish; bu DDP all-reduce'ning CPU muqobili ekanini tushuntiring.

7. (s7) Amdahl: ketma-ket qism = w yangilash + loss log + early-stopping tekshiruvi (Python, har epoxa), parallel = prange epoxa; S(2) dan p, S(8)/S(1024); xotira chegarasi: GB/s o'lchangan vs reduktsiya chegarasi; Gustafson: 50 000 -> 500 000 -> 2 M mijoz (bloklab generatsiya, 512 MB - ehtiyot) ; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

20-variant. Murojaatlar dublikatini topish: n-gramm bitset'lari ustida Jaccard o'xshashligi (AND + popcount) o'rta

Soha: Davlat xizmatlari

Davlat xizmatlari portaliga kuniga minglab murojaat keladi; takroriy va nusxa ko'chirilgan murojaatlarni topish uchun har matn 3-gramm to'plamiga, so'ng 2 048 bitli bitset'ga (32 ta uint64 so'z, xesh bilan) aylantiriladi va barcha juftliklar uchun Jaccard ~ popcount(A & B) / popcount(A | B) hisoblanadi. 8 000 murojaat uchun 32 M juft x 32 so'z = 1 G bit amal - sof Python to'plamlar bilan bu daqiqalar, Numba bitset bilan soniya; bu butun sonli (bit) yadro, FLOP emas - unumdorlikni 'juft/s' va 'GB/s' da o'lchaysiz. Matnlar sintetik: 50 ta shablon + tasodifiy so'z almashtirishlar (seed 19), shuning uchun haqiqiy dublikat juftlar ma'lum va to'g'rilik tekshiriladi.

Yadro	bitset AND/OR + popcount barcha juftliklar; Jaccard >= 0,8 juftlarni sanash (reduktsiya)
O'lchamlar	murojaat 1 000 / 8 000, bitset 1 024 / 2 048 bit (16 / 32 uint64 so'z)
Amallar	8 000^2/2 = 32 M juft x 32 so'z x 3 amal ~ 3 G butun son amal
Iplar/jarayonlar	Numba 1/2, torch 1/2 (bit amallari uint8/int64), Pool 1/2
Blok	qator bloklari 100 / 500 / 2 000 (blok x 8 000 x 1 bayt natija)
Vaqt chegarasi	8 000 murojaat <= 3 s
Ma'lumot	50 shablon + so'z almashtirish, seed 19

QADAMLAR

1. (s1) Sof Python: to'plamlar (set & set) bilan Jaccard 1 000 murojaat (0,5 M juft) vs Python int bitset (bin(a & b).count('1')) vs NumPy uint8 bitset (np.bitwise_and + np.unpackbits/popcount jadvali, bloklab) 1 000/8 000; ms, M juft/s, tezlanish; dublikat juftlar soni haqiqiy bilan mos (to'g'rilik).
2. (s2) @njit bitset Jaccard (popcount: 'bit hiylasi' SWAR formulasi yoki 16 bitli jadval) ikki sikl va prange qatorlar bo'yicha 1/2 ip (ichki sikl j > i - uchburchak, yuk muvozanati!); JIT vaqti; M juft/s, GB/s; dublikatlar soni prange reduktsiya 1 vs 2 ip; prange schedule muvozanatini i tartibini aralashtirib tekshiring.
3. (s3) Lokallik: bitset'larni (n, 32) uint64 C-tartibda vs so'z-birinchi (32, n) tartibda; qator bloklari 100/500/2 000 (blok x 256 bayt keshda, ichki sikl barcha j bo'ylab); 1 024 vs 2 048 bit; popcount jadval (64 KB, keshda) vs SWAR (hisob) - qaysi biri tez; jadval.
4. (s4) Jaccard'ni 4 usulda: Numba 1 ip, prange 2 ip, NumPy bloklab (bitwise_and + popcount jadval), torch (bitwise_and int64 + popcount jadval orqali yoki unpackbits muqobili: uint8 ko'rinishida (n, 256) va 256 li jadval indekslash) 1/2 ip; muqobil 'GEMM hiylasi': 0/1 float32 matritsa (n x 2 048) uchun A @ A.T = kesishma soni - BLAS bilan; natijalar mosligi (dublikat to'plami tengligi).
5. (s5) torch 1 vs 2 ip (1 000/8 000), int64 bitset vs uint8 vs 0/1 float32 GEMM vs bfloat16 GEMM (kesishma sonlari butun - xato bormi?); Jaccard eager (and, or, popcount, popcount, bo'lish) vs |A|+|B|-|A&B| hiylasi bilan bitta popcount - GB/s.

6. (s6) Pool(1/2) qator bloklari bo'yicha (16 blok x 500 qator; bitset'lar fork orqali meros; natija - dublikat juftlar ro'yxati), chunksize 1/2/8; uchburchak ish taqsimoti tufayli muvozanat - bloklarni 'oldingi + oxirgi' juftlab bering; tezlanish, samaradorlik.
7. (s7) Amdahl: ketma-ket qism = matnlarni n-grammga va bitset'ga aylantirish (Python, 8 000 matn) + dublikat guruhlarini birlashtirish (union-find, Python), parallel = prange Jaccard; profil ulushlari va S(2) dan p; S(8)/S(1024); Gustafson: 1 000 -> 8 000 -> 30 000 murojaat (ekstrapolyatsiya); grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A3

Sun'iy intellekt tizimlarini optimallashtirish va unumdorligini baholash

Bajarish bo‘yicha qo‘llanma: 6 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A3
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A3
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu virtual laboratoriyada siz o'qitilgan neyron tarmoqni ishlab chiqarishga tayyorlash zanjirini to'liq bajarasiz: PyTorch modelini ONNX'ga eksport qilish, ONNX Runtime'da grafik optimallashtirish darajalari va paket o'lchamini o'lchash, PyTorch va ONNX Runtime'da INT8 kvantlash (dinamik va statik, turli kalibrlash usullari), so'ngra aniqlik-tezlik-hajm jadvali va Pareto tahlili. Real labdan farqi: ResNet-50 va ImageNet o'rniga sklearn digits (8x8) va kichik CNN/MLP, GPU/TensorRT/OpenVINO yo'q — ular o'rniga CPU muqobillari va «GPU'da bu qadam qanday bo'ladi» izohlari; sandbox CPU'da AVX/VNNI yo'qligi sababli INT8 tezlanishi kutilmaydi (bu o'zi muhim saboq).

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A3`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A3 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Etalon: PyTorch FP32 o'qitish va ONNX eksport
2. 2-qadam. ONNX Runtime: optimallashtirish darajalari, paket, oqimlar, Little qonuni
3. 3-qadam. PyTorch'da INT8 dinamik kvantlash (Linear)
4. 4-qadam. ONNX Runtime'da kvantlash: dinamik va statik (MinMax, Percentile, shovqin)
5. 5-qadam. Yakuniy jadval, Pareto tahlili va grafik
6. 6-qadam. Energiya bahosi, nusxalar rejasi va GPU/TensorRT yo'l xaritasi
7. Qo‘shimcha mashqlar (10)
8. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Etalon: PyTorch FP32 o'qitish va ONNX eksport

MAQSAD

Kichik CNN sklearn digits to'plamida 3 epoxa o'qitiladi, batch 1 kechikishi (p50/p95) o'lchanadi, so'ngra model `torch.onnx.export` bilan dinamik paket o'qi va opset 17 bilan eksport qilinadi. `onnx.checker` grafikni tekshiradi, ONNX Runtime chiqishi PyTorch bilan `np.allclose` orqali solishtiriladi. O'zgartirib ko'ring: epoxa sonini, `dynamic_axes` ni olib tashlab kirish shaklini, `do_constant_folding=False` da tugunlar sonini.

> **Kutilgan natija:** 3 epoxadan keyin ~94 % aniqlik, PyTorch eager p50 ~0,15 ms; ONNX grafikda 10 tugun (BatchNorm Conv'ga birlashgan), maksimal farq ~1e-6, «MOS».

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Etalon: kichik CNN'ni PyTorch'da o'qitish, ONNX'ga eksport qilish va moslikni
tekshirish
import warnings; warnings.filterwarnings("ignore")
import os, time, numpy as np, torch, torch.nn as nn, onnx, onnxruntime as ort
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); np.random.seed(0)
torch.set_num_threads(2)                  # sandbox'da 2 yadro bor
torch.backends.nnpack.set_flags(False)    # eski CPU'da NNPACK ogohlantirishini o'chiramiz

# --- Ma'lumot: sklearn digits — 1797 ta 8x8 kulrang rasm, 10 sinf (ImageNet o'rniga, CPU
uchun)
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype(np.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Xtr, Xte, ytr, yte = map(torch.tensor, (Xtr, Xte, ytr, yte))

# --- Model: ResNet-50 o'rniga kichik CNN (Conv-BN-ReLU-Pool x2 + 2 ta Linear)
class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(16 * 2 * 2, 32), nn.ReLU(),
nn.Linear(32, 10))

    def forward(self, x):
        return self.tasnif(self.belgilar(x))

def oqit(model, epoxa=3, lr=3e-3):
    """Oddiy o'qitish sikli: Adam, paket 64, har epoxadan keyin test aniqligi."""
    opt = torch.optim.Adam(model.parameters(), lr); lossf = nn.CrossEntropyLoss()
    for ep in range(epoxa):
        model.train(); perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), 64):
            idx = perm[i:i + 64]
            opt.zero_grad(); loss = lossf(model(Xtr[idx]), ytr[idx]); loss.backward();
opt.step()
        model.eval()
        with torch.inference_mode():
```

```
            acc = (model(Xte).argmax(1) == yte).float().mean().item()
        print(f"  epoxa {ep + 1}: loss {loss.item():.3f}   test aniqlik {acc * 100:.1f} %")
    return acc

print("=== PyTorch FP32 etalon ===")
model = KichikCNN()
t0 = time.perf_counter(); acc_fp32 = oqit(model)
print(f"O'qitish vaqti: {time.perf_counter() - t0:.1f} s")
print(f"Parametrlar soni: {sum(p.numel() for p in model.parameters())}")

# Latency: batch 1, 300 takror (avval 30 ta isinish)
x1 = Xte[:1]
with torch.inference_mode():
    for _ in range(30): model(x1)
    t = []
    for _ in range(300):
        a = time.perf_counter(); model(x1); t.append(time.perf_counter() - a)
t = np.array(t) * 1e3
print(f"PyTorch eager, batch 1: p50 {np.percentile(t, 50):.3f} ms   p95 {np.percentile(t,
95):.3f} ms")

print("\n=== ONNX eksport ===")
# dynamic_axes — paket o'lchami o'zgaruvchan (keyingi qadamlarda batch 1/16/64 bilan sinaymiz)
# dynamo=False — TorchScript asosidagi eksport (sandbox'da onnxscript o'rnatilmagan)
torch.onnx.export(model, x1, "cnn_fp32.onnx", input_names=["input"], output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
                  opset_version=17, do_constant_folding=True, dynamo=False)
m = onnx.load("cnn_fp32.onnx"); onnx.checker.check_model(m)
print("Opset:", m.opset_import[0].version, "  tugunlar soni:", len(m.graph.node))
print("Operatorlar:", [n.op_type for n in m.graph.node])
print(f"Fayl hajmi: {os.path.getsize('cnn_fp32.onnx') / 1024:.1f} KB")
print("Kirish shakli:", [d.dim_param or d.dim_value for d in
m.graph.input[0].type.tensor_type.shape.dim])

# Moslik: PyTorch va ONNX Runtime chiqishlari bir xilmi?
sess = ort.InferenceSession("cnn_fp32.onnx", providers=["CPUExecutionProvider"])
with torch.inference_mode():
    ref = model(Xte[:64]).numpy()
out = sess.run(None, {"input": Xte[:64].numpy()})[0]
farq = np.abs(ref - out).max()
mos = np.allclose(ref, out, rtol=1e-3, atol=1e-4)
print(f"PyTorch vs ORT maksimal farq: {farq:.2e}  -> {'MOS' if mos else 'MOS EMAS'}")
acc_ort = (sess.run(None, {"input": Xte.numpy()})[0].argmax(1) == yte.numpy()).mean()
print(f"Test aniqlik: PyTorch {acc_fp32 * 100:.1f} %   ORT {acc_ort * 100:.1f} %")
print("\nIzoh: eval rejimida BatchNorm eksport paytida Conv ichiga birlashtirildi — ONNX
grafikda BN tuguni yo'q.")
print("GPU bo'lsa: xuddi shu fayl TensorRT (trtexec --onnx=cnn_fp32.onnx) va ORT
CUDAExecutionProvider'da ishlaydi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== PyTorch FP32 etalon ===`, `=== ONNX eksport ===`, `MOS`

TAJRIBA QILIB KO‘RING

- `model.eval()` ni eksportdan oldin olib tashlang (train rejimida) — farq va tugunlar qanday o'zgaradi?
- Kanallar sonini 8/16 dan 32/64 ga oshiring: parametrlar, hajm va latency qanday o'sadi?
- `opset_version=13` bilan eksport qiling va operatorlar ro'yxatini solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. ONNX Runtime: optimallashtirish darajalari, paket, oqimlar, Little qonuni

MAQSAD

ONNX Runtime'ning 4 ta grafik optimallashtirish darajasi (DISABLE_ALL, BASIC, EXTENDED, ALL) batch 1 da o'lchanadi va optimallashtirilgan grafikdagi tugunlar soni sanaladi; keyin paket 1/16/64/256 uchun throughput va Little qonuni (N = X·R), intra_op oqimlar 1 va 2 taqqoslanadi (GPU parallelligi o'rniga). Oxirida 2000 so'rov/s talabi uchun nusxalar soni rejalashtiriladi.

> **Kutilgan natija:** ORT ALL PyTorch eager'dan 6–9 marta tez; EXTENDED/ALL da tugunlar 10 → 7–8; batch 256 da throughput 10+ marta yuqori, latency ham oshadi; 2 oqim ~1,9x.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. ONNX Runtime: grafik optimallashtirish darajalari, paket o'lchami, oqimlar soni va
Little qonuni
import warnings; warnings.filterwarnings("ignore")
import time, math, numpy as np, pandas as pd, torch, torch.nn as nn, onnx, onnxruntime as ort

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# Latency vaznlarga bog'liq emas, shuning uchun bu qadamda model o'qitilmaydi — faqat
arxitektura.
class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
10))
    def forward(self, x):
        return self.tasnif(self.belgilar(x))

model = KichikCNN().eval()
x1 = torch.randn(1, 1, 8, 8)
torch.onnx.export(model, x1, "cnn_fp32.onnx", input_names=["input"], output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)

def olch(fn, takror=300, isinish=30):
    """fn() ni takror marta chaqirib, kechikish taqsimotini (ms) qaytaradi."""
    for _ in range(isinish): fn()
    t = []
    for _ in range(takror):
        a = time.perf_counter(); fn(); t.append(time.perf_counter() - a)
    t = np.array(t) * 1e3
    return dict(p50=np.percentile(t, 50), p95=np.percentile(t, 95), p99=np.percentile(t, 99),
mean=t.mean())

def ort_sess(opt_level, threads=2, saqla=None):
    so = ort.SessionOptions()
    so.graph_optimization_level = opt_level
    so.intra_op_num_threads = threads
    so.log_severity_level = 3                              # faqat xatolar
```

```
    if saqla: so.optimized_model_filepath = saqla        # optimallashtirilgan grafikni faylga
yozib tugunlarni sanaymiz
    return ort.InferenceSession("cnn_fp32.onnx", so, providers=["CPUExecutionProvider"])

print("=== ONNX Runtime o'lchovlari ===")
print("Mavjud provayderlar:", ort.get_available_providers(), "(GPU yo'q -> faqat CPU)")
L = ort.GraphOptimizationLevel
qatorlar = []
# 0) PyTorch eager etalon
with torch.inference_mode():
    r = olch(lambda: model(x1))
qatorlar.append(dict(dvigatel="PyTorch eager", opt="-", tugun="-", **{k: round(v, 3) for k, v
in r.items()}))
# 1) ORT 4 ta optimallashtirish darajasi, batch 1
xn = x1.numpy()
for nomi, lvl in (("DISABLE_ALL", L.ORT_DISABLE_ALL), ("BASIC", L.ORT_ENABLE_BASIC),
                  ("EXTENDED", L.ORT_ENABLE_EXTENDED), ("ALL", L.ORT_ENABLE_ALL)):
    s = ort_sess(lvl, saqla=f"opt_{nomi}.onnx")
    tugun = len(onnx.load(f"opt_{nomi}.onnx").graph.node)
    r = olch(lambda: s.run(None, {"input": xn}))
    qatorlar.append(dict(dvigatel="ORT CPU", opt=nomi, tugun=tugun, **{k: round(v, 3) for k, v
in r.items()}))
df = pd.DataFrame(qatorlar)
print("\nBatch 1, 2 oqim (ms):"); print(df.to_string(index=False))
tez = df.loc[0, "p50"] / df.loc[4, "p50"]
print(f"ORT ALL PyTorch eager'dan {tez:.1f} marta tez (Python xarajati yo'q, grafik oldindan
tuzilgan).")

# 2) Paket o'lchami va throughput (Little qonuni: tizimdagi so'rovlar = throughput x latency)
print("\n--- Paket o'lchami (ORT ALL, 2 oqim) ---")
s = ort_sess(L.ORT_ENABLE_ALL)
qat = []
for b in (1, 16, 64, 256):
    xb = np.random.rand(b, 1, 8, 8).astype(np.float32)
    r = olch(lambda: s.run(None, {"input": xb}), takror=100)
    thr = b / (r["mean"] / 1e3)
    qat.append(dict(batch=b, p50_ms=round(r["p50"], 3), p99_ms=round(r["p99"], 3),
throughput=round(thr),
                    little_N=round(thr * r["mean"] / 1e3, 1)))
dfb = pd.DataFrame(qat); print(dfb.to_string(index=False))
print("little_N = throughput x latency = paketdagi so'rovlar soni (Little qonuni sinxron siklda
aynan batch'ga teng).")

# 3) Oqimlar soni: 1 va 2 (GPU parallelligi o'rniga CPU parallelligi)
print("\n--- intra_op oqimlar: 1 vs 2 (batch 256) ---")
xb = np.random.rand(256, 1, 8, 8).astype(np.float32)
nat = {}
for th in (1, 2):
    s = ort_sess(L.ORT_ENABLE_ALL, threads=th)
    nat[th] = olch(lambda: s.run(None, {"input": xb}), takror=100)["mean"]
    print(f"  {th} oqim: o'rtacha {nat[th]:.3f} ms  -> {256 / nat[th] * 1e3:,.0f} rasm/s")
print(f"  Tezlanish 2 oqimda: {nat[1] / nat[2]:.2f}x (ideal 2.0x; farq — sinxronizatsiya va
kichik model)")

# 4) Rejalashtirish: 2000 so'rov/s va p99 <= 2 ms talabi uchun nechta parallel nusxa kerak?
lam, p99 = 2000, df.loc[4, "p99"]
print(f"\nLittle qonuni bo'yicha reja ({lam} so'rov/s, p99 bo'yicha):")
for nomi, lat in (("bizning CNN", p99), ("ResNet-50 CPU (~20 ms)", 20.0), ("ResNet-50 T4 INT8
(~0.4 ms/rasm)", 0.4)):
    N = lam * lat / 1e3
    print(f"  {nomi:<34} {lam} x {lat:.3f} ms = {N:6.1f} parallel ishlov -> kamida
{math.ceil(N)} nusxa/oqim")
```

```
print("GPU'da: ORT CUDAExecutionProvider / TensorRT'da batch 32 da throughput 5-10 marta
o'sadi, latency 10+ marta oshadi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== ONNX Runtime o'lchovlari ===`, `Paket o'lchami`, `Little qonuni`

TAJRIBA QILIB KO‘RING

- `takror` ni 300 dan 2000 ga oshiring — p99 barqarorlashadimi?
- `inter_op_num_threads` va `execution_mode = ORT_PARALLEL` qo'shib ko'ring.
- Little qonuni rejasiga p50 o'rniga p99 ni emas, o'rtachani qo'ysangiz nusxalar soni qanday o'zgaradi va nega p99 to'g'riroq?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. PyTorch'da INT8 dinamik kvantlash (Linear)

MAQSAD

MLP (Linear og'ir model) `torch.ao.quantization.quantize_dynamic` bilan INT8 ga o'tkaziladi: kvantlash formulasi q = round(w/s) qo'lda tekshiriladi, hajm (state_dict baytlari), aniqlik va batch 1/256 kechikishi FP32 bilan solishtiriladi. Sandbox CPU'da AVX yo'qligi sababli `qnnpack` dvigateli majburan tanlanadi — real serverda bu qator kerak emas.

> **Kutilgan natija:** Hajm ~3,8 marta kichik (335 → 89 KB), aniqlik o'zgarmaydi (±0,3 p.p.), tezlik bu CPU'da 0,7–1,0x (VNNI yo'q).

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. PyTorch'da dinamik INT8 kvantlash (Linear qatlamlar): hajm, aniqlik, tezlik
import warnings; warnings.filterwarnings("ignore")
import io, time, numpy as np, pandas as pd, torch, torch.nn as nn
from torch.ao.quantization import quantize_dynamic
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)
# Sandbox CPU'da AVX/VNNI yo'q: fbgemm/onednn dvigatellari ishlamaydi, qnnpack (ARM/eski x86
uchun) ishlaydi.
# Haqiqiy serverda (AVX-512 VNNI) bu qator kerak emas — 'x86' dvigateli avtomatik tanlanadi.
torch.backends.quantized.engine = "qnnpack"
print("Kvantlash dvigateli:", torch.backends.quantized.engine, "| mavjud:",
torch.backends.quantized.supported_engines)

X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype(np.float32)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Xtr, Xte, ytr, yte = map(torch.tensor, (Xtr, Xte, ytr, yte))

# MLP — Linear (MatMul) og'ir model: dinamik kvantlash aynan shunday modellarga mo'ljallangan
(BERT, LSTM kabi)
model = nn.Sequential(nn.Linear(64, 256), nn.ReLU(), nn.Linear(256, 256), nn.ReLU(),
nn.Linear(256, 10))
opt = torch.optim.Adam(model.parameters(), 2e-3); lossf = nn.CrossEntropyLoss()
for ep in range(3):
    model.train(); perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lossf(model(Xtr[idx]), ytr[idx]).backward();
opt.step()
model.eval()

def aniqlik(m):
    with torch.inference_mode():
        return (m(Xte).argmax(1) == yte).float().mean().item() * 100

def hajm_kb(m):
    b = io.BytesIO(); torch.save(m.state_dict(), b); return b.tell() / 1024

def latency(m, x, takror=300):
    with torch.inference_mode():
        for _ in range(30): m(x)
        t = []
```

```
        for _ in range(takror):
            a = time.perf_counter(); m(x); t.append(time.perf_counter() - a)
    return np.percentile(np.array(t) * 1e3, 50)

print("\n=== PyTorch INT8 dinamik kvantlash ===")
qmodel = quantize_dynamic(model, {nn.Linear}, dtype=torch.qint8)   # vaznlar INT8,
faollashuvlar ish vaqtida kvantlanadi
print("Kvantlangan model tuzilmasi:")
for nomi, mod in qmodel.named_children():
    print(f"  {nomi}: {repr(mod)}")

# Kvantlash formulasini qo'lda tekshiramiz: q = round(w / s), s = max|w| / 127 (simmetrik, per-
tensor)
w = model[0].weight.detach()
s = w.abs().max() / 127
q = torch.round(w / s).clamp(-128, 127)
print(f"\n1-qatlam vazni: diapazon [{w.min():.3f}, {w.max():.3f}]  masshtab s = {s:.5f}")
print(f"  INT8 -> FP32 qayta tiklash xatosi: max {((q * s) - w).abs().max():.5f}, o'rtacha {((q
* s) - w).abs().mean():.6f}")
print(f"  Noyob qiymatlar: FP32 {w.unique().numel()} ta -> INT8 {q.unique().numel()} ta")

x1, xb = Xte[:1], Xte[:256]
rows = []
for nomi, m in (("FP32", model), ("INT8 dinamik", qmodel)):
    rows.append(dict(model=nomi, hajm_KB=round(hajm_kb(m), 1), aniqlik=round(aniqlik(m), 2),
                     p50_batch1_ms=round(latency(m, x1), 3), p50_batch256_ms=round(latency(m,
xb, 100), 3)))
df = pd.DataFrame(rows); print("\n" + df.to_string(index=False))
print(f"\nHajm {df.hajm_KB[0] / df.hajm_KB[1]:.1f} marta kichraydi (nazariy chegara 4x; bias va
masshtablar FP32 da qoladi).")
print(f"Aniqlik farqi: {df.aniqlik[1] - df.aniqlik[0]:+.2f} p.p.   Tezlik (batch 256):
{df.p50_batch256_ms[0] / df.p50_batch256_ms[1]:.2f}x")
print("Izoh: bu CPU'da VNNI buyruqlari yo'q, shuning uchun INT8 tezlanishi kichik yoki hatto
salbiy bo'lishi mumkin —")
print("  bu A3 matnidagi 5-xato ('VNNI yo'qligini tekshirmaslik') ning jonli namunasi. VNNI/AMX
CPU'da 2-3x kutiladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== PyTorch INT8 dinamik kvantlash ===`, `DynamicQuantizedLinear`, `Hajm`

TAJRIBA QILIB KO‘RING

- Qatlam kengligini 256 dan 1024 ga oshiring — hajm nisbati 4x ga yaqinlashadimi, tezlik nisbati o'zgaradimi?
- `dtype=torch.float16` bilan `quantize_dynamic` chaqirib ko'ring (CPU'da qo'llab-quvvatlanadimi?).
- Per-channel masshtab bilan qo'lda kvantlab, qayta tiklash xatosini per-tensor bilan solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. ONNX Runtime'da kvantlash: dinamik va statik (MinMax, Percentile, shovqin)

MAQSAD

CNN ONNX modeli `quant_pre_process` dan o'tkazilib, `quantize_dynamic` va `quantize_static` (QDQ format) bilan kvantlanadi: per-channel MinMax, per-channel Percentile, per-tensor MinMax va ataylab noto'g'ri — tasodifiy shovqin bilan kalibrlash (A3 matnidagi 3-xato). Har variant uchun hajm, aniqlik, p50, throughput va Q/DQ tugunlar soni jadvalga yig'iladi.

> **Kutilgan natija:** Hajm 17 → 11–13 KB; real kalibrlash aniqlikni saqlaydi (±0,3 p.p.), shovqinli kalibrlash 1 p.p. atrofida yo'qotadi; INT8 bu CPU'da tezroq emas.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. ONNX Runtime'da kvantlash: dinamik va statik (MinMax, Percentile, shovqin bilan
kalibrlash)
import warnings; warnings.filterwarnings("ignore")
import io, os, time, logging, contextlib, numpy as np, pandas as pd, torch, torch.nn as nn,
onnx, onnxruntime as ort
from onnxruntime.quantization import (quantize_dynamic, quantize_static, QuantType,
QuantFormat,
                                      CalibrationDataReader, CalibrationMethod)
from onnxruntime.quantization.shape_inference import quant_pre_process
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

logging.disable(logging.WARNING)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)

X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype(np.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Ttr, Tte, Tytr = torch.tensor(Xtr), torch.tensor(Xte), torch.tensor(ytr)

class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
10))
    def forward(self, x):
        return self.tasnif(self.belgilar(x))

# O'qitish (2 epoxa) va eksport
model = KichikCNN(); opt = torch.optim.Adam(model.parameters(), 3e-3); lossf =
nn.CrossEntropyLoss()
for ep in range(3):
    model.train(); perm = torch.randperm(len(Ttr))
    for i in range(0, len(Ttr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lossf(model(Ttr[idx]), Tytr[idx]).backward();
opt.step()
model.eval()
```

```
torch.onnx.export(model, Tte[:1], "cnn_fp32.onnx", input_names=["input"], output_names=
["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)
quant_pre_process("cnn_fp32.onnx", "cnn_prep.onnx")       # shakl inferensi + optimallashtirish
(kvantlash sifati uchun)

class Reader(CalibrationDataReader):
    """Kalibrlash to'plami: ORT har safar bitta {kirish: massiv} lug'at so'raydi."""
    def __init__(self, arr, paket=32):
        self.it = iter([{"input": arr[i:i + paket]} for i in range(0, len(arr), paket)])
    def get_next(self):
        return next(self.it, None)

def statik(chiqish, kalib, usul, per_channel=True):
    with contextlib.redirect_stdout(io.StringIO()):               # kalibrator loglarini
yashiramiz
        quantize_static("cnn_prep.onnx", chiqish, Reader(kalib), quant_format=QuantFormat.QDQ,
                        activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8,
                        per_channel=per_channel, calibrate_method=usul)

print("=== ONNX Runtime kvantlash ===")
kalib = Xtr[:320]                                                  # 320 ta real namuna
shovqin = (np.random.rand(320, 1, 8, 8) * 8).astype(np.float32)     # 3-xato: real taqsimotga
mos kelmaydigan shovqin (0..8, real 0..1)
quantize_dynamic("cnn_prep.onnx", "cnn_dyn.onnx", weight_type=QuantType.QInt8)
statik("cnn_s_minmax.onnx", kalib, CalibrationMethod.MinMax)
statik("cnn_s_pct.onnx", kalib, CalibrationMethod.Percentile)
statik("cnn_s_pt.onnx", kalib, CalibrationMethod.MinMax, per_channel=False)
statik("cnn_s_noise.onnx", shovqin, CalibrationMethod.MinMax)

def baho(fayl):
    so = ort.SessionOptions(); so.intra_op_num_threads = 2
    s = ort.InferenceSession(fayl, so, providers=["CPUExecutionProvider"])
    acc = (s.run(None, {"input": Xte})[0].argmax(1) == yte).mean() * 100
    x1 = Xte[:1]
    for _ in range(30): s.run(None, {"input": x1})
    t = []
    for _ in range(300):
        a = time.perf_counter(); s.run(None, {"input": x1}); t.append(time.perf_counter() - a)
    xb = Xte[:256]
    for _ in range(5): s.run(None, {"input": xb})
    a = time.perf_counter()
    for _ in range(20): s.run(None, {"input": xb})
    thr = 256 * 20 / (time.perf_counter() - a)
    ops = [n.op_type for n in onnx.load(fayl).graph.node]
    return dict(hajm_KB=round(os.path.getsize(fayl) / 1024, 1), aniqlik=round(acc, 2),
                p50_ms=round(np.percentile(np.array(t) * 1e3, 50), 3), throughput=round(thr),
                QDQ=sum(o in ("QuantizeLinear", "DequantizeLinear", "DynamicQuantizeLinear")
for o in ops))

rows = []
for nomi, f in (("FP32", "cnn_prep.onnx"), ("Dinamik INT8", "cnn_dyn.onnx"),
                ("Statik, per-channel, MinMax", "cnn_s_minmax.onnx"),
                ("Statik, per-channel, Percentile", "cnn_s_pct.onnx"),
                ("Statik, per-tensor, MinMax", "cnn_s_pt.onnx"),
                ("Statik, SHOVQIN bilan kalibrlash", "cnn_s_noise.onnx")):
    rows.append(dict(konfiguratsiya=nomi, **baho(f)))
df = pd.DataFrame(rows)
df["farq_pp"] = (df.aniqlik - df.aniqlik[0]).round(2)
print(df.to_string(index=False))
print("\nOperatorlar (dinamik):", [n.op_type for n in onnx.load("cnn_dyn.onnx").graph.node]
[:9], "...")
```

```
print("Operatorlar (statik QDQ):", [n.op_type for n in
onnx.load("cnn_s_minmax.onnx").graph.node][:8], "...")
print("\nXulosa: statik kvantlash kalibrlash to'plamiga bog'liq — noto'g'ri taqsimotli shovqin
bilan kalibrlangan model aniqlikni yo'qotadi;")
print("  per-tensor vs per-channel farqi kichik modelda sezilmasligi mumkin, ResNet-50 da 1-2
p.p. beradi.")
print("  INT8 tezlanishi bu CPU'da (VNNI yo'q) kutilmaydi; VNNI/AMX serverda statik INT8 2-4x
beradi, GPU'da TensorRT INT8 ishlatiladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== ONNX Runtime kvantlash ===`, `SHOVQIN`, `Operatorlar (statik QDQ)`

TAJRIBA QILIB KO‘RING

- Kalibrlash to'plamini 320 dan 32 va 1000 ga o'zgartiring — aniqlik qanday?
- `CalibrationMethod.Entropy` qo'shing va uch usulni solishtiring.
- `nodes_to_exclude` bilan birinchi Conv'ni FP32 da qoldiring (sezgir qatlam) — aniqlik tiklanadimi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Yakuniy jadval, Pareto tahlili va grafik

MAQSAD

6 ta konfiguratsiya (PyTorch FP32, PyTorch INT8 dinamik, ORT FP32, ORT INT8 dinamik, ORT statik MinMax/Percentile) bitta jadvalga yig'iladi: hajm, aniqlik, p50 (batch 1), throughput (batch 256), tezlanish. Pareto-optimal nuqtalar (latency kichik, aniqlik katta) aniqlanadi va ikki o'qli grafik chiziladi: latency–aniqlik va hajm–throughput.

> **Kutilgan natija:** ORT variantlari PyTorch'dan 4–7 marta tez; Pareto chegarasida ORT FP32 va statik Percentile; grafikda qizil nuqtalar — Pareto.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Yakuniy jadval: aniqlik vs tezlik vs hajm, Pareto tahlili va grafik
import warnings; warnings.filterwarnings("ignore")
import io, os, time, logging, contextlib, numpy as np, pandas as pd, torch, torch.nn as nn,
onnxruntime as ort
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from torch.ao.quantization import quantize_dynamic as torch_qdyn
from onnxruntime.quantization import (quantize_dynamic, quantize_static, QuantType,
QuantFormat,
                                      CalibrationDataReader, CalibrationMethod)
from onnxruntime.quantization.shape_inference import quant_pre_process
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

logging.disable(logging.WARNING)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)
torch.backends.quantized.engine = "qnnpack"      # sandbox CPU'da AVX yo'q (3-qadamga qarang)

X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype(np.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Ttr, Tte, Tytr, Tyte = torch.tensor(Xtr), torch.tensor(Xte), torch.tensor(ytr),
torch.tensor(yte)

class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
10))
    def forward(self, x):
        return self.tasnif(self.belgilar(x))

model = KichikCNN(); opt = torch.optim.Adam(model.parameters(), 3e-3); lossf =
nn.CrossEntropyLoss()
for ep in range(3):
    model.train(); perm = torch.randperm(len(Ttr))
    for i in range(0, len(Ttr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lossf(model(Ttr[idx]), Tytr[idx]).backward();
opt.step()
model.eval()
```

```
torch.onnx.export(model, Tte[:1], "cnn_fp32.onnx", input_names=["input"], output_names=
["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)
quant_pre_process("cnn_fp32.onnx", "cnn_prep.onnx")

class Reader(CalibrationDataReader):
    def __init__(self, arr, paket=32):
        self.it = iter([{"input": arr[i:i + paket]} for i in range(0, len(arr), paket)])
    def get_next(self):
        return next(self.it, None)

quantize_dynamic("cnn_prep.onnx", "cnn_dyn.onnx", weight_type=QuantType.QInt8)
for nomi, usul in (("minmax", CalibrationMethod.MinMax), ("pct",
CalibrationMethod.Percentile)):
    with contextlib.redirect_stdout(io.StringIO()):
        quantize_static("cnn_prep.onnx", f"cnn_s_{nomi}.onnx", Reader(Xtr[:320]),
quant_format=QuantFormat.QDQ,
                        activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8,
per_channel=True, calibrate_method=usul)

def olch(fn, x1, xb):
    """p50 latency (batch 1, ms) va throughput (batch 256, rasm/s)."""
    for _ in range(30): fn(x1)
    t = []
    for _ in range(300):
        a = time.perf_counter(); fn(x1); t.append(time.perf_counter() - a)
    for _ in range(5): fn(xb)
    a = time.perf_counter()
    for _ in range(20): fn(xb)
    return np.percentile(np.array(t) * 1e3, 50), 256 * 20 / (time.perf_counter() - a)

def torch_hajm(m):
    b = io.BytesIO(); torch.save(m.state_dict(), b); return b.tell() / 1024

rows = []
# PyTorch variantlari
qmodel = torch_qdyn(model, {nn.Linear}, dtype=torch.qint8)
for nomi, m in (("PyTorch FP32", model), ("PyTorch INT8 dinamik (Linear)", qmodel)):
    with torch.inference_mode():
        acc = (m(Tte).argmax(1) == Tyte).float().mean().item() * 100
        p50, thr = olch(lambda x: m(x), Tte[:1], Tte[:256])
    rows.append(dict(konfiguratsiya=nomi, hajm_KB=round(torch_hajm(m), 1), aniqlik=round(acc,
2), p50_ms=round(p50, 3), throughput=round(thr)))
# ONNX Runtime variantlari
for nomi, f in (("ORT FP32 (ALL)", "cnn_prep.onnx"), ("ORT INT8 dinamik", "cnn_dyn.onnx"),
                ("ORT INT8 statik MinMax", "cnn_s_minmax.onnx"), ("ORT INT8 statik Percentile",
"cnn_s_pct.onnx")):
    so = ort.SessionOptions(); so.intra_op_num_threads = 2
    s = ort.InferenceSession(f, so, providers=["CPUExecutionProvider"])
    acc = (s.run(None, {"input": Xte})[0].argmax(1) == yte).mean() * 100
    p50, thr = olch(lambda x: s.run(None, {"input": x}), Xte[:1], Xte[:256])
    rows.append(dict(konfiguratsiya=nomi, hajm_KB=round(os.path.getsize(f) / 1024, 1),
aniqlik=round(acc, 2), p50_ms=round(p50, 3), throughput=round(thr)))

df = pd.DataFrame(rows)
df["tezlanish"] = (df.p50_ms[0] / df.p50_ms).round(2)
print("=== Yakuniy jadval ===")
print(df.to_string(index=False))

# Pareto: latency kichik, aniqlik katta — hech kim ikkala ko'rsatkichda ham yaxshiroq bo'lmagan
nuqtalar
def pareto(df):
```

```
    opt = []
    for i, r in df.iterrows():
        ustun = ((df.p50_ms <= r.p50_ms) & (df.aniqlik >= r.aniqlik) & ((df.p50_ms < r.p50_ms)
| (df.aniqlik > r.aniqlik))).any()
        if not ustun: opt.append(r.konfiguratsiya)
    return opt
print("\n=== Pareto tahlili ===")
par = pareto(df)
print("Pareto-optimal (latency-aniqlik):", par)
eng_tez = df.loc[df.p50_ms.idxmin()]; eng_kichik = df.loc[df.hajm_KB.idxmin()]
print(f"Eng tez: {eng_tez.konfiguratsiya} ({eng_tez.p50_ms} ms, {eng_tez.tezlanish}x);  eng
kichik: {eng_kichik.konfiguratsiya} ({eng_kichik.hajm_KB} KB)")
print("Tavsiya: tibbiy tasniflash (aniqlik birinchi) -> ORT FP32; chekka qurilma (hajm/energiya
birinchi) -> statik INT8;")
print("  server CPU'da VNNI bo'lsa statik INT8 ham tezroq bo'ladi; GPU'da TensorRT FP16 odatda
Pareto chegarasida.")

# Grafik: ikki o'q — latency vs aniqlik, hajm vs throughput
fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
for _, r in df.iterrows():
    rang = "tab:red" if r.konfiguratsiya in par else "tab:blue"
    ax[0].scatter(r.p50_ms, r.aniqlik, s=70, color=rang); ax[0].annotate(r.konfiguratsiya,
(r.p50_ms, r.aniqlik), fontsize=7, xytext=(4, 3), textcoords="offset points")
    ax[1].scatter(r.hajm_KB, r.throughput, s=70, color=rang); ax[1].annotate(r.konfiguratsiya,
(r.hajm_KB, r.throughput), fontsize=7, xytext=(4, 3), textcoords="offset points")
ax[0].set_xlabel("p50 latency, ms (batch 1)"); ax[0].set_ylabel("Test aniqlik, %");
ax[0].set_title("Latency - aniqlik (qizil = Pareto)"); ax[0].set_xscale("log");
ax[0].grid(alpha=.3)
ax[1].set_xlabel("Model hajmi, KB"); ax[1].set_ylabel("Throughput, rasm/s (batch 256)");
ax[1].set_title("Hajm - throughput"); ax[1].grid(alpha=.3)
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Yakuniy jadval ===`, `=== Pareto tahlili ===`, `Pareto-optimal`

TAJRIBA QILIB KO‘RING

- Pareto mezonini (hajm, aniqlik) juftligiga o'zgartiring — qaysi konfiguratsiyalar qoladi?
- Uchinchi o'q sifatida hajmni nuqta kattaligi bilan ko'rsating.
- O'qitish epoxasini 1 ga tushiring: aniqlik farqlari kvantlash tufaylimi yoki o'qitish shovqinimi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Energiya bahosi, nusxalar rejasi va GPU/TensorRT yo'l xaritasi

MAQSAD

RAPL/nvidia-smi yo'qligi sababli quvvat MODEL bilan baholanadi: P = P_bosh + (P_to'liq − P_bosh)·CPU_bandlik, bandlik `time.process_time()/devor vaqti` dan olinadi. FP32 va INT8 dinamik, 1 va 2 oqim, batch 1 va 64 uchun µJ/inferens (to'liq va dinamik) hamda inferens/s/W hisoblanadi; Little qonuni bilan 5000 so'rov/s uchun nusxalar va quvvat; oxirida GPU'da TensorRT FP16/INT8 va quvvat o'lchash qadamlari (buyruqlar va kutilgan raqamlar).

> **Kutilgan natija:** Batch 64 da energiya batch 1 dan 5–15 marta kam; 2 oqim batch 1 da energiya ortadi (bandlik 2x, tezlik oshmaydi); ustunli grafik.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Energiya bahosi (CPU vaqti asosida), nusxalar rejasi va GPU/TensorRT yo'l xaritasi
import warnings; warnings.filterwarnings("ignore")
import time, math, logging, numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as
ort
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from onnxruntime.quantization import quantize_dynamic, QuantType
from onnxruntime.quantization.shape_inference import quant_pre_process

logging.disable(logging.WARNING)
torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# Energiya latency kabi vaznlarga bog'liq emas — modelni o'qitmaymiz
class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
10))
    def forward(self, x):
        return self.tasnif(self.belgilar(x))

model = KichikCNN().eval(); x1 = torch.randn(1, 1, 8, 8)
torch.onnx.export(model, x1, "cnn_fp32.onnx", input_names=["input"], output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)
quant_pre_process("cnn_fp32.onnx", "cnn_prep.onnx")
quantize_dynamic("cnn_prep.onnx", "cnn_dyn.onnx", weight_type=QuantType.QInt8)

# --- Quvvat modeli. Sandbox'da RAPL/nvidia-smi yo'q, shuning uchun quvvatni MODEL bilan
baholaymiz:
#     P = P_bosh + (P_tolik - P_bosh) * CPU_bandlik.  Real labda bu qiymatlar RAPL (CPU) yoki
NVML (GPU) dan o'qiladi.
P_TOLIK_W = 15.0      # 2 yadro to'liq yuklangan server CPU ulushi (taxmin; o'zgartirib
ko'ring)
P_BOSH_W = 5.0        # bo'sh holat

def energiya(fn, batch, takror):
    for _ in range(20): fn()
```

```
    w0, c0 = time.perf_counter(), time.process_time()      # devor vaqti va CPU vaqti (barcha
oqimlar yig'indisi)
    for _ in range(takror): fn()
    wall, cpu = time.perf_counter() - w0, time.process_time() - c0
    bandlik = min(cpu / wall / 2, 1.0)                        # 2 yadroga nisbatan
    P = P_BOSH_W + (P_TOLIK_W - P_BOSH_W) * bandlik
    n = batch * takror
    return dict(latency_ms=wall / takror * 1e3, throughput=n / wall, cpu_bandlik=bandlik * 100,
                P_W=P, uJ_tolik=P * wall / n * 1e6, uJ_dinamik=(P - P_BOSH_W) * wall / n * 1e6,
inf_s_W=n / wall / P)

print("=== Energiya bahosi ===")
rows = []
for nomi, fayl in (("ORT FP32", "cnn_prep.onnx"), ("ORT INT8 dinamik", "cnn_dyn.onnx")):
    for th in (1, 2):
        so = ort.SessionOptions(); so.intra_op_num_threads = th
        s = ort.InferenceSession(fayl, so, providers=["CPUExecutionProvider"])
        for b in (1, 64):
            xb = np.random.rand(b, 1, 8, 8).astype(np.float32)
            r = energiya(lambda: s.run(None, {"input": xb}), b, 200 if b == 1 else 60)
            rows.append(dict(konfiguratsiya=nomi, oqim=th, batch=b, **{k: round(v, 3 if k ==
"latency_ms" else 1) for k, v in r.items()}))
df = pd.DataFrame(rows); print(df.to_string(index=False))
eng = df.loc[df.uJ_tolik.idxmin()]
print(f"\nEng tejamkor: {eng.konfiguratsiya}, {eng.oqim} oqim, batch {eng.batch}:
{eng.uJ_tolik} uJ/inferens (to'liq; 1 mJ = 1000 uJ), {eng.inf_s_W} inferens/s/W")
print("Kuzatuv: batch 64 da inferens boshiga energiya batch 1 ga nisbatan ancha kam — doimiy
xarajat (Python, sessiya chaqiruvi) taqsimlanadi.")
print("  uJ_tolik — TCO uchun (bo'sh quvvat ham hisobga olinadi), uJ_dinamik — algoritm
samaradorligi uchun. ResNet-50 da bu 10-100 mJ bo'ladi.")

# --- Little qonuni bo'yicha rejalashtirish
print("\n=== Nusxalar rejasi (Little qonuni) ===")
lam = 5000                                                  # kerakli yuklama, so'rov/s
rejalar = [(r.konfiguratsiya, r.latency_ms, r.P_W) for _, r in df[(df.batch == 1) & (df.oqim ==
2)].iterrows()]
rejalar.append(("ResNet-50 FP32 CPU (taxmin 20 ms, 15 W)", 20.0, 15.0))
rejalar.append(("ResNet-50 INT8 VNNI CPU (taxmin 8 ms, 15 W)", 8.0, 15.0))
for nomi, lat, P in rejalar:
    N = lam * lat / 1e3
    print(f"  {nomi:<44} {lam} x {lat:.3f} ms = {N:7.1f} parallel -> {math.ceil(N):4d} nusxa (2
yadro), quvvat ~{math.ceil(N) * P:.0f} W")

# --- GPU/TensorRT'da bu qadam qanday bo'ladi (sandbox'da GPU yo'q — faqat yo'l xaritasi)
print("\n=== GPU'da bu qadam qanday bo'ladi ===")
gpu = pd.DataFrame([
    dict(bosqich="Dvigatel qurish", buyruq="trtexec --onnx=cnn_fp32.onnx --saveEngine=fp32.plan
--minShapes=input:1x1x8x8 --optShapes=input:32x1x8x8 --maxShapes=input:256x1x8x8"),
    dict(bosqich="FP16", buyruq="trtexec ... --fp16   (Tensor Core, ResNet-50 da ~2.7x, aniqlik
-0.1 p.p.)"),
    dict(bosqich="INT8", buyruq="trtexec --onnx=cnn_s_pct.onnx --int8 --fp16   (Q/DQ tugunli
ONNX kalibrlashsiz qabul qilinadi)"),
    dict(bosqich="Quvvat", buyruq="nvidia-smi --query-gpu=power.draw --format=csv -lms 100
(parallel oqimda, 10-30 s davomida o'rtacha)"),
    dict(bosqich="Energiya", buyruq="mJ/rasm = P_ortacha / throughput * 1000;  rasm/s/W =
throughput / P_ortacha"),
])
for _, r in gpu.iterrows():
    print(f"  {r.bosqich:<16} {r.buyruq}")
print("  Kutilgan (T4, ResNet-50, batch 8): FP32 119 mJ -> FP16 45 mJ -> INT8 27 mJ/rasm;
quvvat deyarli o'zgarmaydi (~70 W), energiya throughput'ga proporsional.")

# Grafik: mJ/inferens ustunlari
```

```
fig, ax = plt.subplots(figsize=(8, 4))
lab = [f"{r.konfiguratsiya}\n{r.oqim} oqim, b={r.batch}" for _, r in df.iterrows()]
ax.bar(range(len(df)), df.uJ_tolik, color=["tab:blue" if "FP32" in l else "tab:orange" for l in
lab], label="to'liq")
ax.bar(range(len(df)), df.uJ_dinamik, color="k", alpha=.35, width=.4, label="dinamik")
ax.set_xticks(range(len(df))); ax.set_xticklabels(lab, fontsize=7); ax.set_ylabel("uJ /
inferens (model bo'yicha)")
ax.set_yscale("log"); ax.set_title("Energiya bahosi: to'liq va dinamik"); ax.legend();
ax.grid(axis="y", alpha=.3)
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Energiya bahosi ===`, `=== Nusxalar rejasi (Little qonuni) ===`, `=== GPU'da bu qadam qanday bo'ladi ===`

TAJRIBA QILIB KO‘RING

- P_TOLIK_W va P_BOSH_W ni o'z noutbukingiz TDP'siga moslang — xulosa o'zgaradimi?
- `psutil.Process().cpu_times()` bilan user/system vaqtni ajrating.
- Batch 256 va 1024 qo'shing: µJ/inferens qachon to'yinadi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Opset va tugunlar** 1-qadam kodida modelni opset 13, 15 va 17 bilan eksport qiling. Har biri uchun tugunlar soni, operatorlar ro'yxati, fayl hajmi va PyTorch bilan maksimal farqni jadvalga yozing. Qaysi opset'da farq bor-yo'qligini izohlang.	oson	5
2	**Optimallashtirish darajalari jadvali** 2-qadamni `takror=1000` bilan 3 marta qayta bajaring va har daraja uchun p50/p95/p99 o'rtachasini va tarqalishini (min–max) hisoblang. Qaysi daraja barqaror tezlanish beradi? Natijani 4 qatorli jadval va 2–3 jumla xulosa bilan topshiring.	oson	5
3	**Kvantlash formulasi** 3-qadamdagi qo'lda kvantlash kodini asimmetrik variantga (s = (max−min)/255, z = round(−min/s)) o'zgartiring va simmetrik bilan qayta tiklash xatosini (max, o'rtacha) uchala Linear qatlam uchun solishtiring. Qaysi qatlamda farq katta va nega (vaznlar taqsimotini gistogramma bilan ko'rsating)?	oson	5
4	**Kalibrlash to'plami hajmi** 4-qadamda statik kvantlashni 8, 32, 128, 320 va 1000 namunali kalibrlash to'plamlari bilan (MinMax va Percentile) bajaring. Aniqlik va kvantlash vaqtini jadvalga yozing, aniqlik–hajm grafigini chizing va kalibrlash to'plamining minimal yetarli hajmini asoslang.	o'rta	10
5	**Sezgir qatlam tahlili** 4-qadam kodida `quantize_static` ning `nodes_to_exclude` parametri bilan har bir Conv/Gemm tugunini navbatma-navbat FP32 da qoldiring (qolganlari INT8). Har konfiguratsiya uchun aniqlik va hajmni o'lchang, eng sezgir qatlamni aniqlang va «INT8 + 1 ta FP32 qatlam» aralash variantini to'liq INT8 bilan solishtiring.	o'rta	10
6	**Paket va kechikish byudjeti** 2-qadamni batch 1, 2, 4, 8, 16, 32, 64, 128, 256 uchun bajarib, latency va throughput egri chiziqlarini chizing. p99 kechikish 0,1 ms dan oshmasligi talabi bilan maksimal paket o'lchamini toping va Little qonuni orqali 10 000 so'rov/s uchun kerakli nusxalar sonini ikki variantda (batch 1 va topilgan maksimal batch) hisoblang.	o'rta	10
7	**MLP vs CNN: dinamik kvantlash farqi** 3- va 4-qadamlar asosida MLP (Linear og'ir) va CNN (Conv og'ir) uchun dinamik INT8 kvantlashning hajm, aniqlik va tezlik ta'sirini bir xil metodika bilan o'lchang (PyTorch va ORT ikkalasida). A3 matnidagi «dinamik kvantlash MatMul og'ir modellarga mos» tezisini o'z raqamlaringiz bilan tasdiqlang yoki rad eting.	o'rta	10
8	**QAT bilan aniqlikni tiklash** 4-qadamdagi shovqin bilan kalibrlangan (yoki ataylab per-tensor, 4 bitli ta'sirni simulyatsiya qilish uchun vaznlarni round(w/s) bilan buzilgan) modelni oling va PyTorch'da soxta kvantlash (fake quantization: forward'da vaznlarni kvantlab-qayta tiklash, backward oddiy) bilan 2 epoxa qayta o'qiting. INT8 aniqligini FP32 ga 0,5 p.p. ichida yaqinlashtiring; o'qitish egri chizig'i va yakuniy jadval bilan hisobot bering.	qiyin	15

9	**Latency taqsimoti va ishonch intervali** 5-qadamdagi 6 konfiguratsiyani har biri 5 marta, 1000 takror bilan o'lchang. p50 va p99 uchun bootstrap (1000 qayta tanlash) 95 % ishonch intervalini hisoblang, konfiguratsiyalar orasidagi farq statistik ahamiyatlimi — aniqlang (intervallar kesishadimi). Shuningdek, 450 ta test namunasida 0,2 p.p. aniqlik farqining ishonch intervalini (binomial) baholang va «qancha namuna kerak» savoliga javob bering.	qiyin	15
10	**To'liq Pareto hisoboti o'z modelingiz uchun** O'zingiz tanlagan boshqa arxitektura (masalan, 3 qatlamli kengroq CNN yoki `make_classification` dagi 100 belgili MLP) uchun 1–6 qadamlarning barchasini takrorlang: eksport, ORT darajalari, PyTorch va ORT kvantlash (kamida 2 kalibrlash usuli), yakuniy jadval, Pareto grafigi, energiya bahosi. 4–6 betlik hisobotda aniq ssenariy (masalan, A1 dagi SHV tizimi) uchun konfiguratsiya tanlang va «GPU bo'lsa nima o'zgaradi» bo'limini yozing.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: ko'krak saratoni skrining MLP — ONNX eksport va ORT darajalari oson

Soha: Tibbiyot (onkologiya)

Klinika laboratoriyasi 30 ta biopsiya belgisiga ko'ra xavfli/xavfsiz o'smani tasniflovchi modelni analizator qurilmasidagi CPU'ga joylashtirmoqchi. Qurilma ishlab chiqaruvchisi faqat ONNX formatini qabul qiladi va bitta namuna uchun p99 kechikish 5 ms dan oshmasligini talab qiladi. Sizdan FP32 etalonni o'qitish, ONNX'ga to'g'ri eksport qilish, ORT optimallashtirish darajalarini o'lchash va qaysi daraja qurilma talabiga mos kelishini asoslash so'raladi. Aniqlik birinchi o'rinda: recall 0,95 dan pastga tushmasligi kerak.

Model	MLP 30-64-32-2 (ReLU, Dropout 0,2)
Ma'lumot	sklearn load_breast_cancer (569 namuna, 30 belgi), StandardScaler
Epoxa	15, Adam lr 1e-3, paket 32
Opset	13 va 17 (solishtiriladi)
Batch	1, 8, 64
SLO	p99 <= 5 ms (batch 1), recall >= 0,95
ORT darajalari	DISABLE_ALL, BASIC, EXTENDED, ALL

QADAMLAR

1. Laboratoriya s1 kabi MLP'ni o'qiting (15 epoxa), test to'plamida aniqlik va recall'ni, PyTorch eager batch 1 p50/p95/p99 kechikishini o'lchang.
2. Modelni opset 13 va opset 17 bilan ONNX'ga eksport qiling (dinamik batch o'qi), onnx.checker bilan tekshiring, tugunlar sonini, operatorlar ro'yxatini va fayl hajmini ikkala opset uchun jadvalga yozing.
3. PyTorch va ORT chiqishlarini 64 namunada np.allclose (rtol 1e-3) bilan solishtiring; Dropout eval rejimida grafikdan yo'qolganini ko'rsating.
4. Laboratoriya s2 kabi ORT 4 ta optimallashtirish darajasini batch 1 da o'lchang (takror 500), har daraja uchun p50/p95/p99 va optimallashtirilgan grafikdagi tugunlar sonini jadvalga yozing.
5. Batch 1, 8, 64 uchun throughput va Little qonuni bo'yicha N ni hisoblang; SLO p99 <= 5 ms bajariladimi, qaysi darajada eng katta zaxira qolishini ko'rsating.
6. Xulosa: qurilmaga qaysi opset va daraja tavsiya etiladi, recall saqlanganmi, GPU bo'lsa nima o'zgarardi (2-3 jumla).

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

2-variant. Agro: tuproq sensorlari bo'yicha sug'orish tavsiyasi — chekka qurilma uchun dinamik INT8 oson

Soha: Agro

Fermer xo'jaligi dalalarida 20 ta ko'rsatkichni (namlik, harorat, EC, pH, azot va boshqalar) o'lchaydigan avtonom sensor stansiyalari bor. Stansiya ichidagi kichik ARM protsessor va 2 MB flesh xotira uchun 4 sinfli sug'orish tavsiyasi modeli kerak: model fayli 100 KB dan kichik bo'lishi, batareyada ishlashi kerak. Siz MLP'ni o'qitib, PyTorch dinamik INT8 kvantlashini qo'llaysiz, hajm va aniqlikni solishtirasiz hamda qnnpack dvigatelida (ARM uchun aynan shu dvigatel ishlatiladi) tezlikni o'lchaysiz.

Model	MLP 20-256-128-4
Ma'lumot	make_classification(n_samples=6000, n_features=20, n_informative=12, n_classes=4, random_state=7)
Epoxa	8, Adam lr 2e-3
Hajm cheklovi	<= 100 KB (state_dict)
Batch	1 va 128
Kvantlash dvigateli	qnnpack (ARM muqobili)
Kutilgan aniqlik	>= 85 %

QADAMLAR

1. Sintetik tuproq ma'lumotini yarating (random_state=7), 75/25 ga bo'ling, MLP'ni 8 epoxa o'qiting va FP32 aniqligini yozing.
2. Laboratoriya s3 kabi quantize_dynamic({nn.Linear}, qint8) qo'llang; kvantlangan model tuzilmasini chop eting va DynamicQuantizedLinear qatlamlar sonini ayting.
3. Birinchi qatlam vazni uchun simmetrik kvantlash formulasini qo'lda tekshiring (s = max|w|/127): qayta tiklash xatosi (max, o'rtacha), noyob qiymatlar soni.
4. FP32 va INT8 uchun hajm (KB), aniqlik, p50 kechikish batch 1 va batch 128 ni jadvalga yozing; hajm 100 KB chegarasiga sig'adimi, tekshiring.
5. Qatlam kengligini 256 dan 512 ga oshirib takrorlang: hajm nisbati 4x ga yaqinlashadimi, aniqlik o'sadimi, chegara buziladimi.
6. Xulosa: stansiyaga qaysi konfiguratsiya joylanadi, VNNI yo'q CPU'da tezlik nega oshmagani, ARM NEON'da nimani kutish mumkin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

3-variant. Transport: yo'l belgilarini tanish CNN — statik INT8 va uch kalibrlash usuli o'rta

Soha: Transport

Avtomobil ichidagi yordamchi tizim kameradan kesib olingan kichik belgilar tasvirini tasniflaydi; ishlab chiqaruvchi bort kompyuterida faqat INT8 inferensga ruxsat beradi (issiqlik va quvvat byudjeti tufayli). Siz digits to'plamini belgilar surrogati sifatida ishlatib, kengroq CNN o'qitasiz, ONNX Runtime'da statik kvantlashni MinMax, Percentile va Entropy usullari bilan bajarasiz va noto'g'ri kalibrlash (boshqa yoritilganlik — pikselga doimiy siljish) qanchalik zarar qilishini ko'rsatasiz. Talab: p95 kechikish 2 ms dan kam, aniqlik yo'qotish 0,5 p.p. dan oshmasin.

Model	CNN Conv(1->16)-Conv(16->32)-FC 128-10
Ma'lumot	sklearn digits 8x8, 70/30 bo'linish, random_state=3
Kalibrlash to'plami	256 namuna
Kalibrlash usullari	MinMax, Percentile, Entropy + noto'g'ri (pikselga +0,5 siljish)
SLO	p95 <= 2 ms (batch 1), aniqlik farqi <= 0,5 p.p.
Format	QDQ, per-channel, QUInt8 faollashuv / QInt8 vazn

QADAMLAR

1. Laboratoriya s1/s4 asosida CNN'ni 4 epoxa o'qiting, ONNX'ga eksport qiling, quant_pre_process dan o'tkazing; FP32 aniqligi va p50/p95 ni yozing.
2. 256 namunali real kalibrlash to'plami bilan quantize_static ni MinMax, Percentile va Entropy usullarida bajaring (QDQ, per-channel).
3. Noto'g'ri kalibrlash: xuddi shu 256 namunaga +0,5 doimiy siljish qo'shib (yoritilganlik o'zgarishi) MinMax bilan kvantlang.
4. Har 5 konfiguratsiya (FP32 + 4 INT8) uchun hajm, aniqlik, p50/p95 batch 1, throughput batch 256 va Q/DQ tugunlar sonini jadvalga yig'ing.
5. Kalibrlash to'plami hajmini 32 va 1000 ga o'zgartirib Percentile uchun aniqlikni qayta o'lchang; aniqlik-hajm grafigini chizing.
6. Xulosa: qaysi usul SLO ga mos, noto'g'ri kalibrlash qancha p.p. yo'qotdi, bort kompyuterida (VNNI/NEON) INT8 tezlanishini qanday tekshirish kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

4-variant. Telekom: tarmoq trafigi anomaliyasi uchun autoenkoder — ONNX va yuqori throughput o'rta

Soha: Telekom

Mobil operatorning yadro tarmog'ida har soniyada 40 000 ta oqim yozuvi (flow record) hosil bo'ladi; ularni 16 belgili autoenkoder bilan tekshirib, rekonstruksiya xatosi katta bo'lganlarini anomaliya deb belgilash kerak. Model GPU'siz 2 yadroli virtual mashinada ishlaydi, shuning uchun throughput asosiy mezon: nechta nusxa kerakligini Little qonuni bilan hisoblaysiz. Siz autoenkoderni sintetik trafikda o'qitasiz, ONNX'ga eksport qilasiz, paket o'lchami va oqimlar sonini o'lchaysiz va dinamik INT8 ta'sirini tekshirasiz.

Model	Autoenkoder 16-32-8-32-16 (MSE)
Ma'lumot	sintetik: normal oqimlar make_blobs(1 markaz, 16 belgi, std 1,0) 20000 ta + 2 % anomaliya (std 4,0)
Anomaliya chegarasi	normal xatolarning 99-foizili
Batch	1, 64, 512, 2048
Oqimlar	intra_op 1 va 2
Talab	40 000 yozuv/s, p99 (batch) <= 50 ms
Metrika	precision/recall anomaliya bo'yicha

QADAMLAR

1. Sintetik trafikni yarating, autoenkoderni faqat normal yozuvlarda 10 epoxa o'qiting; test to'plamida rekonstruksiya xatosi va 99-foizil chegara bo'yicha precision/recall ni hisoblang.
2. ONNX'ga eksport qiling (dinamik batch), onnx.checker, PyTorch bilan np.allclose mosligi; tugunlar soni va hajm.
3. Laboratoriya s2 kabi ORT 4 darajasini batch 1 da o'lchang, keyin batch 1/64/512/2048 uchun p50/p99 va throughput (yozuv/s) jadvalini tuzing.
4. intra_op 1 va 2 oqimni batch 2048 da solishtiring, tezlanish koeffitsientini yozing.
5. ORT quantize_dynamic bilan INT8 variant yarating: precision/recall o'zgaradimi, throughput o'zgaradimi (VNNI yo'q — izohlang).
6. Little qonuni: 40 000 yozuv/s uchun har batch o'lchamida nechta nusxa kerak; p99 <= 50 ms sharti bilan optimal batch'ni tanlang.
7. Xulosa va grafik: batch - throughput, batch - p99; tavsiya etilgan konfiguratsiya.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

5-variant. Energetika: soatlik elektr iste'moli prognozi — regressiya modelida kvantlash xatosi oson

Soha: Energetika

Taqsimot kompaniyasi 24 soatlik oyna va ob-havo belgilaridan keyingi soat iste'molini bashorat qiluvchi modelni 1200 ta transformator podstansiyasidagi kontrollerga joylashtirmoqchi. Kontrollerlar eski x86 CPU'li, xotira 256 MB; har 5 daqiqada bitta prognoz kerak, ya'ni tezlik emas, hajm va aniqlik muhim. Siz MLP regressorni o'qitasiz, ONNX'ga eksport qilib ORT'da tekshirasiz, PyTorch va ORT dinamik kvantlashning MAE/MAPE ga ta'sirini o'lchaysiz va tasniflashdan farqli ravishda regressiyada kvantlash xatosi qanday namoyon bo'lishini tushuntirasiz.

Model	MLP 30-128-64-1 (regressiya, MSE)
Ma'lumot	make_regression(n_samples=8000, n_features=30, n_informative=20, noise=8, random_state=11), nishon MinMax [0,1]
Epoxa	20, Adam lr 1e-3
Metrika	MAE, MAPE, R^2
Batch	1 va 256
Cheklov	hajm <= 60 KB, MAE o'sishi <= 2 %
Opset	17

QADAMLAR

1. Sintetik iste'mol ma'lumotini yarating (random_state=11), nishonni [0,1] ga normallang, MLP'ni 20 epoxa o'qiting; test MAE, MAPE, R^2 ni yozing.
2. ONNX eksport (opset 17, dinamik batch), onnx.checker; ORT va PyTorch chiqishlarini 256 namunada maksimal farq bilan solishtiring.
3. ORT ALL darajasida batch 1 va 256 uchun p50 kechikish va throughput o'lchang (laboratoriya s2 funksiyasi bilan).
4. PyTorch quantize_dynamic va ORT quantize_dynamic bilan ikki INT8 variant yarating; hajm, MAE, MAPE, R^2, p50 ni FP32 bilan bitta jadvalga yig'ing.
5. Kvantlash xatosini tahlil qiling: FP32 va INT8 bashoratlari farqining gistogrammasini chizing, eng katta farq qaysi diapazonda ekanini ayting.
6. Xulosa: kontrollerga qaysi variant mos, hajm chegarasi bajarildimi, regressiyada «aniqlik o'zgarmadi» deyish nega noto'g'ri bo'lishi mumkin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

6-variant. Moliya: kredit skoring — tasodifiy o'rmondan MLP distillyatsiyasi va ONNX qiyin

Soha: Moliya (bank krediti)

Bank kredit skoringini sklearn RandomForest bilan bajaradi, lekin yangi onlayn xizmat har bir so'rovga 1 ms dan kam kechikish talab qiladi va faqat ONNX Runtime'ni qo'llab-quvvatlaydi; sandbox'da skl2onnx yo'q. Siz daraxt ansamblini «o'qituvchi» sifatida olib, uning ehtimolliklarini MLP «o'quvchi»ga distillyatsiya qilasiz, MLP'ni ONNX'ga eksport qilasiz va sklearn predict_proba bilan ORT kechikishini solishtirasiz. Keyin o'quvchini INT8 ga kvantlab, AUC va «tasdiqlangan arizalar ulushi» o'zgarishini tekshirasiz — bank uchun AUC 0,01 dan ko'p pasaymasligi shart.

O'qituvchi	RandomForest 200 daraxt, max_depth 10
O'quvchi	MLP 24-128-64-2 (soft-label KL yo'qotish, T=2)
Ma'lumot	make_classification(n_samples=20000, n_features=24, n_informative=14, weights=[0.8,0.2], random_state=21)
Metrika	AUC, aniqlik, tasdiqlash chegarasi 0,5 da tasdiqlanganlar ulushi
SLO	p99 <= 1 ms (batch 1)
Batch	1, 32, 1024
Kvantlash	ORT dinamik + statik Percentile (512 namuna)

QADAMLAR

1. Sintetik kredit ma'lumotini yarating, RandomForest'ni o'qiting, test AUC va bitta so'rov uchun predict_proba p50/p99 kechikishini (300 takror) o'lchang.
2. O'qituvchining ehtimolliklari bilan MLP'ni distillyatsiya qiling (KL + CE aralash, 15 epoxa); o'quvchi AUC'si o'qituvchidan qancha farq qilishini yozing. Taqqoslash uchun oddiy yorliqlar bilan o'qitilgan MLP ham qo'shing.
3. O'quvchini ONNX'ga eksport qiling (laboratoriya s1), ORT 4 daraja va batch 1/32/1024 uchun kechikish va throughput jadvalini tuzing; sklearn RF bilan tezlik nisbatini hisoblang.
4. ORT dinamik va statik (Percentile, 512 namuna) INT8 kvantlash: hajm, AUC, p50, tasdiqlanganlar ulushi; FP32 bilan farq.
5. Pareto tahlili (p50 - AUC) va grafik; RF, MLP FP32, MLP INT8 nuqtalarini bitta grafikda ko'rsating.
6. Tasdiqlangan arizalar ulushi kvantlashdan keyin o'zgargan bo'lsa, bu nechta mijozga ta'sir qiladi (20 000 ta ariza misolida) — hisoblang va izohlang.
7. Xulosa: distillyatsiya qancha aniqlik yo'qotdi va qancha tezlik berdi, skl2onnx bo'lsa qanday bo'lardi, bank uchun tavsiya.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); O'qituvchi-o'quvchi taqqoslash jadvali (AUC, kechikish, hajm)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

7-variant. Ta'lim: talaba javoblari ketma-ketligi uchun kichik transformer bloki — ONNX, fusion va MatMul kvantlash qiyin

Soha: Ta'lim

Adaptiv test platformasi talabaning oxirgi 16 ta javobi (savol identifikatori va to'g'ri/noto'g'ri) asosida keyingi savolga javob ehtimolini bashorat qiladi. Model — bitta self-attention bloki va ikki Linear qatlam; platformaning inferens serveri GPU'siz, bir vaqtda 1500 ta talaba ishlaydi. Siz transformer blokini sintetik ketma-ketliklarda o'qitasiz, ONNX'ga eksport qilasiz, ORT EXTENDED/ALL darajalarida Attention/LayerNorm birlashuvi (fusion) bo'lgan-bo'lmaganini tugunlar bo'yicha tekshirasiz, MatMul og'ir modelda dinamik INT8 kvantlashni baholaysiz va p99 10 ms SLO uchun nusxalar sonini hisoblaysiz.

Model	Embedding(200, 32) + MultiheadAttention(32, 4 bosh) + LayerNorm + FFN 32-64-32 + Linear 32-1
Ma'lumot	sintetik: 20000 ketma-ketlik, uzunlik 16, savol ID 0..199, javob yorlig'i talaba qobiliyati va savol qiyinligidan logistik model bilan
Epoxa	6, Adam lr 1e-3, BCE
Batch	1, 16, 128
SLO	p99 <= 10 ms, 1500 so'rov/s
Opset	17
Kvantlash	PyTorch quantize_dynamic (Linear), ORT quantize_dynamic (MatMul)

QADAMLAR

1. Sintetik javob ketma-ketliklarini yarating (qobiliyat ~ N(0,1), qiyinlik ~ N(0,1), p = sigmoid(qobiliyat - qiyinlik)), modelni 6 epoxa o'qiting, test AUC ni yozing.
2. ONNX'ga eksport qiling (dinamik batch, opset 17); onnx.checker; operatorlar ro'yxatini chop eting (MatMul, Softmax, LayerNormalization yoki uning qismlari bormi).
3. ORT 4 darajasida optimallashtirilgan grafikni faylga saqlab tugunlarni sanang; EXTENDED/ALL da qaysi birlashgan operatorlar (fused) paydo bo'lganini jadvalga yozing.
4. Batch 1/16/128 uchun p50/p99 va throughput; intra_op 1 va 2 oqim taqqosi.
5. PyTorch va ORT dinamik INT8: hajm, AUC, p50 batch 1, throughput batch 128; embedding qatlami kvantlandimi, tekshiring.
6. Little qonuni: 1500 so'rov/s va p99 bo'yicha nusxalar soni FP32 va INT8 uchun; Pareto grafigi (p50 - AUC).
7. Xulosa: fusion qancha tezlik berdi, MatMul og'ir model uchun dinamik kvantlash tezisi tasdiqlandimi, GPU/VNNI bo'lsa nimalar o'zgaradi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatorlar/fusion jadvali (4 daraja)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

8-variant. Sanoat: podshipnik vibratsiyasi bo'yicha nosozlik tasnifi — 1D CNN va sezgir qatlam tahlili o'rta

Soha: Sanoat (mashinasozlik)

Zavod nasos stansiyasidagi 300 ta dvigatelning vibratsiya signallari (256 nuqtalik oynalar) 4 sinfga (soz, ichki halqa, tashqi halqa, sharcha nuqsoni) tasniflanadi. Model sexdagi sanoat kompyuteriga joylanadi, u yerda xotira kam va INT8 ma'qul, lekin nosozlikni o'tkazib yuborish qimmat: har sinf bo'yicha recall 0,9 dan pastga tushmasin. Siz 1D CNN'ni sintetik vibratsiya signallarida o'qitasiz, ONNX statik kvantlashni bajarasiz va nodes_to_exclude bilan qaysi Conv qatlami kvantlashga eng sezgir ekanini aniqlab, aralash (INT8 + 1 FP32 qatlam) variantni taklif qilasiz.

Model	1D CNN: Conv1d(1->8,k7)-Pool-Conv1d(8->16,k5)-Pool-Conv1d(16->32,k3)-GAP-Linear 32-4
Ma'lumot	sintetik vibratsiya: 256 nuqta, asosiy sinusoida 50 Hz + sinfga xos zarba chastotasi (0/107/162/141 Hz) + Gauss shovqin std 0,3; 4000 namuna
Epoxa	8, Adam lr 2e-3
Kalibrlash	MinMax va Percentile, 200 namuna, per-channel
Metrika	aniqlik, sinf bo'yicha recall
Cheklov	recall >= 0,9 har sinfda, hajm minimal
Batch	1 va 128

QADAMLAR

1. Sintetik vibratsiya generatorini yozing (4 sinf, zarba chastotalari), 1D CNN'ni o'qiting, test aniqligi va sinflar bo'yicha recall jadvali.
2. ONNX eksport (dinamik batch), quant_pre_process; FP32 ORT p50 batch 1 va throughput batch 128.
3. Laboratoriya s4 kabi statik kvantlash (MinMax, Percentile, QDQ per-channel) va dinamik kvantlash; hajm, aniqlik, recall, p50, Q/DQ tugunlar soni jadvali.
4. nodes_to_exclude bilan uchta Conv va Gemm tugunini navbatma-navbat FP32 da qoldirib 4 ta aralash variant yarating; har biri uchun aniqlik va eng past recall'ni o'lchang.
5. Eng sezgir qatlamni aniqlang va «INT8 + 1 FP32 qatlam» variantini to'liq INT8 va FP32 bilan Pareto (hajm - eng past recall) grafigida ko'rsating.
6. Xulosa: sanoat kompyuteriga qaysi variant, recall talabi bajarildimi, nega birinchi qatlam ko'pincha sezgir bo'ladi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Sezgir qatlam jadvali (4 aralash variant)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

9-variant. Kiberxavfsizlik: login hodisalari bo'yicha hujum aniqlash — keng MLP va 4x hajm chegarasi oson

Soha: Xavfsizlik

Korporativ SIEM tizimi har soniyada 25 000 ta autentifikatsiya hodisasini (40 belgi: vaqt, geolokatsiya, qurilma, xatolar soni va h.k.) hujum/normal deb tasniflaydi. Tasniflagich SIEM agentining ichida ishlaydi, agent paketi o'lchami qat'iy cheklangan, shuning uchun model hajmi kamida 3,5 marta qisqarishi kerak. Siz keng MLP'ni sintetik hodisalarda o'qitasiz, PyTorch dinamik INT8 kvantlash bilan hajm, aniqlik va tezlikni o'lchaysiz hamda batch 1 da p99 1 ms SLO ni tekshirasiz.

Model	MLP 40-512-512-2
Ma'lumot	make_classification(n_samples=30000, n_features=40, n_informative=25, weights=[0.95,0.05], flip_y=0.01, random_state=5)
Epoxa	5, Adam lr 1e-3, paket 128
Hajm talabi	>= 3,5x qisqarish
SLO	p99 <= 1 ms (batch 1)
Batch	1, 256
Metrika	aniqlik, hujum sinfi uchun recall va F1

QADAMLAR

1. Sintetik hodisalarni yarating (muvozanatsiz, 5 % hujum), MLP'ni o'qiting; aniqlik, recall va F1 (hujum sinfi) ni yozing.
2. Laboratoriya s3 kabi quantize_dynamic qo'llang (qnnpack), kvantlangan tuzilmani chop eting; 2-qatlam (512x512) vazni uchun masshtab s, qayta tiklash xatosi va noyob qiymatlar sonini hisoblang.
3. FP32 va INT8: hajm KB, aniqlik, recall, F1, p50/p99 batch 1 va p50 batch 256 jadvali; hajm nisbati 3,5x dan katta-kichikligini tekshiring.
4. Per-channel masshtab bilan qo'lda kvantlab (har qator uchun alohida s) qayta tiklash xatosini per-tensor bilan solishtiring.
5. ONNX'ga eksport qilib ORT ALL da batch 1 p99 ni o'lchang; SLO 1 ms uchun PyTorch INT8, ORT FP32 qaysi biri mos — jadvalda ko'rsating.
6. Xulosa: agentga qaysi format, recall saqlanganmi, VNNI yo'q CPU'da tezlik natijasi va serverda nimani kutish kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

10-variant. Chakana savdo: mahsulot tavsiyasi uchun embedding + MLP — Gather kvantlash va katta paket o'rta

Soha: Chakana savdo

Onlayn do'kon 5000 ta mahsulot va 20 000 ta foydalanuvchi uchun «sotib oladi/olmaydi» ehtimolini hisoblaydi; tungi qayta hisoblashda 100 million juftlik baholanadi, kunduzi esa bitta foydalanuvchi uchun 200 ta nomzodni 20 ms ichida saralash kerak. Model — ikkita embedding va MLP; eng katta hajm embeddinglarda. Siz modelni sintetik o'zaro ta'sirlar (interactions) ustida o'qitasiz, ONNX'ga eksport qilasiz, katta paketlarda throughput'ni, dinamik va statik kvantlashda embeddinglar (Gather) kvantlanadimi-yo'qmi tekshirasiz va tungi hisob uchun vaqtni baholaysiz.

Model	Embedding(20000, 16) + Embedding(5000, 16) + MLP 32-64-1
Ma'lumot	sintetik: foydalanuvchi va mahsulot yashirin vektorlari N(0,1), yorliq = sigmoid(skalyar ko'paytma + shovqin) > 0,5; 300 000 juftlik
Epoxa	3, Adam lr 3e-3, paket 512
Batch (onlayn)	200 nomzod bitta so'rovda
Batch (tungi)	4096, 16384
SLO	p95 <= 20 ms (200 nomzod), tungi 100 mln juftlik <= 2 soat 1 nusxada
Metrika	AUC

QADAMLAR

1. Sintetik o'zaro ta'sirlarni yarating, modelni 3 epoxa o'qiting, test AUC ni yozing; parametrlar sonini embedding va MLP bo'yicha ajrating.
2. ONNX'ga eksport qiling (ikki kirish: user_id, item_id, dinamik batch), onnx.checker; operatorlar ichida Gather borligini ko'rsating; hajm.
3. ORT ALL: batch 200 uchun p50/p95 (SLO tekshiruvi), batch 4096 va 16384 uchun throughput (juftlik/s); 100 mln juftlik uchun vaqtni hisoblang.
4. ORT quantize_dynamic va quantize_static (MinMax, 1024 juftlik kalibrlash): hajm, AUC, throughput; Gather/embedding vaznlari kvantlanganmi — grafik tugunlarini tekshiring.
5. Agar embedding kvantlanmagan bo'lsa, embedding jadvallarini qo'lda INT8 ga (per-qator masshtab) kvantlab qayta tiklang va AUC ni qayta o'lchang; hajm nisbatini hisoblang.
6. Pareto (hajm - AUC) va (throughput - AUC) grafiklari; onlayn va tungi rejim uchun alohida tavsiya bering.
7. Xulosa: hajm qayerda, qaysi kvantlash foyda berdi, 1 nusxada tungi hisob ulgurmasa nechta nusxa kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

11-variant. Logistika: qo'l skanerida yetkazib berish vaqti prognozi — energiya bahosi va batareya umri o'rta

Soha: Logistika

Kuryerlik kompaniyasi 4000 ta batareyali qo'l skaneriga yetkazib berish muddatini (daqiqa) bashorat qiluvchi modelni o'rnatmoqda; har skaner kuniga 1500 ta prognoz qiladi va bir zaryadda kamida 12 soat ishlashi kerak. Shuning uchun asosiy mezon — inferens boshiga energiya (uJ). Siz MLP regressorni o'qitasiz, ONNX FP32 va INT8 dinamik variantlarini 1 va 2 oqimda, batch 1 va 32 da laboratoriya s6 quvvat modeli bilan baholaysiz, lekin quvvat parametrlarini mobil protsessorga moslaysiz (P_bosh 0,4 Vt, P_to'liq 2,5 Vt) va kunlik energiya sarfini mAh ga aylantirasiz.

Model	MLP 18-96-48-1 (regressiya)
Ma'lumot	make_regression(n_samples=10000, n_features=18, n_informative=12, noise=5, random_state=13); nishon 10..180 daqiqaga masshtablangan
Quvvat modeli	P_bosh 0,4 Vt, P_to'liq 2,5 Vt (2 yadro)
Batareya	3000 mAh, 3,8 V
Batch	1 va 32
Oqimlar	1 va 2
Kunlik yuklama	1500 prognoz/skaner
Metrika	MAE (daqiqa)

QADAMLAR

1. Sintetik ma'lumotni yarating, MLP'ni 15 epoxa o'qiting, test MAE ni daqiqada yozing.
2. ONNX eksport + quant_pre_process + ORT quantize_dynamic; FP32 va INT8 uchun hajm va MAE.
3. Laboratoriya s6 energiya funksiyasini mobil quvvat parametrlari bilan ishlating: FP32/INT8 x 1/2 oqim x batch 1/32 uchun latency, CPU bandlik, P, uJ_tolik, uJ_dinamik, inferens/s/Vt jadvali.
4. Kunlik energiya: 1500 prognoz x uJ_tolik ni J va mAh (3,8 V) ga aylantiring; batareyaning necha foizi inferensga ketishini har konfiguratsiya uchun hisoblang.
5. Ustunli grafik (uJ/inferens, log o'q) va eng tejamkor konfiguratsiyani belgilang; 2 oqim batch 1 da nega energiya ko'payganini tushuntiring.
6. Xulosa: skanerga tavsiya, 12 soat talabi bajariladimi, ARM qurilmada RAPL o'rniga qanday o'lchash mumkin (batareya hisoblagichi).

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Energiya jadvali (8 konfiguratsiya) va kunlik mAh hisobi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

12-variant. Shahar: chorraha trafik xaritasi bo'yicha tiqilinch bashorati — kichik CNN, oqimlar va Little qonuni oson

Soha: Shahar

Shahar transport markazi 600 ta chorrahadan har 2 soniyada 6x6 katakli band-bo'shlik xaritasini oladi va 3 sinfli tiqilinch darajasini bashorat qiladi; markaziy server 2 yadroli virtual mashina, GPU yo'q. Umumiy yuklama 3000 so'rov/s ga yetishi mumkin. Siz kichik 2D CNN'ni sintetik xaritalarda o'qitasiz, ONNX'ga eksport qilasiz, ORT darajalari, paket o'lchami va oqimlar sonini o'lchaysiz hamda Little qonuni bilan nechta nusxa kerakligini p99 bo'yicha hisoblaysiz.

Model	CNN Conv(1->8,3x3)-ReLU-Conv(8->16,3x3)-ReLU-GAP-Linear 16-3
Ma'lumot	sintetik 6x6 xarita: sinf bo'yicha bandlik ehtimoli 0,2/0,5/0,8 + fazoviy klaster; 9000 namuna
Epoxa	5
Batch	1, 32, 128, 512
Oqimlar	intra_op 1 va 2
Talab	3000 so'rov/s, p99 <= 3 ms (batch 1)
ORT darajalari	4 ta

QADAMLAR

1. Sintetik 6x6 trafik xaritalari generatorini yozing (3 sinf), CNN'ni o'qiting, aniqlikni yozing.
2. ONNX eksport (dinamik batch, opset 17), onnx.checker, PyTorch bilan moslik; tugunlar soni (BatchNorm ishlatgan bo'lsangiz birlashganini ko'rsating).
3. Laboratoriya s2 kabi 4 ta ORT darajasini batch 1 da o'lchang (takror 500) va tugunlar sonini sanang; PyTorch eager bilan tezlanish.
4. Batch 1/32/128/512 uchun p50/p99, throughput va little_N jadvali; 1 va 2 oqim batch 512 da taqqos.
5. Little qonuni: 3000 so'rov/s uchun batch 1 (p99 bo'yicha) va eng yaxshi batch uchun nusxalar soni; p99 <= 3 ms sharti qaysi batch'gacha bajariladi.
6. Grafik (batch - throughput, batch - p99) va xulosa: markaziy serverga nechta nusxa, GPU bo'lsa qanday o'zgaradi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

13-variant. Ekologiya: havo sifati indeksi tasnifi — INT8 kvantlash va magnitudali siyraklashtirish (pruning) taqqosi o'rta

Soha: Ekologiya

Shahar ekologiya xizmati 150 ta ko'cha sensor qutisiga havo sifati indeksini 5 sinfga ajratuvchi modelni joylashtirmoqda; qutilar 4G orqali yangilanadi va trafik qimmat, shuning uchun model fayli iloji boricha kichik bo'lishi kerak. Sizdan ikki yo'lni solishtirish so'raladi: INT8 kvantlash va vaznlarni magnituda bo'yicha siyraklashtirish (30/60/90 %), hamda ularning kombinatsiyasi. Siz MLP'ni sintetik sensor ma'lumotida o'qitasiz, torch.nn.utils.prune bilan siyraklashtirib qayta o'qitasiz (fine-tune), ONNX'ga eksport qilasiz va hajm (siqilgan .onnx.gz ham), aniqlik, kechikishni Pareto grafigida taqqoslaysiz.

Model	MLP 13-256-128-5
Ma'lumot	make_classification(n_samples=12000, n_features=13, n_informative=9, n_classes=5, random_state=17)
Siyraklashtirish	L1 unstructured 30 %, 60 %, 90 % + 2 epoxa fine-tune
Kvantlash	PyTorch dinamik INT8, ORT dinamik INT8
Hajm o'lchovi	.onnx va gzip siqilgan .onnx.gz (KB)
Batch	1 va 256
Cheklov	aniqlik yo'qotish <= 1 p.p.

QADAMLAR

1. Sintetik havo sifati ma'lumotini yarating, MLP'ni 10 epoxa o'qiting, FP32 aniqligi, hajm va ORT p50 batch 1.
2. torch.nn.utils.prune.l1_unstructured bilan 30/60/90 % siyraklashtiring (prune.remove bilan niqobni qotiring), har birini 2 epoxa fine-tune qiling; nol vaznlar ulushini tekshiring.
3. Har siyrak modelni ONNX'ga eksport qiling; .onnx hajmi o'zgarmaganini ko'rsating va gzip bilan siqilgan hajmni o'lchang (siyraklik siqilishda namoyon bo'ladi).
4. ORT quantize_dynamic ni FP32 va 60 % siyrak modelga qo'llang; jami 8 konfiguratsiya uchun hajm (.onnx, .gz), aniqlik, p50 batch 1, throughput batch 256 jadvali.
5. Pareto (gz hajm - aniqlik) grafigi; 4G orqali 150 qutiga yangilash trafigini (MB) har konfiguratsiya uchun hisoblang.
6. Xulosa: siyraklashtirish CPU'da tezlik beradimi (nega yo'q), qaysi kombinatsiya 1 p.p. chegarasida eng kichik, strukturaviy siyraklashtirish qachon kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 8 konfiguratsiyali jadval (hajm .onnx/.gz, aniqlik, tezlik)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

14-variant. Sport: akselerometr bo'yicha harakat turini tanish — 1D CNN + GRU, statik kvantlash per-tensor vs per-channel qiyin

Soha: Sport

Fitnes bilaguzuk ishlab chiqaruvchisi 3 o'qli akselerometrning 2 soniyalik oynasi (64 qadam x 3 kanal) bo'yicha 6 ta harakat turini (yurish, yugurish, zinadan chiqish, o'tirish, turish, velosiped) aniqlaydi. Bilaguzuk protsessori faqat INT8 ni tez bajaradi va rekurrent qatlamlar kvantlashda nozik. Siz 1D CNN + GRU modelini sintetik signallarda o'qitasiz, ONNX'ga eksport qilasiz (GRU operatori), ORT statik kvantlashni per-tensor va per-channel rejimlarida, MinMax va Percentile usullarida bajarasiz, GRU tugunini kvantlashdan chiqarib ko'rasiz va 6 sinf bo'yicha chalkashlik matritsasi orqali qaysi harakatlar INT8 da ko'proq zarar ko'rishini aniqlaysiz.

Model	Conv1d(3->16,k5)-ReLU-Pool-GRU(16->32)-Linear 32-6
Ma'lumot	sintetik: 64 qadam x 3 o'q, har sinf uchun chastota/amplituda/ofset naqshi + shovqin std 0,25; 6000 namuna
Epoxa	10, Adam lr 2e-3
Kalibrlash	MinMax va Percentile, 300 namuna
Rejimlar	per-tensor, per-channel, GRU FP32 da (nodes_to_exclude)
Batch	1 va 64
Cheklov	har sinf recall >= 0,85

QADAMLAR

1. Sintetik akselerometr generatorini yozing (6 sinf), modelni o'qiting, aniqlik va chalkashlik matritsasini chop eting.
2. ONNX eksport (dinamik batch, opset 17), onnx.checker; operatorlar ro'yxatida GRU borligini va ORT bilan moslikni ko'rsating; GRU eksportida yuzaga kelgan muammolarni (bo'lsa) hujjatlang.
3. quant_pre_process va ORT dinamik kvantlash; keyin statik kvantlash 4 variant: per-tensor MinMax, per-channel MinMax, per-channel Percentile, per-channel Percentile + GRU FP32 da.
4. Har variant uchun hajm, aniqlik, sinflar bo'yicha recall, p50 batch 1, throughput batch 64, Q/DQ tugunlar soni jadvali.
5. FP32 va eng yomon INT8 variant chalkashlik matritsalarini yonma-yon chizing; qaysi juft sinflar aralashib ketganini va nima uchun (o'xshash chastota) tushuntiring.
6. Pareto (hajm - eng past recall) grafigi; bilaguzukka tavsiya va «recall >= 0,85» talabi bajarilganligini tekshiring.
7. Xulosa: per-tensor va per-channel farqi qancha, GRU ni FP32 da qoldirish qancha hajm/aniqlik savdosini beradi, QAT qachon kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Chalkashlik matritsalari (FP32 va INT8)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

15-variant. Media: sharhlar kayfiyati tasnifi — so'zlar sumkasi MLP va dinamik kvantlash oson

Soha: Media

Yangiliklar portali kuniga 2 million foydalanuvchi sharhini ijobiy/salbiy/neytral deb belgilaydi; model 1000 o'lchamli so'zlar sumkasi (bag-of-words) vektorini oladi va keng Linear qatlamlardan iborat, ya'ni dinamik kvantlash uchun ideal nomzod. Portal serverlarida GPU yo'q, xizmat 2 yadroli konteynerlarda ishlaydi. Siz sintetik siyrak matn belgilarini yaratasiz, MLP'ni o'qitasiz, PyTorch va ORT dinamik INT8 kvantlashini qo'llab hajm, aniqlik va tezlikni o'lchaysiz hamda kunlik 2 million sharhni qayta ishlash uchun nechta konteyner-soat kerakligini hisoblaysiz.

Model	MLP 1000-512-128-3
Ma'lumot	make_classification(n_samples=20000, n_features=1000, n_informative=60, n_classes=3, random_state=23), belgilar >= 0 qilib siyraklashtirilgan (80 % nol)
Epoxa	4, Adam lr 1e-3, paket 128
Batch	1, 64, 512
Kunlik yuklama	2 000 000 sharh
Kvantlash	PyTorch quantize_dynamic, ORT quantize_dynamic
Metrika	makro F1

QADAMLAR

1. Sintetik so'zlar sumkasi ma'lumotini yarating (manfiy qiymatlarni nolga kesib siyraklashtiring), MLP'ni o'qiting; aniqlik va makro F1.
2. Laboratoriya s3 kabi PyTorch dinamik kvantlash: tuzilma, 1-qatlam (1000x512) masshtab va qayta tiklash xatosi, hajm nisbati (4x ga qanchalik yaqin).
3. ONNX eksport + ORT quantize_dynamic; FP32 PyTorch, INT8 PyTorch, FP32 ORT, INT8 ORT — 4 konfiguratsiya uchun hajm, makro F1, p50 batch 1, throughput batch 64 va 512.
4. Har konfiguratsiya uchun 2 mln sharhni qayta ishlash vaqtini (soat) va 2 yadroli konteyner-soatlarni hisoblang; batch 512 da ishlash qancha tejaydi.
5. Grafik: hajm - throughput va Pareto (p50 - F1).
6. Xulosa: portalga qaysi konfiguratsiya, bu CPU'da INT8 tezlik bermagani sababi, VNNI serverda kutilgan tezlanish.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

16-variant. Tibbiyot: EKG aritmiya detektori — 1D CNN, uch mezonli Pareto va implant hajm chegarasi o'rta

Soha: Tibbiyot (kardiologiya)

Kiyiladigan kardiomonitor 1 soniyalik EKG segmentini (250 nuqta) normal/ekstrasistola/fibrillyatsiya deb tasniflaydi; monitor ichidagi mikrokontroller uchun model 50 KB dan oshmasligi, bitta segment 10 ms ichida qayta ishlanishi va xavfli sinflar uchun recall 0,95 dan kam bo'lmasligi kerak. Siz sintetik EKG segmentlarini yaratib 1D CNN'ni o'qitasiz, ONNX eksport, ORT FP32, dinamik INT8 va statik INT8 (ikki kalibrlash usuli) variantlarini tayyorlaysiz va uch mezonli (hajm, kechikish, recall) Pareto tahlilini o'tkazib, hajmni nuqta kattaligi bilan ko'rsatuvchi grafik chizasiz.

Model	Conv1d(1->8,k9)-Pool4-Conv1d(8->16,k7)-Pool4-Conv1d(16->16,k5)-GAP-Linear 16-3
Ma'lumot	sintetik EKG: 250 nuqta, Gauss QRS + P/T to'lqinlar; ekstrasistola = erta keng QRS, fibrillyatsiya = tartibsiz RR + P yo'q; 6000 segment
Epoxa	8
Cheklovlar	hajm <= 50 KB, p99 <= 10 ms, recall (2 xavfli sinf) >= 0,95
Kalibrlash	MinMax, Percentile, 256 segment
Batch	1 (asosiy), 64
Pareto	3 mezon: hajm, p50, eng past recall

QADAMLAR

1. Sintetik EKG generatorini yozing (3 sinf), modelni o'qiting, aniqlik va sinflar bo'yicha recall.
2. ONNX eksport, onnx.checker, PyTorch bilan moslik; hajm va tugunlar soni.
3. ORT 4 daraja batch 1 (p50/p99), batch 64 throughput; SLO p99 <= 10 ms tekshiruvi.
4. Dinamik va statik (MinMax, Percentile, per-channel) INT8; FP32 bilan birga 4 konfiguratsiya uchun hajm, recall (har sinf), p50, Q/DQ tugunlar jadvali.
5. Uch mezonli Pareto: laboratoriya s5 pareto funksiyasini (hajm, p50, eng past recall) uchligiga kengaytiring; grafikda x = p50, y = recall, nuqta kattaligi = hajm.
6. Hajm 50 KB dan oshsa kanallarni 8/16 dan 4/8 ga kamaytirib qayta o'qiting va jadvalga qo'shing; chegara ichidagi eng yaxshi recall qaysi.
7. Xulosa: monitorga tavsiya, aniqlik yo'qotish qaysi sinfda, kalibrlash to'plamida xavfli sinflar yetarli bo'lmasa nima bo'ladi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Uch mezonli Pareto grafigi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

17-variant. Aviatsiya: dvigatel sensorlari bo'yicha qoldiq resurs (RUL) prognozi — ONNX, kvantlash va energiya o'rta

Soha: Aviatsiya

Texnik xizmat kompaniyasi dvigatelning 21 ta sensor ko'rsatkichidan qoldiq ish sikllari sonini (RUL) bashorat qiladi; model parvoz paytida bort ma'lumot yig'uvchi blokida (passiv sovutish, quvvat byudjeti 3 Vt) har soniyada ishlaydi va parvoz tugagach natija yuboriladi. Talab: RMSE 15 sikldan oshmasin, inferens energiyasi minimal bo'lsin. Siz sintetik degradatsiya ma'lumotini yaratib MLP regressorni o'qitasiz, ONNX FP32 va INT8 variantlarini tayyorlaysiz, laboratoriya s6 quvvat modelini bort bloki parametrlariga moslab uJ/inferens va 10 soatlik parvoz uchun Wh ni hisoblaysiz.

Model	MLP 21-128-64-1 (regressiya, Huber yo'qotish)
Ma'lumot	sintetik: 300 dvigatel x 200 sikl; sensorlar = asosiy qiymat + degradatsiya egri chizig'i (RUL kamaygani sari eksponensial siljish) + shovqin
Metrika	RMSE (sikl), MAE
Quvvat modeli	P_bosh 0,8 Vt, P_to'liq 3,0 Vt
Parvoz	10 soat, 1 inferens/s
Batch	1 va 60 (1 daqiqalik to'plam)
Oqimlar	1 va 2

QADAMLAR

1. Sintetik degradatsiya generatorini yozing (RUL = 200 - sikl), dvigatellar bo'yicha 80/20 bo'ling (bir dvigatel ikkala to'plamga tushmasin), MLP'ni o'qiting, RMSE va MAE.
2. ONNX eksport, moslik, ORT ALL p50 batch 1; ORT quantize_dynamic va quantize_static (MinMax, 500 namuna) INT8 variantlari uchun RMSE va hajm.
3. Laboratoriya s6 energiya funksiyasini bort bloki quvvat parametrlari bilan FP32/INT8 statik x 1/2 oqim x batch 1/60 uchun ishlating; uJ_tolik, uJ_dinamik, inferens/s/Vt jadvali.
4. 10 soatlik parvoz uchun energiya (Wh) va 3 Vt byudjetdagi o'rtacha quvvat ulushini hisoblang: har soniyada bitta inferens (batch 1) va har daqiqada 60 talik to'plam (batch 60) rejimlarini solishtiring.
5. Ustunli grafik (uJ/inferens) va RMSE - energiya nuqtali grafigi; Pareto nuqtalarini belgilang.
6. Xulosa: bort blokiga qaysi rejim va format, batch 60 rejimida natija kechikishi (1 daqiqa) qabul qilinadimi, VNNI/ARM da nimani kutish mumkin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Energiya jadvali va parvoz uchun Wh hisobi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

18-variant. Fintech: tranzaksiya firibgarligini aniqlash — kvantlashning recall va chegaraga ta'siri, Entropy kalibrlash qiyin

Soha: Fintech (to'lovlar)

To'lov tizimi har soniyada 8000 tranzaksiyani real vaqtda tekshiradi; firibgarlik ulushi 1 % va model natijasi ehtimollik chegarasi orqali «bloklash/o'tkazish» qaroriga aylanadi. Infratuzilma jamoasi INT8 ga o'tishni taklif qilmoqda, lekin risk bo'limi «aniqlik o'zgarmadi» degan hisobotga ishonmaydi: muvozanatsiz ma'lumotda aniqlik 99 % bo'lib qolaveradi, recall esa tushishi mumkin. Siz MLP'ni o'qitasiz, ORT dinamik va statik (MinMax, Percentile, Entropy) kvantlashni bajarasiz, har variant uchun PR-egri, recall@precision 0,9 va optimal chegarani hisoblaysiz, hamda kvantlashdan keyin chegarani qayta kalibrlash kerakligini ko'rsatasiz.

Model	MLP 28-128-64-1 (BCE, pos_weight)
Ma'lumot	make_classification(n_samples=60000, n_features=28, n_informative=16, weights=[0.99,0.01], flip_y=0.002, random_state=29)
Epoxa	6, paket 256
Kalibrlash	MinMax, Percentile, Entropy; 1000 namuna (tasodifiy) va 1000 namuna (50 % firibgarlik bilan boyitilgan)
Metrika	AUC-PR, recall@precision=0,9, F1 optimal chegarada
SLO	8000 tranz/s, p99 <= 5 ms
Batch	1 va 256

QADAMLAR

1. Muvozanatsiz sintetik tranzaksiyalarni yarating, MLP'ni pos_weight bilan o'qiting; test aniqlik (nega yuqori), AUC-PR, recall@precision 0,9, optimal chegara.
2. ONNX eksport, quant_pre_process, ORT dinamik kvantlash; statik kvantlash 6 variant: 3 usul x 2 kalibrlash to'plami (tasodifiy / boyitilgan).
3. Har variant uchun FP32 chegarasida recall/precision, keyin shu variant uchun qayta tanlangan optimal chegarada recall/precision; farqni jadvalga yozing.
4. PR-egri chiziqlarini bitta grafikda chizing (FP32 + 7 INT8); AUC-PR farqlari.
5. Laboratoriya s2 kabi batch 1 p99 va batch 256 throughput; Little qonuni bilan 8000 tranz/s uchun nusxalar soni.
6. Pareto (p50 - recall@precision 0,9) va xulosa: qaysi kalibrlash to'plami yaxshiroq va nega, chegarani qayta kalibrlash qancha recall qaytardi, risk bo'limiga yakuniy tavsiya.
7. Bootstrap (500 qayta tanlash) bilan FP32 va eng yaxshi INT8 recall farqining 95 % ishonch intervalini hisoblang — farq statistik ahamiyatlimi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); PR-egri grafigi va chegara jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

19-variant. Oziq-ovqat sanoati: konveyerda nuqsonli mahsulotni ajratish — ikki sinfli CNN, ORT va 500 dona/s oson

Soha: Oziq-ovqat sanoati

Qandolat fabrikasi konveyeridan soniyasiga 500 ta mahsulot o'tadi; har birining 8x8 kulrang issiqlik xaritasi bo'yicha «soz/nuqsonli» qarori 4 ms ichida chiqishi kerak, aks holda ajratuvchi mexanizm ulgurmaydi. Siz digits to'plamini ikki sinfga aylantirasiz (masalan 0,6,8,9 raqamlari «nuqsonli» surrogati), kengroq CNN o'qitasiz, ONNX eksport qilasiz, ORT darajalari va oqimlarini o'lchaysiz, paketlash orqali throughput'ni oshirasiz va Little qonuni bilan 500 dona/s uchun nusxalar sonini hisoblaysiz.

Model	CNN Conv(1->32)-BN-ReLU-Pool-Conv(32->64)-BN-ReLU-Pool-FC 256-64-2
Ma'lumot	sklearn digits -> 2 sinf (0,6,8,9 = nuqsonli), stratifikatsiya, random_state=31
Epoxa	4
SLO	p99 <= 4 ms (batch 1), 500 dona/s
Batch	1, 4, 16, 64
Oqimlar	1 va 2
Metrika	aniqlik, nuqsonli sinf recall

QADAMLAR

1. Digits'ni ikki sinfga aylantiring, CNN'ni o'qiting; aniqlik va nuqsonli sinf recall'ini yozing; parametrlar soni.
2. ONNX eksport (dinamik batch, opset 17), onnx.checker, BatchNorm birlashganini tugunlar ro'yxatida ko'rsating; PyTorch bilan moslik.
3. Laboratoriya s2 kabi ORT 4 ta darajasini batch 1 da (takror 500) o'lchang, tugunlar soni; PyTorch eager bilan tezlanish.
4. Batch 1/4/16/64 uchun p50/p99 va throughput; 1 va 2 oqim batch 64 da.
5. Little qonuni: 500 dona/s uchun batch 1 da nechta nusxa; batch 4 da paket to'lish kutishi (4/500 s) qo'shilganda p99 SLO ga sig'adimi — hisoblang.
6. Grafik va xulosa: konveyer kompyuteriga konfiguratsiya tavsiyasi; kanallar 32/64 bo'lgani uchun model lab CNN'idan necha marta sekin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

20-variant. Konchilik: NIR spektr bo'yicha ruda tarkibi regressiyasi — to'liq optimallashtirish zanjiri va ishonch intervallari qiyin

Soha: Konchilik

Boyitish fabrikasida konveyer ustidagi spektrometr har 0,5 soniyada 200 kanalli NIR spektr beradi, model ruda tarkibidagi foydali komponent foizini bashorat qiladi va natija flotatsiya reagentini boshqaradi. Model fabrikadagi sanoat kompyuteriga (CPU, GPU yo'q) joylanadi; talablar: RMSE 0,8 % dan oshmasin, p99 20 ms, 6 ta spektrometr. Siz laboratoriyaning 1-6 qadamlarini to'liq takrorlaysiz: etalon va eksport, ORT darajalari va paket, PyTorch dinamik kvantlash, ORT dinamik va statik kvantlash (3 usul), yakuniy jadval va Pareto, energiya bahosi; qo'shimcha ravishda kechikish o'lchovlariga bootstrap ishonch intervallari qo'shasiz.

Model	MLP 200-256-64-1 (regressiya) va 1D CNN Conv1d(1->8,k11)-Pool-Conv1d(8->16,k7)-GAP-Linear (ikki arxitektura)
Ma'lumot	sintetik NIR: 200 kanal, 4 ta Gauss yutilish cho'qqisi, amplitudasi tarkib foizi bilan chiziqli + bazaviy chiziq siljishi + shovqin; 8000 spektr, tarkib 2..40 %
Metrika	RMSE, MAE (foiz birligida)
SLO	p99 <= 20 ms, 12 spektr/s jami
Kalibrlash	MinMax, Percentile, Entropy; 400 spektr
Quvvat modeli	P_bosh 6 Vt, P_to'liq 20 Vt
Statistika	bootstrap 1000 qayta tanlash, 95 % II (p50, p99)
Batch	1, 6, 64

QADAMLAR

1. Sintetik NIR generatorini yozing, ikkala arxitekturani o'qiting, RMSE/MAE; ikkala modelni ONNX'ga eksport qiling va moslikni tekshiring (s1).
2. Ikkala model uchun ORT 4 daraja batch 1, batch 1/6/64 throughput, 1 va 2 oqim (s2); har o'lchovni 5 marta takrorlab p50/p99 uchun bootstrap 95 % ishonch intervalini hisoblang.
3. MLP uchun PyTorch dinamik kvantlash (s3): formula tekshiruvi, hajm, RMSE, tezlik.
4. Ikkala model uchun ORT dinamik va statik kvantlash 3 usul bilan (s4): hajm, RMSE, p50, Q/DQ tugunlar; kalibrlash to'plamini faqat past tarkibli (2..10 %) spektrlardan olib qayta kvantlang — yuqori tarkibda xato qanday o'sadi.
5. Yakuniy jadval (kamida 10 konfiguratsiya) va Pareto (p50 - RMSE), grafikda ishonch intervallarini xato chiziqlari sifatida ko'rsating (s5).
6. Energiya bahosi (s6) sanoat kompyuteri quvvat parametrlari bilan: FP32/INT8, 1/2 oqim, batch 1/6; 6 spektrometr uchun yillik kVt-soat va narx (0,08 USD/kVt-soat).
7. 4-6 betlik hisobot: fabrika uchun konfiguratsiya tavsiyasi, statistik ahamiyatli farqlar qaysilar, «GPU bo'lsa nima o'zgaradi» bo'limi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Yakuniy jadval (>= 10 konfiguratsiya) ishonch intervallari bilan; 4-6 betlik hisobot (PDF)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A4

Edge AI va Cloud AI platformalarida sun'iy intellekt tizimlarini joylashtirish

Bajarish bo‘yicha qo‘llanma: 6 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A4
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A4
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

KTM laboratoriyalari — Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

Laboratoriya haqida

Bu virtual laboratoriyada siz inferens modelini xizmat sifatida joylashtirish zanjirini jarayon ichida bajarasiz: FastAPI xizmati (/healthz, /readyz, /predict, /metrics) `TestClient` bilan sinaladi va yuklama ostida o'lchanadi, Dockerfile va Kubernetes manifestlari (Deployment/Service/HPA/ConfigMap) `pyyaml` bilan tahlil qilinib muhandislik qoidalariga tekshiriladi, HPA avtomasshtablash navbat modeli bilan simulyatsiya qilinadi, Cloud/Edge/Gibrid arxitekturalar TCO va kechikish kalkulyatori bilan solishtiriladi. Real labdan farqi: Docker/minikube/Jetson/Coral yo'q — konteyner va klaster o'rniga jarayon ichidagi xizmat va simulyatsiya; model — A3 dagi digits MLP (ONNX).

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A4`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A4 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. FastAPI inferens xizmati: probe'lar, predict, metrikalar
2. 2-qadam. Yuklama sinovi: ketma-ket va parallel so'rovlar, Little qonuni, paketlash
3. 3-qadam. Dockerfile tahlili va tasvir hajmini baholash
4. 4-qadam. Kubernetes manifestlari: pyyaml bilan validatsiya va 18 qoida
5. 5-qadam. HPA avtomasshtablash simulyatsiyasi (navbat modeli)
6. 6-qadam. Cloud vs Edge vs Gibrid: kechikish, trafik, energiya va TCO kalkulyatori
7. Qo‘shimcha mashqlar (10)
8. Mustaqil topshiriqlar — 20 variant

1 1-qadam. FastAPI inferens xizmati: probe'lar, predict, metrikalar

MAQSAD

Kichik MLP ONNX'ga eksport qilinadi va FastAPI xizmatiga o'raladi: `lifespan` ichida model yuklanadi va isitiladi, `/healthz` (liveness) va `/readyz` (readiness) ajratilgan, `/predict` JSON so'rovni qabul qilib top-3 va vaqt taqsimotini (preprocess/inference) qaytaradi, `/metrics` Prometheus matn formatida so'rovlar, xatolar, kechikish kvantillarini beradi. TestClient model yuklanmasdan oldin (`/readyz` → 503) va keyin (`with` blok) chaqiriladi.

> **Kutilgan natija:** Yuklanmasdan oldin /readyz 503, /healthz 200; keyin 200; 5 namunadan 4–5 tasi to'g'ri, 200 namunada ~90 % aniqlik; noto'g'ri so'rovlar 422; /metrics matni.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. FastAPI inferens xizmati: /healthz, /readyz, /predict, /metrics — jarayon ichida
TestClient bilan
import warnings; warnings.filterwarnings("ignore")
import os, time, json, numpy as np, torch, torch.nn as nn, onnxruntime as ort
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from fastapi.responses import PlainTextResponse
from fastapi.testclient import TestClient
from pydantic import BaseModel
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# --- A3 dan "optimallashtirilgan model": kichik MLP -> ONNX (digits, 8x8). Real labda bu
resnet50_int8.onnx
X, y = load_digits(return_X_y=True); X = (X / 16.0).astype(np.float32)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
model = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 10))
opt = torch.optim.Adam(model.parameters(), 3e-3); lossf = nn.CrossEntropyLoss()
Ttr, Tytr = torch.tensor(Xtr), torch.tensor(ytr)
for ep in range(4):
    perm = torch.randperm(len(Ttr))
    for i in range(0, len(Ttr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lossf(model(Ttr[idx]), Tytr[idx]).backward();
opt.step()
model.eval()
torch.onnx.export(model, Ttr[:1], "digits.onnx", input_names=["input"], output_names=
["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)

# --- 12 omilli ilova: barcha sozlamalar muhit o'zgaruvchilaridan (Docker/K8s'da env orqali
beriladi)
MODEL_PATH = os.getenv("MODEL_PATH", "digits.onnx")
ORT_THREADS = int(os.getenv("ORT_THREADS", "2"))
state = {"ready": False, "session": None, "sorovlar": 0, "xatolar": 0, "kechikish": []}

@asynccontextmanager
async def lifespan(app):
    """startup: model yuklash + isinish; shutdown: tozalash. K8s readiness probe shu holatga
```

```
qaraydi."""
    t0 = time.perf_counter()
    so = ort.SessionOptions(); so.intra_op_num_threads = ORT_THREADS
    so.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
    state["session"] = ort.InferenceSession(MODEL_PATH, so, providers=["CPUExecutionProvider"])
    state["session"].run(None, {"input": np.zeros((1, 64), np.float32)})      # isinish —
birinchi so'rov sekin bo'lmasin
    state["ready"] = True
    print(f"[startup] model yuklandi: {MODEL_PATH}, {ORT_THREADS} oqim, {(time.perf_counter() -
t0) * 1e3:.0f} ms")
    yield
    state["ready"] = False
    print("[shutdown] xizmat to'xtadi")

app = FastAPI(title="digits-infer", lifespan=lifespan)

class Sorov(BaseModel):
    pixels: list[float]          # 64 ta qiymat, 0..16

@app.get("/healthz")             # liveness: jarayon tirik
def healthz():
    return {"ok": True}

@app.get("/readyz")              # readiness: model yuklangan va so'rov qabul qila oladi
def readyz():
    if not state["ready"]:
        raise HTTPException(503, "model not loaded")
    return {"ready": True, "model": os.path.basename(MODEL_PATH), "threads": ORT_THREADS}

@app.post("/predict")
def predict(s: Sorov):
    if not state["ready"]:
        raise HTTPException(503, "not ready")
    if len(s.pixels) != 64:
        state["xatolar"] += 1
        raise HTTPException(422, "64 ta piksel kerak")
    t0 = time.perf_counter()
    x = (np.asarray(s.pixels, np.float32) / 16.0)[None]                          # oldindan
ishlov
    t1 = time.perf_counter()
    logits = state["session"].run(None, {"input": x})[0][0]                      # inferens
    t2 = time.perf_counter()
    top = logits.argsort()[-3:][::-1]
    state["sorovlar"] += 1; state["kechikish"].append((t2 - t0) * 1e3)
    return {"top3": [{"label": int(i), "score": round(float(logits[i]), 3)} for i in top],
            "timing_ms": {"preprocess": round((t1 - t0) * 1e3, 3), "inference": round((t2 - t1)
* 1e3, 3)}}

@app.get("/metrics", response_class=PlainTextResponse)   # Prometheus matn formati (4 signal:
so'rovlar, xatolar, kechikish, resurs)
def metrics():
    k = state["kechikish"] or [0.0]
    return (f"# TYPE infer_requests_total counter\ninfer_requests_total {state['sorovlar']}\n"
            f"# TYPE infer_errors_total counter\ninfer_errors_total {state['xatolar']}\n"
            f"# TYPE infer_latency_ms summary\ninfer_latency_ms{{quantile=\"0.5\"}}
{np.percentile(k, 50):.3f}\n"
            f"infer_latency_ms{{quantile=\"0.95\"}} {np.percentile(k, 95):.3f}\n"
            f"# TYPE infer_ready gauge\ninfer_ready {int(state['ready'])}\n")

print("=== FastAPI xizmati ===")
# Lifespan ishga tushmagan mijoz — Pod endi yaratilgan, model hali yuklanmagan holat
sovuq = TestClient(app)
r = sovuq.get("/readyz"); print(f"Model yuklanmasdan oldin GET /readyz -> {r.status_code}
```

```
{r.json()}  (K8s trafik yubormaydi)")
print(f"                        GET /healthz -> {sovuq.get('/healthz').status_code}  (Pod
tirik — o'ldirilmaydi)")

with TestClient(app) as c:            # with-blok lifespan'ni ishga tushiradi (startup -> yield
-> shutdown)
    print("GET /readyz ->", c.get("/readyz").status_code, c.get("/readyz").json())
    print("\nPOST /predict (5 ta test namunasi):")
    togri = 0
    for i in range(5):
        r = c.post("/predict", json={"pixels": (Xte[i] * 16).tolist()})
        j = r.json(); bashorat = j["top3"][0]["label"]; togri += bashorat == yte[i]
        print(f"  #{i} haqiqiy {yte[i]}  bashorat {bashorat}  {'OK' if bashorat == yte[i] else
'XATO'}  timing {j['timing_ms']}")
    print("Noto'g'ri so'rov (3 ta piksel) ->", c.post("/predict", json={"pixels": [1, 2,
3]}).status_code)
    print("Yaroqsiz JSON             ->", c.post("/predict", json={"foo": 1}).status_code)
    # Butun test to'plami bo'yicha aniqlik — xizmat orqali
    acc = np.mean([c.post("/predict", json={"pixels": (Xte[i] * 16).tolist()}).json()["top3"]
[0]["label"] == yte[i] for i in range(200)])
    print(f"\nXizmat orqali 200 namunada aniqlik: {acc * 100:.1f} %")
    print("\nGET /metrics:"); print(c.get("/metrics").text)
    print("OpenAPI yo'llari:", sorted(p for p in c.get("/openapi.json").json()["paths"]))
print("\nIzoh: real labda bu xizmat uvicorn bilan 8000-portda, Docker ichida ishlaydi; bu yerda
TestClient uni jarayon ichida chaqiradi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== FastAPI xizmati ===`, `GET /readyz -> 200`, `infer_requests_total`, `OpenAPI yo'llari`

TAJRIBA QILIB KO‘RING

- `/predict` ga `batch` maydonli variant qo'shing (bir so'rovda N ta rasm).
- `ORT_THREADS` ni `os.environ` orqali 1 ga qo'ying — timing qanday o'zgaradi?
- `/metrics` ga xotira (`psutil.Process().memory_info().rss`) gauge'ini qo'shing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Yuklama sinovi: ketma-ket va parallel so'rovlar, Little qonuni, paketlash

MAQSAD

`hey -c N` o'rniga `ThreadPoolExecutor` bilan 1, 2, 4, 8, 16 parallel mijoz simulyatsiya qilinadi: throughput, p50/p95/p99 va Little qonuni (N = X·R) tekshiriladi; to'liq kechikish ichidagi sof inferens ulushi ko'rsatiladi; 64 ta alohida so'rov va bitta paketli so'rov solishtiriladi; p95 ≤ 50 ms SLO tekshiriladi. Grafik: throughput va kechikish.

> **Kutilgan natija:** c=1: p50 ~2–3 ms, ~400 so'rov/s (sof inferens 0,1 ms — qolgani HTTP/JSON); throughput 2 yadroda to'yinadi, p99 mijozlar bilan chiziqli o'sadi; paket 20–30x tezroq.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Yuklama sinovi: ketma-ket va parallel so'rovlar, p50/p95/p99, throughput, Little
qonuni
import warnings; warnings.filterwarnings("ignore")
import os, time, numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as ort
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from concurrent.futures import ThreadPoolExecutor
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel
from sklearn.datasets import load_digits

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# Model: latency uchun o'qitish shart emas — tasodifiy vaznli MLP -> ONNX
model = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 10)).eval()
torch.onnx.export(model, torch.zeros(1, 64), "digits.onnx", input_names=["input"],
output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)
X, _ = load_digits(return_X_y=True)

state = {"ready": False, "session": None}

@asynccontextmanager
async def lifespan(app):
    so = ort.SessionOptions(); so.intra_op_num_threads = 1       # har so'rov 1 oqim — parallel
so'rovlar bir-biriga xalaqit bermasin
    state["session"] = ort.InferenceSession("digits.onnx", so, providers=
["CPUExecutionProvider"])
    state["ready"] = True
    yield

app = FastAPI(lifespan=lifespan)

class Sorov(BaseModel):
    pixels: list[float]

@app.post("/predict")
def predict(s: Sorov):
    if not state["ready"]: raise HTTPException(503)
```

```
    t0 = time.perf_counter()
    x = (np.asarray(s.pixels, np.float32) / 16.0)[None]
    logits = state["session"].run(None, {"input": x})[0][0]
    return {"label": int(logits.argmax()), "inference_ms": (time.perf_counter() - t0) * 1e3}

@app.post("/predict_batch")            # paketli so'rov: bir HTTP chaqiruvda N ta rasm
def predict_batch(s: list[Sorov]):
    x = np.asarray([q.pixels for q in s], np.float32) / 16.0
    return {"labels": state["session"].run(None, {"input": x})[0].argmax(1).tolist()}

def foiz(t):
    t = np.array(t) * 1e3
    return dict(p50=np.percentile(t, 50), p95=np.percentile(t, 95), p99=np.percentile(t, 99))

print("=== Yuklama sinovi ===")
with TestClient(app) as c:
    body = {"pixels": X[0].tolist()}
    for _ in range(50): c.post("/predict", json=body)                  # isinish

    # 1) Ketma-ket 300 so'rov: to'liq kechikish va uning ichidagi sof inferens
    t, inf = [], []
    for _ in range(300):
        a = time.perf_counter(); r = c.post("/predict", json=body);
t.append(time.perf_counter() - a)
        inf.append(r.json()["inference_ms"])
    f = foiz(t)
    print(f"Ketma-ket (c=1): p50 {f['p50']:.2f} ms  p95 {f['p95']:.2f} ms  p99 {f['p99']:.2f}
ms  -> {1 / np.mean(t):.0f} so'rov/s")
    print(f"  shundan sof inferens p50 {np.percentile(inf, 50):.3f} ms; qolgani ({f['p50'] -
np.percentile(inf, 50):.2f} ms) — HTTP, JSON, pydantic validatsiya")

    # 2) Parallel mijozlar (hey -c N o'rniga ThreadPoolExecutor): throughput va kechikish
    rows = []
    for cc in (1, 2, 4, 8, 16):
        n = 60 * cc
        def bitta(_):
            a = time.perf_counter(); c.post("/predict", json=body); return time.perf_counter()
- a
        a0 = time.perf_counter()
        with ThreadPoolExecutor(cc) as ex: tt = list(ex.map(bitta, range(n)))
        wall = time.perf_counter() - a0; f = foiz(tt); thr = n / wall
        rows.append(dict(mijozlar=cc, sorov_s=round(thr), p50_ms=round(f["p50"], 2),
p95_ms=round(f["p95"], 2), p99_ms=round(f["p99"], 2),
                         little_N=round(thr * np.mean(tt), 1)))
    df = pd.DataFrame(rows); print("\nParallel mijozlar:"); print(df.to_string(index=False))
    print("little_N = throughput x o'rtacha kechikish = tizimdagi so'rovlar soni (Little
qonuni) — mijozlar soniga yaqin bo'lishi kerak.")
    print("Kuzatuv: 2 yadro va GIL tufayli throughput tez to'yinadi, mijozlar ko'paygani sari
faqat navbat (p99) o'sadi.")

    # 3) Paketlash: 64 ta rasm bitta so'rovda vs 64 ta alohida so'rov
    paket = [{"pixels": X[i].tolist()} for i in range(64)]
    a = time.perf_counter()
    for _ in range(10): c.post("/predict_batch", json=paket)
    tb = (time.perf_counter() - a) / 10
    a = time.perf_counter()
    for _ in range(10):
        for q in paket: c.post("/predict", json=q)
    ts = (time.perf_counter() - a) / 10
    print(f"\n64 rasm: alohida so'rovlar {ts * 1e3:.1f} ms, bitta paketli so'rov {tb * 1e3:.1f}
ms -> {ts / tb:.1f}x tezroq (lekin paket to'lguncha kutish kerak).")

    # 4) SLO tekshiruvi
```

```
    SLO = 50.0
    print(f"\nSLO p95 <= {SLO} ms: ", ", ".join(f"c={int(r.mijozlar)}: {'OK' if r.p95_ms <= SLO
else 'BUZILDI'}" for _, r in df.iterrows()))

fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
ax[0].plot(df.mijozlar, df.sorov_s, "o-"); ax[0].set_xlabel("parallel mijozlar");
ax[0].set_ylabel("so'rov/s"); ax[0].set_title("Throughput"); ax[0].grid(alpha=.3)
ax[1].plot(df.mijozlar, df.p50_ms, "o-", label="p50"); ax[1].plot(df.mijozlar, df.p95_ms, "s-",
label="p95"); ax[1].plot(df.mijozlar, df.p99_ms, "^-", label="p99")
ax[1].axhline(SLO, color="r", ls="--", label="SLO"); ax[1].set_xlabel("parallel mijozlar");
ax[1].set_ylabel("ms"); ax[1].set_title("Kechikish"); ax[1].legend(); ax[1].grid(alpha=.3)
plt.tight_layout()
print("\nReal labda: hey -z 30s -c 8 -m POST ... yoki locust; bu yerda TestClient + oqimlar.
Docker --cpus=2 cheklovi sandbox'ning 2 yadrosiga mos.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Yuklama sinovi ===`, `Parallel mijozlar`, `little_N`, `SLO p95`

TAJRIBA QILIB KO‘RING

- SLO ni 10 ms ga tushiring — nechta mijozda buziladi?
- `so.intra_op_num_threads = 2` qiling va parallel natijalarni solishtiring (oqimlar raqobati).
- Har so'rovga 5 ms sun'iy ishlov (`time.sleep`) qo'shib ResNet-50 xizmatini taqlid qiling va Little qonunini qayta tekshiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Dockerfile tahlili va tasvir hajmini baholash

MAQSAD

Docker bo'lmagani uchun Dockerfile matni dasturiy tahlil qilinadi: ko'p bosqichli qurish, qotirilgan versiya, yengil asos, root bo'lmagan USER, pip/apt kesh, HEALTHCHECK, EXPOSE, exec CMD, qatlamlar tartibi, ENV sozlamalar — 11 qoida. Yaxshi (slim + ONNX Runtime) va yomon (pytorch:latest, root) variantlar solishtiriladi; tasvir hajmi, tortish vaqti va 20 tugunli klasterga tarqatish hajmi baholanadi.

> **Kutilgan natija:** Yaxshi variant 11/11, yomon 1/11; hajm bahosi ~270 MB vs ~8 GB (30x); 100 Mbit/s da tortish 22 s vs 11 daqiqa.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Dockerfile tahlili: ko'p bosqichli qurish qoidalari va tasvir hajmini baholash
(Docker'siz)
import re, pandas as pd

# Ikki variant: yaxshi (ko'p bosqichli, root'siz, ORT) va yomon (bitta bosqich, root, to'liq
PyTorch)
YAXSHI = """
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
RUN useradd -m -u 10001 infer && mkdir -p /models && chown infer /models
COPY --from=builder /install /usr/local
WORKDIR /app
COPY app/ /app/
USER infer
ENV MODEL_PATH=/models/digits_int8.onnx ORT_THREADS=2 PYTHONUNBUFFERED=1
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --start-period=20s CMD python -c "import
urllib.request;urllib.request.urlopen('http://127.0.0.1:8000/readyz')"
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "1"]
"""
YOMON = """
FROM pytorch/pytorch:latest
WORKDIR /app
COPY . /app
RUN pip install fastapi uvicorn onnxruntime
RUN apt-get update && apt-get install -y curl
EXPOSE 8000
CMD uvicorn main:app --host 0.0.0.0 --port 8000
"""
# requirements.txt tarkibi (hajm bahosi uchun)
TALABLAR = {"yaxshi": ["fastapi", "uvicorn", "onnxruntime", "numpy", "pydantic"],
            "yomon": ["fastapi", "uvicorn", "onnxruntime", "numpy", "pydantic", "torch",
"torchvision"]}
# Taxminiy o'rnatilgan hajmlar, MB (pip show / du bilan o'lchanadi; CPU g'ildiraklar)
HAJM_MB = {"python:3.12-slim": 130, "python:3.12": 1020, "pytorch/pytorch:latest": 7200,
"nvidia/cuda:12.4.1-runtime-ubuntu22.04": 2900,
           "fastapi": 15, "uvicorn": 10, "onnxruntime": 60, "numpy": 35, "pydantic": 20,
"torch": 900, "torchvision": 25, "apt_curl_kesh": 45}
```

```
def tahlil(df_text):
    satrlar = [l.strip() for l in df_text.strip().splitlines() if l.strip() and not
l.startswith("#")]
    froms = [l for l in satrlar if l.upper().startswith("FROM ")]
    oxirgi_from = satrlar.index(froms[-1]) if froms else 0
    yakuniy = satrlar[oxirgi_from:]                      # faqat oxirgi bosqich tasvirga
kiradi
    asos = froms[-1].split()[1]
    q = {}
    q["Ko'p bosqichli qurish (>=2 FROM)"] = len(froms) >= 2
    q["Asos tasvir 'latest' emas (versiya qotirilgan)"] = not any(f.split()
[1].endswith(":latest") or ":" not in f.split()[1] for f in froms)
    q["Yengil asos tasvir (slim/runtime)"] = any(k in asos for k in ("slim", "runtime",
"alpine", "distroless"))
    q["USER root emas (yakuniy bosqichda)"] = any(l.upper().startswith("USER ") and l.split()
[1] != "root" for l in yakuniy)
    q["pip --no-cache-dir"] = all("--no-cache-dir" in l for l in satrlar if "pip install" in l)
    q["HEALTHCHECK bor"] = any(l.upper().startswith("HEALTHCHECK") for l in yakuniy)
    q["EXPOSE bor"] = any(l.upper().startswith("EXPOSE") for l in yakuniy)
    q["CMD exec (JSON) shaklida"] = any(l.startswith("CMD [") for l in yakuniy)
    q["Bog'liqliklar koddan OLDIN ko'chiriladi (kesh)"] = any("requirements" in l for l in
satrlar) and \
        next((i for i, l in enumerate(satrlar) if "requirements" in l), 99) < next((i for i, l
in enumerate(satrlar) if re.match(r"COPY (app/?|\.) ", l)), 0)
    q["apt keshi tozalangan"] = all("rm -rf /var/lib/apt/lists" in l for l in satrlar if "apt-
get install" in l)
    q["Sozlamalar ENV orqali (12 omil)"] = any(l.startswith("ENV ") and "MODEL_PATH" in l for l
in yakuniy)
    return q, asos

def hajm(asos, paketlar, apt_kesh=False):
    mb = HAJM_MB.get(asos, 500) + sum(HAJM_MB.get(p, 20) for p in paketlar) +
(HAJM_MB["apt_curl_kesh"] if apt_kesh else 0)
    return mb

print("=== Dockerfile tekshiruvi ===")
natija = {}
for nomi, text in (("yaxshi", YAXSHI), ("yomon", YOMON)):
    q, asos = tahlil(text); natija[nomi] = q
    ball = sum(q.values())
    print(f"\n[{nomi.upper()}] asos: {asos}   ball {ball}/{len(q)}")
    for k, v in q.items():
        print(f"  {'PASS' if v else 'FAIL'}  {k}")
jad = pd.DataFrame({"qoida": list(natija["yaxshi"]), "yaxshi": list(natija["yaxshi"].values()),
"yomon": list(natija["yomon"].values())})
print("\nQiyosiy jadval:"); print(jad.replace({True: "+", False: "-"}).to_string(index=False))

print("\n=== Tasvir hajmi bahosi (MB) ===")
rows = [dict(variant="yaxshi: slim + ORT (ko'p bosqichli)", MB=hajm("python:3.12-slim",
TALABLAR["yaxshi"])),
        dict(variant="o'rta: python:3.12 (to'liq) + ORT", MB=hajm("python:3.12",
TALABLAR["yaxshi"])),
        dict(variant="yomon: pytorch/pytorch + hamma narsa", MB=hajm("pytorch/pytorch:latest",
TALABLAR["yomon"], apt_kesh=True)),
        dict(variant="GPU: cuda-runtime + onnxruntime-gpu", MB=hajm("nvidia/cuda:12.4.1-
runtime-ubuntu22.04", TALABLAR["yaxshi"]) + 250)]
dh = pd.DataFrame(rows); dh["nisbat"] = (dh.MB / dh.MB[0]).round(1)
# Tortish vaqti 100 Mbit/s tarmoqda va 20 tugunli klasterga tarqatish
dh["tortish_s_100Mbit"] = (dh.MB * 8 / 100).round(0); dh["20_tugun_GB"] = (dh.MB * 20 /
1024).round(1)
print(dh.to_string(index=False))
print("\nXulosa: inferens uchun PyTorch kerak emas — ONNX Runtime yetarli; yengil asos + ko'p
bosqichli qurish tasvirni ~15x kichraytiradi,")
```

```
print("  bu K8s'da yangi Pod'ning ishga tushish vaqtini (tortish + start) va HPA reaksiyasini bevosita belgilaydi.")
print("Real labda: docker build -t digits-infer:1.0 . && docker images digits-infer && docker run --cpus=2 --memory=1g ...")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Dockerfile tekshiruvi ===`, `[YAXSHI]`, `[YOMON]`, `=== Tasvir hajmi bahosi (MB) ===`

TAJRIBA QILIB KO‘RING

- O'zingizning Dockerfile'ingizni `YAXSHI` o'rniga qo'yib tekshiring.
- Yangi qoida qo'shing: `COPY . .` taqiqlangan (.dockerignore'siz sirlar kiradi).
- GPU varianti uchun `nvidia/cuda:*-devel` asosini qo'shib hajm farqini ko'ring (devel ~2x runtime).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Kubernetes manifestlari: pyyaml bilan validatsiya va 18 qoida

MAQSAD

Deployment/ConfigMap/Service/HPA YAML'i `yaml.safe_load_all` bilan o'qilib 18 qoida bo'yicha tekshiriladi: replicas ≥ 2, selector/label mosligi, requests/limits, readiness va liveness farqi va vaqtlari, runAsNonRoot, RollingUpdate maxUnavailable 0, Service port/selector mosligi, HPA min/max/behavior/target, Prometheus annotatsiyalari. Tipik xatolar bilan yozilgan ikkinchi manifest 2/18 oladi. HPA chegarasi millicore'da hisoblanadi.

> **Kutilgan natija:** To'g'ri manifest 18/18 PASS, xatoli 2/18; requests 1500m × 60 % = 900m chegara; 2 nusxa 95 % da → 4 nusxa.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Kubernetes manifestlari: Deployment / ConfigMap / Service / HPA — pyyaml bilan
tahlil va qoidalar tekshiruvi
import yaml, pandas as pd

MANIFEST = """
apiVersion: apps/v1
kind: Deployment
metadata: { name: digits-infer, labels: { app: digits-infer } }
spec:
  replicas: 2
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
  selector: { matchLabels: { app: digits-infer } }
  template:
    metadata:
      labels: { app: digits-infer }
      annotations: { prometheus.io/scrape: "true", prometheus.io/port: "8000",
prometheus.io/path: "/metrics" }
    spec:
      securityContext: { runAsNonRoot: true, runAsUser: 10001 }
      containers:
        - name: infer
          image: digits-infer:1.0
          imagePullPolicy: IfNotPresent
          ports: [{ containerPort: 8000 }]
          env:
            - { name: MODEL_PATH, valueFrom: { configMapKeyRef: { name: digits-config, key:
model_path } } }
            - { name: ORT_THREADS, value: "2" }
          resources:
            requests: { cpu: "1500m", memory: "600Mi" }
            limits:   { cpu: "2",     memory: "1Gi" }
          readinessProbe: { httpGet: { path: /readyz, port: 8000 }, initialDelaySeconds: 5,
periodSeconds: 5, failureThreshold: 12 }
          livenessProbe:  { httpGet: { path: /healthz, port: 8000 }, initialDelaySeconds: 30,
periodSeconds: 15 }
          volumeMounts: [{ name: models, mountPath: /models, readOnly: true }]
      volumes:
        - name: models
          hostPath: { path: /data/models, type: Directory }
---
```

```
apiVersion: v1
kind: ConfigMap
metadata: { name: digits-config }
data: { model_path: /models/digits_int8.onnx }
---
apiVersion: v1
kind: Service
metadata: { name: digits-infer }
spec:
  type: NodePort
  selector: { app: digits-infer }
  ports: [{ port: 80, targetPort: 8000, nodePort: 30080 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: digits-infer }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: digits-infer }
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 60 } }
  behavior:
    scaleUp:   { stabilizationWindowSeconds: 30 }
    scaleDown: { stabilizationWindowSeconds: 180 }
"""

# "Yangi boshlovchi" manifesti — tipik xatolar bilan
XATOLI = """
apiVersion: apps/v1
kind: Deployment
metadata: { name: digits-infer }
spec:
  replicas: 1
  selector: { matchLabels: { app: digits-infer } }
  template:
    metadata: { labels: { app: digits } }
    spec:
      containers:
        - name: infer
          image: digits-infer:latest
          ports: [{ containerPort: 8000 }]
          livenessProbe: { httpGet: { path: /healthz, port: 8000 }, initialDelaySeconds: 2,
periodSeconds: 5 }
---
apiVersion: v1
kind: Service
metadata: { name: digits-infer }
spec:
  selector: { app: digits-infer }
  ports: [{ port: 80, targetPort: 9000 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: digits-infer }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: digits-infer }
  minReplicas: 1
  maxReplicas: 3
  metrics: [{ type: Resource, resource: { name: cpu, target: { type: Utilization,
averageUtilization: 60 } } }]
"""
```

```
def cpu_m(v):
    """CPU qiymatini millicore'ga: '1500m' -> 1500, '2' -> 2000."""
    v = str(v); return int(v[:-1]) if v.endswith("m") else int(float(v) * 1000)

def tekshir(text):
    obj = {d["kind"]: d for d in yaml.safe_load_all(text) if d}
    q = {}
    dep = obj.get("Deployment", {}); tpl = dep.get("spec", {}).get("template", {}); pod =
tpl.get("spec", {})
    k = pod.get("containers", [{}])[0]; res = k.get("resources", {})
    q["Deployment, Service, HPA, ConfigMap mavjud"] = all(x in obj for x in ("Deployment",
"Service", "HorizontalPodAutoscaler", "ConfigMap"))
    q["replicas >= 2 (bitta nuqta nosozligi yo'q)"] = dep.get("spec", {}).get("replicas", 0) >=
2
    q["selector == template labels"] = dep.get("spec", {}).get("selector",
{}).get("matchLabels") == tpl.get("metadata", {}).get("labels")
    q["image tegi 'latest' emas"] = ":" in k.get("image", "") and not
k["image"].endswith(":latest")
    q["resources.requests (CPU va xotira) — HPA uchun shart"] = all(x in res.get("requests",
{}) for x in ("cpu", "memory"))
    q["resources.limits (CPU va xotira)"] = all(x in res.get("limits", {}) for x in ("cpu",
"memory"))
    q["limits >= requests"] = bool(res.get("limits")) and cpu_m(res["limits"]["cpu"]) >=
cpu_m(res["requests"]["cpu"])
    q["readinessProbe bor"] = "readinessProbe" in k
    q["livenessProbe bor va readiness'dan kechroq boshlanadi"] = "livenessProbe" in k and \
        k["livenessProbe"].get("initialDelaySeconds", 0) >= k.get("readinessProbe",
{}).get("initialDelaySeconds", 0) + 10
    q["readiness va liveness yo'llari har xil"] = k.get("readinessProbe", {}).get("httpGet",
{}).get("path") != k.get("livenessProbe", {}).get("httpGet", {}).get("path")
    q["runAsNonRoot"] = pod.get("securityContext", {}).get("runAsNonRoot") is True
    q["RollingUpdate maxUnavailable: 0 (xizmat to'xtamaydi)"] = dep.get("spec",
{}).get("strategy", {}).get("rollingUpdate", {}).get("maxUnavailable") == 0
    svc = obj.get("Service", {}).get("spec", {})
    q["Service targetPort == containerPort"] = svc.get("ports", [{}])[0].get("targetPort") ==
k.get("ports", [{}])[0].get("containerPort")
    q["Service selector Pod labellariga mos"] = svc.get("selector") == tpl.get("metadata",
{}).get("labels")
    hpa = obj.get("HorizontalPodAutoscaler", {}).get("spec", {})
    q["HPA minReplicas >= 2 va max > min"] = hpa.get("minReplicas", 0) >= 2 and
hpa.get("maxReplicas", 0) > hpa.get("minReplicas", 0)
    q["HPA behavior (stabilizatsiya oynalari) bor"] = "behavior" in hpa
    q["HPA nishoni Deployment nomiga mos"] = hpa.get("scaleTargetRef", {}).get("name") ==
dep.get("metadata", {}).get("name")
    q["Prometheus scrape annotatsiyalari"] = tpl.get("metadata", {}).get("annotations",
{}).get("prometheus.io/scrape") == "true"
    return q, obj

print("=== Kubernetes manifest tekshiruvi ===")
jami = {}
for nomi, text in (("to'g'ri", MANIFEST), ("xatoli", XATOLI)):
    q, obj = tekshir(text); jami[nomi] = q
    print(f"\n[{nomi.upper()}] ob'ektlar: {list(obj)}   ball {sum(q.values())}/{len(q)}")
    for k, v in q.items():
        print(f"  {'PASS' if v else 'FAIL'}  {k}")

print("\nQiyosiy jadval:")
jad = pd.DataFrame({"qoida": list(jami["to'g'ri"]), "to'g'ri": list(jami["to'g'ri"].values()),
"xatoli": list(jami["xatoli"].values())})
print(jad.replace({True: "+", False: "-"}).to_string(index=False))

# HPA hisobi: foiz requests'ga nisbatan — chegara qancha millicore?
```

```
obj = {d["kind"]: d for d in yaml.safe_load_all(MANIFEST)}
req = cpu_m(obj["Deployment"]["spec"]["template"]["spec"]["containers"][0]["resources"]
["requests"]["cpu"])
hpa = obj["HorizontalPodAutoscaler"]["spec"]; target = hpa["metrics"][0]["resource"]["target"]
["averageUtilization"]
print(f"\n=== HPA hisobi ===\nrequests.cpu = {req}m, nishon {target} % -> masshtablash
chegarasi har Pod'da {req * target // 100}m CPU.")
print(f"Formula: kerakli_nusxa = ceil(joriy_nusxa x joriy_foiz / {target}). Masalan 2 nusxa 95
% da -> ceil(2 x 95/60) = {-(-2 * 95 // 60)} nusxa.")
print(f"Klaster sig'imi: maxReplicas {hpa['maxReplicas']} x limits 2 CPU = {hpa['maxReplicas']
* 2} vCPU kerak bo'lishi mumkin — tugunlar yetarlimi, tekshiring.")
print("\nReal labda: kubectl apply -f k8s/ && kubectl rollout status deployment/digits-infer &&
kubectl get hpa -w")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Kubernetes manifest tekshiruvi ===`, `[TO'G'RI]`, `[XATOLI]`, `=== HPA hisobi ===`

TAJRIBA QILIB KO‘RING

- GPU varianti uchun `limits: {nvidia.com/gpu: 1}` qo'shing va «GPU Pod'da HPA CPU bo'yicha ishlamaydi» qoidasini yozing.
- PodDisruptionBudget va Ingress ob'ektlarini qo'shib, ularga 2 tadan qoida yozing.
- `kubectl apply --dry-run=client` nima tekshiradi, sizning skriptingiz nima tekshiradi — farqni 3 jumlada yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. HPA avtomasshtablash simulyatsiyasi (navbat modeli)

MAQSAD

Minikube o'rniga diskret-vaqtli simulyatsiya: so'rov oqimi λ(t) (40 → 340 → 40 so'rov/s), har nusxa sig'imi μ = 1000/14 ms, CPU foizi = λ/(R·μ), HPA har 15 s da `ceil(R·cpu/nishon)` hisoblaydi, scaleUp/scaleDown stabilizatsiya oynalari va Pod start vaqti (20 s) hisobga olinadi, navbat va p95 kechikish Little qonuni bilan baholanadi. 7 ta konfiguratsiya (nishon 40/60/80 %, minReplicas 4, sekin start 60 s, maxReplicas 3, oynasiz) SLO buzilish vaqti va pod-soniya narxi bo'yicha solishtiriladi.

> **Kutilgan natija:** Asosiy konfiguratsiyada reaksiya ~65 s, SLO ~200 s buziladi; minReplicas 4 SLO buzilishini 4x kamaytiradi, narx +23 %; maxReplicas 3 — eng yomon.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. HPA avtomasshtablash simulyatsiyasi: so'rov oqimi -> CPU foizi -> nusxalar soni ->
navbat va kechikish
import math, numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

# --- Parametrlar (K8s manifestidan va 2-qadam o'lchovlaridan). O'zgartirib ko'ring!
XIZMAT_MS = 14.0            # bitta so'rovga xizmat vaqti (inferens + ishlov), ms  -> sig'im
1000/14 ~ 71 so'rov/s/nusxa
MU = 1000.0 / XIZMAT_MS
MIN_R, MAX_R = 2, 6         # HPA minReplicas / maxReplicas
NISHON = 60                 # averageUtilization, %
UP_OYNA, DOWN_OYNA = 30, 180   # stabilizationWindowSeconds
HPA_DAVR = 15               # HPA metrikani necha soniyada bir o'qiydi
START_S = 20                # yangi Pod tayyor bo'lgunicha (tasvir tortish + model yuklash +
readiness)
SLO_MS = 200                # p95 kechikish talabi
T = 900                     # simulyatsiya davomiyligi, s

def yuklama(t):
    """So'rov oqimi, so'rov/s: tinch -> keskin o'sish -> plato -> yana pasayish (video
kuzatuvda smena boshlanishi kabi)."""
    if t < 120: return 40
    if t < 180: return 40 + (t - 120) * 5          # 60 s ichida 40 -> 340
    if t < 480: return 340
    if t < 540: return 340 - (t - 480) * 5
    return 40

def simulyatsiya(min_r=MIN_R, max_r=MAX_R, nishon=NISHON, start_s=START_S, up_oyna=UP_OYNA,
down_oyna=DOWN_OYNA):
    tayyor, kutilayotgan = min_r, []            # kutilayotgan: [tayyor bo'lish vaqti, ...]
    navbat = 0.0; tavsiyalar = []; log = []
    for t in range(T):
        kutilayotgan_yangi = [x for x in kutilayotgan if x > t]
        tayyor += len(kutilayotgan) - len(kutilayotgan_yangi); kutilayotgan =
kutilayotgan_yangi
        lam = yuklama(t); sigim = tayyor * MU
        # Navbat dinamikasi (1 s qadam): kelgan - xizmat ko'rsatilgan
        navbat = max(0.0, navbat + lam - sigim)
        cpu = min(lam / sigim, 1.0) * 100 if sigim else 100     # foiz requests'ga nisbatan
(requests ~= limits deb olamiz)
        kutish_ms = navbat / sigim * 1000 if sigim else 1e4     # Little: navbatdagi so'rov
kutish vaqti = N / throughput
```

```
        p95_ms = XIZMAT_MS + kutish_ms + (XIZMAT_MS * 2 if cpu > 90 else 0)   # to'yinganda p95
keskin o'sadi (empirik qo'shimcha)
        # HPA boshqaruvchisi
        if t % HPA_DAVR == 0:
            istalgan = math.ceil(tayyor * cpu / nishon) if tayyor else min_r
            istalgan = min(max(istalgan, min_r), max_r)
            tavsiyalar.append((t, istalgan))
            jami = tayyor + len(kutilayotgan)
            if istalgan > jami:        # scaleUp: oxirgi up_oyna ichidagi tavsiyalarning ENG
KICHIGI (keskin sakrashlardan himoya)
                xavfsiz = min(v for tt, v in tavsiyalar if tt >= t - up_oyna)
                if xavfsiz > jami:
                    kutilayotgan += [t + start_s] * (xavfsiz - jami)
            elif istalgan < jami:      # scaleDown: oxirgi down_oyna ichidagi ENG KATTASI (erta
qisqartirishdan himoya)
                xavfsiz = max(v for tt, v in tavsiyalar if tt >= t - down_oyna)
                if xavfsiz < jami:
                    tayyor = max(min_r, xavfsiz)
        log.append(dict(t=t, lam=lam, tayyor=tayyor, cpu=cpu, navbat=navbat, p95_ms=p95_ms))
    df = pd.DataFrame(log)
    return df, dict(max_nusxa=int(df.tayyor.max()), max_p95_ms=round(df.p95_ms.max()),
SLO_buzilgan_s=int((df.p95_ms > SLO_MS).sum()),
                    max_navbat=int(df.navbat.max()), pod_soniya=int(df.tayyor.sum()),
                    reaksiya_s=int(df[df.tayyor > min_r].t.min() - 120) if (df.tayyor >
min_r).any() else None)

print("=== HPA simulyatsiyasi ===")
print(f"Nusxa sig'imi: {MU:.0f} so'rov/s; cho'qqi yuklama 340 so'rov/s -> kamida {math.ceil(340
/ MU / (NISHON / 100))} nusxa kerak ({NISHON} % nishonda)")
asosiy, x = simulyatsiya()
print("\nAsosiy konfiguratsiya (min 2, max 6, nishon 60 %, start 20 s):")
for k, v in x.items(): print(f"  {k}: {v}")
print("\nVaqt bo'yicha (har 60 s):")
print(asosiy[asosiy.t % 60 == 0][["t", "lam", "tayyor", "cpu", "navbat",
"p95_ms"]].round(0).to_string(index=False))

# Konfiguratsiyalarni solishtirish
print("\n=== Konfiguratsiyalar taqqosi ===")
rows = []
for nomi, kw in (("asosiy: min2 max6 nishon60 start20", {}),
                 ("nishon 80 % (tejamkor)", dict(nishon=80)),
                 ("nishon 40 % (zaxirali)", dict(nishon=40)),
                 ("minReplicas 4 (oldindan zaxira)", dict(min_r=4)),
                 ("sekin start 60 s (GPU Pod, TensorRT)", dict(start_s=60)),
                 ("maxReplicas 3 (klaster kichik)", dict(max_r=3)),
                 ("scaleUp oynasi 0 s (tez reaksiya)", dict(up_oyna=0))):
    _, r = simulyatsiya(**kw); rows.append(dict(konfiguratsiya=nomi, **r))
dk = pd.DataFrame(rows); dk["narx_nisbati"] = (dk.pod_soniya / dk.pod_soniya[0]).round(2)
print(dk.to_string(index=False))
print("\nXulosa: HPA reaktiv — yuklama o'sgach, metrika davri + stabilizatsiya + Pod start
vaqti o'tgachgina sig'im paydo bo'ladi;")
print("  shu oraliqda navbat to'planib p95 SLO buziladi. Yechimlar: minReplicas zaxirasi (narx
oshadi), pastroq nishon, tez start (kichik tasvir),")
print("  yoki so'rov/s bo'yicha prognozli masshtablash (KEDA). GPU Pod'larda start 30-60 s —
muammo yanada keskin.")

fig, ax = plt.subplots(3, 1, figsize=(9, 7.5), sharex=True)
ax[0].plot(asosiy.t, asosiy.lam, label="so'rov/s"); ax[0].plot(asosiy.t, asosiy.tayyor * MU, "-
-", label="sig'im (nusxa x mu)"); ax[0].set_ylabel("so'rov/s"); ax[0].legend();
ax[0].grid(alpha=.3)
ax[1].step(asosiy.t, asosiy.tayyor, where="post", color="tab:green", label="tayyor nusxalar");
ax[1].set_ylabel("nusxa"); ax[1].set_ylim(0, MAX_R + 1)
ax1b = ax[1].twinx(); ax1b.plot(asosiy.t, asosiy.cpu, color="tab:gray", alpha=.6, label="CPU
```

```
%"); ax1b.axhline(NISHON, color="tab:gray", ls=":"); ax1b.set_ylabel("CPU %");
ax[1].legend(loc="upper left"); ax[1].grid(alpha=.3)
ax[2].plot(asosiy.t, asosiy.p95_ms, color="tab:red"); ax[2].axhline(SLO_MS, color="k", ls="--",
label=f"SLO {SLO_MS} ms"); ax[2].set_yscale("log"); ax[2].set_ylabel("p95, ms");
ax[2].set_xlabel("vaqt, s"); ax[2].legend(); ax[2].grid(alpha=.3)
fig.suptitle("HPA simulyatsiyasi: yuklama, nusxalar, kechikish"); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== HPA simulyatsiyasi ===`, `=== Konfiguratsiyalar taqqosi ===`, `narx_nisbati`

TAJRIBA QILIB KO‘RING

- `yuklama(t)` ni sinusoidal (kunlik sikl) qiling — HPA qanday kuzatadi?
- So'rov/s bo'yicha masshtablash (KEDA) qo'shing: istalgan = ceil(λ/(μ·0,6)) — CPU bilan solishtiring.
- Navbat chegarasi qo'shing (masalan 500): ortiqcha so'rovlar 503 bilan rad etilsin va rad etilganlar sonini hisoblang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Cloud vs Edge vs Gibrid: kechikish, trafik, energiya va TCO kalkulyatori

MAQSAD

A1 SHV ssenariysi (16 kamera, 160 inferens/s, 3 yil) uchun uch arxitektura hisoblanadi: uchdan-uchga kechikish (RTT + navbat + inferens), tarmoq trafigi, mJ/inferens (PUE bilan), TCO (GPU soati, egress, qo'shimcha xizmatlar; Jetson narxi, energiya, texnik xizmat, zaxira). TCO 1/2/3/5 yil, sezgirlik tahlili (kameralar x2/x8, bulut narxi −30 %, hodisa bo'yicha oqim, xizmat narxi, RTT) va kesishish nuqtasi. Jetson/TPU yo'q — bu simulyatsiya, raqamlar o'zgartiriladigan parametrlar.

> **Kutilgan natija:** Cloud TCO Edge'dan 30–50 marta qimmat, kechikish ~6 marta katta; bulut narxi −30 % ham nisbatni saqlaydi; grafik: TCO yillar bo'yicha va sezgirlik ustunlari.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Cloud vs Edge vs Gibrid: kechikish, trafik, energiya va TCO kalkulyatori (A1 SHV
ssenariysi) + sezgirlik tahlili
import math, numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

# --- Ssenariy va narxlar (ochiq narxlar, taxminiy; hisobotda sana va manba ko'rsatiladi).
Hammasini o'zgartirib ko'ring.
S = dict(
    kameralar=16, fps_har_kamera=10,          # 16 kamera x 10 kadr/s = 160 inferens/s
    yillar=3, kvt_soat_usd=0.08, pue_cloud=1.3,
    # Cloud (T4 GPU, K8s)
    gpu_soat_usd=0.50, gpu_sigim_inf_s=380, zaxira_nusxa=2, rtt_ms=80, bulut_qoshimcha_foiz=30,
# egress, LB, loglar, monitoring
    oqim_mbit_kamera=4.0, egress_usd_gb=0.09, gpu_quvvat_w=70, inferens_ms_cloud=3.0,
navbat_ms_cloud=25,
    # Edge (Jetson Orin NX)
    edge_qurilma_usd=700, edge_korpus_usd=300, edge_quvvat_w=25, edge_sigim_inf_s=380,
edge_inferens_ms=19,
    edge_xizmat_usd_yil=150, edge_zaxira=1,     # texnik xizmat + bitta zaxira qurilma
    # Gibrid: chekkada hamma, bulutga shubhali kadrlar ulushi
    shubhali_ulush=0.05,
)

def hisob(S):
    inf_s = S["kameralar"] * S["fps_har_kamera"]
    soat = S["yillar"] * 365 * 24
    # ---- Cloud
    gpu_n = max(S["zaxira_nusxa"], math.ceil(inf_s / S["gpu_sigim_inf_s"]))
    gpu_usd = gpu_n * S["gpu_soat_usd"] * soat
    trafik_mbit = S["kameralar"] * S["oqim_mbit_kamera"]                                #
uzluksiz oqim
    egress_gb = trafik_mbit / 8 / 1024 * 3600 * soat
    cloud_usd = (gpu_usd + egress_gb * S["egress_usd_gb"]) * (1 + S["bulut_qoshimcha_foiz"] /
100)
    cloud_lat = S["rtt_ms"] + S["navbat_ms_cloud"] + S["inferens_ms_cloud"]
    cloud_mj = S["gpu_quvvat_w"] * gpu_n / inf_s * 1000 * S["pue_cloud"]                #
to'liq (GPU band bo'lmasa ham ishlaydi)
    # ---- Edge
    edge_n = math.ceil(inf_s / S["edge_sigim_inf_s"])
    kwh = S["edge_quvvat_w"] * edge_n * soat / 1000
```

```
    edge_usd = (edge_n + S["edge_zaxira"]) * (S["edge_qurilma_usd"] + S["edge_korpus_usd"]) +
kwh * S["kvt_soat_usd"] + S["edge_xizmat_usd_yil"] * S["yillar"] * edge_n
    edge_lat = S["edge_inferens_ms"]
    edge_mj = S["edge_quvvat_w"] * edge_n / inf_s * 1000
    # ---- Gibrid: Edge + bulutda faqat shubhali kadrlar (kichik GPU ulushi, egress kam)
    sh_inf_s = inf_s * S["shubhali_ulush"]
    gib_gpu_usd = 1 * S["gpu_soat_usd"] * soat * max(S["shubhali_ulush"] * 4, 0.1)        # 1
GPU qisman band (serverless/spot taxmini)
    gib_egress_gb = egress_gb * S["shubhali_ulush"] / 10                                   #
faqat kesilgan kadrlar
    gib_usd = edge_usd + (gib_gpu_usd + gib_egress_gb * S["egress_usd_gb"]) * (1 +
S["bulut_qoshimcha_foiz"] / 100)
    return pd.DataFrame([
        dict(arxitektura="Cloud (T4, K8s)", kechikish_p95_ms=cloud_lat,
trafik_mbit_s=round(trafik_mbit, 1), TCO_usd=round(cloud_usd),
             mJ_inferens=round(cloud_mj), oflayn="yo'q", maxfiylik="video chiqadi", nusxa=f"
{gpu_n} GPU"),
        dict(arxitektura="Edge (Jetson Orin NX)", kechikish_p95_ms=edge_lat,
trafik_mbit_s=0.01, TCO_usd=round(edge_usd),
             mJ_inferens=round(edge_mj), oflayn="ha", maxfiylik="video binoda", nusxa=f"
{edge_n}+{S['edge_zaxira']} qurilma"),
        dict(arxitektura="Gibrid (Edge + bulut tekshiruv)", kechikish_p95_ms=f"{edge_lat} /
{cloud_lat}", trafik_mbit_s=round(trafik_mbit * S["shubhali_ulush"] / 10, 2),
             TCO_usd=round(gib_usd), mJ_inferens=round(edge_mj * 1.1), oflayn="asosiy ha",
maxfiylik="faqat kadrlar", nusxa=f"{edge_n} qurilma + 1 GPU qisman"),
    ])

print("=== Cloud vs Edge ===")
print(f"Ssenariy: {S['kameralar']} kamera x {S['fps_har_kamera']} kadr/s = {S['kameralar'] *
S['fps_har_kamera']} inferens/s, {S['yillar']} yil")
df = hisob(S); print(df.to_string(index=False))
nisbat = df.TCO_usd[0] / df.TCO_usd[1]
print(f"\nCloud TCO Edge'dan {nisbat:.1f} marta qimmat; kechikish {df.kechikish_p95_ms[0] /
df.kechikish_p95_ms[1]:.1f} marta katta.")
print("Cloud mJ/inferens katta, chunki 2 ta zaxira GPU 160 inferens/s da kam band (sig'im
760/s) — bo'sh quvvat ham hisobga kiradi (PUE 1,3 bilan).")

# --- TCO yillar bo'yicha (1, 3, 5): Edge kapital xarajat oldinda, Cloud chiziqli o'sadi
print("\n=== TCO yillar bo'yicha (USD) ===")
rows = []
for y in (1, 2, 3, 5):
    d = hisob({**S, "yillar": y}); rows.append(dict(yil=y, cloud=d.TCO_usd[0],
edge=d.TCO_usd[1], gibrid=d.TCO_usd[2]))
dy = pd.DataFrame(rows); print(dy.to_string(index=False))

# --- Sezgirlik tahlili
print("\n=== Sezgirlik tahlili (3 yil, TCO USD) ===")
rows = []
for nomi, kw in (("asosiy", {}), ("kameralar x2", dict(kameralar=32)), ("kameralar x8 (128)",
dict(kameralar=128)),
                 ("bulut narxi -30 %", dict(gpu_soat_usd=0.35, egress_usd_gb=0.063)), ("hodisa
bo'yicha oqim (0.5 Mbit/kamera)", dict(oqim_mbit_kamera=0.5)),
                 ("Edge xizmat 500 $/yil", dict(edge_xizmat_usd_yil=500)), ("RTT 150 ms (uzoq
mintaqa)", dict(rtt_ms=150))):
    d = hisob({**S, **kw}); rows.append(dict(holat=nomi, cloud=d.TCO_usd[0], edge=d.TCO_usd[1],
gibrid=d.TCO_usd[2],
                                             cloud_edge=round(d.TCO_usd[0] / d.TCO_usd[1], 1),
cloud_p95=d.kechikish_p95_ms[0]))
ds = pd.DataFrame(rows); print(ds.to_string(index=False))

# Qachon Cloud arzonroq? Kameralar soni juda kam bo'lsa (bitta kamera uchun ham Jetson sotib
olinadi) yoki yuklama notekis bo'lsa
print("\n=== Kesishish nuqtasi ===")
```

```
for kam in (1, 2, 4, 8):
    d = hisob({**S, "kameralar": kam, "zaxira_nusxa": 1, "edge_zaxira": 0})
    print(f"  {kam} kamera, zaxirasiz: cloud {d.TCO_usd[0]:>7} $   edge {d.TCO_usd[1]:>6} $   -
> {'Cloud' if d.TCO_usd[0] < d.TCO_usd[1] else 'Edge'} arzon")
print("Xulosa: SHV ssenariysida (uzluksiz 160 inferens/s, maxfiy video) Edge aniq ustun; Cloud
10 000 foydalanuvchili mobil ilova kabi")
print("  notekis, geografik tarqoq yuklamada oqilona. Gibrid: chekkada 95 % qaror, bulutda
sifat nazorati va qayta o'qitish uchun ma'lumot.")
print("Eslatma: bu simulyatsiya — Jetson/TPU/GPU o'lchovlari emas; real labda raqamlar
tegrastats, nvidia-smi va bulut hisob-fakturasidan olinadi.")

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(dy.yil, dy.cloud, "o-", label="Cloud"); ax[0].plot(dy.yil, dy.edge, "s-",
label="Edge"); ax[0].plot(dy.yil, dy.gibrid, "^-", label="Gibrid")
ax[0].set_xlabel("yil"); ax[0].set_ylabel("TCO, USD"); ax[0].set_title("TCO yillar bo'yicha");
ax[0].legend(); ax[0].grid(alpha=.3)
xx = np.arange(len(ds)); w = 0.27
ax[1].bar(xx - w, ds.cloud, w, label="Cloud"); ax[1].bar(xx, ds.edge, w, label="Edge");
ax[1].bar(xx + w, ds.gibrid, w, label="Gibrid")
ax[1].set_xticks(xx); ax[1].set_xticklabels(ds.holat, rotation=25, ha="right", fontsize=7);
ax[1].set_yscale("log"); ax[1].set_ylabel("TCO 3 yil, USD (log)")
ax[1].set_title("Sezgirlik tahlili"); ax[1].legend(); ax[1].grid(axis="y", alpha=.3)
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Cloud vs Edge ===`, `=== TCO yillar bo'yicha (USD) ===`, `=== Sezgirlik tahlili`, `=== Kesishish nuqtasi ===`

TAJRIBA QILIB KO‘RING

- Yuklama notekis (kuniga 2 soat cho'qqi) ssenariysini qo'shing: bulutda GPU faqat ishlaganda to'lansin — kim yutadi?
- Muhandis vaqti (OTA, flot boshqaruvi) xarajatini Edge'ga 1000 qurilma uchun qo'shing.
- Ma'lumotlar markazi PUE 1,1 va 1,5 bilan energiya ustunini qayta hisoblang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Probe'lar ssenariysi** 1-qadam xizmatiga `/readyz` ni vaqtincha 503 ga o'tkazuvchi (model qayta yuklanmoqda) `/admin/reload` endpointi qo'shing. TestClient bilan ketma-ketlikni ko'rsating: ready → reload → 503 → ready. Har holatda K8s nima qilishini (trafik, qayta ishga tushirish) 1 jumlada izohlang.	oson	5
2	**Dockerfile qoidalarini kengaytirish** 3-qadam tahlilchisiga 3 ta yangi qoida qo'shing (masalan: `COPY . .` taqiqi, `ADD` o'rniga `COPY`, `WORKDIR` mavjudligi, `pip` versiyasi qotirilgan `==`). Ikkala Dockerfile'ni qayta baholang va qoidalar jadvalini topshiring.	oson	5
3	**TCO parametrlarini yangilash** 6-qadam kalkulyatoridagi GPU soati, egress va Jetson narxini bugungi ochiq narxlar bilan (manba va sana ko'rsatib) almashtiring, 3 yillik TCO jadvalini qayta hisoblang va nisbat qanday o'zgarganini 2–3 jumlada tushuntiring.	oson	5
4	**Kechikish byudjeti dekompozitsiyasi** 2-qadamda to'liq so'rov kechikishini qismlarga ajrating: JSON parse, pydantic validatsiya, oldindan ishlov, inferens, javob serializatsiyasi (har qismga `perf_counter` qo'ying). c=1 va c=8 uchun qismlar ulushini ustunli diagramma bilan ko'rsating va eng katta qismni kamaytirish usulini (masalan, `orjson`, raw bytes kirish) sinab natija bering.	o'rta	10
5	**Manifest validatori kengaytmasi** 4-qadam tekshiruvchisiga PodDisruptionBudget, Ingress (TLS) va NetworkPolicy ob'ektlarini qo'shing, ularga kamida 6 ta yangi qoida yozing (masalan, PDB minAvailable ≥ 1, Ingress TLS secret mavjud, NetworkPolicy faqat 8000-portga ruxsat). Jami 24+ qoidali hisobot va «to'g'ri» manifestni 100 % PASS holatiga keltiring.	o'rta	10
6	**KEDA uslubida so'rov/s bo'yicha masshtablash** 5-qadam simulyatsiyasiga ikkinchi masshtablash siyosati qo'shing: istalgan = ceil(λ/(μ·0,6)) (so'rov/s bo'yicha, CPU emas). CPU-HPA bilan bir xil yuklama profilida SLO buzilish vaqti, maksimal navbat va pod-soniya narxini solishtiring; GPU xizmatlari uchun qaysi biri to'g'ri ekanini asoslang.	o'rta	10
7	**Rolling update simulyatsiyasi** 5-qadam modeliga yangilanish (rollout) qo'shing: t=400 s da `maxSurge=1, maxUnavailable=0` bilan nusxalar birma-bir almashtiriladi (yangi Pod 20 s da tayyor, keyin eski o'chadi). `maxUnavailable=1` varianti bilan solishtiring: yangilanish davomida sig'im, navbat va SLO buzilishi grafigi; GPU'lar soni cheklangan (2 GPU) holatda qaysi strategiya ishlashini tushuntiring.	o'rta	10
8	**Chekka qurilma simulyatori va gibrid mezon** 1-qadam xizmatining «Edge» nusxasini yarating: ORT 1 oqim, har so'rovga ARM CPU oldindan ishlovini taqlid qiluvchi 15 ms kechikish, 2 Vt quvvat modeli. «Cloud» nusxasi: 2 oqim + 80 ms RTT. Gibrid marshrutlash qoidasini yozing (ishonch <	qiyin	15

	chegara bo'lsa bulutga yuboriladi) va chegarani 0,5–0,99 oralig'ida o'zgartirib, bulutga ketgan so'rovlar ulushi, o'rtacha kechikish, umumiy aniqlik va trafikni grafikda ko'rsating; optimal chegarani asoslang.		
9	**Xizmatga nosozlik kiritish (chaos) sinovi** 2-qadam yuklama sinoviga nosozlik ssenariylari qo'shing: (a) 10 % so'rovlarda sun'iy 500 ms kechikish, (b) 5 % so'rovlarda 500 xato, (c) t=5 s da model qayta yuklash (readyz 503, 2 s). Har ssenariy uchun p50/p95/p99, xatolar ulushi va `/metrics` dagi ko'rsatkichlarni jadvalga yig'ing; mijoz tomonida timeout + 1 marta qayta urinish (retry) strategiyasini qo'shib natijani solishtiring. 4–5 betlik hisobot.	qiyin	15
10	**To'liq joylashtirish hisoboti o'z ssenariyingiz uchun** A1 da tanlagan tizimingiz uchun 1–6 qadamlarni to'liq takrorlang: o'z ONNX modelingiz bilan xizmat (probe'lar, metrikalar), yuklama sinovi (kamida 5 parallellik darajasi), o'z Dockerfile va K8s manifestlaringizni skriptlar bilan 100 % PASS holatiga keltirish, o'z yuklama profilingiz bilan HPA simulyatsiyasi (kamida 3 konfiguratsiya), Cloud/Edge/Gibrid TCO 1/3/5 yil va sezgirlik tahlili. 6–8 betlik hisobot: yakuniy arxitektura tavsiyasi va A1 strukturaviy sxemasining yangilangan varianti bilan.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: rentgen tasvirlarini triaj qilish xizmati — probe'lar, yuklama va kasalxona ichidagi Edge o'rta

Soha: Tibbiyot (radiologiya)

Viloyat kasalxonasi rentgen tasvirlarini «shoshilinch/odatiy» deb saralaydigan xizmatni ishga tushirmoqda; tasvirlar surrogati — digits 8x8 CNN. Ertalab 8:00-10:00 da yuklama 5 so'rov/s dan 60 so'rov/s ga ko'tariladi, qolgan vaqtda 5 so'rov/s. Bemor ma'lumotlari binodan chiqmasligi kerak, shuning uchun bulut varianti faqat anonimlashtirilgan tasvirlar bilan ko'rib chiqiladi. Siz FastAPI xizmatini yozib TestClient bilan sinaysiz, yuklama sinovi o'tkazasiz, Dockerfile va K8s manifestlarini tekshirasiz, HPA ni ertalabki profil bilan simulyatsiya qilasiz va kasalxona serveri (Edge) bilan bulutni TCO va maxfiylik bo'yicha solishtirasiz.

Model	kichik CNN (A3 s1) -> ONNX, 2 sinf (shoshilinch = raqamlar 1,4,7)
Xizmat turi	tasvir tasnifi, /predict 64 piksel
SLO	p95 <= 100 ms, mavjudlik 99,9 %
Yuklama profili	5 -> 60 -> 5 so'rov/s (ertalabki cho'qqi 2 soat, o'sish 10 daqiqa)
Xizmat vaqti (sim)	25 ms
HPA	min 2, max 5, nishon 55 %, start 25 s
TCO	3 yil, kasalxona serveri 2500 USD + 40 Vt vs bulut GPU 0,50 USD/soat, RTT 60 ms

QADAMLAR

1. Laboratoriya s1 kabi xizmatni yozing: /healthz, /readyz, /predict (tasvir -> sinf + ishonch + timing), /metrics; readyz yuklanishdan oldin 503 ekanini TestClient bilan ko'rsating; 200 tasvirda aniqlik.
2. Laboratoriya s2 kabi yuklama sinovi: ketma-ket 300 so'rov (sof inferens ulushi), parallel 1/2/4/8/16 mijoz jadvali (so'rov/s, p50/p95/p99, little_N), SLO p95 <= 100 ms tekshiruvi.
3. Dockerfile tahlili (s3): o'z Dockerfile'ingizni yozing (slim + ORT, root'siz, HEALTHCHECK /readyz) va 11 qoidadan o'tkazing; tasvir hajmi bahosi va kasalxona 100 Mbit/s tarmog'ida tortish vaqti.
4. K8s manifestlari (s4): Deployment/ConfigMap/Service/HPA (min 2, max 5, nishon 55 %) 18 qoidadan 100 % o'tsin; HPA chegarasini millicore'da hisoblang.
5. HPA simulyatsiyasi (s5): yuklama(t) ni ertalabki profilga o'zgartiring (T = 3 soat, 10 daqiqada 5 -> 60), xizmat vaqti 25 ms; asosiy + 3 konfiguratsiya (nishon 40 %, min 3, start 60 s) uchun SLO buzilish vaqti va pod-soniya.
6. TCO (s6): kasalxona serveri (Edge) vs bulut vs gibrid (faqat anonim shubhali tasvirlar bulutga) 1/3/5 yil; kechikish (RTT 60 ms) va maxfiylik ustunlari; xulosa va tavsiya.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Dockerfile va K8s YAML (PASS natijasi bilan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

2-variant. Chakana savdo: mahsulot tavsiya xizmati — top-5, Qora juma yuklamasi va bulut tanlovi o'rta

Soha: Chakana savdo

Onlayn do'kon har sahifa ochilganda foydalanuvchiga 5 ta tavsiya ko'rsatadi; model — foydalanuvchi va mahsulot embeddinglari ustidagi MLP (A3 kabi, sintetik). Odatiy yuklama 150 so'rov/s, Qora juma kuni 20 daqiqa ichida 3000 so'rov/s ga chiqadi va 6 soat davom etadi. Har so'rov 200 nomzodni baholaydi, tavsiya 80 ms ichida kelmasa sahifa tavsiyasiz ochiladi (daromad yo'qoladi). Siz /recommend endpointini yozasiz, yuklama sinovida nomzodlar sonining kechikishga ta'sirini o'lchaysiz, manifestlarni tekshirasiz, HPA ni Qora juma profilida simulyatsiya qilasiz va faqat bulut variantlari (doimiy nusxa vs avtomasshtab vs oldindan rejalashtirilgan masshtab) TCO sini solishtirasiz.

Model	Embedding(20000,16)+Embedding(5000,16)+MLP 32-64-1 -> ONNX
Xizmat turi	tavsiya, /recommend {user_id, item_ids[200]} -> top-5
SLO	p95 <= 80 ms
Yuklama profili	150 -> 3000 so'rov/s (20 daqiqada), 6 soat plato, keyin 150
Xizmat vaqti (sim)	6 ms (200 nomzod)
HPA	min 3, max 40, nishon 65 %, start 30 s
Bulut narxi	vCPU-soat 0,045 USD, nusxa 2 vCPU

QADAMLAR

1. A3 dagi embedding+MLP modelini (sintetik) o'qitib ONNX'ga eksport qiling; FastAPI xizmati: /readyz, /recommend (200 nomzodni paketda baholab top-5 qaytaradi), /metrics (so'rovlar, p95, nomzodlar soni gistogrammasi).
2. Yuklama sinovi: nomzodlar soni 20/50/100/200/500 uchun ketma-ket p50/p95; keyin 200 nomzod bilan parallel 1/2/4/8/16 mijoz jadvali va SLO tekshiruvi.
3. Dockerfile (s3) 11 qoida va K8s manifest (s4) 18 qoida — o'z fayllaringiz 100 % PASS; HPA max 40 uchun klaster sig'imi (vCPU) hisobi.
4. HPA simulyatsiyasi (s5): Qora juma profili (T = 8 soat, 1 s qadam yoki 5 s qadam), xizmat vaqti 6 ms, min 3 / max 40; SLO buzilish vaqti, maksimal navbat, pod-soniya.
5. Uch siyosat taqqosi: (a) CPU-HPA, (b) oldindan rejalashtirilgan masshtab (t = -30 daqiqada 30 nusxaga), (c) doimiy 40 nusxa; har biri uchun SLO buzilgan soniyalar va vCPU-soat narxi.
6. TCO (s6) ni bulut ichida qayta yozing: 3 siyosat uchun yillik narx (oddiy kunlar + 5 ta aksiya kuni), yo'qotilgan tavsiyalar soni (SLO buzilgan vaqt x so'rov/s) va har yo'qotilgan tavsiya 0,02 USD deb daromad yo'qotishini hisoblang; xulosa.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Uch masshtablash siyosati taqqos jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

3-variant. Sanoat: vibratsiya anomaliyasini baholash xizmati — sex shlyuzida Edge va ARM tasviri o'rta

Soha: Sanoat

Zavodda 200 ta dvigatel sensorlari har soniyada bittadan 256 nuqtalik oyna yuboradi (jami 200 so'rov/s, doimiy). Autoenkoder (A3 kabi) rekonstruksiya xatosini hisoblab anomaliya ballini qaytaradi; qaror 500 ms ichida kerak, zavod tarmog'i internetga 10 Mbit/s kanal orqali ulangan va tez-tez uziladi. Siz /score xizmatini yozasiz, yuklamani sinaysiz, ARM shlyuzi uchun Dockerfile (python:3.12-slim arm64, ORT) tahlil qilasiz, K8s o'rniga bitta tugunli k3s manifestini tekshirasiz, HPA simulyatsiyasida sensorlar sonining o'sishini (200 -> 600) modellashtirasiz va Edge shlyuz vs bulut TCO ni oflayn ishlash talabi bilan solishtirasiz.

Model	Autoenkoder 256-64-16-64-256 -> ONNX (anomaliya = MSE > chegara)
Xizmat turi	anomaliya balli, /score {signal[256]} -> {score, is_anomaly}
SLO	p95 <= 500 ms, oflayn ishlash majburiy
Yuklama profili	200 so'rov/s doimiy; 2-yilda 600
Xizmat vaqti (sim)	4 ms
Shlyuz	ARM 4 yadro, 4 GB (Edge), 450 USD + 12 Vt
Bulut	2 vCPU nusxa 0,09 USD/soat, egress 0,09 USD/GB, RTT 90 ms
Trafik	256 x 4 bayt x 200/s

QADAMLAR

1. Autoenkoderni sintetik vibratsiyada o'qiting (A3 4-variant usuli), chegarani 99-foizil qilib ONNX'ga eksport qiling; FastAPI /score, /readyz, /metrics (anomaliya ulushi gauge bilan).
2. Yuklama sinovi: parallel 1/2/4/8/16 mijoz; 200 so'rov/s talabini qaysi parallellikda bajarish mumkin; little_N; /score_batch (50 oyna bitta so'rovda) bilan solishtiring.
3. Dockerfile: arm64 uchun slim + onnxruntime (torch'siz) variant va «hamma narsa bilan» variant; 11 qoida va hajm bahosi; 10 Mbit/s kanalda tasvirni shlyuzga yetkazish vaqti.
4. k3s manifesti (bitta tugun): Deployment replicas 2 (ikki jarayon), Service, HPA max 3; 18 qoidadan o'ting, bitta tugunda replicas >= 2 nimani beradi va nimani bermaydi — izohlang.
5. HPA simulyatsiyasi: yuklama 200 -> 600 so'rov/s 1 yil davomida chiziqli o'sadi deb (simulyatsiyada 600 s ga siqilgan), xizmat vaqti 4 ms; shlyuz sig'imi qachon tugaydi, ikkinchi shlyuz qachon kerak.
6. TCO (s6): Edge shlyuz (1+1 zaxira) vs bulut vs gibrid (shlyuzda ball, bulutga faqat anomal oynalar) 1/3/5 yil; trafik GB/oy; oflayn uzilishda (yiliga 40 soat) bulut variantida nechta oyna yo'qoladi; xulosa.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Ikki Dockerfile taqqos jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

4-variant. Telekom: SMS spam filtri xizmati — doimiy 2000 so'rov/s, paketli endpoint va K8s manifest oson

Soha: Telekom

Mobil operator SMS markazidan o'tayotgan xabarlarni spam/oddiy deb belgilaydi: doimiy oqim 2000 xabar/s, kechikish talabi yumshoq (p95 300 ms), lekin xizmat to'xtamasligi kerak. Model — 1000 o'lchamli so'zlar sumkasi MLP (A3 15-variant kabi, sintetik). SMS markazi xabarlarni 100 talik paketlarda yuborishi mumkin. Siz /classify va /classify_batch endpointlarini yozasiz, yuklama sinovida yakka va paketli rejimni solishtirasiz, Dockerfile va K8s manifestlarini qoidalardan o'tkazasiz va paketli rejimda nechta nusxa kerakligini Little qonuni bilan hisoblaysiz.

Model	MLP 1000-256-2 -> ONNX
Xizmat turi	matn tasnifi, /classify {vector[1000]} va /classify_batch [100 ta]
SLO	p95 <= 300 ms, mavjudlik 99,95 %
Yuklama profili	2000 xabar/s doimiy
Paket	1 va 100
K8s	replicas 3, HPA min 3 max 8, nishon 60 %, PDB minAvailable 2
Nusxa	2 vCPU, 1 GiB

QADAMLAR

1. Sintetik so'zlar sumkasi ma'lumotida MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI xizmati /readyz, /classify, /classify_batch, /metrics (xabarlar soni, spam ulushi).
2. Yuklama sinovi (s2): yakka so'rov parallel 1/2/4/8/16 mijoz; paketli (100 ta) so'rov parallel 1/2/4 mijoz; xabar/s ko'rsatkichini ikkala rejim uchun jadvalga yozing.
3. Little qonuni: 2000 xabar/s uchun yakka va paketli rejimda nusxalar soni (o'lchangan throughput/nusxa asosida); paketli rejimda 100 ta xabar yig'ilish kutishi (100/2000 s) kechikishga qo'shiladi — SLO ga sig'adimi.
4. Dockerfile (s3): o'z variantingiz 11 qoidadan o'tsin; tasvir hajmi va 8 tugunga tarqatish GB.
5. K8s manifest (s4): Deployment replicas 3, PDB (minAvailable 2) qo'shing va unga 2 ta yangi qoida yozing (jami 20); HPA chegarasi millicore'da.
6. Xulosa: operatorga paketli yoki yakka rejim, nechta nusxa, PDB nega kerak (tugun yangilanishida xizmat to'xtamasligi), GPU kerakmi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); K8s YAML (PDB bilan) va 20 qoidali tekshiruv natijasi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

5-variant. Energetika: aqlli hisoblagichlar uchun iste'mol prognozi — 15 daqiqalik burst, HPA va batch-job taqqosi qiyin

Soha: Energetika

Elektr tarmog'i operatori 40 000 ta aqlli hisoblagichdan har 15 daqiqada ko'rsatkich oladi va har biri uchun keyingi soat prognozini hisoblaydi; so'rovlar 15 daqiqalik oynaning birinchi 2 daqiqasida keladi (burst 330 so'rov/s), qolgan 13 daqiqa deyarli nol. Model — 24 soatlik oyna MLP regressori (A3 5-variant kabi). Siz /forecast xizmatini yozasiz, yuklama sinovi o'tkazasiz, HPA ni burst profilida simulyatsiya qilasiz (HPA bunday profilda yomon ishlaydi), muqobil sifatida har 15 daqiqada ishga tushadigan batch Job (K8s CronJob) rejimini modellashtirasiz va ikkala rejim uchun narx va kechikishni solishtirasiz.

Model	MLP 30-128-64-1 -> ONNX (regressiya)
Xizmat turi	vaqt qatori prognozi, /forecast {window[24], features[6]}
SLO	barcha 40 000 prognoz oyna boshidan 5 daqiqa ichida tayyor
Yuklama profili	har 15 daqiqa: 2 daqiqa 330 so'rov/s, 13 daqiqa 2 so'rov/s
Xizmat vaqti (sim)	8 ms
HPA	min 1, max 10, nishon 60 %, start 20 s, scaleDown 300 s
CronJob	4 parallel pod, 2 daqiqa ishlaydi
Narx	vCPU-soat 0,045 USD

QADAMLAR

1. MLP regressorni sintetik ma'lumotda o'qitib ONNX'ga eksport qiling; FastAPI /forecast, /forecast_batch (1000 hisoblagich), /readyz, /metrics; TestClient bilan MAE tekshiruvi.
2. Yuklama sinovi: yakka parallel 1/2/4/8/16; paketli 1000 talik so'rov; 40 000 prognoz uchun ikkala rejimda vaqt.
3. K8s manifestlari: Deployment + HPA (18 qoida) va qo'shimcha CronJob manifesti (schedule */15, parallelism 4, activeDeadlineSeconds 600) — CronJob uchun 4 ta yangi qoida yozing.
4. HPA simulyatsiyasi (s5): yuklama(t) ni 15 daqiqalik davriy burst qiling (T = 1 soat), xizmat vaqti 8 ms; SLO «5 daqiqada hammasi» ni navbat orqali tekshiring; scaleDown oynasi 300 s va 60 s ni solishtiring.
5. Batch rejimi simulyatsiyasi: har 15 daqiqada 4 pod 20 s da ishga tushib 40 000 prognozni o'lchangan throughput bilan bajaradi; tugash vaqti, pod-soniya.
6. Narx taqqosi: HPA (min 1 va min 4) vs CronJob vs doimiy 6 nusxa — kunlik vCPU-soat va USD; qaysi rejim SLO ni arzonroq bajaradi.
7. Xulosa: davriy burst yuklamada reaktiv HPA nega yomon, KEDA cron-trigger nima beradi, hisoblagichlar 400 000 bo'lsa nima o'zgaradi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); HPA vs CronJob taqqos jadvali va grafigi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

6-variant. Fintech: tranzaksiya firibgarligi skoring xizmati — qattiq p99, ikki mintaqa va kanareyka relizi qiyin

Soha: Fintech (to'lovlar)

To'lov tizimi har tranzaksiyani o'tkazishdan oldin firibgarlik ballini so'raydi: p99 20 ms dan oshsa tranzaksiya tekshiruvsiz o'tkaziladi (xavf), xizmat ikki mintaqada ishlaydi va yangi model versiyasi avval 10 % trafikka (kanareyka) chiqariladi. Yuklama 500 so'rov/s, oyning oxirgi kunida 5000 so'rov/s. Model — A3 18-variantdagi MLP. Siz xizmatga model versiyasi yorlig'ini va /metrics da versiya bo'yicha hisoblagichlarni qo'shasiz, yuklama sinovida p99 ni alohida kuzatasiz, ikki Deployment (stable/canary) manifestini tekshirasiz, HPA ni oy oxiri profilida simulyatsiya qilasiz va ikki mintaqali bulut TCO sini hisoblaysiz.

Model	MLP 28-128-64-1 -> ONNX, versiyalar v1 (FP32) va v2 (INT8 statik)
Xizmat turi	anomaliya/skoring, /score {features[28]} -> {score, decision, model_version}
SLO	p99 <= 20 ms, xatolar < 0,1 %
Yuklama profili	500 so'rov/s, oy oxiri 5000 so'rov/s (30 daqiqada o'sish, 4 soat plato)
Xizmat vaqti (sim)	3 ms
Kanareyka	stable 90 % / canary 10 % (Service selector + replicas nisbati 9:1)
HPA	min 4, max 30, nishon 50 %, start 20 s
Mintaqalar	2, har birida to'liq nusxa, RTT 25 ms yaqin mintaqa

QADAMLAR

1. Ikki ONNX versiya tayyorlang (FP32 va INT8); FastAPI xizmati MODEL_VERSION env bilan ishga tushadi, /score javobida versiya, /metrics da infer_requests_total{version=...} va p99.
2. Yuklama sinovi: parallel 1/2/4/8/16/32 mijoz, p99 ni alohida ustun qilib SLO 20 ms qaysi parallellikda buziladi; v1 va v2 ni solishtiring.
3. K8s: ikkita Deployment (score-stable replicas 9, score-canary replicas 1, umumiy label app=score + version label), bitta Service, HPA faqat stable'ga; 18 qoidaga kanareyka uchun 3 ta yangi qoida qo'shing (versiya label, canary replicas <= 10 %, canary'da alohida readiness).
4. Kanareyka tahlili: TestClient bilan 1000 so'rovni 9:1 nisbatda yo'naltirib (tasodifiy marshrutlash), versiya bo'yicha p99 va qaror ulushini /metrics dan oling; v2 qarorlari v1 dan qancha farq qiladi (tranzaksiya soni).
5. HPA simulyatsiyasi (s5): oy oxiri profili, xizmat vaqti 3 ms, min 4/max 30; SLO p99 ni «p95_ms» o'rniga navbat + 2x xizmat vaqti bilan baholang; min 4 va min 10 taqqosi.
6. TCO (s6) ikki mintaqa uchun: bulut nusxalar (oddiy kun va oy oxiri), mintaqalararo trafik, 3 yil; bitta mintaqa + zaxira bilan solishtiring; mintaqa uzilsa (yiliga 4 soat) yo'qotilgan tranzaksiyalar.
7. Xulosa: kanareyka qachon 100 % ga o'tkaziladi (mezon), p99 uchun HPA yetarlimi, GPU kerakmi (yo'q — asoslang).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Kanareyka manifestlari va versiya bo'yicha metrikalar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

7-variant. Agro: issiqxona sug'orish tavsiyasi — Raspberry Pi shlyuzi simulyatsiyasi va 5 yillik TCO oson

Soha: Agro

Agrofirma 50 ta issiqxonada har 5 daqiqada sensorlar (A3 2-variant: 20 belgi, 4 sinf tavsiya) bo'yicha sug'orish qarorini oladi; internet qishloqda beqaror (kuniga 1-2 soat uziladi), qaror kechiksa ekin zarar ko'radi. Siz /advise xizmatini yozasiz va uni ikki rejimda sinaysiz: «Edge» (1 oqim + har so'rovga 12 ms sun'iy ishlov — Raspberry Pi taqlidi) va «Cloud» (2 oqim + 100 ms RTT taqlidi). Yuklama kichik (jami 0,17 so'rov/s), shuning uchun masshtablash emas, ishonchlilik va narx asosiy. Dockerfile arm64 uchun tekshiriladi, HPA simulyatsiyasi o'rniga uzilishlar simulyatsiyasi, TCO 5 yil.

Model	MLP 20-256-128-4 -> ONNX
Xizmat turi	tavsiya, /advise {sensors[20]} -> {action, confidence}
SLO	qaror 5 daqiqa ichida, oflayn ishlash
Yuklama	50 issiqxona x 1 so'rov / 5 daqiqa
Edge taqlidi	ORT 1 oqim + 12 ms sleep
Cloud taqlidi	ORT 2 oqim + 100 ms sleep (RTT)
Uzilishlar	kuniga 90 daqiqa, tasodifiy
TCO	Raspberry Pi 4 (90 USD + 5 Vt) x 50 + zaxira 5 vs bulut 1 nusxa 0,045 USD/soat + 4G modemlar

QADAMLAR

1. MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /advise, /readyz, /metrics; EDGE_MODE env bilan 12 ms yoki 100 ms sun'iy kechikish qo'shiladi.
2. Ikkala rejimda ketma-ket 200 so'rov p50/p95 va parallel 1/2/4 mijoz; sof inferens ulushi; Little qonuni tekshiruvi.
3. Dockerfile (s3): python:3.12-slim arm64 + ORT varianti 11 qoida; tasvir hajmi va 4G (5 Mbit/s) orqali 50 qurilmaga yangilash vaqti va trafik.
4. Uzilishlar simulyatsiyasi (s5 o'rniga): 7 kunlik vaqt o'qi (1 daqiqa qadam), kuniga 90 daqiqa tasodifiy uzilish; Cloud rejimida uzilish paytidagi so'rovlar yo'qoladi yoki navbatga tushadi (10 daqiqagacha); Edge rejimida yo'qolmaydi; yo'qolgan/kechikkan qarorlar sonini hisoblang.
5. TCO (s6) 1/3/5 yil: Edge (50+5 Pi, elektr, yiliga 10 % almashtirish) vs Cloud (1 nusxa + 50 modem x 3 USD/oy) vs gibrid (Pi'da qaror, bulutda monitoring); sezgirlik: issiqxona 200 ta, uzilish kuniga 4 soat.
6. Xulosa: agrofirmaga tavsiya, uzilishlar narxini (zarar ko'rgan qarorlar) TCO ga qanday qo'shish mumkin, Jetson kerakmi (yo'q — asoslang).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Uzilishlar simulyatsiyasi jadvali (yo'qolgan qarorlar)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

8-variant. Transport: avtobus bortida yo'lovchilarni sanash — Jetson taqlidi, 4G RTT va 300 avtobus TCO o'rta

Soha: Transport

Shahar avtobus parki 300 ta avtobusning eshik kamerasidan yo'lovchilar sonini hisoblaydi; kadr surrogati — digits CNN (sinf = yo'lovchilar soni 0..9). Har eshik ochilganda 2 soniya davomida 5 kadr/s, kuniga avtobus boshiga 400 to'xtash. Bulut varianti 4G orqali ishlaydi (RTT 120 ms, tunnel va chekka hududlarda uzilish), Edge varianti — bortdagi kichik kompyuter (Jetson Nano taqlidi: 1 oqim + 8 ms). Siz /count xizmatini yozasiz, ikkala rejimni sinaysiz, bulut uchun K8s manifest va HPA ni ertalabki/kechki cho'qqi profilida tekshirasiz va 300 avtobus uchun 3 yillik TCO va trafikni hisoblaysiz.

Model	kichik CNN (digits) -> ONNX, 10 sinf
Xizmat turi	tasvir, /count {pixels[64]} -> {count, confidence}
SLO	p95 <= 400 ms (bulut), oflayn ishlash istalgan
Yuklama profili	cho'qqi 07-09 va 17-19: 300 avtobus x 10 kadr / 30 s = 100 so'rov/s; kunduzi 30; tunda 2
Xizmat vaqti (sim)	5 ms
HPA	min 2, max 8, nishon 60 %
Edge	bort kompyuter 180 USD + 8 Vt, 1 oqim + 8 ms
Trafik	kadr 3 KB, 4G 1,5 USD/GB

QADAMLAR

1. CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /count, /readyz, /metrics; EDGE_MODE (1 oqim + 8 ms) va CLOUD_MODE (2 oqim + 120 ms RTT taqlidi).
2. Yuklama sinovi ikkala rejimda: parallel 1/2/4/8/16; cloud rejimida RTT tufayli little_N qanday o'sadi; SLO 400 ms tekshiruvi.
3. K8s manifest (s4) bulut xizmati uchun 18 qoida; HPA chegarasi; Dockerfile (s3) 11 qoida.
4. HPA simulyatsiyasi (s5): 24 soatlik profil (tun 2, kunduz 30, cho'qqi 100 so'rov/s, 30 daqiqalik o'tish), 1 daqiqa qadam; SLO buzilish, pod-soniya; min 2 vs min 4.
5. Trafik va TCO (s6): bulut — kuniga 300 x 400 x 10 kadr x 3 KB egress + 4G narxi + nusxalar; Edge — 300 bort kompyuter + 10 % zaxira + quvvat; gibrid — bortda sanash, bulutga faqat soatlik yig'indi; 1/3/5 yil.
6. Xulosa: avtobus parkiga tavsiya, 4G trafik ulushi TCO da qancha, uzilishlar bo'lsa bulut variantida ma'lumot yo'qolishi, Jetson haqiqiy qurilmada nimani o'lchash kerak (tegrastats).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

9-variant. Ta'lim: insholarni avtomatik baholash xizmati — imtihon kuni, sekin start va oldindan masshtablash o'rta

Soha: Ta'lim

Milliy test markazi imtihon kuni 2 soat ichida 50 000 talabaning insholarini (matn belgilari MLP, A3 15-variant kabi 1000 o'lchamli vektor) 5 ballik shkalada baholaydi; talabalar inshoni topshirgach 30 soniya ichida dastlabki ball ko'rsatilishi kerak. Yuklama imtihon boshlanishidan 90 daqiqa o'tgach keskin oshadi (ko'pchilik oxirgi 30 daqiqada topshiradi): 5 so'rov/s dan 40 so'rov/s gacha. Model tasviri katta (til resurslari bilan 1,2 GB), shuning uchun Pod start 90 s. Siz /grade xizmatini yozasiz, yuklama sinovi o'tkazasiz, HPA ni sekin start bilan simulyatsiya qilasiz va oldindan masshtablash (imtihondan 15 daqiqa oldin) hamda tasvirni kichraytirish (start 20 s) variantlarini solishtirasiz.

Model	MLP 1000-512-128-5 -> ONNX
Xizmat turi	matn baholash, /grade {vector[1000]} -> {score 1..5, confidence}
SLO	p95 <= 30 s (navbat bilan), ball yo'qolmasligi
Yuklama profili	0-90 daqiqa 5 so'rov/s, 90-120 daqiqa 40 so'rov/s
Xizmat vaqti (sim)	120 ms (matn ishlovi bilan)
HPA	min 2, max 12, nishon 60 %, start 90 s (katta tasvir) yoki 20 s (slim)
Tasvir	1,2 GB vs 280 MB
Narx	vCPU-soat 0,045 USD

QADAMLAR

1. MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /grade, /readyz (model + «til resursi» yuklash taqlidi 3 s), /metrics; TestClient bilan readyz 503 -> 200 ketma-ketligi.
2. Yuklama sinovi: parallel 1/2/4/8/16 mijoz, har so'rovga 120 ms sun'iy ishlov qo'shilgan; Little qonuni bilan N; bitta nusxa sig'imi (so'rov/s).
3. Dockerfile (s3): ikki variant — til resurslari tasvir ichida (1,2 GB) va alohida volume'dan yuklanadi (280 MB); 11 qoida va tortish vaqti 1 Gbit/s da; start vaqtiga ta'siri.
4. K8s manifest (s4) 18 qoida; readinessProbe initialDelay/failureThreshold ni 90 s startga moslang va buni qoida sifatida tekshiring.
5. HPA simulyatsiyasi (s5): imtihon profili (T = 2 soat, 1 s qadam), xizmat vaqti 120 ms; 4 konfiguratsiya: start 90 s, start 20 s, oldindan masshtab (t = 75 daqiqada 10 nusxa), min 10; SLO 30 s buzilgan vaqt, maksimal navbat, pod-soniya.
6. Narx va xulosa: har konfiguratsiya uchun imtihon kuni vCPU-soat va USD; kechikkan ballar soni; test markaziga tavsiya (oldindan masshtab + slim tasvir), GPU kerakmi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 4 konfiguratsiyali HPA taqqos jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

10-variant. Xavfsizlik: turniketda yuz embeddinglarini solishtirish — oflayn Edge va 40 turniket oson

Soha: Xavfsizlik

Korxona 40 ta turniketda xodimlarni yuz embeddingi (128 o'lchamli vektor, sintetik make_blobs bilan 500 xodim) bo'yicha tekshiradi: /verify so'rovi embeddingni oladi, bazadagi eng yaqin xodimni kosinus o'xshashlik bilan topadi va chegara bo'yicha «ruxsat/rad» qaytaradi. Ertalab 08:00-09:00 da 40 turniketdan jami 20 so'rov/s, qolgan vaqt 1 so'rov/s; javob 300 ms ichida bo'lmasa navbat hosil bo'ladi; turniket internet uzilganda ham ishlashi shart. Siz xizmatni yozasiz, yuklamani sinaysiz, Edge (turniket kontrolleri) va Cloud rejimlarini solishtirasiz, K8s manifest va HPA ni ertalabki profil bilan tekshirasiz va TCO hisoblaysiz.

Model	kosinus o'xshashlik + chegara 0,8 (sklearn make_blobs: 500 markaz, 128 o'lcham, std 0,15), ONNX o'rniga numpy matritsa
Xizmat turi	tasvir/embedding, /verify {embedding[128]} -> {employee_id, similarity, allowed}
SLO	p95 <= 300 ms, oflayn ishlash majburiy
Yuklama profili	08:00-09:00 20 so'rov/s, boshqa vaqt 1 so'rov/s
Xizmat vaqti (sim)	2 ms (Edge 1 oqim + 20 ms ARM taqlidi)
HPA	min 1, max 3, nishon 50 %
Edge	turniket kontrolleri 120 USD + 4 Vt x 40
Baza yangilanishi	kuniga 1 marta, 500 x 128 x 4 bayt

QADAMLAR

1. 500 xodim embedding bazasini make_blobs bilan yarating, test so'rovlari = markaz + shovqin (xodim) va tasodifiy vektor (begona); FastAPI /verify, /readyz, /metrics (ruxsat/rad hisoblagichlari); TestClient bilan FAR/FRR ni chegara 0,7/0,8/0,9 da o'lchang.
2. Yuklama sinovi: parallel 1/2/4/8 mijoz Edge va Cloud taqlidida (Cloud: +80 ms RTT); SLO 300 ms; Little qonuni.
3. Dockerfile (s3) arm64 slim variant 11 qoida; baza faylini volume orqali ulash (ENV + ConfigMap) — 12 omil qoidasi.
4. K8s manifest (s4) bulut varianti uchun 18 qoida; HPA min 1 bo'lsa qaysi qoida buziladi va nega bu xizmat uchun qabul qilinadi/qilinmaydi.
5. HPA simulyatsiyasi (s5): 24 soat, 1 daqiqa qadam, ertalabki cho'qqi; Cloud rejimida SLO buzilish; Edge rejimida har turniket o'z yukini (0,5 so'rov/s) ko'taradi — masshtablash kerak emasligini ko'rsating.
6. TCO (s6) 1/3/5 yil: Edge 40 kontroller + bulut sinxronizatsiya vs Cloud + 40 yupqa mijoz; internet uzilishida (yiliga 30 soat) Cloud rejimida nechta xodim kira olmaydi; xulosa va maxfiylik ustuni (embeddinglar binodan chiqadimi).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); FAR/FRR jadvali (3 chegara)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

11-variant. Logistika: ombor konveyerida paket nuqsonini aniqlash — 8 liniya, PDB va rolling update simulyatsiyasi qiyin

Soha: Logistika

Saralash ombori 8 ta konveyer liniyasida soniyasiga 20 tadan paketning tasvirini (digits surrogati, 2 sinf) tekshiradi — jami 160 so'rov/s, 24/7, qaror 50 ms ichida kerak, aks holda paket qayta aylanadi (liniya unumdorligi tushadi). Xizmat ombor ichidagi 3 tugunli k3s klasterida ishlaydi; model haftada bir yangilanadi va yangilanish paytida liniyalar to'xtamasligi shart. Siz xizmatni yozasiz, yuklamani sinaysiz, Deployment + PDB + HPA manifestlarini tekshirasiz, HPA simulyatsiyasiga rolling update (maxSurge 1 / maxUnavailable 0 vs maxUnavailable 1) va tugun nosozligini (3 tugundan biri 10 daqiqaga o'chadi) qo'shasiz, qayta aylangan paketlar sonini hisoblaysiz.

Model	kichik CNN (digits -> 2 sinf) -> ONNX
Xizmat turi	tasvir, /inspect {pixels[64]} -> {defect, confidence}
SLO	p95 <= 50 ms, qayta aylangan paketlar < 0,5 %
Yuklama profili	160 so'rov/s doimiy
Xizmat vaqti (sim)	6 ms
K8s	3 tugun, replicas 4, HPA min 4 max 9, PDB minAvailable 3, podAntiAffinity
Rollout	maxSurge 1 / maxUnavailable 0 vs maxUnavailable 1, yangi Pod 25 s
Nosozlik	t=600 s da 1 tugun 10 daqiqaga o'chadi

QADAMLAR

1. CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /inspect, /readyz, /metrics (nuqson ulushi, p95); TestClient bilan 200 paketda aniqlik.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16; bitta nusxa sig'imi; Little qonuni bilan 160 so'rov/s uchun nusxalar soni p95 <= 50 ms sharti bilan.
3. K8s manifestlari (s4): Deployment (replicas 4, RollingUpdate), PDB, HPA, Service; podAntiAffinity (tugun bo'yicha tarqatish) va PDB uchun 4 ta yangi qoida — jami 22 qoida 100 % PASS; Dockerfile 11 qoida.
4. HPA simulyatsiyasi (s5) doimiy 160 so'rov/s da: asosiy holat SLO ichida ekanini tasdiqlang (bazaviy chiziq).
5. Rolling update simulyatsiyasi: t=300 s da yangilanish, nusxalar birma-bir almashadi (yangi Pod 25 s), ikki strategiya; sig'im, navbat, p95 grafigi; qayta aylangan paketlar (p95 > 50 ms bo'lgan soniyalar x 160) soni.
6. Tugun nosozligi simulyatsiyasi: t=600 s da 1 tugun o'chadi (uning 1-2 Pod'i yo'qoladi), HPA va PDB qanday javob beradi, qayta ko'tarilish vaqti; antiAffinity bo'lmasa (barcha Pod bitta tugunda) nima bo'lishini solishtiring.
7. Xulosa: omborga tavsiya (replicas, PDB, strategiya), haftalik yangilanish qaysi vaqtda, GPU/Jetson kerakmi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Rolling update va tugun nosozligi grafiklari; 22 qoidali manifest tekshiruvi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

12-variant. Shahar: chorraha svetofor kontrollerlari uchun tiqilinch bashorati — 400 chorraha, Edge kontroller vs markaz oson

Soha: Shahar

Shahar transport markazi 400 ta chorrahadan har 2 soniyada 6x6 trafik xaritasini oladi (A3 12-variant CNN) va tiqilinch darajasini qaytaradi; natija svetofor fazasini sozlaydi, shuning uchun 500 ms ichida kelishi kerak. Yuklama doimiy 200 so'rov/s, ertalab va kechqurun 1,5 baravar. Markaziy serverga ulanish optik tola, lekin ba'zi chorrahalar radiokanal (RTT 150 ms, paketlar yo'qoladi). Siz /congestion xizmatini yozasiz, yuklamani sinaysiz, Dockerfile va manifestlarni tekshirasiz, HPA ni kunlik profilda simulyatsiya qilasiz va «hamma chorrahada kontroller» vs «markazda K8s» vs «gibrid (radiokanalli 60 chorrahada kontroller)» TCO sini hisoblaysiz.

Model	CNN 6x6 -> 3 sinf -> ONNX
Xizmat turi	vaqt qatori/tasvir, /congestion {grid[36]} -> {level, probs}
SLO	p95 <= 500 ms
Yuklama profili	200 so'rov/s, 07-09 va 17-19 da 300
Xizmat vaqti (sim)	3 ms
HPA	min 2, max 6, nishon 60 %
Edge kontroller	140 USD + 6 Vt, 1 oqim
Radiokanal	60 chorraha, RTT 150 ms, 2 % yo'qotish

QADAMLAR

1. Sintetik 6x6 xaritalarda CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /congestion, /readyz, /metrics; TestClient aniqlik.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16; 200 va 300 so'rov/s uchun nusxalar soni; radiokanal taqlidi (150 ms sleep + 2 % so'rovda 503) bilan p95 va xatolar ulushi.
3. Dockerfile (s3) 11 qoida; K8s manifest (s4) 18 qoida; HPA chegarasi millicore'da.
4. HPA simulyatsiyasi (s5): 24 soat, 1 daqiqa qadam, ikki cho'qqi; SLO buzilish, pod-soniya; nishon 60 % vs 40 %.
5. TCO (s6) 1/3/5 yil: 400 kontroller vs markaz (K8s, 3 tugun server 4000 USD har biri + elektr) vs gibrid (60 kontroller + markaz); radiokanalli chorrahalarda yo'qolgan qarorlar soni markaz variantida (2 % x so'rovlar).
6. Xulosa: shaharga tavsiya, radiokanal muammosini Edge qanday hal qiladi, 400 kontrollerni yangilash (OTA) xarajati TCO ga qanday qo'shiladi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

13-variant. Ekologiya: havo sifati prognozi — juda kam yuklama, scale-to-zero va serverless TCO o'rta

Soha: Ekologiya

Ekologiya agentligi 120 ta havo monitoring stansiyasidan daqiqada bitta o'lchov oladi va har biri uchun 6 soatlik prognoz hisoblaydi — jami 2 so'rov/s, tunda 0,5 so'rov/s; natija veb-saytda ko'rsatiladi, 3 soniya kechikish qabul qilinadi. Doimiy ishlaydigan 2 nusxa bunday yuklama uchun isrof. Siz /aqi_forecast xizmatini yozasiz, yuklamani sinaysiz, K8s manifestida minReplicas 1 va scale-to-zero (KEDA/Knative taqlidi: 5 daqiqa so'rovsiz bo'lsa 0 nusxa, sovuq start 15 s) variantlarini simulyatsiya qilasiz va doimiy pod, scale-to-zero hamda serverless funksiya (so'rov boshiga narx) TCO sini solishtirasiz.

Model	MLP 13-256-128-5 (AQI sinfi) + regressiya boshi -> ONNX
Xizmat turi	vaqt qatori prognozi, /aqi_forecast {features[13]} -> {aqi_class, values[6]}
SLO	p95 <= 3 s
Yuklama profili	kunduzi 2 so'rov/s, tunda 0,5; har 10 daqiqada 60 s so'rovsiz oraliq
Xizmat vaqti (sim)	10 ms
Scale-to-zero	5 daqiqa so'rovsiz -> 0 nusxa, sovuq start 15 s
Serverless narxi	0,0000002 USD/so'rov + 0,0000167 USD/GB-s (256 MB)
Doimiy pod	0,5 vCPU 0,0225 USD/soat

QADAMLAR

1. MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /aqi_forecast, /readyz (sovuq start taqlidi: model yuklash 2 s), /metrics; TestClient bilan birinchi so'rov (sovuq) va keyingi (iliq) kechikishini alohida o'lchang.
2. Yuklama sinovi (s2): parallel 1/2/4 mijoz (yuklama kichik), bitta nusxa sig'imi; 2 so'rov/s uchun CPU bandlik foizi (requests 500m ga nisbatan).
3. K8s manifest (s4): minReplicas 1 varianti 18 qoida (replicas >= 2 qoidasi buziladi — asoslang); Dockerfile (s3) hajmi sovuq startga qanday ta'sir qiladi.
4. Simulyatsiya (s5 o'zgartirilgan): 24 soat, 1 s qadam, 3 siyosat — doimiy 1 nusxa, doimiy 2 nusxa, scale-to-zero (5 daqiqa bo'shliqdan keyin 0, sovuq start 15 s); SLO 3 s buzilgan so'rovlar soni, pod-soniya.
5. TCO (s6) yillik: doimiy 1 va 2 pod, scale-to-zero (pod-soniya bo'yicha), serverless (so'rov soni x narx + GB-s); stansiyalar 1200 ta bo'lsa kesishish nuqtasi qayerda.
6. Xulosa: agentlikka tavsiya, sovuq start SLO ni qancha buzadi, Edge (stansiyada hisoblash) nega bu yerda ma'nosiz.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 3 siyosat taqqos jadvali (SLO, pod-soniya, USD)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

14-variant. Sport: fitnes ilovasi uchun harakat tasnifi — 1 mln foydalanuvchi, kunlik sinusoidal yuklama va on-device taqqosi qiyin

Soha: Sport

Fitnes ilovasi 1 million foydalanuvchining akselerometr oynalarini (A3 14-variant 1D CNN + GRU, 6 sinf) tasniflaydi; mashg'ulot paytida har 2 soniyada bitta so'rov. Bir vaqtda faol foydalanuvchilar kun davomida sinusoidal o'zgaradi: tunda 2000, ertalab 7:00 va kechqurun 19:00 da 60 000 (jami 30 000 so'rov/s cho'qqi). Muqobil — modelni telefonda ishlatish (on-device): server yuklamasi nolga tushadi, lekin batareya sarfi va yangilash murakkabligi paydo bo'ladi. Siz /activity xizmatini yozasiz, yuklamani sinaysiz, HPA ni sinusoidal profil bilan simulyatsiya qilasiz (nishon va oynalarni sozlab), bulut TCO ni hisoblaysiz va on-device variant bilan (telefon energiyasi A3 s6 modeli bilan) solishtirasiz.

Model	Conv1d+GRU -> ONNX (64x3 -> 6 sinf)
Xizmat turi	vaqt qatori, /activity {window[64][3]} -> {activity, probs}
SLO	p95 <= 200 ms
Yuklama profili	sinusoidal: 1000 + 29000 x (1 + sin) / 2 so'rov/s, ikki cho'qqi (12 soat davr), 1 daqiqa qadam, 24 soat
Xizmat vaqti (sim)	5 ms
HPA	min 10, max 300, nishon 50/70 %, scaleUp oynasi 0/60 s, scaleDown 300/900 s
Bulut narxi	2 vCPU nusxa 0,09 USD/soat
On-device	telefon 0,3 Vt x inferens vaqti, 1 so'rov / 2 s, 1 soat mashg'ulot

QADAMLAR

1. 1D CNN + GRU modelini sintetik akselerometrda o'qitib ONNX'ga eksport qiling; FastAPI /activity, /activity_batch, /readyz, /metrics; TestClient aniqlik.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16/32; bitta nusxa sig'imi; Little qonuni bilan cho'qqi 30 000 so'rov/s uchun nusxalar soni; paketli so'rov (telefon 10 oynani birga yuborsa) sig'imni qancha oshiradi.
3. K8s manifest (s4) 18 qoida, maxReplicas 300 uchun klaster vCPU hisobi; Dockerfile (s3).
4. HPA simulyatsiyasi (s5): sinusoidal profil, 1 daqiqa qadam (HPA davri 15 s ni mos ravishda moslang), 6 konfiguratsiya (nishon 50/70 %, scaleUp 0/60, scaleDown 300/900); SLO buzilish daqiqalari, maksimal nusxa, pod-soat; HPA sinusoidani qanchalik kechikib kuzatadi (fazaviy siljish daqiqada).
5. Bulut TCO yillik: eng yaxshi HPA konfiguratsiyasi vs doimiy 300 nusxa vs prognozli masshtab (ideal — sig'im = yuklama / 0,7); grafik.
6. On-device variant: A3 s6 energiya modeli bilan telefon uchun uJ/inferens (P_bosh 0,1 Vt, P_to'liq 0,3 Vt), 1 soatlik mashg'ulotda mAh (3,85 V, 4000 mAh batareya) va foiz; model yangilash trafigi (1 mln x hajm); server narxi 0 — lekin qaysi xarajatlar qoladi.

7. Xulosa: ilova uchun gibrid taklif (on-device tasnif + serverda yig'ma tahlil), HPA sozlamalari, GPU bu xizmatga kerakmi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 6 konfiguratsiyali HPA jadvali; Bulut vs on-device taqqos jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

15-variant. Media: yangiliklar sarlavhasini teglash xizmati — viral portlash, navbat chegarasi va 503 o'rta

Soha: Media

Yangiliklar agregatori yangi sarlavhalarni 8 ta mavzu tegiga ajratadi (matn vektori MLP, sintetik). Odatiy yuklama 50 so'rov/s, viral voqea paytida 5 daqiqa ichida 30 baravar (1500 so'rov/s) oshadi va 20 daqiqa davom etadi. Teg kechiksa yangilik lentaga «tegsiz» tushadi — bu qabul qilinadi, lekin xizmat butunlay qulab qolishi mumkin emas. Siz /tag xizmatini yozasiz, yuklamani sinaysiz, HPA simulyatsiyasiga navbat chegarasini (ortiqcha so'rovlar 503 bilan rad etiladi) qo'shasiz, scaleUp oynasi va nishonni sozlab rad etilganlar sonini kamaytirasiz va mijoz tomonida qayta urinish (retry) strategiyasining ta'sirini TestClient bilan o'lchaysiz.

Model	MLP 1000-256-8 -> ONNX
Xizmat turi	matn tasnifi, /tag {vector[1000]} -> {tags top-2, probs}
SLO	p95 <= 500 ms, rad etilganlar < 5 % (viral vaqtda)
Yuklama profili	50 -> 1500 so'rov/s (5 daqiqada), 20 daqiqa, keyin 50
Xizmat vaqti (sim)	7 ms
Navbat chegarasi	har nusxada 200 so'rov, ortig'i 503
HPA	min 2, max 25, nishon 60 %, scaleUp oynasi 0/30/60 s
Retry	mijoz 503 da 1 marta 200 ms dan keyin qayta uradi

QADAMLAR

1. MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /tag, /readyz, /metrics (rad etilganlar hisoblagichi); xizmatga navbat chegarasi qo'shing (bir vaqtdagi so'rovlar semafori, ortig'i 503).
2. Yuklama sinovi (s2): parallel 1/2/4/8/16/32 mijoz; semafor 4 bo'lganda 503 ulushi; mijozda retry (1 marta, 200 ms) bilan va retry'siz p95 va muvaffaqiyatli so'rovlar ulushi.
3. K8s manifest (s4) 18 qoida; Dockerfile (s3); HPA chegarasi.
4. HPA simulyatsiyasi (s5) navbat chegarasi bilan: har soniyada navbat > 200 x nusxa bo'lsa ortiqcha so'rovlar rad etiladi; viral profil; scaleUp oynasi 0/30/60 s va nishon 40/60 % uchun rad etilganlar soni, SLO buzilish, pod-soniya.
5. Retry ta'siri simulyatsiyada: rad etilgan so'rovlarning 100 % i 1 soniyadan keyin qaytib keladi — yuklama qanday o'sadi (retry bo'roni), rad etilganlar soni qanday o'zgaradi; eksponensial kechiktirish (1, 2, 4 s) bilan solishtiring.
6. Xulosa: agregatorga tavsiya (navbat chegarasi, HPA sozlamalari, retry siyosati), rad etish nega qulashdan yaxshi, prognozli masshtablash mumkinmi (viral oldindan ma'lum emas).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Rad etilganlar va retry bo'roni grafiklari

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

16-variant. Tibbiyot: palata EKG monitoringi — 200 bemor, hayot uchun kritik probe'lar, PDB va bedside Edge qiyin

Soha: Tibbiyot (reanimatsiya)

Kasalxona 200 ta palata monitoridan EKG oqimini (A3 16-variant 1D CNN, 3 sinf) har soniyada bir segment sifatida tahlil qiladi — 200 so'rov/s, 24/7; fibrillyatsiya aniqlanganda 2 soniya ichida signal berilishi shart, xizmat 1 daqiqadan ortiq to'xtasa hayot xavfi. Siz xizmatni yozasiz (readiness/liveness farqi, graceful shutdown — preStop paytida yangi so'rovlar qabul qilinmaydi, lekin joriylar tugaydi), yuklamani sinaysiz, Deployment + PDB + PriorityClass manifestlarini qoidalardan o'tkazasiz, HPA simulyatsiyasiga Pod nosozliklarini (tasodifiy o'lim, 2 daqiqa qayta tiklanish) va rolling update'ni qo'shasiz, bedside Edge (har monitor o'zi hisoblaydi) bilan markaziy xizmatni ishonchlilik va TCO bo'yicha solishtirasiz.

Model	1D CNN (250 nuqta -> 3 sinf) -> ONNX
Xizmat turi	vaqt qatori/anomaliya, /ecg {segment[250]} -> {class, alarm, confidence}
SLO	signal <= 2 s, mavjudlik 99,99 %, yo'qolgan segment < 0,01 %
Yuklama profili	200 so'rov/s doimiy
Xizmat vaqti (sim)	4 ms
K8s	replicas 4, PDB minAvailable 3, PriorityClass yuqori, terminationGracePeriodSeconds 30, HPA min 4 max 8
Nosozlik	har 30 daqiqada tasodifiy 1 Pod o'ladi, 60 s da qayta tiklanadi
Edge	monitor ichidagi modul 60 USD + 2 Vt x 200

QADAMLAR

1. 1D CNN'ni sintetik EKG'da o'qitib ONNX'ga eksport qiling; FastAPI /ecg, /healthz, /readyz, /metrics (alarm hisoblagichi); graceful shutdown: /admin/drain readyz'ni 503 qiladi, joriy so'rovlar tugaydi — TestClient bilan ketma-ketlikni ko'rsating.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16; 200 so'rov/s uchun nusxalar soni p95 va alarm kechikishi 2 s sharti bilan.
3. K8s manifestlari (s4): Deployment (preStop sleep 5, terminationGracePeriodSeconds 30), PDB, PriorityClass, HPA, Service; 5 ta yangi qoida (PDB, PriorityClass, preStop, grace period, antiAffinity) — jami 23 qoida 100 % PASS.
4. Dockerfile (s3) 11 qoida; tasvir hajmi va qayta tiklanish vaqtiga ta'siri.
5. Simulyatsiya (s5 kengaytirilgan): 2 soat, 1 s qadam, doimiy 200 so'rov/s; tasodifiy Pod o'limi har 30 daqiqada (60 s tiklanish); replicas 2/3/4 va PDB bor/yo'q (yangilanish paytida) uchun yo'qolgan segmentlar soni va eng uzun uzilish; rolling update t=3600 s.
6. TCO va ishonchlilik (s6): markaziy xizmat (3 tugun + zaxira) vs bedside Edge (200 modul) vs gibrid (bedside aniqlash + markazda arxiv va ikkinchi fikr); mavjudlik hisobi (ketma-ket/parallel komponentlar), 1/3/5 yil.

7. Xulosa: kasalxonaga tavsiya, nima uchun hayot uchun kritik xizmatda Edge + markaz dublikati kerak, qaysi K8s mexanizmlari mavjudlikni qanchaga oshiradi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 23 qoidali manifest tekshiruvi; Nosozlik simulyatsiyasi jadvali (yo'qolgan segmentlar)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

17-variant. Bank: chat-bot niyat tasnifi xizmati — ish vaqti profili, versiya bo'yicha metrikalar va kanareyka 10 % o'rta

Soha: Bank xizmatlari

Bank mijozlarga xizmat ko'rsatish chat-boti har xabarni 20 ta niyatdan biriga ajratadi (matn vektori MLP, sintetik); yuklama 9:00-18:00 da 120 so'rov/s, tushlikda 180, tunda 10. Javob 150 ms ichida bo'lmasa bot «bir oz kuting» deydi va mijoz qoniqishi tushadi. Yangi model versiyasi har 2 haftada chiqadi va avval 10 % trafikda sinaladi; versiyalar bo'yicha aniqlik va kechikish alohida kuzatilishi kerak. Siz xizmatga versiya yorlig'ini qo'shasiz, yuklamani sinaysiz, stable/canary manifestlarini tekshirasiz, HPA ni ish kuni profilida simulyatsiya qilasiz va «ish vaqti bo'yicha rejalashtirilgan masshtab» (CronHPA taqlidi) bilan solishtirasiz.

Model	MLP 1000-256-20 -> ONNX, v1 va v2 (v2 = boshqa seed bilan o'qitilgan)
Xizmat turi	matn tasnifi, /intent {vector[1000]} -> {intent, confidence, model_version}
SLO	p95 <= 150 ms
Yuklama profili	tun 10, 9:00-18:00 120, 13:00-14:00 180 so'rov/s; o'tishlar 15 daqiqa
Xizmat vaqti (sim)	6 ms
Kanareyka	stable 9 nusxa / canary 1 nusxa
HPA	min 2, max 12, nishon 60 %
CronHPA	8:45 da min 6, 18:15 da min 2

QADAMLAR

1. Ikki versiya MLP'ni o'qitib (seed 0 va 1) ONNX'ga eksport qiling; FastAPI MODEL_VERSION env; /intent, /readyz, /metrics da versiya yorlig'i bilan so'rovlar, xatolar, p95; TestClient bilan har versiya aniqligi.
2. Yuklama sinovi (s2): parallel 1/2/4/8/16; SLO 150 ms; bitta nusxa sig'imi; tushlik 180 so'rov/s uchun nusxalar soni.
3. K8s manifestlari (s4): intent-stable (9) va intent-canary (1), umumiy Service, HPA stable'ga; 18 + 3 kanareyka qoidasi; Dockerfile (s3).
4. Kanareyka tahlili: TestClient bilan 2000 so'rovni 9:1 marshrutlab versiya bo'yicha aniqlik, p95 va «ishonch < 0,5» ulushini /metrics dan oling; v2 ni 100 % ga chiqarish mezonini (aniqlik >= v1 - 0,5 p.p., p95 <= v1 x 1,1) tekshiring.
5. HPA simulyatsiyasi (s5): 24 soat, 1 daqiqa qadam, ish kuni profili; CPU-HPA vs CronHPA (min 6 ish vaqtida) vs ikkalasi birga; SLO buzilish daqiqalari, pod-soat.
6. Xulosa: bankka tavsiya (CronHPA + HPA), kanareyka natijasi, hafta oxiri profili uchun nima o'zgaradi, yillik vCPU-soat narxi (0,045 USD).

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Versiya bo'yicha metrikalar jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

18-variant. Konchilik: konveyer tasmasi anomaliyasi — sun'iy yo'ldosh kanali, majburiy Edge va gibrid sinxronizatsiya oson

Soha: Konchilik

Uzoq hududdagi kon konveyer tasmasining 40 ta sensor tugunidan (vibratsiya + harorat, A3 8-variant 1D CNN 4 sinf) har 2 soniyada ma'lumot keladi — 20 so'rov/s. Kon internetga faqat sun'iy yo'ldosh orqali ulangan: 2 Mbit/s, RTT 600 ms, oyiga 20 GB trafik limiti, 1 GB ortig'i 8 USD. Nosozlik aniqlansa konveyer 1 soniyada to'xtatilishi kerak — bulut varianti RTT tufayli imkonsiz. Siz /belt xizmatini yozasiz, Edge (kon serveri) va bulut (RTT taqlidi) rejimlarini sinaysiz, trafikni hisoblaysiz, bitta tugunli k3s manifestini tekshirasiz va gibrid variantda bulutga nima yuborish mumkinligini (soatlik yig'indi, anomal segmentlar) trafik limiti ichida hisoblaysiz.

Model	1D CNN (256 nuqta -> 4 sinf) -> ONNX
Xizmat turi	anomaliya, /belt {signal[256], temp} -> {class, stop}
SLO	to'xtatish qarori <= 1 s, oflayn ishlash majburiy
Yuklama profili	20 so'rov/s doimiy
Xizmat vaqti (sim)	5 ms
Kanal	2 Mbit/s, RTT 600 ms, 20 GB/oy limit, 8 USD/GB ortig'i
Segment hajmi	256 x 4 bayt + sarlavha ~ 1,1 KB
Edge server	sanoat kompyuteri 1200 USD + 35 Vt, zaxira 1

QADAMLAR

1. 1D CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /belt, /readyz, /metrics (to'xtatish hisoblagichi); TestClient aniqlik va recall (nosozlik sinflari).
2. Yuklama sinovi (s2): Edge rejimi parallel 1/2/4/8; bulut taqlidi (600 ms sleep) bilan p95 — SLO 1 s qaysi parallellikda buziladi; Little qonuni bulut rejimida (little_N katta — nega).
3. Trafik hisobi: barcha segmentlar bulutga yuborilsa oyiga GB va ortiqcha to'lov; gibrid — faqat anomal segmentlar (2 %) + soatlik yig'indi (1 KB) — oyiga GB; model yangilash (hajm) trafigi.
4. Dockerfile (s3) 11 qoida; k3s manifesti (s4) bitta tugun, replicas 2, hostPath model volume, 18 qoida.
5. Simulyatsiya (s5 o'rniga): 7 kun, 1 daqiqa qadam, sun'iy yo'ldosh kanali kuniga 2 soat uziladi (ob-havo); bulut rejimida qancha so'rov javobsiz qoladi va necha marta konveyer «xavfsizlik rejimida» to'xtaydi; Edge rejimida 0.
6. TCO (s6) 1/3/5 yil: Edge (1+1 server) vs bulut (nusxa + trafik ortig'i) vs gibrid; xulosa: konga tavsiya va bulutga nima uchun faqat arxiv/qayta o'qitish ma'lumotlari ketadi.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Trafik hisobi jadvali (to'liq / gibrid)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

19-variant. Oziq-ovqat: restoranlar tarmog'ida talab prognozi — 300 filial, kuniga 3 burst va 300 ta Edge tasvirini tarqatish oson

Soha: Oziq-ovqat

Tez ovqatlanish tarmog'i 300 filialda har kuni 3 marta (06:00, 11:00, 16:00) keyingi 4 soat uchun taomlar talabini prognoz qiladi (vaqt qatori MLP, sintetik: hafta kuni, soat, ob-havo, aksiya). Har burst 5 daqiqa davom etadi: 300 filial x 40 taom = 12 000 so'rov, ya'ni 40 so'rov/s; qolgan vaqt 0,2 so'rov/s. Tarmoq xizmatni markaziy bulutda yoki har filialdagi kassa kompyuterida (Edge) ishlatishi mumkin. Siz /demand xizmatini yozasiz, yuklamani sinaysiz, Dockerfile tahlilida slim va to'liq variantlarni 300 filialga tarqatish hajmi va vaqti bo'yicha solishtirasiz, K8s manifestini tekshirasiz, HPA ni 3 burst profilida simulyatsiya qilasiz va TCO hisoblaysiz.

Model	MLP 12-128-64-1 (regressiya) -> ONNX
Xizmat turi	vaqt qatori prognozi, /demand {features[12]} -> {qty}; /demand_batch (40 taom)
SLO	burst 5 daqiqada tugashi, p95 <= 1 s
Yuklama profili	3 x kuniga 40 so'rov/s (5 daqiqa), boshqa vaqt 0,2
Xizmat vaqti (sim)	5 ms
HPA	min 1, max 4, nishon 60 %, start 20 s
Tasvir	slim+ORT (~270 MB) vs pytorch (~8 GB)
Filial kanali	20 Mbit/s

QADAMLAR

1. Sintetik talab ma'lumotida MLP'ni o'qitib ONNX'ga eksport qiling; FastAPI /demand, /demand_batch, /readyz, /metrics; TestClient MAE.
2. Yuklama sinovi (s2): yakka parallel 1/2/4/8; paketli (40 taom) so'rov; 12 000 prognozni ikkala rejimda bajarish vaqti.
3. Dockerfile (s3): slim va to'liq variantlar 11 qoida; 300 filialga tarqatish: jami GB, 20 Mbit/s kanalda har filial uchun tortish vaqti, registr trafigi; haftalik model yangilanishi bilan yillik trafik.
4. K8s manifest (s4) bulut varianti 18 qoida (minReplicas 1 ni asoslang yoki 2 qiling); Dockerfile HEALTHCHECK /readyz.
5. HPA simulyatsiyasi (s5): 24 soat, 1 daqiqa qadam (HPA davri 15 s ni moslang), 3 burst; min 1 vs min 2 vs CronHPA (burstdan 3 daqiqa oldin 3 nusxa); SLO buzilish, pod-soat.
6. TCO (s6) 1/3/5 yil: bulut (HPA) vs Edge (kassa kompyuteri allaqachon bor — faqat dastur + yangilash xarajati) vs gibrid; xulosa: tarmoqqa tavsiya va 300 filialga tasvir tarqatish nega slim bo'lishi shart.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Tasvir tarqatish jadvali (300 filial)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

20-variant. Davlat xizmatlari: hujjat raqamlarini tanish — kuniga 1 mln hujjat, onlayn xizmat vs batch job TCO oson

Soha: Davlat boshqaruvi

Davlat arxivi skanerlangan hujjatlardagi raqamli maydonlarni (digits CNN, 10 sinf) tanib bazaga yozadi: kuniga 1 million hujjat, har birida 12 ta raqam — 12 mln inferens; bundan tashqari, fuqarolar portalidan kuniga 20 000 ta onlayn so'rov (javob 2 s ichida) keladi. Siz /ocr_digit va /ocr_batch (bir hujjat 12 raqam) endpointlarini yozasiz, yuklamani sinab bitta nusxa sig'imini o'lchaysiz, kunlik arxivni onlayn xizmat orqali (doimiy nusxalar) yoki tungi batch Job (K8s Job, parallelism N) orqali qayta ishlash vaqtini va narxini hisoblaysiz, manifestlarni tekshirasiz va Edge (skaner stansiyasida tanish) variantini TCO ga qo'shasiz.

Model	kichik CNN (digits) -> ONNX, 10 sinf
Xizmat turi	tasvir, /ocr_digit {pixels[64]} va /ocr_batch [12 ta]
SLO	onlayn p95 <= 2 s; arxiv 1 mln hujjat 8 soatlik tun oynasida
Yuklama profili	onlayn 0,25 so'rov/s (kunduzi 0,6), arxiv 12 mln inferens/kun
Xizmat vaqti (sim)	2 ms / raqam
K8s Job	parallelism 2/4/8, har pod 2 vCPU
Narx	vCPU-soat 0,045 USD
Skaner stansiyalari	20 ta, har birida kompyuter bor

QADAMLAR

1. CNN'ni o'qitib ONNX'ga eksport qiling; FastAPI /ocr_digit, /ocr_batch (12 raqam, paket inferens), /readyz, /metrics; TestClient bilan 200 hujjatda raqam bo'yicha va hujjat bo'yicha (12 tasi ham to'g'ri) aniqlik.
2. Yuklama sinovi (s2): yakka va paketli so'rovlar parallel 1/2/4/8/16; bitta nusxa sig'imi (hujjat/s); 1 mln hujjat uchun 1 nusxada soatlar.
3. K8s manifestlari (s4): onlayn Deployment + HPA (18 qoida) va alohida Job manifesti (parallelism, completions, backoffLimit, activeDeadlineSeconds) — Job uchun 4 ta qoida; Dockerfile (s3).
4. Batch hisobi: parallelism 2/4/8 uchun tungi oynada tugash vaqti (o'lchangan throughput asosida) va vCPU-soat; 8 soat oynasiga sig'adigan minimal parallelism.
5. HPA simulyatsiyasi (s5): arxiv oqimi onlayn xizmatga kunduzi 10 soat davomida tekis yuborilsa (12 mln / 36 000 s = 333 inferens/s) va onlayn so'rovlar bilan aralashsa — onlayn p95 SLO buziladimi; ajratilgan Job bilan solishtiring.
6. TCO (s6) yillik: doimiy nusxalar (kunduzi arxiv + onlayn) vs onlayn kichik + tungi Job vs Edge (20 skaner stansiyasida tanish, bulutga faqat natija); xulosa: arxivga tavsiya va onlayn/batch ajratish nega to'g'ri.

Topshiriladi: Kod (.py: xizmat + yuklama sinovi + tekshiruvchi skriptlar); Natijalar jadvali (yuklama, HPA, TCO); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Batch Job hisobi

jadvali (parallelism 2/4/8)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A5

Taqsimlangan sun'iy intellekt tizimlarini ishlab chiqish

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A5
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A5
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

KTM laboratoriyalari — Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

Laboratoriya haqida

Bu virtual laboratoriyada talaba taqsimlangan o'qitish va parallel inferensning asosiy mexanizmlarini CPU sandbox'da bajaradi: torch.distributed (gloo backend) bilan 2 jarayonli DDP, kuchli/kuchsiz masshtablash o'lchovi, halqa all-reduce algoritmini qo'lda yozish, gradient to'plash ekvivalentligi, multiprocessing Pool bilan parallel inferens, Amdal/Gustafson qonunlari va GPU klasteri uchun aloqa/hisob kalkulyatori. Real labdan farqi: GPU va NCCL yo'q — 2 ta CPU yadro «2 ta GPU» rolini o'ynaydi, modellar kichik MLP, ma'lumot sintetik yoki sklearn digits; mantiq (process group, DistributedSampler, DDP o'rami, all-reduce) esa haqiqiy klasterdagi bilan bir xil.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A5`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A5 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. DDP: gloo backend bilan 2 jarayonli taqsimlangan o'qitish
2. 2-qadam. Masshtablash: 1 vs 2 jarayon, S(N) va E(N)
3. 3-qadam. Halqa all-reduce ni NumPy'da qo'lda yozish va aloqa hajmi
4. 4-qadam. Gradient to'plash va ma'lumotlar parallelligi ekvivalentligi
5. 5-qadam. Parallel inferens: sharding, ishchilar puli, natijalarni yig'ish
6. 6-qadam. Amdal va Gustafson qonunlari grafigi
7. 7-qadam. Aloqa/hisob nisbati kalkulyatori (GPU klasteri uchun baho)
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. DDP: gloo backend bilan 2 jarayonli taqsimlangan o'qitish

MAQSAD

torch.multiprocessing orqali 2 ta jarayon (rank 0 va 1) ishga tushiriladi, har biri jarayonlar guruhiga qo'shiladi, DistributedSampler ma'lumotni kesishmaydigan ikki qismga bo'ladi, DDP o'rami gradientlarni har qadamda all-reduce qiladi. Oxirida parametrlar barcha ranklarda bir xil ekani (max farq 0) va jami ko'rilgan namunalar soni tekshiriladi. Talaba `EPOCHS`, `BS`, LR masshtablash qoidasini (`0.05 * world`) va `set_epoch` chaqiruvini o'zgartirib ko'rishi mumkin.

> **Kutilgan natija:** Ikkala rank o'z 4096 namunasini ko'radi, jami 8192; aniqlik ~0.95; ranklar orasidagi parametr farqi 0.00e+00 (DDP sinxron); o'tkazuvchanlik bir necha o'n ming namuna/s.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. DDP: 2 jarayonli taqsimlangan o'qitish (gloo backend, CPU)
# GPU va NCCL yo'q — shuning uchun CPU uchun mo'ljallangan gloo backend ishlatiladi.
# Mantiq (process group, DistributedSampler, DDP o'rami, all-reduce) haqiqiy GPU klasteridagi
bilan bir xil.
import os, time, random
import torch, torch.nn as nn, torch.distributed as dist
import torch.multiprocessing as mp
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data.distributed import DistributedSampler
from torch.nn.parallel import DistributedDataParallel as DDP

N_SAMPLES, N_FEAT, N_CLS = 4096, 64, 8
EPOCHS, BS = 2, 64          # BS — har bir jarayon uchun paket (kuchsiz masshtablash rejimi)

def make_data():
    """Sintetik tasniflash ma'lumoti — barcha ranklarda bir xil urug' bilan yaratiladi."""
    g = torch.Generator().manual_seed(0)
    centers = torch.randn(N_CLS, N_FEAT, generator=g) * 0.6
    y = torch.randint(0, N_CLS, (N_SAMPLES,), generator=g)
    X = centers[y] + torch.randn(N_SAMPLES, N_FEAT, generator=g) * 1.5
    return TensorDataset(X, y)

def make_model():
    torch.manual_seed(0)
    return nn.Sequential(nn.Linear(N_FEAT, 256), nn.ReLU(), nn.Linear(256, 256), nn.ReLU(),
nn.Linear(256, N_CLS))

def worker(rank, world, port):
    """Har bir jarayon shu funksiyani bajaradi — torchrun'dagi bitta 'rank' ga teng."""
    os.environ["MASTER_ADDR"] = "127.0.0.1"
    os.environ["MASTER_PORT"] = str(port)
    os.environ["GLOO_SOCKET_IFNAME"] = "lo"      # tarmoq yo'q — faqat loopback interfeysi
    torch.set_num_threads(1)                      # har jarayonga 1 yadro (2 yadro = 2 jarayon)
    dist.init_process_group("gloo", rank=rank, world_size=world)

    ds = make_data()
    sampler = DistributedSampler(ds, num_replicas=world, rank=rank, shuffle=True, seed=0)
    dl = DataLoader(ds, batch_size=BS, sampler=sampler, drop_last=True)
    model = DDP(make_model())                     # 0-rank parametrlari hammaga broadcast
qilinadi
```

```
    opt = torch.optim.SGD(model.parameters(), lr=0.05 * world, momentum=0.9)   # chiziqli LR
qoidasi
    lossf = nn.CrossEntropyLoss()

    t0 = time.time(); seen = 0
    for ep in range(EPOCHS):
        sampler.set_epoch(ep)                    # har epoxada boshqa aralashtirish
        for X, y in dl:
            opt.zero_grad(set_to_none=True)
            loss = lossf(model(X), y)
            loss.backward()                      # gradientlar chelaklarda avtomatik all-
reduce qilinadi
            opt.step(); seen += X.size(0)
    dt = time.time() - t0

    # Global ko'rsatkichlarni yig'amiz: namunalar soni va oxirgi yo'qotish
    stats = torch.tensor([seen, loss.item()], dtype=torch.float64)
    dist.all_reduce(stats, op=dist.ReduceOp.SUM)
    # Parametrlar barcha ranklarda bir xilmi? (DDP to'g'ri ishlaganining isboti)
    flat = torch.cat([p.detach().flatten() for p in model.parameters()])
    mx = flat.clone(); mn = flat.clone()
    dist.all_reduce(mx, op=dist.ReduceOp.MAX); dist.all_reduce(mn, op=dist.ReduceOp.MIN)
    diff = (mx - mn).abs().max().item()
    # Aniqlik — to'liq to'plamda (har rank hisoblaydi, 0-rank chiqaradi)
    with torch.no_grad():
        acc = (model(ds.tensors[0]).argmax(1) == ds.tensors[1]).float().mean().item()
    print(f"rank={rank} world={world}  namunalar(shu rank)={seen}  vaqt={dt:.2f}s", flush=True)
    if rank == 0:
        print("=== DDP natijasi (gloo, 2 jarayon) ===")
        print(f"jami ko'rilgan namunalar : {int(stats[0].item())}  ({EPOCHS} epoxa x
{N_SAMPLES})")
        print(f"o'rtacha oxirgi yo'qotish: {stats[1].item() / world:.4f}")
        print(f"aniqlik (to'liq to'plam) : {acc:.3f}")
        print(f"ranklar orasidagi parametr farqi: {diff:.2e}  -> {'BIR XIL (DDP sinxron)' if
diff < 1e-6 else 'FARQ BOR!'}")
        print(f"o'tkazuvchanlik: {stats[0].item() / dt:.0f} namuna/s")
    dist.destroy_process_group()

if __name__ == "__main__":
    port = random.randint(20000, 60000)          # port band bo'lmasligi uchun tasodifiy
    world = 2
    print(f"Ishga tushirish: world_size={world}, backend=gloo, MASTER_PORT={port}")
    mp.start_processes(worker, args=(world, port), nprocs=world, start_method="fork",
join=True)
    print("Izoh: GPU klasterida backend='nccl', ishga tushirish torchrun bilan bo'ladi; kod
mantig'i o'zgarmaydi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== DDP natijasi (gloo, 2 jarayon) ===`, `BIR XIL (DDP sinxron)`

TAJRIBA QILIB KO‘RING

- `sampler.set_epoch(ep)` satrini izohga olib, ikki epoxada bir xil aralashtirish bo'lishini tekshiring (yo'qotish qanday o'zgaradi?).
- `DDP(make_model())` o'rniga oddiy `make_model()` qo'ying — parametr farqi endi nolga teng bo'ladimi? Nega?
- `lr=0.05 * world` ni `lr=0.05` qilib, yakuniy yo'qotishni solishtiring (chiziqli LR qoidasi).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Masshtablash: 1 vs 2 jarayon, S(N) va E(N)

MAQSAD

Bir xil MLP uchun to'rt konfiguratsiya o'lchanadi: 1 jarayon (1 yadro) — baza T(1); 2 jarayonli DDP kuchli rejimda (global paket o'zgarmaydi, har rankka yarmi); kuchsiz rejimda (har rankda to'liq paket, global 2 baravar); va taqqoslash uchun 1 jarayon 2 yadro bilan (intra-op parallellik). Tezlashish S(N) va samaradorlik E(N) jadvalda chiqadi. Talaba `HID`, `GB`, `STEPS` ni o'zgartirib, model kattalashganda samaradorlik qanday o'zgarishini ko'radi.

> **Kutilgan natija:** Baza ~4-5 s; DDP kuchli rejimda S(2)≈1.6-1.8 (E≈80-90%), kuchsiz rejimda E≈90%; intra-op 2 yadro ham ~1.9x. Qadam 15-25 s davom etadi.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Masshtablash: 1 jarayon vs 2 jarayon (kuchli va kuchsiz masshtablash), S(N) va E(N)
# CPU sandbox: 2 yadro. Har jarayon 1 yadro oladi — bu "2 ta GPU" ning CPU muqobili.
import os, time, random
import torch, torch.nn as nn, torch.distributed as dist
import torch.multiprocessing as mp
from torch.nn.parallel import DistributedDataParallel as DDP

N_FEAT, HID, N_CLS = 64, 1024, 8
STEPS = 60                      # o'lchanadigan optimizatsiya qadamlari soni

def make_model():
    torch.manual_seed(0)
    return nn.Sequential(nn.Linear(N_FEAT, HID), nn.ReLU(), nn.Linear(HID, HID), nn.ReLU(),
nn.Linear(HID, N_CLS))

def train_steps(model, bs, steps):
    """bs hajmli paketda `steps` ta qadam; vaqtni qaytaradi (2 ta isitish qadami hisobga
olinmaydi)."""
    opt = torch.optim.SGD(model.parameters(), lr=0.01)
    lossf = nn.CrossEntropyLoss()
    g = torch.Generator().manual_seed(1)
    X = torch.randn(bs, N_FEAT, generator=g); y = torch.randint(0, N_CLS, (bs,), generator=g)
    for i in range(steps + 2):
        if i == 2: t0 = time.time()
        opt.zero_grad(set_to_none=True)
        lossf(model(X), y).backward(); opt.step()
    return time.time() - t0

def worker(rank, world, port, bs_per_rank, q):
    os.environ.update(MASTER_ADDR="127.0.0.1", MASTER_PORT=str(port), GLOO_SOCKET_IFNAME="lo")
    torch.set_num_threads(1)
    dist.init_process_group("gloo", rank=rank, world_size=world)
    model = DDP(make_model())
    dt = train_steps(model, bs_per_rank, STEPS)
    if rank == 0: q.put(dt)
    dist.destroy_process_group()

def run_ddp(world, bs_per_rank):
    q = mp.get_context("fork").Queue()
    port = random.randint(20000, 60000)
    mp.start_processes(worker, args=(world, port, bs_per_rank, q), nprocs=world,
start_method="fork", join=True)
```

```
    return q.get()

if __name__ == "__main__":
    torch.set_num_threads(1)
    GB = 256                                          # global paket (kuchli masshtablash
uchun)
    t1 = train_steps(make_model(), GB, STEPS)         # T(1): bitta jarayon, 1 yadro
    t2_strong = run_ddp(2, GB // 2)                   # kuchli: global paket o'zgarmaydi -
> har rankka 128
    t2_weak = run_ddp(2, GB)                          # kuchsiz: har rankda 256 -> global
512
    torch.set_num_threads(2)
    t1_2thr = train_steps(make_model(), GB, STEPS)    # taqqoslash: 1 jarayon, lekin 2
yadro (intra-op)

    rows = [("1 jarayon (1 yadro) — baza", GB, t1, 1),
            ("2 jarayon DDP, kuchli (global 256)", GB, t2_strong, 2),
            ("2 jarayon DDP, kuchsiz (global 512)", 2 * GB, t2_weak, 2),
            ("1 jarayon, 2 yadro (intra-op)", GB, t1_2thr, 2)]
    print("=== Masshtablash jadvali (gloo, CPU) ===")
    print(f"{'Konfiguratsiya':38s} {'global paket':>12s} {'vaqt,s':>8s} {'namuna/s':>10s}
{'S(N)':>6s} {'E(N)':>7s}")
    for name, gb, t, n in rows:
        thr = gb * STEPS / t
        # kuchsiz rejimda tezlashish o'tkazuvchanlik nisbati bilan hisoblanadi
        s = (thr / (GB * STEPS / t1))
        print(f"{name:38s} {gb:12d} {t:8.2f} {thr:10.0f} {s:6.2f} {100 * s / n:6.0f}%")
    print("\nIzoh: S(N)=T(1)/T(N) (kuchsiz rejimda namuna/s nisbati), E(N)=S(N)/N.")
    print("Kuchli masshtablashda har rankka kamroq ish tegadi — samaradorlik odatda pastroq.")
    print("Haqiqiy GPU klasterida bir tugunda E(2) ~ 90-95%, tugunlar orasida 70-90%
kutiladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Masshtablash jadvali (gloo, CPU) ===`

TAJRIBA QILIB KO‘RING

- `HID` ni 1024 dan 256 ga tushiring: kichik modelda aloqa ulushi oshib, E(2) pasayishini kuzating.
- `GB` ni 64 qiling — kuchli masshtablashda har rankka 32 namuna tegadi; samaradorlik nima bo'ladi?
- DDP o'rnida `bucket_cap_mb=1` parametri bilan sinab ko'ring (ko'proq, kichikroq all-reduce).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Halqa all-reduce ni NumPy'da qo'lda yozish va aloqa hajmi

MAQSAD

N ta ishchining gradient vektorlari uchun halqa all-reduce (N-1 reduce-scatter + N-1 all-gather qadami) simulyatsiya qilinadi, natija oddiy yig'indi bilan solishtiriladi va har ishchi yuborgan baytlar `2(N-1)/N * S` formulasiga mos kelishi ko'rsatiladi. So'ng ResNet-50 gradientlari (102 MB) uchun NVLink, InfiniBand va 10 GbE tarmoqlarda aloqa vaqti hisoblanadi.

> **Kutilgan natija:** N=2,4,8 uchun «natija to'g'ri=True» va yuborilgan baytlar formulaga aynan teng; 10 GbE da 8 GPU uchun ~143 ms, NVLink'da 0.3 ms.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Halqa (ring) all-reduce ni NumPy'da qo'lda amalga oshirish va aloqa hajmini
hisoblash
import numpy as np

def ring_allreduce(grads):
    """grads: ro'yxat, har bir ishchining gradient vektori (bir xil uzunlik).
    Qaytaradi: har ishchidagi natija (yig'indi) va har ishchi yuborgan baytlar soni."""
    N = len(grads); L = len(grads[0])
    chunks = np.array_split(np.arange(L), N)           # vektor N bo'lakka bo'linadi
    buf = [g.astype(np.float32).copy() for g in grads]  # har ishchining mahalliy buferi
    sent = [0] * N
    # 1-bosqich: reduce-scatter — N-1 qadam, har qadamda ishchi i bo'lak (i-k) ni o'ng
qo'shnisiga yuboradi
    for k in range(N - 1):
        pkts = []
        for i in range(N):
            c = chunks[(i - k) % N]
            pkts.append((c, buf[i][c].copy())); sent[i] += len(c) * 4
        for i in range(N):
            c, data = pkts[(i - 1) % N]                # chap qo'shnidan kelgan bo'lak
            buf[i][c] += data                          # qo'shib qo'yiladi (reduce)
    # 2-bosqich: all-gather — N-1 qadam, tayyor bo'lgan bo'lak halqa bo'ylab tarqatiladi
    for k in range(N - 1):
        pkts = []
        for i in range(N):
            c = chunks[(i + 1 - k) % N]
            pkts.append((c, buf[i][c].copy())); sent[i] += len(c) * 4
        for i in range(N):
            c, data = pkts[(i - 1) % N]
            buf[i][c] = data                           # almashtiriladi (gather)
    return buf, sent

print("=== Ring all-reduce tekshiruvi ===")
rng = np.random.default_rng(0)
for N in (2, 4, 8):
    L = 1000
    grads = [rng.standard_normal(L) for _ in range(N)]
    out, sent = ring_allreduce(grads)
    ref = np.sum(grads, axis=0)
    ok = all(np.allclose(o, ref, atol=1e-4) for o in out)
    S = L * 4
```

```
    theory = 2 * (N - 1) / N * S
    print(f"N={N:2d}: natija to'g'ri={ok}  har ishchi yubordi={sent[0]} bayt  formula 2(N-
1)/N*S={theory:.0f} bayt  "
          f"(sodda 'hammaga yuborish' bo'lsa {(N-1)*S} bayt)")

print("\n=== Aloqa hajmi va vaqti: ResNet-50 (25.6 mln parametr, FP32 = 102 MB) ===")
S_bytes = 25.6e6 * 4
nets = {"NVLink (600 GB/s)": 600e9, "InfiniBand 100 Gbit/s": 100e9 / 8, "Ethernet 10 Gbit/s":
10e9 / 8}
print(f"{'N GPU':>6s} {'bayt/GPU, MB':>13s} " + " ".join(f"{k:>24s}" for k in nets))
for N in (2, 4, 8, 16, 64):
    per = 2 * (N - 1) / N * S_bytes
    print(f"{N:6d} {per / 1e6:13.1f} " + " ".join(f"{per / bw * 1e3:22.1f}ms" for bw in
nets.values()))
print("\nXulosa: N oshganda har GPU yuboradigan hajm 2S ga intiladi (ishchilar sonidan deyarli
bog'liq emas),")
print("lekin tarmoq sekin bo'lsa (10 GbE) qadam sayin ~160 ms aloqa — hisoblash vaqtiga teng
yoki undan ko'p.")
print("BF16 gradient yuborilsa hajm 2 baravar kamayadi; gradient to'plash all-reduce
chastotasini kamaytiradi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Ring all-reduce tekshiruvi ===`, `natija to'g'ri=True`

TAJRIBA QILIB KO‘RING

- `ring_allreduce` ni o'rtacha (SUM/N) qaytaradigan qilib o'zgartiring — DDP aynan shuni qiladi.
- Parametr-server sxemasini yozing (hamma 0-ishchiga yuboradi, u qaytaradi) va 0-ishchi yuboradigan hajmni N bo'yicha solishtiring.
- S_bytes ni BF16 (2 bayt) uchun qayta hisoblang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Gradient to'plash va ma'lumotlar parallelligi ekvivalentligi

MAQSAD

Bir xil 128 namunali global paket uchun uch usul taqqoslanadi: bitta katta paket; 4 ta mikro-paketda gradient to'plash (yo'qotish 1/K ga ko'paytiriladi); 2 «ishchi»da 64 tadan hisoblab gradientlarni o'rtachalash (DDP all-reduce'ning qo'lda nusxasi). Farqlar 1e-7 dan kichik chiqishi, 1/K ga bo'lmaslik esa gradientni K baravar kattalashtirishi ko'rsatiladi.

> **Kutilgan natija:** max|gA-gB| va max|gA-gC| ~1e-8; xato variantda farq ~0.3 va norma nisbati 4.00.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Gradient to'plash (accumulation) va ma'lumotlar parallelligi ekvivalentligi
# Uch usul bir xil 128 namunali global paket uchun bir xil gradient berishi kerak:
#  (A) bitta katta paket 128;  (B) 4 ta mikro-paket 32 — gradient to'planadi;
#  (C) "2 ishchi" har biri 64 — gradientlar o'rtachalanadi (DDP all-reduce ning qo'lda
nusxasi).
import torch, torch.nn as nn
torch.manual_seed(0)
N_FEAT, N_CLS, GB = 32, 5, 128
X = torch.randn(GB, N_FEAT); y = torch.randint(0, N_CLS, (GB,))
lossf = nn.CrossEntropyLoss()     # 'mean' — o'rtacha yo'qotish, shuning uchun bo'laklar vazni
muhim

def fresh():
    torch.manual_seed(1)
    return nn.Sequential(nn.Linear(N_FEAT, 64), nn.ReLU(), nn.Linear(64, N_CLS))

def flat_grad(m):
    return torch.cat([p.grad.flatten() for p in m.parameters()])

# (A) to'liq paket
mA = fresh(); lossf(mA(X), y).backward(); gA = flat_grad(mA)

# (B) gradient to'plash: 4 x 32, har mikro-paket yo'qotishi 1/4 ga ko'paytiriladi
mB = fresh(); K = 4
for i in range(K):
    xb, yb = X[i*32:(i+1)*32], y[i*32:(i+1)*32]
    (lossf(mB(xb), yb) / K).backward()        # .grad ichida yig'iladi — zero_grad
chaqirilmaydi
gB = flat_grad(mB)

# (C) ma'lumotlar parallelligi: 2 ishchi, har biri 64 namuna, so'ng all-reduce (o'rtacha)
workers = [fresh(), fresh()]                   # DDP broadcast — bir xil boshlang'ich
og'irliklar
grads = []
for r, m in enumerate(workers):
    lossf(m(X[r*64:(r+1)*64]), y[r*64:(r+1)*64]).backward()
    grads.append(flat_grad(m))
gC = torch.stack(grads).mean(0)                # all-reduce(SUM)/world_size

# (B') xato variant: 1/K ga bo'lmasdan to'plash — gradient K baravar katta chiqadi
mE = fresh()
for i in range(K):
    lossf(mE(X[i*32:(i+1)*32]), y[i*32:(i+1)*32]).backward()
gE = flat_grad(mE)
```

```
print("=== Gradient ekvivalentligi ===")
print(f"||gA|| (to'liq paket 128)            = {gA.norm():.6f}")
print(f"max|gA - gB| (to'plash 4x32)         = {(gA - gB).abs().max():.2e}")
print(f"max|gA - gC| (2 ishchi x 64, o'rtacha)= {(gA - gC).abs().max():.2e}")
print(f"max|gA - gE| (to'plash, 1/K siz!)    = {(gA - gE).abs().max():.2e}   <- xato:
||gE||/||gA|| = {gE.norm() / gA.norm():.2f}")
print("\nXulosa: 1 x 128 == 4 x 32 (to'plash) == 2 ishchi x 64 (DDP). Shuning uchun")
print("GPU yetmasa gradient to'plash bilan katta global paketni taqlid qilish mumkin, aloqa
chastotasi esa kamayadi.")
print("Eslatma: BatchNorm bo'lsa ekvivalentlik buziladi (statistika mikro-paketga bog'liq).")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Gradient ekvivalentligi ===`

TAJRIBA QILIB KO‘RING

- Modelga `nn.BatchNorm1d(64)` qo'shing va ekvivalentlik buzilishini ko'ring.
- `lossf = nn.CrossEntropyLoss(reduction='sum')` qilib, to'plashda 1/K kerak bo'lmay qolishini tekshiring.
- K=8, 2 ishchi x 4 mikro-paket kombinatsiyasini yozing (haqiqiy DDP+accumulation).

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Parallel inferens: sharding, ishchilar puli, natijalarni yig'ish

MAQSAD

Digits ustida o'qitilgan MLP og'irliklari NumPy'ga eksport qilinadi, ~360 ming so'rov 2048 talik bo'laklarga bo'linib multiprocessing Pool'ga tarqatiladi (scatter), natijalar tartib raqami bo'yicha yig'iladi (gather). 1 va 2 ishchi uchun o'tkazuvchanlik, S(N), E(N) va bo'lak kechikishining p50/p99 qiymatlari jadvalda chiqadi.

> **Kutilgan natija:** 1 ishchi ~0.5-0.6 s, 2 ishchi ~0.3 s; S(2)≈1.7-1.9, E≈85-95%; aniqlik ikkala holatda bir xil (0.993).

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Parallel inferens: ma'lumotni bo'lib yuborish (sharding), ishchilar puli,
natijalarni yig'ish
# Model — digits ustida o'qitilgan kichik MLP; inferens NumPy'da (har ishchi o'z nusxasi
bilan).
import os
os.environ["OMP_NUM_THREADS"] = "1"        # har ishchi 1 yadro — BLAS ichki parallelligi
o'chiriladi
import time, numpy as np, torch, torch.nn as nn
import multiprocessing as mp
from sklearn.datasets import load_digits

def train_model():
    torch.manual_seed(0); torch.set_num_threads(1)
    d = load_digits(); X = torch.tensor(d.data / 16.0, dtype=torch.float32); y =
torch.tensor(d.target)
    m = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 10))
    opt = torch.optim.Adam(m.parameters(), lr=3e-3)
    for _ in range(150):
        opt.zero_grad(); nn.functional.cross_entropy(m(X), y).backward(); opt.step()
    acc = (m(X).argmax(1) == y).float().mean().item()
    W = [(l.weight.detach().numpy().T.copy(), l.bias.detach().numpy().copy()) for l in m if
isinstance(l, nn.Linear)]
    return W, d.data / 16.0, d.target, acc

W_GLOBAL = None
def init_worker(W):
    global W_GLOBAL; W_GLOBAL = W          # fork: model nusxasi har ishchiga ko'chiriladi

def predict_shard(args):
    idx, X = args                          # idx — bo'lakning tartib raqami (natijani to'g'ri
tartibda yig'ish uchun)
    (W1, b1), (W2, b2) = W_GLOBAL
    t0 = time.perf_counter()
    h = np.maximum(X @ W1 + b1, 0); out = (h @ W2 + b2).argmax(1)
    return idx, out, time.perf_counter() - t0

if __name__ == "__main__":
    W, X, y, acc = train_model()
    # inferens yukini kattalashtiramiz: 1797 ta rasmni 200 marta takrorlab ~360k so'rov
    Xbig = np.tile(X, (200, 1)).astype(np.float32); ybig = np.tile(y, 200)
    SHARD = 2048
```

```
    shards = [(i, Xbig[s:s + SHARD]) for i, s in enumerate(range(0, len(Xbig), SHARD))]
    print(f"model aniqligi={acc:.3f}; so'rovlar={len(Xbig)}; bo'laklar={len(shards)} x
{SHARD}")
    print("=== Parallel inferens o'tkazuvchanligi ===")
    print(f"{'ishchilar':>9s} {'vaqt,s':>8s} {'so\'rov/s':>10s} {'S(N)':>6s} {'E(N)':>6s} {'p50
bo\'lak,ms':>13s} {'p99 bo\'lak,ms':>13s} {'aniqlik':>8s}")
    base = None
    for nw in (1, 2):
        ctx = mp.get_context("fork")
        with ctx.Pool(nw, initializer=init_worker, initargs=(W,)) as pool:
            pool.map(predict_shard, shards[:2])          # isitish
            t0 = time.perf_counter()
            res = pool.map(predict_shard, shards)         # scatter -> hisob -> gather
            dt = time.perf_counter() - t0
        res.sort(key=lambda r: r[0])                      # all-gather: tartib bo'yicha yig'ish
        pred = np.concatenate([r[1] for r in res]); lat = np.array([r[2] for r in res]) * 1e3
        thr = len(Xbig) / dt
        base = base or thr
        s = thr / base
        print(f"{nw:9d} {dt:8.3f} {thr:10.0f} {s:6.2f} {100*s/nw:5.0f}%
{np.percentile(lat,50):13.2f} {np.percentile(lat,99):13.2f} {(pred==ybig).mean():8.3f}")
    print("\nXulosa: inferensda qadam sayin sinxronizatsiya yo'q ('noqulay parallel'), shuning
uchun")
    print("masshtablash o'qitishga qaraganda yaxshiroq; cheklov — bo'laklarni tarqatish/yig'ish
(IPC) xarajati.")
    print("Real xizmatda esa p50/p99 kechikish va dinamik paketlash (dynamic batching) ham
o'lchanadi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Parallel inferens o'tkazuvchanligi ===`

TAJRIBA QILIB KO‘RING

- `SHARD` ni 128 va 16384 qilib, IPC xarajati va p99 kechikishga ta'sirini o'lchang.
- Ishchilar sonini 4 qiling (2 yadroda) — samaradorlik nega tushadi?
- `res.sort(...)` satrini olib tashlang va `pool.imap_unordered` bilan aniqlik hisobida nima buzilishini ko'ring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Amdal va Gustafson qonunlari grafigi

MAQSAD

Parallel ulush p=0.90/0.95/0.99 uchun Amdal (kuchli masshtablash) va Gustafson (kuchsiz) tezlashish egri chiziqlari N=1..256 oralig'ida hisoblanadi va log-log grafikda chiziladi; har N uchun E(N) jadvali va $N \to \infty$ chegarasi chiqadi. Talaba o'z o'lchovlaridan (2-qadam) p ni baholab, grafikka qo'shishi mumkin.

> **Kutilgan natija:** Jadval: p=0.90 da S(256)≈9.7 (chegara 10x), p=0.99 da 72; Gustafson deyarli chiziqli. Ikki panelli grafik.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Amdal va Gustafson qonunlari: nazariy tezlashish chegaralari (grafik + jadval)
import numpy as np, matplotlib.pyplot as plt

N = np.array([1, 2, 4, 8, 16, 32, 64, 128, 256])
fracs = [0.90, 0.95, 0.99]           # parallel qilinadigan ulush p

def amdahl(p, n):    return 1.0 / ((1 - p) + p / n)            # kuchli masshtablash (ish hajmi
o'zgarmas)
def gustafson(p, n): return (1 - p) + p * n                     # kuchsiz masshtablash (ish
hajmi N bilan o'sadi)

print("=== Amdal qonuni: S(N) = 1 / ((1-p) + p/N) ===")
print(f"{'N':>4s} " + " ".join(f"p={p:.2f}  S / E%" for p in fracs))
for n in N:
    print(f"{n:4d} " + " ".join(f"{amdahl(p, n):8.2f} /{100*amdahl(p, n)/n:4.0f}%" for p in
fracs))
print("\n=== Gustafson qonuni: S(N) = (1-p) + p*N ===")
for n in (8, 64, 256):
    print(f"N={n:3d}: " + ", ".join(f"p={p:.2f} -> S={gustafson(p, n):6.1f}" for p in fracs))

print("\nAmdal chegarasi (N -> cheksiz): " + ", ".join(f"p={p:.2f} -> {1/(1-p):.0f}x" for p in
fracs))
# Masalan: ResNet-18 DDP da aloqa ulushi ~12% (4 GPU, PCIe) -> ketma-ket ulush ~0.12 ->
maksimal ~8x
p_ddp = 1 - 0.12
print(f"Misol: aloqa ulushi 12% (ketma-ket deb olsak) -> p={p_ddp:.2f}, S(4)=
{amdahl(p_ddp,4):.2f}, E(4)={100*amdahl(p_ddp,4)/4:.0f}%, chegara {1/(1-p_ddp):.1f}x")

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
for p in fracs:
    ax[0].plot(N, amdahl(p, N), marker="o", label=f"p={p}")
    ax[1].plot(N, gustafson(p, N), marker="s", label=f"p={p}")
ax[0].plot(N, N, "k--", lw=1, label="ideal"); ax[1].plot(N, N, "k--", lw=1, label="ideal")
ax[0].set(xscale="log", yscale="log", title="Amdal (kuchli masshtablash)", xlabel="N
(ishchilar)", ylabel="S(N)")
ax[1].set(xscale="log", yscale="log", title="Gustafson (kuchsiz masshtablash)", xlabel="N
(ishchilar)")
for a in ax: a.grid(alpha=.3); a.legend()
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Amdal qonuni`, `=== Gustafson qonuni`

TAJRIBA QILIB KO‘RING

- 2-qadamdagi E(2) dan p ni toping: p = 2(1 - 1/S(2)) va Amdal egri chizig'iga nuqta sifatida qo'shing.
- Aloqa vaqtini N bilan o'suvchi qilib (masalan, p ulushi N oshganda kamayadi) egri chiziqning pasayishini modellashtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Aloqa/hisob nisbati kalkulyatori (GPU klasteri uchun baho)

MAQSAD

Analitik model: T_step = T_comp + (1-overlap)·T_comm, bunda T_comp FLOPs/(cho'qqi·MFU), T_comm — halqa all-reduce hajmi/tarmoq o'tkazuvchanligi. ResNet-18/50, ViT-B, GPT-2 1.5B uchun 4 xil tarmoqda r = T_comm/T_comp va E(N) hisoblanadi, gradient to'plashning ta'siri ko'rsatiladi, ResNet-50 uchun E(N) grafigi chiziladi. Talaba o'z klasteri parametrlarini (PEAK, MFU, BATCH, BYTES_PER_GRAD) kiritadi.

> **Kutilgan natija:** NVLink'da E≈99%, 10 GbE da ResNet-50 uchun E≈26% (accum=8 bilan 74%); katta model (ViT-B, GPT-2) yaxshiroq masshtablanadi.

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Aloqa/hisob nisbati kalkulyatori — GPU klasteri uchun taxminiy baho (CPU'da faqat
arifmetika)
# Model: T_step = T_comp + (1 - overlap) * T_comm,  T_comm = 2(N-1)/N * S / BW,  T_comp =
FLOPs_step / (peak * MFU)
import numpy as np, matplotlib.pyplot as plt

MODELS = {   # nom: (parametrlar, FLOPs bir namuna uchun forward, GFLOP)
    "ResNet-18": (11.7e6, 1.8), "ResNet-50": (25.6e6, 4.1), "ViT-B/16": (86e6, 17.6), "GPT-2
1.5B (seq 1024)": (1.5e9, 3.0e3)}
NETS = {"NVLink 600GB/s": 600e9, "IB 400Gbit/s": 50e9, "IB 100Gbit/s": 12.5e9, "Eth 10Gbit/s":
1.25e9}
PEAK, MFU = 312e12, 0.40          # A100 BF16 cho'qqi va amaldagi foydalanish (Model FLOPs
Utilization)
BATCH, BYTES_PER_GRAD, OVERLAP, ACCUM = 128, 2, 0.5, 1   # paket/GPU, BF16 gradient, overlap
ulushi, to'plash qadamlari

def estimate(params, gflop, N, bw, batch=BATCH, bpg=BYTES_PER_GRAD, accum=ACCUM,
overlap=OVERLAP):
    t_comp = 3 * gflop * 1e9 * batch * accum / (PEAK * MFU)         # forward + backward ~ 3x
forward
    t_comm = (2 * (N - 1) / N) * params * bpg / bw if N > 1 else 0.0  # bitta all-reduce
    t_step = t_comp + (1 - overlap) * t_comm
    return t_comp, t_comm, t_comp / (t_comp + (1 - overlap) * t_comm)   # samaradorlik bahosi
E(N)

print("=== Aloqa/hisob nisbati (N=8 GPU, paket 128/GPU, BF16 gradient, overlap 50%) ===")
print(f"{'Model':24s} {'T_comp,ms':>10s} " + " ".join(f"{k[:14]:>21s}" for k in NETS))
for name, (p, g) in MODELS.items():
    cells = []
    for bw in NETS.values():
        tc, tm, e = estimate(p, g, 8, bw)
        cells.append(f"{tm*1e3:7.1f}ms r={tm/tc:5.2f} E={e*100:3.0f}%")
    print(f"{name:24s} {tc*1e3:10.2f} " + " ".join(f"{c:>21s}" for c in cells))
print("r = T_comm/T_comp (aloqa/hisob nisbati); r << 1 bo'lsa masshtablash yaxshi.")

print("\n=== Gradient to'plash ta'siri: ResNet-50, 8 GPU, 10 GbE ===")
for acc in (1, 2, 4, 8):
    tc, tm, e = estimate(*MODELS["ResNet-50"], 8, NETS["Eth 10Gbit/s"], accum=acc)
    print(f"accum={acc}: T_comp={tc*1e3:6.1f}ms  T_comm={tm*1e3:6.1f}ms  r={tm/tc:.2f}  E=
{e*100:.0f}%")
```

```
# Grafik: samaradorlik N ga bog'liq, ResNet-50, turli tarmoqlar
Ns = [1, 2, 4, 8, 16, 32, 64]
plt.figure(figsize=(7, 4))
for net, bw in NETS.items():
    plt.plot(Ns, [100 * estimate(*MODELS["ResNet-50"], n, bw)[2] for n in Ns], marker="o",
label=net)
plt.xscale("log", base=2); plt.ylim(0, 105); plt.xlabel("N GPU"); plt.ylabel("E(N), %")
plt.title("ResNet-50 DDP samaradorligi bahosi (taxminiy model)"); plt.grid(alpha=.3);
plt.legend(fontsize=8)
plt.tight_layout()
print("\nIzoh: bu analitik model; haqiqiy qiymat profiler (torch.profiler, Nsight) bilan
o'lchanadi.")
print("Kattaroq model (ViT-B) kichigidan (ResNet-18) yaxshiroq masshtablanadi — hisob tezroq
o'sadi, aloqa emas.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Aloqa/hisob nisbati`, `=== Gradient to'plash ta'siri`

TAJRIBA QILIB KO‘RING

- BYTES_PER_GRAD=4 (FP32) va 1 (FP8/siqish) uchun jadvalni qayta hisoblang.
- OVERLAP=0 va 0.9 — DDP chelak overlap'i qanchalik muhim?
- O'zingizning model (parametr soni, GFLOP) va tarmog'ingizni qo'shib, E(8) ni baholang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Epoxa va paket ta'siri** 1-qadam kodida EPOCHS=4 va BS=32, 64, 128 uchun DDP ni ishga tushiring. Har variant uchun o'tkazuvchanlik (namuna/s), oxirgi yo'qotish va aniqlikni jadvalga yozing; paket kattalashganda o'tkazuvchanlik nega oshishini 2-3 jumla bilan tushuntiring.	oson	5
2	**Aloqa hajmi jadvali** 3-qadam formulasidan foydalanib ResNet-18 (11.7 mln), ViT-B/16 (86 mln) va 7B parametrli model uchun N=2, 8, 64 GPU'da har GPU yuboradigan hajmni (FP32 va BF16) va 100 Gbit/s InfiniBand'dagi vaqtni hisoblang. Natijani 3x3 jadval ko'rinishida chiqaring va 7B model uchun DDP nega yaroqsiz ekanini izohlang.	oson	5
3	**Amdal bo'yicha p ni baholash** 2-qadamdagi o'lchovingizdan (kuchli rejim) S(2) ni olib, Amdal formulasidan ketma-ket ulush (1-p) ni toping. Shu p bilan S(4), S(8), S(64) ni hisoblang va 6-qadam grafigiga o'lchangan nuqtani qo'shing. Hisobotda p qiymati va maksimal nazariy tezlashish keltiriladi.	oson	5
4	**Kuchli vs kuchsiz masshtablash egri chizig'i** 2-qadam kodini global paket GB=64, 128, 256, 512 uchun tsiklda ishga tushiring va har GB uchun kuchli hamda kuchsiz rejimda E(2) ni o'lchang. Ikki egri chiziqni bitta matplotlib grafikda chizing (x — GB, y — E(2) %). Qaysi paketdan boshlab kuchli masshtablash «to'yinmay» qolishini aniqlang.	o'rta	10
5	**Parametr-server vs halqa** 3-qadamdagi NumPy simulyatsiyasiga parametr-server (PS) sxemasini qo'shing: barcha ishchilar gradientni 0-ishchiga yuboradi, u yig'ib qaytaradi. N=2, 4, 8, 16, 32 uchun eng band tugun yuboradigan+qabul qiladigan hajmni ikkala sxemada hisoblang, natijani jadval va grafikda ko'rsating va PS sxemasi qaysi N dan boshlab 10 GbE da 100 ms dan oshishini toping.	o'rta	10
6	**Gradient to'plash bilan DDP** 4-qadam g'oyasini 1-qadamdagi haqiqiy DDP kodiga ko'chiring: `model.no_sync()` konteksti yordamida K=4 mikro-paketdan keyin bitta all-reduce bajaring. K=1 va K=4 uchun 2 jarayonli o'qitish vaqtini va yakuniy yo'qotishni solishtiring; all-reduce chaqiruvlari soni K baravar kamayganini tushuntiring.	o'rta	10
7	**Inferens kechikish profili** 5-qadam kodida bo'lak hajmini SHARD = 64, 256, 1024, 4096, 16384 qilib, 1 va 2 ishchi uchun o'tkazuvchanlik hamda p50/p99 kechikishni o'lchang. Ikki o'qli grafik (o'tkazuvchanlik va p99) chizing va onlayn xizmat uchun (p99 < 5 ms sharti) optimal bo'lak hajmini asoslang.	o'rta	10
8	**Qo'lda DDP: all_reduce bilan** DDP o'ramisiz 2 jarayonli o'qitish yozing: 1-qadam modelida har `backward()` dan keyin har parametr gradienti uchun `dist.all_reduce(p.grad)` va `/ world_size` bajaring, boshida esa 0-rank parametrlarini `dist.broadcast` bilan tarqating. Natija	qiyin	15

	DDP varianti bilan bir xil parametrlarga (farq < 1e-6) olib kelishini ko'rsating va ikkala variant vaqtini solishtiring (DDP chelak/overlap afzalligi).		
9	**Sekin ishchi (straggler) simulyatsiyasi** 1-qadam DDP kodida rank 1 ga har qadamda `time.sleep(0.002)` qo'shib sun'iy sekinlashtiring. 2 jarayon vaqti qanday o'zgaradi va nega rank 0 ham sekinlashadi (sinxron SGD)? Keyin 3 ta turli kechikish (0, 1, 3 ms) uchun E(2) jadvalini tuzing va sekin ishchi muammosini kamaytiradigan ikkita yondashuvni (masalan, asinxron SGD, elastik o'qitish) taqqoslab yozing.	qiyin	15
10	**Klaster konfiguratsiyasi tanlovi** 7-qadam kalkulyatorini kengaytiring: berilgan model (parametr soni va GFLOP talaba tanlaydi), byudjet (GPU soni ≤ 32) va uchta tarmoq varianti uchun qadam vaqtini, E(N) ni va «1 epoxa narxi»ni (GPU-soat) hisoblaydigan funksiya yozing. Eng kam GPU-soat beradigan (N, tarmoq, accum) kombinatsiyasini toping, natijani issiqlik xaritasi (N x accum) ko'rinishida chizing va tanlovni asoslang.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Telekom: abonent segmentlari tasnifi — 2 jarayonli DDP va paket ta'siri

oson

Soha: Telekom (mobil operator)

Mobil operator abonentlarning kunlik chaqiruv va trafik yozuvlaridan (CDR) 6 ta tarif segmentini bashorat qiluvchi MLP'ni 2 GPU'li serverda o'qitmoqchi. Sandbox'da GPU yo'q, shuning uchun 2 CPU yadro va gloo backend bilan DDP mantig'i aynan shu tarzda tekshiriladi. Talab: har rank ma'lumotning kesishmaydigan yarmini ko'rishi, parametrlar sinxron bo'lishi (farq 0), o'tkazuvchanlik uchta paket hajmi uchun o'lchanishi va 10 Gb/s tarmoqda aloqa vaqti baholanishi.

Ma'lumot	sintetik CDR: make_classification(n=6000, 40 belgi, n_informative=20, 6 sinf, random_state=7)
Model	MLP 40-192-192-6 (taxminan 46 ming parametr)
Paket (har rank)	32, 64, 128
Epoxa	3
LR qoidasi	0.03 * world_size, momentum 0.9
Tarmoq (kalkulyator)	10 Gb/s
Ishchilar	2 jarayon, gloo, 1 yadro/jarayon

QADAMLAR

1. Laboratoriya s1 kodini make_classification ma'lumotiga moslang (X ni float32, standartlang). BS=32, 64, 128 va EPOCHS=3 uchun DDP'ni ishga tushiring; har holat uchun jami ko'rilgan namunalar (36000 bo'lishi kerak), namuna/s, oxirgi yo'qotish, aniqlik va ranklar orasidagi parametr farqini jadvalga yozing.
2. s2 kabi HID=192, GB=128, STEPS=60 bilan 4 konfiguratsiyani (1 jarayon; DDP kuchli; DDP kuchsiz; 1 jarayon 2 yadro) o'lchang, S(2) va E(2) ni hisoblang.
3. s3 ring all-reduce funksiyasini N=2, 4, 8 uchun tekshiring (natija to'g'ri=True), so'ng o'z modelingiz gradienti (46 ming parametr, FP32) va ResNet-18 (11.7 mln) uchun 1, 10, 100 Gb/s tarmoqlarda bitta all-reduce vaqtini N=2, 8, 32 da hisoblang.
4. s4 kabi 96 namunali global paket uchun 1x96, 3x32 (to'plash, 1/K bilan va 1/K siz) va 2 ishchi x 48 gradientlarini solishtiring; max farqlarni chiqaring.
5. s5 kabi o'qitilgan modelni NumPy'ga eksport qilib, 300 ming sintetik CDR so'rovini SHARD=1024 bilan 1 va 2 ishchida bajaring; so'rov/s, S(2), E(2), p50/p99 bo'lak kechikishini jadvalga kiriting.
6. 2-qadamdagi kuchli rejim S(2) dan Amdal bo'yicha p ni baholang, S(8) va S(32) hamda chegarani hisoblang; s6 grafigiga o'lchangan nuqtani qo'shing.
7. Xulosa: paket kattalashganda o'tkazuvchanlik nega oshadi, 10 Gb/s tarmoq shu model uchun to'siq bo'ladimi, 2 GPU serverga qaysi paketni tavsiya qilasiz.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun

tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

2-variant. Agro: tuproq namligi sensorlari — kuchli va kuchsiz masshtablash

oson

Soha: Agro

Agroklaster 8 ta tuproq sensorining 24 soatlik o'lchovlari (192 belgi) bo'yicha dalaning sug'orish holatini 4 sinfga (shoshilinch, bugun, 2 kundan keyin, kerak emas) ajratuvchi MLP'ni fermer xo'jaligi serverida (2 ta GPU, tugunlar orasida 1 Gb/s tarmoq) o'qitmoqchi. Sizdan: CPU sandbox'da kuchli va kuchsiz masshtablashni ikki model o'lchami uchun o'lchash va 1 Gb/s tarmoqda aloqa vaqtini baholab, qaysi rejim maqsadga muvofiqligini aytish talab qilinadi.

Ma'lumot	sintetik sensor matritsasi: 8000 x 192 (sinus kunlik sikl + shovqin), 4 sinf
Model	MLP 192-HID-HID-4, HID = 256 va 768
Global paket	128 va 256
STEPS	50
Tarmoq	1 Gb/s (tugunlar orasida), NVLink muqobili yo'q
Gradient formati	FP32 va BF16

QADAMLAR

1. Sensor generatorini yozing: har sensor uchun kunlik sinus + tasodifiy shovqin, sinf namlik o'rtachasi bo'yicha; s1 DDP kodini shu ma'lumotga moslang, EPOCHS=2, BS=64; parametr farqi 0 ekanini ko'rsating.
2. s2 kodini HID=256 va HID=768, GB=128 va 256 uchun (4 kombinatsiya) ishga tushiring; har biri uchun T(1), kuchli va kuchsiz DDP vaqtini, S(2), E(2) ni jadvalga yozing.
3. E(2) ni HID bo'yicha ikki rejimda ustunli diagrammada chizing; kichik modelda kuchli rejim samaradorligi nega past ekanini izohlang.
4. s3 formulasidan HID=768 modeli gradient hajmini (parametrlar sonini kod bilan hisoblang) FP32 va BF16 da toping va 1 Gb/s tarmoqda N=2, 4, 8 uchun all-reduce vaqtini hisoblang; ring simulyatsiyasini N=4 da tekshiring.
5. s4 kabi gradient to'plash K=2 va K=4 ekvivalentligini 128 namunali paketda tekshiring (farq < 1e-6).
6. s6 Gustafson formulasi bilan kuchsiz rejim uchun S(N) ni N=2..64 da chizing, o'lchangan E(2) dan p ni baholang.
7. Xulosa: fermer xo'jaligi klasteri uchun kuchsiz masshtablash + BF16 + gradient to'plash kombinatsiyasini raqamlar bilan asoslang.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

3-variant. Moliya: firibgarlik detektori gradientlari uchun halqa all-reduce va tarmoq tanlovi oson

Soha: Moliya (to'lov tizimi)

To'lov tizimi uchta nomzod modelni ko'rib chiqmoqda: kichik MLP (2.1 mln parametr), jadval-transformer (45 mln) va matnli izohlar uchun 7 mlrd parametrli til modeli. Infratuzilma bo'limi 1, 10 va 100 Gb/s tarmoq variantlarini taklif qiladi. Sizdan har model va tarmoq uchun bitta all-reduce vaqtini hisoblash, NumPy ring all-reduce simulyatsiyasini tekshirish va qaysi model DDP bilan umuman masshtablanmasligini ko'rsatish talab qilinadi.

Modellar	MLP 2.1 mln; TabTransformer 45 mln; LLM 7 mlrd parametr
Tarmoqlar	1, 10, 100 Gb/s
N (GPU)	2, 4, 8, 16, 64
Gradient formati	FP32 (4 bayt) va BF16 (2 bayt)
Simulyatsiya vektori	L = 2400 element, N = 2, 3, 6
Hisob vaqti (taqqoslash)	T_comp = 120 ms/qadam

QADAMLAR

1. s3 ring_allreduce funksiyasini L=2400 va N=2, 3, 6 (N ga bo'linmaydigan uzunlik!) uchun ishga tushiring; natija to'g'ri=True va yuborilgan baytlar 2(N-1)/N*S formulasiga mos ekanini ko'rsating.
2. Funksiyani SUM/N (o'rtacha) qaytaradigan qilib o'zgartiring va DDP shu rejimda ishlashini izohlang.
3. Uchta model x uchta tarmoq x N=2,4,8,16,64 uchun har GPU yuboradigan hajm (MB) va aloqa vaqti (ms) jadvalini FP32 da chiqaring; BF16 uchun takrorlang.
4. T_comp=120 ms deb olib, r = T_comm/T_comp va E(N) = T_comp/(T_comp+T_comm) ni hisoblang; E(8) < 50% bo'lgan kombinatsiyalarni belgilang.
5. s1 DDP kodini make_classification(n=5000, 30 belgi, 2 sinf, weights=[0.97, 0.03]) firibgarlik ma'lumotiga moslang va 2 jarayonda parametr sinxronligini tekshiring (DDP mantig'i kichik modelda).
6. Xulosa: 7 mlrd model uchun DDP nega yaroqsiz (xotira va 2S hajm), FSDP/ZeRO qanday yordam beradi, 45 mln model uchun minimal tarmoq qaysi.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

4-variant. Tibbiyot: EKG segmentlari tasnifi — gradient to'plash ekvivalentligi

oson

Soha: Tibbiyot (kardiologiya)

Kardiologiya markazi 256 nuqtali EKG segmentlarini 3 sinfga (normal, aritmiya, artefakt) ajratuvchi modelni o'qitadi. Klinika serverida xotira kam, shuning uchun 192 namunali global paketni bir yo'la sig'dirib bo'lmaydi — gradient to'plash yoki 2 GPU'da ma'lumotlar parallelligi kerak. Sizdan to'plash, DDP o'rtachalash va bitta katta paket bir xil gradient berishini isbotlash, xato variantlarni (1/K siz, BatchNorm bilan) ko'rsatish talab qilinadi.

Ma'lumot	sintetik EKG: 256 nuqta, sinus + QRS impuls + shovqin, 3 sinf, n=3000
Model	MLP 256-96-3 (BatchNorm bilan va BatchNorm'siz)
Global paket	192
K (mikro-paket)	3 va 6
Ishchilar (qo'lda DDP)	2 x 96 va 4 x 48
Loss reduction	mean va sum

QADAMLAR

1. EKG generatorini yozing (har sinf uchun boshqa impuls shakli); s4 kodini 192 namunali paketga moslang: (A) 1x192, (B) K=3 to'plash 1/K bilan, (C) 2 ishchi x 96 o'rtacha; max farqlarni chiqaring (< 1e-6).
2. K=6 va 4 ishchi x 48 variantlarini qo'shing; (B') 1/K siz to'plashda ||gE||/||gA|| nisbati K ga teng ekanini ko'rsating.
3. Modelga nn.BatchNorm1d(96) qo'shing va ekvivalentlik buzilishini raqam bilan ko'rsating; nega buzilishini izohlang (SyncBatchNorm haqida 2 jumla).
4. reduction='sum' bilan to'plashda 1/K kerak emasligini tekshiring va LR qanday o'zgarishi kerakligini yozing.
5. s1 DDP kodini shu EKG ma'lumotiga moslang (BS=48, EPOCHS=3) va 2 jarayonda parametr farqi 0 ekanini ko'rsating; namuna/s ni o'lchang.
6. s3 formulasi bilan model gradienti (parametr sonini hisoblang) uchun 1 Gb/s klinika tarmog'ida all-reduce vaqtini N=2 da toping va to'plash K=6 aloqa chastotasini qanday kamaytirishini jadvalda ko'rsating.
7. Xulosa: xotira cheklangan klinika serverida qaysi usulni (to'plash yoki DDP) tavsiya qilasiz va nima uchun.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

5-variant. Chakana savdo: mahsulot toifasini aniqlash inferensi — ishchilar puli

oson

Soha: Chakana savdo

Onlayn do'kon har kecha 500 ming yangi mahsulot kartochkasini 12 toifaga avtomatik ajratadi. Hozir bu bitta jarayonda 40 daqiqa olmoqda; rahbariyat 2 va 4 yadroli serverda parallel inferens qanchalik tezlashtirishini bilmoqchi. Sizdan multiprocessing Pool bilan sharding, natijalarni tartib bilan yig'ish va bo'lak hajmining o'tkazuvchanlik hamda kechikishga ta'sirini o'lchash talab qilinadi.

Ma'lumot	make_classification(n=5000, 50 belgi, 12 sinf, n_informative=30, random_state=3), 100 marta takrorlab 500 ming so'rov
Model	MLP 50-128-12, Adam, 200 qadam
SHARD	512, 2048, 8192
Ishchilar	1, 2 (va 4 — 2 yadroda sinov)
SLO	p99 bo'lak kechikishi < 8 ms
Yig'ish	tartiblangan map va imap_unordered

QADAMLAR

1. s5 kodini make_classification ma'lumotiga moslang: modelni o'qiting (aniqlikni yozing), og'irliklarni NumPy'ga eksport qiling, 500 ming so'rovni tayyorlang.
2. SHARD=2048 bilan 1 va 2 ishchi uchun so'rov/s, S(2), E(2), p50/p99 bo'lak kechikishini o'lchang; aniqlik ikkala holatda bir xil ekanini tekshiring.
3. SHARD=512 va 8192 uchun takrorlang; 3x2 jadval tuzing va ikki o'qli grafik (so'rov/s va p99) chizing; SLO p99 < 8 ms ni qondiruvchi eng katta bo'lakni tanlang.
4. 4 ishchini 2 yadroda ishga tushiring — E(4) nega 50% atrofida yoki past bo'lishini izohlang.
5. pool.imap_unordered bilan res.sort'siz yig'ing va aniqlik qanday buzilishini ko'rsating; so'ng idx bo'yicha tiklang.
6. s6 Amdal formulasi bilan o'lchangan S(2) dan p ni baholang va 8 yadroli server uchun S(8) ni bashorat qiling; 40 daqiqa qancha bo'lishini hisoblang.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

6-variant. Ta'lim: o'quvchi natijalarini bashorat qiluvchi klaster — Amdal va Gustafson oson

Soha: Ta'lim

Respublika ta'lim portali 2 mln o'quvchi yozuvi bo'yicha o'zlashtirish xavfini (3 daraja) bashorat qiluvchi modelni o'qitadi va IT bo'limi 4, 16 yoki 64 GPU sotib olish masalasini hal qilmoqda. Sizdan CPU'da 2 jarayonli o'lchov o'tkazib, ketma-ket ulushni baholash va Amdal hamda Gustafson qonunlari asosida har uchala klaster hajmi uchun kutilayotgan tezlashishni hisoblab berish talab qilinadi.

Ma'lumot	sintetik o'quvchi yozuvlari: 12000 x 24 belgi (davomat, baholar, ijtimoiy ko'rsatkichlar), 3 sinf
Model	MLP 24-256-256-3
p (nazariy)	0.80, 0.92, 0.98
N	1, 2, 4, 8, 16, 32, 64, 128, 512
Global paket (kuchli)	192
STEPS	60

QADAMLAR

1. s1 DDP kodini o'quvchi ma'lumotiga moslang (EPOCHS=2, BS=96) va 2 jarayonda to'g'ri ishlashini (parametr farqi 0, jami 24000 namuna) ko'rsating.
2. s2 kodi bilan HID=256, GB=192, STEPS=60 uchun kuchli va kuchsiz rejimda S(2), E(2) ni o'lchang.
3. Kuchli rejim S(2) dan Amdal bo'yicha ketma-ket ulushni toping: 1-p = (2/S(2) - 1); p qiymatini yozing.
4. s6 kodini p=0.80, 0.92, 0.98 va o'lchangan p uchun N=1..512 oralig'ida ishga tushiring; S(N), E(N) jadvali va log-log grafik (o'lchangan nuqta bilan).
5. Gustafson formulasi bilan kuchsiz rejim uchun S(4), S(16), S(64) ni hisoblang; ikki qonun farqini bitta grafikda ko'rsating.
6. Xulosa: 4, 16, 64 GPU uchun kutilayotgan E(N) jadvali; Amdal bo'yicha 64 GPU sotib olish o'zini oqlaydimi, kuchsiz rejimda-chi.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

7-variant. Energetika: elektr yuklama prognozi — aloqa/hisob kalkulyatori (V100 klasteri) oson

Soha: Energetika

Energiya kompaniyasi 3 ta vaqt qatori modelini (kichik LSTM, TCN, Transformer) 8 ta V100 GPU'li klasterda o'qitmoqchi; tugunlar orasidagi tarmoq 1, 10 yoki 100 Gb/s bo'lishi mumkin. Sizdan s7 analitik modeli bilan har model va tarmoq uchun T_comp, T_comm, r va E(8) ni hisoblash, gradient to'plash ta'sirini ko'rsatish va byudjet doirasida tarmoq tanlash talab qilinadi.

Modellar	LSTM-kichik 2.3 mln / 0.5 GFLOP; TCN 9 mln / 1.2 GFLOP; Transformer 110 mln / 22 GFLOP (bir namuna forward)
GPU	V100 FP16 cho'qqi 125 TFLOPS, MFU 0.35
Tarmoqlar	1, 10, 100 Gb/s
Paket/GPU	64
Gradient	BF16 (2 bayt)
Overlap	0.5
Accum	1, 2, 4, 8

QADAMLAR

1. s7 kodidagi MODELS, NETS, PEAK, MFU, BATCH ni variant qiymatlariga almashtiring va N=8 uchun 3x3 jadvalni (T_comm, r, E) chiqaring.
2. Transformer uchun accum=1, 2, 4, 8 da 10 Gb/s tarmoqda E(8) ni hisoblang va to'plash qaysi qiymatdan boshlab E > 80% berishini toping.
3. Har tarmoq uchun N=1..64 da E(N) egri chiziqlarini TCN modeli uchun chizing (log2 o'q).
4. BYTES_PER_GRAD=4 (FP32) va OVERLAP=0 holatlarini qo'shib, eng yomon va eng yaxshi holat jadvalini tuzing.
5. s3 ring simulyatsiyasini N=8 da tekshiring va LSTM-kichik gradient hajmi uchun 1 Gb/s da haqiqiy all-reduce vaqtini formula bilan solishtiring.
6. Xulosa: 10 Gb/s tarmoq qaysi modellar uchun yetarli, 100 Gb/s qachon zarur, kichik LSTM uchun 8 GPU umuman kerakmi.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

8-variant. Transport: yo'l tirbandligi bashorati — sekin ishchi (straggler) ta'siri

o'rta

Soha: Transport

Shahar transport markazi 300 ta yo'l sensoridan kelayotgan oqim bo'yicha 15 daqiqalik tirbandlik darajasini (5 sinf) bashorat qiluvchi modelni 2 GPU'da o'qitadi, lekin GPU'lardan biri boshqa xizmat bilan birga ishlaydi va vaqti-vaqti bilan sekinlashadi. Sizdan sinxron SGD'da bitta sekin ishchi butun o'qitishni qanday tutib qolishini o'lchash va kechikish darajasiga qarab E(2) jadvalini tuzish, so'ng yumshatish usullarini taqqoslash talab qilinadi.

Ma'lumot	sintetik sensor oqimi 10000 x 300, 5 sinf (soat va hafta kuni bo'yicha naqsh)
Model	MLP 300-384-384-5
Sun'iy kechikish (rank 1)	0, 1, 2, 4 ms har qadamda
BS (har rank)	64
EPOCHS	2
Qadam vaqti (baza)	o'lchanadi
Tarmoq (kalkulyator)	10 Gb/s

QADAMLAR

1. s1 DDP kodini transport ma'lumotiga moslang va kechikishsiz baza vaqtini hamda bitta qadam o'rtacha vaqtini (ms) o'lchang.
2. rank 1 ga har qadamda time.sleep(d) qo'shing (d = 1, 2, 4 ms); har d uchun ikkala rank vaqtini, namuna/s va E(2) = T_baza/T_d ni jadvalga yozing. Rank 0 nega ham sekinlashadi — all-reduce to'siq (barrier) tushunchasi bilan izohlang.
3. Kechikishning qadam vaqtiga nisbati (d / T_qadam) bo'yicha E(2) grafigini chizing va chiziqli model E = 1/(1 + d/T_qadam) bilan solishtiring.
4. s2 kodi bilan kechikishsiz kuchli/kuchsiz S(2) ni o'lchang va straggler ta'sirini shu jadvalga qo'shimcha ustun qiling.
5. s3 formulasi bilan model gradienti hajmini va 10 Gb/s da aloqa vaqtini hisoblang; aloqa vaqti bilan 4 ms kechikishni taqqoslang.
6. s6 Amdal modeliga straggler'ni ketma-ket ulush sifatida qo'shing va N=8 da kutilayotgan E(8) ni baholang.
7. Xulosa: asinxron SGD, zaxira ishchilar (backup workers) va elastik o'qitishning afzallik/kamchiliklarini jadvalda taqqoslang va transport markazi uchun tanlov bering.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot

sifati (grafik, tuzilma, til) — 20

9-variant. Logistika: ombor buyurtmalarini marshrutlash inferensi — bo'lak hajmi va p99 o'rta

Soha: Logistika

Katta ombor har soatda 1.2 mln buyurtma qatorini 16 ta yig'ish zonasiga taqsimlovchi modelni ishlatadi; onlayn rejimda har bo'lak uchun p99 kechikish 4 ms dan oshmasligi kerak, kechki qayta ishlashda esa maksimal o'tkazuvchanlik muhim. Sizdan bo'lak hajmi bo'yicha o'tkazuvchanlik va p50/p99 profilini 1 va 2 ishchida o'lchash, ikki rejim uchun turli optimal bo'lakni asoslash talab qilinadi.

Ma'lumot	make_classification(n=6000, 36 belgi, 16 sinf, n_informative=24, random_state=11), 200 marta takror (1.2 mln so'rov)
Model	MLP 36-160-16
SHARD	64, 256, 1024, 4096, 16384
Ishchilar	1, 2
SLO onlayn	p99 bo'lak < 4 ms
Maxsus	imap chunksize ta'siri

QADAMLAR

1. s5 kodini moslang: modelni o'qiting, 1.2 mln so'rov tayyorlang; SHARD ro'yxati bo'yicha 1 va 2 ishchida so'rov/s, S(2), E(2), p50, p99 ni 5x2 jadvalga yozing.
2. Ikki o'qli grafik: x — SHARD (log), chap o'q — so'rov/s, o'ng o'q — p99 ms; 1 va 2 ishchi egri chiziqlari.
3. Onlayn rejim (p99 < 4 ms) va kechki rejim (maks. so'rov/s) uchun optimal SHARD ni tanlang va asoslang.
4. pool.map o'rniga pool.imap(..., chunksize=c) bilan c=1, 4, 16 uchun kichik SHARD=64 da IPC xarajati qanday kamayishini o'lchang.
5. 4 ishchi 2 yadroda: E(4) ni o'lchang va kontekst almashinuvi ta'sirini izohlang.
6. s6 Amdal bilan eng yaxshi S(2) dan p ni baholang va 8 yadroli server uchun soatlik 1.2 mln buyurtma necha daqiqada bajarilishini bashorat qiling.
7. Xulosa: ikki rejim uchun konfiguratsiya jadvali (SHARD, ishchilar, kutilgan p99, so'rov/s) va dinamik paketlash haqida tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

10-variant. Sanoat: dastgoh nosozligini bashorat — masshtablash egri chizig'i (paket va model o'lchami) o'rta

Soha: Sanoat

Mashinasozlik zavodi 120 ta vibratsiya belgisidan dastgoh nosozligini 7 sinfga ajratuvchi modelni o'qitadi va 2 GPU'li stansiyada kuchli masshtablash qachon to'yinishini bilmoqchi. Sizdan global paket va model kengligi bo'yicha E(2) egri chiziqlarini ikki rejimda o'lchash, to'yinish nuqtasini aniqlash va bucket_cap_mb ta'sirini tekshirish talab qilinadi.

Ma'lumot	sintetik vibratsiya spektrlari 8000 x 120, 7 sinf
Model	MLP 120-HID-HID-7, HID = 256 va 1024
Global paket	32, 64, 128, 256, 512
STEPS	40
DDP sozlamasi	bucket_cap_mb = 1 va 25 (standart)
Tarmoq (kalkulyator)	100 Gb/s

QADAMLAR

1. s2 kodini tsiklga o'rab, HID=256 uchun GB ro'yxati bo'yicha T(1), kuchli va kuchsiz DDP vaqtini o'lchang; E(2) ni ikki rejimda hisoblang (jami vaqt 90 s dan oshsa STEPS ni 25 ga tushiring).
2. HID=1024 uchun takrorlang (kerak bo'lsa GB ro'yxatini 64..512 qiling).
3. Bitta grafikda 4 egri chiziq: (HID, rejim) bo'yicha E(2) vs GB; kuchli rejim qaysi GB dan boshlab to'yinishini (E > 85%) har HID uchun aniqlang.
4. DDP(model, bucket_cap_mb=1) bilan HID=1024, GB=256 ni qayta o'lchang va standart bilan solishtiring; chelak soni ko'payganda nima o'zgaradi.
5. s1 kodini vibratsiya ma'lumotiga moslang (EPOCHS=2) va parametr sinxronligi hamda aniqlikni ko'rsating.
6. s3 formulasi bilan HID=1024 model gradienti uchun 100 Gb/s da all-reduce vaqtini va s7 modeli bilan r nisbatini baholang; o'lchangan E(2) bilan mos keladimi.
7. Xulosa: zavod stansiyasi uchun GB va rejim tavsiyasi, to'yinish nuqtasi va chiziqli LR qoidasi haqida eslatma.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

11-variant. Video-kuzatuv: kadr embedding tasnifi — parametr-server va halqa taqqoslovi o'rta

Soha: Xavfsizlik

Xavfsizlik kompaniyasi 2048 o'lchamli kadr embedding'laridan 9 ta hodisa turini aniqlovchi modelni ko'p GPU'li klasterda o'qitadi; arxitektura guruhi parametr-server (PS) va halqa all-reduce sxemalari orasida tanlov qilmoqda, tugunlar orasida 10 Gb/s tarmoq bor. Sizdan NumPy'da ikkala sxemani yozish, eng band tugun trafigini N bo'yicha solishtirish va PS qaysi N dan boshlab 100 ms chegarasidan oshishini topish talab qilinadi.

Ma'lumot	sintetik embedding 6000 x 2048 (9 ta markaz + shovqin), 9 sinf
Model (gradient hajmi)	embedding tasniflagich 2048-512-9 (taxminan 1.05 mln) va ResNet-50 23 mln
N	2, 4, 8, 16, 32, 64
Tarmoq	10 Gb/s
Chegara	100 ms / all-reduce
Simulyatsiya	L = 4096, N = 2, 4, 8

QADAMLAR

1. s3 ring_allreduce ni L=4096, N=2,4,8 da tekshiring; so'ng parametr-server sxemasini yozing (hamma 0-ishchiga yuboradi, u yig'ib hammaga qaytaradi) va natija bir xil ekanini ko'rsating.
2. Har ikki sxema uchun eng band tugun yuboradigan + qabul qiladigan baytlarni N ro'yxati bo'yicha jadvalga yozing (S = model hajmi FP32, ikkala model uchun).
3. 10 Gb/s da vaqtni hisoblang; PS sxemasi ResNet-50 uchun qaysi N dan 100 ms dan oshadi; halqa uchun chegara 2S/BW ekanini grafikda ko'rsating (log-log).
4. s1 DDP kodini embedding ma'lumotiga moslang (BS=64, EPOCHS=2) va parametr sinxronligini tekshiring; DDP ichida qaysi sxema ishlatilishini izohlang.
5. s2 bilan HID=512 uchun S(2), E(2) ni o'lchang.
6. s7 kalkulyatoriga PS sxemasini qo'shing (T_comm_PS = 2(N-1)*S/BW server uchun) va ResNet-50, N=8 da E(8) ni ikki sxemada solishtiring.
7. Xulosa: qaysi N gacha PS maqbul, halqa all-reduce'ning kamchiligi (kechikish, straggler) va kompaniya uchun tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

12-variant. Media: yangiliklar matnini tasniflash — no_sync bilan gradient to'plash DDP'da o'rta

Soha: Media

Yangiliklar agentligi 3000 o'lchamli so'z-sumka vektorlaridan 8 ta rubrikani aniqlovchi modelni 2 GPU'da o'qitadi; tarmoq sekin bo'lgani uchun har qadamda all-reduce qilish qimmat. Sizdan haqiqiy DDP kodida model.no_sync() konteksti yordamida K mikro-paketdan keyin bitta all-reduce bajarish, K bo'yicha vaqt va yakuniy yo'qotishni o'lchash hamda all-reduce soni K baravar kamayganini ko'rsatish talab qilinadi.

Ma'lumot	sintetik so'z-sumka: 6000 x 3000 (siyrak, 8 rubrika markazi), float32
Model	MLP 3000-256-8 (taxminan 770 ming parametr)
K (to'plash)	1, 2, 4, 8
Mikro-paket (har rank)	32
EPOCHS	2
Tarmoq (kalkulyator)	1 Gb/s
Tekshiruv	all-reduce soni = qadamlar / K

QADAMLAR

1. s1 DDP kodini so'z-sumka ma'lumotiga moslang (BS=32, EPOCHS=2) va K=1 baza vaqtini, namuna/s va yakuniy yo'qotishni o'lchang.
2. O'qitish siklini qayta yozing: K-1 mikro-paket model.no_sync() ichida backward, K-chisi oddiy backward + opt.step(); yo'qotishni 1/K ga ko'paytiring. K=2, 4, 8 uchun vaqt, namuna/s, yakuniy yo'qotish jadvali.
3. All-reduce chaqiruvlari sonini hisoblang (DDP hook yoki hisoblagich bilan) va K baravar kamayganini ko'rsating; chiziqli LR qoidasi global paket K*32*2 uchun qanday bo'lishi kerakligini yozing.
4. s4 kabi K=4 to'plash + 2 ishchi kombinatsiyasining bitta 256 namunali paket bilan gradient ekvivalentligini tekshiring (farq < 1e-6).
5. s3 formulasi bilan 770 ming parametr gradienti uchun 1 Gb/s da bitta all-reduce vaqtini hisoblang va K bo'yicha qadamga to'g'ri keladigan aloqa vaqtini jadvalga qo'shing.
6. s6 Amdal modelida aloqa ulushini K ga bo'lib, N=8 uchun kutilgan E(8) ni K=1 va K=8 da baholang.
7. Xulosa: K ning ortishi qaysi nuqtadan boshlab yaqinlashuvga zarar bera boshlaydi (yo'qotish jadvali), agentlik uchun tavsiya etilgan K.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot

sifati (grafik, tuzilma, til) — 20

13-variant. Aqlli shahar: kamera oqimlari uchun klaster kalkulyatori (1/10/100 Gb/s) o'rta

Soha: Shahar

Shahar boshqaruv markazi 400 ta kameradan kelayotgan kadrlarni tahlil qiluvchi uchta modelni (kichik YOLO, BERT-base matn uchun, ViT-L) 16 GPU'li A100 klasterida o'qitishni rejalashtiradi; tarmoq uchun 1, 10 va 100 Gb/s variantlari va overlap darajasi noma'lum. Sizdan s7 kalkulyatorini kengaytirib, N=4, 8, 16 da E(N) ni hisoblash, overlap va gradient formatining ta'sirini ko'rsatish va har model uchun minimal yetarli tarmoqni tanlash talab qilinadi.

Modellar	YOLO-kichik 7 mln / 8 GFLOP; BERT-base 110 mln / 11 GFLOP (seq 128); ViT-L 304 mln / 61 GFLOP
GPU	A100 BF16 312 TFLOPS, MFU 0.4
Tarmoqlar	1, 10, 100 Gb/s
N	4, 8, 16
Overlap	0.3 va 0.7
Gradient	FP32, BF16
Paket/GPU	32

QADAMLAR

1. s7 kodidagi MODELS, NETS, BATCH, OVERLAP ni variant qiymatlariga almashtiring; N=8, overlap=0.5 uchun 3x3 jadval (T_comm, r, E).
2. Har model uchun N=4, 8, 16 x 3 tarmoq E(N) jadvalini tuzing va E > 80% shartini qondiruvchi minimal tarmoqni belgilang.
3. Overlap 0.3 va 0.7 uchun ViT-L, 10 Gb/s da E(16) ni solishtiring; DDP chelaklari overlap'ni qanday ta'minlashini izohlang.
4. BYTES_PER_GRAD=4 va 2 uchun BERT-base natijasini solishtiring; gradient siqish 1 Gb/s da yetarli bo'ladimi.
5. Accum=1,2,4,8 uchun YOLO-kichik, 1 Gb/s da E(16) ni hisoblang va grafik chizing.
6. s3 ring simulyatsiyasini N=16 da tekshiring va YOLO-kichik hajmi uchun har GPU yuboradigan baytlar 2S ga yaqinlashishini ko'rsating.
7. Xulosa: har model uchun (tarmoq, accum, gradient formati) tavsiyasi va umumiy klaster uchun bitta tarmoq tanlansa qaysi biri.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

14-variant. Ekologiya: sun'iy yo'ldosh tasvirlarini tasniflash — gradient siqish (FP32/BF16/FP8) o'rta

Soha: Ekologiya

Ekologik monitoring markazi 512 o'lchamli sun'iy yo'ldosh tasvir belgilaridan 6 ta yer qoplami turini aniqlovchi modelni 4 ta viloyat serverida (tugunlar orasida 1 Gb/s) o'qitmoqchi; aloqa to'siq bo'lgani uchun gradientlarni BF16 yoki FP8 ga siqish ko'rib chiqilmoqda. Sizdan NumPy ring all-reduce'da past aniqlikli formatlarning hajm va xatolik ta'sirini o'lchash, aloqa vaqtini hisoblash va DDP'da xatolik to'planishini baholash talab qilinadi.

Ma'lumot	sintetik tasvir belgilari 6000 x 512, 6 sinf
Model	MLP 512-384-6 (taxminan 199 ming parametr)
Gradient formati	FP32, BF16 (torch.bfloat16 orqali), FP8-taqlid (int8 kvantlash, masshtab bilan)
Tarmoq	1 Gb/s
N	2, 4, 8, 16
Xatolik o'lchovi	nisbiy L2 xato, kosinus o'xshashlik

QADAMLAR

1. s3 ring_allreduce ga 'dtype' parametri qo'shing: yuborishdan oldin bo'lakni BF16 (torch orqali) yoki int8 (max-abs masshtab) ga aylantirib, qabulda qaytaring; N=4, L=8192 da uch format uchun yuborilgan baytlar va natijaning aniq yig'indiga nisbiy xatosini jadvalga yozing.
2. Formula bilan 199 ming parametr modeli va ResNet-50 (25.6 mln) uchun 1 Gb/s da N=2,4,8,16 all-reduce vaqtini uch formatda hisoblang.
3. s4 gradient ekvivalentligi kodida 2 ishchi gradientini BF16 va int8 ga siqib o'rtachalang; max farq va kosinus o'xshashlikni FP32 bilan solishtiring.
4. s1 DDP kodini tasvir belgilariga moslang va DDP'ga BF16 siqish hook'i (register_comm_hook, default_hooks.bf16_compress_hook) qo'shing; vaqt va yakuniy aniqlikni hook'siz holat bilan solishtiring.
5. s7 kalkulyatorida BYTES_PER_GRAD = 4, 2, 1 uchun ResNet-50, 1 Gb/s, N=4 da E(4) ni hisoblang.
6. Xulosa: qaysi format aniqlik yo'qotmasdan aloqa vaqtini yetarlicha kamaytiradi, xatolik-fikr (error feedback) nima uchun kerak, viloyat klasteri uchun tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

15-variant. Sport: harakat sensorlari bo'yicha mashq turini aniqlash — DDP'ni qo'lda yozish o'rta

Soha: Sport

Sport texnologiyalari startapi bilak sensorlaridan (6 o'q x 50 nuqta = 300 belgi) 10 ta mashq turini aniqlovchi modelni o'qitadi va jamoa DDP o'rami ichida nima bo'layotganini tushunishi kerak. Sizdan DDP'siz 2 jarayonli o'qitishni qo'lda yozish (broadcast + har gradient uchun all_reduce), natijasi DDP bilan bir xil ekanini isbotlash va vaqt farqini (chelak/overlap afzalligi) o'lchash talab qilinadi.

Ma'lumot	sintetik IMU signallari 5000 x 300 (10 mashq uchun turli chastotali sinuslar), 10 sinf
Model	MLP 300-256-128-10
BS (har rank)	64
EPOCHS	2
Taqqoslash	DDP o'rami vs qo'lda broadcast + all_reduce(p.grad)/world
Tekshiruv	parametr farqi < 1e-6, bir xil urug' va sampler

QADAMLAR

1. s1 DDP kodini IMU ma'lumotiga moslang va DDP bilan yakuniy parametrlarni (flat vektor) faylga saqlang; vaqtni o'lchang.
2. Qo'lda variant yozing: DDP o'ramisiz, boshida har parametr uchun dist.broadcast(p.data, src=0); har backward'dan keyin har p.grad uchun dist.all_reduce(SUM) va /world; sampler va urug' bir xil bo'lsin.
3. Ikkala variant yakuniy parametrlarini solishtiring (max farq < 1e-6) va vaqtni jadvalga yozing; DDP tezroq bo'lsa — chelaklar va overlap tufayli ekanini izohlang.
4. Qo'lda variantda gradientlarni bitta flat vektorga yig'ib (torch.cat) bitta all_reduce qiling; vaqt qanday o'zgaradi — all-reduce soni va kechikish (latency) rolini tushuntiring.
5. s3 formulasi bilan model gradienti hajmini va 10 Gb/s da all-reduce vaqtini hisoblang; parametrlar soni bo'yicha chaqiruvlar soni (qatlam soni) bilan solishtiring.
6. s2 kodi bilan S(2), E(2) ni DDP va qo'lda (flat) variant uchun o'lchang va jadvalga kiriting.
7. Xulosa: DDP nimani avtomatlashtiradi, qo'lda yozilgan variant qachon foydali (maxsus siqish, gradient klipping), startap uchun tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (DDP sinxron, tekshiruvlar o'tadi) — 30, O'lchovlar va jadval (to'liq, takrorlanuvchan) — 25, Tahlil va xulosa (formulalar bilan asoslangan) — 25, Hisobot sifati (grafik, tuzilma, til) — 20

16-variant. Bank: to'lov skoring modeli — eng arzon klaster konfiguratsiyasi (GPU-soat) qiyin

Soha: Moliya (bank)

Bankning ML platformasi 180 mln parametrli jadval-transformer skoring modelini 40 mln yozuvda 5 epoxa o'qitadi; byudjet 32 GPU'gacha, tarmoq variantlari 10, 25, 100 Gb/s, GPU-soat narxi tarmoqqa qarab farq qiladi. Sizdan s7 kalkulyatorini kengaytirib, (N, tarmoq, accum) kombinatsiyalari uchun qadam vaqti, E(N), epoxa davomiyligi va umumiy narxni hisoblash, eng arzon va eng tez variantlarni topish hamda natijani issiqlik xaritasida ko'rsatish talab qilinadi.

Model	TabTransformer 180 mln parametr, 6 GFLOP/namuna forward
Ma'lumot	40 mln yozuv, 5 epoxa
GPU	A100, 312 TFLOPS BF16, MFU 0.38; narx 2.1 $/GPU-soat
Tarmoq va qo'shimcha narx	10 Gb/s +0 %, 25 Gb/s +8 %, 100 Gb/s +20 %
N	1, 2, 4, 8, 16, 32
Accum	1, 2, 4, 8, 16
Paket/GPU	256
Cheklov	devor vaqti <= 12 soat

QADAMLAR

1. s7 estimate funksiyasini kengaytiring: qadam vaqti, 1 epoxa vaqti = (yozuvlar / (N*paket*accum)) * T_step, umumiy devor vaqti (5 epoxa), GPU-soat va narx (tarmoq ustamasi bilan).
2. N x tarmoq x accum (6 x 3 x 5 = 90 kombinatsiya) uchun jadval tuzing; devor vaqti 12 soatdan oshgan variantlarni chiqarib tashlang.
3. Eng arzon va eng tez kombinatsiyani toping; narx va vaqt bo'yicha Pareto chegarasini nuqtali grafikda chizing.
4. 10 Gb/s tarmoq uchun N x accum issiqlik xaritasi (rang — narx, $) va E(N) uchun ikkinchi xarita.
5. s3 formulasi bilan 180 mln parametr gradienti (BF16) uchun har tarmoqda all-reduce vaqtini hisoblang va kalkulyator T_comm bilan mos ekanini tekshiring; s6 Amdal bilan N=32 da nazariy chegarani solishtiring.
6. Sezgirlik tahlili: MFU 0.3 va 0.5, overlap 0.3 va 0.8 uchun eng arzon variant o'zgaradimi — jadval.
7. Xulosa (bank rahbariyati uchun 1 sahifa): tavsiya etilgan N, tarmoq, accum; nega katta accum bepul emas (yaqinlashuv, global paket); cheklovlar.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya asosliligi — 20

17-variant. Genomika: variant patogenligi tasnifi — DDP va gradient to'plash kombinatsiyasi qiyin

Soha: Genomika (biotibbiyot)

Genomika laboratoriyasi 1024 o'lchamli ketma-ketlik belgilari bo'yicha mutatsiya patogenligini 4 sinfga ajratadi; 2 GPU'da xotira faqat 32 namunali mikro-paketga yetadi, lekin tajribalar 512 global paket talab qiladi. Sizdan haqiqiy DDP + no_sync bilan 2 ishchi x K mikro-paket sxemasini yozish, uning bitta 512 paket bilan gradient ekvivalentligini isbotlash, all-reduce sonini, vaqtni va yaqinlashuvni o'lchash hamda BatchNorm/SyncBatchNorm masalasini tahlil qilish talab qilinadi.

Ma'lumot	sintetik genomik belgilar 8192 x 1024 (4 sinf, siyrak motivlar), float32
Model	MLP 1024-512-128-4 (BatchNorm'li va BatchNorm'siz)
Mikro-paket (har rank)	32
Global paket	512 (2 ishchi x K=8)
K	1, 2, 4, 8
EPOCHS	2
LR qoidasi	0.02 * (global paket / 64)
Tekshiruv	max|g_512 - g_ddp_accum| < 1e-5

QADAMLAR

1. s1 DDP kodini genomik ma'lumotga moslang; BS=32 va K=1 baza vaqtini o'lchang.
2. DDP + no_sync bilan K=2, 4, 8 to'plashni yozing (oxirgi mikro-paketda sinxron); har K uchun vaqt, all-reduce soni, yakuniy yo'qotish va aniqlik jadvali; LR ni qoidaga ko'ra masshtablang.
3. Ekvivalentlik isboti: bir xil 512 namuna uchun (a) bitta jarayon 1x512 gradienti, (b) 2 ishchi x 8 x 32 DDP+accum gradienti (opt.step oldidan p.grad ni yig'ib oling) — max farq < 1e-5 ekanini ko'rsating.
4. Modelga BatchNorm1d qo'shing: ekvivalentlik buzilishini o'lchang; nn.SyncBatchNorm.convert_sync_batchnorm qo'llang va farq qanday o'zgarishini (to'plash ichida baribir qoladi) izohlang.
5. s3 formulasi bilan model gradienti hajmi (parametrlar sonini hisoblang) uchun 10 Gb/s da all-reduce vaqtini hisoblang; K bo'yicha epoxa uchun jami aloqa vaqtini jadvalga qo'shing.
6. s6 Amdal va s7 kalkulyatori bilan N=8 GPU, 10 Gb/s da K=1 va K=8 uchun E(8) ni baholang; o'lchangan 2 jarayon natijasi bilan taqqoslang.
7. Xulosa: 2 GPU xotira cheklovida 512 global paketga erishish rejasi, yaqinlashuv uchun LR/warmup eslatmasi, BatchNorm o'rniga LayerNorm tavsiyasi.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya asosliligi — 20

18-variant. 5G tarmoq: anomaliya detektori — straggler va asinxron SGD simulyatsiyasi qiyin

Soha: Telekom (5G infratuzilma)

5G operatori 64 ta bazaviy stansiya serverida tarmoq anomaliyalarini aniqlovchi modelni taqsimlangan tarzda o'qitadi; serverlar bir xil emas, ba'zilari 2-3 marta sekin. Sizdan sinxron SGD, zaxira ishchili sinxron SGD (eng sekin k tasini kutmaslik) va asinxron (eskirgan gradientli) SGD'ni NumPy'da simulyatsiya qilish, bir xil hisoblash byudjetida yaqinlashuv va devor vaqtini solishtirish hamda 2 jarayonli haqiqiy DDP'da straggler o'lchovini olish talab qilinadi.

Ma'lumot	sintetik KPI oqimi 20000 x 48 (2 sinf: normal/anomaliya, 5 % anomaliya)
Model (simulyatsiya)	logistik regressiya 48 og'irlik, NumPy'da qo'lda gradient
Ishchilar (simulyatsiya)	N = 16, qadam vaqti log-normal (sigma = 0.3 va 0.8)
Sxemalar	sinxron; sinxron + 2 zaxira (eng sekin 2 tasi kutilmaydi); asinxron (eskirish tau <= 4)
Haqiqiy DDP	2 jarayon, rank 1 kechikishi 0/2/5 ms
Byudjet	1600 gradient hisobi

QADAMLAR

1. KPI generatorini yozing va NumPy'da mini-paket gradientli logistik regressiyani amalga oshiring; 1 ishchi baza yaqinlashuv egri chizig'ini (yo'qotish vs gradient soni) oling.
2. Sinxron SGD simulyatsiyasi: har qadamda 16 ishchi vaqti log-normal taqsimotdan olinadi, qadam vaqti = maksimum; devor vaqti va yo'qotish egri chizig'i (sigma=0.3 va 0.8).
3. Zaxira ishchilar: har qadamda eng tez 14 ta gradient o'rtachalanadi; devor vaqti va yaqinlashuvni solishtiring.
4. Asinxron SGD: har ishchi o'z tezligida tugatganda eskirgan parametr bo'yicha hisoblangan gradient qo'llanadi (eskirish tau ni yozib boring); yo'qotish va o'rtacha tau jadvali; LR ni tau ga qarab kamaytirish ta'sirini sinang.
5. Haqiqiy o'lchov: s1 DDP kodini KPI ma'lumotiga moslang, rank 1 ga 0/2/5 ms sleep qo'shib E(2) jadvalini tuzing va simulyatsiya modeli E = T_qadam/(T_qadam + d) bilan solishtiring.
6. Uch sxema uchun 'bir xil devor vaqtida erishilgan yo'qotish' grafigini chizing (sigma=0.8 holatida) va s6 Amdal modeliga straggler ulushini qo'shib N=64 uchun E(64) ni baholang.
7. Xulosa: operator uchun qaysi sxema (yoki kombinatsiya) tavsiya etiladi, asinxron rejimning narxi (eskirgan gradient, yaqinlashuv kafolati), amaliy vositalar (torch elastic, Horovod) haqida izoh.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya

asosliligi — 20

19-variant. Aviatsiya: dvigatel telemetriyasi — ierarxik all-reduce va alfa-beta kechikish modeli qiyin

Soha: Aviatsiya

Aviakompaniya dvigatel telemetriyasidan qolgan resursni bashorat qiluvchi 95 mln parametrli modelni 4 tugun x 8 GPU klasterida o'qitadi: tugun ichida NVLink (600 GB/s), tugunlar orasida 10 yoki 100 Gb/s. Oddiy halqa 32 ta GPU'ni bitta halqaga tizadi va sekin bog'lamlar hammani tutib qoladi. Sizdan alfa-beta (kechikish + o'tkazuvchanlik) aloqa modelini yozish, yassi halqa va ierarxik (tugun ichida reduce-scatter/all-gather, tugunlar orasida halqa) sxemalarni NumPy'da simulyatsiya qilib vaqtini taqqoslash talab qilinadi.

Model	95 mln parametr (BF16 gradient = 190 MB), 14 GFLOP/namuna
Topologiya	4 tugun x 8 GPU; tugun ichida 600 GB/s, alfa 5 mks; tugunlar orasida 10 va 100 Gb/s, alfa 20 mks
Aloqa modeli	T = alfa * qadamlar + baytlar / BW (har bosqich uchun)
Sxemalar	yassi halqa (N=32); ierarxik 2 bosqich (8 ichida, 4 orasida)
Chelak hajmi	1, 25, 100 MB (qadamlar soniga ta'sir)
Simulyatsiya	L = 4096, 32 ishchi
GPU	A100, MFU 0.4, paket 64

QADAMLAR

1. s3 ring_allreduce asosida ierarxik all-reduce yozing: 1) har tugun ichida 8 ishchi reduce-scatter, 2) tugunlar orasida 4 ta bo'lak egasi halqa all-reduce, 3) tugun ichida all-gather; L=4096, 32 ishchi uchun natija to'g'riligini tekshiring va har ishchi yuborgan baytlarni bog'lam turi bo'yicha (ichki/tashqi) ajratib hisoblang.
2. Alfa-beta modelini amalga oshiring: har qadam vaqti = alfa + baytlar/BW; yassi halqa (31+31 qadam, eng sekin bog'lam tezligi bilan) va ierarxik sxema uchun 190 MB gradientda jami vaqtni 10 va 100 Gb/s uchun hisoblang.
3. Chelak hajmi 1, 25, 100 MB uchun all-reduce soni va alfa ulushini hisoblang; kichik chelakda kechikish (alfa) hukmron bo'lishini grafikda ko'rsating.
4. s7 kalkulyatoriga ierarxik T_comm ni qo'shing va N=32 uchun E(32) ni yassi va ierarxik sxemalarda, ikki tarmoqda solishtiring; overlap 0.5.
5. s1 DDP kodini telemetriya sintetik ma'lumotiga (6000 x 200, regressiya o'rniga 5 sinf) moslang va 2 jarayonda to'g'ri ishlashini ko'rsating; bucket_cap_mb=1 va 25 uchun vaqt farqini o'lchang (alfa ta'sirining CPU muqobili).
6. s6 Amdal bilan aloqa ulushidan N=32 va N=128 uchun chegarani baholang.
7. Xulosa: aviakompaniya klasteri uchun sxema, chelak hajmi va tarmoq tavsiyasi; NCCL ierarxik algoritmlarni qanday tanlashi haqida izoh.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun

tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya asosliligi — 20

20-variant. E-hukumat: murojaatlarni tasniflash — o'qitish va inferens uchun to'liq masshtablash hisoboti qiyin

Soha: Davlat xizmatlari

Davlat xizmatlari portali kuniga 2 mln fuqaro murojaatini 24 ta idoraga yo'naltiradi; modelni 2 GPU'li serverda o'qitish va 8 yadroli serverda inferens qilish rejalashtirilgan, byudjet qat'iy. Sizdan laboratoriyaning barcha bosqichlarini bitta ssenariyga birlashtirish: DDP o'lchovi, masshtablash jadvali, gradient to'plash, parallel inferens profili, Amdal bahosi va kalkulyator — va yakuniy texnik hisobot (o'qitish SLA 4 soat, inferens p99 < 6 ms) tayyorlash talab qilinadi.

Ma'lumot	sintetik murojaat vektorlari 12000 x 768 (24 idora markazi + shovqin), 24 sinf
Model	MLP 768-512-256-24 (taxminan 532 ming parametr)
O'qitish	2 jarayon DDP, BS 64/128, EPOCHS 3, K = 1 va 4
Inferens	2 mln so'rov, SHARD 256/1024/4096, 1 va 2 ishchi, SLO p99 < 6 ms
Tarmoq (kalkulyator)	1, 10, 100 Gb/s
Real klaster bahosi	N = 2..16, ResNet-50 o'rniga shu model + BERT-base 110 mln
Hisobot	Markdown, 2 jadval + 3 grafik

QADAMLAR

1. s1 DDP: murojaat ma'lumotida BS=64 va 128, EPOCHS=3 uchun namuna/s, yo'qotish, aniqlik, parametr sinxronligi jadvali.
2. s2 masshtablash: HID=512 (ikki yashirin qatlam), GB=256 uchun 4 konfiguratsiya, S(2), E(2); kuchli rejim S(2) dan Amdal p ni toping.
3. s4 + DDP no_sync: K=4 to'plash bilan vaqt va all-reduce soni; 1x256 bilan gradient ekvivalentligini tekshiring.
4. s5 inferens: 2 mln so'rovni SHARD=256, 1024, 4096 da 1 va 2 ishchida; so'rov/s, p50/p99 jadvali; SLO p99 < 6 ms ni qondiruvchi eng katta SHARD; 8 yadro uchun S(8) ni Amdal bilan bashorat qiling va kunlik 2 mln so'rov necha daqiqa olishini hisoblang.
5. s3 + s7: 532 ming va 110 mln parametr uchun 1/10/100 Gb/s da all-reduce vaqti, N=2..16 da E(N) egri chiziqlari (kalkulyator) va ring simulyatsiyasi tekshiruvi.
6. Barcha natijalarni Markdown hisobotga yig'ing: o'qitish SLA 4 soat (12000 yozuvdan 2 mln ga ekstrapolyatsiya qilib, epoxa vaqtini namuna/s dan hisoblang) bajariladimi, inferens SLO bajariladimi, qaysi konfiguratsiya tavsiya etiladi.
7. Xulosa: ikki alohida tavsiya (o'qitish va inferens), xavflar (straggler, tarmoq, xotira) va keyingi qadamlar (profiler bilan tekshirish).

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (konfiguratsiya, vaqt, namuna/s, S(N), E(N)); Grafik(lar) (matplotlib, PNG); Xulosa (>= 150 so'z): o'lchovlar talqini va klaster uchun tavsiya

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadval va grafiklar — 25, Chuqur tahlil (analitik model vs o'lchov, cheklovlar) — 30, Hisobot sifati va tavsiya asosliligi — 20

70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A6

Sun'iy intellekt tizimlarining xavfsizligi va ishonchliligini tahlil qilish

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A6
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A6
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

KTM laboratoriyalari — Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

Laboratoriya haqida

Bu virtual laboratoriyada talaba sklearn digits (8x8 rasm) ustida kichik PyTorch MLP'ga FGSM va PGD hujumlarini qo'lda yozadi, aniqlikning epsilon'ga bog'liqlik egri chizig'ini quradi, qisqa adversarial o'qitish bilan mustahkamlikni taqqoslaydi, Grad-CAM o'rniga gradient-saliency xaritasini chizadi, sintetik «kredit» ma'lumotida fairlearn MetricFrame va ThresholdOptimizer bilan adolatlilikni o'lchaydi va tuzatadi, shap TreeExplainer bilan daraxt modelini tushuntiradi hamda 20 bandli etik tekshiruv ro'yxatini avtomatik baholaydi. Real labdan farqi: CIFAR-10/ResNet-18 va Adult o'rniga kichik, tarmoqsiz yuklanadigan ma'lumotlar; torchattacks va grad-cam kutubxonalari yo'q — hujumlar va saliency qo'lda yoziladi (bu o'rganish uchun foydaliroq).

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A6`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A6 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Bazaviy model va FGSM: aniqlikning epsilon'ga bog'liqligi
2. 2-qadam. PGD hujumini qo'lda yozish va FGSM bilan taqqoslash
3. 3-qadam. Adversarial o'qitish (qisqa) va mustahkamlik murozasi
4. 4-qadam. Gradient-saliency xaritasi: toza va hujum ostidagi raqamlar
5. 5-qadam. Adolatlilik: MetricFrame va ThresholdOptimizer
6. 6-qadam. SHAP TreeExplainer: global muhimlik, lokal izoh, additivlik
7. 7-qadam. Etik tekshiruv ro'yxatini avtomatik baholash
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Bazaviy model va FGSM: aniqlikning epsilon'ga bog'liqligi

MAQSAD

Digits ma'lumoti [0,1] piksel fazosiga keltiriladi (1 kulrang daraja = 1/16), MLP 300 qadam Adam bilan o'qitiladi (test ~97%). FGSM qo'lda yoziladi: `x + eps*sign(grad)`, [0,1] ga qirqish. 7 ta epsilon uchun aniqlik jadvali va mustahkamlik egri chizig'i chiziladi; shu eps'dagi tasodifiy ±eps shovqin bilan taqqoslanadi. Talaba model chuqurligini, epsilon to'rini va o'qitish qadamlarini o'zgartirib ko'radi.

> **Kutilgan natija:** eps=0.05 da aniqlik ~0.77, eps=0.1 da ~0.3, eps=0.15 da ~0.1; tasodifiy shovqin eps=0.15 da ~0.93 qoldiradi. Grafik: keskin tushuvchi egri chiziq.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Bazaviy model (digits 8x8, MLP) va FGSM hujumi: aniqlikning epsilon ga bog'liqligi
# Piksellar [0,1] oraliqqa keltiriladi (data/16) — epsilon shu fazoda ma'noga ega (1 kulrang
daraja = 1/16 = 0.0625).
import torch, torch.nn as nn, torch.nn.functional as F, numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
torch.manual_seed(0); torch.set_num_threads(2)

d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16.0, d.target, test_size=0.3, random_state=0,
stratify=d.target)
Xtr, Xte = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xte, dtype=torch.float32)
ytr, yte = torch.tensor(ytr), torch.tensor(yte)

def train(X, y, steps=300):
    m = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 64), nn.ReLU(),
nn.Linear(64, 10))
    opt = torch.optim.Adam(m.parameters(), lr=3e-3)
    for _ in range(steps):
        opt.zero_grad(); F.cross_entropy(m(X), y).backward(); opt.step()
    return m.eval()

def fgsm(model, x, y, eps):
    """x_adv = x + eps * sign(grad_x L) — bitta gradient qadami, [0,1] ga qirqiladi."""
    x = x.clone().detach().requires_grad_(True)
    loss = F.cross_entropy(model(x), y)
    grad, = torch.autograd.grad(loss, x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

def acc(model, x, y):
    with torch.no_grad(): return (model(x).argmax(1) == y).float().mean().item()

model = train(Xtr, ytr)
print(f"Toza aniqlik: o'quv={acc(model, Xtr, ytr):.3f}  test={acc(model, Xte, yte):.3f}")
EPS = [0.0, 0.02, 0.05, 0.1, 0.15, 0.2, 0.3]
print("=== FGSM: aniqlik vs epsilon ===")
print(f"{'eps':>6s} {'eps*16 (daraja)':>16s} {'aniqlik':>8s} {'o\'rtacha L_inf':>14s}")
res = []
for e in EPS:
    xa = fgsm(model, Xte, yte, e) if e > 0 else Xte
```

```
    a = acc(model, xa, yte); res.append(a)
    print(f"{e:6.2f} {e*16:16.1f} {a:8.3f} {(xa - Xte).abs().amax(1).mean():14.3f}")
# Tasodifiy shovqin bilan taqqoslash: xuddi shu eps, lekin yo'nalish tasodifiy
e = 0.15; xr = (Xte + e * torch.sign(torch.randn_like(Xte))).clamp(0, 1)
print(f"\nTaqqoslash eps={e}: FGSM aniqlik={res[EPS.index(e)]:.3f}  tasodifiy +-eps shovqin=
{acc(model, xr, yte):.3f}")
print("Xulosa: gradient yo'nalishidagi kichik buzilish tasodifiy shovqindan ko'p marta
xavfliroq.")

plt.figure(figsize=(6, 4))
plt.plot(EPS, res, marker="o", label="FGSM")
plt.axhline(acc(model, xr, yte), ls="--", c="gray", label=f"tasodifiy shovqin (eps={e})")
plt.xlabel("epsilon (L_inf, [0,1] piksel fazosi)"); plt.ylabel("test aniqligi"); plt.ylim(0,
1.02)
plt.title("Mustahkamlik egri chizig'i: digits MLP, FGSM"); plt.grid(alpha=.3); plt.legend();
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== FGSM: aniqlik vs epsilon ===`, `Taqqoslash eps=`

TAJRIBA QILIB KO‘RING

- `train(..., steps=50)` — kam o'qitilgan model hujumga ko'proq yoki kamroq chidamlimi?
- Modelga `nn.Dropout(0.3)` qo'shing va eval rejimini unutib (`model.train()`), FGSM aniqligini solishtiring — «soxta mustahkamlik».
- Yo'naltirilgan (targeted) FGSM yozing: gradient ishorasini teskari olib, maqsad sinf yo'qotishini kamaytiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. PGD hujumini qo'lda yozish va FGSM bilan taqqoslash

MAQSAD

PGD: tasodifiy boshlang'ich nuqta, K qadam `alpha*sign(grad)`, har qadimdan keyin eps-sharga proyeksiya va [0,1] qirqish. FGSM, PGD-10 (tasodifiy boshlanish bilan/siz) va PGD-40 bir xil epsilon to'rida solishtiriladi, har hujum uchun L_inf chegarasi `assert` bilan tekshiriladi. Talaba alpha, qadamlar soni va boshlang'ich nuqtaning ta'sirini ko'radi.

> **Kutilgan natija:** PGD har eps'da FGSM dan past yoki teng aniqlik beradi (eps=0.15: FGSM ~0.11, PGD ~0.05; eps=0.2: PGD ~0.00). Assert o'tadi. 4 egri chiziqli grafik.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. PGD hujumini qo'lda amalga oshirish va FGSM bilan taqqoslash
import torch, torch.nn as nn, torch.nn.functional as F, numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
torch.manual_seed(0); torch.set_num_threads(2)

d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16.0, d.target, test_size=0.3, random_state=0,
stratify=d.target)
Xtr, Xte = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xte, dtype=torch.float32)
ytr, yte = torch.tensor(ytr), torch.tensor(yte)

def train(X, y, steps=300):
    m = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 64), nn.ReLU(),
nn.Linear(64, 10))
    opt = torch.optim.Adam(m.parameters(), lr=3e-3)
    for _ in range(steps):
        opt.zero_grad(); F.cross_entropy(m(X), y).backward(); opt.step()
    return m.eval()

def fgsm(model, x, y, eps):
    x = x.clone().detach().requires_grad_(True)
    grad, = torch.autograd.grad(F.cross_entropy(model(x), y), x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

def pgd(model, x, y, eps, alpha=None, steps=10, random_start=True):
    """Iterativ FGSM + proyeksiya: har qadamdan keyin x atrofidagi eps-sharga va [0,1] ga
qaytariladi."""
    alpha = alpha or 2.5 * eps / steps
    x0 = x.clone().detach()
    xa = (x0 + torch.empty_like(x0).uniform_(-eps, eps)) if random_start else x0.clone()
    xa = xa.clamp(0, 1)
    for _ in range(steps):
        xa.requires_grad_(True)
        grad, = torch.autograd.grad(F.cross_entropy(model(xa), y), xa)
        xa = xa.detach() + alpha * grad.sign()
        xa = torch.min(torch.max(xa, x0 - eps), x0 + eps).clamp(0, 1)   # proyeksiya
    return xa.detach()

def acc(model, x, y):
    with torch.no_grad(): return (model(x).argmax(1) == y).float().mean().item()
```

```
model = train(Xtr, ytr)
EPS = [0.02, 0.05, 0.1, 0.15, 0.2]
attacks = {"FGSM": lambda e: fgsm(model, Xte, yte, e),
           "PGD-10 (tasodifiy boshlanmasdan)": lambda e: pgd(model, Xte, yte, e, steps=10,
random_start=False),
           "PGD-10": lambda e: pgd(model, Xte, yte, e, steps=10),
           "PGD-40": lambda e: pgd(model, Xte, yte, e, steps=40)}
print(f"Toza test aniqligi: {acc(model, Xte, yte):.3f}")
print("=== Aniqlik: FGSM vs PGD (epsilon bo'yicha) ===")
print(f"{'hujum':34s} " + " ".join(f"eps={e:<5}" for e in EPS))
curves = {}
for name, fn in attacks.items():
    row = []
    for e in EPS:
        torch.manual_seed(1); xa = fn(e)
        assert (xa - Xte).abs().max() <= e + 1e-6, "L_inf chegarasi buzildi!"   # tahdid modeli
tekshiruvi
        row.append(acc(model, xa, yte))
    curves[name] = row
    print(f"{name:34s} " + " ".join(f"{a:9.3f}" for a in row))
print("L_inf chegarasi barcha hujumlarda saqlangan (assert o'tdi).")
print("Xulosa: PGD har doim FGSM dan kuchli yoki teng; qadamlar soni oshsa hujum kuchayadi,
lekin narx K baravar.")
print("Agar qo'lda yozilgan PGD FGSM dan kuchsiz chiqsa — xato bor (eval() yo'q, detach yo'q
yoki proyeksiya noto'g'ri).")

plt.figure(figsize=(6.5, 4))
for name, row in curves.items(): plt.plot(EPS, row, marker="o", label=name)
plt.xlabel("epsilon"); plt.ylabel("test aniqligi"); plt.ylim(0, 1.02); plt.grid(alpha=.3);
plt.legend(fontsize=8)
plt.title("FGSM vs PGD — toza o'qitilgan model"); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Aniqlik: FGSM vs PGD`, `L_inf chegarasi barcha hujumlarda saqlangan`

TAJRIBA QILIB KO‘RING

- `alpha = eps` (juda katta qadam) qilib PGD kuchsizlanishini kuzating.
- Proyeksiya satrini olib tashlang — assert nega yiqiladi?
- L_2 normali PGD yozing: gradientni normalashtirib, buzilishni eps radiusli sharga proyeksiya qiling.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Adversarial o'qitish (qisqa) va mustahkamlik murozasi

MAQSAD

Madry uslubi: har qadamda avval PGD-5 (eps=0.1) bilan adversarial o'quv paketi yasaladi, model aynan unda o'qitiladi. Toza va adversarial modellar FGSM/PGD-10 ostida 5 ta eps uchun taqqoslanadi (4 egri chiziq), o'qitish vaqti nisbati chiqadi. Talaba EPS_TRAIN, k va qadamlar sonini o'zgartirib, murozani o'rganadi.

> **Kutilgan natija:** Adversarial o'qitish ~6x qimmat; PGD-10 eps=0.1 ostida aniqlik ~0.27 dan ~0.83 ga ko'tariladi; eps=0.2 da himoya zaiflashadi (~0.2).

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Adversarial o'qitish (Madry uslubi, qisqa) va mustahkamlikni taqqoslash
import time, torch, torch.nn as nn, torch.nn.functional as F
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
torch.manual_seed(0); torch.set_num_threads(2)

d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16.0, d.target, test_size=0.3, random_state=0,
stratify=d.target)
Xtr, Xte = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xte, dtype=torch.float32)
ytr, yte = torch.tensor(ytr), torch.tensor(yte)
EPS_TRAIN = 0.1                     # o'qitishdagi tahdid modeli

def new_model():
    torch.manual_seed(0)
    return nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 64), nn.ReLU(),
nn.Linear(64, 10))

def pgd(model, x, y, eps, steps=10, random_start=True):
    alpha = 2.5 * eps / steps
    x0 = x.clone().detach()
    xa = (x0 + torch.empty_like(x0).uniform_(-eps, eps)).clamp(0, 1) if random_start else
x0.clone()
    for _ in range(steps):
        xa.requires_grad_(True)
        grad, = torch.autograd.grad(F.cross_entropy(model(xa), y), xa)
        xa = torch.min(torch.max(xa.detach() + alpha * grad.sign(), x0 - eps), x0 +
eps).clamp(0, 1)
    return xa.detach()

def fgsm(model, x, y, eps):
    x = x.clone().detach().requires_grad_(True)
    grad, = torch.autograd.grad(F.cross_entropy(model(x), y), x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

def train(adv=False, steps=300, k=5):
    """adv=True: har qadamda avval PGD-k bilan adversarial paket yasaladi, model aynan unda
o'qitiladi (min-max)."""
    m = new_model(); opt = torch.optim.Adam(m.parameters(), lr=3e-3)
    t0 = time.time()
    for _ in range(steps):
        x = pgd(m.eval(), Xtr, ytr, EPS_TRAIN, steps=k) if adv else Xtr
```

```
        m.train(); opt.zero_grad(); F.cross_entropy(m(x), ytr).backward(); opt.step()
    return m.eval(), time.time() - t0

def acc(model, x, y):
    with torch.no_grad(): return (model(x).argmax(1) == y).float().mean().item()

clean, t_clean = train(adv=False)
robust, t_adv = train(adv=True)
print(f"O'qitish vaqti: toza={t_clean:.1f}s  adversarial(PGD-5)={t_adv:.1f}s  (~{t_adv /
t_clean:.0f}x qimmat)")
EPS = [0.0, 0.05, 0.1, 0.15, 0.2]
print("=== Toza model vs adversarial o'qitilgan model ===")
print(f"{'eps':>5s} | {'toza: FGSM':>10s} {'toza: PGD-10':>12s} | {'adv: FGSM':>9s} {'adv: PGD-
10':>11s}")
curves = {k: [] for k in ("toza/FGSM", "toza/PGD", "adv/FGSM", "adv/PGD")}
for e in EPS:
    vals = []
    for name, m in (("toza", clean), ("adv", robust)):
        torch.manual_seed(1)
        af = acc(m, fgsm(m, Xte, yte, e) if e else Xte, yte); ap = acc(m, pgd(m, Xte, yte, e)
if e else Xte, yte)
        curves[f"{name}/FGSM"].append(af); curves[f"{name}/PGD"].append(ap); vals += [af, ap]
    print(f"{e:5.2f} | {vals[0]:10.3f} {vals[1]:12.3f} | {vals[2]:9.3f} {vals[3]:11.3f}")
print(f"\nMuroza: toza aniqlik {curves['toza/PGD'][0]:.3f} -> {curves['adv/PGD'][0]:.3f}; "
      f"PGD-10 (eps={EPS_TRAIN}) ostida {curves['toza/PGD'][2]:.3f} -> {curves['adv/PGD']
[2]:.3f}")
print("Kichik va oson to'plamda (digits) adversarial o'qitish regulyarizator vazifasini
bajarib, toza aniqlikni")
print("pasaytirmasligi ham mumkin; CIFAR-10/ResNet kabi real masalalarda toza aniqlik odatda 5-
15 bandga tushadi.")
print("Eslatma: himoya faqat o'qitilgan tahdid modeli (L_inf, eps<=0.1) doirasida; eps=0.2 da
yana pasayadi.")

plt.figure(figsize=(6.5, 4))
for (k, v), st in zip(curves.items(), ("o--", "o-", "s--", "s-")): plt.plot(EPS, v, st,
label=k)
plt.xlabel("epsilon"); plt.ylabel("test aniqligi"); plt.ylim(0, 1.02); plt.grid(alpha=.3);
plt.legend()
plt.title("Adversarial o'qitish ta'siri (digits MLP)"); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Toza model vs adversarial o'qitilgan model ===`, `Muroza:`

TAJRIBA QILIB KO‘RING

- EPS_TRAIN=0.05 va 0.2 bilan qayta o'qiting: himoya radiusi va toza aniqlik qanday o'zgaradi?
- k=1 (FGSM adversarial training) — «katastrofik overfitting» kuzatiladimi (FGSM'ga chidamli, PGD'ga zaif)?
- Toza va adversarial yo'qotishni 50/50 aralashtirib (TRADES-ga o'xshash) murozani yumshating.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Gradient-saliency xaritasi: toza va hujum ostidagi raqamlar

MAQSAD

MLP uchun Grad-CAM o'rniga gradient-saliency (|d logit/d x|, 8x8) hisoblanadi. 8 ta test raqami uchun toza rasm, uning saliency xaritasi, PGD (eps=0.15) bilan buzilgan rasm va uning xaritasi 4 qatorli panelda chiziladi; toza/adv xaritalar korrelyatsiyasi va entropiyasi jadvalda. Talaba tushuntirishning hujum ostida «ko'chishini» ko'radi.

> **Kutilgan natija:** Adversarial rasmlar ko'zga toza bilan deyarli bir xil, lekin bashorat noto'g'ri; saliency korrelyatsiyasi ~0 atrofida (xarita ko'chgan). Panel rasmi.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Gradient-saliency xaritasi (Grad-CAM ning MLP uchun muqobili): toza va hujum
ostidagi raqamlar
# Saliency = |d logit_y / d x| — har piksel bashoratga qanchalik ta'sir qilishini ko'rsatadi.
import torch, torch.nn as nn, torch.nn.functional as F, numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
torch.manual_seed(0); torch.set_num_threads(2)

d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16.0, d.target, test_size=0.3, random_state=0,
stratify=d.target)
Xtr, Xte = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xte, dtype=torch.float32)
ytr, yte = torch.tensor(ytr), torch.tensor(yte)

m = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 64), nn.ReLU(), nn.Linear(64,
10))
opt = torch.optim.Adam(m.parameters(), lr=3e-3)
for _ in range(300):
    opt.zero_grad(); F.cross_entropy(m(Xtr), ytr).backward(); opt.step()
m.eval()

def pgd(model, x, y, eps, steps=10):
    alpha = 2.5 * eps / steps; x0 = x.clone().detach()
    xa = (x0 + torch.empty_like(x0).uniform_(-eps, eps)).clamp(0, 1)
    for _ in range(steps):
        xa.requires_grad_(True)
        grad, = torch.autograd.grad(F.cross_entropy(model(xa), y), xa)
        xa = torch.min(torch.max(xa.detach() + alpha * grad.sign(), x0 - eps), x0 +
eps).clamp(0, 1)
    return xa.detach()

def saliency(model, x, cls):
    """Bashorat qilingan sinf logitining kirish bo'yicha gradienti moduli (8x8)."""
    x = x.clone().detach().requires_grad_(True)
    logit = model(x)[torch.arange(len(x)), cls].sum()
    grad, = torch.autograd.grad(logit, x)
    return grad.abs()

def entropy(s):
    p = s / s.sum(1, keepdim=True); return -(p * torch.log(p + 1e-12)).sum(1)
```

```
N = 8; x, y = Xte[:N], yte[:N]
xa = pgd(m, x, y, eps=0.15)
with torch.no_grad(): p_clean, p_adv = m(x).argmax(1), m(xa).argmax(1)
s_clean, s_adv = saliency(m, x, p_clean), saliency(m, xa, p_adv)
corr = torch.tensor([np.corrcoef(s_clean[i].numpy(), s_adv[i].numpy())[0, 1] for i in
range(N)])
print("=== Saliency xaritasi: toza vs adversarial (eps=0.15) ===")
print(f"{'#':>2s} {'haqiqiy':>7s} {'toza':>5s} {'adv':>4s} {'korr(toza,adv)':>15s} {'entropiya
toza':>15s} {'entropiya adv':>14s}")
for i in range(N):
    print(f"{i:2d} {y[i].item():7d} {p_clean[i].item():5d} {p_adv[i].item():4d} {corr[i]:15.3f}
{entropy(s_clean)[i]:15.3f} {entropy(s_adv)[i]:14.3f}")
print(f"O'rtacha korrelyatsiya={corr.mean():.3f}; o'rtacha entropiya: toza=
{entropy(s_clean).mean():.3f}, adv={entropy(s_adv).mean():.3f}")
print("Kuzatuv: hujumdan keyin xarita 'ko'chadi' (korrelyatsiya past) — tushuntirishning o'zi
ham hujumga zaif;")
print("xarita tarqoqligi (entropiya) adversarial namunani aniqlash belgisi sifatida
ishlatilishi mumkin.")

fig, ax = plt.subplots(4, N, figsize=(1.6 * N, 6.8))
for i in range(N):
    ax[0, i].imshow(x[i].view(8, 8), cmap="gray"); ax[0, i].set_title(f"toza:
{p_clean[i].item()}", fontsize=9)
    ax[1, i].imshow(s_clean[i].view(8, 8), cmap="hot")
    ax[2, i].imshow(xa[i].view(8, 8), cmap="gray"); ax[2, i].set_title(f"adv:
{p_adv[i].item()}", fontsize=9)
    ax[3, i].imshow(s_adv[i].view(8, 8), cmap="hot")
for a in ax.flat: a.axis("off")
for r, t in enumerate(("toza rasm", "saliency (toza)", "adversarial rasm", "saliency (adv)")):
ax[r, 0].set_ylabel(t)
plt.suptitle("Gradient-saliency: toza va PGD (eps=0.15) namunalar"); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Saliency xaritasi: toza vs adversarial`, `O'rtacha korrelyatsiya=`

TAJRIBA QILIB KO‘RING

- SmoothGrad yozing: 20 ta shovqinli nusxa bo'yicha saliency o'rtachasi — xarita silliqroqmi?
- Adversarial o'qitilgan model (3-qadam) uchun xaritalarni solishtiring: korrelyatsiya yuqoriroq chiqadimi?
- Eng muhim 8 pikselni 0 qilib (deletion) bashorat ehtimoli qanchaga tushishini o'lchang — sodiqlik metrikasi.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Adolatlilik: MetricFrame va ThresholdOptimizer

MAQSAD

Sintetik kredit ma'lumotida sezgir atribut A, u bilan bog'liq vakil belgi (income) va tarixiy tarafkashlik (A=1 guruhda ijobiylarning 25% noto'g'ri rad etilgan) modellashtiriladi. Logistik regressiya uchun MetricFrame guruhlar kesimida aniqlik, tanlov ulushi, TPR, FPR beradi; DP va EO farqlari hisoblanadi. To'rt variant taqqoslanadi: A bilan, A siz (unawareness), ThresholdOptimizer DP va EO cheklovlari bilan.

> **Kutilgan natija:** Baza: DP farqi ~0.25, EO ~0.27; A olib tashlanganda DP ~0.13 (proxy orqali saqlanadi); ThresholdOptimizer(DP) DP ni ~0.01 ga, (EO) EO ni ~0.02 ga tushiradi, aniqlik 1-3 bandga pasayadi.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Adolatlilik: Fairlearn MetricFrame va ThresholdOptimizer (sintetik "kredit"
ma'lumoti)
# Sezgir atribut A (0/1). Tarixiy tarafkashlik: A=1 guruhda belgilangan maqsad (y) past
baholangan.
import numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from fairlearn.metrics import (MetricFrame, selection_rate, true_positive_rate,
false_positive_rate,
                               demographic_parity_difference, equalized_odds_difference)
from fairlearn.postprocessing import ThresholdOptimizer

rng = np.random.default_rng(0); n = 6000
A = rng.binomial(1, 0.4, n)                                  # sezgir atribut (masalan,
hudud/jins)
income = rng.normal(5.0 - 0.8 * A, 1.0, n)                   # A bilan bog'liq (vakil belgi —
proxy)
history = rng.normal(0, 1, n); debt = rng.normal(0, 1, n)
score = 0.9 * income + 1.0 * history - 0.8 * debt
y_true_ability = (score + rng.normal(0, 1, n) > 4.5).astype(int)   # haqiqiy to'lov qobiliyati
# tarixiy ma'lumotdagi yorliq: A=1 guruhda haqiqiy ijobiylarning 25% noto'g'ri "rad" deb
yozilgan
flip = (A == 1) & (y_true_ability == 1) & (rng.random(n) < 0.25)
y = np.where(flip, 0, y_true_ability)
X = pd.DataFrame({"income": income, "history": history, "debt": debt, "A": A})
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, random_state=0, stratify=y)
Atr, Ate = Xtr["A"].values, Xte["A"].values
print(f"Bazaviy ulush P(y=1|A): A=0 -> {y[A==0].mean():.3f}, A=1 -> {y[A==1].mean():.3f}")

def report(name, pred):
    mf = MetricFrame(metrics={"aniqlik": accuracy_score, "tanlov": selection_rate,
                              "TPR": true_positive_rate, "FPR": false_positive_rate},
                     y_true=yte, y_pred=pred, sensitive_features=Ate)
    dp = demographic_parity_difference(yte, pred, sensitive_features=Ate)
    eo = equalized_odds_difference(yte, pred, sensitive_features=Ate)
    print(f"\n--- {name}: umumiy aniqlik={accuracy_score(yte, pred):.4f}  DP farqi={dp:.3f}  EO
farqi={eo:.3f}")
    print(mf.by_group.round(3).to_string())
    return accuracy_score(yte, pred), dp, eo

print("=== Adolatlilik tahlili (MetricFrame) ===")
```

```
rows = {}
clf = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
rows["Baza (A belgi sifatida kiradi)"] = report("Baza (A belgi sifatida kiradi)",
clf.predict(Xte))

feats = ["income", "history", "debt"]                    # "bilmaslik orqali adolat": A olib
tashlanadi
clf2 = LogisticRegression(max_iter=1000).fit(Xtr[feats], ytr)
rows["A olib tashlangan (unawareness)"] = report("A olib tashlangan (unawareness)",
clf2.predict(Xte[feats]))

for c in ("demographic_parity", "equalized_odds"):
    to = ThresholdOptimizer(estimator=clf2, constraints=c, prefit=True,
predict_method="predict_proba")
    to.fit(Xtr[feats], ytr, sensitive_features=Atr)
    pred = to.predict(Xte[feats], sensitive_features=Ate, random_state=0)
    rows[f"ThresholdOptimizer ({c})"] = report(f"ThresholdOptimizer ({c})", pred)

print("\n=== Yakuniy jadval ===")
print(f"{'Usul':42s} {'aniqlik':>8s} {'DP farqi':>9s} {'EO farqi':>9s}")
for k, (a, dp, eo) in rows.items(): print(f"{k:42s} {a:8.4f} {dp:9.3f} {eo:9.3f}")
print("\nXulosa: A ni olib tashlash farqni yo'qotmaydi (income — vakil belgi). Post-processing
guruhga xos")
print("chegaralar bilan tanlangan mezonni kamaytiradi, lekin aniqlik biroz tushadi va
inferensda A kerak bo'ladi.")
print("DP va EO ni bir vaqtda to'liq qondirib bo'lmaydi — mezon masala kontekstiga qarab
tanlanadi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Adolatlilik tahlili (MetricFrame) ===`, `=== Yakuniy jadval ===`

TAJRIBA QILIB KO‘RING

- Tarafkashlik ulushini 0.25 dan 0 ga tushiring — bazaviy farq qoladimi (haqiqiy bazaviy ulush farqi tufayli)?
- `fairlearn.reductions.ExponentiatedGradient` + `EqualizedOdds(difference_bound=0.02)` bilan in-processing qo'shing va jadvalga kiriting.
- `income` ni belgilar ro'yxatidan ham olib tashlang: aniqlik va DP farqi qanday o'zgaradi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. SHAP TreeExplainer: global muhimlik, lokal izoh, additivlik

MAQSAD

breast_cancer (569x30) ustida GradientBoosting o'qitiladi, TreeExplainer log-odds fazosida SHAP qiymatlarini beradi. Global o'rtacha |SHAP| top-10, bitta namuna uchun top-6 hissalar, additivlik tekshiruvi (bazaviy + sum(SHAP) = decision_function, farq ~1e-15) va 1% shovqinda tushuntirish barqarorligi chiqadi; beeswarm grafigi chiziladi.

> **Kutilgan natija:** Aniqlik ~0.94; top belgilar: worst perimeter, worst concave points, mean concave points; additivlik farqi ~1e-15; barqarorlik ~0.99.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. SHAP (TreeExplainer) bilan daraxt modelini tushuntirish: global muhimlik, lokal
izoh, additivlik
import numpy as np, pandas as pd, shap, matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split

d = load_breast_cancer(as_frame=True)
X, y = d.data, d.target                                    # 569 namuna, 30 belgi, 1 = xavfsiz
(benign)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
clf = GradientBoostingClassifier(n_estimators=150, max_depth=3, random_state=0).fit(Xtr, ytr)
print(f"Model aniqligi (test): {clf.score(Xte, yte):.4f}")

ex = shap.TreeExplainer(clf)                               # daraxtlar uchun aniq va polinomial
TreeSHAP
sv = ex.shap_values(Xte)                                   # log-odds fazosida, shakli (n, 30)
sv = sv[..., 1] if sv.ndim == 3 else sv
base = np.ravel(ex.expected_value)[-1]

print("=== SHAP: global muhimlik (o'rtacha |SHAP|) — top-10 ===")
imp = pd.Series(np.abs(sv).mean(0), index=X.columns).sort_values(ascending=False)
for i, (k, v) in enumerate(imp.head(10).items(), 1): print(f"{i:2d}. {k:28s} {v:.4f}")

i = 0
margin = clf.decision_function(Xte.iloc[[i]])[0]
print(f"\n=== Lokal tushuntirish: test namunasi #{i} (haqiqiy={yte.iloc[i]}, bashorat=
{clf.predict(Xte.iloc[[i]])[0]}) ===")
print(f"bazaviy qiymat (log-odds) = {base:.4f};  bazaviy + sum(SHAP) = {base +
sv[i].sum():.4f};  model decision_function = {margin:.4f}")
print(f"Additivlik tekshiruvi: farq = {abs(base + sv[i].sum() - margin):.2e}")
contrib = pd.Series(sv[i], index=X.columns).sort_values(key=np.abs, ascending=False)
for k, v in contrib.head(6).items():
    print(f"  {k:28s} qiymat={Xte.iloc[i][k]:9.3f}  hissa={v:+.4f}  ({'xavfsiz tomon' if v > 0
else 'xavfli tomon'})")

# Barqarorlik: kichik shovqinda tushuntirish qanchalik o'zgaradi?
rng = np.random.default_rng(0)
Xn = Xte + rng.normal(0, 0.01, Xte.shape) * Xte.std().values
svn = ex.shap_values(Xn); svn = svn[..., 1] if svn.ndim == 3 else svn
top_same = np.mean([len(set(np.argsort(-np.abs(sv[j]))[:5]) & set(np.argsort(-np.abs(svn[j])))
```

```
[:5])) / 5 for j in range(len(sv))])
print(f"\nTushuntirish barqarorligi: 1% shovqinda top-5 belgilarning o'rtacha mosligi =
{top_same:.2f}")

shap.summary_plot(sv, Xte, max_display=12, show=False)
plt.title("SHAP beeswarm — GradientBoosting, breast_cancer"); plt.tight_layout()
print("Xulosa: TreeSHAP lokal aniqlik aksiomasini qondiradi (hissalar yig'indisi = bashorat),
beeswarm global ko'rinish beradi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== SHAP: global muhimlik`, `Additivlik tekshiruvi`

TAJRIBA QILIB KO‘RING

- RandomForestClassifier bilan takrorlang — TreeExplainer ehtimollik fazosida qaytaradi; additivlikni qayta tekshiring.
- Korrelyatsiyalangan belgilar (worst radius / worst perimeter / worst area) hissalari qanday taqsimlangan? Bittasini olib tashlab qayta o'qiting.
- `shap.KernelExplainer` bilan 20 namunada solishtiring va vaqt farqini o'lchang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Etik tekshiruv ro'yxatini avtomatik baholash

MAQSAD

20 bandli ro'yxat (ISO/IEC 42001, 23894, EU AI Act, NIST AI RMF) skript bilan baholanadi: texnik bandlar (7, 8, 9, 11, 20) skript ichida tez o'lchangan metrikalardan (toza/FGSM aniqlik, DP/EO farqi, urug' va versiyalar) chegaralar bo'yicha avtomatik, tashkiliy bandlar talaba to'ldiradigan DALIL lug'atidan olinadi; dalilsiz «ha» avtomatik «qisman»ga tushiriladi. Umumiy ball /100 va xulosa chiqadi.

Kutilgan natija: 20 qatorli jadval, umumiy ball ~60/100, bajarilmagan bandlar ro'yxati (masalan 9, 12, 14, 18) va xulosa.

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Etik tekshiruv ro'yxatini (20 band) avtomatik baholovchi skript
# Texnik bandlar (7, 8, 9, 11, 20) o'lchov natijalaridan avtomatik aniqlanadi; tashkiliy
bandlar DALIL lug'atidan.
# Talaba DALIL lug'atini o'z tizimi uchun to'ldiradi: holat ("ha"/"qisman"/"yo'q") va dalil
matni.
import numpy as np, torch, torch.nn as nn, torch.nn.functional as F
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from fairlearn.metrics import demographic_parity_difference, equalized_odds_difference
torch.manual_seed(0); np.random.seed(0)

# ---- 1) Tezkor o'lchovlar (laboratoriya tizimi uchun) ----
d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16.0, d.target, test_size=0.3, random_state=0,
stratify=d.target)
Xtr, Xte, ytr, yte = map(torch.tensor, (Xtr.astype(np.float32), Xte.astype(np.float32), ytr,
yte))
m = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 10)); opt =
torch.optim.Adam(m.parameters(), 3e-3)
for _ in range(250): opt.zero_grad(); F.cross_entropy(m(Xtr), ytr).backward(); opt.step()
m.eval()
def fgsm(x, y, eps):
    x = x.clone().requires_grad_(True); g, = torch.autograd.grad(F.cross_entropy(m(x), y), x)
    return (x + eps * g.sign()).clamp(0, 1).detach()
with torch.no_grad(): clean_acc = (m(Xte).argmax(1) == yte).float().mean().item()
adv_acc = (m(fgsm(Xte, yte, 0.1)).argmax(1) == yte).float().mean().item()

rng = np.random.default_rng(0); n = 4000
A = rng.binomial(1, 0.4, n); inc = rng.normal(5 - 0.8 * A, 1, n); h = rng.normal(0, 1, n)
yf = ((0.9 * inc + h + rng.normal(0, 1, n)) > 4.5).astype(int)
Xf = np.c_[inc, h]; clf = LogisticRegression().fit(Xf[:3000], yf[:3000]); pf =
clf.predict(Xf[3000:])
dp = demographic_parity_difference(yf[3000:], pf, sensitive_features=A[3000:])
eo = equalized_odds_difference(yf[3000:], pf, sensitive_features=A[3000:])
OLCHOV = {"toza_aniqlik": clean_acc, "fgsm_aniqlik_eps0.1": adv_acc, "dp_farqi": dp,
"eo_farqi": eo,
          "urug_belgilangan": True, "versiyalar_qayd": {"torch": torch.__version__}}

# ---- 2) Dalillar (talaba to'ldiradi) ----
DALIL = {
 1: ("ha", "model_card.md: maqsad — raqam tasnifi, foydalanuvchi — laboratoriya"),
 2: ("ha", "xavf toifasi: minimal (o'quv demo); kredit modeli bo'lsa — yuqori (AI Act III
```

```
ilova)"),
 3: ("ha", "sklearn digits (BSD litsenziya), sintetik kredit ma'lumoti (generator kodi)"),
 4: ("qisman", "sinflar vakilligi tekshirilgan, guruhlar bo'yicha yo'q"),
 5: ("ha", "sezgir atribut A, vakil belgi — income (korr. bilan)"),
 6: ("ha", "mezon: equalized odds — noto'g'ri rad etish zarari og'irroq"),
 10: ("ha", "gradient-saliency xaritasi (4-qadam)"),
 12: ("yo'q", ""), 13: ("qisman", "interfeysda AI haqida yozuv rejalashtirilgan"),
 14: ("yo'q", ""), 15: ("qisman", "drift monitoringi rejasi yozilgan, amalga oshirilmagan"),
 16: ("ha", "shaxsiy ma'lumot yo'q (ochiq to'plam)"), 17: ("qisman", "model card bor, datasheet
yo'q"),
 18: ("yo'q", ""), 19: ("qisman", "mas'ul shaxs ko'rsatilgan, shikoyat mexanizmi yo'q"),
}
BANDLAR = {1: "Maqsad va ta'sir ko'lami hujjatlashtirilgan", 2: "Xavf darajasi aniqlangan", 3:
"Ma'lumot manbai/litsenziyasi",
 4: "Guruhlar vakilligi tahlili", 5: "Sezgir atributlar va proxy aniqlangan", 6: "Adolatlilik
mezoni asoslangan",
 7: "Adolatlilik ko'rsatkichlari o'lchangan (chegara: EO<0.05)", 8: "Toza va adversarial
aniqlik o'lchangan",
 9: "Himoya chorasi sinovdan o'tgan (adv aniqlik >= 50%)", 10: "XAI tushuntirish mexanizmi
mavjud",
 11: "Tushuntirish ishonchliligi tekshirilgan", 12: "Inson nazorati nuqtalari", 13:
"Foydalanuvchi AI'ni biladi",
 14: "Audit izi (jurnal)", 15: "Drift/adolatlilik monitoringi", 16: "Shaxsiy ma'lumot
minimallashtirilgan",
 17: "Model card va datasheet", 18: "Nosozlik rejimi (fallback)", 19: "Mas'ul shaxs va shikoyat
mexanizmi",
 20: "Takrorlanuvchanlik (urug', versiyalar)"}

def baholash(n):
    """Har band uchun (holat, dalil) qaytaradi. Texnik bandlar avtomatik."""
    o = OLCHOV
    if n == 7:  return ("ha" if o["eo_farqi"] < 0.05 else "qisman"), f"EO farqi=
{o['eo_farqi']:.3f}, DP farqi={o['dp_farqi']:.3f}"
    if n == 8:  return "ha", f"toza={o['toza_aniqlik']:.3f}, FGSM eps=0.1 ->
{o['fgsm_aniqlik_eps0.1']:.3f}"
    if n == 9:  return ("ha" if o["fgsm_aniqlik_eps0.1"] >= 0.5 else "yo'q"), f"adversarial
aniqlik={o['fgsm_aniqlik_eps0.1']:.3f}"
    if n == 11: return "qisman", "saliency korrelyatsiyasi o'lchangan, sodiqlik (deletion)
o'lchanmagan"
    if n == 20: return ("ha" if o["urug_belgilangan"] and o["versiyalar_qayd"] else "yo'q"),
f"seed=0, torch={o['versiyalar_qayd']['torch']}"
    holat, dalil = DALIL.get(n, ("yo'q", ""))
    if holat == "ha" and not dalil.strip(): holat = "qisman"      # dalilsiz "bajarilgan" qabul
qilinmaydi
    return holat, dalil or "-"

BALL = {"ha": 5, "qisman": 2.5, "yo'q": 0}
print("=== Etik tekshiruv ro'yxati (ISO/IEC 42001, 23894, EU AI Act, NIST AI RMF) ===")
print(f"{'№':>2s} {'Band':58s} {'Holat':8s} Dalil"); jami = 0; yoq = []
for n in range(1, 21):
    h, dl = baholash(n); jami += BALL[h]; yoq += [n] if h == "yo'q" else []
    print(f"{n:2d} {BANDLAR[n]:58s} {h:8s} {dl[:60]}")
print(f"\nUmumiy ball: {jami:.1f} / 100   bajarilmagan bandlar: {yoq}")
daraja = "yetarli (ishlab chiqarishga tayyor emas, lekin audit boshlanishi mumkin)" if jami >=
70 else \
         "qoniqarsiz — yuqori xavfli sohada ishlatish mumkin emas" if jami < 50 else "qisman —
tashkiliy bandlar to'ldirilsin"
print(f"Xulosa: {daraja}")
print("Eslatma: yuqori xavfli tizim (kredit, tibbiyot, bandlik) uchun 12, 14, 18-bandlar
majburiy (AI Act 9, 12, 14-m.).")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Etik tekshiruv ro'yxati`, `Umumiy ball:`

TAJRIBA QILIB KO‘RING

- DALIL lug'atini o'z tizimingiz uchun to'ldiring va 80+ ballga chiqing — qaysi bandlar texnik ish talab qiladi?
- 9-band chegarasini qondirish uchun 3-qadamdagi adversarial o'qitishni skriptga qo'shing.
- Xavf toifasini (minimal/cheklangan/yuqori) kiritish parametri qiling va yuqori xavfda 12, 14, 18-bandlar «yo'q» bo'lsa umumiy xulosani «taqiqlanadi» deb chiqaring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Epsilon to'rini kengaytirish** 1-qadam kodida epsilon to'rini 0 dan 0.3 gacha 0.01 qadam bilan kengaytiring, FGSM aniqligini har nuqtada o'lchang va aniqlik 50% dan pastga tushadigan eng kichik epsilon'ni (eps_50) toping. Grafik va eps_50 qiymati (kulrang darajalarda ham) hisobotga kiritiladi.	oson	5
2	**Guruhlar kesimida metrikalar** 5-qadam MetricFrame'iga `false_negative_rate` va `precision` metrikalarini qo'shing, to'rt variant uchun guruhlar jadvalini qayta chiqaring va «kredit» konteksti uchun qaysi metrika farqi eng muhim ekanini 3-4 jumla bilan asoslang.	oson	5
3	**SHAP lokal izohlar** 6-qadam kodida 3 ta namuna (bitta to'g'ri «xavfsiz», bitta to'g'ri «xavfli», bitta noto'g'ri bashorat) uchun top-5 SHAP hissalarini chiqaring, har biri uchun additivlikni tekshiring va noto'g'ri bashoratda qaysi belgilar modelni chalg'itganini izohlang.	oson	5
4	**Adversarial o'qitish radiusi** 3-qadam kodini EPS_TRAIN = 0.05, 0.1, 0.2 uchun ishga tushiring va har model uchun toza aniqlik hamda PGD-10 aniqligini eps = 0.05, 0.1, 0.15, 0.2 da o'lchang. 3x4 jadval va 3 egri chiziqli grafik tuzing; o'qitish radiusi kattalashganda toza aniqlik va kichik eps'dagi mustahkamlik qanday o'zgarishini tushuntiring.	o'rta	10
5	**Saliency asosida adversarial detektor** 4-qadam saliency funksiyasidan foydalanib, 200 toza va 200 PGD (eps=0.15) test namunasi uchun xarita entropiyasi va eng katta 5 piksel ulushini hisoblang. Shu ikki belgi bo'yicha oddiy chegara-detektor (yoki logistik regressiya) yasang, ROC-AUC ni (sklearn.metrics.roc_auc_score) o'lchang va ROC egri chizig'ini chizing.	o'rta	10
6	**In-processing tuzatish** 5-qadamga fairlearn.reductions.ExponentiatedGradient (EqualizedOdds, difference_bound=0.02) va GridSearch variantlarini qo'shing. Olti usul uchun aniqlik, DP farqi, EO farqi jadvalini tuzing, aniqlik-EO murozasini nuqtali grafikda ko'rsating va post- vs in-processing afzallik/kamchiliklarini (inferensda A kerakmi?) yozing.	o'rta	10
7	**Vakil belgilarni aniqlash** 5-qadam ma'lumotida har belgining sezgir atribut A bilan korrelyatsiyasini va A ni belgilar orqali bashorat qiluvchi yordamchi modelning AUC qiymatini hisoblang (proxy kuchi). So'ng 6-qadam SHAP yondashuvini 5-qadam modeliga (GradientBoosting) qo'llab, A olib tashlanganda ham A ta'siri qaysi belgi orqali o'tayotganini SHAP hissalari bilan ko'rsating.	o'rta	10
8	**Qora quti hujum: ko'chuvchanlik** Bir xil ma'lumotda ikki xil arxitekturali model (2-qadam MLP va kengroq/chuqurroq MLP yoki sklearn MLPClassifier) o'qiting. Birinchi modelga qarshi yasalgan FGSM va PGD namunalarini ikkinchisiga berib, ko'chuvchanlik (transfer) aniqligini 4 ta eps uchun o'lchang; oq quti natijasi bilan jadval va grafikda solishtiring va qora quti tahdid modeli uchun xulosa yozing.	qiyin	15

9	**Mos kelmaslik teoremasi amalda** 5-qadam generatorida bazaviy ulushlar farqini boshqaradigan parametr kiriting (A=1 uchun income siljishi 0, 0.4, 0.8, 1.2). Har holat uchun ThresholdOptimizer(DP) va ThresholdOptimizer(EO) ni qo'llab, DP farqi, EO farqi va guruhlar bo'yicha kalibrlash (bashorat ehtimoli bo'yicha haqiqiy ijobiy ulush, 5 ta bin) ni hisoblang. Bazaviy ulushlar farqi oshganda uchala mezonni bir vaqtda qondirib bo'lmasligini jadval va grafik bilan ko'rsating.	qiyin	15
10	**To'liq ishonchlilik hisoboti generatori** 7-qadam skriptini kengaytiring: u 1-6 qadam funksiyalarini chaqirib toza/FGSM/PGD aniqligini (3 eps), adversarial o'qitilgan modelni, DP/EO farqlarini (baza va tuzatilgan), SHAP top-5 va saliency korrelyatsiyasini o'lchasin, so'ng 20 bandni dalillar bilan to'ldirib Markdown formatida model kartasi + tekshiruv ro'yxati hisobotini print qilsin. Xavf toifasi (minimal/yuqori) kirish parametri bo'lsin; yuqori xavfda majburiy bandlar bajarilmasa «joriy etish mumkin emas» xulosasi chiqsin. Umumiy ish vaqti 60 s dan oshmasin.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Mikrokredit: hudud bo'yicha adolatlilik va raqamli imzo rasmlariga FGSM oson

Soha: Moliya (mikrokredit)

Mikromoliya tashkiloti arizachilarning skanerlangan raqamli kodlarini (digits) avtomatik o'qiydi va alohida model bilan kredit berish qarorini chiqaradi. Sezgir atribut — hudud (qishloq = 1, shahar = 0); tarixiy ma'lumotda qishloq arizachilarining to'lovga qodir qismi 20 % hollarda noto'g'ri rad etilgan. Sizdan raqam o'qiydigan modelga FGSM/PGD hujumi, qisqa adversarial o'qitish, saliency, kredit modelida DP mezoni bo'yicha tuzatish, SHAP tushuntirish va etik ro'yxatni to'ldirish talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf, [0,1] piksel
Epsilon to'ri	0, 0.02, 0.04, 0.06, 0.08, 0.1, 0.12
PGD	7 qadam, alpha = 2.5*eps/7, tasodifiy boshlanish
EPS_TRAIN	0.06
Sezgir atribut	hudud (A=1 qishloq, ulushi 0.45)
Tarafkashlik	A=1 ijobiylarining 20 % noto'g'ri rad (yorliq flip)
Vakil belgi	oylik daromad = N(4.8 - 0.6*A, 1)
Adolatlilik mezoni	demographic parity

QADAMLAR

1. s1 kodini variant epsilon to'ri bilan ishga tushiring; FGSM aniqlik jadvali (eps, eps*16 daraja, aniqlik) va tasodifiy shovqin bilan taqqoslash eps=0.08 da; aniqlik 50 % dan tushadigan eps_50 ni toping.
2. s2 kodida PGD-7 va PGD-20 ni FGSM bilan shu to'rda solishtiring; L_inf assert o'tsin; 3 egri chiziqli grafik.
3. s3 adversarial o'qitishni EPS_TRAIN=0.06, k=5 bilan bajaring; toza vs adversarial model jadvali (eps = 0, 0.04, 0.08, 0.12) va o'qitish vaqti nisbati.
4. s4 saliency panelini 8 ta raqam uchun eps=0.08 PGD bilan chizing; o'rtacha korrelyatsiya va entropiyani yozing.
5. s5 generatorini variant parametrlariga (ulush 0.45, flip 0.2, daromad siljishi 0.6) moslang; baza, unawareness, ThresholdOptimizer(DP) va (EO) uchun aniqlik, DP farqi, EO farqi jadvali; DP nega tanlanganini izohlang.
6. s6 SHAP'ni kredit ma'lumotida o'qitilgan GradientBoosting (A'siz belgilar) uchun bajaring: global top-3, bitta rad etilgan arizachi uchun lokal izoh, additivlik tekshiruvi.
7. s7 ro'yxatini mikrokredit tizimi uchun to'ldiring (xavf toifasi: yuqori), umumiy ball va bajarilmagan majburiy bandlar.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli,

beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

2-variant. Ishga qabul: rezyume skoringi — jins bo'yicha EO va hujjat rasmlariga PGD oson

Soha: Bandlik (ishga qabul)

HR-platforma nomzodlar yuklagan hujjatlardagi raqamli maydonlarni (digits) o'qiydi va alohida skoring modeli suhbatga taklif qilish qarorini chiqaradi. Sezgir atribut — jins (A=1 ayollar, ulushi 0.35); tarixiy ma'lumotda ayol nomzodlarning muvaffaqiyatli qismi 15 % hollarda rad etilgan, qo'shimcha ravishda vakil belgi sifatida 'ish tajribasi yillari' A bilan bog'liq. Sizdan hujjat o'quvchi modelga hujum tahlili va skoring modelida equalized odds bo'yicha tuzatish talab qilinadi.

Rasm ma'lumoti	sklearn digits, faqat 0-4 sinflar (5 sinf)
Epsilon to'ri	0, 0.03, 0.06, 0.1, 0.15, 0.2
PGD	10 va 30 qadam, alpha = 2*eps/qadam
EPS_TRAIN	0.1
Sezgir atribut	jins (A=1 ulushi 0.35)
Tarafkashlik	A=1 ijobiylarining 15 % flip
Vakil belgi	tajriba = N(6 - 1.2*A, 2)
Adolatlilik mezoni	equalized odds

QADAMLAR

1. s1 kodini 0-4 sinflarga cheklang (maska bilan), chiqish qatlami 5 ta; FGSM aniqlik jadvali variant to'rida; 5 sinfli masala 10 sinfliga qaraganda chidamliroqmi — izohlang.
2. s2 PGD-10 va PGD-30 (alpha = 2*eps/qadam) vs FGSM; L_inf assert; grafik.
3. s3 adversarial o'qitish EPS_TRAIN=0.1; toza/adversarial jadval; muroza qiymatlari.
4. s4 saliency: eps=0.1 PGD; 8 ta namuna paneli, korrelyatsiya o'rtachasi.
5. s5 generatori: jins ulushi 0.35, flip 0.15, tajriba belgisi A bilan bog'liq; MetricFrame'ga false_negative_rate qo'shing; 4 usul jadvali; EO nega ishga qabul uchun mos — 3 jumla.
6. s6 SHAP (GradientBoosting, skoring ma'lumoti): tajriba belgisi A ta'sirini qanday o'tkazayotganini SHAP bilan ko'rsating; additivlik.
7. s7 ro'yxati: ishga qabul = yuqori xavf (AI Act III ilova); ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

3-variant. Tibbiy triaj: shoshilinch yordam ustuvorligi — yosh guruhi va o'lchov tarafkashligi oson

Soha: Tibbiyot (triaj)

Shoshilinch yordam bo'limi bemorlarni 'shoshilinch / kutishi mumkin' sinfiga ajratuvchi model va analizator ko'rsatkichlarini rasmdan o'qiydigan OCR moduli ishlatadi. Sezgir atribut — yosh guruhi (A=1: 65 yoshdan katta, ulushi 0.3); tarafkashlik yorliqda emas, o'lchovda: keksa bemorlarda asosiy belgi (simptom bali) ko'proq shovqin bilan yozilgan. Sizdan noto'g'ri 'kutishi mumkin' (FN) zarari og'ir bo'lgani uchun TPR tengligi mezonini tekshirish talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.01, 0.03, 0.05, 0.08, 0.12
PGD	10 qadam, tasodifiy boshlanish bilan va boshlanishsiz
EPS_TRAIN	0.05
Sezgir atribut	yosh guruhi (A=1 ulushi 0.30)
Tarafkashlik	o'lchov shovqini: simptom bali A=1 da N(0, 1.8), A=0 da N(0, 0.6)
Adolatlilik mezoni	true positive rate parity (fairlearn: true_positive_rate_parity)
SHAP ma'lumoti	breast_cancer, GradientBoosting

QADAMLAR

1. s1 FGSM jadvali variant to'rida; kichik eps (0.01, 0.03) da aniqlik pasayishini kulrang darajalarda izohlang.
2. s2 PGD-10 tasodifiy boshlanish bilan/siz vs FGSM; assert; grafik; boshlanish ta'siri 2 jumla.
3. s3 adversarial o'qitish EPS_TRAIN=0.05; jadval; eps=0.12 da himoya zaiflashishini ko'rsating.
4. s4 saliency eps=0.08; korrelyatsiya va entropiya; entropiya detektor belgisi bo'la oladimi — izoh.
5. s5 generatorini o'zgartiring: yorliq flip yo'q, A=1 guruhda simptom bali shovqini kattaroq; baza, unawareness, ThresholdOptimizer(true_positive_rate_parity) va (equalized_odds) uchun aniqlik, TPR farqi, FNR farqi jadvali.
6. s6 SHAP'ni breast_cancer GradientBoosting uchun bajaring: top-10, bitta 'xavfli' namuna lokal izohi, additivlik, 1 % shovqin barqarorligi.
7. s7 ro'yxati triaj tizimi uchun (yuqori xavf, 12 va 18-bandlar majburiy): ball, xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

4-variant. Ta'lim: stipendiya tanlovi — maktab turi bo'yicha DP va imtihon varaqlari OCR hujumi oson

Soha: Ta'lim

Universitet stipendiya komissiyasi imtihon varaqlaridagi raqamlarni OCR bilan o'qiydi va alohida model stipendiyaga tavsiya qiladi. Sezgir atribut — maktab turi (A=1 qishloq maktabi, ulushi 0.5); tarixiy qarorlarda qishloq maktabi bitiruvchilarining munosib qismi 30 % hollarda rad etilgan, vakil belgi — 'ingliz tili sertifikati bali'. Sizdan OCR modelining mustahkamligini va stipendiya modelida DP tuzatishini baholash talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf, model MLP 64-96-10 (bitta yashirin qatlam)
Epsilon to'ri	0, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3
PGD	10 va 40 qadam
EPS_TRAIN	0.15
Sezgir atribut	maktab turi (A=1 ulushi 0.50)
Tarafkashlik	A=1 ijobiylarining 30 % flip
Vakil belgi	ingliz tili bali = N(60 - 12*A, 15)
Adolatlilik mezoni	demographic parity

QADAMLAR

1. s1 modelini bitta yashirin qatlamli (64-96-10) qiling; FGSM jadvali variant to'rida; chuqurroq lab modeli bilan eps=0.1 da solishtiring.
2. s2 PGD-10 va PGD-40 vs FGSM; assert; grafik.
3. s3 EPS_TRAIN=0.15 bilan adversarial o'qitish; toza aniqlik qancha tushdi; eps=0.05 da himoya.
4. s4 saliency eps=0.15; panel; korrelyatsiya.
5. s5 generatori: ulush 0.5, flip 0.3, ingliz tili bali vakil belgi; 4 usul jadvali; MetricFrame'ga precision qo'shing; DP tanlovini 3 jumla bilan asoslang.
6. s6 SHAP (stipendiya ma'lumoti, GradientBoosting): ingliz tili bali hissasi guruhlar bo'yicha o'rtacha qanday farq qiladi; additivlik.
7. s7 ro'yxati ta'lim tizimi uchun (yuqori xavf: ta'limga kirish); ball va tavsiyalar.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

5-variant. Avto sug'urta: narxlash skoringi — yosh guruhi va haqiqiy bazaviy farq

oson

Soha: Sug'urta

Sug'urta kompaniyasi avto polis uchun 'yuqori xavfli haydovchi' bashoratini qiladi va haydovchilik guvohnomasi raqamlarini rasmdan o'qiydi. Sezgir atribut — yosh (A=1: 25 yoshgacha, ulushi 0.25). Bu variantda yorliq flip yo'q: guruhlar orasida haqiqiy bazaviy ulush farqi bor (yoshlarda hodisalar ko'proq), shuning uchun DP va EO bir vaqtda qondirilmaydi. Sizdan hujum tahlili bilan birga qaysi mezon sug'urta uchun to'g'ri ekanini raqamlar bilan asoslash talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.02, 0.05, 0.08, 0.1, 0.15
PGD	10 qadam, alpha = 3*eps/10
EPS_TRAIN	0.08
Sezgir atribut	yosh guruhi (A=1 ulushi 0.25)
Tarafkashlik	flip 0; bazaviy farq: xavf bali = 0.9*staj + 0.7*A + shovqin
Adolatlilik mezoni	equalized odds vs demographic parity (taqqoslash)
SHAP ma'lumoti	sug'urta sintetik, GradientBoosting

QADAMLAR

1. s1 FGSM jadvali; tasodifiy shovqin taqqoslash eps=0.1.
2. s2 PGD-10 (alpha = 3*eps/10) vs FGSM; alpha kattaligining ta'siri haqida izoh; assert; grafik.
3. s3 EPS_TRAIN=0.08 adversarial o'qitish; jadval; vaqt nisbati.
4. s4 saliency eps=0.1; panel; korrelyatsiya.
5. s5 generatorini o'zgartiring: flip yo'q, A to'g'ridan-to'g'ri xavfga ta'sir qiladi (haqiqiy bazaviy farq); 4 usul jadvali; ThresholdOptimizer(DP) aniqlikni qancha tushiradi va EO nega sug'urta uchun ma'qulroq — asoslang.
6. s6 SHAP (GradientBoosting, sug'urta ma'lumoti, A'siz): staj belgisining hissasi; bitta yosh haydovchi uchun lokal izoh; additivlik.
7. s7 ro'yxati (sug'urta narxlash: yuqori xavf AI Act bo'yicha); ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

6-variant. Reklama: ish e'lonlarini ko'rsatish — jins bo'yicha tanlov ulushi va banner raqamlariga hujum oson

Soha: Reklama (marketing)

Reklama platformasi foydalanuvchiga yuqori maoshli ish e'lonini ko'rsatish-ko'rsatmaslikni bashorat qiladi va bannerlardagi raqamli kodlarni rasmdan tekshiradi. Sezgir atribut — jins (A=1 ulushi 0.5); tarixiy bosish (click) yorlig'ida ayollar uchun ijobiy natijalar 25 % kam qayd etilgan, vakil belgi — 'ko'rilgan toifalar' soni. Sizdan tanlov ulushi (selection rate) farqini o'lchash va DP tuzatishi reklama samaradorligiga ta'sirini baholash talab qilinadi.

Rasm ma'lumoti	sklearn digits, juft/toq (2 sinf)
Epsilon to'ri	0, 0.05, 0.1, 0.15, 0.2, 0.3
PGD	10 qadam
EPS_TRAIN	0.1
Sezgir atribut	jins (A=1 ulushi 0.50)
Tarafkashlik	A=1 ijobiylarining 25 % flip
Vakil belgi	toifalar soni = Poisson(7 - 2*A)
Adolatlilik mezoni	demographic parity (selection rate)

QADAMLAR

1. s1 modelini 2 sinfli (juft/toq) qiling; FGSM jadvali; 2 sinfli masalada tasodifiy taxmin 50 % ekanini hisobga olib eps_50 o'rniga eps_75 ni toping.
2. s2 PGD-10 vs FGSM; assert; grafik.
3. s3 EPS_TRAIN=0.1 adversarial o'qitish; jadval; muroza.
4. s4 saliency eps=0.15, 8 ta namuna; korrelyatsiya.
5. s5 generatori: ulush 0.5, flip 0.25, Poisson vakil belgi; MetricFrame: selection_rate, TPR, FPR; 4 usul; DP tuzatishdan keyin umumiy tanlov ulushi va aniqlik qanday o'zgardi.
6. s6 SHAP (reklama ma'lumoti, GradientBoosting): toifalar soni belgisi hissasi; beeswarm; additivlik.
7. s7 ro'yxati (reklama: cheklangan xavf, lekin bandlikka ta'sir bo'lsa yuqori); ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

7-variant. Telekom: chegirma taklifi modeli — til/hudud bo'yicha adolatlilik va SIM-karta raqamlari OCR oson

Soha: Telekom

Mobil operator ketish ehtimoli yuqori abonentlarga chegirma taklif qiladi; SIM-karta raqamlari rasmdan o'qiladi. Sezgir atribut — xizmat tili/hudud (A=1 ulushi 0.4); tarixda A=1 guruhdagi 'saqlab qolinadigan' abonentlarning 18 % noto'g'ri belgilangan, vakil belgi — 'oylik to'lov'. Sizdan OCR mustahkamligi va chegirma modelida EO tuzatishini baholash hamda wine ma'lumotida SHAP tushuntirish talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf, o'qitish 150 qadam (kam o'qitilgan model)
Epsilon to'ri	0, 0.02, 0.05, 0.1, 0.15, 0.2
PGD	10 qadam
EPS_TRAIN	0.1, k = 3
Sezgir atribut	til/hudud (A=1 ulushi 0.40)
Tarafkashlik	A=1 ijobiylarining 18 % flip
Vakil belgi	oylik to'lov = N(5.2 - 0.9*A, 1.2)
SHAP ma'lumoti	sklearn wine (3 sinf), GradientBoosting

QADAMLAR

1. s1 modelini 150 qadam o'qiting; FGSM jadvali; 300 qadamli model bilan eps=0.1 da solishtiring (kam o'qitilgan model chidamliroqmi?).
2. s2 PGD-10 vs FGSM; assert; grafik.
3. s3 EPS_TRAIN=0.1, k=3 adversarial o'qitish; k=5 bilan vaqt va mustahkamlikni solishtiring.
4. s4 saliency eps=0.15; panel; korrelyatsiya.
5. s5 generatori: ulush 0.4, flip 0.18, oylik to'lov vakil belgi; 4 usul jadvali; EO tanlovini asoslang.
6. s6 SHAP'ni wine (3 sinf) GradientBoosting uchun bajaring: sinf bo'yicha SHAP massivini to'g'ri tanlang, top-5, additivlik ko'p sinfli holatda.
7. s7 ro'yxati (telekom chegirma: cheklangan xavf); ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

8-variant. Jamoat xavfsizligi: tekshiruv ustuvorligi modeli — hudud bo'yicha FPR tengligi va fikr-mulohaza halqasi o'rta

Soha: Xavfsizlik (jamoat tartibi)

Shahar xavfsizlik xizmati qaysi murojaatlarni birinchi tekshirishni bashorat qiluvchi model va hujjat raqamlarini o'qiydigan OCR ishlatadi. Sezgir atribut — hudud (A=1 ulushi 0.35); tarafkashlik fikr-mulohaza halqasi ko'rinishida: A=1 hududda tekshiruvlar ko'proq o'tkazilgani uchun 'ijobiy' yorliqlar 1.6 baravar ko'p qayd etilgan (haqiqiy xavf teng). Noto'g'ri tekshiruv (FP) zarari og'ir, shuning uchun FPR tengligi asosiy mezon.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.02, 0.05, 0.1, 0.15
PGD	10 va 40 qadam; qo'shimcha L_2 PGD (eps_2 = 1.0, 2.0)
EPS_TRAIN	0.1
Sezgir atribut	hudud (A=1 ulushi 0.35)
Tarafkashlik	A=1 guruhda haqiqiy salbiylarning 20 % 'ijobiy' deb yozilgan (ortiqcha nazorat)
Adolatlilik mezoni	false positive rate parity
SHAP ma'lumoti	xavfsizlik sintetik, GradientBoosting

QADAMLAR

1. s1 FGSM jadvali variant to'rida.
2. s2 PGD-10, PGD-40 vs FGSM (L_inf) va qo'shimcha L_2 PGD (gradientni normalashtirib, eps_2 radiusli sharga proyeksiya) eps_2 = 1.0, 2.0; ikkala tahdid modelini bitta jadvalda.
3. s3 adversarial o'qitish EPS_TRAIN=0.1; L_inf'ga o'qitilgan model L_2 hujumga qanchalik chidamli — jadval.
4. s4 saliency eps=0.1; panel; korrelyatsiya va entropiya.
5. s5 generatorini o'zgartiring: A=1 guruhda salbiylarning 20 % ijobiy deb flip (ortiqcha nazorat); MetricFrame: FPR, FNR, selection_rate; baza, unawareness, ThresholdOptimizer(false_positive_rate_parity), (equalized_odds); FPR farqi va aniqlik jadvali.
6. Fikr-mulohaza halqasi simulyatsiyasi: tuzatilmagan model bashoratlarini 3 'yil' davomida yangi yorliq sifatida qayta o'qiting va FPR farqi qanday o'sishini grafikda ko'rsating; tuzatilgan model bilan takrorlang.
7. s6 SHAP (sintetik, GradientBoosting): A olib tashlanganda ham qaysi belgi hudud ta'sirini o'tkazadi; s7 ro'yxati (yuqori xavf, huquqni muhofaza qilish — AI Act III ilova): ball, majburiy bandlar.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

9-variant. Uy-joy ijarasi: ijarachini tanlash platformasi — oilaviy holat va vakil belgilar o'rta

Soha: Uy-joy (ijara platformasi)

Onlayn ijara platformasi uy egalariga ijarachi arizasini 'tavsiya / tavsiya emas' deb baholaydi va pasport raqamlarini rasmdan o'qiydi. Sezgir atribut — bolalar bilan oila (A=1 ulushi 0.3); tarixiy rad etishlarda A=1 guruhning to'lovga qodir qismi 22 % flip qilingan, ikkita vakil belgi bor: 'oila a'zolari soni' va 'kerakli xonalar soni'. Sizdan vakil belgilar kuchini o'lchash (A ni bashorat qiluvchi yordamchi model AUC) va DP tuzatish talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.03, 0.06, 0.1, 0.15, 0.2
PGD	10 qadam, alpha = 2.5*eps/10
EPS_TRAIN	0.08
Sezgir atribut	bolalar bilan oila (A=1 ulushi 0.30)
Tarafkashlik	A=1 ijobiylarining 22 % flip
Vakil belgilar	oila a'zolari = Poisson(2 + 2*A), xonalar = 1 + a'zolar/2 + shovqin
Adolatlilik mezoni	demographic parity; proxy kuchi: A ni belgilar orqali bashorat qiluvchi LogisticRegression AUC

QADAMLAR

1. s1 FGSM jadvali; s2 PGD-10 vs FGSM; assert; grafik (ikkala qadam bitta skriptda).
2. s3 adversarial o'qitish EPS_TRAIN=0.08; jadval; muroza.
3. s4 saliency eps=0.1; panel; SmoothGrad (10 shovqinli nusxa) bilan korrelyatsiya qanday o'zgaradi.
4. s5 generatori: ikki vakil belgi; har belgining A bilan korrelyatsiyasi va A ni bashorat qiluvchi yordamchi model AUC (proxy kuchi) jadvali.
5. 4 usul jadvali (baza, unawareness, ThresholdOptimizer DP va EO); vakil belgilarni ham olib tashlagan 5-usulni qo'shing: aniqlik va DP farqi.
6. s6 SHAP (ijara ma'lumoti, GradientBoosting, A'siz): oila a'zolari va xonalar hissasi A=0/A=1 guruhlarda o'rtacha; additivlik.
7. s7 ro'yxati (uy-joyga kirish: yuqori xavf); ball va xulosa — vakil belgilar bilan nima qilish kerak.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

10-variant. Universitetga qabul: kirish ballari modeli — maktab tili va test balining vakilligi o'rta

Soha: Ta'lim (qabul)

Universitet qabul komissiyasi abituriyentning birinchi kursni muvaffaqiyatli tugatish ehtimolini bashorat qiladi va test varaqlaridagi raqamlarni OCR bilan o'qiydi. Sezgir atribut — maktab ta'lim tili (A=1 ulushi 0.4); tarafkashlik ikki qatlamli: test bali A=1 uchun o'rtacha 8 ballga past o'lchangan (o'lchov tarafkashligi), yorliq flip 10 %. Sizdan EO tuzatish, in-processing (ExponentiatedGradient) qo'shish va OCR modeliga qora quti hujumni baholash talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf; ikkinchi model sklearn MLPClassifier (qora quti)
Epsilon to'ri	0, 0.05, 0.1, 0.15, 0.2
PGD	10 qadam
EPS_TRAIN	0.1
Sezgir atribut	maktab tili (A=1 ulushi 0.40)
Tarafkashlik	test bali = haqiqiy - 8*A + N(0, 5); flip 10 %
Adolatlilik mezoni	equalized odds; in-processing ExponentiatedGradient(EqualizedOdds, difference_bound=0.02)
SHAP ma'lumoti	qabul sintetik, GradientBoosting

QADAMLAR

1. s1 FGSM va s2 PGD-10 jadvallari; assert; grafik.
2. Ko'chuvchanlik: PyTorch MLP uchun yasalgan FGSM/PGD namunalarini sklearn MLPClassifier (hidden 100, 64 belgi) ga bering; 4 eps uchun transfer aniqligi jadvali va oq quti bilan taqqoslash.
3. s3 adversarial o'qitish EPS_TRAIN=0.1; adversarial model uchun transfer hujum qanchalik ishlaydi.
4. s4 saliency eps=0.1; panel; korrelyatsiya.
5. s5 generatori: o'lchov tarafkashligi (test bali - 8*A) + flip 0.1; baza, unawareness, ThresholdOptimizer(EO), ExponentiatedGradient(EO, 0.02) — 4 usul; aniqlik-EO muroza nuqtali grafigi.
6. s6 SHAP (GradientBoosting, qabul ma'lumoti): test bali hissasi A guruhlari bo'yicha; additivlik; shovqin barqarorligi.
7. s7 ro'yxati (ta'limga kirish: yuqori xavf); ball va xulosa — o'lchov tarafkashligini qaysi bosqichda tuzatish kerak (ma'lumot yoki model).

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

11-variant. Sog'liq sug'urtasi: xarajat prognozi — surunkali kasallik guruhi va regressiya SHAP o'rta

Soha: Sug'urta (tibbiy)

Tibbiy sug'urta kompaniyasi mijozning yillik xarajatini bashorat qiladi va retsept raqamlarini rasmdan o'qiydi. Sezgir atribut — surunkali kasallik mavjudligi (A=1 ulushi 0.2); kompaniya 'yuqori xarajat' sinfini chegara bilan aniqlaydi va A=1 guruhda tarixiy yorliqlar 20 % ortiqcha 'yuqori' deb belgilangan. Sizdan tasniflash adolatliligi bilan birga diabetes regressiya ma'lumotida SHAP tushuntirish (TreeExplainer, GradientBoostingRegressor) talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.02, 0.04, 0.08, 0.12, 0.16
PGD	10 qadam; 20 qadam
EPS_TRAIN	0.08
Sezgir atribut	surunkali kasallik (A=1 ulushi 0.20)
Tarafkashlik	A=1 salbiylarining 20 % 'yuqori xarajat' deb flip
Adolatlilik mezoni	equalized odds, FPR farqiga alohida e'tibor
SHAP ma'lumoti	sklearn diabetes (regressiya), GradientBoostingRegressor

QADAMLAR

1. s1 FGSM va s2 PGD-10/PGD-20 jadvallari; assert; grafik.
2. s3 adversarial o'qitish EPS_TRAIN=0.08; jadval; k=1 (FGSM-AT) bilan 'katastrofik overfitting' bor-yo'qligini tekshiring.
3. s4 saliency eps=0.08; panel; eng muhim 8 pikselni 0 qilib (deletion) bashorat ehtimoli qanchaga tushishini o'lchang.
4. s5 generatori: A=1 salbiylarining 20 % ijobiy flip; MetricFrame: FPR, FNR, selection_rate; baza, unawareness, ThresholdOptimizer(EO), (DP); FPR farqi jadvali.
5. s6 SHAP'ni diabetes GradientBoostingRegressor uchun bajaring: expected_value + sum(SHAP) = predict (additivlik), top-5 belgilar, bitta bemor uchun lokal izoh, dependence plot (bmi).
6. Barqarorlik: 1 % va 5 % shovqinda top-5 mosligi; regressiya SHAP'da qiymatlar birligi (xarajat birligida) haqida izoh.
7. s7 ro'yxati (tibbiy sug'urta: yuqori xavf); ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi

asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

12-variant. E-tijorat: to'lov firibgarligi detektori — mamlakat/hudud bo'yicha FPR va nomutanosib sinflar o'rta

Soha: E-tijorat (to'lovlar)

To'lov shlyuzi tranzaksiyani 'firibgarlik' deb bloklash qarorini chiqaradi va karta raqamlarini rasmdan o'qiydi. Sezgir atribut — hudud (A=1 ulushi 0.3); firibgarlik 4 % (nomutanosib), A=1 hududda tarixiy bloklash qoidalari tufayli salbiylarning 12 % 'firibgarlik' deb yozilgan. Halol mijozni bloklash (FP) biznesga zarar, shuning uchun FPR tengligi asosiy; sizdan nomutanosib sinflarda MetricFrame bilan to'g'ri o'lchash va chegara tanlash talab qilinadi.

Rasm ma'lumoti	sklearn digits, faqat 1 vs 7 (2 sinf, chalkash juftlik)
Epsilon to'ri	0, 0.03, 0.06, 0.1, 0.15, 0.2
PGD	10 qadam; yo'naltirilgan (targeted) FGSM 1 -> 7
EPS_TRAIN	0.1
Sezgir atribut	hudud (A=1 ulushi 0.30)
Tarafkashlik	ijobiy ulush 4 %; A=1 salbiylarining 12 % flip
Adolatlilik mezoni	false positive rate parity
SHAP ma'lumoti	firibgarlik sintetik, GradientBoosting (class_weight o'rniga sample_weight)

QADAMLAR

1. s1 modelini 1 vs 7 ikki sinfli qiling; FGSM jadvali; yo'naltirilgan FGSM (1 -> 7, gradientni maqsad sinf yo'qotishi bo'yicha teskari) muvaffaqiyat ulushi eps bo'yicha.
2. s2 PGD-10 vs FGSM; assert; grafik.
3. s3 adversarial o'qitish EPS_TRAIN=0.1; jadval.
4. s4 saliency eps=0.1 uchun 1 va 7 rasmlari; qaysi piksellar hal qiluvchi; korrelyatsiya.
5. s5 generatori: ijobiy ulush 4 % (chegara bilan), A=1 salbiylarining 12 % flip; LogisticRegression(class_weight='balanced'); MetricFrame: FPR, TPR, precision, selection_rate; baza, unawareness, ThresholdOptimizer(false_positive_rate_parity), (equalized_odds).
6. Nomutanosiblik ta'siri: umumiy aniqlik 96 % bo'lsa ham FPR farqi katta bo'lishi mumkinligini ko'rsating; balanced accuracy qo'shing; s6 SHAP (GradientBoosting, sample_weight) top-5 va additivlik.
7. s7 ro'yxati (to'lov bloklash: yuqori xavf — moliyaviy xizmatlarga kirish); ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi

asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

13-variant. Ijtimoiy himoya: subsidiya ustuvorligi — oila kattaligi/hudud va kalibrlash o'rta

Soha: Davlat xizmatlari

Ijtimoiy himoya agentligi oilalarni subsidiya uchun 'ustuvor' deb belgilaydi va arizadagi raqamlarni OCR bilan o'qiydi. Sezgir atribut — hudud (A=1 ulushi 0.55, ko'pchilik guruh); tarixiy ro'yxatlarda A=1 hududning muhtoj oilalari 28 % hollarda ro'yxatga kiritilmagan, vakil belgi — 'kommunal to'lov qarzi'. Agentlik ehtimollik bahosiga tayanadi, shuning uchun guruhlar bo'yicha kalibrlash ham tekshiriladi.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.02, 0.05, 0.1, 0.15, 0.2, 0.3
PGD	10 qadam
EPS_TRAIN	0.1
Sezgir atribut	hudud (A=1 ulushi 0.55)
Tarafkashlik	A=1 ijobiylarining 28 % flip; vakil belgi: kommunal qarz = N(2 + 1.1*A, 1)
Adolatlilik mezoni	demographic parity + guruhlar bo'yicha kalibrlash (5 bin)
SHAP ma'lumoti	subsidiya sintetik, GradientBoosting

QADAMLAR

1. s1 FGSM va s2 PGD-10 jadvallari; assert; grafik.
2. s3 adversarial o'qitish EPS_TRAIN=0.1; jadval; muroza.
3. s4 saliency eps=0.15; panel; korrelyatsiya va entropiya; adversarial o'qitilgan model uchun xaritalar korrelyatsiyasi yuqoriroqmi.
4. s5 generatori: ulush 0.55, flip 0.28, kommunal qarz vakil belgi; 4 usul jadvali (aniqlik, DP, EO).
5. Kalibrlash: har usul uchun predict_proba bo'yicha 5 bin, har binda haqiqiy ijobiy ulush A=0/A=1 uchun; kalibrlash egri chiziqlari; ThresholdOptimizer kalibrlashni buzadimi.
6. s6 SHAP (GradientBoosting, A'siz): kommunal qarz hissasi; bitta rad etilgan oila uchun lokal izoh (fuqaroga tushunarli tilda yozib bering); additivlik.
7. s7 ro'yxati (davlat xizmatlariga kirish: yuqori xavf; 13, 19-bandlar muhim); ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

14-variant. Taksi platformasi: haydovchi reytingi va buyurtma taqsimoti — jins/yosh kesishmasi o'rta

Soha: Transport (platforma)

Taksi platformasi haydovchiga 'premium buyurtma' berish qarorini modellashtiradi va avtomobil raqamlarini rasmdan o'qiydi. Ikki sezgir atribut: jins (A1, ulushi 0.2) va yosh guruhi 50+ (A2, ulushi 0.25); tarafkashlik faqat kesishmada — A1=1 va A2=1 bo'lgan haydovchilarning munosib qismi 35 % flip. Sizdan MetricFrame'da kesishgan guruhlar (4 ta) bo'yicha tahlil, ThresholdOptimizer'ni birlashgan atribut bilan qo'llash va avtomobil raqami OCR mustahkamligi talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.04, 0.08, 0.12, 0.16, 0.2
PGD	10 qadam, alpha = 2.5*eps/10
EPS_TRAIN	0.12
Sezgir atributlar	jins A1 (ulushi 0.20), yosh 50+ A2 (ulushi 0.25) — kesishma 4 guruh
Tarafkashlik	faqat A1=1 va A2=1 kesishmada ijobiylarning 35 % flip
Vakil belgi	o'rtacha reyting = N(4.6 - 0.15*A1 - 0.1*A2, 0.2)
Adolatlilik mezoni	equalized odds (kesishgan guruhlar)

QADAMLAR

1. s1 FGSM va s2 PGD-10 jadvallari; assert; grafik.
2. s3 adversarial o'qitish EPS_TRAIN=0.12; jadval; muroza.
3. s4 saliency eps=0.12; panel; korrelyatsiya.
4. s5 generatorini ikki atributli qiling; MetricFrame'ni sensitive_features sifatida DataFrame [A1, A2] bilan chaqiring — 4 kesishgan guruh uchun aniqlik, TPR, FPR, selection_rate; faqat A1 yoki faqat A2 bo'yicha tahlil kesishma muammosini yashirishini ko'rsating.
5. ThresholdOptimizer(EO) ni birlashgan atribut (A1*2 + A2) bilan qo'llang; 4 usul jadvali; guruhlar soni oshganda har guruhda namuna kamayishi va chegaralar barqarorligi haqida izoh (bootstrap 5 marta).
6. s6 SHAP (GradientBoosting, A'siz): reyting belgisi hissasi 4 guruhda; additivlik.
7. s7 ro'yxati (platforma ishchilari: yuqori xavf — bandlik); ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

15-variant. Kontent moderatsiyasi: izohlarni bloklash — til bo'yicha FPR va matn belgilariga hujum o'rta

Soha: Media (moderatsiya)

Ijtimoiy tarmoq izohlarni 'zararli' deb avtomatik yashiradi; model so'z-sumka belgilarida ishlaydi, rasmli kodlar uchun OCR bor. Sezgir atribut — izoh tili (A=1 ulushi 0.45); tarafkashlik: A=1 tilidagi zararsiz izohlarning 15 % tarixan noto'g'ri bloklangan, vakil belgi — 'noodatiy belgilar ulushi'. Bu variantda adversarial hujum jadval (so'z-sumka) modeliga ham qo'llanadi: belgilar fazosida L_inf buzilish va chegaralangan to'liq sonli (so'z qo'shish) hujum.

Rasm ma'lumoti	sklearn digits, 10 sinf (OCR moduli)
Matn ma'lumoti	sintetik so'z-sumka 6000 x 200 (hisoblagichlar 0-5), 2 sinf
Epsilon to'ri (rasm)	0, 0.05, 0.1, 0.15, 0.2
Epsilon (matn, standartlangan fazo)	0, 0.1, 0.25, 0.5, 1.0; to'liq sonli hujum: 1, 3, 5 so'z qo'shish
EPS_TRAIN	0.1 (rasm)
Sezgir atribut	izoh tili (A=1 ulushi 0.45)
Tarafkashlik	A=1 salbiylarining 15 % flip (ortiqcha bloklash)
Adolatlilik mezoni	false positive rate parity

QADAMLAR

1. s1/s2: digits OCR uchun FGSM va PGD-10 jadvali; assert; grafik.
2. Matn modeli: so'z-sumka generatori + MLP 200-64-2; standartlangan fazoda FGSM/PGD (eps ro'yxati) va to'liq sonli hujum (gradient bo'yicha eng ta'sirli k ta so'z hisoblagichini +1) uchun aniqlik jadvali; qaysi tahdid modeli realistik — izoh.
3. s3 adversarial o'qitish (rasm, EPS_TRAIN=0.1) va matn modeli uchun to'liq sonli hujumga qarshi augmentatsiya; jadval.
4. s4 saliency: rasm uchun panel; matn uchun eng ta'sirli 10 so'z indeksini (|gradient|) chiqaring.
5. s5 generatori: til ulushi 0.45, salbiylarning 15 % flip, noodatiy belgilar ulushi vakil belgi; MetricFrame FPR/TPR; baza, unawareness, ThresholdOptimizer(false_positive_rate_parity), (EO).
6. s6 SHAP (GradientBoosting, moderatsiya sintetik ma'lumoti): noodatiy belgilar hissasi; additivlik; 1 % shovqin barqarorligi.
7. s7 ro'yxati (so'z erkinligi, DSA talablari — cheklangan/yuqori xavf); 13 va 19-bandlar (foydalanuvchi xabardorligi, shikoyat) majburiy; ball va xulosa.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (hujumlar L_inf chegarasida, assert o'tadi) — 30, O'lchovlar va jadvallar (mustahkamlik, adolatlilik, SHAP) — 25, Tahlil va xulosa (mezon tanlovi asoslangan) — 25, Hisobot va tekshiruv ro'yxati sifati — 20

16-variant. Bank: qarz undirish ustuvorligi — kalibrlash, DP va EO mos kelmasligi amalda qiyin

Soha: Moliya (bank)

Bank muddati o'tgan qarzdorlar orasidan 'qattiq undirish' choralarini qo'llanadiganlarni bashorat qiladi; sezgir atribut — hudud (A=1 ulushi 0.4). Generatorda A=1 uchun daromad siljishi parametri bor (0, 0.5, 1.0, 1.5): bazaviy ulushlar farqi oshganda DP, EO va kalibrlashni bir vaqtda qondirib bo'lmasligini ko'rsatish kerak. Qo'shimcha: bank OCR moduliga qora quti (transfer) hujum va in-processing tuzatish talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf; qora quti: kengroq MLP 64-256-256-10
Epsilon to'ri	0, 0.05, 0.1, 0.15, 0.2
PGD	10 va 40 qadam
EPS_TRAIN	0.1
Sezgir atribut	hudud (A=1 ulushi 0.40)
Daromad siljishi (A=1)	0, 0.5, 1.0, 1.5
Tarafkashlik	flip 10 % (doimiy)
Mezonlar	DP, EO, kalibrlash (5 bin), ExponentiatedGradient(EqualizedOdds, 0.02)

QADAMLAR

1. s1/s2: FGSM, PGD-10, PGD-40 jadvali; assert; ko'chuvchanlik: lab MLP'ga qarshi yasalgan namunalarni kengroq MLP'ga bering, 4 eps uchun transfer aniqligi.
2. s3 adversarial o'qitish EPS_TRAIN=0.1; adversarial modelga transfer hujum aniqligi; s4 saliency korrelyatsiyasi toza va adversarial modellar uchun.
3. s5 generatoriga siljish parametri kiriting; har siljish (0, 0.5, 1.0, 1.5) uchun bazaviy ulushlar P(y=1|A) jadvali.
4. Har siljish uchun baza, ThresholdOptimizer(DP), ThresholdOptimizer(EO), ExponentiatedGradient(EO) — DP farqi, EO farqi, aniqlik va guruhlar bo'yicha kalibrlash xatosi (5 bin, |haqiqiy ulush - bashorat ehtimoli| o'rtachasi); 4 x 4 jadval.
5. Grafik: x — siljish, y — DP farqi / EO farqi / kalibrlash xatosi, har usul alohida; siljish oshganda uchala mezonni birga qondirib bo'lmasligini (Kleinberg-Chouldechova) raqamlar bilan ko'rsating.
6. s6 SHAP (GradientBoosting, siljish 1.0 holati): daromad hissasi guruhlar bo'yicha; additivlik; barqarorlik.
7. s7 ro'yxati (qarz undirish: yuqori xavf) + bank uchun 1 sahifalik tavsiya: qaysi mezon, nega, qanday monitoring; ball.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadvallar va grafiklar — 25, Chuqur tahlil (murozalar, mos kelmaslik, cheklovlar) — 30, Hisobot, model kartasi va tekshiruv ro'yxati sifati — 20

17-variant. Kasalxona: qayta yotqizish xavfi — yosh va jins kesishmasi, ko'chuvchan hujum va detektor qiyin

Soha: Tibbiyot (klinika)

Kasalxona 30 kun ichida qayta yotqizish xavfini bashorat qiladi; ikki sezgir atribut (yosh 65+, jins). Tibbiy ko'rsatkichlar OCR orqali rasmdan olinadi va hujumchi OCR modelining arxitekturasini bilmaydi (qora quti). Sizdan transfer hujum, saliency asosida adversarial detektor (ROC-AUC), kesishgan guruhlar adolatliligi (EO), breast_cancer SHAP va yuqori xavfli tibbiy tizim uchun to'liq tekshiruv ro'yxati talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf; qora quti: sklearn MLPClassifier(hidden 128,64) va o'zgacha PyTorch MLP 64-64-64-10
Epsilon to'ri	0, 0.05, 0.1, 0.15, 0.2
PGD	10 qadam; EPS_TRAIN 0.1
Detektor	saliency entropiyasi + top-5 piksel ulushi, 200 toza + 200 PGD namuna, LogisticRegression, ROC-AUC
Sezgir atributlar	yosh 65+ (0.3), jins (0.5) — 4 kesishgan guruh
Tarafkashlik	keksa ayollar guruhida ijobiylarning 25 % flip; o'lchov shovqini keksalarda 1.5x
Adolatlilik mezoni	equalized odds (kesishma)
SHAP ma'lumoti	breast_cancer, GradientBoosting

QADAMLAR

1. s1/s2 oq quti jadvali; so'ng ikki qora quti modelga transfer aniqligi 4 eps uchun; qaysi arxitektura juftligi ko'proq ko'chadi — izoh.
2. s3 adversarial o'qitish EPS_TRAIN=0.1; adversarial model uchun transfer; jadval va grafik (6 egri chiziq).
3. s4 saliency asosida detektor: 200 toza + 200 PGD (eps=0.15) namuna uchun entropiya va top-5 ulush; LogisticRegression detektor; ROC-AUC va ROC egri chizig'i; eps=0.05 namunalarda AUC qanday tushadi.
4. s5 generatori ikki atributli (yosh, jins), kesishmada flip 0.25 va o'lchov shovqini; MetricFrame 4 guruh (TPR, FPR, FNR); baza, unawareness, ThresholdOptimizer(EO) birlashgan atribut bilan, ExponentiatedGradient(EO).
5. Kalibrlash 4 guruh bo'yicha (5 bin) va FNR farqi — klinik zarar nuqtai nazaridan qaysi metrika birinchi; jadval + grafik.
6. s6 SHAP breast_cancer: top-10, 3 ta namuna (to'g'ri xavfsiz, to'g'ri xavfli, noto'g'ri) lokal izohlari; additivlik; korrelyatsiyalangan belgilar (worst radius/perimeter/area) hissalari taqsimoti.
7. s7 ro'yxati tibbiy tizim uchun (yuqori xavf, MDR/AI Act): 12, 14, 18-bandlar majburiy, xavf toifasi kiritish parametri, bajarilmasa 'joriy etish mumkin emas' xulosasi; ball.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadvallar va grafiklar — 25, Chuqur tahlil (murozalar, mos kelmaslik, cheklovlar) — 30, Hisobot, model kartasi va tekshiruv ro'yxati sifati — 20

18-variant. HR: rezyume saralash tizimi — to'liq ishonchlilik hisoboti va model kartasi generatori qiyin

Soha: Bandlik (HR tizimi)

Yirik kompaniya rezyumelarni avtomatik saralaydi (jins va yosh — sezgir atributlar) va skanerlangan hujjatlardagi raqamlarni OCR bilan o'qiydi. Yuridik bo'lim audit oldidan Markdown formatida model kartasi + 20 bandli tekshiruv ro'yxatini avtomatik ishlab chiqaruvchi skript talab qiladi: u 1-6 qadam funksiyalarini chaqirib barcha metrikalarni o'lchasin, xavf toifasi parametriga qarab yakuniy xulosa chiqarsin va 60 s ichida ishlasin.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.05, 0.1, 0.2
PGD	10 qadam
EPS_TRAIN	0.1, k = 3 (vaqt chegarasi uchun)
Sezgir atributlar	jins (A1, 0.4) va yosh 45+ (A2, 0.3)
Tarafkashlik	A1=1 ijobiylarining 20 % flip; A2=1 uchun 'oxirgi ish davomiyligi' belgisi siljigan
Adolatlilik mezoni	equalized odds, DP qo'shimcha
Chiqish	Markdown hisobot: model kartasi, metrikalar jadvali, 20 band, xulosa; xavf toifasi = minimal/yuqori; vaqt chegarasi 60 s

QADAMLAR

1. 1-4 qadam funksiyalarini bitta modulga yig'ing (train, fgsm, pgd, adv_train, saliency); toza/FGSM/PGD aniqligini 3 eps uchun, adversarial modelni (k=3) va saliency korrelyatsiyasini o'lchaydigan funksiya yozing; vaqtini o'lchang.
2. s5 generatorini ikki atributli HR ssenariysiga moslang; baza va ThresholdOptimizer(EO) uchun DP/EO farqlari va kesishgan guruhlar jadvali funksiya ko'rinishida.
3. s6 SHAP (GradientBoosting, HR ma'lumoti): top-5 belgilar va additivlik farqini qaytaruvchi funksiya.
4. s7 ro'yxatini kengaytiring: texnik bandlar (7, 8, 9, 11, 20) o'lchovlardan avtomatik, 2 ta yangi band qo'shing (kesishgan guruhlar tahlili; adversarial detektor mavjudligi) — jami 22 band, ball 100 ga normallashtirilsin.
5. Markdown generator: model kartasi (maqsad, ma'lumot, metrikalar, cheklovlar), metrikalar jadvali, tekshiruv ro'yxati jadvali, bajarilmagan bandlar, xulosa; xavf toifasi 'yuqori' bo'lsa 12, 14, 18-bandlar bajarilmaganda 'joriy etish mumkin emas'.
6. Skriptni ikki rejimda ishga tushiring (minimal, yuqori) va ikkala hisobotni saqlang; umumiy vaqt 60 s dan oshmasligini ko'rsating (vaqt jadvali: qaysi qadam qancha).
7. Xulosa: HR tizimi uchun qaysi bandlar texnik ish talab qiladi, 80+ ballga chiqish rejasi; hisobotning auditor uchun yetarliligi haqida tanqidiy baho.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadvallar va grafiklar — 25, Chuqur tahlil (murozalar, mos kelmaslik, cheklovlar) — 30, Hisobot, model kartasi va tekshiruv ro'yxati sifati — 20

19-variant. Mobil mikromoliya: telefon ma'lumotlari skoringi — vakil belgilarni SHAP bilan fosh qilish qiyin

Soha: Moliya (fintech)

Fintech ilovasi telefon foydalanish ma'lumotlaridan (qo'ng'iroqlar, ilovalar, joylashuv) mikroqarz berishni hal qiladi; sezgir atribut A (hudud, 0.4) belgilar ro'yxatida yo'q, lekin 6 ta belgidan 3 tasi A bilan turlicha kuchda bog'liq. Sizdan vakil belgilar kuchini o'lchash (korrelyatsiya, yordamchi model AUC), SHAP orqali A ta'siri qaysi belgi orqali o'tishini ko'rsatish, vakil belgilarni bosqichma-bosqich olib tashlab aniqlik-adolatlilik egri chizig'ini qurish va OCR moduliga L_2 hamda L_inf hujumlarni taqqoslash talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	L_inf: 0, 0.05, 0.1, 0.15; L_2: 0, 0.5, 1.0, 2.0
PGD	10 qadam (L_inf va L_2 versiyalari); EPS_TRAIN 0.1 (L_inf)
Sezgir atribut	hudud (A=1 ulushi 0.40), belgilarda yo'q
Vakil belgilar	tungi qo'ng'iroqlar (korr 0.6), ilovalar soni (korr 0.35), zaryad sikli (korr 0.15) + 3 ta neytral belgi
Tarafkashlik	A=1 ijobiylarining 20 % flip
Adolatlilik mezoni	demographic parity
SHAP	GradientBoosting, TreeExplainer, guruhlar bo'yicha o'rtacha hissa

QADAMLAR

1. s1/s2: L_inf FGSM/PGD jadvali va L_2 PGD (gradient normalashtirilgan, eps_2 radiusli sharga proyeksiya) jadvali; L_2 va L_inf buzilishlarining ko'rinishini panelda solishtiring.
2. s3 adversarial o'qitish (L_inf 0.1); L_2 hujumga chidamlilik; s4 saliency korrelyatsiyasi.
3. Generator: 6 belgi, uchtasi A bilan berilgan korrelyatsiyada (Cholesky yoki aralashtirish bilan); har belgining A bilan korrelyatsiyasi va A ni bashorat qiluvchi yordamchi model AUC jadvali.
4. s5: baza (A'siz, 6 belgi), ThresholdOptimizer(DP), ExponentiatedGradient(DP); so'ng vakil belgilarni kuchi bo'yicha birin-ketin olib tashlab (3 bosqich) aniqlik va DP farqi egri chizig'ini chizing.
5. s6 SHAP (GradientBoosting, 6 belgi): har belgi uchun o'rtacha SHAP hissasi A=0 va A=1 guruhlarida — farq eng katta belgi A ta'sirini o'tkazuvchi ekanini ko'rsating; additivlik; barqarorlik.
6. Taqqoslash: vakil belgini olib tashlash vs post-processing vs in-processing — aniqlik, DP farqi, inferensda A kerakmi, tushuntirish murakkabligi; jadval.
7. s7 ro'yxati (mikromoliya: yuqori xavf), 5-band (proxy aniqlangan) uchun o'z o'lchovlaringizni dalil qiling; ball va fintech uchun tavsiya.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli,

beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadvallar va grafiklar — 25, Chuqur tahlil (murozalar, mos kelmaslik, cheklovlar) — 30, Hisobot, model kartasi va tekshiruv ro'yxati sifati — 20

20-variant. Jinoyat sodir etish xavfini baholash: COMPAS uslubidagi sintetik tizim — kalibrlash vs xatolik tengligi qiyin

Soha: Huquqni muhofaza qilish

Sud-huquq tizimi uchun takroriy huquqbuzarlik xavfini baholovchi vositaning sintetik nusxasi tahlil qilinadi: sezgir atribut — guruh (A=1 ulushi 0.45), bazaviy ulushlar haqiqatan farq qiladi (0.45 vs 0.30) va tarixiy hibs yozuvlari A=1 da 1.4 baravar ko'p (nazorat tarafkashligi). Sizdan guruhlar bo'yicha kalibrlangan, lekin FPR/FNR tengsiz model (COMPAS bahsi) ni takrorlash, uchta mezonni birga qondirib bo'lmasligini isbotlash, qarorlar uchun tushuntirish berish va hujjat OCR'iga adversarial o'qitish murozasini EPS_TRAIN bo'yicha o'lchash talab qilinadi.

Rasm ma'lumoti	sklearn digits, 10 sinf
Epsilon to'ri	0, 0.05, 0.1, 0.15, 0.2 (PGD 10 qadam)
EPS_TRAIN	0.05, 0.1, 0.2 (uchta model)
Sezgir atribut	guruh (A=1 ulushi 0.45)
Bazaviy ulushlar	P(y=1|A=0)=0.30, P(y=1|A=1)=0.45
Tarafkashlik	A=1 da 'oldingi hibslar' belgisi 1.4x ko'paytirilgan (nazorat); flip 0
Mezonlar	kalibrlash (10 bin, Brier guruhlar bo'yicha), FPR farqi, FNR farqi, DP, EO
SHAP	GradientBoosting, lokal izohlar 3 shaxs uchun

QADAMLAR

1. s1/s2 FGSM va PGD-10 jadvali; s3 ni EPS_TRAIN = 0.05, 0.1, 0.2 uchun uch marta bajaring: toza aniqlik va PGD aniqligi 4 eps da — 3 x 4 jadval va grafik; radius oshganda muroza.
2. s4 saliency uchta adversarial model uchun: korrelyatsiya o'rtachasi EPS_TRAIN bilan qanday o'zgaradi.
3. Generator: bazaviy ulushlar 0.30/0.45, 'oldingi hibslar' belgisi A=1 da 1.4x; GradientBoosting va LogisticRegression o'qiting; guruhlar bo'yicha kalibrlash (10 bin) va Brier — model kalibrlanganini ko'rsating.
4. MetricFrame: FPR, FNR, selection_rate, TPR guruhlar bo'yicha — kalibrlangan modelda FPR A=1 da sezilarli yuqori ekanini (COMPAS effekti) ko'rsating; nazorat tarafkashligini olib tashlab (1.4x -> 1.0x) farq qancha qoladi.
5. ThresholdOptimizer(EO) va (DP) qo'llang va kalibrlash buzilishini o'lchang; 'kalibrlash + EO + DP' uchala shart bazaviy ulushlar farqlanganda mos kelmasligini jadval va grafikda ko'rsating (nazariy formula bilan solishtiring: FPR/FNR va bazaviy ulush bog'liqligi).
6. s6 SHAP: 3 shaxs (yuqori xavf A=1, yuqori xavf A=0, chegaradagi holat) uchun lokal izohlar; 'oldingi hibslar' hissasi; additivlik; izohni sudya uchun tushunarli tilda yozing.
7. s7 ro'yxati (huquqni muhofaza qilish: yuqori xavf, ba'zi qo'llanishlar taqiqlangan — AI Act 5-modda) : qaysi bandlar texnik jihatdan umuman qondirib bo'lmaydi; umumiy ball va 'joriy etish mumkin emas' asoslari.

Topshiriladi: Kod (.py fayllar, har qadam alohida); Natijalar jadvali (aniqlik vs epsilon, guruhlar kesimida metrikalar, SHAP top-belgilar); Grafik(lar): mustahkamlik egri chizig'i, saliency paneli, beeswarm; To'ldirilgan etik tekshiruv ro'yxati (20 band, dalillar bilan); Xulosa (>= 150 so'z): xavf, adolatlilik mezoni tanlovi va tavsiyalar

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha tekshiruvlar o'tadi) — 25, O'lchovlar, jadvallar va grafiklar — 25, Chuqur tahlil (murozalar, mos kelmaslik, cheklovlar) — 30, Hisobot, model kartasi va tekshiruv ro'yxati sifati — 20

70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A7

Yakuniy kompleks loyihani shakllantirish: ma'lumot -> model -> API -> monitoring -> adolatlilik

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A7
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A7
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

KTM laboratoriyalari — Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

Laboratoriya haqida

Bu virtual laboratoriyada siz tibbiy tasvir triaji loyihasining to'liq ML mahsulot ketma-ketligini mini-loyiha sifatida boshidan oxirigacha o'tasiz: sxema tekshiruvi va bemor bo'yicha sizishsiz bo'lish, model reestri va promote darvozasi, ONNX eksport, FastAPI xizmati metrikalari va audit jurnali, PSI/KS drift ogohlantirishi, fairlearn bilan kesimlar hisoboti va yakunda avtomatik tuziladigan loyiha pasporti. Real labdan farqi: sandbox'da rentgen tasvirlari, GPU va Docker yo'q — har tasvir o'rniga undan olingan 6 ta sintetik belgi, DenseNet o'rniga kichik MLP, uvicorn o'rniga TestClient ishlatiladi; lekin muhandislik qarorlari (sizish, chegara, darvoza, drift qoidasi, ko'lam cheklash) aynan haqiqiy loyihadagidek.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, fastapi, httpx. (pandera, mlflow, prometheus_client, docker YO'Q — ularning vazifasi qadamlarda oddiy kod bilan bajariladi.)

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A7`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A7 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Ma'lumot sxemasi validatsiyasi va bemor bo'yicha bo'lish
2. 2-qadam. Model o'qitish, eksperiment jurnali, model reestri va promote darvozasi
3. 3-qadam. PyTorch MLP -> ONNX eksport va onnxruntime bilan inferens tekshiruvi
4. 4-qadam. FastAPI xizmati: /predict, /healthz, Prometheus metrikalari va audit jurnali
5. 5-qadam. Monitoring: PSI/KS bilan drift aniqlash va ogohlantirish mantiqi
6. 6-qadam. Adolatlilik: fairlearn MetricFrame hisoboti va yumshatish
7. 7-qadam. Loyiha pasporti va model kartasi avtomatik generatsiyasi, rubrika bo'yicha o'z-o'zini baholash
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Ma'lumot sxemasi validatsiyasi va bemor bo'yicha bo'lish

MAQSAD

Sintetik «bemor» manifesti yaratiladi (har bemorda 1–3 tasvir, 6 ta tasvir belgisi, yosh, jins, topilma). Pandera uslubidagi sxema tekshiruvi toza ma'lumotda o'tadi, ataylab buzilgan partiyada esa har xatoni aniq nomlaydi. So'ng `GroupShuffleSplit` bilan BEMOR bo'yicha bo'linadi va sizish `assert` bilan tekshiriladi; taqqoslash uchun tasodifiy (tasvir bo'yicha) bo'lishda nechta bemor sizishi hisoblanadi. O'zgartirib ko'ring: `test_size`, buzilgan qiymatlar, `n_bemor`.

> **Kutilgan natija:** Sxema toza ma'lumotda O'TDI, buzilgan partiyada 6 ta xato satri; bemor bo'yicha bo'lishda sizish 0, tasodifiy bo'lishda ~35% bemor sizadi; split × sex va yosh guruhi × split jadvallari.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Ma'lumot sxemasi validatsiyasi va BEMOR bo'yicha bo'lish (sizishsiz)
# Sandbox'da haqiqiy rentgen tasvirlari yo'q: har tasvir o'rniga undan olingan 6 ta
# sonli belgi (opaklik, o'pka maydoni, kardiotorakal nisbat, tekstura, yorqinlik, qirra
zichligi)
# sintetik tarzda yaratiladi. Bitta bemorda 1–3 ta tasvir bo'ladi — xuddi NIH ChestX-ray14
dagidek.
import numpy as np, pandas as pd
from sklearn.model_selection import GroupShuffleSplit

FEATS = ["opacity", "lung_area", "ct_ratio", "texture", "intensity", "edge_density"]

def sintetik_bemorlar(n_bemor=1500, seed=0):
    """Bemor-darajali sintetik manifest: image_id, patient_id, age, sex, 6 belgi, finding."""
    rng = np.random.default_rng(seed)
    rows = []
    for pid in range(n_bemor):
        age = int(np.clip(rng.normal(50, 18), 1, 95))
        sex = "M" if rng.random() < 0.56 else "F"
        z = rng.normal(0, 1) + 0.02 * (age - 50)          # bemorning yashirin "patologiya"
omili
        for k in range(int(rng.integers(1, 4))):            # 1–3 ta tasvir
            shovqin = 2.2 if age < 18 else 1.0               # bolalar tasviri shovqinliroq
(kesim farqi uchun)
            e = rng.normal(0, shovqin, 6)
            f = [0.9*z+e[0], -0.6*z+e[1], 0.5*z+e[2], 0.7*z+e[3], 0.2*z+e[4], 0.4*z+e[5]]
            finding = int(z + rng.normal(0, 0.6) > 0.3)
            rows.append([f"{pid:08d}_{k:03d}.png", pid, age, sex, *np.round(f, 4), finding])
    return pd.DataFrame(rows, columns=["image_id", "patient_id", "age", "sex", *FEATS,
"finding"])

# --- Sxema: Pandera uslubidagi kichik tekshiruvchi (sandbox'da pandera yo'q) ---
SXEMA = {
    "image_id":   dict(dtype="str",   regex=r"^\d{8}_\d{3}\.png$", unique=True),
    "patient_id": dict(dtype="int",   min=0),
    "age":        dict(dtype="int",   min=0, max=100),
    "sex":        dict(dtype="str",   isin={"M", "F"}),
    "finding":    dict(dtype="int",   isin={0, 1}),
    **{f: dict(dtype="float", min=-15, max=15) for f in FEATS},
}
```

```
def validate(df, sxema=SXEMA):
    """Har ustun uchun qoidalarni tekshiradi; buzilishlar ro'yxatini qaytaradi (bo'sh =
to'g'ri)."""
    xatolar = []
    for col, q in sxema.items():
        if col not in df.columns:
            xatolar.append(f"{col}: ustun yo'q"); continue
        s = df[col]
        if s.isna().any():
            xatolar.append(f"{col}: {int(s.isna().sum())} ta bo'sh qiymat")
        if q["dtype"] in ("int", "float") and not pd.api.types.is_numeric_dtype(s):
            xatolar.append(f"{col}: son emas"); continue
        if "min" in q and (s < q["min"]).any():
            xatolar.append(f"{col}: {int((s < q['min']).sum())} ta qiymat < {q['min']}")
        if "max" in q and (s > q["max"]).any():
            xatolar.append(f"{col}: {int((s > q['max']).sum())} ta qiymat > {q['max']}")
        if "isin" in q and (~s.isin(q["isin"])).any():
            xatolar.append(f"{col}: ruxsatsiz qiymatlar
{sorted(set(s[~s.isin(q['isin'])].astype(str)))[:5]}")
        if "regex" in q and (~s.astype(str).str.match(q["regex"])).any():
            xatolar.append(f"{col}: {int((~s.astype(str).str.match(q['regex'])).sum())} ta nom
formati noto'g'ri")
        if q.get("unique") and s.duplicated().any():
            xatolar.append(f"{col}: {int(s.duplicated().sum())} ta takror")
    return xatolar

df = sintetik_bemorlar()
print("=== Manifest ===")
print(f"tasvirlar: {len(df)}, bemorlar: {df.patient_id.nunique()}, topilma ulushi:
{df.finding.mean():.3f}")
print(df.head(5).to_string(index=False))

print("\n=== Sxema tekshiruvi (toza ma'lumot) ===")
x = validate(df)
print("natija:", "O'TDI" if not x else x)

# Buzilgan «yangi partiya»: shifoxona arxividan kelgan ma'lumotda tipik xatolar
buzuq = df.sample(200, random_state=1).copy()
buzuq.loc[buzuq.index[:3], "age"] = [412, -1, 150]        # yosh xato kiritilgan
buzuq.loc[buzuq.index[3:5], "sex"] = ["U", "m"]            # noma'lum / kichik harf
buzuq.loc[buzuq.index[5], "image_id"] = "scan_final.jpg"   # nom formati
buzuq.loc[buzuq.index[6], "finding"] = 2                   # yorliq kodi
buzuq.loc[buzuq.index[7], "opacity"] = np.nan              # yo'qolgan belgi
print("\n=== Sxema tekshiruvi (buzilgan partiya) ===")
for xat in validate(buzuq):
    print(" -", xat)

# --- Bo'lish: BEMOR bo'yicha (GroupShuffleSplit), so'ng testning yarmi validatsiya ---
gss = GroupShuffleSplit(n_splits=1, test_size=0.3, random_state=42)
tr_idx, rest_idx = next(gss.split(df, groups=df["patient_id"]))
df["split"] = "train"
df.iloc[rest_idx, df.columns.get_loc("split")] = "test"
val_mask = (df["split"] == "test") & (df["patient_id"] % 2 == 0)
df.loc[val_mask, "split"] = "val"

# Sizish testi: bitta bemor ikki to'plamda bo'lmasligi shart
tr_p = set(df[df.split == "train"].patient_id)
assert tr_p.isdisjoint(df[df.split == "test"].patient_id) and tr_p.isdisjoint(df[df.split ==
"val"].patient_id)

# Taqqoslash uchun NOTO'G'RI (tasodifiy, tasvir bo'yicha) bo'lish — nechta bemor sizadi?
rng = np.random.default_rng(0)
tasodifiy = rng.random(len(df)) < 0.7
```

```
sizgan = len(set(df[tasodifiy].patient_id) & set(df[~tasodifiy].patient_id))

print("\n=== Bo'lish natijasi ===")
jad = df.groupby(["split", "sex"])["finding"].agg(count="count", mean="mean").round(3)
print(jad.to_string())
print("\nBemorlar soni:", df.groupby("split")["patient_id"].nunique().to_dict())
print(f"Bemor bo'yicha bo'lish: train∩test bemorlar = 0 (sizish yo'q)")
print(f"Tasodifiy (tasvir bo'yicha) bo'lishda sizadigan bemorlar: {sizgan} ta
({sizgan/df.patient_id.nunique():.0%})")

df["age_group"] = pd.cut(df.age, [0, 18, 40, 60, 100], labels=["0-18", "19-40", "41-60",
"61+"])
print("\n=== Vakillik jadvali: yosh guruhi × split (tasvirlar soni) ===")
print(pd.crosstab(df.age_group, df.split).to_string())

import os
os.makedirs("/tmp/triage/data", exist_ok=True)
df.to_csv("/tmp/triage/data/manifest.csv", index=False)
print("\nSaqlandi: /tmp/triage/data/manifest.csv", df.shape)
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Sxema tekshiruvi (buzilgan partiya) ===`, `=== Bo'lish natijasi ===`, `sizish yo'q`

TAJRIBA QILIB KO‘RING

- `SXEMA` ga yangi qoida qo'shing: `finding=1` bo'lgan tasvirlarda `opacity` manfiy bo'lmasin — necha qator buziladi?
- `test_size` ni 0.2 va 0.5 qilib, yosh guruhi × split jadvalidagi 0-18 guruhi hajmini kuzating.
- Bo'lishni `patient_id` o'rniga `image_id` bo'yicha qiling va `assert` nima deyishini ko'ring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Model o'qitish, eksperiment jurnali, model reestri va promote darvozasi

MAQSAD

Uchta eksperiment (logreg, 2 belgili zaif baseline, HistGradientBoosting) o'qitiladi, chegara validatsiyada sezgirlik >= 0.90 bo'ladigan qilib tanlanadi. Har model /tmp/triage/registry ichiga pickle + SHA-256 xesh bilan JSON reestrga «staging» sifatida yoziladi. `promote` darvozasi SLO (AUC >= 0.75, recall >= 0.90, jins bo'yicha TPR farqi <= 0.05) tekshiradi va joriy production'dan yomon bo'lsa almashtirmaydi. O'zgartirib ko'ring: SLO qiymatlari, yangi eksperiment, xeshni buzish.

> **Kutilgan natija:** Jadval: v1 logreg -> production, v2 -> rejected (auc < 0.75), v3 hgb -> staging (v1 yaxshiroq); reestr holati va butunlik tekshiruvi MOS.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Model o'qitish, eksperiment jurnali, oddiy MODEL REESTRI va «promote» darvozasi
# MLflow/Model Registry o'rniga JSON fayl: har versiya uchun metrika, artefakt xeshi va bosqich
(stage).
# Model reestrga "staging" sifatida kiradi; SLO darvozasidan o'tsa — "production" ga
ko'tariladi.
import json, os, hashlib, pickle, time
import numpy as np, pandas as pd
from sklearn.model_selection import GroupShuffleSplit
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import roc_auc_score, recall_score, precision_score, brier_score_loss

FEATS = ["opacity", "lung_area", "ct_ratio", "texture", "intensity", "edge_density"]

def sintetik_bemorlar(n_bemor=1500, seed=0):
    """1-qadamdagi generator: bemor-darajali sintetik manifest (har qadam mustaqil bo'lishi
uchun qayta yaratiladi)."""
    rng = np.random.default_rng(seed); rows = []
    for pid in range(n_bemor):
        age = int(np.clip(rng.normal(50, 18), 1, 95)); sex = "M" if rng.random() < 0.56 else
"F"
        z = rng.normal(0, 1) + 0.02 * (age - 50)
        for k in range(int(rng.integers(1, 4))):
            e = rng.normal(0, 2.2 if age < 18 else 1.0, 6)
            f = [0.9*z+e[0], -0.6*z+e[1], 0.5*z+e[2], 0.7*z+e[3], 0.2*z+e[4], 0.4*z+e[5]]
            rows.append([f"{pid:08d}_{k:03d}.png", pid, age, sex, *np.round(f, 4), int(z +
rng.normal(0, 0.6) > 0.3)])
    return pd.DataFrame(rows, columns=["image_id", "patient_id", "age", "sex", *FEATS,
"finding"])

def bolish(df):
    """Bemor bo'yicha train/val/test (1-qadam bilan bir xil)."""
    tr, rest = next(GroupShuffleSplit(1, test_size=0.3, random_state=42).split(df,
groups=df.patient_id))
    df = df.copy(); df["split"] = "train"; df.iloc[rest, df.columns.get_loc("split")] = "test"
    df.loc[(df.split == "test") & (df.patient_id % 2 == 0), "split"] = "val"
    return df
```

```
df = bolish(sintetik_bemorlar(3000))
tr, va, te = (df[df.split == s] for s in ("train", "val", "test"))
Xtr, ytr, Xva, yva, Xte, yte = tr[FEATS].values, tr.finding.values, va[FEATS].values,
va.finding.values, te[FEATS].values, te.finding.values

# --- SLO (loyiha pasportidan): sezgirlik >= 0.90, AUC >= 0.75, jins bo'yicha TPR farqi <= 0.05
---
SLO = {"recall_min": 0.90, "auc_min": 0.75, "tpr_gap_max": 0.05}

def chegara_tanla(p, y, recall_min):
    """Validatsiyada sezgirlik >= recall_min bo'ladigan ENG KATTA chegarani tanlaydi (aniqlikni
maksimal saqlaydi)."""
    for t in np.arange(0.95, 0.01, -0.01):
        if recall_score(y, p >= t) >= recall_min:
            return round(float(t), 2)
    return 0.01

def baholash(model, X, y, meta, t):
    p = model.predict_proba(X)[:, 1]; pred = (p >= t).astype(int)
    tpr = {s: recall_score(y[meta.sex.values == s], pred[meta.sex.values == s]) for s in ("M",
"F")}
    return dict(auc=round(roc_auc_score(y, p), 3), recall=round(recall_score(y, pred), 3),
                precision=round(precision_score(y, pred), 3), brier=round(brier_score_loss(y,
p), 3),
                tpr_gap_sex=round(abs(tpr["M"] - tpr["F"]), 3))

# --- Reestr: /tmp/triage/registry/registry.json ---
REG_DIR = "/tmp/triage/registry"; os.makedirs(REG_DIR, exist_ok=True)
REG = os.path.join(REG_DIR, "registry.json")
reestr = json.load(open(REG)) if os.path.exists(REG) else {"name": "cxr-triage", "versions":
[]}

def royxatga_ol(model, nom, params, t, m_val, m_test):
    """Artefaktni saqlaydi, SHA-256 xeshini hisoblaydi va reestrga 'staging' bosqichida
yozadi."""
    v = len(reestr["versions"]) + 1
    yol = os.path.join(REG_DIR, f"triage-v{v}.pkl")
    with open(yol, "wb") as f: pickle.dump(model, f)
    xesh = hashlib.sha256(open(yol, "rb").read()).hexdigest()[:16]
    yoz = dict(version=v, name=nom, params=params, threshold=t, val=m_val, test=m_test,
               artifact=yol, sha256=xesh, created=time.strftime("%Y-%m-%d %H:%M:%S"),
stage="staging")
    reestr["versions"].append(yoz); return yoz

def promote(yoz):
    """Darvoza: SLO shartlari VALIDATSIYA metrikalarida bajarilsa -> production, aks holda rad
etiladi."""
    m = yoz["val"]; sabab = []
    if m["recall"] < SLO["recall_min"]: sabab.append(f"recall {m['recall']} <
{SLO['recall_min']}")
    if m["auc"] < SLO["auc_min"]: sabab.append(f"auc {m['auc']} < {SLO['auc_min']}")
    if m["tpr_gap_sex"] > SLO["tpr_gap_max"]: sabab.append(f"TPR farqi {m['tpr_gap_sex']} >
{SLO['tpr_gap_max']}")
    if sabab:
        yoz["stage"] = "rejected"; yoz["reason"] = "; ".join(sabab); return False
    joriy = [v for v in reestr["versions"] if v["stage"] == "production"]
    if joriy and m["auc"] < joriy[-1]["val"]["auc"]:    # joriy production yaxshiroq bo'lsa —
almashtirilmaydi
        yoz["stage"] = "staging"; yoz["reason"] = f"v{joriy[-1]['version']} val AUC {joriy[-1]
['val']['auc']} yuqoriroq"; return False
    for old in joriy:                                   # faqat bitta production bo'ladi
        old["stage"] = "archived"
    yoz["stage"] = "production"; return True
```

```
# --- 3 ta eksperiment (rubrika: kamida 3 ta taqqoslash) ---
eksperimentlar = [
    ("logreg",        dict(C=1.0),                        make_pipeline(StandardScaler(),
LogisticRegression(C=1.0, max_iter=500))),
    ("logreg-2feat",  dict(C=1.0, feats=2),               None),   # ataylab zaif: faqat 2
belgi
    ("hgb",           dict(max_iter=150, lr=0.05, depth=3),
HistGradientBoostingClassifier(max_iter=150, learning_rate=0.05, max_depth=3, random_state=0)),
]
print("=== Eksperiment jurnali ===")
print(f"{'v':>2} {'model':<13} {'chegara':>7} {'valAUC':>7} {'valRec':>7} {'valPrec':>7}
{'TPRfarq':>7} {'testAUC':>7} {'stage':<10} sabab")
for nom, params, model in eksperimentlar:
    if model is None:   # zaif baseline: 2 ta belgi bilan
        from sklearn.compose import ColumnTransformer
        model = make_pipeline(ColumnTransformer([("sel", "passthrough", [4, 5])]),
StandardScaler(), LogisticRegression(max_iter=500))
    t0 = time.perf_counter(); model.fit(Xtr, ytr); fit_s = time.perf_counter() - t0
    p_val = model.predict_proba(Xva)[:, 1]
    t = chegara_tanla(p_val, yva, SLO["recall_min"])
    m_val, m_test = baholash(model, Xva, yva, va, t), baholash(model, Xte, yte, te, t)
    yoz = royxatga_ol(model, nom, {**params, "fit_s": round(fit_s, 2)}, t, m_val, m_test)
    ok = promote(yoz)
    print(f"{yoz['version']:>2} {nom:<13} {t:>7} {m_val['auc']:>7} {m_val['recall']:>7}
{m_val['precision']:>7} "
          f"{m_val['tpr_gap_sex']:>7} {m_test['auc']:>7} {yoz['stage']:<10} {yoz.get('reason',
'')}")

json.dump(reestr, open(REG, "w"), indent=1)
prod = [v for v in reestr["versions"] if v["stage"] == "production"]
print("\n=== Reestr holati ===")
for v in reestr["versions"]:
    print(f"v{v['version']} {v['name']:<13} stage={v['stage']:<10} sha256={v['sha256']}
chegara={v['threshold']}")
print("\nProduction versiya:", f"v{prod[-1]['version']} ({prod[-1]['name']})" if prod else
"YO'Q")
print("Reestr fayli:", REG, "| artefaktlar:", sorted(os.listdir(REG_DIR)))

# Artefakt butunligi: xesh mos kelmasa — yuklash rad etiladi (runbook'dagi tekshiruv)
if prod:
    p = prod[-1]; hozir = hashlib.sha256(open(p["artifact"], "rb").read()).hexdigest()[:16]
    print("Butunlik tekshiruvi:", "MOS" if hozir == p["sha256"] else "BUZILGAN")
    print(f"Production test metrikalari: {p['test']}")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Eksperiment jurnali ===`, `=== Reestr holati ===`, `Butunlik tekshiruvi`

TAJRIBA QILIB KO‘RING

- `SLO['recall_min']` ni 0.95 qiling — chegara va aniqlik qanday o'zgaradi, qaysi modellar darvozadan o'tadi?
- To'rtinchi eksperiment sifatida `RandomForestClassifier` qo'shing va reestrda nechanchi bo'lib production bo'lishini tekshiring.
- Saqlangan pickle faylga bir bayt qo'shib (`open(..., 'ab').write(b'x')`) butunlik tekshiruvi BUZILGAN deyishini ko'ring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. PyTorch MLP -> ONNX eksport va onnxruntime bilan inferens tekshiruvi

MAQSAD

Normalizatsiyasi grafga «tikilgan» kichik MLP 30 epoxa CPU'da o'qitiladi, dinamik batch bilan ONNX'ga eksport qilinadi, `onnx.checker` va PyTorch–ONNX parity (maks. farq < 1e-4) tekshiriladi, ikkala artefakt xeshi chiqariladi. Keyin PyTorch eager va ONNX Runtime kechikishi batch 1 va 64 uchun p50/p99 bilan taqqoslanadi. O'zgartirib ko'ring: qatlam o'lchamlari, `opset_version`, batch hajmi.

> **Kutilgan natija:** Test AUC ~0.84; parity farqi ~1e-7 (MOS); ONNX Runtime batch 1 p50 ~10 µs, PyTorch ~40 µs; triage.onnx ~4 KB.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. PyTorch MLP -> ONNX eksport -> onnxruntime bilan inferens tekshiruvi va kechikish
# Haqiqiy loyihada DenseNet-121 tasvirdan ishlaydi; sandbox'da 6 ta tasvir belgisi ustida
kichik MLP.
# Muhim: eksportdan keyin ONNX va PyTorch chiqishlari mos kelishi (parity) va artefakt xeshi
tekshiriladi.
import os, time, hashlib, json, warnings
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, torch, torch.nn as nn, onnx, onnxruntime as ort
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import roc_auc_score
torch.manual_seed(0); torch.set_num_threads(2)

FEATS = ["opacity", "lung_area", "ct_ratio", "texture", "intensity", "edge_density"]

def sintetik_bemorlar(n_bemor=1500, seed=0):
    """1-qadamdagi generator: bemor-darajali sintetik manifest."""
    rng = np.random.default_rng(seed); rows = []
    for pid in range(n_bemor):
        age = int(np.clip(rng.normal(50, 18), 1, 95)); sex = "M" if rng.random() < 0.56 else
"F"
        z = rng.normal(0, 1) + 0.02 * (age - 50)
        for k in range(int(rng.integers(1, 4))):
            e = rng.normal(0, 2.2 if age < 18 else 1.0, 6)
            f = [0.9*z+e[0], -0.6*z+e[1], 0.5*z+e[2], 0.7*z+e[3], 0.2*z+e[4], 0.4*z+e[5]]
            rows.append([f"{pid:08d}_{k:03d}.png", pid, age, sex, *np.round(f, 4), int(z +
rng.normal(0, 0.6) > 0.3)])
    return pd.DataFrame(rows, columns=["image_id", "patient_id", "age", "sex", *FEATS,
"finding"])

df = sintetik_bemorlar(3000)
tr_i, te_i = next(GroupShuffleSplit(1, test_size=0.3, random_state=42).split(df,
groups=df.patient_id))
tr, te = df.iloc[tr_i], df.iloc[te_i]
mu, sd = tr[FEATS].mean().values, tr[FEATS].std().values          # normalizatsiya — modelga
«tikiladi»

class TriageMLP(nn.Module):
    """Normalizatsiya + 2 qatlamli MLP. Chiqish — logit (sigmoid serverda yoki grafda)."""
    def __init__(self, mu, sd):
        super().__init__()
        self.register_buffer("mu", torch.tensor(mu, dtype=torch.float32))
```

```
        self.register_buffer("sd", torch.tensor(sd, dtype=torch.float32))
        self.net = nn.Sequential(nn.Linear(6, 32), nn.ReLU(), nn.Linear(32, 16), nn.ReLU(),
nn.Linear(16, 1))
    def forward(self, x):
        return self.net((x - self.mu) / self.sd)

model = TriageMLP(mu, sd)
Xtr = torch.tensor(tr[FEATS].values, dtype=torch.float32); ytr =
torch.tensor(tr.finding.values, dtype=torch.float32)
Xte = torch.tensor(te[FEATS].values, dtype=torch.float32); yte = te.finding.values
pos_w = torch.tensor([(1 - ytr.mean()) / ytr.mean()])
lossf = nn.BCEWithLogitsLoss(pos_weight=pos_w); opt = torch.optim.AdamW(model.parameters(),
lr=3e-3)
print("=== O'qitish (CPU, 30 epoxa, mini-batch 128) ===")
t0 = time.perf_counter()
for ep in range(30):
    model.train(); perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 128):
        b = perm[i:i+128]; opt.zero_grad(); loss = lossf(model(Xtr[b]).squeeze(1), ytr[b]);
loss.backward(); opt.step()
    if ep % 10 == 9:
        model.eval()
        with torch.no_grad(): auc = roc_auc_score(yte,
torch.sigmoid(model(Xte)).squeeze(1).numpy())
        print(f"epoxa {ep+1:2d}  loss={loss.item():.3f}  test AUC={auc:.3f}")
print(f"o'qitish vaqti: {time.perf_counter()-t0:.1f} s")
model.eval()

# --- ONNX eksport (dinamik batch) ---
os.makedirs("/tmp/triage/models", exist_ok=True)
ONNX = "/tmp/triage/models/triage.onnx"; PT = "/tmp/triage/models/triage.pt"
torch.save(model.state_dict(), PT)
dummy = torch.randn(1, 6)
torch.onnx.export(model, dummy, ONNX, input_names=["features"], output_names=["logit"],
                  dynamic_axes={"features": {0: "batch"}, "logit": {0: "batch"}},
opset_version=17, dynamo=False)
onnx.checker.check_model(onnx.load(ONNX))              # graf tuzilmasi to'g'riligi
sess = ort.InferenceSession(ONNX, providers=["CPUExecutionProvider"])

print("\n=== ONNX eksport tekshiruvi ===")
with torch.no_grad(): ref = model(Xte).numpy()
out = sess.run(None, {"features": Xte.numpy()})[0]
farq = float(np.abs(out - ref).max())
print(f"graf kirish: {[i.name for i in sess.get_inputs()]}, shakl: {sess.get_inputs()
[0].shape}")
print(f"operatorlar: {sorted({n.op_type for n in onnx.load(ONNX).graph.node})}")
print(f"PyTorch va ONNX chiqishlari orasidagi maks. farq: {farq:.2e}  -> {'MOS (parity OK)' if
farq < 1e-4 else 'NOMOS'}")
print(f"test AUC (PyTorch) = {roc_auc_score(yte, ref):.4f}   test AUC (ONNX) =
{roc_auc_score(yte, out):.4f}")
for yol in (PT, ONNX):
    print(f"{os.path.basename(yol):<12} {os.path.getsize(yol):>6} bayt  sha256=
{hashlib.sha256(open(yol,'rb').read()).hexdigest()[:16]}")

# --- Kechikish: PyTorch vs ONNX Runtime, batch 1 va 64 (p50/p99) ---
def olchov(fn, n=300):
    for _ in range(20): fn()                               # isitish (birinchi chaqiruvlar
sekin)
    t = []
    for _ in range(n):
        s = time.perf_counter(); fn(); t.append((time.perf_counter() - s) * 1e6)
    return np.percentile(t, 50), np.percentile(t, 99)
print("\n=== Inferens kechikishi (mikrosekund) ===")
```

```
print(f"{'variant':<22} {'batch':>5} {'p50':>9} {'p99':>9}")
for bs in (1, 64):
    xb = Xte[:bs]; xn = xb.numpy()
    with torch.no_grad():
        p50, p99 = olchov(lambda: model(xb))
    print(f"{'PyTorch eager':<22} {bs:>5} {p50:>9.1f} {p99:>9.1f}")
    p50, p99 = olchov(lambda: sess.run(None, {"features": xn}))
    print(f"{'ONNX Runtime CPU':<22} {bs:>5} {p50:>9.1f} {p99:>9.1f}")

# Model metadata — serverga va model kartasiga kerak
meta = {"model_version": "triage-mlp-1.0", "onnx": ONNX, "input": "features[batch,6]", "feats":
FEATS,
        "test_auc": round(float(roc_auc_score(yte, out)), 4), "parity_max_diff": farq}
json.dump(meta, open("/tmp/triage/models/meta.json", "w"), indent=1)
print("\nSaqlandi:", sorted(os.listdir("/tmp/triage/models")))
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== ONNX eksport tekshiruvi ===`, `parity OK`, `=== Inferens kechikishi (mikrosekund) ===`

TAJRIBA QILIB KO‘RING

- MLP'ni 6-128-128-1 qiling: AUC o'zgaradimi, ONNX fayl hajmi va p50 qanchaga oshadi?
- `sess_opts = ort.SessionOptions(); sess_opts.intra_op_num_threads = 1` bilan sessiya yarating va 2 oqim bilan solishtiring.
- Eksportda `dynamic_axes` ni olib tashlang va batch 64 bilan `sess.run` chaqirganda qanday xato chiqishini ko'ring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. FastAPI xizmati: /predict, /healthz, Prometheus metrikalari va audit jurnali

MAQSAD

Xizmat kodi haqiqiy loyihadagidek yoziladi: API kalit, pydantic kontrakt, kirish nazorati, ONNX Runtime inferens, Prometheus matn formatidagi Counter/Histogram (o'z amalga oshirishimiz), audit jurnali (kirishning xeshi, jins/yosh sarlavhadan). TestClient bilan 300 so'rov yuboriladi — to'liq va server ichidagi kechikish p50/p99 taqqoslanadi, /metrics chiqishi va jurnal o'qiladi. O'zgartirib ko'ring: chegara, gistogramma chelaklari, noto'g'ri so'rov turlari.

> **Kutilgan natija:** /healthz ok; 401/400/422 holatlar; 300 so'rov p50 ~4 ms (TestClient) vs ~0.1 ms (server ichida); /metrics da counter va 2 histogram; jurnalda 300 yozuv.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. FastAPI inferens xizmati: /predict, /healthz, /metrics (Prometheus matn formati),
audit jurnali
# Docker/uvicorn o'rniga TestClient — xizmat kodi aynan shunday, faqat tarmoq o'rniga jarayon
ichida chaqiriladi.
import os, time, json, uuid, hashlib, warnings
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as ort
from fastapi import FastAPI, Header, HTTPException, Response
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field
torch.manual_seed(0)

FEATS = ["opacity", "lung_area", "ct_ratio", "texture", "intensity", "edge_density"]

def sintetik_bemorlar(n_bemor=1500, seed=0):
    """1-qadamdagi generator: bemor-darajali sintetik manifest."""
    rng = np.random.default_rng(seed); rows = []
    for pid in range(n_bemor):
        age = int(np.clip(rng.normal(50, 18), 1, 95)); sex = "M" if rng.random() < 0.56 else
"F"
        z = rng.normal(0, 1) + 0.02 * (age - 50)
        for k in range(int(rng.integers(1, 4))):
            e = rng.normal(0, 2.2 if age < 18 else 1.0, 6)
            f = [0.9*z+e[0], -0.6*z+e[1], 0.5*z+e[2], 0.7*z+e[3], 0.2*z+e[4], 0.4*z+e[5]]
            rows.append([f"{pid:08d}_{k:03d}.png", pid, age, sex, *np.round(f, 4), int(z +
rng.normal(0, 0.6) > 0.3)])
    return pd.DataFrame(rows, columns=["image_id", "patient_id", "age", "sex", *FEATS,
"finding"])

# --- Model artefakti (3-qadam qisqacha): tez o'qitiladigan MLP -> ONNX ---
df = sintetik_bemorlar(1500)
X = torch.tensor(df[FEATS].values, dtype=torch.float32); y = torch.tensor(df.finding.values,
dtype=torch.float32)
mu, sd = X.mean(0), X.std(0)
net = nn.Sequential(nn.Linear(6, 32), nn.ReLU(), nn.Linear(32, 1))
opt = torch.optim.Adam(net.parameters(), lr=5e-3); lossf = nn.BCEWithLogitsLoss()
for ep in range(60):
    opt.zero_grad(); loss = lossf(net((X - mu) / sd).squeeze(1), y); loss.backward();
opt.step()
class Wrap(nn.Module):
```

```
    def __init__(s): super().__init__(); s.net = net; s.register_buffer("mu", mu);
s.register_buffer("sd", sd)
    def forward(s, x): return torch.sigmoid(s.net((x - s.mu) / s.sd))      # grafning o'zi
ehtimol qaytaradi
os.makedirs("/tmp/triage/models", exist_ok=True); os.makedirs("/tmp/triage/logs",
exist_ok=True)
ONNX = "/tmp/triage/models/triage.onnx"
torch.onnx.export(Wrap().eval(), torch.randn(1, 6), ONNX, input_names=["features"],
output_names=["prob"],
                  dynamic_axes={"features": {0: "b"}, "prob": {0: "b"}}, opset_version=17,
dynamo=False)

# --- Prometheus uslubidagi metrikalar (prometheus_client o'rniga minimal amalga oshirish) ---
class Counter:
    def __init__(s, name, help_, labels): s.name, s.help, s.labels_, s.v = name, help_, labels,
{}
    def inc(s, **lab): k = tuple(lab[l] for l in s.labels_); s.v[k] = s.v.get(k, 0) + 1
    def expose(s):
        out = [f"# HELP {s.name} {s.help}", f"# TYPE {s.name} counter"]
        for k, n in sorted(s.v.items()):
            out.append(f"{s.name}{{{','.join(f'{l}=\"{x}\"' for l, x in zip(s.labels_, k))}}}
{n}")
        return out
class Histogram:
    def __init__(s, name, help_, buckets): s.name, s.help, s.b, s.cnt, s.sum, s.vals = name,
help_, buckets, [0]*len(buckets), 0.0, []
    def observe(s, x):
        s.sum += x; s.vals.append(x)
        for i, ub in enumerate(s.b):
            if x <= ub: s.cnt[i] += 1
    def expose(s):
        out = [f"# HELP {s.name} {s.help}", f"# TYPE {s.name} histogram"]
        out += [f'{s.name}_bucket{{le="{ub}"}} {c}' for ub, c in zip(s.b, s.cnt)]
        out += [f'{s.name}_bucket{{le="+Inf"}} {len(s.vals)}', f"{s.name}_sum {s.sum:.6f}", f"
{s.name}_count {len(s.vals)}"]
        return out

REQ = Counter("triage_requests_total", "so'rovlar soni holat bo'yicha", ["status"])
LAT = Histogram("triage_latency_seconds", "to'liq so'rov kechikishi", [.00005, .0001, .0002,
.0005, .001, .002, .005, .01])
SCORE = Histogram("triage_score", "bashorat ehtimoli taqsimoti", [i / 10 for i in range(1,
11)])

# --- Xizmat ---
MODEL_VERSION, THRESHOLD, API_KEY = "triage-mlp-1.0", 0.30, os.environ.get("TRIAGE_API_KEY",
"demo-key")
AUDIT = "/tmp/triage/logs/predictions.jsonl"
app = FastAPI(title="CXR Triage API", version=MODEL_VERSION)
sess = ort.InferenceSession(ONNX, providers=["CPUExecutionProvider"])

class Kirish(BaseModel):
    """API kontrakti: 6 ta belgi (haqiqiy tizimda — tasvir fayli); qiymatlar chegaralangan."""
    features: list[float] = Field(min_length=6, max_length=6)

@app.post("/predict")
def predict(k: Kirish, x_api_key: str = Header(None), x_patient_sex: str = Header(None),
x_patient_age: int = Header(None)):
    if x_api_key != API_KEY:
        REQ.inc(status="unauthorized"); raise HTTPException(401, "API kalit noto'g'ri")
    t0 = time.perf_counter()
    x = np.asarray(k.features, dtype=np.float32)[None]
    if not np.isfinite(x).all() or np.abs(x).max() > 15:            # kirish nazorati (sxema
bilan bir xil chegara)
```

```
        REQ.inc(status="bad_input"); raise HTTPException(400, "belgilar diapazondan
tashqarida")
    p = float(sess.run(None, {"features": x})[0][0, 0]); flag = p >= THRESHOLD
    dt = time.perf_counter() - t0
    LAT.observe(dt); SCORE.observe(p); REQ.inc(status="ok")
    rec = {"id": str(uuid.uuid4())[:8], "ts": round(time.time(), 3), "model": MODEL_VERSION,
"score": round(p, 4),
           "flag": bool(flag), "latency_ms": round(dt * 1000, 2), "sex": x_patient_sex, "age":
x_patient_age,
           "input_sha": hashlib.sha256(x.tobytes()).hexdigest()[:16]}   # kirishning o'zi emas,
xeshi saqlanadi
    with open(AUDIT, "a") as f: f.write(json.dumps(rec) + "\n")
    return {"id": rec["id"], "score": rec["score"], "priority": "HIGH" if flag else "ROUTINE",
            "model_version": MODEL_VERSION, "disclaimer": "Faqat navbatni tartiblash uchun;
tashxis emas."}

@app.get("/healthz")
def healthz():
    return {"status": "ok", "model": MODEL_VERSION, "onnx_loaded": sess is not None,
"threshold": THRESHOLD}

@app.get("/metrics")
def metrics():
    return Response("\n".join(REQ.expose() + LAT.expose() + SCORE.expose()) + "\n",
media_type="text/plain; version=0.0.4")

# --- Sinov: TestClient bilan 300 so'rov (Locust yuk sinovining jarayon ichidagi muqobili) ---
c = TestClient(app)
print("=== /healthz ===", c.get("/healthz").json())
print("=== /predict namunasi ===")
r = c.post("/predict", json={"features": df[FEATS].iloc[0].tolist()},
           headers={"X-API-Key": "demo-key", "X-Patient-Sex": "F", "X-Patient-Age": "54"})
print(r.status_code, r.json())
print("kalitsiz so'rov ->", c.post("/predict", json={"features": [0]*6}).status_code,
      "| noto'g'ri kirish ->", c.post("/predict", json={"features": [99]*6}, headers={"X-API-
Key": "demo-key"}).status_code,
      "| 5 ta belgi ->", c.post("/predict", json={"features": [0]*5}, headers={"X-API-Key":
"demo-key"}).status_code)

if os.path.exists(AUDIT): os.remove(AUDIT)
REQ.v.clear(); LAT.__init__(LAT.name, LAT.help, LAT.b); SCORE.__init__(SCORE.name, SCORE.help,
SCORE.b)
sample = df.sample(300, random_state=1)
t_all = []
for _, row in sample.iterrows():
    s = time.perf_counter()
    c.post("/predict", json={"features": row[FEATS].tolist()},
           headers={"X-API-Key": "demo-key", "X-Patient-Sex": row.sex, "X-Patient-Age":
str(row.age)})
    t_all.append((time.perf_counter() - s) * 1000)
c.post("/predict", json={"features": [0]*6})          # bitta ruxsatsiz — metrikada ko'rinsin
print(f"\n=== Yuk sinovi: 300 so'rov ===")
print(f"to'liq (TestClient) kechikish: p50={np.percentile(t_all,50):.2f} ms  p99=
{np.percentile(t_all,99):.2f} ms  "
      f"o'tkazuvchanlik ~{1000/np.mean(t_all):.0f} so'rov/s (bitta oqim)")
print(f"server ichidagi kechikish:     p50={np.percentile(LAT.vals,50)*1000:.3f} ms  p99=
{np.percentile(LAT.vals,99)*1000:.3f} ms")
print("-> tizim kechikishi >> model kechikishi (HTTP/JSON qayta ishlash ulushi katta)")

print("\n=== /metrics (Prometheus matn formati) ===")
print(c.get("/metrics").text.strip())

log = pd.read_json(AUDIT, lines=True)
```

```
print("\n=== Audit jurnali ===")
print(f"fayl: {AUDIT}, yozuvlar: {len(log)}, ustunlar: {list(log.columns)}")
print(log[["id", "model", "score", "flag", "latency_ms", "sex", "age",
"input_sha"]].head(3).to_string(index=False))
print("HIGH ulushi:", round(log.flag.mean(), 3), "| jins bo'yicha HIGH ulushi:",
log.groupby("sex").flag.mean().round(3).to_dict())
print("\nOpenAPI yo'llari:", sorted(p for p in c.get("/openapi.json").json()["paths"]))
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Yuk sinovi: 300 so'rov ===`, `triage_requests_total{status="ok"} 300`, `=== Audit jurnali ===`

TAJRIBA QILIB KO‘RING

- `/predict` ga `X-Patient-Age` < 19 bo'lsa 422 va «ko'lamdan tashqari» xabari qaytaradigan qoida qo'shing; metrikada `status="out_of_scope"` ko'rinsin.
- `/metrics` dan p99 ni gistogramma chelaklaridan (Prometheus `histogram_quantile` mantig'i bilan) qo'lda hisoblang va `LAT.vals` dagi haqiqiy p99 bilan solishtiring.
- Jurnalga `patient_id` ham yoza boshlang — bu shaxsiy ma'lumotni minimallashtirish tamoyilini qanday buzadi? Muqobil: tuzli xesh.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Monitoring: PSI/KS bilan drift aniqlash va ogohlantirish mantiqi

MAQSAD

Validatsiya to'plami mos yozuv bo'ladi. 8 kunlik ishlab chiqarish oqimi simulyatsiya qilinadi: 4-kundan «yangi apparat» (opaklik kontrasti −30% va siljish, yorqinlik oshishi) kuchayib boradi. Har kun uchun belgilar va model ballari PSI, KS-test p-qiymati, HIGH ulushi hisoblanib, darajali qoida (OK/WARN/ALERT) ishlaydi; sezgirlik yorliq bilan qayta hisoblanib, driftning haqiqiy zarari ko'rsatiladi. O'zgartirib ko'ring: siljish kuchi, qoidalar chegarasi, chelaklar soni.

> **Kutilgan natija:** 1–4 kun OK, 5-kundan ALERT (intensity/opacity PSI > 0.3), 8-kunda ball PSI ~0.23, KS p ~1e-14, sezgirlik 0.86 -> 0.68; kun × belgi PSI jadvali va grafik.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Monitoring: PSI va KS-test bilan drift aniqlash, ogohlantirish mantiqi
# Mos yozuv (reference) — validatsiya to'plamidagi belgilar va model ballari.
# Simulyatsiya: shifoxona yangi rentgen apparati o'rnatdi -> kontrast pasaydi, yorqinlik
siljidi
# (bizda: 'opacity' va 'intensity' belgilari siljiydi). Drift kun sayin kuchayadi; har kun —
monitoring oynasi.
import numpy as np, pandas as pd, json, warnings
warnings.filterwarnings("ignore")
from scipy.stats import ks_2samp
from sklearn.model_selection import GroupShuffleSplit
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import roc_auc_score, recall_score
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

FEATS = ["opacity", "lung_area", "ct_ratio", "texture", "intensity", "edge_density"]

def sintetik_bemorlar(n_bemor=1500, seed=0):
    """1-qadamdagi generator: bemor-darajali sintetik manifest."""
    rng = np.random.default_rng(seed); rows = []
    for pid in range(n_bemor):
        age = int(np.clip(rng.normal(50, 18), 1, 95)); sex = "M" if rng.random() < 0.56 else
"F"
        z = rng.normal(0, 1) + 0.02 * (age - 50)
        for k in range(int(rng.integers(1, 4))):
            e = rng.normal(0, 2.2 if age < 18 else 1.0, 6)
            f = [0.9*z+e[0], -0.6*z+e[1], 0.5*z+e[2], 0.7*z+e[3], 0.2*z+e[4], 0.4*z+e[5]]
            rows.append([f"{pid:08d}_{k:03d}.png", pid, age, sex, *np.round(f, 4), int(z +
rng.normal(0, 0.6) > 0.3)])
    return pd.DataFrame(rows, columns=["image_id", "patient_id", "age", "sex", *FEATS,
"finding"])

def psi(ref, cur, bins=10):
    """Population Stability Index: mos yozuv kvantillari bo'yicha chelaklar; <0.1 barqaror,
0.1–0.2 kuzatuv, >=0.2 drift."""
    e = np.quantile(ref, np.linspace(0, 1, bins + 1)); e[0], e[-1] = -np.inf, np.inf
    r = np.histogram(ref, e)[0] / len(ref) + 1e-6; c = np.histogram(cur, e)[0] / len(cur) + 1e-
6
    return float(np.sum((c - r) * np.log(c / r)))
```

```
# --- Model va mos yozuv ---
df = sintetik_bemorlar(3000)
tr_i, rest = next(GroupShuffleSplit(1, test_size=0.3, random_state=42).split(df,
groups=df.patient_id))
tr, ref = df.iloc[tr_i], df.iloc[rest]
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=500)).fit(tr[FEATS],
tr.finding)
ref_scores = model.predict_proba(ref[FEATS])[:, 1]
THRESHOLD = 0.30
print(f"Mos yozuv: {len(ref)} tasvir, o'rtacha ball {ref_scores.mean():.3f}, HIGH ulushi
{(ref_scores>=THRESHOLD).mean():.3f}, AUC {roc_auc_score(ref.finding, ref_scores):.3f}")

# --- 8 kunlik ishlab chiqarish oqimi: 4-kundan yangi apparat (kontrast −30%, yorqinlik
siljishi) ---
QOIDA = {"psi_warn": 0.10, "psi_alert": 0.20, "ks_p": 0.01, "pos_rate_rel": 0.25}   #
ogohlantirish qoidalari

def yangi_apparat(df, kuch):
    """Boshqa apparat taqlidi: opaklik kontrasti −30% va siljish, yorqinlik oshadi, tekstura
silliqlashadi (kuch 0..1)."""
    cur = df.copy()
    cur["opacity"] = cur["opacity"] * (1 - 0.30 * kuch) - 0.9 * kuch
    cur["intensity"] = cur["intensity"] + 0.9 * kuch
    cur["texture"] = cur["texture"] - 0.5 * kuch
    return cur

jurnal, psi_jadval = [], []
pos_ref = float((ref_scores >= THRESHOLD).mean())
print("\n=== Kunlik monitoring oynalari ===")
print(f"{'kun':>3} {'n':>4} {'PSI(ball)':>9} {'PSI maks belgi':>22} {'KS p(ball)':>10} {'HIGH
ulushi':>11} {'sezg.*':>6}  holat")
for kun in range(1, 9):
    kuch = 0.0 if kun <= 3 else min(1.0, (kun - 3) / 3)             # 4-kundan boshlab
kuchayadigan siljish
    cur = yangi_apparat(sintetik_bemorlar(250, seed=100 + kun), kuch)
    sc = model.predict_proba(cur[FEATS])[:, 1]
    psi_f = {f: round(psi(ref[f].values, cur[f].values), 3) for f in FEATS};
psi_jadval.append({"kun": kun, **psi_f})
    fmax = max(psi_f, key=psi_f.get)
    psi_s = psi(ref_scores, sc); ks_p = ks_2samp(ref_scores, sc).pvalue
    pos = float((sc >= THRESHOLD).mean())
    rec = recall_score(cur.finding, sc >= THRESHOLD)                  # * haqiqiy yorliq odatda
kechikib keladi; bu yerda «oltin» yorliq bor deb faraz
    # Ogohlantirish mantiqi: darajali (OK / WARN / ALERT) + sabab
    sabab = []
    if psi_s >= QOIDA["psi_alert"] or psi_f[fmax] >= QOIDA["psi_alert"]: holat = "ALERT";
sabab.append("PSI>=0.2")
    elif psi_s >= QOIDA["psi_warn"] or psi_f[fmax] >= QOIDA["psi_warn"]: holat = "WARN";
sabab.append("PSI>=0.1")
    else: holat = "OK"
    if ks_p < QOIDA["ks_p"]: sabab.append("KS p<0.01"); holat = "ALERT" if holat != "OK" else
"WARN"
    if abs(pos - pos_ref) / pos_ref > QOIDA["pos_rate_rel"]: sabab.append("HIGH ulushi ±25%");
holat = "ALERT" if holat != "OK" else "WARN"
    jurnal.append(dict(kun=kun, n=len(cur), psi_score=round(psi_s, 3), psi_feat_max=fmax,
psi_feat=psi_f[fmax],
                       ks_p=float(ks_p), pos_rate=round(pos, 3), recall=round(rec, 3),
status=holat, reasons=sabab))
    print(f"{kun:>3} {len(cur):>4} {psi_s:>9.3f} {fmax:>14} {psi_f[fmax]:>7.3f} {ks_p:>10.1e}
{pos:>11.3f} {rec:>6.3f}  {holat} {' '.join(sabab)}")

birinchi = next((j for j in jurnal if j["status"] == "ALERT"), None)
```

```
print("\n=== Ogohlantirish xulosasi ===")
print("Qoidalar:", QOIDA)
print("Birinchi ALERT:", f"{birinchi['kun']}-kun ({', '.join(birinchi['reasons'])})" if
birinchi else "yo'q")
print("Eng ko'p siljigan belgi:", pd.Series([j["psi_feat_max"] for j in jurnal[3:]]).mode()[0],
      "-> runbook: apparat sozlamalarini tekshirish, mos yozuvni yangilash yoki qayta
o'qitish")
print(f"Sezgirlik: 1-kun {jurnal[0]['recall']} -> 8-kun {jurnal[-1]['recall']} (SLO 0.90).
Yorliq kechikib keladi — PSI buni yorliqsiz, oldinroq ko'rsatdi.")

print("\n=== PSI belgilar bo'yicha (kun × belgi) ===")
print(pd.DataFrame(psi_jadval).set_index("kun").to_string())

# Grafik: ball PSI va HIGH ulushi dinamikasi
fig, ax = plt.subplots(1, 2, figsize=(9, 3.2))
ax[0].plot([j["kun"] for j in jurnal], [j["psi_score"] for j in jurnal], "o-");
ax[0].axhline(0.2, color="r", ls="--", label="ALERT 0.2"); ax[0].axhline(0.1, color="orange",
ls="--", label="WARN 0.1")
ax[0].set_title("PSI (model balli)"); ax[0].set_xlabel("kun"); ax[0].legend()
ax[1].plot([j["kun"] for j in jurnal], [j["pos_rate"] for j in jurnal], "s-", color="purple");
ax[1].axhline(pos_ref, color="gray", ls=":", label="mos yozuv")
ax[1].set_title("HIGH ulushi"); ax[1].set_xlabel("kun"); ax[1].legend(); plt.tight_layout()
json.dump(jurnal, open("/tmp/drift_log.json", "w"), indent=1)
print("\nDrift jurnali saqlandi: /tmp/drift_log.json")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Kunlik monitoring oynalari ===`, `=== Ogohlantirish xulosasi ===`, `Birinchi ALERT:`

TAJRIBA QILIB KO‘RING

- Siljishni faqat `edge_density` ga bering (model unga kam bog'liq) — belgi PSI ALERT beradi, lekin sezgirlik tushmaydi: bu «yolg'on ogohlantirish»mi? Qoidani qanday to'g'rilaysiz?
- Mos yozuvni har 3 kunda yangilab turadigan «sirpanuvchi» rejim qiling va sekin driftni u sezmay qolishini ko'rsating.
- `psi` da `bins=20` va `bins=5` bilan 8-kun qiymatlarini solishtiring; oyna hajmi `n=60` bo'lsa PSI qanchalik shovqinli bo'ladi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Adolatlilik: fairlearn MetricFrame hisoboti va yumshatish

MAQSAD

Test to'plamida jins va yosh guruhlari bo'yicha n, TPR, FPR, tanlov ulushi va AUC `MetricFrame` bilan chiqariladi, TPR farqi SLO (<= 0.05) bilan solishtiriladi, bootstrap bilan 95% ishonch oralig'i hisoblanadi. Ikki yumshatish yo'li taqqoslanadi: (A) `ThresholdOptimizer` guruhga xos chegaralar, (B) ko'lamni 19+ bilan cheklash va chegarani qayta tanlash. O'zgartirib ko'ring: cheklov turi, chegara, bolalar shovqin koeffitsienti generator ichida.

> **Kutilgan natija:** Jins bo'yicha farq ~0.01 (OK), yosh bo'yicha ~0.18 (BUZILDI, 0-18 TPR ~0.70, keng ishonch oralig'i); A: farq ~0.02, lekin sezgirlik 0.77; B: farq ~0.03, sezgirlik 0.90.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Adolatlilik: fairlearn MetricFrame bilan kesimlar hisoboti, bootstrap ishonch
oralig'i va yumshatish
# Ikki yo'l taqqoslanadi: (A) guruhga xos chegara (ThresholdOptimizer, post-processing) va
# (B) ko'lamni cheklash (0–18 yosh guruhini tizimdan chiqarish, model kartasida
«mo'ljallanmagan foydalanish»).
import numpy as np, pandas as pd, warnings
warnings.filterwarnings("ignore")
from sklearn.model_selection import GroupShuffleSplit
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import roc_auc_score, recall_score, precision_score
from fairlearn.metrics import MetricFrame, true_positive_rate, false_positive_rate,
selection_rate, count
from fairlearn.postprocessing import ThresholdOptimizer

FEATS = ["opacity", "lung_area", "ct_ratio", "texture", "intensity", "edge_density"]

def sintetik_bemorlar(n_bemor=1500, seed=0):
    """1-qadamdagi generator: bemor-darajali sintetik manifest (bolalar tasviri
shovqinliroq)."""
    rng = np.random.default_rng(seed); rows = []
    for pid in range(n_bemor):
        age = int(np.clip(rng.normal(50, 18), 1, 95)); sex = "M" if rng.random() < 0.56 else
"F"
        z = rng.normal(0, 1) + 0.02 * (age - 50)
        for k in range(int(rng.integers(1, 4))):
            e = rng.normal(0, 2.2 if age < 18 else 1.0, 6)
            f = [0.9*z+e[0], -0.6*z+e[1], 0.5*z+e[2], 0.7*z+e[3], 0.2*z+e[4], 0.4*z+e[5]]
            rows.append([f"{pid:08d}_{k:03d}.png", pid, age, sex, *np.round(f, 4), int(z +
rng.normal(0, 0.6) > 0.3)])
    return pd.DataFrame(rows, columns=["image_id", "patient_id", "age", "sex", *FEATS,
"finding"])

df = sintetik_bemorlar(4000)
df["age_group"] = pd.cut(df.age, [0, 18, 40, 60, 100], labels=["0-18", "19-40", "41-60",
"61+"]).astype(str)
tr_i, te_i = next(GroupShuffleSplit(1, test_size=0.35, random_state=42).split(df,
groups=df.patient_id))
tr, te = df.iloc[tr_i].reset_index(drop=True), df.iloc[te_i].reset_index(drop=True)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=500)).fit(tr[FEATS],
tr.finding)
```

```
te = te.assign(score=model.predict_proba(te[FEATS])[:, 1])
THRESHOLD = 0.30; te["pred"] = (te.score >= THRESHOLD).astype(int)
SLO_GAP = 0.05

def hisobot(y, pred, score, sens, nom):
    """MetricFrame: kesimlar bo'yicha n, TPR, FPR, tanlov ulushi, AUC; TPR farqi (max-min)."""
    mf = MetricFrame(metrics={"n": count, "tpr": true_positive_rate, "fpr":
false_positive_rate, "sel": selection_rate},
                     y_true=y, y_pred=pred, sensitive_features=sens)
    auc = pd.Series({g: roc_auc_score(y[sens == g], score[sens == g]) for g in
np.unique(sens)}, name="auc")
    rep = mf.by_group.join(auc).round(3)
    gap = float(mf.difference()["tpr"])
    print(f"\n-- {nom}: kesim = {sens.name}")
    print(rep.to_string())
    print(f"TPR farqi (max-min) = {gap:.3f}  SLO <= {SLO_GAP}: {'BAJARILDI' if gap <= SLO_GAP
else 'BUZILDI'}")
    return gap

def bootstrap_tpr(y, pred, sens, n=300, seed=0):
    """Har guruh uchun TPR 95% ishonch oralig'i (bemor emas, tasvir bo'yicha oddiy
bootstrap)."""
    rng = np.random.default_rng(seed); out = {}
    for g in np.unique(sens):
        m = (sens == g) & (y == 1); yy, pp = y[m], pred[m]; vals = []
        for _ in range(n):
            i = rng.integers(0, len(yy), len(yy)); vals.append(pp[i].mean())
        out[g] = (round(pp.mean(), 3), round(float(np.percentile(vals, 2.5)), 3),
round(float(np.percentile(vals, 97.5)), 3), int(m.sum()))
    return out

y, s, p = te.finding.values, te.score.values, te.pred.values
print("=== Dastlabki model (chegara 0.30) ===")
print(f"test: n={len(te)}, AUC={roc_auc_score(y, s):.3f}, sezgirlik={recall_score(y, p):.3f},
aniqlik={precision_score(y, p):.3f}")
gap_sex = hisobot(y, p, s, te.sex, "Dastlabki")
gap_age = hisobot(y, p, s, te.age_group, "Dastlabki")
print("\nTPR 95% ishonch oralig'i (yosh guruhi): ", {g: f"{v[0]} [{v[1]}; {v[2]}] n+={v[3]}"
for g, v in bootstrap_tpr(y, p, te.age_group.values).items()})
print("-> 0-18 guruhi: namunalar kam, ishonch oralig'i keng, TPR past — bu guruh uchun model
ishonchsiz")

# --- Yumshatish A: ThresholdOptimizer (yosh guruhiga xos chegara, TPR tenglik cheklovi) ---
print("\n=== Yumshatish A: guruhga xos chegara (ThresholdOptimizer, true_positive_rate_parity)
===")
to = ThresholdOptimizer(estimator=model, constraints="true_positive_rate_parity",
objective="balanced_accuracy_score",
                        predict_method="predict_proba", prefit=True)
to.fit(tr[FEATS], tr.finding, sensitive_features=tr.age_group)
pA = to.predict(te[FEATS], sensitive_features=te.age_group, random_state=0).astype(int)
print(f"umumiy: sezgirlik={recall_score(y, pA):.3f}, aniqlik={precision_score(y, pA):.3f}")
gapA = hisobot(y, pA, s, te.age_group, "A: ThresholdOptimizer")
print("Guruh chegaralari (tasodifiy interpolyatsiya: p0*[>t0] + p1*[>t1]):")
for g, v in to.interpolated_thresholder_.interpolation_dict.items():
    print(f"   {g:<6} p0={float(v['p0']):.2f} t0={float(v['operation0'].threshold):.2f}   p1=
{float(v['p1']):.2f} t1={float(v['operation1'].threshold):.2f}")

# --- Yumshatish B: ko'lamni cheklash (18+ bemorlar), chegarani sezgirlik >= 0.90 uchun qayta
tanlash ---
print("\n=== Yumshatish B: ko'lam cheklash — tizim faqat 19+ yoshdagi bemorlar uchun ===")
m = te.age >= 19; yB, sB = y[m], s[m]
tB = max(t for t in np.arange(0.05, 0.95, 0.01) if recall_score(yB, sB >= t) >= 0.90)
pB = (sB >= tB).astype(int)
```

```
print(f"chiqarilgan tasvirlar: {(~m).sum()} ({(~m).mean():.1%}), yangi chegara {tB:.2f},
sezgirlik={recall_score(yB, pB):.3f}, aniqlik={precision_score(yB, pB):.3f}")
gapB = hisobot(yB, pB, sB, te.age_group[m].reset_index(drop=True), "B: 19+ ko'lam")

print("\n=== Taqqoslash ===")
print(f"{'variant':<28} {'TPR farqi (yosh)':>16} {'sezgirlik':>10} {'aniqlik':>8}  SLO")
for nom, g, pp, yy in (("Dastlabki", gap_age, p, y), ("A: ThresholdOptimizer", gapA, pA, y),
("B: 19+ ko'lam cheklash", gapB, pB, yB)):
    print(f"{nom:<28} {g:>16.3f} {recall_score(yy, pp):>10.3f} {precision_score(yy, pp):>8.3f}
{'OK' if g <= SLO_GAP else 'BUZILDI'}")
print("\nQaror (ADR uchun): B yo'li halolroq — bolalar uchun ma'lumot yetarli emas; model
kartasida «mo'ljallanmagan foydalanish: 18 yoshgacha».")
print("A yo'li esa kam ma'lumotli guruhga sun'iy past chegara qo'yib, yolg'on ijobiylarni
oshiradi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Dastlabki model (chegara 0.30) ===`, `TPR farqi (max-min)`, `=== Taqqoslash ===`

TAJRIBA QILIB KO‘RING

- `constraints="equalized_odds"` va `"demographic_parity"` bilan A yo'lini qayta bajaring — sezgirlik va aniqlikka ta'sirini jadvalga yozing.
- Generatorda bolalar shovqinini 2.2 dan 1.0 ga tushiring — yosh bo'yicha farq SLO'ga sig'adimi? Demak muammo modeldami yoki ma'lumotdami?
- Bootstrap'ni tasvir emas, BEMOR bo'yicha qiling (bemor indekslarini qayta tanlab) va ishonch oralig'i kengayishini kuzating.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Loyiha pasporti va model kartasi avtomatik generatsiyasi, rubrika bo'yicha o'z-o'zini baholash

MAQSAD

Butun ketma-ketlik qisqa shaklda qayta bajariladi va barcha o'lchovlar (sizish, AUC, sezgirlik, chegara, parity, p50/p99, PSI, TPR farqlari, ko'lamdan tashqari guruh) bitta `0` lug'atiga yig'iladi. Shundan Markdown pasport + model kartasi (Mitchell va boshq. formati) va 10 mezonli rubrika jadvali avtomatik tuziladi: avtomatik tekshiriladigan mezonlar SLO bo'yicha to'liq/yarim ball oladi, qolganlari «qo'lda» belgilanadi. O'zgartirib ko'ring: SLO qiymatlari, model o'lchami, drift kuchi — hujjat o'zi qayta yoziladi.

> **Kutilgan natija:** Markdown hujjat stdout'da va /tmp/triage/docs/model_card.md da; test AUC ~0.85, sezgirlik ~0.91, TPR farqlari SLO ichida, drift ALERT ishlaydi; jami ~84/100 (qo'lda mezonlar yarim ball).

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Yakuniy «loyiha pasporti» + model kartasi avtomatik generatsiyasi va rubrika
bo'yicha o'z-o'zini baholash
# Butun ketma-ketlik qisqa shaklda qayta bajariladi (ma'lumot -> model -> ONNX/API -> drift ->
adolatlilik),
# o'lchangan RAQAMLAR SLO bilan solishtiriladi va Markdown hujjat (docs/model_card.md)
yoziladi.
# Hujjatdagi har raqam koddan olinadi — «qo'lda yozilgan» metrika yo'q (himoyada shuni talab
qilishadi).
import os, time, json, hashlib, warnings
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as ort
from scipy.stats import ks_2samp
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import roc_auc_score, recall_score, precision_score
from fairlearn.metrics import MetricFrame, true_positive_rate
torch.manual_seed(0)

FEATS = ["opacity", "lung_area", "ct_ratio", "texture", "intensity", "edge_density"]

def sintetik_bemorlar(n_bemor=1500, seed=0):
    """1-qadamdagi generator: bemor-darajali sintetik manifest."""
    rng = np.random.default_rng(seed); rows = []
    for pid in range(n_bemor):
        age = int(np.clip(rng.normal(50, 18), 1, 95)); sex = "M" if rng.random() < 0.56 else
"F"
        z = rng.normal(0, 1) + 0.02 * (age - 50)
        for k in range(int(rng.integers(1, 4))):
            e = rng.normal(0, 2.2 if age < 18 else 1.0, 6)
            f = [0.9*z+e[0], -0.6*z+e[1], 0.5*z+e[2], 0.7*z+e[3], 0.2*z+e[4], 0.4*z+e[5]]
            rows.append([f"{pid:08d}_{k:03d}.png", pid, age, sex, *np.round(f, 4), int(z +
rng.normal(0, 0.6) > 0.3)])
    return pd.DataFrame(rows, columns=["image_id", "patient_id", "age", "sex", *FEATS,
"finding"])

def psi(ref, cur, bins=10):
    e = np.quantile(ref, np.linspace(0, 1, bins + 1)); e[0], e[-1] = -np.inf, np.inf
    r = np.histogram(ref, e)[0] / len(ref) + 1e-6; c = np.histogram(cur, e)[0] / len(cur) + 1e-
6
```

```
    return float(np.sum((c - r) * np.log(c / r)))

SLO = {"auc": 0.75, "recall": 0.90, "tpr_gap": 0.05, "p99_ms": 800, "rps": 20, "psi_alert":
0.2}
O = {}   # o'lchovlar — pasportga kiradi

# 1) Ma'lumot: bemor bo'yicha bo'lish, sizish testi, ko'lam = 19+ (6-qadam qarori)
df = sintetik_bemorlar(3000); df["age_group"] = pd.cut(df.age, [0, 18, 40, 60, 100], labels=
["0-18", "19-40", "41-60", "61+"]).astype(str)
tr_i, rest = next(GroupShuffleSplit(1, test_size=0.3, random_state=42).split(df,
groups=df.patient_id))
df["split"] = "train"; df.iloc[rest, df.columns.get_loc("split")] = "test"
df.loc[(df.split == "test") & (df.patient_id % 2 == 0), "split"] = "val"
O["leak"] = len(set(df[df.split == "train"].patient_id) & set(df[df.split !=
"train"].patient_id))
O["n_img"], O["n_pat"], O["pos"] = len(df), int(df.patient_id.nunique()),
round(float(df.finding.mean()), 3)
O["data_sha"] = hashlib.sha256(pd.util.hash_pandas_object(df[["image_id",
"finding"]]).values.tobytes()).hexdigest()[:12]
tr, va, te = (df[(df.split == s) & (df.age >= 19)] for s in ("train", "val", "test"))

# 2) Model: MLP (CPU), chegara validatsiyada sezgirlik >= 0.90
X = {k: torch.tensor(d[FEATS].values, dtype=torch.float32) for k, d in (("tr", tr), ("va", va),
("te", te))}
mu, sd = X["tr"].mean(0), X["tr"].std(0); ytr = torch.tensor(tr.finding.values,
dtype=torch.float32)
net = nn.Sequential(nn.Linear(6, 32), nn.ReLU(), nn.Linear(32, 16), nn.ReLU(), nn.Linear(16,
1))
opt = torch.optim.Adam(net.parameters(), lr=5e-3); lossf = nn.BCEWithLogitsLoss()
t0 = time.perf_counter()
for ep in range(150):
    opt.zero_grad(); loss = lossf(net((X["tr"] - mu) / sd).squeeze(1), ytr); loss.backward();
opt.step()
O["train_s"] = round(time.perf_counter() - t0, 1)
class Wrap(nn.Module):
    def __init__(s): super().__init__(); s.net = net; s.register_buffer("mu", mu);
s.register_buffer("sd", sd)
    def forward(s, x): return torch.sigmoid(s.net((x - s.mu) / s.sd))
w = Wrap().eval()
with torch.no_grad(): p_va, p_te = w(X["va"]).squeeze(1).numpy(), w(X["te"]).squeeze(1).numpy()
T = max(t for t in np.arange(0.05, 0.95, 0.01) if recall_score(va.finding, p_va >= t) >=
SLO["recall"])
O.update(threshold=round(float(T), 2), val_auc=round(roc_auc_score(va.finding, p_va), 3),
test_auc=round(roc_auc_score(te.finding, p_te), 3),
         recall=round(recall_score(te.finding, p_te >= T), 3),
precision=round(precision_score(te.finding, p_te >= T), 3))

# 3) ONNX + inferens kechikishi (API bosqichi uchun model qismi), artefakt xeshi
os.makedirs("/tmp/triage/docs", exist_ok=True); ONNX = "/tmp/triage/triage.onnx"
torch.onnx.export(w, torch.randn(1, 6), ONNX, input_names=["features"], output_names=["prob"],
                  dynamic_axes={"features": {0: "b"}, "prob": {0: "b"}}, opset_version=17,
dynamo=False)
sess = ort.InferenceSession(ONNX, providers=["CPUExecutionProvider"])
O["onnx_sha"] = hashlib.sha256(open(ONNX, "rb").read()).hexdigest()[:12]
O["parity"] = float(np.abs(sess.run(None, {"features": X["te"].numpy()})[0].squeeze(1) -
p_te).max())
xs = X["te"][:1].numpy(); lat = []
for _ in range(500):
    s = time.perf_counter(); sess.run(None, {"features": xs}); lat.append((time.perf_counter()
- s) * 1000)
O["p50_ms"], O["p99_ms"] = round(float(np.percentile(lat, 50)), 3),
round(float(np.percentile(lat, 99)), 3)
O["rps"] = int(1000 / np.mean(lat))
```

```
# 4) Drift simulyatsiyasi (yangi apparat) — PSI ogohlantirish ishlaydimi?
cur = sintetik_bemorlar(400, seed=9); cur = cur[cur.age >= 19].copy(); cur["opacity"] =
cur.opacity * 0.7 - 0.9; cur["intensity"] += 0.9
with torch.no_grad(): p_cur = w(torch.tensor(cur[FEATS].values,
dtype=torch.float32)).squeeze(1).numpy()
# Ogohlantirish 5-qadamdagi kabi: ball PSI yoki istalgan belgi PSI >= 0.2
psi_feat = {f: psi(te[f].values, cur[f].values) for f in FEATS}
O["psi_score_drift"] = round(psi(p_te, p_cur), 3); O["psi_feat_max"] = max(psi_feat,
key=psi_feat.get)
O["psi_drift"] = round(max(psi(p_te, p_cur), max(psi_feat.values())), 3); O["psi_normal"] =
round(psi(p_te, p_va), 3)
O["ks_p_drift"] = float(ks_2samp(p_te, p_cur).pvalue)

# 5) Adolatlilik: jins va yosh (19+) bo'yicha TPR farqi; 0–18 uchun alohida hisobot (ko'lamdan
tashqari)
pred = (p_te >= T).astype(int)
for attr in ("sex", "age_group"):
    mf = MetricFrame(metrics=true_positive_rate, y_true=te.finding.values, y_pred=pred,
sensitive_features=te[attr].values)
    O[f"tpr_{attr}"] = {k: round(float(v), 3) for k, v in mf.by_group.items()};
O[f"gap_{attr}"] = round(float(mf.difference()), 3)
kids = df[(df.split == "test") & (df.age < 19)]
with torch.no_grad(): p_k = w(torch.tensor(kids[FEATS].values,
dtype=torch.float32)).squeeze(1).numpy()
O["kids_tpr"], O["kids_n"] = round(recall_score(kids.finding, p_k >= T), 3), len(kids)

# --- Rubrika bo'yicha o'z-o'zini baholash (avtomatik tekshiriladigan qismi) ---
def baho(ok, ball): return ball if ok else ball // 2
rubrika = [
    ("Masala va talablar", "SLO jadvali raqamli, xavf toifasi, mo'ljallanmagan foydalanish
yozilgan", True, 8),
    ("Ma'lumot muhandisligi", f"bemor bo'yicha bo'lish, sizgan bemor = {O['leak']}, sxema
tekshiruvi", O["leak"] == 0, 12),
    ("Model va eksperimentlar", f"chegara {O['threshold']} SLO bo'yicha; test AUC
{O['test_auc']} >= {SLO['auc']}; sezgirlik {O['recall']} >= {SLO['recall']}",
        O["test_auc"] >= SLO["auc"] and O["recall"] >= SLO["recall"] - 0.02, 12),
    ("Optimallashtirish va unumdorlik", f"ONNX parity {O['parity']:.1e}; p99 {O['p99_ms']} ms
<= {SLO['p99_ms']}; {O['rps']} so'rov/s >= {SLO['rps']}",
        O["parity"] < 1e-4 and O["p99_ms"] <= SLO["p99_ms"] and O["rps"] >= SLO["rps"], 12),
    ("API va joylashtirish", "FastAPI /predict, /healthz, API kalit, OpenAPI (4-qadam); Docker
— sandbox'da yo'q", None, 12),
    ("Monitoring", f"PSI normal {O['psi_normal']} < 0.1, drift {O['psi_drift']} >=
{SLO['psi_alert']} -> ogohlantirish ishladi",
        O["psi_normal"] < 0.1 and O["psi_drift"] >= SLO["psi_alert"], 12),
    ("Ishonchlilik va adolatlilik", f"TPR farqi jins {O['gap_sex']}, yosh(19+)
{O['gap_age_group']} <= {SLO['tpr_gap']}; ko'lam qarori asoslangan",
        O["gap_sex"] <= SLO["tpr_gap"] and O["gap_age_group"] <= SLO["tpr_gap"], 14),
    ("Hujjatlar", "pasport + model kartasi avtomatik; arxitektura hujjati va runbook — qo'lda
to'ldiriladi", None, 8),
    ("Jamoaviy ish", "Git tarixi / PR — sandbox'da baholanmaydi", None, 5),
    ("Himoya", "jonli namoyish — sandbox'da baholanmaydi", None, 5),
]
jami, maks_avto = 0, 0
satrlar = []
for nom, dalil, ok, ball in rubrika:
    if ok is None: b, holat = ball // 2, "qo'lda"      # avtomatik tekshirib bo'lmaydi — yarim
ball, o'qituvchi to'ldiradi
    else: b, holat = baho(ok, ball), ("to'liq" if ok else "yarim"); maks_avto += ball
    jami += b; satrlar.append((nom, dalil, holat, b, ball))

# --- Markdown hujjat ---
md = [f"# Loyiha pasporti va model kartasi — CXR Triage (sandbox versiyasi)", f"Yaratildi:
```

```
{time.strftime('%Y-%m-%d %H:%M')} | ma'lumot xeshi: {O['data_sha']} | ONNX xeshi:
{O['onnx_sha']}", "",
      "## 1. Pasport", "| Maydon | Mazmun |", "|---|---|",
      "| Nomi | CXR Triage — rentgenogrammalarni birlamchi saralash (sintetik belgilar bilan
mini-loyiha) |",
      "| Foydalanuvchi | Radiologiya bo'limi; ish ro'yxatini tartiblash moduli |",
      f"| Model SLO | sezgirlik >= {SLO['recall']}; AUC >= {SLO['auc']}; kesimlar bo'yicha TPR
farqi <= {SLO['tpr_gap']} |",
      f"| Tizim SLO | p99 <= {SLO['p99_ms']} ms (CPU); >= {SLO['rps']} so'rov/s |",
      "| Xavf toifasi | Yuqori (EU AI Act, III ilova — tibbiyot); inson nazorati majburiy |",
      f"| Ma'lumot | sintetik, {O['n_img']} tasvir / {O['n_pat']} bemor, topilma ulushi
{O['pos']}; bo'lish bemor bo'yicha, sizish = {O['leak']} |",
      "| Mo'ljallanmagan foydalanish | Tashxis qo'yish; 18 yoshgacha bemorlar (6-qadam qarori);
boshqa apparat tasvirlari qayta tekshiruvsiz |", "",
      "## 2. Model kartasi (Mitchell va boshq., 2019)",
      f"- **Model:** MLP 6-32-16-1, PyTorch -> ONNX (opset 17), parity maks. farq
{O['parity']:.1e}, o'qitish {O['train_s']} s (CPU)",
      f"- **Chegara:** {O['threshold']} (validatsiyada sezgirlik >= {SLO['recall']})",
      f"- **Baholash (test, 19+):** AUC {O['test_auc']} (val {O['val_auc']}), sezgirlik
{O['recall']}, aniqlik {O['precision']}",
      f"- **Kesimlar (TPR):** jins {O['tpr_sex']} (farq {O['gap_sex']}); yosh
{O['tpr_age_group']} (farq {O['gap_age_group']})",
      f"- **Ko'lamdan tashqari guruh (0–18, n={O['kids_n']}):** TPR {O['kids_tpr']} — model
ishonchsiz, tizim bu bemorlarni qabul qilmaydi",
      f"- **Unumdorlik:** ONNX Runtime CPU, batch 1: p50 {O['p50_ms']} ms, p99 {O['p99_ms']}
ms, ~{O['rps']} so'rov/s (bitta oqim)",
      f"- **Drift:** PSI oddiy oqimda {O['psi_normal']}; yangi apparat taqlidida ball PSI
{O['psi_score_drift']}, maks. belgi PSI {O['psi_drift']} ({O['psi_feat_max']}), KS p=
{O['ks_p_drift']:.1e} -> {'ALERT ishlaydi' if O['psi_drift'] >= SLO['psi_alert'] else 'ALERT
ISHLAMADI'}",
      "- **Cheklovlar:** sintetik ma'lumot; yorliqlar shovqinsiz (haqiqiy NIH yorliqlari
shovqinli); bitta oqimda o'lchangan kechikish", "",
      "## 3. Rubrika bo'yicha o'z-o'zini baholash", "| № | Mezon | Dalil (koddan) | Holat |
Ball |", "|---|---|---|---|---|"]
md += [f"| {i+1} | {n} | {d} | {h} | {b}/{m} |" for i, (n, d, h, b, m) in enumerate(satrlar)]
md += ["", f"**Jami: {jami}/100** (avtomatik tekshirilgan mezonlar maksimumi {maks_avto};
«qo'lda» mezonlar yarim ball bilan hisoblangan)"]
yol = "/tmp/triage/docs/model_card.md"; open(yol, "w").write("\n".join(md))
json.dump(O, open("/tmp/triage/docs/measurements.json", "w"), indent=1, default=str)

print("=== Loyiha pasporti va model kartasi ===")
print("\n".join(md))
print("\n=== Yakun ===")
print(f"Hujjat yozildi: {yol} ({os.path.getsize(yol)} bayt); o'lchovlar:
/tmp/triage/docs/measurements.json")
buz = [n for n, d, h, b, m in satrlar if h == "yarim"]
print("SLO buzilgan mezonlar:", buz if buz else "yo'q — barcha avtomatik mezonlar to'liq ball")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Loyiha pasporti va model kartasi ===`, `## 3. Rubrika bo'yicha o'z-o'zini baholash`, `**Jami:`

TAJRIBA QILIB KO‘RING

- `SLO['tpr_gap']` ni 0.03 qiling — qaysi mezon yarim ballga tushadi va hujjatda qanday aks etadi?
- Rubrikaga 11-mezon «Takrorlanuvchanlik» qo'shing: kodni ikki marta bajarib `data_sha` va `test_auc` bir xil chiqsa to'liq ball.
- Model kartasiga «Kalibrlash» bo'limi qo'shing: 10 chelakli reliability jadvali (o'rtacha ball vs haqiqiy ulush) va Brier ko'rsatkichi.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Sxemaga 3 yangi qoida** 1-qadam `SXEMA` ga kamida 3 ta yangi mantiqiy qoida qo'shing (masalan: bir bemorda jins o'zgarmasligi, bir bemorda yosh farqi <= 1, `edge_density` va `texture` orasidagi korrelyatsiya chegarasi). Toza va buzilgan partiyada tekshiruv natijalarini jadval qilib (qoida, buzilgan qatorlar soni) hisobotda keltiring. Virtual laboratoriyada bajariladi.	oson	5
2	**Chegara–sezgirlik egri chizig'i** 2-qadam production modelida chegarani 0.05 dan 0.95 gacha 0.05 qadam bilan o'zgartirib, validatsiyada sezgirlik, aniqlik va HIGH ulushini jadval va grafikda ko'rsating. Sezgirlik 0.85, 0.90, 0.95 uchun tanlangan chegaralarni va har birida radiolog ko'radigan «yolg'on HIGH» sonini (1000 tasvirga) hisoblang. Virtual laboratoriyada bajariladi.	oson	5
3	**API kontrakt testlari** 4-qadam xizmati uchun kamida 8 ta kontrakt testi yozing (to'g'ri so'rov, kalitsiz, noto'g'ri kalit, 5 va 7 belgi, NaN, diapazondan tashqari, yosh < 19 ko'lamdan tashqari). Har test kutilgan HTTP kodi va javob maydonlarini tekshirsin; natijani «test nomi — kutilgan — olingan — PASS/FAIL» jadvali qilib bering va /metrics da holatlar hisoblagichi testlardan keyin qanday ko'rinishini qo'shing. Virtual laboratoriyada bajariladi.	oson	5
4	**Reestr rollback va versiyalar taqqoslash** 2-qadam reestriga `rollback(version)` funksiyasini qo'shing: joriy production'ni «archived» qilib, ko'rsatilgan eski versiyani qaytaradi va sababini jurnalga yozadi. So'ng yangi, ataylab yomonroq model (masalan, `C=0.001`) yarating, uni promote qilishga urinib, darvoza rad etishini, keyin qo'lda majburlab production qilib, rollback bilan qaytarishni ko'rsating. Hisobot: reestr JSON'ning har bosqichdagi holati, versiyalar metrika jadvali, xeshlar. Virtual laboratoriyada bajariladi.	o'rta	10
5	**ONNX optimallashtirish va aniqlik variantlari** 3-qadam modelini FP32 ONNX bilan birga onnxruntime grafik optimallashtirishi (`GraphOptimizationLevel.ORT_ENABLE_ALL`) va dinamik INT8 kvantlash (`onnxruntime.quantization.quantize_dynamic`) variantlarida tayyorlang. Jadval: variant, fayl hajmi, p50/p99 batch 1 va 64, test AUC, PyTorch bilan maks. farq. Qaysi variant SLO'ni buzmay eng tez ekanini asoslang. Virtual laboratoriyada bajariladi.	o'rta	10
6	**Drift qoidalarini sozlash: yolg'on va o'tkazib yuborilgan ogohlantirishlar** 5-qadam simulyatsiyasini 30 kunga cho'zing va uch xil ssenariy kiriting: (a) sekin chiziqli drift, (b) bir kunlik keskin sakrash va qaytish, (c) faqat model ta'sir qilmaydigan belgida siljish. Uch qoida to'plami (faqat PSI ball; PSI ball + belgi; PSI + KS + HIGH ulushi) uchun yolg'on ogohlantirish va o'tkazib yuborish (sezgirlik < 0.85 bo'lgan kun ALERT bo'lmagan) sonini hisoblang. Hisobot: ssenariy × qoida jadvali va tavsiya etilgan qoida. Virtual laboratoriyada bajariladi.	o'rta	10

7	**Bemor bo'yicha bootstrap va kesimlar ishonch oralig'i** 6-qadamdagi bootstrap'ni bemor (klaster) bo'yicha qayta yozing va jins hamda yosh guruhlari uchun TPR, FPR va AUC ning 95% ishonch oraliqlarini hisoblang. TPR farqi uchun ham ishonch oralig'i tuzing va «farq SLO'dan katta» xulosasi statistik jihatdan ishonchli yoki yo'qligini har kesim uchun ayting. Hisobot: jadval + qisqa talqin (qaysi guruhlarda namunalar kamligi xulosani noaniq qiladi). Virtual laboratoriyada bajariladi.	o'rta	10
8	**Monitoring paneli: metrikalardan SLO hisoboti** 4-qadam xizmatiga 2000 so'rov yuboring (ularning 20% ida 5-qadamdagi «yangi apparat» siljishi bo'lsin), so'ng /metrics matnini va audit jurnalini o'qib, Grafana panelining matnli muqobilini tuzing: p50/p95/p99 (gistogramma chelaklaridan interpolyatsiya bilan), so'rovlar holat bo'yicha, HIGH ulushi 100 so'rovlik oynalarda, ball PSI oynalar bo'yicha, jins bo'yicha HIGH ulushi. Har ko'rsatkich yoniga SLO va holat (OK/BUZILDI) qo'ying, kamida bitta ogohlantirish qoidasi ishlashini ko'rsating. Virtual laboratoriyada bajariladi.	qiyin	15
9	**Qayta o'qitish quvuri: drift -> retrain -> darvoza -> promote** 5-qadam ALERT bergandan keyin ishga tushadigan avtomatik quvur yozing: siljigan oqimning «yorliqlangan» qismini train'ga qo'shib qayta o'qitish, 2-qadam reestriga yangi versiya sifatida yozish, SLO darvozasi (shu jumladan 6-qadamdagi TPR farqi) va eski vs yangi modelni siljigan va siljimagan ma'lumotda taqqoslash (AUC, sezgirlik, PSI). Yangi model faqat ikkala to'plamda ham yomonlashmasa promote bo'lsin. Hisobot: quvur bosqichlari jurnali, taqqoslash jadvali va rollback sharti. Virtual laboratoriyada bajariladi.	qiyin	15
10	**O'z mavzungiz uchun to'liq mini-loyiha va pasport** Transport (bekatda kutish vaqti) yoki sanoat monitoringi (sensorlar bo'yicha nosozlik) mavzusini tanlab, 1–7-qadamlar tuzilmasini sintetik ma'lumot bilan takrorlang: o'z sxemangiz va guruh identifikatoringiz (bekat/jihoz), kamida 3 eksperimentli reestr, ONNX + FastAPI xizmati, drift simulyatsiyasi, kamida bitta sezgir atribut bo'yicha adolatlilik tahlili va 7-qadamdagi kabi avtomatik pasport + rubrika. Hisobotda model kartasi, SLO jadvali, ADR ko'rinishida kamida 2 ta qaror (masalan, chegara tanlovi, ko'lam cheklash) bo'lsin. Virtual laboratoriyada bajariladi.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Kredit skoring: mikroqarz defolt xavfi oson

Soha: Moliya (mikromoliya)

Mikromoliya tashkiloti 12 oylik iste'mol qarzlari uchun defolt (90+ kun kechikish) ehtimolini baholaydigan xizmat yaratmoqda. Bitta mijozda 1–4 ta qarz tarixi bor, shuning uchun bo'lish mijoz bo'yicha bo'lishi shart. Model kredit menejeriga «qo'shimcha tekshiruv» bayrog'ini beradi, rad etish qarori emas; jins va yosh bo'yicha kamsitish qonunan taqiqlangan. Yangi kreditlar oqimida inflyatsiya tufayli daromad/qarz nisbati siljiydi — monitoring buni yorliq kelguncha (90 kun) sezishi kerak.

Ma'lumot generatori	2500 mijoz x 1–4 qarz; belgilar: oylik daromad, qarz/daromad nisbati, kredit tarixi oyi, oldingi kechikishlar soni, ish staji, qarz miqdori; yashirin omil z = to'lov qobiliyati
Guruhlash kaliti	customer_id (GroupShuffleSplit, test 0.3)
Sezgir atribut	jins (M/F, 48/52) va yosh guruhi (18-25, 26-40, 41-60, 61+)
Model	logreg (baseline), logreg faqat 2 belgi (zaif), HistGradientBoosting; ONNX uchun MLP 6-32-16-1
SLO	AUC >= 0.78, recall(defolt) >= 0.80, TPR farqi (jins) <= 0.05, API p99 <= 20 ms
Drift ssenariysi	10 kunlik oqim, 5-kundan daromad −15% va qarz/daromad nisbati +0.12 (inflyatsiya), kuch chiziqli o'sadi
Adolatlilik kesimi	jins va yosh guruhi; 18-25 guruhida kredit tarixi qisqa (shovqin x1.8)
Promote qoidasi	SLO bajarilsa va val AUC joriy production'dan kamida 0.005 yuqori bo'lsa

QADAMLAR

1. Lab s1 kabi generator yozing (customer_id, loan_id, age, sex, 6 belgi, default). Sxemaga 8 qoida: loan_id regex ^L\d{7}$, age 18–85, daromad > 0, nisbat 0–3, default {0,1}. Buzilgan partiyada (200 qator, 6 xato turi) har xatoni nomlang. customer_id bo'yicha bo'ling, sizish assert, tasodifiy bo'lishda sizgan mijozlar ulushini hisoblang; split x jins jadvali.
2. Uch eksperimentni o'qiting, validatsiyada recall >= 0.80 beradigan eng katta chegarani tanlang. Reestrga (JSON + SHA-256) staging yozing; promote darvozasi SLO va «AUC +0.005» qoidasini tekshirsin. Jadval: versiya, chegara, valAUC, valRecall, TPR farqi, stage, sabab.
3. MLP 6-32-16-1 ni 30 epoxa o'qiting, normalizatsiyani grafga tiking, ONNX (opset 17, dinamik batch) eksport qiling; onnx.checker va parity (maks. farq < 1e-4); PyTorch vs ONNX Runtime p50/p99 batch 1 va 64.
4. FastAPI: /predict (X-API-Key, pydantic 6 belgi, diapazon nazorati), /healthz, /metrics (Counter + 2 Histogram), audit jurnalida kirish xeshi, jins va yosh guruhi. TestClient bilan 300 so'rov: to'liq va server ichidagi p50/p99; 401/400/422 holatlar.
5. Drift: validatsiya mos yozuv, 10 kun oqim, 5-kundan inflyatsiya siljishi. Har kun belgi PSI, ball PSI, KS p, bayroq ulushi, recall; OK/WARN/ALERT qoidasi (PSI 0.1/0.2, KS p < 0.01, bayroq ulushi ±25%). Birinchi ALERT kunini va eng ko'p siljigan belgini toping.

6. fairlearn MetricFrame: jins va yosh guruhi bo'yicha n, TPR, FPR, tanlov ulushi, AUC; bootstrap 95% IO. Yumshatish A: ThresholdOptimizer (true_positive_rate_parity), B: 18-25 guruhi uchun «qo'lda ko'rib chiqish» yo'li (ko'lam cheklash). Taqqoslash jadvali.
7. Barcha o'lchovlarni bitta lug'atga yig'ib Markdown model kartasi + loyiha pasporti + 10 mezonli rubrika jadvalini avtomatik tuzing; SLO bo'yicha to'liq/yarim ball.

Topshiriladi: Kod papkasi: s1_data.py … s7_passport.py (har biri mustaqil ishlaydi); registry.json, drift_log.json, predictions.jsonl; Avtomatik tuzilgan model_card.md (model kartasi + pasport + rubrika); Hisobot PDF: jadvallar (eksperiment, kechikish, PSI kun x belgi, MetricFrame), 2 grafik, xulosa (>= 200 so'z), ADR: nega B yoki A yumshatish tanlandi

Baholash (100 ball): Sxema validatsiyasi va mijoz bo'yicha sizishsiz bo'lish — 10, Eksperimentlar, reestr, promote darvozasi to'g'ri ishlaydi — 15, ONNX eksport, parity va kechikish o'lchovlari — 10, FastAPI xizmati: kontrakt, metrikalar, audit — 15, Drift monitoring va ogohlantirish mantiqi — 15, Adolatlilik hisoboti va yumshatish taqqoslash — 15, Model kartasi, pasport va hisobot sifati — 20

2-variant. Telekom: abonent ketishi (churn) bashorati oson

Soha: Telekommunikatsiya

Mobil operator keyingi 30 kunda raqamini boshqa operatorga ko'chirish ehtimoli yuqori abonentlarni aniqlab, ularga saqlab qolish taklifi yubormoqchi. Har abonentda 1–3 oylik hisob-kitob yozuvi bor — bo'lish abonent bo'yicha. Taklif byudjeti cheklangan: bayroq ulushi 25% dan oshmasin, lekin ketayotganlarning kamida 75% ushlansin. Tarif yangilanganda foydalanish belgilari siljiydi; shahar va qishloq abonentlari o'rtasida xizmat sifati farq qilmasligi kerak.

Ma'lumot generatori	3000 abonent x 1–3 oy; belgilar: oylik daqiqa, internet GB, qo'ng'iroq markaziga murojaatlar, to'lov kechikishi kuni, tarif narxi, shartnoma yoshi oyi; churn ~18%
Guruhlash kaliti	subscriber_id
Sezgir atribut	hudud turi (shahar 60% / qishloq 40%); qishloqda internet belgisi shovqinliroq (x1.6)
Model	logreg, HistGradientBoosting, RandomForest(200); ONNX MLP 6-24-1
SLO	AUC >= 0.80, recall >= 0.75, bayroq ulushi <= 0.25, TPR farqi (hudud) <= 0.06, p99 <= 15 ms
Drift ssenariysi	8 kun, 4-kundan yangi tarif: internet GB +40%, daqiqa −20%; 7-kunda qo'ng'iroq markazi murojaatlari keskin x2 (bir kunlik sakrash)
Adolatlilik kesimi	hudud turi va shartnoma yoshi guruhi (0-6, 7-24, 25+ oy)
Promote qoidasi	SLO + bayroq ulushi cheklovi; joriy production'dan val AUC past bo'lsa staging'da qoladi

QADAMLAR

1. Generator (subscriber_id, month, region, 6 belgi, churn). Sxema: GB 0–500, daqiqa 0–20000, kechikish 0–90, tarif narxi {29000, 49000, 79000, 119000} so'm, churn {0,1}. Buzilgan partiya: 7 xato turi. subscriber_id bo'yicha bo'lish, sizish assert, split x hudud jadvali.
2. Uch eksperiment; chegara: recall >= 0.75 VA bayroq ulushi <= 0.25 ikkalasini qanoatlantiradigan chegara oralig'ini toping (yo'q bo'lsa sababini yozing). Reestr JSON + SHA-256, promote darvozasi; jadval.
3. MLP 6-24-1 -> ONNX, parity, p50/p99 batch 1/64; ONNX fayl hajmi va xeshi.
4. FastAPI /predict, /healthz, /metrics; javobda offer = RETAIN / NONE; audit jurnalida hudud va shartnoma yoshi; 300 so'rov p50/p99; 401/400/422.
5. Drift: 8 kun, tarif siljishi + 7-kun sakrashi. Belgi PSI jadvali, ball PSI, KS, bayroq ulushi; sakrash va asta siljish uchun qoida qanday farq qilishini izohlang; grafik.
6. MetricFrame hudud va shartnoma yoshi bo'yicha; bootstrap IO. Yumshatish A: ThresholdOptimizer, B: qishloq uchun internet belgisini olib tashlab qayta o'qitish. Taqqoslash.
7. Model kartasi + pasport + rubrika avtomatik generatsiya.

Topshiriladi: Kod papkasi (7 bosqich); registry.json, drift_log.json, predictions.jsonl, model.onnx; model_card.md avtomatik; Hisobot PDF: jadvallar, PSI grafigi, byudjet–recall kelishuvi tahlili, xulosa (>= 200 so'z)

Baholash (100 ball): Sxema va abonent bo'yicha bo'lish — 10, Eksperimentlar, ikki shartli chegara, reestr va promote — 15, ONNX eksport va kechikish — 10, FastAPI xizmati va metrikalar — 15, Drift: siljish va sakrashni farqlash — 15, Adolatlilik va yumshatish — 15, Model kartasi va hisobot — 20

3-variant. Ta'lim: talabaning o'qishni tashlab ketish xavfi oson

Soha: Ta'lim (oliy ta'lim)

Universitet dekanati birinchi va ikkinchi kurs talabalari orasida o'qishni tashlab ketish xavfi yuqori bo'lganlarni semestr o'rtasida aniqlab, tyutor biriktirmoqchi. Har talabada 1–4 semestr yozuvi bor — bo'lish talaba bo'yicha. Model «yordam kerak» ro'yxatini tuzadi, hech qanday jazo choralari uchun ishlatilmaydi. Ta'lim tili va jins bo'yicha teng xizmat talab qilinadi; yangi o'quv yilida baholash tizimi o'zgarsa (5 ballikdan 100 ballikka) belgilar siljiydi.

Ma'lumot generatori	2000 talaba x 1–4 semestr; belgilar: davomat %, o'rtacha ball, topshirilmagan ishlar soni, LMS'ga kirishlar/hafta, qarzdor fanlar, kutubxona olishlari; dropout ~14%
Guruhlash kaliti	student_id
Sezgir atribut	jins (M/F) va ta'lim tili (o'zbek 70% / rus 20% / ingliz 10%)
Model	logreg, DecisionTree(depth 4, tushuntiriladigan), HistGradientBoosting; ONNX MLP 6-16-1
SLO	AUC >= 0.77, recall >= 0.85, TPR farqi (til) <= 0.07, p99 <= 15 ms
Drift ssenariysi	6 hafta, 4-haftadan o'rtacha ball shkalasi 5 -> 100 ballikka o'tadi (masshtab siljishi), LMS kirishlar +30%
Adolatlilik kesimi	ta'lim tili va jins; ingliz guruhi kichik (n kam, IO keng)
Promote qoidasi	SLO bajarilsa; tushuntiriladigan model (daraxt) AUC farqi <= 0.02 bo'lsa unga ustunlik beriladi

QADAMLAR

1. Generator, sxema (davomat 0–100, ball 2–5, LMS 0–200, qarzdor 0–8, student_id regex ^S\d{6}$), buzilgan partiya (6 xato), student_id bo'yicha bo'lish, sizish taqqoslash, split x til jadvali.
2. Uch eksperiment, chegara recall >= 0.85, reestr + SHA-256, promote darvozasi «tushuntiriladigan modelga ustunlik» qoidasi bilan; jadval va asoslash.
3. MLP 6-16-1 -> ONNX; parity, kechikish, xeshlar.
4. FastAPI: /predict javobida risk_level (LOW/MEDIUM/HIGH, ikki chegara) va tavsiya matni; /metrics, audit jurnali (student_id EMAS — tuzli xesh); 300 so'rov.
5. Drift: 6 hafta, ball shkalasi siljishi — PSI ALERT va recall keskin tushishi; sxema tekshiruvi (max=5) buni API darajasida 400 bilan ushlashini ko'rsating; ikki himoya qatlamini taqqoslang.
6. MetricFrame til va jins; bootstrap; A: ThresholdOptimizer, B: ingliz guruhi uchun natija «tyutor qo'lda ko'radi» rejimi. Taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json, predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, grafiklar, «ikki himoya qatlami» tahlili, xulosa (>= 200 so'z)

Baholash (100 ball): Sxema va talaba bo'yicha bo'lish — 10, Eksperimentlar, reestr, promote (tushuntirish qoidasi) — 15, ONNX va kechikish — 10, FastAPI, uch darajali javob, metrikalar, audit — 15, Drift va sxema himoyasi taqqoslash — 15, Adolatlilik va yumshatish — 15, Model kartasi va hisobot — 20

4-variant. Ekologiya: ichimlik suvi namunasi ifloslanganligi oson

Soha: Ekologiya (suv sifati)

Viloyat sanitariya laboratoriyasi quduq va suv tarmog'i nuqtalaridan olinadigan namunalar uchun «bakteriologik ifloslanish ehtimoli» xizmatini yaratmoqda — ijobiy natija qimmat laborator ekish tahliliga yuboriladi. Bir nuqtadan yiliga 2–6 namuna olinadi, bo'lish nuqta bo'yicha. Sezgirlik 0.92 dan past bo'lmasin (o'tkazib yuborish xavfli), lekin ekish laboratoriyasi quvvati cheklangan. Yomg'irli mavsumda loyqalik siljiydi; shahar va qishloq nuqtalari teng xizmat olishi kerak.

Ma'lumot generatori	1200 nuqta x 2–6 namuna; belgilar: loyqalik NTU, pH, qoldiq xlor mg/l, harorat, elektr o'tkazuvchanlik, nitrat mg/l; ifloslanish ~12%
Guruhlash kaliti	site_id
Sezgir atribut	nuqta turi (shahar tarmog'i 55% / qishloq qudug'i 45%)
Model	logreg, GaussianNB, HistGradientBoosting; ONNX MLP 6-32-1
SLO	recall >= 0.92, AUC >= 0.82, TPR farqi (nuqta turi) <= 0.05, p99 <= 15 ms
Drift ssenariysi	12 kun, 6-kundan yomg'ir mavsumi: loyqalik x1.8, qoldiq xlor −0.2; 10-kunda qishloq nuqtalarida xlorlash boshlanadi (qisman qaytish)
Adolatlilik kesimi	nuqta turi va mavsum (quruq/yomg'irli)
Promote qoidasi	SLO bajarilsa va val recall joriy production'dan kam bo'lmasa (recall-birinchi qoida)

QADAMLAR

1. Generator, sxema (NTU 0–100, pH 5–10, xlor 0–5, harorat 0–40, nitrat 0–100, site_id regex ^W\d{5}$), buzilgan partiya 6 xato, site_id bo'yicha bo'lish, sizish taqqoslash, split x nuqta turi jadvali.
2. Uch eksperiment, chegara recall >= 0.92, reestr + xesh, promote «recall-birinchi» qoidasi; jadval; GaussianNB nega zaif ekanini izohlang.
3. MLP 6-32-1 -> ONNX, parity, p50/p99, xeshlar.
4. FastAPI /predict (javob: action = CULTURE_TEST / OK), /healthz, /metrics, audit (nuqta turi, mavsum); 300 so'rov; 401/400/422.
5. Drift 12 kun: yomg'ir siljishi va qisman qaytish; PSI kun x belgi, ball PSI, KS, bayroq ulushi; qaytishdan keyin WARN'ga tushadimi?
6. MetricFrame nuqta turi va mavsum bo'yicha; bootstrap; A: ThresholdOptimizer, B: qishloq quduqlari uchun alohida chegara qo'lda; taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json, predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, PSI va bayroq ulushi grafigi, xulosa (>= 200 so'z)

Baholash (100 ball): Sxema va nuqta bo'yicha bo'lish — 10, Eksperimentlar, reestr, recall-birinchi promote — 15, ONNX va kechikish — 10, FastAPI va metrikalar — 15, Drift (siljish va qaytish) — 15, Adolatlilik — 15, Model kartasi va hisobot — 20

5-variant. Chakana savdo: do'kon-mahsulot haftalik talab prognozi oson

Soha: Chakana savdo

Supermarketlar tarmog'i 40 ta do'konda 30 ta tez buziladigan mahsulot uchun keyingi hafta sotuvini bashorat qiluvchi regressiya xizmatini yaratmoqda. Bitta do'kon-mahsulot juftligida 20 haftalik tarix bor — bo'lish do'kon bo'yicha (yangi do'kon ochilganda model ishlashi kerak). SLO MAPE bilan beriladi; bayram haftalarida talab sakraydi, yangi do'konlar ochilganda taqsimot siljiydi. Markaz va chekka do'konlar uchun xatolik teng bo'lishi talab qilinadi.

Ma'lumot generatori	40 do'kon x 30 mahsulot x 20 hafta = 24000 qator; belgilar: o'tgan hafta sotuvi, 4 haftalik o'rtacha, narx, chegirma %, bayram bayrog'i, do'kon maydoni m2; maqsad: keyingi hafta sotuvi (dona)
Guruhlash kaliti	store_id (GroupShuffleSplit, test 0.25)
Sezgir atribut	do'kon joylashuvi (markaz 40% / chekka 60%)
Model	Ridge, HistGradientBoostingRegressor, RandomForestRegressor; ONNX MLP 6-32-16-1 (regressiya)
SLO	MAPE <= 18%, MAE joylashuv bo'yicha farqi <= 15% nisbiy, p99 <= 15 ms, batch 64 p99 <= 2 ms (ONNX)
Drift ssenariysi	10 hafta oqim, 5-haftadan 8 ta yangi kichik do'kon (maydon −40%, sotuv −35%); 8-hafta bayram sakrashi x1.6
Adolatlilik kesimi	joylashuv va do'kon hajmi (kichik/o'rta/katta)
Promote qoidasi	val MAPE SLO ichida va joriy production MAPE dan kamida 0.5 foiz bandi yaxshi

QADAMLAR

1. Generator (store_id, sku, week, 6 belgi, sales). Sxema: sotuv >= 0, chegirma 0–70, narx > 0, bayram {0,1}, (store_id, sku, week) unikal. Buzilgan partiya 6 xato. store_id bo'yicha bo'lish; tasodifiy bo'lishda nechta do'kon ikki to'plamga tushishini hisoblang.
2. Uch regressiya eksperimenti; metrikalar MAE, MAPE, R2; reestr + xesh; promote «MAPE −0.5 bandi» qoidasi; jadval.
3. Regressiya MLP -> ONNX (chiqish log1p masshtabida, grafda expm1 bilan qaytariladi); parity, p50/p99 batch 1 va 64.
4. FastAPI /predict javobi: forecast, lower, upper (±MAPE), /metrics (Histogram forecast_units), audit (do'kon joylashuvi); 300 so'rov.
5. Drift 10 hafta: belgi PSI, prognoz PSI, KS; MAPE haftalik; bayram sakrashini «kutilgan hodisa» sifatida qoidadan istisno qiling (holiday kalendari) va yolg'on ALERT kamayishini ko'rsating.
6. MetricFrame regressiya uchun: guruh bo'yicha MAE, MAPE, n (metrics lug'atiga mean_absolute_error); bootstrap IO; yumshatish: guruh bo'yicha vazn (sample_weight) bilan qayta o'qitish vs alohida model kichik do'konlar uchun.
7. Model kartasi, pasport, rubrika (regressiya metrikalari bilan).

Topshiriladi: Kod papkasi; registry.json, drift_log.json, predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, haftalik MAPE va PSI grafigi, bayram istisnosi tahlili, xulosa (>= 200 so'z)

Baholash (100 ball): Sxema va do'kon bo'yicha bo'lish — 10, Regressiya eksperimentlari, reestr, promote — 15, ONNX (log masshtab) va kechikish — 10, FastAPI prognoz oralig'i va metrikalar — 15, Drift va bayram istisnosi — 15, Guruhlar bo'yicha xatolik adolati — 15, Model kartasi va hisobot — 20

6-variant. Sport: futbolchi jarohat xavfi (haftalik yuklama) oson

Soha: Sport tibbiyoti

Futbol klubi akademiyasi va asosiy jamoasi o'yinchilari uchun keyingi 7 kunda muskul jarohati xavfini baholaydigan xizmat qurmoqda; xavf yuqori bo'lsa murabbiy mashg'ulot yuklamasini kamaytiradi. Har o'yinchida 10–30 haftalik GPS va so'rovnoma yozuvlari bor — bo'lish o'yinchi bo'yicha. Jarohatlar kam (6%), shuning uchun sezgirlik muhim, lekin har haftada jamoaning 30% dan ko'pini «dam oldirish» mumkin emas. Yangi GPS jiletlari o'rnatilganda masofa belgilari siljiydi; o'smir (U-17) va kattalar guruhlari teng himoyalanishi kerak.

Ma'lumot generatori	150 o'yinchi x 10–30 hafta = ~3000 qator; belgilar: haftalik masofa km, yuqori tezlikdagi masofa m, ACWR (o'tkir/surunkali nisbat), uyqu sifati 1-10, charchoq 1-10, o'tgan jarohatlar soni; jarohat ~6%
Guruhlash kaliti	player_id
Sezgir atribut	yosh guruhi (U-17 30% / kattalar 70%) va jins (erkaklar 65% / ayollar jamoasi 35%)
Model	logreg (class_weight=balanced), HistGradientBoosting, kNN(15); ONNX MLP 6-16-8-1
SLO	recall >= 0.80, bayroq ulushi <= 0.30, AUC >= 0.75, TPR farqi (yosh) <= 0.08, p99 <= 15 ms
Drift ssenariysi	8 hafta, 4-haftadan yangi GPS jilet: masofa +8%, yuqori tezlik masofasi +25% (kalibrovka)
Adolatlilik kesimi	yosh guruhi va jins; U-17 da so'rovnoma shovqini x2
Promote qoidasi	SLO + bayroq ulushi; nomutanosiblik tufayli AUC o'rniga PR-AUC bilan taqqoslanadi

QADAMLAR

1. Generator, sxema (masofa 0–60 km, ACWR 0.3–2.5, uyqu va charchoq 1–10, player_id regex ^P\d{4}$, bir o'yinchida jins o'zgarmasligi — qo'shimcha mantiqiy qoida), buzilgan partiya, player_id bo'yicha bo'lish, sizish taqqoslash.
2. Uch eksperiment (class_weight), chegara recall >= 0.80 va bayroq <= 0.30; reestr + xesh; promote PR-AUC bilan; jadval; nomutanosiblikda AUC va PR-AUC farqini izohlang.
3. MLP -> ONNX, parity, kechikish, xeshlar.
4. FastAPI /predict (javob: risk, recommendation = REDUCE_LOAD / NORMAL), /metrics, audit (yosh guruhi, jins); 300 so'rov.
5. Drift 8 hafta: GPS kalibrovka siljishi; PSI kun x belgi; ACWR nisbat belgisi siljimasligini (nisbat!) va shu sababli model ta'siri kichikligini ko'rsating — bu «belgi drifti, lekin zarar kam» holatimi?
6. MetricFrame yosh va jins bo'yicha; bootstrap (o'yinchi bo'yicha!); A: ThresholdOptimizer, B: U-17 uchun shifokor ko'rigi majburiy rejimi; taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json, predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, PR egri chizig'i, PSI grafigi, xulosa (>= 200 so'z)

Baholash (100 ball): Sxema (mantiqiy qoida bilan) va o'yinchi bo'yicha bo'lish — 10, Nomutanosib eksperimentlar, reestr, PR-AUC promote — 15, ONNX va kechikish — 10, FastAPI va metrikalar — 15, Drift: nisbat belgisi tahlili — 15, Adolatlilik, o'yinchi bo'yicha bootstrap — 15, Model kartasi va hisobot — 20

7-variant. Energetika: bino energiya samaradorligi sinfi oson

Soha: Energetika (binolar)

Shahar hokimligi ko'p qavatli uylar uchun «yuqori isitish sarfi» (normadan 30% ko'p) ehtimolini baholaydigan xizmat buyurdi — natija energiya auditi navbatini tuzish uchun ishlatiladi. Har binoda 1–3 isitish mavsumi yozuvi bor, bo'lish bino bo'yicha. Qadimgi (1970 gacha) va yangi binolar uchun sezgirlik teng bo'lishi kerak; aqlli hisoblagichlar o'rnatilgach, o'lchov aniqligi oshib belgilar taqsimoti siljiydi.

Ma'lumot generatori	1800 bino x 1–3 mavsum; belgilar: umumiy maydon m2, qavatlar, deraza/devor nisbati, izolyatsiya indeksi 0–1, o'rtacha tashqi harorat, kvartiralar soni; yuqori sarf ~28%
Guruhlash kaliti	building_id
Sezgir atribut	qurilgan davri (1970 gacha 35% / 1970–2000 40% / 2000 dan keyin 25%) va tuman (markaz/chekka)
Model	logreg, HistGradientBoosting, ExtraTrees(300); ONNX MLP 6-32-1
SLO	AUC >= 0.80, recall >= 0.85, TPR farqi (davr) <= 0.06, p99 <= 15 ms
Drift ssenariysi	9 oy, 5-oydan aqlli hisoblagich: belgilar shovqini −50% (taqsimot torayadi), o'rtacha o'zgarmaydi
Adolatlilik kesimi	qurilgan davri va tuman
Promote qoidasi	SLO bajarilsa va Brier ko'rsatkichi joriy production'dan yomon bo'lmasa (kalibrovka qoidasi)

QADAMLAR

1. Generator, sxema (maydon 300–30000, qavat 2–25, nisbat 0.05–0.6, izolyatsiya 0–1, building_id regex ^B\d{5}$), buzilgan partiya 6 xato, building_id bo'yicha bo'lish, sizish taqqoslash, split x davr jadvali.
2. Uch eksperiment; chegara recall >= 0.85; reestr + xesh; promote darvozasi Brier kalibrovka qoidasi bilan; jadval; kalibrovka egri chizig'i (5 chelak).
3. MLP -> ONNX, parity, kechikish, xeshlar.
4. FastAPI /predict (javob: audit_priority), /healthz, /metrics, audit jurnali (davr, tuman); 300 so'rov p50/p99.
5. Drift 9 oy: taqsimot torayishi (o'rtacha o'zgarmaydi) — PSI va KS qanday javob beradi? Recall o'zgarishini hisoblang va «o'rtacha emas, shakl drifti» xulosasini chiqaring.
6. MetricFrame davr va tuman; bootstrap; A: ThresholdOptimizer, B: davr bo'yicha alohida kalibrovka (CalibratedClassifierCV guruh bo'yicha); taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json, predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, kalibrovka va PSI grafiklari, xulosa (>= 200 so'z)

Baholash (100 ball): Sxema va bino bo'yicha bo'lish — 10, Eksperimentlar, reestr, kalibrovka promote — 15, ONNX va kechikish — 10, FastAPI va metrikalar — 15, Shakl drifti tahlili — 15, Adolatlilik va guruh kalibrovkasi — 15, Model kartasi va hisobot — 20

8-variant. HR: ishga qabul uchun rezyume skrining o'rta

Soha: Inson resurslari

IT kompaniyasi junior dasturchi vakansiyasiga kelgan arizalarni «suhbatga taklif qilish» ehtimoli bo'yicha saralaydigan yordamchi tizim qurmoqda. Bir nomzod bir necha vakansiyaga (1–3) ariza beradi — bo'lish nomzod bo'yicha. Tarixiy yorliqlar HR xodimlari qarorlaridan olingan va ularda jins va yosh bo'yicha tarixiy noxolislik bor (sintetik generatorda ataylab kiritiladi). Tizim talabi: jins bo'yicha tanlov ulushi farqi 0.05 dan oshmasin (demografik paritet), yosh bo'yicha TPR farqi 0.06 dan oshmasin. Tashqi auditor model kartasini talab qiladi.

Ma'lumot generatori	2500 nomzod x 1–3 ariza; belgilar: tajriba yili, test ball 0–100, GitHub faolligi, ta'lim darajasi kodi, til sertifikati, oldingi ish joylari soni; yorliqda jins bo'yicha −0.4 logit tarixiy noxolislik
Guruhlash kaliti	candidate_id
Sezgir atribut	jins (M 62% / F 38%) va yosh guruhi (18-25, 26-35, 36-45, 46+)
Model	logreg, HistGradientBoosting, logreg sezgir atributga korrelyatsion belgilarsiz (proxy olib tashlangan); ONNX MLP 6-32-1
SLO	AUC >= 0.78, recall >= 0.80, tanlov ulushi farqi (jins) <= 0.05, TPR farqi (yosh) <= 0.06, p99 <= 15 ms
Drift ssenariysi	8 hafta, 4-haftadan universitet bitiruvi: 18-25 guruhi ulushi 25% -> 55%, test ball o'rtachasi −6
Adolatlilik kesimi	jins (demographic_parity) va yosh (TPR); kesishma jins x yosh jadvali
Promote qoidasi	barcha SLO; adolatlilik SLO buzilsa AUC qanchalik yuqori bo'lmasin rad etiladi (qattiq darvoza)

QADAMLAR

1. Generator (yorliqda ataylab noxolislik), sxema (ball 0–100, tajriba 0–30, candidate_id regex), buzilgan partiya, candidate_id bo'yicha bo'lish, sizish taqqoslash; dastlabki ma'lumotda jins bo'yicha ijobiy yorliq ulushini jadvalda ko'rsating (noxolislik dalili).
2. Uch eksperiment, shu jumladan proxy belgilarsiz model; chegara recall >= 0.80; reestr + xesh; qattiq adolatlilik darvozasi: tanlov ulushi farqini promote ichida hisoblang; jadval — qaysi model rad etildi va nima uchun.
3. MLP -> ONNX, parity, kechikish; ONNX kirishida sezgir atribut yo'qligini kontrakt bilan kafolatlang.
4. FastAPI /predict (javob: shortlist ehtimoli + «inson qaror qabul qiladi» eslatmasi), /metrics, audit jurnali (jins, yosh guruhi faqat statistik hisobot uchun, alohida fayl); 300 so'rov.
5. Drift 8 hafta: yosh taqsimoti siljishi — ball PSI ALERT, lekin AUC deyarli o'zgarmaydi (populyatsiya drifti vs kontsept drifti); tanlov ulushi farqini har hafta monitoring qiling — adolatlilik drifti ham ogohlantirsin.
6. MetricFrame jins (selection_rate, demographic_parity_difference) va yosh (TPR); kesishma jadvali; A: ThresholdOptimizer(demographic_parity), B:

ExponentiatedGradient(DemographicParity) in-processing; C: proxy belgilarsiz model; uchlik taqqoslash: AUC, recall, farqlar.

7. Model kartasi auditor uchun: mo'ljallangan foydalanish, noxolislik manbai, yumshatish, cheklovlar; pasport; rubrika.

Topshiriladi: Kod papkasi; registry.json (rad etilgan versiyalar sabablari bilan), drift_log.json (adolatlilik drifti ustuni bilan), predictions.jsonl; model_card.md auditor formati; Hisobot PDF: noxolislik dalili, uchlik yumshatish taqqoslash, kesishma jadvali, ADR, xulosa (>= 250 so'z)

Baholash (100 ball): Sxema, nomzod bo'yicha bo'lish, noxolislik dalili — 10, Eksperimentlar va qattiq adolatlilik darvozasi — 15, ONNX va kontrakt kafolati — 10, FastAPI, metrikalar, ma'lumot minimallashtirish — 15, Populyatsiya drifti va adolatlilik drifti monitoringi — 15, Uch yumshatish usuli taqqoslash — 15, Auditor model kartasi va hisobot — 20

9-variant. Energetika: podstansiya soatlik yuklama prognozi o'rta

Soha: Energetika (elektr tarmog'i)

Hududiy elektr tarmoqlari 60 ta podstansiya uchun keyingi soat yuklamasini (MVt) bashorat qiluvchi regressiya xizmatini yaratmoqda; prognoz dispetcherga generatsiya rejasini tuzishga yordam beradi. Har podstansiyada 30 kunlik soatlik tarix bor — bo'lish podstansiya bo'yicha (yangi podstansiya ulanganda ishlashi uchun). SLO MAPE va alohida «pik soat» xatoligi bilan beriladi. Yoz issig'i boshlanganda konditsioner yuklamasi taqsimotni siljitadi; shahar va qishloq podstansiyalari uchun xatolik teng bo'lishi kerak.

Ma'lumot generatori	60 podstansiya x 30 kun x 24 soat = 43200 qator; belgilar: oldingi soat yuklama, 24 soat oldingi yuklama, haftalik o'rtacha, tashqi harorat, soat (sin/cos), dam olish kuni; maqsad: keyingi soat MVt
Guruhlash kaliti	substation_id (test 0.25)
Sezgir atribut	podstansiya turi (shahar 45% / qishloq 55%); qishloqda o'lchov shovqini x1.5
Model	Ridge, HistGradientBoostingRegressor, MLPRegressor(64,32); ONNX MLP 7-64-32-1
SLO	MAPE <= 6%, pik soat (18–22) MAPE <= 9%, MAE farqi (tur) <= 20% nisbiy, batch 64 p99 <= 1.5 ms (ONNX), API p99 <= 15 ms
Drift ssenariysi	14 kun, 7-kundan issiq to'lqin: harorat +9 °C, pik yuklama +22%; 12-kunda 5 qishloq podstansiyada quyosh panellari — kunduzgi yuklama −15%
Adolatlilik kesimi	podstansiya turi va kun turi (ish/dam)
Promote qoidasi	umumiy MAPE va pik MAPE ikkalasi SLO ichida; joriy production'dan pik MAPE yomonlashsa rad etiladi

QADAMLAR

1. Generator (vaqt qatori, soat sin/cos), sxema (yuklama 0–500 MVt, harorat −25…45, soat 0–23, (substation_id, ts) unikal, ketma-ket soatlar orasida bo'shliq yo'q — mantiqiy qoida), buzilgan partiya, substation_id bo'yicha bo'lish; vaqt bo'yicha sizish haqida ham izoh (oxirgi 5 kun test).
2. Uch regressiya eksperimenti; metrikalar MAE, MAPE, pik MAPE; reestr + xesh; promote ikki MAPE qoidasi; jadval.
3. MLP 7-64-32-1 -> ONNX; parity; batch 1 va 64 p50/p99 (dispetcher 60 podstansiyani bir so'rovda yuboradi — batch 64 muhim).
4. FastAPI /predict_batch (ro'yxat kirish, 1–64), /healthz, /metrics (Histogram batch_size), audit (podstansiya turi, soat); 300 so'rov (har biri batch 60).
5. Drift 14 kun: issiq to'lqin va quyosh panellari; belgi PSI, prognoz PSI, qoldiq (residual) taqsimoti KS; soatlik MAPE; «harorat drifti modelga kiritilgan belgi, demak kutilgan» — qoidani harorat belgisi uchun yumshating va asoslang.
6. MetricFrame tur va kun turi bo'yicha MAE/MAPE; bootstrap; yumshatish: tur bo'yicha sample_weight vs alohida qishloq modeli; taqqoslash.

7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json, predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, soatlik MAPE grafigi, PSI grafigi, xulosa (>= 250 so'z)

Baholash (100 ball): Vaqt qatori sxemasi va podstansiya bo'yicha bo'lish — 10, Regressiya eksperimentlari, pik MAPE, reestr — 15, ONNX batch inferens — 10, Batch API va metrikalar — 15, Drift: kutilgan vs kutilmagan siljish — 15, Guruhlar bo'yicha xatolik adolati — 15, Model kartasi va hisobot — 20

10-variant. Agro: barg tasviri belgilaridan kasallik aniqlash o'rta

Soha: Qishloq xo'jaligi

Agro-maslahat xizmati fermerlar telefonda suratga olgan g'o'za barglaridan zang kasalligi ehtimolini qaytaradigan API qurmoqda. Sandbox'da tasvir o'rniga undan olingan 6 ta belgi (yashil ulushi, dog' maydoni, dog' soni, rang dispersiyasi, qirra zichligi, yorqinlik) sintetik yaratiladi. Bir daladan 2–8 surat keladi — bo'lish dala bo'yicha. Yangi smartfon modeli chiqqanda kamera ranglari siljiydi. Kichik dehqon xo'jaliklari (eski telefon, shovqinli surat) yirik xo'jaliklar bilan teng sifatli xizmat olishi kerak.

Ma'lumot generatori	800 dala x 2–8 surat = ~4000 tasvir; 6 belgi; kasallik ~22%; kichik xo'jalik suratlarida shovqin x1.7
Guruhlash kaliti	field_id
Sezgir atribut	xo'jalik turi (kichik dehqon 55% / fermer 30% / klaster 15%) va viloyat (3 ta)
Model	logreg, SVC(rbf, probability=True), HistGradientBoosting; ONNX MLP 6-32-16-1
SLO	recall >= 0.88, AUC >= 0.82, TPR farqi (xo'jalik turi) <= 0.07, p99 <= 20 ms
Drift ssenariysi	10 kun, 5-kundan yangi smartfon: yorqinlik +0.8, rang dispersiyasi −25%; 8-kunda kechki suratlar ulushi oshadi (yorqinlik −0.5)
Adolatlilik kesimi	xo'jalik turi va viloyat
Promote qoidasi	SLO + kichik xo'jalik kesimida recall >= 0.85 (kesim darvozasi)

QADAMLAR

1. Generator, sxema (image_id regex ^F\d{4}_\d{3}\.jpg$, yashil ulushi 0–1, dog' maydoni 0–1, dog' soni 0–200, label {0,1}), buzilgan partiya 7 xato, field_id bo'yicha bo'lish, sizish taqqoslash, split x xo'jalik turi.
2. Uch eksperiment; chegara recall >= 0.88; reestr + xesh; promote kesim darvozasi (kichik xo'jalik recall) bilan; jadval; SVC o'qitish vaqtini ham yozing.
3. MLP -> ONNX, parity, kechikish; haqiqiy loyihada MobileNet bo'lishi va telefonda ONNX Runtime Mobile ishlatilishi haqida izoh.
4. FastAPI /predict (javob: disease_prob, action = TREAT / MONITOR / OK, ikki chegara), /metrics, audit (xo'jalik turi, viloyat, telefon modeli sarlavhasi); 300 so'rov.
5. Drift 10 kun: kamera siljishi va kechki suratlar — ikki yo'nalishli yorqinlik siljishi PSI'da qanday ko'rinadi; sabab topish uchun telefon modeli bo'yicha kesimli PSI hisoblang.
6. MetricFrame xo'jalik turi va viloyat; bootstrap dala bo'yicha; A: ThresholdOptimizer, B: shovqinli suratlar uchun «qayta suratga oling» javobi (sifat darvozasi); taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json, predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, telefon modeli bo'yicha PSI, grafiklar, xulosa (>= 250 so'z)

Baholash (100 ball): Sxema va dala bo'yicha bo'lish — 10, Eksperimentlar, kesim darvozasi, reestr — 15, ONNX va kechikish — 10, FastAPI uch darajali javob va metrikalar — 15, Kesimli drift tahlili — 15, Adolatlilik va sifat darvozasi — 15, Model kartasi va hisobot — 20

11-variant. Transport: chorrahada tirbandlik darajasi bashorati o'rta

Soha: Transport (shahar harakati)

Shahar transport markazi 120 ta chorraha uchun keyingi 15 daqiqada «og'ir tirbandlik» (o'rtacha tezlik 10 km/soat dan past) ehtimolini bashorat qiluvchi xizmat qurmoqda; natija svetofor rejimini almashtirish uchun ishlatiladi. Har chorrahada 14 kunlik 15 daqiqalik oynalar bor — bo'lish chorraha bo'yicha. Yo'l ta'miri boshlanganda ayrim chorrahalarda oqim keskin o'zgaradi. Markaziy va chekka tumanlar chorrahalari uchun sifat teng bo'lishi talab qilinadi.

Ma'lumot generatori	120 chorraha x 14 kun x 96 oyna = ~161000 qator (10% namuna olib ishlang — 16000); belgilar: oldingi oyna hajmi (mashina/soat), tezlik, kun soati sin/cos, dam olish, yomg'ir, yaqin tadbir bayrog'i; og'ir tirbandlik ~20%
Guruhlash kaliti	intersection_id
Sezgir atribut	tuman turi (markaz 35% / chekka 65%)
Model	logreg, HistGradientBoosting, kNN(25); ONNX MLP 7-32-1
SLO	AUC >= 0.85, recall >= 0.80, aniqlik >= 0.60, TPR farqi (tuman) <= 0.05, p99 <= 10 ms (real vaqt)
Drift ssenariysi	10 kun, 4-kundan 15 chorrahada yo'l ta'miri: hajm −40%, tezlik −30% (lokal drift); 8-kunda yomg'ir (global drift)
Adolatlilik kesimi	tuman turi va kun soati guruhi (tun/ertalab pik/kunduz/kechki pik)
Promote qoidasi	SLO + p99 <= 10 ms (kechikish darvozasi — kechikish ham reestrga yoziladi)

QADAMLAR

1. Generator (vaqt oynalari), sxema (hajm 0–4000, tezlik 0–90, (intersection_id, ts) unikal, tezlik > 0 bo'lsa hajm >= 0 — mantiqiy), buzilgan partiya, intersection_id bo'yicha bo'lish, sizish taqqoslash.
2. Uch eksperiment; chegara recall >= 0.80 va aniqlik >= 0.60 oralig'i; reestr + xesh + kechikish (ONNX p99 batch 1); promote kechikish darvozasi bilan; kNN nega kechikish bo'yicha rad etilishini ko'rsating.
3. MLP -> ONNX, parity, kechikish batch 1 va 120 (barcha chorrahalar bir so'rovda).
4. FastAPI /predict va /predict_batch, /metrics (Histogram batch_size va latency), audit (tuman, soat guruhi); 300 so'rov; p99 SLO tekshiruvi.
5. Drift 10 kun: lokal (15 chorraha) vs global (yomg'ir) drift — umumiy PSI lokal driftni sezmasligini, chorraha bo'yicha kesimli PSI esa sezishini ko'rsating; qoida: «kamida 10% guruhda PSI >= 0.2 bo'lsa WARN».
6. MetricFrame tuman va soat guruhi; bootstrap chorraha bo'yicha; A: ThresholdOptimizer, B: tuman bo'yicha alohida chegara qo'lda (ikki chegara); taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json (kechikish ustuni bilan), drift_log.json (kesimli PSI), predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, lokal vs global drift grafigi, xulosa (>= 250 so'z)

Baholash (100 ball): Vaqt oynasi sxemasi va chorraha bo'yicha bo'lish — 10, Eksperimentlar va kechikish darvozasi — 15, ONNX batch inferens — 10, Real vaqt API va metrikalar — 15, Lokal vs global drift — 15, Adolatlilik — 15, Model kartasi va hisobot — 20

12-variant. Sug'urta: avtosug'urta da'vo xavfi va narxlash yordamchisi o'rta

Soha: Sug'urta

Sug'urta kompaniyasi avto polislar uchun keyingi yilda da'vo bo'lish ehtimolini baholaydigan modelni anderrayterga yordamchi sifatida joylamoqchi. Bir polis egasida 1–3 ta avtomobil polisi bor — bo'lish polis egasi bo'yicha. Qonunga ko'ra jins narxlashda ishlatilmaydi, yosh esa ishlatiladi, lekin 18-24 yosh guruhida yolg'on ijobiy ulushi (FPR) boshqa guruhlardan 0.08 dan ko'p farq qilmasin. Yangi elektromobillar bozorga kirganda belgilar siljiydi.

Ma'lumot generatori	2200 polis egasi x 1–3 polis; belgilar: haydovchilik staji, yillik yurish km, avto yoshi, dvigatel quvvati, oldingi da'volar soni, hudud xavf indeksi; da'vo ~15%
Guruhlash kaliti	policyholder_id
Sezgir atribut	jins (modelga kirmaydi, faqat auditda) va yosh guruhi (18-24, 25-40, 41-60, 61+)
Model	logreg, HistGradientBoosting, GradientBoosting(100); ONNX MLP 6-32-16-1
SLO	AUC >= 0.76, recall >= 0.70, FPR farqi (yosh) <= 0.08, tanlov ulushi farqi (jins) <= 0.04, p99 <= 20 ms
Drift ssenariysi	12 oy, 6-oydan elektromobillar: dvigatel quvvati belgisi +60%, avto yoshi −3; 9-oyda da'vo ulushi 15% -> 21% (kontsept drifti, yorliq 3 oy kechikadi)
Adolatlilik kesimi	yosh guruhi (FPR) va jins (tanlov ulushi — proxy tekshiruvi)
Promote qoidasi	SLO + jins proxy testi: model bashorati bilan jins orasidagi AUC <= 0.60 bo'lishi shart

QADAMLAR

1. Generator (jins ma'lumotda bor, belgilarga kirmaydi), sxema (staj 0–60, km 0–100000, quvvat 40–600 kVt, da'vo {0,1}, policy_id regex ^INS\d{7}$), buzilgan partiya, policyholder_id bo'yicha bo'lish, sizish taqqoslash.
2. Uch eksperiment; chegara recall >= 0.70; reestr + xesh; promote darvozasida jins proxy testi (bashoratdan jinsni bashorat qilish AUC); jadval.
3. MLP -> ONNX, parity, kechikish, xeshlar.
4. FastAPI /predict (javob: risk_score, tier = A/B/C), /metrics, audit (yosh guruhi; jins ALOHIDA audit faylida faqat hisobot uchun); 300 so'rov.
5. Drift 12 oy: belgi drifti (6-oy) va kontsept drifti (9-oy, yorliq kechikadi) — PSI birinchi driftni ko'radi, ikkinchisini ko'rmaydi; kechikkan yorliq kelganda AUC tushishini ko'rsating va «yorliqsiz monitoring chegarasi» haqida xulosa.
6. MetricFrame yosh (FPR, TPR) va jins (selection_rate); bootstrap; A: ThresholdOptimizer(false_positive_rate_parity), B: 18-24 uchun anderrayter qo'lda ko'rishi; taqqoslash.

7. Model kartasi (regulyator uchun bo'lim: qaysi atributlar ishlatilmagan va buni qanday isbotlash mumkin), pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json (proxy test natijasi bilan), drift_log.json, predictions.jsonl, audit_sens.jsonl; model_card.md regulyator bo'limi bilan; Hisobot PDF: jadvallar, belgi vs kontsept drifti grafigi, xulosa (>= 250 so'z)

Baholash (100 ball): Sxema va polis egasi bo'yicha bo'lish — 10, Eksperimentlar va proxy darvozasi — 15, ONNX va kechikish — 10, FastAPI, ajratilgan sezgir audit — 15, Belgi vs kontsept drifti — 15, FPR adolati va yumshatish — 15, Regulyator model kartasi va hisobot — 20

13-variant. Logistika: yetkazib berish kechikishi bashorati o'rta

Soha: Logistika

Marketpleys yetkazib berish buyurtmalarining 24 soatdan ko'p kechikish ehtimolini bashorat qilib, mijozga oldindan xabar bermoqchi va kuryer kompaniyasiga ustuvorlik bermoqchi. Bir marshrutda (shahar juftligi + kuryer kompaniyasi) yuzlab buyurtma bor — bo'lish marshrut bo'yicha, aks holda model marshrutni «yodlab» oladi. Qish mavsumi va bayram oldi talab o'sishi kechikishni oshiradi. Viloyatlarga yetkazish uchun bashorat sifati shaharlardagidek bo'lishi kerak.

Ma'lumot generatori	300 marshrut x 20–60 buyurtma = ~12000 qator; belgilar: masofa km, posilka og'irligi, kuryer yuklamasi %, hafta kuni, ob-havo indeksi, ombor qayta ishlash soati; kechikish ~17%
Guruhlash kaliti	route_id
Sezgir atribut	manzil turi (poytaxt 40% / viloyat markazi 35% / tuman 25%)
Model	logreg, HistGradientBoosting, RandomForest(300, min_samples_leaf 20); ONNX MLP 6-32-1
SLO	AUC >= 0.80, recall >= 0.75, TPR farqi (manzil) <= 0.06, batch 256 p99 <= 5 ms (ONNX, tungi paketli hisob), API p99 <= 15 ms
Drift ssenariysi	12 hafta, 6-haftadan qish: ob-havo indeksi +1.2, qayta ishlash soati +30%; 11-haftada bayram: kuryer yuklamasi +45%, kechikish ulushi 17% -> 30%
Adolatlilik kesimi	manzil turi va posilka og'irligi guruhi (yengil/o'rta/og'ir)
Promote qoidasi	SLO + batch 256 kechikish darvozasi; tuman kesimida recall >= 0.70

QADAMLAR

1. Generator, sxema (masofa 1–1500, og'irlik 0.1–50, yuklama 0–150, order_id regex ^ORD\d{8}$ unikal, ombor soati 0–72), buzilgan partiya, route_id bo'yicha bo'lish; tasodifiy bo'lishda marshrut «yodlash» ta'sirini AUC farqi bilan ko'rsating (marshrut bo'yicha vs tasodifiy test AUC).
2. Uch eksperiment; chegara recall >= 0.75; reestr + xesh + batch 256 kechikish; promote kesim va kechikish darvozasi; jadval.
3. MLP -> ONNX; parity; batch 1 va 256 p50/p99; fayl hajmi.
4. FastAPI /predict va /predict_batch (<= 256), /metrics, audit (manzil turi); 300 so'rov; tungi paket uchun o'tkazuvchanlik (buyurtma/s) hisoblang.
5. Drift 12 hafta: qish (asta) va bayram (sakrash); PSI, KS, bayroq ulushi; yorliq kechikishi 2 hafta — haftalik recall «kechikib» keladi, buni jadvalda alohida ustun qilib ko'rsating.
6. MetricFrame manzil va og'irlik guruhi; bootstrap marshrut bo'yicha; A: ThresholdOptimizer, B: tuman manzillari uchun alohida chegara; taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json, predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, «yodlash» dalili, PSI grafigi, xulosa (>= 250 so'z)

Baholash (100 ball): Sxema, marshrut bo'yicha bo'lish, yodlash dalili — 10, Eksperimentlar, kesim va kechikish darvozasi — 15, ONNX katta batch — 10, Batch API va o'tkazuvchanlik — 15, Drift va kechikkan yorliq — 15, Adolatlilik — 15, Model kartasi va hisobot — 20

14-variant. Ekologiya: havo ifloslanishi xavfli darajasi (PM2.5) o'rta

Soha: Ekologiya (atmosfera)

Shahar ekologiya boshqarmasi 25 ta monitoring stansiyasi ma'lumotidan keyingi 6 soatda PM2.5 «zararli» darajadan (75 mkg/m3) oshish ehtimolini bashorat qilib, aholiga SMS ogohlantirish yubormoqchi. Har stansiyada 60 kunlik soatlik yozuv bor — bo'lish stansiya bo'yicha. Yolg'on ogohlantirish ishonchni yo'qotadi (aniqlik >= 0.65), o'tkazib yuborish esa sog'liqqa zarar (recall >= 0.85). Sensorlar vaqt o'tishi bilan kalibrovkadan chiqadi (sekin drift). Sanoat va turar-joy tumanlari stansiyalari uchun sifat teng bo'lishi kerak.

Ma'lumot generatori	25 stansiya x 60 kun x 24 soat = 36000 qator; belgilar: joriy PM2.5, 6 soat oldingi PM2.5, shamol tezligi, namlik, harorat inversiyasi bayrog'i, kun soati sin; zararli ~13%
Guruhlash kaliti	station_id
Sezgir atribut	tuman turi (sanoat 30% / turar-joy 50% / shahar atrofi 20%)
Model	logreg, HistGradientBoosting, AdaBoost(100); ONNX MLP 6-32-16-1
SLO	recall >= 0.85, aniqlik >= 0.65, AUC >= 0.88, TPR farqi (tuman) <= 0.06, p99 <= 15 ms
Drift ssenariysi	20 kun, sensor kalibrovka drifti: joriy PM2.5 belgisi har kun +1.5% (sekin, kumulyativ 30%); 15-kunda 5 stansiya qayta kalibrovka (qisman qaytish)
Adolatlilik kesimi	tuman turi va mavsum (kun/tun)
Promote qoidasi	SLO + aniqlik; bir vaqtda recall va aniqlik bajariladigan chegara mavjud bo'lmasa model rad etiladi

QADAMLAR

1. Generator, sxema (PM2.5 0–1000, shamol 0–30, namlik 0–100, (station_id, ts) unikal, 6 soat oldingi qiymat joriy jadvaldagi qiymat bilan mos — mantiqiy), buzilgan partiya, station_id bo'yicha bo'lish, sizish taqqoslash.
2. Uch eksperiment; recall va aniqlik ikkalasini qanoatlantiradigan chegara oralig'i; reestr + xesh; promote; jadval; oraliq bo'sh bo'lgan modelni ko'rsating.
3. MLP -> ONNX, parity, kechikish, xeshlar.
4. FastAPI /predict (javob: alert = SMS / NONE, confidence), /metrics (Counter alert turi bo'yicha), audit (tuman, stansiya xeshi); 300 so'rov.
5. Drift 20 kun sekin kumulyativ: kunlik PSI (kechagi kun bilan) deyarli hech narsani ko'rmaydi, mos yozuv bilan PSI 12-kunda WARN, 16-kunda ALERT; «sirpanuvchi mos yozuv tuzog'i» ni ikkala usul grafigi bilan ko'rsating.
6. MetricFrame tuman va kun/tun; bootstrap stansiya bo'yicha; A: ThresholdOptimizer, B: sanoat tumanida qo'shimcha sensor (belgi) bilan qayta o'qitish taqlidi; taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json (ikki PSI usuli), predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, sirpanuvchi vs qat'iy mos yozuv grafigi, xulosa (>= 250 so'z)

Baholash (100 ball): Sxema va stansiya bo'yicha bo'lish — 10, Ikki shartli chegara va promote — 15, ONNX va kechikish — 10, FastAPI va metrikalar — 15, Sekin drift va mos yozuv tuzog'i — 15, Adolatlilik — 15, Model kartasi va hisobot — 20

15-variant. Shahar xizmatlari: fuqaro murojaatlari ustuvorligi o'rta

Soha: Shahar boshqaruvi (elektron murojaat)

Shahar hokimligining murojaatlar portali kelib tushgan arizalar ichidan 48 soat ichida hal qilinishi shart bo'lgan «shoshilinch» (suv uzilishi, yo'l o'pirilishi, gaz hidi) murojaatlarni avtomatik ajratmoqchi. Sandbox'da matn o'rniga undan olingan 6 ta belgi (kalit so'z ballari, kategoriya kodi, takror murojaat soni, geolokatsiya zichligi, soat, foto bor-yo'qligi) sintetik yaratiladi. Bir fuqaro bir nechta murojaat yuboradi — bo'lish fuqaro bo'yicha. Kam daromadli mahallalardan kelgan murojaatlar bir xil sifatda ko'rib chiqilishi kerak. Yangi kategoriya (masalan, issiqlik tarmog'i avariyasi) paydo bo'lganda taqsimot siljiydi.

Ma'lumot generatori	3000 fuqaro x 1–5 murojaat = ~9000 qator; 6 belgi; shoshilinch ~11%; kam daromadli mahallalarda foto kam (belgi shovqini)
Guruhlash kaliti	citizen_id
Sezgir atribut	mahalla daromad darajasi (past 30% / o'rta 50% / yuqori 20%) va murojaat tili (o'zbek/rus)
Model	logreg, MultinomialNB (kalit so'z ballari uchun), HistGradientBoosting; ONNX MLP 6-32-1
SLO	recall >= 0.90, AUC >= 0.85, TPR farqi (daromad) <= 0.05, tanlov ulushi farqi (til) <= 0.04, p99 <= 15 ms
Drift ssenariysi	10 kun, 6-kundan yangi kategoriya kodi (sxemada yo'q!) murojaatlarning 15% ida; 8-kunda sovuq: takror murojaatlar x2
Adolatlilik kesimi	mahalla daromad darajasi va til
Promote qoidasi	SLO; qo'shimcha: past daromadli kesimda recall >= 0.88

QADAMLAR

1. Generator, sxema (kategoriya kodi isin {1..12}, kalit so'z balli 0–1, takror 0–20, zichlik 0–1, foto {0,1}, ticket_id regex), buzilgan partiya, citizen_id bo'yicha bo'lish, sizish taqqoslash, split x daromad jadvali.
2. Uch eksperiment; chegara recall >= 0.90; reestr + xesh; promote kesim qoidasi bilan; jadval.
3. MLP -> ONNX, parity, kechikish, xeshlar.
4. FastAPI /predict (javob: priority = URGENT / NORMAL, sla_hours), /metrics, audit (daromad darajasi, til; citizen_id tuzli xesh); 300 so'rov; noma'lum kategoriya kodi 422 qaytarsin.
5. Drift 10 kun: yangi kategoriya — sxema/API 422 bilan rad etadi, demak modelgacha yetmaydi; rad etilgan ulushni metrikada (status=schema_reject) kuzatib, u 10% dan oshsa ALERT beradigan qoida qo'shing; takror murojaat sakrashi uchun PSI.
6. MetricFrame daromad (TPR) va til (selection_rate); bootstrap; A: ThresholdOptimizer, B: foto yo'q murojaatlar uchun alohida chegara; taqqoslash.
7. Model kartasi, pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json (schema_reject ulushi bilan), predictions.jsonl; model_card.md; Hisobot PDF: jadvallar, rad etish ulushi grafigi, xulosa (>= 250 so'z)

Baholash (100 ball): Sxema va fuqaro bo'yicha bo'lish — 10, Eksperimentlar va kesim promote — 15, ONNX va kechikish — 10, FastAPI, 422 himoyasi, metrikalar — 15, Sxema rad etish drifti — 15, Adolatlilik — 15, Model kartasi va hisobot — 20

16-variant. Moliya: karta tranzaksiyalarida firibgarlik aniqlash qiyin

Soha: Moliya (to'lov xavfsizligi)

To'lov tizimi real vaqtda karta tranzaksiyalarini firibgarlik ehtimoli bo'yicha baholab, shubhalilarni SMS-tasdiqqa yuboradi. Firibgarlik juda kam (1.5%), shuning uchun SLO «FPR <= 1% da recall >= 0.80» ko'rinishida; har soat 300 ming tranzaksiya — p99 kechikishi 5 ms dan oshmasin. Bir kartada o'nlab tranzaksiya bor — bo'lish karta bo'yicha, aks holda model kartani yodlaydi. Firibgarlar taktikani o'zgartiradi (kontsept drifti): yangi sxema paydo bo'lganda eski belgilar ishlamay qoladi. Yoshi katta mijozlar uchun yolg'on bloklash ulushi boshqalardan ko'p bo'lmasin.

Ma'lumot generatori	5000 karta x 10–40 tranzaksiya = ~125000 qator; belgilar: summa (log), o'rtacha summadan og'ish, oxirgi 1 soatdagi tranzaksiya soni, yangi savdogar bayrog'i, geografik masofa km, tungi vaqt; firibgarlik 1.5%
Guruhlash kaliti	card_id (test 0.3)
Sezgir atribut	mijoz yosh guruhi (18-30, 31-50, 51-65, 66+) va hudud (poytaxt/viloyat)
Model	logreg(class_weight), HistGradientBoosting(class_weight), IsolationForest (nazoratsiz baseline); ONNX MLP 6-32-16-1 pos_weight bilan
SLO	recall >= 0.80 @ FPR <= 0.01, PR-AUC >= 0.45, FPR farqi (yosh) <= 0.005 absolyut, ONNX batch 1 p99 <= 0.2 ms, API p99 <= 5 ms
Drift ssenariysi	14 kun, 6-kundan yangi sxema: firibgar tranzaksiyalar kichik summali va kunduzgi bo'ladi (summa belgisi ishlamaydi), firibgarlik ulushi 1.5% -> 2.5%; belgi PSI kichik, lekin recall 0.82 -> 0.55
Adolatlilik kesimi	yosh guruhi (FPR — yolg'on bloklash) va hudud
Promote qoidasi	SLO @ FPR 1% + kechikish darvozasi; yangi versiya joriy production'dan PR-AUC bo'yicha 0.02 yaxshi bo'lsa almashtiriladi

QADAMLAR

1. Generator (nomutanosib, karta bo'yicha), sxema (summa > 0, soatlik son 0–50, masofa 0–20000, tx_id regex ^TX\d{10}$ unikal, bir kartada hudud o'zgarmasligi — mantiqiy), buzilgan partiya 7 xato, card_id bo'yicha bo'lish; tasodifiy bo'lishda sizish ulushi va PR-AUC farqi (yodlash dalili).
2. Uch eksperiment (IsolationForest nazoratsiz baseline sifatida); chegara: FPR <= 1% bo'ladigan eng kichik chegara, so'ng recall o'lchanadi; reestr + xesh + ONNX p99; promote darvozasi; jadval; AUC va PR-AUC farqini izohlang.
3. MLP pos_weight bilan -> ONNX; parity; batch 1 p50/p99 (SLO 0.2 ms) va batch 512; intra_op_num_threads=1 va 2 taqqoslash.
4. FastAPI /score (javob: fraud_prob, action = BLOCK / VERIFY_SMS / ALLOW, uch chegara), /metrics (Counter action bo'yicha, Histogram latency 0.1–10 ms chelaklari), audit (summa EMAS — xesh; yosh guruhi); 300 so'rov p99 <= 5 ms tekshiruvi.
5. Drift 14 kun kontsept drifti: belgi PSI deyarli o'zgarmaydi, ball PSI biroz, lekin VERIFY_SMS ulushi va (kechikkan) recall keskin o'zgaradi — «belgi monitoringi kontsept driftini ko'rmaydi»

xulosasi; qo'shimcha proksi-signal: mijoz SMS rad etish ulushi (sintetik) bilan ogohlantirish.

6. MetricFrame yosh (FPR absolyut farq!) va hudud; bootstrap karta bo'yicha; A: ThresholdOptimizer(false_positive_rate_parity), B: 66+ guruhi uchun yuqori chegara + qo'ng'iroq tasdig'i; taqqoslash: recall yo'qotish vs adolat.
7. Model kartasi, pasport (kechikish byudjeti jadvali bilan), rubrika.

Topshiriladi: Kod papkasi; registry.json, drift_log.json (proksi-signal ustuni), predictions.jsonl; model_card.md; Hisobot PDF: PR egri chizig'i, kechikish byudjeti, kontsept drifti grafigi, kesimlar, xulosa (>= 300 so'z)

Baholash (100 ball): Nomutanosib sxema, karta bo'yicha bo'lish, yodlash dalili — 10, FPR-chegara, PR-AUC, reestr, promote — 15, ONNX 0.2 ms darvozasi va oqimlar — 10, Uch harakatli API va 5 ms p99 — 15, Kontsept drifti va proksi-signal — 15, FPR adolati — 15, Model kartasi va hisobot — 20

17-variant. Sanoat: liniyada mahsulot nuqsonini sensorlardan aniqlash qiyin

Soha: Sanoat (sifat nazorati)

Kabel zavodi ekstruziya liniyasida har mahsulot partiyasi uchun 6 ta sensor (harorat, bosim, tezlik, vibratsiya, diametr og'ishi, tok) o'rtacha qiymatlaridan nuqson ehtimolini bashorat qilib, nuqsonli partiyani laboratoriyaga yuboradi. Bir smena ichida sensorlar o'zaro bog'liq — bo'lish smena bo'yicha (yangi smenada model ishlashi uchun). Sensorlar eskirishi bilan bosim va vibratsiya o'lchovlari sekin siljiydi, bir sensor to'satdan buzilishi ham mumkin (NaN yoki doimiy qiymat). Kunduzgi va tungi smenalar uchun sifat teng bo'lishi kerak; ikki liniya (eski va yangi) bor.

Ma'lumot generatori	400 smena x 30–60 partiya = ~18000 qator; 6 sensor belgisi; nuqson ~8%; tungi smenada vibratsiya shovqini x1.5; 2 liniya (eski liniyada diametr og'ishi kattaroq)
Guruhlash kaliti	shift_id
Sezgir atribut	smena (kunduz 55% / tun 45%) va liniya (eski/yangi)
Model	logreg, HistGradientBoosting, OneClassSVM (faqat normal partiyalarda o'qitilgan nazoratsiz baseline); ONNX MLP 6-32-16-1
SLO	recall >= 0.90, aniqlik >= 0.50, AUC >= 0.90, TPR farqi (smena) <= 0.05, API p99 <= 10 ms, uzilishsiz: sensor NaN bo'lsa xizmat 400 emas, «degraded» javob
Drift ssenariysi	15 kun, 1-kundan bosim +0.5%/kun va vibratsiya +1%/kun (sekin); 9-kunda vibratsiya sensori buziladi: doimiy 0 qiymat (dispersiya nolga tushadi); 12-kunda almashtiriladi
Adolatlilik kesimi	smena va liniya; kesishma smena x liniya
Promote qoidasi	SLO + sensor buzilish testi: har bir belgini NaN qilib yuborganda model (imputer bilan) recall >= 0.80 saqlasa — «mustahkamlik» darvozasi

QADAMLAR

1. Generator, sxema (harorat 150–260, bosim 50–400 bar, tezlik 5–120 m/min, vibratsiya 0–20, diametr og'ishi −1…1 mm, batch_id regex ^L[12]-S\d{4}-\d{3}$, bir smenada liniya o'zgarmasligi), buzilgan partiya 7 xato (shu jumladan doimiy qiymat — dispersiya 0 qoidasi), shift_id bo'yicha bo'lish, sizish taqqoslash.
2. Uch eksperiment; chegara recall >= 0.90 va aniqlik >= 0.50; reestr + xesh; promote mustahkamlik darvozasi (har belgini NaN qilib imputer bilan recall); jadval 6 ustun (har sensor yo'qligida recall).
3. MLP (kirishda NaN -> o'rtacha almashtirish grafga tikiladi: torch.where(isnan)) -> ONNX; parity NaN li kirishda ham; kechikish.
4. FastAPI /predict (javob: defect_prob, status = OK / DEGRADED (sensor yo'q) / REJECT), /metrics (Counter status), audit (smena, liniya, yo'q sensor ro'yxati); 300 so'rov, 30 tasi NaN li.
5. Drift 15 kun: sekin drift (PSI 8-kundan WARN), sensor buzilishi (dispersiya 0 — PSI juda katta, KS p ~0) va almashtirish; «buzilgan sensor drifti» ni «jarayon drifti» dan ajratuvchi qoida (dispersiya nolga yaqin bo'lsa — SENSOR_FAULT, drift emas).

6. MetricFrame smena va liniya, kesishma; bootstrap smena bo'yicha; A: ThresholdOptimizer, B: tungi smena uchun vibratsiya belgisini silliqlash (3 partiya o'rtachasi) bilan qayta o'qitish; taqqoslash.
7. Model kartasi, pasport (sensor buzilishi runbook bo'limi), rubrika.

Topshiriladi: Kod papkasi; registry.json (mustahkamlik jadvali bilan), drift_log.json (SENSOR_FAULT holati), predictions.jsonl; model_card.md + runbook bo'limi; Hisobot PDF: jadvallar, drift va sensor buzilish grafigi, kesishma jadvali, xulosa (>= 300 so'z)

Baholash (100 ball): Sxema (dispersiya qoidasi), smena bo'yicha bo'lish — 10, Eksperimentlar va mustahkamlik darvozasi — 15, NaN-bardosh ONNX graf — 10, DEGRADED rejimli API — 15, Sensor buzilishi vs jarayon drifti — 15, Adolatlilik va kesishma — 15, Model kartasi, runbook, hisobot — 20

18-variant. Tibbiyot: tez yordam chaqiruvlari triaji qiyin

Soha: Tibbiyot (shoshilinch yordam)

103 xizmati dispetcherlik markazi chaqiruv paytida operator kiritgan 6 ta strukturaviy belgi (yosh, hushsizlik, nafas qiyinligi balli, ko'krak og'rig'i, surunkali kasallik soni, chaqiruv davomiyligi) asosida «hayot uchun xavfli» ehtimolini hisoblab, brigadani ustuvor yuboradi. Bir bemor yiliga bir necha marta chaqiradi — bo'lish bemor bo'yicha. O'tkazib yuborish qimmat: recall >= 0.95; lekin barcha chaqiruvlarni shoshilinch deb belgilash mumkin emas (bayroq ulushi <= 0.40). Bolalar (0-12) va keksalar (75+) uchun model ishonchli bo'lishi shart, aks holda ko'lamdan chiqariladi. Qish epidemiyasida belgilar taqsimoti siljiydi.

Ma'lumot generatori	6000 bemor x 1–4 chaqiruv = ~15000 qator; 6 belgi; xavfli ~14%; 0-12 va 75+ guruhlarida belgilar shovqinliroq (x1.8), ularning ulushi 10% va 15%
Guruhlash kaliti	patient_id
Sezgir atribut	yosh guruhi (0-12, 13-40, 41-74, 75+) va jins; qo'shimcha: hudud (shahar/qishloq — brigada yetib borish vaqti)
Model	logreg, HistGradientBoosting (monotonic_cst: nafas balli va hushsizlik bo'yicha monoton), ekspert qoidasi baseline (hushsizlik yoki nafas balli >= 7 -> xavfli); ONNX MLP 6-32-16-1
SLO	recall >= 0.95, bayroq ulushi <= 0.40, AUC >= 0.85, TPR farqi (yosh) <= 0.04, har guruhda TPR >= 0.92, p99 <= 10 ms
Drift ssenariysi	12 kun, 5-kundan gripp epidemiyasi: nafas balli +1.5, chaqiruvlar soni x1.8, xavfli ulushi 14% -> 19%; 10-kunda yangi so'rovnoma shakli: davomiyligi belgisi birliklari daqiqa -> soniya (60x siljish — sxema tutishi kerak)
Adolatlilik kesimi	yosh guruhi (TPR, har guruh alohida chegara bilan) va hudud
Promote qoidasi	SLO + ekspert qoidasidan recall bo'yicha yomon bo'lmasligi + monotonlik tekshiruvi (nafas balli oshganda ehtimol kamaymasin)

QADAMLAR

1. Generator, sxema (yosh 0–105, nafas balli 0–10, hushsizlik {0,1}, surunkali 0–10, davomiylik 10–900 soniya, call_id regex, bir bemorda jins o'zgarmasligi, yosh farqi <= 1), buzilgan partiya 8 xato, patient_id bo'yicha bo'lish, sizish taqqoslash, yosh guruhi x split jadvali.
2. Uch eksperiment (ekspert qoidasi baseline bilan); chegara recall >= 0.95 va bayroq <= 0.40; reestr + xesh; promote: SLO + ekspert bilan taqqoslash + monotonlik testi (100 sintetik nuqtada); jadval.
3. MLP -> ONNX; parity; kechikish; monotonlik ONNX chiqishida ham tekshiriladi.
4. FastAPI /triage (javob: priority = P1 / P2 / P3, uch chegara, eta_minutes hudud bo'yicha), /metrics, audit (yosh guruhi, hudud, kirish xeshi); 300 so'rov; davomiylik 900 dan katta bo'lsa 422.
5. Drift 12 kun: epidemiya (kutilgan, lekin xavfli ulushi oshadi) va birlik xatosi (sxema 422 ushlaydi, rad etish ulushi ALERT); PSI, KS, P1 ulushi, kechikkan recall; brigada yuklamasi (P1 soni x 40 daqiqa) hisoblab resurs ogohlantirishi.

6. MetricFrame yosh (har guruhda TPR >= 0.92 shartini tekshiring) va hudud; bootstrap bemor bo'yicha; A: ThresholdOptimizer(true_positive_rate_parity), B: 0-12 va 75+ uchun «operator majburiy qo'shimcha savollar» rejimi (ko'lam cheklash), C: har guruh uchun alohida chegara qo'lda tanlash; taqqoslash — bayroq ulushi 0.40 dan oshmasligi sharti bilan.
7. Model kartasi (klinik cheklovlar, «tashxis emas, navbat» eslatmasi), pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json (monotonlik va ekspert taqqoslash bilan), drift_log.json (resurs ogohlantirishi), predictions.jsonl; model_card.md klinik bo'lim bilan; Hisobot PDF: jadvallar, guruh bo'yicha TPR IO grafigi, PSI, ADR (A/B/C tanlovi), xulosa (>= 300 so'z)

Baholash (100 ball): Sxema (mantiqiy qoidalar), bemor bo'yicha bo'lish — 10, Eksperimentlar, ekspert baseline, monotonlik darvozasi — 15, ONNX va monotonlik tekshiruvi — 10, Uch ustuvorlikli API — 15, Epidemiya drifti va birlik xatosi himoyasi — 15, Har guruhda TPR >= 0.92 adolati, uch yumshatish — 15, Klinik model kartasi va hisobot — 20

19-variant. Kiberxavfsizlik: tarmoq oqimlarida anomaliya va hujum aniqlash

qiyin

Soha: Kiberxavfsizlik

Korporativ SOC markazi tarmoq oqimi (NetFlow) yozuvlaridan hujum ehtimolini baholab, analitik navbatiga qo'yadigan xizmat qurmoqda. Bir xost (qurilma) minglab oqim yaratadi — bo'lish xost bo'yicha, aks holda model xost IP sini yodlaydi. Hujumlar kam (3%) va turlari o'zgaradi (yangi skaner vositasi — kontsept drifti). SOC analitiklari kuniga 200 tadan ortiq ogohlantirishni ko'ra olmaydi — bu bayroq byudjeti. Bo'limlar (IT, moliya, ishlab chiqarish) uchun yolg'on ogohlantirish ulushi teng bo'lishi kerak, aks holda bir bo'lim doimiy «shubhali» bo'lib qoladi.

Ma'lumot generatori	600 xost x 50–200 oqim = ~75000 qator; belgilar: baytlar (log), paketlar, davomiylik, noyob portlar soni (1 daqiqa), muvaffaqiyatsiz ulanishlar, tashqi IP bayrog'i; hujum 3% (skanerlash, brute-force, eksfiltratsiya)
Guruhlash kaliti	host_id
Sezgir atribut	bo'lim (IT 25% / moliya 20% / ishlab chiqarish 35% / boshqa 20%); IT bo'limi oqimlari tabiatan «shubhaliroq» (ko'p port)
Model	IsolationForest (nazoratsiz), logreg(class_weight), HistGradientBoosting; ikki bosqichli: IsolationForest ball + nazoratli model; ONNX MLP 6-32-16-1
SLO	recall >= 0.85, kuniga ogohlantirish <= 200 (25000 oqimda bayroq ulushi <= 0.8%), PR-AUC >= 0.50, FPR farqi (bo'lim) <= 0.003 absolyut, ONNX batch 1024 p99 <= 3 ms
Drift ssenariysi	14 kun, 6-kundan yangi skaner: noyob portlar belgisi hujumlarda pasayadi (sekin skaner), hujumlar 3% -> 5%; 10-kunda IT bo'limida yangi monitoring agenti — normal oqimlarda tashqi IP ulushi +30% (yolg'on ogohlantirish to'lqini)
Adolatlilik kesimi	bo'lim (FPR va ogohlantirish soni) va vaqt (ish soati / tun)
Promote qoidasi	SLO + ogohlantirish byudjeti darvozasi: validatsiyada kunlik ogohlantirish soni 200 dan oshsa chegara qayta tanlanadi, baribir recall < 0.85 bo'lsa rad etiladi

QADAMLAR

1. Generator (xost bo'yicha, 3 hujum turi), sxema (baytlar >= 0, portlar 0–65535, davomiylik 0–3600, flow_id regex unikal, bir xostda bo'lim o'zgarmasligi), buzilgan partiya, host_id bo'yicha bo'lish; tasodifiy bo'lishda PR-AUC «shishib ketishi» (yodlash) dalili.
2. Uch eksperiment (IsolationForest ballini belgi sifatida ikkinchi bosqichga berish); chegara: kunlik byudjet 200 ga mos bayroq ulushi; recall; reestr + xesh; promote byudjet darvozasi; jadval hujum turi bo'yicha recall.
3. MLP -> ONNX; parity; batch 1 va 1024 p50/p99 (oqimlar paketli keladi); o'tkazuvchanlik oqim/s.
4. FastAPI /score_batch (<= 1024 oqim), javob har oqimga: score, alert {0,1}; /metrics (Counter alert bo'lim bo'yicha, Histogram batch_size), audit (bo'lim, xost xeshi, kunlik alert hisoblagichi); 300 so'rov x 256 oqim.

5. Drift 14 kun: kontsept drifti (sekin skaner) — belgi PSI kichik, hujum turi bo'yicha recall tushadi; IT agenti — bo'lim kesimida FPR va alert soni sakraydi (byudjet buziladi); ikki holat uchun ikki xil ogohlantirish (MODEL_DRIFT vs ALERT_BUDGET) va runbook harakati.
6. MetricFrame bo'lim (FPR absolyut, alert soni) va vaqt; bootstrap xost bo'yicha; A: ThresholdOptimizer(false_positive_rate_parity), B: bo'lim bo'yicha alohida IsolationForest mos yozuvi (kontekstual anomaliya) bilan qayta o'qitish; taqqoslash: recall va byudjet.
7. Model kartasi, pasport (hujum turlari bo'yicha qamrov jadvali), rubrika.

Topshiriladi: Kod papkasi; registry.json (hujum turi bo'yicha recall), drift_log.json (ikki xil holat), predictions.jsonl; model_card.md; Hisobot PDF: PR egri chizig'i, byudjet grafigi, bo'lim bo'yicha FPR, drift turlari, xulosa (>= 300 so'z)

Baholash (100 ball): Sxema, xost bo'yicha bo'lish, yodlash dalili — 10, Ikki bosqichli model, byudjet darvozasi, reestr — 15, ONNX katta batch va o'tkazuvchanlik — 10, Batch API va bo'lim metrikalari — 15, MODEL_DRIFT vs ALERT_BUDGET — 15, Bo'limlar bo'yicha FPR adolati — 15, Model kartasi va hisobot — 20

20-variant. Media: yangiliklar ilovasida kontent tavsiyasi (CTR) qiyin

Soha: Media (tavsiya tizimi)

Yangiliklar ilovasi har foydalanuvchiga ko'rsatiladigan maqolalar ro'yxatini «bosish ehtimoli» (CTR) bo'yicha tartiblaydi. Bir foydalanuvchida o'nlab ko'rsatuv (impression) bor — bo'lish foydalanuvchi bo'yicha, aks holda model foydalanuvchini yodlaydi. Tahririyat talabi: yosh guruhlari va til auditoriyasi (o'zbek/rus) uchun mahalliy yangiliklar ko'rsatilish ulushi (exposure) 0.05 dan ko'p farq qilmasin; shuningdek «mashhurlik halqasi» (ko'p bosilgan maqola yana ko'p ko'rsatiladi) monitoring qilinsin. Yangi kontent turi (qisqa video) paydo bo'lganda taqsimot siljiydi; p99 juda qattiq, chunki ro'yxatga 50 maqola bir so'rovda baholanadi.

Ma'lumot generatori	4000 foydalanuvchi x 20–60 ko'rsatuv = ~160000 qator (30% namuna — 48000); belgilar: maqola yoshi soat, maqola kategoriyasi kodi, foydalanuvchi kategoriya qiziqishi (0–1), sarlavha uzunligi, mashhurlik (oxirgi soat bosishlar), kun soati; bosish ~6%
Guruhlash kaliti	user_id
Sezgir atribut	yosh guruhi (18-24, 25-44, 45+) va til (o'zbek 65% / rus 35%)
Model	logreg, HistGradientBoosting, FM uslubidagi MLP (kategoriya embedding 8 o'lchov) — PyTorch; ONNX MLP batch 50
SLO	AUC >= 0.72, LogLoss <= 0.21, mahalliy kontent exposure farqi (til) <= 0.05, top-10 ichida har kategoriya >= 1 (xilma-xillik), ONNX batch 50 p99 <= 1 ms, API p99 <= 10 ms
Drift ssenariysi	14 kun, 6-kundan qisqa video kontent (yangi kategoriya kodi, maqola yoshi juda kichik) ko'rsatuvlarning 25% i; 10-kunda mashhurlik halqasi: top-5 maqola mashhurligi x3, bosish ulushi 6% -> 9% (o'z-o'zini kuchaytirish)
Adolatlilik kesimi	til (mahalliy kontent exposure — selection_rate analogi) va yosh guruhi (AUC farqi)
Promote qoidasi	SLO + xilma-xillik va exposure darvozasi; AUC yuqori bo'lsa ham exposure farqi > 0.05 bo'lsa rad etiladi

QADAMLAR

1. Generator (foydalanuvchi x maqola ko'rsatuvlari, bosish 6%), sxema (maqola yoshi 0–168 soat, kategoriya isin {1..8}, qiziqish 0–1, impression_id regex unikal, (user_id, article_id, ts) unikal), buzilgan partiya, user_id bo'yicha bo'lish; tasodifiy bo'lishda AUC shishishi dalili.
2. Uch eksperiment (shu jumladan PyTorch embedding model); metrikalar AUC, LogLoss, top-10 xilma-xillik, til bo'yicha mahalliy exposure; reestr + xesh; promote exposure darvozasi; jadval.
3. Embedding MLP -> ONNX (kategoriya kodi int64 kirish + 5 float belgi; Gather operatori); parity; batch 50 p50/p99; fayl hajmi.
4. FastAPI /rank (kirish: foydalanuvchi konteksti + 50 nomzod maqola; javob: tartiblangan ro'yxat + xilma-xillik qayta tartiblash), /metrics (Histogram ro'yxat uzunligi, Counter kategoriya exposure), audit (til, yosh guruhi, top-10 kategoriya ro'yxati); 300 so'rov p99 <= 10 ms.

5. Drift 14 kun: yangi kontent turi (sxema yangi kodni rad etadi — 422 ulushi ALERT, so'ng sxema yangilanib model qayta o'qitiladi: ikki bosqichli runbook) va mashhurlik halqasi (mashhurlik belgisi PSI, top-5 exposure ulushi monitoring); «halqa» ogohlantirish qoidasi: top-5 exposure > 40%.
6. MetricFrame til (mahalliy exposure, selection_rate) va yosh (AUC); bootstrap foydalanuvchi bo'yicha; A: ThresholdOptimizer o'rniga qayta tartiblash (re-ranking) bilan exposure tenglash, B: mashhurlik belgisini log-siqish (damping) bilan qayta o'qitish; taqqoslash: AUC, exposure farqi, xilma-xillik.
7. Model kartasi (tahririyat siyosati bo'limi), pasport, rubrika.

Topshiriladi: Kod papkasi; registry.json (exposure va xilma-xillik bilan), drift_log.json (halqa ogohlantirishi), predictions.jsonl; model_card.md tahririyat bo'limi bilan; Hisobot PDF: jadvallar, exposure va top-5 ulushi grafigi, re-ranking taqqoslash, xulosa (>= 300 so'z)

Baholash (100 ball): Sxema, foydalanuvchi bo'yicha bo'lish, shishish dalili — 10, Embedding model, exposure darvozasi, reestr — 15, Aralash kirishli ONNX (int64 + float) — 10, /rank API va exposure metrikalari — 15, Yangi kontent va mashhurlik halqasi drifti — 15, Exposure adolati va re-ranking — 15, Model kartasi va hisobot — 20